

NixOS

K19G

2026-09-22

Table of contents

Notes on Nix & NixOS	77
What “done” looks like	77
How the book is organized	77
Recommended path if your daily driver is Fedora	78
Lab 0 (recommended before first host work)	78
How I intend to study	79
Prerequisites	79
Related books	79
Formats	79
Acknowledgments	80
Front Matter	81
Syllabus	82
Syllabus — expert path for Nix & NixOS	83
Why this order (expert rationale)	84
How source notes were used	84
Other languages (including Go)	85
Modern stack (defaults for this book)	86
Part map (topic book)	87
Lab 0 (front matter) — before Concepts	89
Concepts — Foundations	90
Nix on Linux — Fedora bridge (recommended before host cutover)	91
NixOS host	92
Home Manager and flake layout	93
Services and security	94
Packaging craft	95
Ops and fleet	96
Internals and currency	97

Capstone and recovery	98
Capstone A — required	98
Capstone B — stretch	99
Projects (optional depth track)	100
Review habit	101
Completeness audit (sources → parts)	102
Explicitly out of scope	103
Resources (primary, not blogs-of-2022)	104
Writing rule for future chapters	105
Lab hardware default (this book)	106
Lab 0 — Overview: roles before the Why Nix exists chapter	107
Lab 0 — Overview: roles before the Why Nix exists chapter	108
Why a lab host exists	108
Philosophy: host is not an appliance	108
Three roles (do not collapse them)	109
Chapters in this section	109
Recommended topology for this curriculum	110
Decision: host OS	110
Hypervisor choice (libvirt vs Incus)	111
What “ready for the Why Nix exists chapter” means	111
Time budget	111
Further reading	112
Next	112
Lab 0A — Journey single-server (KVM + containers)	113
Lab 0A — Journey single-server (KVM + containers)	114
What this lab optimizes for	114
Architecture	114
Part 1 — Host OS install (metal)	115
Option H1 (recommended): NixOS host	115
Option H2: Debian/Ubuntu host	116
Part 2 — Storage plan (critical on 16 GB RAM laptops)	116
If the Elitebook still has a spinning HDD	116
Thin provision	116
Part 3 — KVM: create journey VMs	117
Resource template (16 GB host)	117
Example: <code>virt-install</code> (adjust ISO path)	117
Snapshots = courage	117
Networking for “remote” deploy	117
Part 4 — Containers (Podman)	118

Part 4b — Optional hypervisor: Incus (CLI-first VMs + system containers)	118
When to consider Incus on the lab host	118
NixOS host sketch (Incus)	119
Simulate the host before you reinstall metal	119
If you already run Proxmox	119
Part 5 — Nested virtualization?	120
Part 6 — Services the host should run (boring is good)	120
Part 7 — Daily workflow for the journey	120
Debugging without drift	121
Part 8 — Declarative hardware quirks (host flake)	121
Part 9 — Failure modes checklist	121
Checkpoint — Lab 0A complete	122
Next	122
Lab 0B — NixOS development machine	123
Lab 0B — NixOS development machine	124
Dev machine vs journey guest	124
Part 1 — Install Nix (if host is not NixOS)	124
Part 2 — Essential packages	125
NixOS host module sketch	125
Non-NixOS host	125
Part 3 — Editor + LSP	125
Part 4 — Git layout for the whole journey	126
Part 5 — Build ergonomics	126
Keep host builds from melting a 2012 laptop	126
Substituters	127
Remote / guest as builder (optional, Stage V+)	127
Part 6 — Access to journey VMs	127
Part 7 — Workflow commands you will live with	127
Part 8 — Home Manager on the dev role	128
Part 9 — Binary cache & secrets (preview only)	128
Part 10 — Quality of life on old hardware	128
Dev machine checklist	129
Anti-patterns	129
Next	129
Lab 0C — HP Elitebook 2570p (16 GB) profile	130
Lab 0C — HP Elitebook 2570p · 16 GB RAM lab profile	131
Hardware snapshot (typical 2570p)	131
BIOS checklist (do this first)	132
Disk strategy for <i>this</i> laptop	132
Best	132
Acceptable	132
Partition sketch (NixOS host, single disk)	132
RAM budget (16 GB fixed)	132
Suggested virsh memory	133

CPU budget	133
What this machine is good for (curriculum map)	133
Networking on the Elitebook	134
Power & thermals	134
Firmware & drivers notes	134
Host is fully yours (but still a lab)	134
Concrete “day zero” procedure (you)	135
Optional extras that fit this chassis	135
What not to run on the 2570p	136
Elitebook readiness checklist	136
After this page	136
Lab 0D — Ops cheatsheet	137
Lab 0D — Ops cheatsheet (single-server lab)	138
Daily	138
Snapshots / restore points	138
Disk pressure	139
Guest rebuild (from host flake)	139
When the Elitebook sweats	139
Drift control	139
Emergency	139
Inventory template (fill once)	140
Concepts	141
Concepts — Nix language, store, CLI, flakes	142
Concepts — Nix language, store, CLI, flakes	143
What this part is for	143
Baseline (this book)	143
Learning outcomes	144
Prerequisites	144
Chapter map	144
Suggested learning paths	145
How each chapter is structured	145
Lab workspace convention	146
Mental model map (part-level)	146
What you will <i>not</i> do in this part	147
Suggested checkpoints (part gate)	147
Further depth (this book)	147
How to use this part as a reference	148
Why Nix exists	149
Why Nix exists	150
Why this chapter matters	150

Theory 1 — Three faces of the same ecosystem	150
Analogy (imperfect but useful)	151
Naming traps in the wild	151
Theory 2 — Failure modes of traditional systems	152
Dependency hell	152
Snowflake servers	152
Language-level global installs	152
CI “works on my machine” graphs	152
What “reproducible” can mean (honesty ladder)	153
Theory 3 — Nix’s core bet: pure builds + hashed store	153
Content-addressed intuition	153
Coexistence	154
Environments as compositions	154
What purity buys operations	154
Theory 4 — Evaluation vs realization	154
Substituters (binary caches)	155
Why the split matters for debugging	155
Theory 5 — Installer shapes and multi-user daemon	155
Why a daemon?	155
Config locations (theory)	156
Disk and trust	156
Theory 6 — Flakes as the modern pinning model (preview)	156
Baseline pin you will reuse	156
Theory 7 — CLI eras (so docs don’t gaslight you)	157
Theory 8 — What Nix is <i>not</i>	157
Worked examples	157
Example A — “Install” without polluting a global profile	157
Example B — Why two machines diverge without locks	157
Example C — What Nix does <i>not</i> do	158
Example D — Eval error vs build error (feel the phases)	158
Example E — Minimal REPL tour	158
Example F — Inspect config after enabling features	158
Lab 1 — Install Nix	158
Lab 2 — Enable modern features	159
Lab 3 — First modern commands	159
REPL	159
Lab 4 — Write the model	160
Exercises	160
Common pitfalls	160
Checkpoint	161
Further depth (this book)	161

The Nix store and closures 162

The Nix store and closures 163

Why this chapter matters	163
Theory 1 — The store is the database	163
Invariants (practical)	163

What lives beside the store	164
Theory 2 — Path anatomy	164
Examples (illustrative shapes)	164
Input-addressed vs content-addressed (awareness)	165
Name segment is not a version pin	165
Theory 3 — References and the closure	165
Direct references	165
Closure (requisites)	166
Why closures matter — worked scenarios	166
Example narrative	166
Closure size theory	166
Referrers (reverse edges)	167
Theory 4 — Evaluation vs realization (store-centric)	167
Theory 5 — Profiles, generations, and roots	168
Why GC exists	168
Root types (non-exhaustive)	168
The <code>./result</code> trap (critical theory)	168
Generations theory (preview)	169
Theory 6 — Substituters, signatures, trust (preview)	169
Theory 7 — What “uninstall” means	169
Operational uninstall checklist	170
Theory 8 — Copying closures (ops preview)	170
Worked examples bank	170
Example A — Build and print path	170
Example B — Direct vs full graph	170
Example C — Tree view	171
Example D — Result root lifecycle	171
Example E — Path info metadata	171
Example F — Compare two packages	171
Lab 1 — Realize <code>hello</code> and dissect the path	171
Lab 2 — Closure measurements	172
Lab 3 — Follow one dependency	172
Lab 4 — Roots observation	172
Lab 5 — Teach-back	172
Exercises	173
Common pitfalls	173
Checkpoint	174
Journal prompts	174
Further depth (this book)	174

Nix language: values and attrsets 175

Nix language: values and attrsets 176

Why this chapter matters	176
Theory 1 — Values are few; composition is everything	176
Type inspection	177
Theory 2 — Strings: interpolation and two quote forms	177
Double quotes	177

Indented strings ('' ... '')	177
Escaping notes	177
Paths vs path-looking strings	178
Theory 3 — Lists	178
List builtins you will reuse	178
Theory 4 — Attribute sets (the main event)	179
Access	179
Nested sets	179
Dynamic keys	179
Update / merge intuition	180
Attrset builtins	180
Theory 5 — <code>let ... in</code> (local bindings)	180
Rules that matter	180
Nested lets vs one flat let	181
<code>rec</code> attrsets	181
Theory 6 — <code>inherit</code> (pull names into sets or lets)	181
Into an attrset	181
Rename form	181
Into <code>let</code>	182
Theory 7 — <code>with</code> (know it; use sparingly)	182
Theory 8 — Conditionals and operators (minimal set)	182
Theory 9 — Config-shaped modeling (bridge to NixOS)	183
Theory 10 — Destructuring patterns you will see	184
Worked examples bank	184
Example A — REPL tour	184
Example B — Deep access and missing attrs	184
Example C — <code>inherit</code> package list pattern	185
Example D — Shallow merge trap	185
Example E — Dynamic user keys	185
Lab 1 — Workspace and file evaluation	186
Lab 2 — Nested merge experiments	186
Lab 3 — Attrset utilities in the REPL	187
Lab 4 — Model something real from your life	187
Lab 5 — Teach-back	188
Exercises	188
Common pitfalls	189
Checkpoint	189
Further depth (this book)	190

Nix language: functions **191**

Nix language: functions **192**

Why this chapter matters	192
Theory 1 — Everything callable is a lambda	192
Application is juxtaposition	192
Functions are values	193
Identity and composition (lightweight)	193

Theory 2 — Attrset pattern arguments (the Nix idiom)	193
Destructuring is the point	193
Why not positional for public APIs?	193
Theory 3 — Default arguments with ?	194
Defaults can be expressions	194
Explicit <code>null</code> vs missing key	194
Theory 4 — Ellipsis ... (allow unknown attributes)	195
When to use ellipsis	195
Theory 5 — @ binding (name the whole set)	195
Why @ exists	196
Theory 6 — Composition and higher-order helpers	196
<code>map</code> / <code>filter</code> (builtins)	196
Function returning function	196
<code>enableIf</code> -style helper (preview of module style)	197
Theory 7 — Fixed-point awareness (not mastery)	197
Theory 8 — Call patterns you will see in the wild	197
1. Package “callPackage” shape	197
2. Overlay shape	198
3. Module shape (preview)	198
4. <code>override</code> / <code>overrideAttrs</code> as functions	198
Theory 9 — Error messages as teachers	198
Worked examples bank	199
Example A — Defaults + ellipsis	199
Example B — @ forward	199
Example C — User skeleton helper	199
Example D — Package list merge helper	200
Example E — Strict API for typos	200
Lab 1 — Workspace	200
Lab 2 — Strict vs ellipsis	201
Lab 3 — @ args inventory	201
Lab 4 — Compose three helpers into a “mini host”	202
Lab 5 — Read real shapes (optional but high value)	202
Exercises	202
Common pitfalls	203
Checkpoint	203
Further depth (this book)	204

Laziness, imports, and lib 205

Laziness, imports, and lib 206

Why this chapter matters	206
Theory 1 — Laziness: values are promises	206
Why it matters	206
Forcing	207
Mental model	207
Theory 2 — Imports are different: loading is eager (once forced)	207
Mental model	207
Common patterns	207

Paths	208
Import caching	208
Theory 3 — Multi-file composition patterns	208
Pattern A — Data module	208
Pattern B — Function module	208
Pattern C — Directory + <code>default.nix</code>	209
Pattern D — Layered host-shaped tree (preview)	209
Theory 4 — Infinite recursion (how you will meet it)	209
Debugging habits	210
Fixed points (recognition only)	210
Theory 5 — <code>lib</code> as the standard library (taste, not catalog)	210
Attrsets	210
Lists	211
Strings	211
Example pipeline — users list $\rightarrow$ attrset	211
Why not memorize all of <code>lib</code> ?	211
Theory 6 — <code>NIX_PATH</code> and angle-bracket paths (literacy)	212
Theory 7 — Evaluation vs runtime (again, file-centric)	212
When does “work” happen?	212
Theory 8 — Project layout habits	212
Worked examples bank	213
Example A — Lazy skip	213
Example B — Import function	213
Example C — <code>lib.optional</code> / <code>optionals</code>	213
Example D — <code>mapAttrs</code>	213
Example E — <code>recursiveUpdate</code> vs <code>//</code>	214
Lab 1 — Multi-file project skeleton	214
Lab 2 — Trace forcing	215
Lab 3 — Infinite recursion (safe sandbox)	215
Lab 4 — Real <code>lib</code> if <code>nixpkgs</code> available	216
Lab 5 — Teach-back	216
Exercises	216
Common pitfalls	217
Checkpoint	217
Further depth (this book)	218

Derivations as an idea 219

Derivations as an idea 220

Why this chapter matters	220
Theory 1 — Derivation = build recipe with a promised output path	220
Vocabulary table	221
Theory 2 — Eval vs realize (operational)	221
<code>.drv</code> is not the package	221
Pipeline sketch	221
Theory 3 — <code>mkDerivation</code> mental model (lite)	222
Other trivial helpers (awareness)	222

Theory 4 — Sandbox: why builds are fenced	222
Fixed-output derivations (FODs)	223
Why FODs matter to operators	223
Theory 5 — Substituters vs local build	223
Theory 6 — Outputs and multiple outputs (preview)	224
Theory 7 — What you inspect after a build	224
Reading a log like an operator	224
Theory 8 — Impure vs pure evaluation (awareness)	225
Worked examples bank	225
Example A — Build hello from nixpkgs	225
Example B — Trivial package via flake	225
Example C — Read derivation metadata	225
Example D — Sandbox failure story (conceptual)	225
Example E — <code>runCommand</code> sketch	226
Example F — Wrong FOD hash	226
Lab 1 — Workspace + flake for trivial builds	226
Lab 2 — Inspect plan and log	227
Lab 3 — Compare to nixpkgs hello closure	227
Lab 4 — Break a FOD on purpose (optional)	228
Lab 5 — Teach-back	228
Exercises	228
Common pitfalls	229
Checkpoint	229
Further depth (this book)	230
Modern Nix CLI	231
Modern Nix CLI	232
Why this chapter matters	232
Theory 1 — One CLI, many intents	232
Theory 2 — Installables: how you <i>name</i> what you want	233
# splits flake ref from attribute	233
Ambiguity and defaults	233
Theory 3 — <code>nix build</code> deep enough for daily use	233
Theory 4 — <code>nix run</code> vs <code>nix shell</code> vs <code>nix develop</code>	234
<code>nix run</code>	234
<code>nix shell</code>	234
<code>nix develop</code>	234
Theory 5 — Flake subcommands	235
<code>show</code> vs <code>metadata</code>	235
JSON for scripting	235
Theory 6 — Eval and debugging eval	235
Theory 7 — Store queries you will reuse	236
Theory 8 — Profiles without making them a lifestyle	236
Theory 9 — Search and discovery	237
Theory 10 — Error class → command checklist	237
Worked examples bank	237
Example A — Same package, three ways	237

Example B — Flake introspection	237
Example C — Logs streaming	238
Example D — JSON for scripting	238
Example E — Non-interactive develop	238
Lab 1 — Cheat-sheet construction	238
Lab 2 — Operate a local flake with modern CLI only	239
Lab 3 — Ad-hoc <code>nix shell</code> workflow	239
Lab 4 — Search + run loop	239
Lab 5 — Intent mapping drill (closed book)	240
Exercises	240
Common pitfalls	241
Checkpoint	241
Further depth (this book)	241
Flakes and project devShells	242
Flakes and project devShells	243
Why this chapter matters	243
Theory 1 — What a flake is	243
Evaluation model	243
Baseline for this book	244
Theory 2 — <code>inputs</code>	244
Common URL forms	244
Following transitive inputs	244
Input types (awareness)	244
Theory 3 — <code>outputs</code>	245
Output schema (common)	245
<code>self</code>	245
Naming <code>default</code>	245
Theory 4 — Systems and portability	246
<code>genAttrs</code> pattern	246
Theory 5 — <code>mkShell</code> / <code>mkShellNoCC</code>	246
<code>mkShellNoCC</code>	247
Theory 6 — Lock discipline (intro)	247
Theory 7 — Flakes and git	247
Dirty tree semantics	247
Theory 8 — Registries (light awareness)	248
Theory 9 — Apps vs packages (optional clarity)	248
Theory 10 — What not to put in a first flake	248
Worked examples bank	249
Example A — Minimal full flake	249
Example B — Enter and verify	249
Example C — Metadata	249
Example D — Non-interactive CI style	249
Example E — Package + shell together	250
Lab 1 — Create the project	250
Lab 2 — Make it <i>your</i> project	250
Lab 3 — <code>shellHook</code> and <code>env</code>	251

Lab 4 — Offline-capable after populate (preview)	251
Lab 5 — Teach-back	251
Exercises	251
Common pitfalls	252
Checkpoint	252
Further depth (this book)	253
Classic CLI literacy	254
Classic CLI literacy	255
Why this chapter matters	255
Theory 1 — Same store, two UIs	255
Theory 2 — Command map (core)	256
Intent-first map	256
Theory 3 — Channels as history	256
Channel model	256
Flake model (preferred)	257
When channels still appear	257
Theory 4 — <code>shell.nix</code> / <code>default.nix</code> legacy layouts	257
Classic project shell	257
Modern	258
Dual support pattern (optional)	258
Theory 5 — <code>nix-env</code> and imperative profiles	258
Theory 6 — Purity and flakes	258
Theory 7 — Reading old error messages	259
Theory 8 — GC and store tools (both eras)	259
Theory 9 — Decision policy for this book	259
Worked examples bank	260
Example A — Build hello both ways	260
Example B — Shell both ways	260
Example C — Query store both ways	260
Example D — Profile vs run	260
Example E — Channel list observation	261
Lab 1 — Build the translation table	261
Lab 2 — Dual path for a project shell	261
Lab 3 — Channel inspection (optional, disposable)	262
Lab 4 — Doc archaeology	262
Lab 5 — Commitment statement	262
Exercises	262
Common pitfalls	263
Checkpoint	263
Concepts part — exit criteria	264
Further depth (this book)	264
Nix on Linux	265
Nix on Linux — Fedora bridge path	266

Nix on Linux — Fedora bridge path	267
Why Fedora 44 matters	267
What you will learn here	268
Suggested study path	268
Lab workspace	269
Exit gate (before NixOS host)	269
Further depth (this book)	270
Why Nix on Linux first	271
Why Nix on Linux first	272
Why this chapter matters	272
Three faces again (bridge framing)	273
What Nix is good at <i>on</i> an existing distro	273
What you should still leave to Fedora	273
Dual-stack anti-patterns	274
Skills map: bridge → NixOS	275
Mental model: evaluation vs realization (ops view)	275
Worked decision: “Should this be Nix or dnf?”	276
Lab	276
Checkpoint	276
Further depth	277
Install Nix on Fedora 44	278
Install Nix on Fedora 44	279
Prerequisites	279
Install modes (choose one)	279
Multi-user install (Workstation / Server)	280
1. Install packages	280
2. Enable the daemon	280
3. Groups and shell environment	280
4. First smoke test	281
Configure system vs user	281
Daemon / system config	281
User config	282
Single-user mode (optional)	282
Atomic / immutable desktops (Silverblue, Kinoite, bootc)	283
Verify deeply (acceptance checklist)	283
Upstream installer vs Fedora package	284
Uninstall / reset notes	284
Common failures	285
Lab	285
Checkpoint	286
Further depth	286
Store, profiles, and GC on Linux	287

Store, profiles, and GC on Linux	288
Why this chapter matters	288
Store anatomy (quick ops view)	288
Profiles: what they are for	289
Inspect	289
Install / remove (modern)	289
Upgrade	289
Other roots you will meet	289
Garbage collection	290
Safe everyday	290
Delete old profile generations	290
Disk pressure playbook	291
Channels vs flakes (on Fedora)	291
Binary caches	291
Worked lab: profile vs shell	292
Checkpoint	292
Further depth	292
Daily CLI, flakes, and devShells	293
Daily CLI, flakes, and devShells	294
Modern CLI map (muscle memory)	294
First project flake on Fedora	294
nix develop vs nix shell vs profile	296
direnv + nix-direnv (high leverage)	296
Install tools (profile or HM later)	296
direnvrc	296
Shell hook	296
Project	297
Ergonomic extras (optional)	297
Multi-system flakes (preview)	297
CI-shaped local check	298
Lab	298
Checkpoint	298
Further depth	299
Home Manager on Fedora (standalone)	300
Home Manager on Fedora (standalone)	301
Why HM on Fedora	301
Standalone bootstrap (flake-based)	301
1. Minimal home flake skeleton	301
2. home.nix	302
3. First switch	303
4. Verify	303
What to put in HM vs project flakes	303
Generations and rollback	304
Secrets warning	304

Coexistence with Fedora’s bashrc	304
Lab	304
Checkpoint	305
Further depth	305
dnf and Nix coexistence	306
dnf and Nix coexistence	307
PATH reality	307
Decision matrix (printable)	308
Avoid dual service managers for the same app	308
Language version managers vs Nix	308
Graphics / Electron / browsers	309
Disk layout awareness	309
Updates: two clocks	309
Lab	309
Checkpoint	310
Further depth	310
SELinux, Atomic, and day-2 ops	311
SELinux, Atomic, and day-2 ops	312
SELinux: do not “fix” Nix by disabling it	312
Atomic desktops and bootc	312
Problem	312
Pattern A — bind mount	313
Pattern B — rootless / pure user Nix	313
Pattern C — Nix in a toolbox/distrobox	313
Day-2 operations checklist	313
Daemon health	313
Disk	313
Trust and config	314
Upgrade Nix itself (Fedora)	314
Networking / proxies / corporate TLS	314
Backup thoughts	314
Performance tips (laptop)	315
Lab	315
Checkpoint	315
Further depth	316
Bridge to NixOS	317
Bridge to NixOS	318
What stays the same	318
What is new	318
Recommended transition shapes	319
A — Keep Fedora laptop; NixOS server/lab (common)	319
B — Dual boot / spare disk NixOS	319
C — Full cutover	319

Porting checklist (HM → NixOS)	319
First NixOS learning loop (preview)	320
Dual-stack monorepo sketch	320
Skills you should already have (gate)	321
Lab	321
Checkpoint	322
Further depth	322
NixOS Host	323
NixOS host — install, configure, modules	324
NixOS host — install, configure, modules	325
What this part is for	325
Baseline (this book)	325
Learning outcomes	326
Prerequisites	326
Chapter map	327
Suggested learning paths	327
Architecture sketch (part-level)	328
Safety rules for this part	328
Lab workspace convention	328
Part gate (before Home / flake layout)	329
Further depth (this book only)	329
How to use this part as a reference	330
Installing NixOS	331
Installing NixOS	332
Why this chapter matters	332
Theory 1 — What you are installing	332
Release identity	333
Theory 2 — Disposable VM emphasis	333
VM resources (practical)	333
Networking modes (hypervisor)	333
Theory 3 — UEFI vs legacy BIOS	334
ESP requirements	334
Theory 4 — Disk layout options	334
Manual (classic installer path)	334
Disko (intro only)	334
Suggested simple layout (UEFI)	335
Why not exotic FS on first install?	335
Theory 5 — <code>nixos-generate-config</code> outputs	335
What generate-config is good at	335
Theory 6 — Installer networking and time	336
Theory 7 — First boot mental model	336
Activation vs “files on disk”	336
Theory 8 — Credentials hygiene on a lab	336

Theory 9 — <code>system.stateVersion</code>	337
Theory 10 — Minimal install config knobs	337
Worked examples bank	337
Example A — Download image	337
Example B — Partition sketch (sfdisk/parted-level ideas)	338
Example C — Generate + install skeleton	338
Example D — Minimal configuration knobs to set <i>during</i> install	338
Example E — Mount checklist before generate-config	339
Lab 1 — Plan and snapshot strategy	339
Lab 2 — Install NixOS 26.05	339
Lab 3 — Post-install inventory	340
Lab 4 — Preserve hardware config	340
Lab 5 — Break-glass familiarity	341
Exercises	341
Common pitfalls	341
Checkpoint	342
Further depth (this book)	342

Flake-defined hosts 343

Flake-defined hosts 344

Why this chapter matters	344
Theory 1 — <code>nixosConfigurations</code> output	344
Evaluation sketch	345
Theory 2 — Where the flake lives	345
Sudo and paths	345
Theory 3 — Module list composition	346
<code>imports</code> inside <code>configuration.nix</code>	346
Theory 4 — Minimal flake host contents	347
<code>flake.nix</code>	347
<code>hosts/lab/configuration.nix</code>	347
<code>hardware-configuration.nix</code>	348
Theory 5 — First rebuild flow	348
Safe preview builds	348
Theory 6 — <code>specialArgs</code> / <code>extraSpecialArgs</code> (preview)	348
Theory 7 — Hostname vs flake attribute	349
Theory 8 — Git discipline for hosts	349
Flake purity reminder	349
Theory 9 — Multiple hosts (preview)	349
Theory 10 — Leaving installer <code>/etc/nixos</code> behind	350
Worked examples bank	350
Example A — Build without switching (safe)	350
Example B — Switch	350
Example C — Show config value	350
Example D — Show system path	350
Example E — Trace eval errors	350
Lab 1 — Repository layout on the lab host	351
Lab 2 — First flake rebuild	351

Lab 3 — Prove flake is authority	351
Lab 4 — README for future you	352
Lab 5 — Optional: match <code>/etc/nixos</code> to flake	352
Exercises	352
Common pitfalls	353
Checkpoint	353
Further depth (this book)	354
Rebuild modes and generations	355
Rebuild modes and generations	356
Why this chapter matters	356
Theory 1 — System profile and generations	356
Inspect	356
Theory 2 — Rebuild modes	357
Choosing modes	357
Remote note	357
Theory 3 — Activation vs boot default	357
Decision flowchart	358
Theory 4 — Listing and switching generations	358
Boot menu recovery	358
Theory 5 — What is <i>not</i> rolled back	359
Theory 6 — GC and generations	359
Theory 7 — Flake + rebuild flags worth knowing	360
Theory 8 — Activation scripts and failure modes	360
Theory 9 — Naming generations in ops notes	360
Worked examples bank	361
Example A — Safe build	361
Example B — test then switch	361
Example C — rollback	361
Example D — Compare generations (conceptual)	361
Example E — Logs during rebuild	361
Lab 1 — Inventory generations before chaos	361
Lab 2 — Mode drills (harmless package)	362
Lab 3 — Deliberate break → rollback	362
Lab 4 — Boot menu literacy	362
Lab 5 — Incident card	363
Exercises	363
Common pitfalls	364
Checkpoint	364
Further depth (this book)	364
Users and access	365
Users and access	366
Why this chapter matters	366
Theory 1 — Declarative users	366
System users (preview)	367

Theory 2 — Root vs wheel admin	367
Order of hardening	367
Theory 3 — Passwords and the store	368
Why the store leaks matter	368
Theory 4 — SSH service basics	368
Host keys vs user keys	368
Theory 5 — Authorized keys on NixOS	369
Client config hygiene	369
Theory 6 — Groups you will actually use	369
Theory 7 — Home directories and mutability	370
mutableUsers	370
Theory 8 — Sudo and elevation patterns	370
Theory 9 — Debugging SSH failures	371
Theory 10 — Audit checklist for flakes	371
Worked examples bank	371
Example A — Admin user with key	371
Example B — Disable password SSH	372
Example C — Console password without storing plaintext	372
Example D — System user for a service (preview)	372
Example E — Key file in repo	372
Lab 1 — Generate a lab keypair on the client	372
Lab 2 — Wire user + SSH into flake	373
Lab 3 — Verify identity paths	373
Lab 4 — Lockout drill (console required)	373
Lab 5 — Audit flake for secret smells	374
Lab 6 — users.mutableUsers experiment (optional, snapshot)	374
Lab 7 — SSH hard fail checklist	374
Exercises	375
Common pitfalls	375
Checkpoint	376
Further depth (this book)	376
Networking and firewall	377
Networking and firewall	378
Why this chapter matters	378
Theory 1 — Two common network stacks on NixOS	378
Classic DHCP simplicity	378
networkd file shape (server VM)	379
Theory 2 — Host identity	379
Extra hosts	379
Theory 3 — Firewall model	379
nftables awareness (26.05)	380
Theory 4 — Interface names	380
Theory 5 — Static addressing (optional lab)	381
Theory 6 — DNS and resolution	381
Theory 7 — Firewall vs service modules	381

Theory 8 — Rebuild safety pattern for remote operators	382
Lockout drill ethics	382
Theory 9 — Port audit as continuous practice	382
Theory 10 — Hypervisor port forwards	382
Worked examples bank	383
Example A — Server-like DHCP + SSH	383
Example B — Explicit interface DHCP	383
Example C — Temporary debug open (lab only)	383
Example D — networkd DHCP match	383
Example E — DNS pin	383
Lab 1 — Discover ground truth	383
Lab 2 — Declare firewall explicitly	384
Lab 3 — Port audit	384
Lab 4 — Controlled break (console ready)	384
Lab 5 — Network README snippet	385
Lab 6 — Firewall rule listener map	385
Lab 7 — DNS failure drill (safe)	385
Exercises	386
Common pitfalls	386
Checkpoint	387
Further depth (this book)	387
Packages and programs	388
Packages and programs	389
Why this chapter matters	389
Theory 1 — <code>environment.systemPackages</code>	389
Organization pattern	390
Theory 2 — <code>programs.*</code> modules	390
Discovery	390
Theory 3 — Decision table	391
Theory 4 — Closure discipline (light)	391
Theory 5 — Overlays and pins (awareness only)	391
Theory 6 — Unfree and allow flags	392
Theory 7 — System vs user packages	392
Theory 8 — <code>environment.defaultPackages</code> trap	392
Theory 9 — Package name binary name	393
Theory 10 — Layers interaction with generations	393
Worked examples bank	393
Example A — Baseline admin tools	393
Example B — programs integration	394
Example C — Search package attr	394
Example D — Conditional unfree single package	394
Example E — Avoid double install	394
Lab 1 — Declare your daily CLI set	394
Lab 2 — Convert one package to <code>programs.*</code>	395
Lab 3 — Anti-profile audit	395
Lab 4 — Size awareness	395

Lab 5 — Split comments in config	395
Lab 6 — <code>programs.*</code> option archaeology	396
Lab 7 — Remove one redundant package	396
Exercises	396
Common pitfalls	397
Checkpoint	397
Further depth (this book)	397
systemd via Nix	398
systemd via Nix	399
Why this day exists	399
Theory 1 — Module options generate units	399
Theory 2 — Service essentials	399
Absolute paths matter	400
Theory 3 — Timers	400
Theory 4 — State directories and permissions	401
Theory 5 — Inspecting generated units	401
Theory 6 — Relationship to <code>services.*</code> high-level modules	401
Theory 7 — Failure and rollback	402
Worked examples bank	402
Example A — oneshot + timer writing a log	402
Example B — long-running simple service	402
Example C — <code>path</code> for tools in script	402
Lab 1 — Ship <code>lab-tick</code> timer	403
Lab 2 — <code>systemctl cat</code> literacy	403
Lab 3 — One long-running service (localhost)	403
Lab 4 — Break <code>ExecStart</code> on purpose	404
Lab 5 — Document units in README	404
Theory 8 — Hardening starters for custom units	404
Theory 9 — <code>systemd.tmpfiles</code> for paths units need	405
Theory 10 — Activation vs live <code>systemctl edit</code>	405
Worked example D — oneshot with <code>RemainAfterExit</code>	405
Lab 6 — Timer calendar form	405
Lab 7 — Unit dependency map	406
Common gotchas	406
Checkpoint	406
Commit	407
Modules as a consumer	408
Modules as a consumer	409
Why this day exists	409
Theory 1 — A module is a function (usually)	409
Theory 2 — Options vs config	410
Theory 3 — Merge semantics (intuition)	410
Theory 4 — <code>lib.mkIf</code>	411
Theory 5 — <code>lib.mkDefault</code> and priority	411

Theory 6 — How to read a nixpkgs module	411
Step path	411
Local source on a NixOS system	412
What to extract mentally	412
Theory 7 — Consumer anti-patterns	412
Theory 8 — <code>imports</code> as composition	413
Worked examples bank	413
Example A — Read openssh options	413
Example B — Override a default carefully	413
Example C — When <code>mkForce</code> appears in blogs	413
Lab 1 — Study three modules	414
Lab 2 — Change behavior via options only	414
Lab 3 — Trace an option to source	415
Lab 4 — Priority experiment (safe)	415
Lab 5 — Consumer checklist	415
Common gotchas	416
Checkpoint	416
Commit	416

Writing custom modules 417

Writing custom modules 418

Why this day exists	418
Theory 1 — Minimal custom module shape	418
Theory 2 — <code>mkOption</code> and types (starter set)	419
Theory 3 — <code>cfg = config...</code> pattern	419
Theory 4 — Assertions	420
Theory 5 — Import into the host	420
Theory 6 — Namespace your options	421
Theory 7 — What to extract first	421
Worked examples bank	421
Example A — Enable site	421
Example B — Assertion trip	421
Example C — Package override	421
Lab 1 — Create module file	422
Lab 2 — Add a real assertion	422
Lab 3 — Extract something from your host config	422
Lab 4 — Disable path	423
Lab 5 — Document the module	423
Theory 8 — <code>mkDefault</code> vs bare assignment (producer side)	423
Theory 9 — Option descriptions are UX	424
Theory 10 — Module args you receive	424
Worked example D — warnings for soft policy	424
Lab 6 — <code>nixos-option</code> on your option	425
Lab 7 — Priority conflict drill	425
Common gotchas	425
Checkpoint	425
Commit	426

Boot and hardware	427
Boot and hardware	428
Why this day exists	428
Theory 1 — Boot chain (UEFI lab)	428
Theory 2 — systemd-boot vs GRUB	428
Generation UI	429
Theory 3 — Kernel packages	429
Theory 4 — Firmware	430
Theory 5 — CPU / virt hardware bits	430
Theory 6 — <code>hardware-configuration.nix</code> discipline	430
Theory 7 — Temporary boot parameters	431
Worked examples bank	431
Example A — Document current loader	431
Example B — Kernel identity	431
Example C — Safe param experiment	431
Lab 1 — Boot path document	431
Lab 2 — Verify loader config matches reality	432
Lab 3 — Kernel package awareness	432
Lab 4 — Firmware decision	432
Lab 5 — Recovery dry-run (no panic)	433
Theory 8 — <code>boot.loader.timeout</code> and generation menu	433
Theory 9 — Initrd and LUKS awareness (preview)	433
Theory 10 — <code>hardware-configuration.nix</code> vs flake purity	434
Worked example D — QEMU guest agent (virt)	434
Lab 6 — Loader entries inventory	434
Lab 7 — Kernel param one-shot (snapshot)	434
Common gotchas	435
Checkpoint	435
Commit	435
Filesystems and state	436
Filesystems and state	437
Why this day exists	437
Theory 1 — Three layers of “the disk”	437
Theory 2 — <code>fileSystems</code> options	437
Theory 3 — Why UUID/LABEL	438
Theory 4 — State directories that matter	438
Theory 5 — Impermanence idea (concept only)	439
Why people want it	439
Costs	439
Theory 6 — Store is not a backup of state	440
Theory 7 — tmpfs and ephemeral mounts (awareness)	440
Worked examples bank	440
Example A — Mount inventory	440
Example B — What’s big?	440
Example C — SSH host key paths	440

Lab 1 — Full inventory table	441
Lab 2 — Persist classification	441
Lab 3 — State created by your Stage II work	441
Lab 4 — Impermanence reading (light)	442
Lab 5 — Danger checklist	442
Theory 8 — Btrfs / ZFS awareness without deploying them	442
Theory 9 — /nix on its own mount (optional design)	442
Theory 10 — Bind mounts as proto-impermanence	443
Worked example D — <code>neededForBoot</code> meaning	443
Lab 6 — What survives <code>nixos-rebuild switch</code> ?	444
Lab 7 — Size budget sentence	444
Common gotchas	444
Checkpoint	444
Commit	445

Home and Flake Layout 446

Home Manager and flake layout 447

Home Manager and flake layout 448

What this part is for	448
Prerequisites	449
Chapter map	449
Mental model — three planes	450
Dependency direction (import DAG)	450
Suggested repo tree (target shape)	451
How to work this part	451
Cross-cutting conventions (26.05)	452
What this part does <i>not</i> cover	452
Skills checklist (end of part)	452
How chapters are structured	453
Reading strategies	453
Lab host assumptions	453
Next part	454

Flake repository layout 455

Flake repository layout 456

Why this chapter exists	456
Theory 1 — What “layout” means in a flake monorepo	456
Target tree	457
Theory 2 — Dependency direction (prevent circular imports)	458
What belongs where	458
Theory 3 — Host entry pattern	459
<code>default.nix</code> vs explicit path	460
<code>specialArgs</code> vs module arguments	460
Theory 4 — Module “barrel” files	460
Optional deeper split	461

Theory 5 — Hardware config and purity	461
Theory 6 — Roles vs common vs services	461
Theory 7 — Multi-host readiness (without building the second machine)	462
Theory 8 — Relative paths and flake root	462
Worked example — Minimal SSH + users + nix split	463
Worked example — Thin flake with optional lib	464
Exercises	465
Exercise 1 — Inventory the monolith	465
Exercise 2 — Create the directory skeleton	465
Exercise 3 — Split common modules	465
Exercise 4 — Host entry + thin flake	465
Exercise 5 — Build before switch	465
Exercise 6 — Document the tree	466
Exercise 7 — Role extract (optional stretch)	466
Exercise 8 — Path-break drill	466
Exercise 9 — Diff against backup	466
Exercise 10 — Second-host skeleton (docs only)	466
Exercise 11 — Secrets flag list	466
Exercise 12 — Commit hygiene	467
Common gotchas	467
Checkpoint	467
Journal (optional)	468

Lock file discipline 469

Lock file discipline 470

Why this chapter exists	470
Theory 1 — What <code>flake.lock</code> actually pins	470
Inputs vs follow-ons	471
Anatomy of a lock node (conceptual)	471
Theory 2 — Update operations (precision toolkit)	471
Mental model	472
Theory 3 — How to read a lock diff	472
Nested duplicates	473
Theory 4 — Pin strategies (pick a house rule)	473
Security vs stability	473
Theory 5 — Registries (light literacy)	474
Registry vs input (summary)	474
Theory 6 — Multi-machine and “reproducible”	474
Clone checklist	474
Theory 7 — Commit policy for locks	475
Revert pattern	475
Theory 8 — Inputs that should co-move	475
Worked example — Selective update narrative	476
Worked commands	476
Exercises	476
Exercise 1 — Snapshot current pins	476
Exercise 2 — Read the lock structure	477

Exercise 3 — Selective update of one input	477
Exercise 4 — Prove “update all” is louder	477
Exercise 5 — Document house rules	478
Exercise 6 — Registry awareness (read-only)	478
Exercise 7 — follows fix (if needed)	478
Exercise 8 — Compare URL construction	478
Exercise 9 — Impurity awareness list	478
Exercise 10 — CI anti-pattern note	479
Exercise 11 — Paired restore drill	479
Exercise 12 — Multi-input update plan	479
Common gotchas	479
Checkpoint	480
Journal (optional)	480

Home Manager on NixOS 481

Home Manager on NixOS 482

Why this chapter exists	482
Theory 1 — Three ways people run Home Manager	482
Why module mode first	483
Theory 2 — Input wiring (26.05-aligned)	483
Where to put the HM block	484
Theory 3 — System user vs HM user	484
<code>home.stateVersion</code>	485
Name and path must match	485
Theory 4 — What belongs in HM vs NixOS	485
Theory 5 — Activation model (what rebuild does)	486
Standalone commands (awareness)	486
Theory 6 — Programs modules vs raw packages	487
Shell source of truth	487
Theory 7 — Splitting the home tree	487
Theory 8 — Backup and collision with existing dots	488
Worked example — Minimal useful user	488
Worked example — HM wiring module (multi-host friendly)	489
Exercises	490
Exercise 1 — Add the input and lock	490
Exercise 2 — Stub → real home module	490
Exercise 3 — Rebuild and verify	490
Exercise 4 — Change only HM, rebuild once	490
Exercise 5 — Deliberate failure drill	491
Exercise 6 — Journal: boundary decisions	491
Exercise 7 — Activation log literacy	491
Exercise 8 — Split programs.nix	491
Exercise 9 — Package placement check	491
Exercise 10 — Shell source of truth	491
Exercise 11 — Backup existing dots	491
Exercise 12 — Version alignment proof	492
Exercise 13 — Negative case: standalone not required	492

Common gotchas	492
Checkpoint	492
Journal (optional)	493
Home files and XDG	494
Home files and XDG	495
Why this chapter exists	495
Theory 1 — How HM places files	495
Escape hatches (know, use sparingly)	495
Theory 2 — <code>home.file</code> vs <code>xdg.configFile</code>	496
Minimal examples	496
Theory 3 — Content sources: <code>text</code> , <code>source</code> , <code>templates</code>	497
Recursive directories	497
Theory 4 — XDG directories	497
Classic vs XDG	498
Theory 5 — Permissions and clobber behavior	498
Mode options	499
Theory 6 — Theming (optional, bounded)	499
Theory 7 — Multi-machine user splits	499
Theory 8 — <code>programs.*</code> vs raw files for the same tool	500
Theory 9 — What never goes in managed files	500
Worked example — Migrate git ignores + editor config	501
Worked example — Structured home tree	501
Exercises	502
Exercise 1 — Inventory live dotfiles	502
Exercise 2 — First managed XDG file	502
Exercise 3 — Migrate via <code>source</code>	502
Exercise 4 — Conflict drill	502
Exercise 5 — Secret anti-pattern (fake only)	502
Exercise 6 — Structure the home tree	503
Exercise 7 — Recursive config tree	503
Exercise 8 — Executable home file	503
Exercise 9 — <code>programs</code> vs file for one tool	503
Exercise 10 — Multi-host plan	503
Exercise 11 — Cache vs config	503
Exercise 12 — Activation after migrate	504
Exercise 13 — Documentation	504
Common gotchas	504
Checkpoint	504
Journal (optional)	505
Home Manager user services	506
Home Manager user services	507
Why this chapter exists	507
Theory 1 — Systemd user vs system	507
Lingering	507

Theory 2 — HM service options	508
HM attrset shape	508
Theory 3 — Mapping system knowledge to user bus	509
Theory 4 — Simple vs oneshot vs long-running	509
Theory 5 — Paths and packages in units	510
WorkingDirectory and paths	510
Theory 6 — Prefer HM modules when they exist	510
Theory 7 — Targets, wants, and session lifecycle	511
Theory 8 — Debugging workflow	511
Worked example — Notes fetcher timer	511
Worked example — Calendar timer variants	512
Exercises	512
Exercise 1 — User systemd baseline	512
Exercise 2 — Heartbeat oneshot + timer	513
Exercise 3 — Journal debugging	513
Exercise 4 — One service you actually want	513
Exercise 5 — Boundary check	513
Exercise 6 — Import services.nix	513
Exercise 7 — list-unit-files literacy	514
Exercise 8 — Failure restart policy	514
Exercise 9 — OnCalendar experiment	514
Exercise 10 — Wrong bus drill	514
Exercise 11 — Secret audit of unit text	514
Exercise 12 — Graphical vs headless note	514
Exercise 13 — Comparison table in docs	514
Exercise 14 — Clean disable	514
Common gotchas	515
Checkpoint	515
Journal (optional)	515

direnv and project shells 516

direnv and project shells 517

Why this chapter exists	517
Theory 1 — devShells in a flake	517
Multi-system sketch	518
Why not always put shells in the NixOS flake?	518
Theory 2 — direnv + nix-direnv	519
HM installation pattern	519
Verify hook	519
Theory 3 — .envrc with use flake	519
Named shells	520
.envrc is not for secrets	520
Theory 4 — System packages vs project shells	520
Theory 5 — Pinning discipline for project flakes	521
Theory 6 — nix develop vs nix shell vs HM packages	521
inputsFrom (awareness)	521
Theory 7 — shellHook hygiene	521

Theory 8 — Host flake devShell for Nix ops	522
Worked example — Multi-tool Python shell	522
Worked example — Isolation proof layout	523
Exercises	523
Exercise 1 — Install direnv via HM	523
Exercise 2 — Create a project flake	523
Exercise 3 — Wire direnv	523
Exercise 4 — Prove version isolation	524
Exercise 5 — Thin the profile (careful)	524
Exercise 6 — Document the workflow	524
Exercise 7 — Commit project lock	524
Exercise 8 — Reload after lock bump	524
Exercise 9 — Quiet shellHook	524
Exercise 10 — Host ops shell	524
Exercise 11 — Ad-hoc vs project	525
Exercise 12 — gitignore hygiene	525
Exercise 13 — Failure: allow missing	525
Exercise 14 — Boundary table	525
Common gotchas	525
Checkpoint	526
Journal (optional)	526

flake-parts (optional structure) 527

flake-parts (optional structure) 528

Why this chapter exists	528
Theory 1 — Pain that motivates structure	528
Theory 2 — Hand-rolled discipline (no flake-parts)	529
House rules for hand-rolled	530
Theory 3 — flake-parts approach	530
Costs	531
Split flake-parts modules (optional)	531
Theory 4 — Decision rubric	532
Theory 5 — What not to do	532
Theory 6 — Formatter as a style signal	532
Theory 7 — Overlay injection point	533
Worked example — Split hand-rolled helpers	533
Worked example — Decision record template	533
Exercises	534
Exercise 1 — Audit current outputs	534
Exercise 2 — Pick and write the decision	534
Exercise 3A — If flake-parts: adopt on a branch	534
Exercise 3B — If hand-rolled: extract helpers	534
Exercise 4 — Merge the winner	535
Exercise 5 — Teach the style to “future you”	535
Exercise 6 — Formatter end-to-end	535
Exercise 7 — specialArgs survival	535
Exercise 8 — Show vs build	535

Exercise 9 — follows still holds	535
Exercise 10 — Negative hybrid cleanup	535
Exercise 11 — Systems honesty	536
Exercise 12 — Style PR description	536
Common gotchas	536
Checkpoint	536
Journal (optional)	537
Flake checks	538
Flake checks	539
Why this chapter exists	539
Theory 1 — What checks are	539
Theory 2 — Hand-rolled checks skeleton	540
Host toplevel (strong, heavy)	540
Lighter host check — eval-oriented	540
Theory 3 — flake-parts checks	541
Theory 4 — Format check that actually fails	541
Alternative: treefmt (awareness)	541
Theory 5 — What belongs in check vs push-time CI only	541
Theory 6 — nix flake check UX	542
Common failure modes	542
Theory 7 — Checks and secrets	542
Theory 8 — Wiring consistency with pkgs/overlays	543
Worked example — Practical minimal set	543
Worked example — Script wrapper	543
Exercises	544
Exercise 1 — Baseline without checks	544
Exercise 2 — Add devShell as a check	544
Exercise 3 — Add a format check that can fail	544
Exercise 4 — Optional host toplevel check	544
Exercise 5 — Developer loop script (optional)	544
Exercise 6 — Journal: CI shape	544
Exercise 7 — Single-attr debug	545
Exercise 8 — cleanSource impact	545
Exercise 9 — System gate	545
Exercise 10 — Pre-commit habit	545
Exercise 11 — Document check inventory	545
Exercise 12 — Secret scan of check context	545
Exercise 13 — Fail-closed demo	545
Exercise 14 — Overlay consistency	546
Common gotchas	546
Checkpoint	546
Journal (optional)	547
Overlays (light introduction)	548

Overlays (light introduction)	549
Why this chapter exists	549
Theory 1 — What an overlay is	549
Fixed-point intuition	550
Theory 2 — Wiring overlays on NixOS	550
Flake-level pkgs for shells/checks	550
Theory 3 — <code>overrideAttrs</code> vs <code>override</code> vs new package	551
Theory 4 — Composition order	552
Theory 5 — When not to overlay	552
Theory 6 — Pinning a package from another nixpkgs (light)	552
Theory 7 — Overlays vs <code>packageOverrides</code> (classic)	553
Theory 8 — Infinite recursion patterns	553
Worked example — Shared overlay module	553
Worked example — HM sees the overlay	554
Exercises	555
Exercise 1 — Create overlay file	555
Exercise 2 — Attach to NixOS	555
Exercise 3 — Attach to devShell / checks pkgs	555
Exercise 4 — Prove HM sees it (if useGlobalPkgs)	555
Exercise 5 — Intentional override discipline	555
Exercise 6 — Negative space	555
Exercise 7 — Composition list	556
Exercise 8 — Failure: overlay not wired	556
Exercise 9 — Document overlays	556
Exercise 10 — Check integration	556
Exercise 11 — Prefer wrapper over override	556
Exercise 12 — Search before overlay	556
Exercise 13 — Recursion caution note	556
Exercise 14 — Upgrade readiness	556
Common gotchas	557
Checkpoint	557
Journal (optional)	557
Part close-out	557
 Services and Security	 559
Services and security patterns	560
Services and security patterns	561
What this part is for	561
Prerequisites	562
Chapter map	562
Mental model — service slice	562
Suggested module layout (extends previous part)	563
How to work this part	563
Cross-cutting conventions (26.05)	564
Security non-goals (this part)	564

Skills checklist (end of part)	565
Reading strategies	565
Relationship to other parts	565
Lab host assumptions	566
Next parts	566
The secrets problem	567
The secrets problem	568
Why this chapter exists	568
Theory 1 — The store is not a vault	568
Evaluation vs runtime	569
Theory 2 — Common anti-patterns catalog	569
builtins.readFile trap	570
Theory 3 — What “good” looks like (preview)	570
Theory 4 — Generations and GC	570
Theory 5 — Multi-user threat on a “lab laptop”	571
Theory 6 — Distinguishing config from secret	571
Theory 7 — Process listings and logs	572
Theory 8 — Git history is forever (enough)	572
Worked example — Finding secrets with search	572
Worked example — Safe store readability demo	573
Exercises	573
Exercise 1 — Threat model paragraph	573
Exercise 2 — Audit the flake for anti-patterns	573
Exercise 3 — Prove store readability (safe demo)	573
Exercise 4 — User password hygiene check	573
Exercise 5 — Remove anti-patterns and rebuild	574
Exercise 6 — Incident checklist card	574
Exercise 7 — HM file audit	574
Exercise 8 — Environment variable audit	574
Exercise 9 — Generation awareness	574
Exercise 10 — Backup story note	574
Exercise 11 — Public vs private key check	575
Exercise 12 — Document TODO list for sops	575
Exercise 13 — Peer review simulation	575
Exercise 14 — CI log risk	575
Common gotchas	575
Checkpoint	576
Journal (optional)	576
sops-nix	577
sops-nix	578
Why this chapter exists	578
Theory 1 — Components	578
Flow	579

Theory 2 — Age keys (host-centric pattern)	579
Key backup	579
Theory 3 — Flake input	580
Theory 4 — Minimal NixOS sops config	581
Theory 5 — Creating encrypted files	581
Adding a secret later	582
Theory 6 — HM secrets (awareness)	582
Theory 7 — Bootstrapping chicken-and-egg	582
SSH host key method (awareness)	583
Theory 8 — systemd integration patterns	583
Theory 9 — Multi-host recipients	583
Worked example — Secret for a custom script service	584
Exercises	584
Exercise 1 — Add input and module	584
Exercise 2 — Age keys and <code>.sops.yaml</code>	584
Exercise 3 — First encrypted secret file	585
Exercise 4 — Declare <code>sops.secrets</code> and rebuild	585
Exercise 5 — Runtime consumer	585
Exercise 6 — Failure drill	585
Exercise 7 — Docs	585
Exercise 8 — Selective lock update	586
Exercise 9 — Permission check	586
Exercise 10 — Second secret	586
Exercise 11 — No readFile audit	586
Exercise 12 — Journal hygiene	586
Exercise 13 — Recipient add rehearsal (docs)	586
Exercise 14 — Bootstrap checklist in git	586
Common gotchas	586
Checkpoint	587
Journal (optional)	587

agenix (comparison) 588

agenix (comparison) 589

Why this chapter exists	589
Theory 1 — agenix mental model	589
Flow	590
Theory 2 — Flake input sketch	590
Theory 3 — secrets.nix shape	590
Identity reuse	591
Theory 4 — NixOS declaration	591
Theory 5 — Side-by-side comparison	591
When agenix might still win for you	592
When sops-nix wins for this curriculum	592
Theory 6 — Dual-running risks	592
Theory 7 — Declaring house default	592
Theory 8 — Re-encryption and rotation	593
Theory 9 — Path API discipline (both tools)	593

Worked example — Same placeholder secret both ways	593
Exercises	594
Exercise 1 — Add agenix input	594
Exercise 2 — Public keys and secrets.nix	594
Exercise 3 — Encrypt one age secret	594
Exercise 4 — NixOS module + rebuild	594
Exercise 5 — Consumer parity	594
Exercise 6 — Comparison write-up	595
Exercise 7 — Decommission the non-default	595
Exercise 8 — docs/secrets.md house default section	595
Exercise 9 — follows check	595
Exercise 10 — Incident note	595
Exercise 11 — Team rationale	595
Exercise 12 — Path-only audit	595
Exercise 13 — Rebuild with single stack	596
Exercise 14 — Future secret list	596
Common gotchas	596
Checkpoint	596
Journal (optional)	597

Host hardening patterns 598

Host hardening patterns 599

Why this chapter exists	599
Theory 1 — Threat model for this lab	599
Role split	600
Theory 2 — SSH baseline (NixOS)	600
Optional allowlists	600
Theory 3 — sudo and wheel	601
Theory 4 — Disable what you do not use	601
Theory 5 — Kernel / sysctl care	602
Theory 6 — Automatic updates? (judgment)	602
Theory 7 — Module shape	602
Theory 8 — Assertions as guardrails	603
Theory 9 — Safe apply order	603
Worked example — Recovery card	603
Exercises	604
Exercise 1 — Inventory surface	604
Exercise 2 — Write hardening module	604
Exercise 3 — Apply carefully	604
Exercise 4 — Lockout recovery drill (document)	604
Exercise 5 — Assertions (optional polish)	604
Exercise 6 — Diff and explain	604
Exercise 7 — Key-before-password-off proof	604
Exercise 8 — wheel policy document	605
Exercise 9 — Role wiring	605
Exercise 10 — PermitRootLogin check	605
Exercise 11 — Disable list rationale	605

Exercise 12 — Sysctl minimalism	605
Exercise 13 — Workstation exception note	605
Exercise 14 — Generation rollback rehearsal (console available)	605
Common gotchas	606
Checkpoint	606
Journal (optional)	606
Firewall discipline	607
Firewall discipline	608
Why this chapter exists	608
Theory 1 — NixOS firewall model	608
Prefer service options	608
Theory 2 — Deny by default	609
Theory 3 — Per-service matrix (living document)	609
Theory 4 — Localhost vs LAN vs world	609
Theory 5 — Temporary debug holes	610
Theory 6 — Interaction with containers (preview)	610
Theory 7 — Interfaces and trusted nets	610
Cloud security groups	611
Theory 8 — Connection tracking	611
Theory 9 — Co-locate opens with services	611
Worked example — Tight SSH + HTTPS owned by proxy	611
Worked example — Service openFirewall pattern	612
Exercises	612
Exercise 1 — Audit current opens	612
Exercise 2 — Enable firewall explicitly	612
Exercise 3 — Remove unjustified ports	612
Exercise 4 — Service-tied open (stub)	612
Exercise 5 — Localhost service experiment	613
Exercise 6 — ports.md committed	613
Exercise 7 — Diff firewall after change	613
Exercise 8 — Ping policy decision	613
Exercise 9 — SSH tunnel habit	613
Exercise 10 — Cloud SG note	613
Exercise 11 — Orphan port hunt	613
Exercise 12 — UDP inventory	614
Exercise 13 — test generation safety	614
Exercise 14 — Pre-proxy checklist	614
Common gotchas	614
Checkpoint	614
Journal (optional)	615
TLS and reverse-proxy front door	616
TLS and reverse-proxy front door	617
Why this chapter exists	617
Theory 1 — Choose one proxy	617

Theory 2 — ACME on NixOS	618
Secrets	618
Theory 3 — Caddy pattern (NixOS)	618
Theory 4 — Nginx + ACME pattern	619
Theory 5 — Backend for the lab	619
Theory 6 — Failure modes	620
Theory 7 — Certificate storage and renewal	620
Theory 8 — HTTP → HTTPS redirect	621
Theory 9 — Internal TLS without public ACME	621
Worked example — ports.md dual-write	621
Exercises	621
Exercise 1 — Pick proxy and document	621
Exercise 2 — Localhost backend	622
Exercise 3 — Enable proxy module	622
Exercise 4 — TLS path	622
Exercise 5 — Headers and hygiene (light)	622
Exercise 6 — Break/fix	622
Exercise 7 — ports.md + proxy.md dual-write	622
Exercise 8 — Certificate expiry awareness	622
Exercise 9 — ss audit	623
Exercise 10 — ACME journal	623
Exercise 11 — Staging note	623
Exercise 12 — WebSocket flag (if needed)	623
Exercise 13 — Single proxy enforcement	623
Exercise 14 — Secret path for DNS-01 (plan)	623
Exercise 15 — Smoke from remote	623
Common gotchas	624
Checkpoint	624
Journal (optional)	624

A data service pattern 625

A data service pattern 626

Why this chapter exists	626
Theory 1 — One DB only (pattern depth)	626
Theory 2 — Minimal PostgreSQL module	627
Theory 3 — State directories and disk	627
Theory 4 — Credentials	628
Theory 5 — App connection story	628
Theory 6 — Backup thought (required cognitive work)	628
Theory 7 — Upgrades	629
Theory 8 — <code>ensureDatabases</code> vs migrations	629
Theory 9 — Socket vs TCP local connections	629
Worked example — Service + docs	630
Worked example — Timer sketch for dumps (optional)	630
Exercises	630
Exercise 1 — Enable PostgreSQL	630
Exercise 2 — Database and role	631

Exercise 3 — Wire a secret password path	631
Exercise 4 — Network posture	631
Exercise 5 — Manual dump	631
Exercise 6 — Failure narrative	631
Exercise 7 — Schema independence proof	631
Exercise 8 — Document restore dry-run	632
Exercise 9 — dataDir documentation	632
Exercise 10 — Peer auth admin path	632
Exercise 11 — Package pin note	632
Exercise 12 — Backup location permissions	632
Exercise 13 — App connection string template	632
Exercise 14 — ports.md final for DB	632
Common gotchas	633
Checkpoint	633
Journal (optional)	633

NixOS containers 634

NixOS containers 635

Why this chapter exists	635
Theory 1 — When containers help on NixOS	635
Decision micro-script	636
Theory 2 — <code>containers.<name></code> (NixOS nspawn)	636
Pros / cons	637
Theory 3 — OCI awareness (without next-chapter depth)	637
Theory 4 — Security boundaries (honest)	637
Hardening checklist (guest)	638
Theory 5 — State and containers	638
Theory 6 — Relation to proxy / firewall	638
Theory 7 — Rebuild UX and nested eval	639
Theory 8 — Feature flag for demos	639
Worked example — Prefer native, demo container optional	639
Exercises	639
Exercise 1 — Decision record	639
Exercise 2 — Explore host tools	640
Exercise 3 — One declarative container or dry config	640
Exercise 4 — Reach the guest service	640
Exercise 5 — Compare rebuild UX	640
Exercise 6 — When you would use OCI instead	640
Exercise 7 — Firewall / ports doc	641
Exercise 8 — Nested stateVersion	641
Exercise 9 — Journal path	641
Exercise 10 — Bind mount caution note	641
Exercise 11 — Disable cleanly	641
Exercise 12 — Eval time measurement	641
Exercise 13 — Diagram	641
Exercise 14 — Threat model honesty	641
Common gotchas	641

Checkpoint	642
Journal (optional)	642
Podman and Docker patterns	643
Podman and Docker patterns	644
Why this chapter exists	644
Theory 1 — Podman vs Docker on NixOS	644
Theory 2 — Rootless vs rootful	645
Theory 3 — Declarative containers (OCI)	645
Theory 4 — Images and supply chain (awareness)	646
Theory 5 — Compare to nspawn	646
Theory 6 — Firewall and proxy integration	647
Theory 7 — Stateful volumes	647
Theory 8 — Rootless paths and user namespaces	647
Theory 9 — Imperative snowflakes vs flake	648
Worked example — Full small stack slice	648
Exercises	648
Exercise 1 — Enable Podman (or Docker)	648
Exercise 2 — Declarative OCI container	648
Exercise 3 — Imperative contrast (then undo)	649
Exercise 4 — Wire to proxy (optional but valuable)	649
Exercise 5 — Secret anti-pattern check	649
Exercise 6 — Comparison table filled	649
Exercise 7 — Volume inventory	649
Exercise 8 — Update story one-pager	649
Exercise 9 — autoStart reboot proof	649
Exercise 10 — Port conflict check	649
Exercise 11 — docker group risk note	650
Exercise 12 — Image pin upgrade	650
Exercise 13 — Disable cleanly	650
Exercise 14 — Isolation.md final	650
Common gotchas	650
Checkpoint	650
Journal (optional)	651
Observability patterns	652
Observability patterns	653
Why this chapter exists	653
Theory 1 — Three pillars (light)	653
Theory 2 — Logs-first pattern (valid)	653
Theory 3 — Prometheus pattern	654
Theory 4 — Grafana (optional layer)	655
Theory 5 — What to scrape for <i>your</i> slice	655
Theory 6 — Firewall and multi-tenant risk	656
Theory 7 — Alerting (awareness only)	656
Theory 8 — Retention and disk	656

Theory 9 — Authn in front of Grafana	656
Worked example — Minimal metrics module set	657
Worked example — Runbook skeleton	657
Exercises	657
Exercise 1 — Choose path	657
Exercise 2A — Metrics path	657
Exercise 2B — Logs-first path	658
Exercise 3 — One service deep dive	658
Exercise 4 — Access path	658
Exercise 5 — Update ports and isolation docs	658
Exercise 6 — Capstone foreshadow	658
Exercise 7 — One dashboard or one journal recipe	658
Exercise 8 — Disk use snapshot	659
Exercise 9 — PromQL basics (metrics path)	659
Exercise 10 — Password hygiene for Grafana	659
Exercise 11 — Unit list for the slice	659
Exercise 12 — Error priority journal	659
Exercise 13 — Tunnel muscle memory	659
Exercise 14 — Docs cross-link	659
Common gotchas	660
Checkpoint	660
Journal (optional)	660

k3s (optional awareness) 661

k3s (optional awareness) 662

Why this chapter exists	662
Theory 1 — Decision rubric	662
Theory 2 — What k3s is	663
Theory 3 — NixOS vs k8s configuration	663
Theory 4 — Single-node honesty	664
Theory 5 — Minimal literacy tasks (if enabled)	664
Theory 6 — Skip path is first-class	664
Theory 7 — When NixOS modules beat k8s on a single node	665
Theory 8 — Resource floor honesty	665
Theory 9 — Feature flag module	665
Worked example — Disable / enable toggles	666
Worked example — Tiny workload (if enabled)	666
Exercises	667
Exercise S — Skip path (most readers)	667
Exercise S2 — Skip artifact quality bar	667
Exercise K1 — Opt-in: enable k3s	667
Exercise K2 — Tiny workload	667
Exercise K3 — Tear-down plan	667
Exercise K4 — Compare control planes	668
Exercise K5 — API surface inventory (if enabled)	668
Exercise 6 — Resource estimate	668
Exercise 7 — Secrets boundary note	668

Exercise 8 — Proxy interaction	668
Exercise 9 — Decision commit	668
Exercise 10 — FOMO check	668
Exercise 11 — Revisit calendar	668
Exercise 12 — ports.md consistency	668
Exercise 13 — Disable half-install	669
Exercise 14 — Part retrospective	669
Common gotchas	669
Checkpoint	669
If skipped	669
If enabled	669
Journal (optional)	670
Part close-out	670
Packaging	671
Packaging craft	672
Packaging craft	673
What this part is for	673
Prerequisites	674
Chapter map	674
Mental model — three layers of “a package”	675
stdenv is the spine	675
FODs vs normal builds	675
Caches vs builders (preview)	676
Lab workspace conventions	676
How chapters are structured	677
Reading paths	677
Path A — “I need one internal CLI in nixpkgs style”	677
Path B — “CI is compiling the world”	678
Path C — “We forked half of nixpkgs”	678
Glossary (part-local)	678
Quality bar for “done packaging”	679
Relationship to other parts	679
Anti-patterns to avoid while studying	679
Part checkpoint (before you leave packaging)	680
Suggested lab commit series	680
How to use this part as reference	681
Next	681
stdenv and build phases	682
stdenv and build phases	683
Why it matters	683
Theory 1 — What <code>stdenv.mkDerivation</code> actually is	683
<code>buildInputs</code> vs <code>nativeBuildInputs</code> (cross-aware)	684
Evaluation vs realization	684

Theory 2 — Default phase order	684
Phase responsibilities	685
Hooks API (muscle memory)	685
Theory 3 — Important environment variables in the sandbox	686
Flags that skip work	686
Theory 4 — Minimal patterns you will reuse forever	687
Shell-only “package” (no compile)	687
Autotools / simple C (upstream tarball)	687
Single C file without autotools	688
Flat single-file source (unpack quirks)	688
Theory 5 — <code>finalAttrs</code> vs <code>rec</code>	689
Theory 6 — What fixup actually does (short list)	689
Worked example — flake that exports packages	690
Host consumption (optional)	691
Worked example — intentional phase map document	691
Lab — multi-step	692
Step 1 — Scaffold	692
Step 2 — Write three packages	692
Step 3 — Break phases on purpose	693
Step 4 — Flip input kinds	693
Step 5 — Phase introspection	693
Step 6 — Drop <code>runHook</code> once	693
Step 7 — Parallel build flag	693
Step 8 — Host or shell consumption	694
Exercises	694
Common gotchas	694
Checkpoint	695
Commit	695

Fetchers and fixed-output derivations 696

Fetchers and fixed-output derivations 697

Why it matters	697
Theory 1 — Fixed-output vs normal derivations	697
Why the store path is special	698
Theory 2 — Common fetchers (2026 mental map)	698
<code>fetchFromGitHub</code> pattern	698
Submodules and deep clones	699
Theory 3 — The hash mismatch workflow	699
Correct response	699
Prefetch helpers	700
SRI vs base32	700
Theory 4 — Impure traps and purity	700
Theory 5 — Flake inputs vs package fetchers	701
Theory 6 — <code>outputHashMode</code> (awareness)	702
Theory 7 — Private sources and credentials	702
Worked examples	703
Example A — tarball package skeleton	703

Example B — GitHub + patch FOD	703
Example C — intentional mismatch	704
Example D — flake wiring	704
Lab — multi-step	704
Step 1 — Prefetch a known tarball	704
Step 2 — Package from that tarball	705
Step 3 — Force a mismatch	705
Step 4 — GitHub fetcher	705
Step 5 — Patch as FOD and as path	705
Step 6 — Document the trust boundary	705
Step 7 — Offline check	706
Step 8 — Submodules experiment (optional)	706
Exercises	706
Common gotchas	706
Checkpoint	707
Commit	707

Language ecosystems (part A) 708

Language ecosystems (part A) 709

Why it matters	709
Theory 1 — How language builders relate to stdenv	709
Shared skeleton	710
Theory 2 — Go path (<code>buildGoModule</code>)	710
Local module (lab-friendly)	711
<code>vendorHash</code> workflow	711
Useful knobs	711
Example <code>src/hello-go</code>	712
Theory 3 — Rust path (<code>buildRustPackage</code>)	712
Git dependencies in Cargo	713
Useful knobs	713
Theory 4 — Python path (application vs library)	713
pytest pattern (sketch)	714
Theory 5 — Node (<code>buildNpmPackage</code>) sketch	715
Theory 6 — Version injection and <code>meta.mainProgram</code>	715
Theory 7 — Tests and purity	716
Worked example — flake with Go tool + check	716
Lab — multi-step	717
Step 1 — Choose primary language	717
Step 2 — Minimal upstream project	717
Step 3 — First package expression	717
Step 4 — Wire <code>meta.mainProgram</code> and run	718
Step 5 — Version injection	718
Step 6 — Tests	718
Step 7 — Consume from a devShell	718
Step 8 — Break the vendor/cargo/npm hash	718
Step 9 — Closure glance	718
Exercises	719

Common gotchas	719
Checkpoint	719
Commit	720
Language ecosystems (part B)	721
Language ecosystems (part B)	722
Why it matters	722
Theory 1 — Pattern map across ecosystems	722
Theory 2 — Choose B deliberately	723
Transfer questions (answer in notes)	723
Theory 3 — Wrapping and runtime deps	723
Theory 4 — Overrides preview	724
Theory 5 — Cross-ecosystem FOD discipline	725
Theory 6 — Runtime vs build-time closure	725
Theory 7 — Dual interface pattern	725
Worked example — dual packages in one flake	726
Tiny Python app (if A was compiled)	727
Tiny Node CLI (alternative)	728
Tiny Go (if A was Python)	728
Lab — multi-step	728
Step 1 — Import or copy the part A package	728
Step 2 — Scaffold smaller B project	728
Step 3 — Package with the B builder	729
Step 4 — Comparison table (required)	729
Step 5 — Unified check	729
Step 6 — Optional same interface	729
Step 7 — Closure glance	729
Step 8 — Hash bump drill	730
Step 9 — Light override	730
Exercises	730
Common gotchas	730
Checkpoint	731
Commit	731
Patching and overrides	732
Patching and overrides	733
Why it matters	733
Theory 1 — Three layers of change	733
Theory 2 — <code>overrideAttrs</code>	734
<code>override</code> vs <code>overrideAttrs</code>	734
Composing safely	734
Theory 3 — Patches as first-class inputs	735
Generating a patch	735
Theory 4 — Overlay composition	735
Order matters	736
Infinite recursion patterns	736

NixOS / HM consumption	736
Theory 5 — Pinning and “don’t rebuild the world”	737
Theory 6 — Scoped injection without global overlays	737
Theory 7 — Language-specific override helpers	738
Worked examples	738
Example A — local patch on <code>hello</code>	738
Example B — overlay adding your ecosystem A package	738
Example C — <code>packageOverrides</code> legacy note	739
Example D — flake export	739
Lab — multi-step	739
Step 1 — Baseline	739
Step 2 — Write a patch	739
Step 3 — <code>overrideAttrs</code> only (no overlay)	740
Step 4 — Promote to overlay	740
Step 5 — Stack two overlays	740
Step 6 — Dependency override	740
Step 7 — Maintenance note	740
Step 8 — Intentional drop of upstream patches	740
Exercises	741
Common gotchas	741
Checkpoint	741
Commit	742

Multiple outputs 743

Multiple outputs 744

Why it matters	744
Theory 1 — Outputs are separate store paths	744
First output is special	745
Theory 2 — Conventional output roles	745
Environment variables in the build	745
Theory 3 — Why dependents get smaller	746
Reference leaks	746
Theory 4 — <code>moveToOutput</code> and helpers	746
Theory 5 — Consuming multi-output packages	747
Multiple outputs and <code>nix shell</code>	747
Theory 6 — <code>separateDebugInfo</code>	747
Theory 7 — When single-output is fine	748
Worked example — toy multi-output C library + app	748
Lab — multi-step	750
Step 1 — Inspect an existing multi-out package	750
Step 2 — Build libgreet with three outputs	750
Step 3 — Verify separate paths	751
Step 4 — Build greetapp against libgreet	751
Step 5 — Deliberate mistake	751
Step 6 — Document policy	751
Step 7 — Optional <code>separateDebugInfo</code>	751
Step 8 — pkg-config break/fix	751

Exercises	752
Common gotchas	752
Checkpoint	752
Commit	753
Debugging builds	754
Debugging builds	755
Why it matters	755
Theory 1 — Where information lives	755
Theory 2 — Eval failure vs build failure	755
Theory 3 — Keep failed trees	756
Theory 4 — <code>breakpointHook</code>	756
Theory 5 — Common failure classes	757
Reading a log quickly	757
Theory 6 — Sandbox: don't disable casually	757
Theory 7 — Reproducing CI failures locally	758
Bisect mental model	758
Theory 8 — Save the drv path	759
Worked example — scripted break/fix series	759
Failure dossier template	760
Worked example — personal runbook skeleton	760
Lab — multi-step	760
Step 1 — Baseline success package	760
Step 2 — Inject a compile failure	761
Step 3 — <code>--keep-failed</code>	761
Step 4 — Inject an install failure	761
Step 5 — FOD failure recall	761
Step 6 — <code>breakpointHook</code> (Linux recommended)	761
Step 7 — Write a personal runbook	761
Step 8 — Force local rebuild	761
Step 9 — Complete failure dossier	761
Step 10 — Sandbox proof note	762
Exercises	762
Common gotchas	762
Checkpoint	763
Commit	763
Navigating nixpkgs	764
Navigating nixpkgs	765
Why it matters	765
Theory 1 — Mental map of nixpkgs (2026)	765
by-name rules of thumb	765
Theory 2 — Anatomy of a PR-shaped <code>package.nix</code>	766
Modern habits	767
Theory 3 — <code>callPackage</code> and fixed-points	767
Why function heads matter	767

Theory 4 — Reading existing packages as literature	768
What to look for	768
Theory 5 — Tests and <code>passthru</code> (awareness)	768
Theory 6 — Category tree vs by-name coexistence	769
Theory 7 — Contribution hygiene (even if you never PR)	769
Worked example — flake package laid out like by-name	770
Lab — multi-step	770
Step 1 — Locate packages	771
Step 2 — Reshape your best packaging package	771
Step 3 — Overlay + flake export	771
Step 4 — Meta completeness pass	771
Step 5 — PR-shaped checklist	771
Step 6 — Optional: clone nixpkgs and dry-place the file	771
Step 7 — Compare category tree vs by-name	772
Step 8 — Literature notes	772
Exercises	772
Common gotchas	772
Checkpoint	773
Commit	773

Binary caches 774

Binary caches 775

Why it matters	775
Theory 1 — Substituters vs builders	775
Theory 2 — Trust model (non-negotiable)	776
Theory 3 — Cachix (hosted, popular)	776
Push full runtime closure	777
Theory 4 — Self-hosted: Attic & Harmonia	777
Harmonia sketch (NixOS)	777
Attic sketch (conceptual)	777
Theory 5 — Push what matters	778
Theory 6 — Substituter order and priority	778
Theory 7 — What “cache miss” really means	779
Force local build (contrast)	779
Theory 8 — Daemon vs user config	779
Worked examples	780
NixOS module fragment for team cache	780
Local file NAR cache (always available lab path)	780
Query whether a path would substitute	780
Lab — multi-step	780
Step 1 — Observe default cache	780
Step 2 — Build your package cold-ish	781
Step 3 — Choose push target	781
Step 4 — Second machine / second user story	781
Step 5 — Trust write-up	781
Step 6 — CI note	781
Step 7 — Negative test	781

Step 8 — Document lab cache policy	782
Exercises	782
Common gotchas	782
Checkpoint	783
Commit	783
Remote builders	784
Remote builders	785
Why it matters	785
Theory 1 — Cache vs remote builder	785
Theory 2 — Configuration surfaces (2026)	786
NixOS (<code>nix.buildMachines</code>)	786
Ad-hoc one-shot	786
Features	786
Theory 3 — SSH and store talk	787
Theory 4 — Cross and multi-system	787
Theory 5 — Security notes	788
Decision card — do I need a builder?	788
Theory 6 — Scheduling knobs that matter	788
Upload storms	789
Theory 7 — Builder host minimal config	789
Topology diagram (keep in notes)	789
Worked example — two-VM lab checklist	789
Prove which machine built	790
Lab — multi-step	790
Step 1 — Inventory local capacity	790
Step 2 — Provision builder	790
Step 3 — SSH from daemon-relevant user	791
Step 4 — One-shot remote build	791
Step 5 — Persist config	791
Step 6 — Feature flag experiment	791
Step 7 — Combine with cache (optional)	791
Step 8 — Write topology diagram	791
Step 9 — Failure injection	791
Step 10 — Disk pressure note	792
Exercises	792
Common gotchas	792
Checkpoint	793
Commit	793
Reproducibility (honest view)	794
Reproducibility (honest view)	795
Why it matters	795
Theory 1 — Strength ladder	795
How to talk about this in a runbook	796

Theory 2 — Input-addressed sameness	796
Path sameness vs byte sameness	796
Theory 3 — Why bit-for-bit fails	796
Fixed-output derivations (FODs)	797
Theory 4 — Tools for comparison	797
Theory 5 — Operational reproducibility vs academic	798
Theory 6 — What Nix does <i>not</i> make reproducible	798
Theory 7 — Cache vs reproducibility	798
Worked example — measurement script	799
Worked example — deliberate nondeterminism	799
Worked example — REPRO.md template	800
Two-machine log template	800
Lab — multi-step	800
Step 1 — Environment record	800
Step 2 — Pick three artifacts	801
Step 3 — L1 measurements	801
Step 4 — L2 if you have two environments	801
Step 5 — Substitute false control	801
Step 6 — Deliberate nondeterminism (lab only)	801
Step 7 — Closure drift story	801
Step 8 — Honesty statement	801
Step 9 — Optional diffoscope	802
Step 10 — Cache trust paragraph	802
Step 11 — README claim scrub	802
Exercises	802
Common gotchas	802
Checkpoint	803
Commit	803
ROS 2 robotics with nix-ros-overlay	804
ROS 2 robotics with nix-ros-overlay	805
Why this chapter exists	805
Mental model	806
Status and expectations (be honest)	806
Binary caches (do this first)	807
Official community cache (Cachix)	807
Experimental Attic cache (when Cachix is thin)	807
Fast path: try upstream examples (no project yet)	808
Flake develop (recommended)	808
Classic nix-shell form	808
Project flake: ROS 2 underlay in a devShell	809
Minimal <code>flake.nix</code>	809
direnv	811
Choosing a ROS 2 distro attr	811
Workspace workflow (overlay underlay + your packages)	812
Why not put <i>everything</i> in Nix immediately?	812
Custom ROS packages (direction of travel)	812

Graphics: RViz, Gz, tools on non-NixOS	813
NixOS host integration (what belongs where)	813
Fedora dual-stack note	814
Troubleshooting	814
Security and supply chain	815
Lab A — Hello underlay	815
Lab B — Project flake	815
Lab C — Distro compare (optional)	816
Lab D — Robot appliance sketch (optional)	816
Checkpoint	816
Further depth (this book)	816
Upstream	817
Ops and Fleet	818
Operations and fleet	819
Operations and fleet	820
What this part is for	820
Prerequisites	820
Chapter map	821
Mental model — fleet pipeline	822
Disk and identity	822
Deploy tools at a glance	822
Lab topology suggestions	823
How chapters are structured	824
Reading paths	824
Path A — “Homelab single server → clean reinstall”	824
Path B — “Three nodes with roles”	824
Path C — “Cloud image factory”	825
Glossary (part-local)	825
Safety rules (non-negotiable)	825
Quality bar for “fleet ready”	826
Relationship to other parts	826
Anti-patterns	826
Part checkpoint (before you leave ops)	827
Suggested lab commit series	827
How to use this part as reference	828
Next	828
Disko	829
Disko	830
Why it matters	830
Theory 1 — What Disko is	830
Theory 2 — Minimal GPT + ext4 root (UEFI)	831
Prefer stable device paths	832

Theory 3 — Flake integration	832
Evaluate mounts without wiping	833
Theory 4 — LUKS sketch (common homelab)	833
Theory 5 — btrfs subvolumes (impermanence-ready)	834
ZFS note	834
Theory 6 — Host-specific devices without copy-paste hell	835
Theory 7 — Swap, BIOS, and multi-disk	835
Swap	835
Legacy BIOS / hybrid	836
Multi-disk	836
Theory 8 — Interaction with hardware-configuration.nix	836
Worked example — VM lab checklist	836
DISK.md template	837
Lab — multi-step	837
Step 1 — Snapshot safety	837
Step 2 — Add Disko input	837
Step 3 — Write simplest UEFI layout	837
Step 4 — Dry evaluation	837
Step 5 — Apply on VM	838
Step 6 — Reinstall proof (core)	838
Step 7 — Optional LUKS or btrfs	838
Step 8 — Document recovery	838
Step 9 — Wrong-disk tabletop	838
Exercises	838
Common gotchas	838
Checkpoint	839
Commit	839

Impermanence (idea and patterns) 840

Impermanence (idea and patterns) 841

Why it matters	841
Theory 1 — Model	841
What “disappears” means	842
Theory 2 — <code>impermanence</code> module (nix-community)	842
<code>neededForBoot</code>	843
Theory 3 — btrfs rollback-on-boot pattern	843
Theory 4 — What breaks first (expect it)	843
Theory 5 — Soft impermanence	844
Theory 6 — Interaction with generations and secrets	844
Theory 7 — Home Manager alignment	845
Services checklist (starting set)	845
Worked example — minimal VM proof	846
Worked example — IMPERMANENCE.md template	847
Lab — multi-step	847
Step 1 — Decide commit level	847
Step 2 — State inventory	848
Step 3 — Implement level A or B	848

Step 4 — Proof reboot	848
Step 5 — Secrets path	848
Step 6 — Rollback story interaction	848
Step 7 — If Level C	848
Step 8 — Persist list review	848
Step 9 — Second surprise reboot	848
Exercises	849
Common gotchas	849
Checkpoint	849
Commit	850
deploy-rs	851
deploy-rs	852
Why it matters	852
Theory 1 — Deploy pipeline	852
Theory 2 — Flake skeleton with deploy-rs	853
Theory 3 — SSH prerequisites	854
Theory 4 — Magic rollback / confirmation	855
Theory 5 — Build locality	855
Cache interaction	855
Theory 6 — Profiles beyond <code>system</code>	856
Theory 7 — vs <code>nixos-rebuild --target-host</code>	856
Worked example — trivial visible change	856
Generation awareness	857
Worked example — RUNBOOK seed	857
Lab — multi-step	857
Step 1 — Target ready	857
Step 2 — Add deploy-rs input	857
Step 3 — Wire <code>deploy.nodes</code>	858
Step 4 — First deploy	858
Step 5 — Prove activation	858
Step 6 — Second deploy delta	858
Step 7 — Failure drill (safe)	858
Step 8 — Compare one-liner	858
Step 9 — Document RUNBOOK seed	858
Step 10 — Generation awareness after deploy	858
Step 11 — Cache interaction	859
Step 12 — Multi-node sketch (optional)	859
Exercises	859
Common gotchas	859
Checkpoint	860
Commit	860
Colmena	861
Colmena	862
Why it matters	862

Theory 1 — Hive mental model	862
Theory 2 — Flake-based Colmena (modern)	863
Theory 3 — Tags and blast radius	864
Parallelism	864
Theory 4 — Evaluation vs activation	864
Theory 5 — Relationship to <code>nixosConfigurations</code>	865
Theory 6 — When Colmena vs <code>deploy-rs</code>	865
Theory 7 — Secrets and <code>specialArgs</code>	865
Worked example — two-node hive (one local)	866
Lab — multi-step	867
Step 1 — Install Colmena in a shell	867
Step 2 — Define hive with defaults	867
Step 3 — Two nodes	867
Step 4 — Build only	867
Step 5 — Apply tagged subset	867
Step 6 — Tag experiment	867
Step 7 — Blast-radius note	867
Step 8 — Decision card	867
Step 9 — Optional parallel apply	868
Step 10 — DRY modules	868
Exercises	868
Common gotchas	868
Checkpoint	869
Commit	869

Shared modules across hosts 870

Shared modules across hosts 871

Why it matters	871
Theory 1 — Layering	871
Theory 2 — Role module pattern	871
Theory 3 — Profile module pattern	872
Theory 4 — Options API design	873
Theory 5 — Directory layout that scales	873
Theory 6 — <code>SpecialArgs</code> and lib helpers	874
Theory 7 — Testing shared modules	874
Worked example — before/after host thinness	875
Before (bad)	875
After (good)	875
Worked example — flake wiring both deploy tools	876
Lab — multi-step	876
Step 1 — Inventory duplication	876
Step 2 — Extract <code>profiles/baseline.nix</code>	876
Step 3 — Extract one role	876
Step 4 — Thin hosts	876
Step 5 — Override test	876
Step 6 — Wire into Colmena/ <code>deploy-rs</code>	876
Step 7 — Option docs	877

Step 8 — Negative test	877
Step 9 — README map	877
Exercises	877
Common gotchas	877
Checkpoint	878
Commit	878
Images with nixos-generators	879
Images with nixos-generators	880
Why it matters	880
Theory 1 — Image = NixOS config + format	880
Mental model	881
Theory 2 — nixos-generators integration shapes	881
Flake app / package patterns	881
Module specializations	881
Theory 3 — Same modules, different entrypoints	882
Theory 4 — Cloud-init and metadata boundary	882
Theory 5 — Build performance and caches	883
Theory 6 — ISO as recovery	883
Worked example — qcow lab image (shape)	883
Worked example — consistency checklist	884
Lab — multi-step	884
Step 1 — Pick a format	885
Step 2 — Wire generators input	885
Step 3 — Image module entry	885
Step 4 — Build artifact	885
Step 5 — Boot once	885
Step 6 — Consistency proof	885
Step 7 — Size diet (optional)	885
Step 8 — Document CI policy	885
Step 9 — Stretch cloud	885
Exercises	886
Common gotchas	886
Checkpoint	886
Commit	887
Terraform boundary	888
Terraform boundary	889
Why it matters	889
Theory 1 — Responsibility split	889
Theory 2 — Handoff patterns	890
Pattern A — Image + launch	890
Pattern B — Generic distro image + first deploy	890
Pattern C — NixOS on metal (no TF)	890
Theory 3 — What to put in TF state vs flake lock	890
Theory 4 — OpenTofu vs Terraform	891

Theory 5 — Minimal resource graph (sketch)	891
Theory 6 — Secrets across the boundary	892
Theory 7 — Anti-patterns gallery	892
Worked example — BOUNDARY.md template	893
Worked example — inventory bridge	893
Lab — multi-step	894
Step 1 — Write BOUNDARY.md	894
Step 2 — Draw resource graph	894
Step 3 — Choose pattern A/B/C	894
Step 4 — Optional implement	894
Step 5 — User-data audit	894
Step 6 — State backend note	894
Step 7 — Failure story	894
Step 8 — Stretch destroy	895
Exercises	895
Common gotchas	895
Checkpoint	895
Commit	896
nixosTest	897
nixosTest	898
Why this day exists	898
Theory 1 — What a nixosTest is	898
Theory 2 — Minimal single-node test	898
Theory 3 — testScript essentials	900
Theory 4 — What to test (high value)	900
Theory 5 — Running tests	900
Worked example — web role smoke test	901
Lab — multi-step	901
Step 1 — Confirm virtualization	902
Step 2 — Hello test	902
Step 3 — Port your module	902
Step 4 — Make it fail once	902
Step 5 — Wire checks	902
Step 6 — Document runtime	902
Step 7 — Multi-node stretch	902
Theory 6 — Tests as flake checks	903
Theory 7 — Interactive vs pure test runs	903
Worked example — wait for unit + port	903
Lab — one negative assertion	903
Lab — time the test	904
Common gotchas	904
Checkpoint	904
Commit	904
More testing patterns	905

More testing patterns	906
Why this day exists	906
Theory 1 — Test pyramid for NixOS fleets	906
Theory 2 — Three checks minimum (template)	907
Theory 3 — Lightweight non-VM checks	907
Format / lint	907
Eval / build host configuration	908
Structured eval assertions	908
Theory 4 — Anti-flake tactics	908
Theory 5 — Mapping tests to modules	909
Theory 6 — Suite layout and CI shape	909
Worked example — fail-inject once	910
Lab — multi-step	910
Step 1 — Inventory today’s checks	911
Step 2 — Add check #1 (package)	911
Step 3 — Add check #2 (nixosTest)	911
Step 4 — Add check #3 (config)	911
Step 5 — Fail inject matrix	911
Step 6 — Timing budget	911
Step 7 — README table	911
Step 8 — Stability	911
Step 9 — Secrets policy for tests	912
Common gotchas	912
Checkpoint	912
Commit	912
CI pipelines for flakes	913
CI pipelines for flakes	914
Why this day exists	914
Theory 1 — Minimal trustworthy pipeline	914
Trust boundaries	914
Theory 2 — GitHub Actions workflow (2026-shaped)	915
Theory 3 — Cachix (and friends) in CI	916
House rules	916
Alternatives	917
Theory 4 — What not to run on free runners	917
Timeouts and disks	918
Theory 5 — GitLab CI sketch	918
Theory 6 — CI vs deploy	918
Worked example — selected checks job	919
Worked example — intentional red PR	919
Lab — multi-step	919
Step 1 — Add workflow file	920
Step 2 — First green CI	920
Step 3 — Wire cache	920
Step 4 — PR simulation	920
Step 5 — Secrets hygiene	920

Step 6 — Consumer config	920
Step 7 — Document runner needs	920
Step 8 — Status badge (optional)	920
Step 9 — Deploy from CI? (decide)	921
Common gotchas	921
Checkpoint	921
Commit	921
GC and store hygiene	922
GC and store hygiene	923
Why this day exists	923
Theory 1 — What GC deletes	923
Mental model	923
Theory 2 — Generations vs store GC	924
User vs system profiles	924
Theory 3 — Policy template	924
Theory 4 — Finding what holds space	925
CI-specific roots	926
Theory 5 — optimise (hardlinks)	926
Theory 6 — Incident patterns	926
Worked example — hygiene report script	927
Lab — multi-step	927
Step 1 — Baseline metrics	927
Step 2 — Map roots	927
Step 3 — Safe cleanup	927
Step 4 — Generation policy	928
Step 5 — Enable automatic GC on lab host	928
Step 6 — optimise	928
Step 7 — CI runner note	928
Step 8 — Policy doc	928
Step 9 — Rollback still possible?	928
Step 10 — Profile audit (user + HM)	928
Step 11 — Dry-run before aggressive delete	929
Worked example — min-free incident story	929
Common gotchas	929
Checkpoint	930
Commit	930
Secure Boot and TPM (awareness)	931
Secure Boot and TPM (awareness)	932
Why this day exists	932
Theory 1 — Problem statement	932
Theory 2 — Lanzaboote overview (2026)	933
Theory 3 — TPM2 + LUKS	933
PCR trade-offs	934

Theory 4 — VM lab limits	934
QEMU/swtpm note (advanced lab)	935
Theory 5 — Recovery & break-glass	935
Theory 6 — Relation to Disko, deploy, and images	935
Worked example — dossier template	936
Lab — multi-step	937
Step 1 — Probe hardware	937
Step 2 — Choose track	937
Step 3 — Read upstream (required even for Track C)	937
Step 4 — Recovery assets first	937
Step 5 — Implement or simulate	938
Step 6 — Failure plan drill (written)	938
Step 7 — Link Disko / LUKS	938
Step 8 — Fleet policy paragraph	938
Step 9 — Tabletop: evil maid + firmware update	938
Step 10 — Capstone A vs B honesty line	938
Worked example — recovery USB checklist	939
Common gotchas	939
Checkpoint	939
Commit	940

Backup story 941

Backup story 942

Why this day exists	942
Theory 1 — What to back up	942
Inventory questions	942
Theory 2 — Tool choice (pick one)	943
3-2-1 reminder	943
Theory 3 — Restic on NixOS (shape)	943
Borgmatic sketch	944
ZFS	944
Theory 4 — Consistency	944
Theory 5 — Restore is the product	945
Success criteria	945
Theory 6 — Secrets and backup encryption	945
Worked example — local restic lab (no cloud)	946
Worked example — RTO/RPO honesty	946
Lab — multi-step	947
Step 1 — Choose tool	947
Step 2 — State inventory	947
Step 3 — Declare backup job in Nix	947
Step 4 — First backup	947
Step 5 — Restore drill (required)	947
Step 6 — DB drill (if you have the data service pattern chapter DB)	947
Step 7 — Offsite note	947
Step 8 — Runbook	948
Step 9 — Negative cases table	948

Common gotchas	948
Checkpoint	949
Commit	949
Internals	950
Internals and currency	951
Internals and currency	952
Learning outcomes	952
Chapter map	953
How this part connects	954
Suggested study modes	954
Lab workspace convention	955
Acceptance bar for the part	955
Baseline and policy	956
Common traps in this part	957
Next	957
The evaluator	958
The evaluator	959
Why this chapter exists	959
Theory 1 — Evaluation is a graph of thunks	959
Values vs thunks	959
Why modules love laziness	960
Eager edges you already know	960
Theory 2 — Infinite recursion patterns	961
Pattern A — Direct self-reference	961
Pattern B — Mutual recursion without a base case	961
Pattern C — Attrset self-reference (legal <i>shape</i> , dangerous force)	961
Pattern D — Module / option cycles (production shape)	961
Pattern E — <code>with</code> + shadowing confusion	962
Pattern F — <code>lib.fix</code> / fixed points gone wrong	962
Theory 3 — Debugging toolkit	962
<code>builtins.trace</code>	962
<code>builtins.tryEval</code>	963
<code>seq</code> vs <code>deepSeq</code>	963
<code>lib.debug.traceVal</code> / <code>traceSeq</code> / <code>traceValSeq</code> (<code>nixpkgs</code>)	963
<code>NIX_SHOW_STATS</code> and evaluator metrics	963
<code>nix-instantiate --eval</code> / <code>nix eval</code>	963
Bisect strategy for real flakes	964
Theory 4 — When to force (and when not to)	964
Theory 5 — Eval vs build vs activation (where time goes)	964
Worked example — Construct, diagnose, fix	965
Broken	965
Fixed	965
Module-shaped fix	965

Force a leaf, not the world	966
Lab 0 — Workspace	966
Lab 1 — Watch memoization	966
Lab 2 — Infinite recursion zoo	967
Lab 3 — <code>seq</code> vs <code>deepSeq</code>	967
Lab 4 — Trace a tricky real eval (your flake)	967
Lab 5 — Intentional soft failure with <code>tryEval</code>	968
Lab 6 — Module-shaped cycle (optional, high value)	968
Lab 7 — Stats snapshot	969
Failure modes (quick reference)	969
Common gotchas	969
Checkpoint	970
Commit	970
Next	970
Module system in depth	971
Module system in depth	972
Why this chapter exists	972
Theory 1 — Anatomy of a module (recall with precision)	972
Freeform modules (awareness)	973
Theory 2 — Type catalog (practical reference)	973
Scalars & paths	973
Collections	974
Structure	974
Function-ish / paths to callables	974
Useful helpers	974
Choosing types under pressure	975
Theory 3 — Definitions: <code>mkIf</code> , <code>mkMerge</code> , priorities	975
<code>mkIf</code>	975
<code>mkMerge</code>	975
Priority ladder (high level)	976
Example conflict	976
Debugging silent loss	976
Theory 4 — Assertions & warnings	977
Theory 5 — Submodules & <code>attrsOf</code> services	977
Theory 6 — Import graphs and role modules	978
Worked example — Redesign sketch	978
Before (weak)	978
After (typed)	979
Bad configs that must fail	980
Lab 0 — Workspace	980
Lab 1 — Type catalog cheat sheet (you write it)	980
Lab 2 — Redesign one module	981
Lab 3 — Priority drill	981
Lab 4 — Intentional bad config	982
Lab 5 — Multi-instance sketch (optional)	982
Lab 6 — Description quality audit	982

Failure modes	982
Common gotchas	983
Checkpoint	983
Commit	983
Next	983
Store and daemon	984
Store and daemon	985
Why this chapter exists	985
Theory 1 — Single-user vs multi-user	985
Trusted users	986
Allowed users	986
Threat sketches	986
Theory 2 — Store paths, validity, and roots	986
Path anatomy (recall)	986
Validity	986
GC roots	987
Disk full playbook (operator)	987
Theory 3 — Sandbox (Linux)	987
Why sandbox	987
Mechanisms (Linux)	988
Config knobs	988
Fixed-output derivations (FOD)	988
Nested virt / KVM note	989
Theory 4 — Substituters and signatures	989
Flow	989
Trust policy	989
Signing your own cache (awareness)	990
Theory 5 — Reading build logs	990
Theory 6 — Daemon lifecycle on NixOS	990
Worked example — Trust interview (self)	991
Lab 0 — Workspace	991
Lab 1 — Map live roots	991
Lab 2 — Daemon & trust surface	992
Lab 3 — Sandbox log literacy	992
Lab 4 — Optional impurity demo (VM only)	993
Lab 5 — Signature refusal mental model	993
Lab 6 — Elitebook / KVM lab acceptance	993
Common gotchas	994
Checkpoint	994
Commit	995
Next	995
Nix performance	996
Nix performance	997
Why this chapter exists	997

Theory 1 — The rebuild time budget	997
Elitebook 16 GB reality	998
Theory 2 — Jobs vs cores	998
Definitions	998
Choosing numbers (rules of thumb)	998
Remote builders	999
Theory 3 — Eval performance	999
Eval cache	999
IFD cost	999
Module count	1000
Memory	1000
Theory 4 — Download & store IO	1000
Theory 5 — What <i>not</i> to do	1000
Theory 6 — CI budget design	1001
Worked example — Measurement protocol	1001
Interpreting results	1002
Lab 0 — Workspace	1002
Lab 1 — Baseline timings	1002
Lab 2 — Tune jobs/cores	1003
Lab 3 — Cache hit awareness	1003
Lab 4 — Eval hygiene checklist	1003
Lab 5 — Remote builder decision (write-only if no hardware)	1004
Lab 6 — Memory pressure drill (lab only)	1004
Common gotchas	1004
Checkpoint	1004
Commit	1005
Next	1005

Content-addressed derivations (awareness) 1006

Content-addressed derivations (awareness) 1007

Why this chapter exists	1007
Theory 1 — Input-addressed world (today's default)	1007
FODs vs CA (do not conflate)	1008
Theory 2 — Content-addressed idea	1008
Theory 3 — Experimental feature surface	1008
Reading status (primary sources)	1009
Theory 4 — What CA does <i>not</i> magically fix	1009
Theory 5 — Relationship to Capstone and 26.05	1010
Theory 6 — Trust and substitutes under CA thinking	1010
Theory 7 — Operational decision template	1011
Worked example — Conceptual comparison card	1011
Lab 0 — Workspace	1011
Lab 1 — Version & feature inventory (required)	1012
Lab 2 — Choose a track	1012
Track A — Optional tiny experiment (if feature available)	1012
Track B — Status memo without enabling (always acceptable)	1012
Lab 3 — Teach-back diagram	1013

Lab 4 — Nixpkgs awareness scan	1013
Lab 5 — Explicit non-adoption	1013
Lab 6 — Analogy card (teach others)	1014
Interaction with other internals chapters	1014
Common gotchas	1014
Checkpoint	1015
Commit	1015
Next	1015
nixosTest mastery	1016
nixosTest mastery	1017
Why this chapter exists	1017
Theory 1 — What nixosTest really runs	1017
Theory 2 — Flake (flaky) sources	1018
Theory 3 — Driver API patterns (Python DSL)	1018
Anti-pattern	1019
Multi-node	1019
Theory 4 — Systemd-aware testing	1019
Theory 5 — Secrets and tests	1020
Theory 6 — CI integration	1020
Elitebook / KVM lab acceptance	1020
Theory 7 — Machine objects and multi-node patterns	1021
Theory 8 — Snapshotting vs rebuild cost	1021
Worked example — systemd journal assertion	1021
Worked example — regression that fails when firewall wrong	1021
Lab 0 — Workspace	1022
Lab 1 — Inventory existing tests	1022
Lab 2 — Eliminate sleep-only waits	1022
Lab 3 — Add one regression that would have bitten you	1022
Lab 4 — Multi-node or client/server (stretch)	1023
Lab 5 — Failure UX	1023
Lab 6 — Flake check wiring audit	1023
Lab 7 — Failure screenshot discipline	1023
Common gotchas	1024
Checkpoint	1024
Commit	1024
Next	1024
Ecosystem map	1025
Ecosystem map	1026
Why this chapter exists	1026
Theory 1 — Layers of “ecosystem”	1026
Theory 2 — devenv (project environments)	1027
Compare to house default	1027
Theory 3 — Snowfall (flake library / structure)	1028
Compare to flake-parts	1028

Theory 4 — nix-darwin	1028
Theory 5 — Decision framework (keep/drop)	1029
Theory 6 — Ecosystem postcard (vocabulary)	1029
Theory 7 — Integration risks matrix	1029
Worked example — COMPARE.md skeleton	1030
Lab 0 — Workspace	1031
Lab 1 — Map your current layout (30 min)	1031
Lab 2 — Pick one tool and trial	1031
If devenv	1031
If Snowfall	1032
If nix-darwin	1032
Lab 3 — Comparison table (required)	1032
Lab 4 — Ecosystem postcard	1032
Lab 5 — Capstone protection check	1033
Lab 6 — Teach-back (5 bullets)	1033
Common gotchas	1033
Checkpoint	1034
Commit	1034
Next	1034

Comparative landscape 1035

Comparative landscape 1036

Why this chapter exists	1036
Theory 1 — Comparison dimensions	1036
Theory 2 — NixOS (your baseline)	1037
Theory 3 — Guix System	1037
Theory 4 — Image-based / immutable distros	1038
Theory 5 — Ansible (and friends)	1039
Theory 6 — Honesty checklist (avoid tribal takes)	1039
Theory 7 — Decision time-bounds	1040
Worked example — weighted scorecard fragment	1040
Lab 0 — Workspace	1040
Lab 1 — Primary sources (skimming with intent)	1041
Lab 2 — Scorecard for <i>your</i> context	1041
Lab 3 — Essay (required)	1041
Lab 4 — Elevator version	1042
Lab 5 — Capstone implication	1042
Lab 6 — Scenario cards (forced choice)	1042
Card A — Homelab “weird topology”	1042
Card B — Employer fleet	1042
Card C — Kubernetes worker pool	1042
Lab 7 — Quote hygiene	1043
Worked example — elevator closer	1043
Common gotchas	1043
Checkpoint	1044
Commit	1044
Next	1044

Reading release notes (26.05) 1045

Reading release notes (26.05) 1046

Why this chapter exists	1046
Theory 1 — What “a NixOS release” actually is	1046
Theory 2 — How to read release notes like an operator	1047
Pass A — Inventory (30–45 min)	1047
Pass B — Personal filter (45–60 min)	1047
Pass C — Prove with eval/build	1048
Pass D — Warnings are future breakages	1048
Theory 3 — <code>stateVersion</code> is not a fashion number	1048
Theory 4 — Cross-check Nix itself	1048
Theory 5 — Module renames and aliases (kinds of change)	1049
Theory 6 — Inputs beyond <code>nixpkgs</code>	1049
Theory 7 — Severity and scheduling	1050
Worked example — IMPACT.md row	1050
Lab 0 — Workspace	1051
Lab 1 — Capture current pin	1051
Lab 2 — Official notes intake	1051
Lab 3 — Repo audit scripts	1052
Lab 4 — Build verification	1052
Lab 5 — Personal “gotchas from 26.05”	1052
Lab 6 — Feed upgrade planning	1053
Common gotchas	1053
Checkpoint	1053
Commit	1054
Next	1054

Upgrade planning 1055

Upgrade planning 1056

Why this chapter exists	1056
Theory 1 — What actually changes in a flake upgrade	1056
Theory 2 — Staging strategy	1057
Environments	1057
Git strategy	1057
Companion inputs	1057
Theory 3 — Tests-first gate	1057
Theory 4 — Rollback is part of the plan	1058
Concrete rollback commands (fill for your path)	1058
Theory 5 — <code>stateVersion</code> and stateful services	1059
Theory 6 — Communication & windows	1059
Abort criteria template	1059
Theory 7 — Elitebook / small-lab constraints	1060
Runbook template (you will fill this)	1060
0. Metadata	1060
1. Preconditions	1060
2. Release notes	1061

3. Technical steps (lab first)	1061
4. Health verification	1061
5. Rollback	1061
6. Risks	1061
7. Decision	1061
Lab 0 — Workspace	1062
Lab 1 — Inventory pins	1062
Lab 2 — Write the full runbook	1062
Lab 3 — Dry-run mechanics on lab (optional execution)	1062
Lab 4 — Test plan matrix	1063
Lab 5 — Abort & rollback tabletop	1063
Lab 6 — Disk space math	1063
Common gotchas	1064
Checkpoint	1064
Commit	1064
Next	1064
Troubleshooting drills	1065
Troubleshooting drills	1066
Why this chapter exists	1066
Theory 1 — Failure class matrix	1066
Theory 2 — Golden command palette	1067
Remote when SSH dies	1067
Elitebook lab specifics	1067
Theory 3 — Injection catalog (choose carefully)	1068
Theory 4 — Time discipline	1068
Theory 5 — Playbook entry format	1068
Theory 6 — Rollback as a diagnostic tool	1069
Theory 7 — Layered isolation order	1069
Worked example — playbook stub	1070
Lab 0 — Snapshot	1070
Lab 1 — Baseline health	1070
Lab 2 — Inject & recover #1 (eval or build)	1071
Lab 3 — Inject & recover #2 (service or secrets)	1071
Lab 4 — Inject & recover #3 (network) — console required	1071
Lab 5 — Compile master index	1072
Lab 6 — Timeboxed chaos #4 (state)	1072
Lab 7 — Master index cross-links	1072
Lab 8 — End-state health	1072
Common gotchas	1072
Checkpoint	1073
Commit	1073
Next	1073
Architecture review	1074

Architecture review	1075
Why this chapter exists	1075
Theory 1 — Capstone A bar (recall)	1075
Theory 2 — Coverage matrix (topic parts)	1076
Theory 3 — Architecture views (write these)	1077
A. Context	1077
B. Containers of code	1077
C. Runtime view	1077
D. Trust view	1077
E. Failure view	1077
F. Data view	1078
Theory 4 — Prioritization	1078
Theory 5 — Non-goals for Capstone A	1079
Theory 6 — Evidence standards	1079
Theory 7 — Dossier structure (feeds Capstone)	1079
Lab 0 — Workspace	1080
Lab 1 — Automated inventory	1080
Lab 2 — Coverage matrix	1080
Lab 3 — ARCHITECTURE.md (required)	1081
Lab 4 — Prioritized backlog	1081
Lab 5 — Checklist freeze seed	1082
Lab 6 — Risk register (short)	1082
Lab 7 — Sequence plan	1082
Common gotchas	1083
Checkpoint	1083
Commit	1083
Next	1083
 Capstone	 1084
Capstone and recovery	1085
Capstone and recovery	1086
Capstone A vs optional bar	1086
Learning outcomes	1087
Chapter map	1087
Suggested sequence	1087
Dossier structure (canonical)	1088
Live Capstone A checklist (copy into repo)	1089
Lab workspace convention	1089
Acceptance bar for Capstone A	1089
Elitebook / KVM lab notes	1090
Policy reminders	1090
Next	1090
 Capstone: scope and plan	 1091

Capstone: scope and plan	1092
Why this chapter exists	1092
Theory — Working agreements for Capstone	1092
Suggested repo shape (adapt, don't dogma)	1092
Capstone A checklist (live)	1093
Target scope for this chapter	1093
Explicit non-goals (this chapter)	1094
Worked example — Minimal host flake skeleton	1094
Theory — Capstone A success definition (this chapter)	1095
Theory — Branch discipline	1095
Worked example — smoke script	1095
Rebuild proof steps (skeleton → switch)	1096
Lab 0 — Branch & checklist	1096
Lab 1 — Build green host	1096
Lab 2 — Users story	1097
Track HM (workstation-ish)	1097
Track service users (server-ish)	1097
Lab 3 — Module hygiene	1097
Lab 4 — Update architecture & gaps	1098
Lab 5 — Smoke	1098
Lab 6 — Gap register freeze for next chapter	1098
Lab 7 — Teach-back outline (5 bullets)	1098
Failure modes	1099
Common gotchas	1099
Checkpoint	1099
Commit	1100
Next	1100
Capstone: implement	1101
Capstone: implement	1102
Why this chapter exists	1102
Theory 1 — Secrets E2E definition of done	1102
Threat model (short)	1102
Theory 2 — sops-nix sketch (preferred)	1103
.sops.yaml creation rules (idea)	1103
Theory 3 — agenix sketch (alternative)	1104
Theory 4 — Proxy E2E definition of done	1104
nginx sketch	1104
TLS modes (pick and document in docs/TLS.md)	1105
Theory 5 — Ordering and dependencies	1105
Store discipline reminder	1105
Theory 6 — Module shape for Capstone	1105
Concrete checklists	1106
Secrets checklist	1106
Proxy checklist	1106
Worked example — Proof commands	1107
Lab 0 — Branch	1107

Lab 1 — Secrets path E2E	1107
Lab 2 — Proxy path E2E	1108
Lab 3 — Failure injection (mini)	1108
Lab 4 — Tests stub toward prove-rebuild chapter	1108
Lab 5 — Docs & checklist	1108
Lab 6 — Offline key custody proof	1109
Common gotchas	1109
Checkpoint	1109
Commit	1110
Next	1110
Capstone: harden and document	1111
Capstone: harden and document	1112
Why this chapter exists	1112
Theory 1 — Deploy definition of done	1112
deploy-rs sketch	1112
Colmena sketch	1113
Theory 2 — CI definition of done	1113
Minimal workflow sketch	1114
Theory 3 — Cache push value	1114
Theory 4 — SSH deploy auth checklist	1114
Theory 5 — RUNBOOK / DEPLOY docs minimum	1115
Concrete checklists	1115
Deploy checklist	1115
CI checklist	1116
Lab 0 — Workspace	1116
Lab 1 — Wire deploy tool	1116
Lab 2 — CI workflow	1117
Lab 3 — Hardening pass (light)	1117
Lab 4 — Docs	1117
Lab 5 — Failure tabletop	1118
Lab 6 — Elitebook admin path	1118
Common gotchas	1118
Checkpoint	1119
Commit	1119
Next	1119
Capstone: prove rebuildability	1120
Capstone: prove rebuildability	1121
Why this chapter exists	1121
Theory 1 — “Sufficient” tests for Capstone A	1121
Flake outputs shape	1122
Theory 2 — RUNBOOK audience	1122
Theory 3 — Diagrams that match reality	1122
Theory 4 — Rebuild proof steps (pre-wipe)	1123
Theory 5 — Assertion fixtures	1123

Concrete checklists	1123
Tests checklist	1123
Docs checklist	1124
Lab 0 — Workspace	1124
Lab 1 — Harden tests	1124
Lab 2 — RUNBOOK	1124
Lab 3 — Diagrams	1125
Lab 4 — Paper DR draft	1125
Lab 5 — Freeze readiness review	1125
Lab 6 — Negative eval fixture	1125
Common gotchas	1126
Checkpoint	1126
Commit	1126
Next	1126
Capstone: optional multi-node bar	1127
Capstone: optional multi-node bar	1128
Why this chapter exists	1128
Theory 1 — What “freeze” means	1128
Freeze is a social contract with yourself	1128
Tag shape	1129
Theory 2 — Waiver register	1129
Theory 3 — Optional multi-node bar	1129
Elitebook 16 GB guidance	1130
Shared modules pattern	1130
Theory 4 — Freeze gate commands	1130
Concrete freeze checklist	1131
Lab 0 — Workspace	1131
Lab 1 — Honest pass on CAPSTONE-A	1131
Lab 2 — Green suite at freeze	1131
Lab 3 — Tag	1132
Lab 4 — Optional second node (only if A green)	1132
Lab 5 — Snapshot for DR	1132
Lab 6 — Freeze announcement (to yourself)	1132
Common gotchas	1133
Checkpoint	1133
Commit	1133
Next	1133
Disaster recovery notes	1134
Disaster recovery notes	1135
Why this chapter exists	1135
Theory 1 — What “from flake + keys alone” means	1135
Theory 2 — Recovery paths (pick what you documented)	1136
Path A — Interactive installer + flake	1136
Path B — nixos-anywhere / remote install	1136

Path C — Image rehydrate	1136
Theory 3 — Timing and RTO	1136
Theory 4 — What wipe proves / does not prove	1137
Theory 5 — Data vs system	1137
Concrete DR checklist	1137
Pre-wipe (mandatory)	1137
Post-rebuild health	1138
Lab 0 — Confirm freeze and keys	1138
Lab 1 — Execute wipe per DR-PLAN	1138
Lab 2 — Time the recovery	1138
Lab 3 — Secrets rehydrate	1139
Lab 4 — Health + deploy proof	1139
Lab 5 — Fix docs (the real deliverable)	1139
Lab 6 — Optional: refuse snapshot	1139
Common gotchas	1140
Checkpoint	1140
Commit	1140
Next	1140
Rollback and tests	1141
Rollback and tests	1142
Why this chapter exists	1142
Theory 1 — Rollback layers	1142
Commands (lab)	1143
Theory 2 — Bad change protocol	1143
Theory 3 — Full suite definition	1144
Theory 4 — Troubleshooting checklist (finalize)	1144
Theory 5 — State vs generation (again)	1144
Concrete checklists	1145
Rollback drill	1145
Suite	1145
Lab 0 — Workspace	1145
Lab 1 — Record baseline generation	1145
Lab 2 — Intentional break	1146
Lab 3 — Rollback	1146
Lab 4 — Git hygiene	1146
Lab 5 — Full suite green	1146
Lab 6 — Finalize troubleshooting checklist	1147
Lab 7 — Bootloader path (optional)	1147
Common gotchas	1147
Checkpoint	1147
Commit	1148
Next	1148
Retrospective	1149

Retrospective	1150
Why this chapter exists	1150
Theory 1 — What you should now be able to do	1150
Theory 2 — Retrospective structure (blameless)	1151
Theory 3 — Teach plan (1 session)	1151
Theory 4 — Roadmap options (pick few)	1152
Theory 5 — What not to do today	1152
Lab 0 — Workspace	1152
Lab 1 — Skills matrix	1152
Lab 2 — Full retrospective writeup	1153
Lab 3 — Teach plan	1153
Lab 4 — Roadmap	1153
Lab 5 — Capstone A final sign-off	1153
Lab 6 — Celebrate without thrash	1154
Common gotchas	1154
Checkpoint	1154
Commit	1154
Next	1155
 Projects	 1156
Projects index	1157
Projects: packaging and your own distro	1158
How to use this part	1158
Learning loop	1158
Lab environment checklist	1159
How chapters are structured	1159
Project path	1159
Track A — Derivations & packaging	1159
Track B — Custom NixOS-based distro	1160
Suggested order	1160
Time and difficulty (rough)	1161
Repository hygiene (all projects)	1161
Cross-links into the main book	1162
Done means (global bar)	1162
Trademark and ethics (read once)	1162
Quick start: create the lab root	1163
 Project 1: Hello CLI derivation	 1164
Project 1 — Package a tiny CLI with stdenv.mkDerivation	1165
Why this project exists	1165
Prerequisites	1165
Theory	1166
Derivation vs package vs app	1166
Phases you will skip on purpose	1166
\$out is the contract	1166

<code>callPackage</code> dependency injection	1166
Purity / sandbox	1166
Lab tree	1167
Step 1 — Source: shell tool	1167
Step 2 — Derivation	1167
Optional: clean the source filter	1168
Step 3 — Flake	1168
Step 4 — Build, run, inspect	1170
Step 5 — C variant (stretch, same flake)	1170
Theory checkpoints (self-test)	1170
Pitfalls	1171
Acceptance criteria	1171
Stretch goals	1172
Checkpoint commit message (example)	1172
Project 2: Script tool derivation	1173
Project 2 — Real script tool with deps and wrap	1174
Why this project exists	1174
Prerequisites	1174
Theory	1175
Three ways to ship shell tools	1175
What the wrapper actually does	1175
Closure vs references	1175
Build-time vs run-time (again)	1175
Lab tree	1176
Step 1 — Preferred path: <code>writeShellApplication</code>	1176
Flake	1177
Build and inspect	1177
Step 2 — Fail closed (required demo)	1178
Step 3 — Alternative: <code>stdenv</code> + <code>wrapProgram</code>	1178
Step 4 — Optional live mode (network at runtime)	1179
Lab tasks	1180
Pitfalls	1180
Acceptance criteria	1180
Stretch goals	1181
Theory recap	1181
Project 3: Language package derivation	1182
Project 3 — Package a Go or Python app with language helpers	1183
Why this project exists	1183
Prerequisites	1184
Theory map	1184
Why helpers beat raw <code>stdenv</code> here	1184
Hash discipline	1184
Lab tree (shared idea)	1185

Option A — Go (<code>buildGoModule</code>)	1185
A1. Scaffold module	1185
A2. Package expression	1186
A3. Flake	1186
A4. Hash fix loop	1187
A5. Vendor / tests (optional)	1187
Option B — Python (<code>buildPythonApplication</code>)	1187
Shared finish	1189
REPORT.md (required)	1189
Pitfalls	1189
Acceptance criteria	1190
Stretch goals	1190
Self-check	1190
Project 4: Overlay pin and patch	1192
Project 4 — Overlay: pin or patch a package	1193
Why this project exists	1193
Prerequisites	1194
Theory	1194
Overlay shape	1194
Three layers of change (recap)	1194
Overlay vs separate flake input	1195
Infinite recursion classic	1195
Lab tree	1195
Step 1 — Flake wiring (overlay applied once)	1196
Step 2 — Lab A: message patch on <code>hello</code>	1197
A1. Discover the string to patch	1197
A2. Lab-fast path: <code>postPatch</code> (allowed for green)	1197
A3. Proper patch path (preferred write-up)	1197
A4. Prove it	1198
Step 3 — Lab B: pin via extra input (document, implement lightly)	1198
Step 4 — Optional NixOS consumer + upgrade drill	1199
Gotchas	1199
Acceptance criteria	1200
Stretch goals	1200
Self-check	1200
Project 5: Distro identity and branding	1201
Project 5 — Name your distro: identity, versioning, branding assets	1202
Why this project exists	1202
Prerequisites	1203
Theory	1203
Identity surfaces	1203
Version scheme for derivatives	1203
Modules as product API	1204
os-release merge reality	1204

Product sheet (PRODUCT.md)	1204
Repo layout	1205
Copy sample branding from this book	1205
Step 1 — Identity module	1205
Step 2 — Lab host + flake	1207
Step 3 — Verify identity	1208
Step 4 — Artwork + consistency	1208
Pitfalls	1209
Acceptance criteria	1209
Stretch goals	1210
Self-check	1210
Project 6: Distro ISO and themes	1211
Project 6 — Installer ISO with themes (Plymouth / greeter / console)	1212
Why this project exists	1212
Prerequisites	1213
Theory / architecture	1213
Theme layers (cheap → expensive)	1213
Lab tree (extends Project 5)	1214
Step 1 — ISO host configuration	1214
Step 2 — Themes module	1215
Step 3 — Flake output	1216
Build	1216
Alternative: nixos-generators	1217
Step 4 — Boot test (QEMU)	1217
VirtualBox / UTM	1217
Verification checklist (live session)	1217
Step 5 — Branding matrix	1218
Step 6 — Iterate without tears	1218
Plymouth (optional after minimal ISO is green)	1219
Pitfalls	1219
Acceptance criteria	1219
Stretch goals	1220
Self-check	1220
Project 7: Complete custom distro product	1221
Project 7 — Ship a complete NixOS-based distro flake	1222
Why this project exists	1222
Product definition of done	1223
Target tree	1223
Theory: editions vs hosts	1224
Step 1 — Defaults module (flakes-first)	1224
Step 2 — Edition modules	1225
Minimal	1225
Desktop (or server)	1225

Step 3 — Hosts	1226
Demo VM / installed profile	1226
ISO host	1227
Step 4 — Flake product surface	1227
Step 5 — Optional metapackage (toolkit)	1228
Step 6 — Install story (document, then test once)	1228
Step 7 — README + DEMO.md (required)	1229
Step 8 — Channels of truth	1229
Pitfalls	1230
Acceptance criteria	1230
Stretch goals	1230
Capstone reflection (½ page)	1231
Self-check	1231
Done with the projects track	1231
 Branding	 1232
 Sample distro branding assets (lab only)	 1233
License	1233
Trademark	1233
Wire into a flake	1233
 Plymouth / greeter theme notes (lab)	 1234
Goal	1234
Practical path on 26.05	1234
Custom Plymouth (stretch)	1234
Assets in this folder	1234

Notes on Nix & NixOS

Personal **journey notes** for learning **Nix** and **NixOS** the way practitioners who already live in this ecosystem would sequence the work: early feedback loops, modern defaults, depth when you have something real to debug.

Baseline: Nix **2.34.x** · NixOS / nixpkgs **26.05 “Yarara”** (as of mid-2026).

This is a **normal topic-oriented book** (parts and chapters), not a day-calendar or certification track. Chapter contract: [Syllabus](#).

What “done” looks like

- Explain store, derivations, evaluation vs realization, generations, and GC roots in plain language
- Run **one** flake-defined NixOS 26.05 host; break it, roll back, recover
- Prefer **modern CLI + flakes** as the default; recognize classic commands when you meet them
- Separate **system** (NixOS) and **user** (Home Manager) concerns cleanly
- Package something you actually use; write modules with real options
- Keep secrets out of the store; layout disks with Disko when you reinstall
- Deploy beyond one box, use a binary cache and CI, harden without cargo-cult

How the book is organized

Part	Path	Focus
Front matter	00-front-matter/	Syllabus + Lab 0 (single-server / dev / Elitebook)
Concepts	01-concepts/	Why Nix, store, language, CLI, flakes
Nix on Linux	02-nix-on-linux/	Fedora 44+ official Nix; dual-stack bridge to NixOS
NixOS host	03-nixos-host/	Install, rebuild, users, net, modules, boot

Part	Path	Focus
Home & layout	04-home-and-flake-layout/	Flake layout, locks, Home Manager, direnv
Services & security	05-services-and-security/	Secrets, hardening, TLS front door, containers
Packaging	06-packaging/	stdenv, overlays, caches, language ecosystems, ROS 2 / nix-ros-overlay
Ops & fleet	07-ops-and-fleet/	Disko, deploy, tests, CI, images, backups
Internals	08-internals/	Evaluator, module system, upgrades, troubleshooting
Capstone	09-capstone/	Rebuildability proof and recovery notes
Projects	99-projects/	Derivations + branded distro (ISO, themes, editions)

Chapters are **topics**, not “Day 1...Day 90”. Read in order when learning; use the sidebar as a reference index when you already know where to jump.

Recommended path if your daily driver is Fedora

1. **Concepts** — store, language, flakes
2. **Nix on Linux** — `dnf install nix` on **Fedora 44+**, profiles, devShells, standalone Home Manager, `dnf` coexistence
3. **NixOS host** — when ready, same skills apply to a full OS flake

See [Nix on Linux — Fedora bridge path](#).

Lab 0 (recommended before first host work)

If you have a spare machine (example profile: **HP Elitebook 2570p · 16 GB**), set up the lab first:

1. Roles & topology — [01a-lab-overview-and-roles.qmd](#)
2. Journey single-server (KVM, VMs, containers) — [01b-lab-journey-single-server.qmd](#)
3. NixOS **dev** machine — [01c-lab-nixos-dev-machine.qmd](#)
4. Elitebook hardware profile — [01d-lab-elitebook-2570p.qmd](#)
5. Ops cheatsheet — [01e-lab-ops-cheatsheet.qmd](#)

How I intend to study

Slice	Share	Purpose
Concept	~25%	Manuals, mental models, <i>why</i>
Hands-on	~55%	Configs and builds that can fail
Review	~20%	Refactor modules, commit, write gotchas

Rules that match expert practice:

- **Disposable VM** (or spare disk) for breakage; keep a daily driver safe
- **Flakes + modern CLI by default** from early chapters
- **One change → rebuild → observe** — avoid huge untested diffs
- Budget disk for `/nix/store`
- Pin **26.05** deliberately; upgrade only with a written plan

Prerequisites

- Comfortable Unix shell
- Processes, files, SSH, basic networking
- Willingness to reinstall a VM

Related books

Book	Overlap
Linux	Classic ops literacy before (and beside) NixOS
VCS	Flake repos, CI, and signed history
Networking	Lab networks you may declare on NixOS hosts

Formats

HTML (primary), PDF, and EPUB via the monorepo library portal when built.

Acknowledgments

Path designed as an expert learning sequence for Nix/NixOS. Topic inventory drawn from personal study notes and production practice.

Official docs (nixos.org, nix.dev, release notes) win when notes and reality disagree. No exam-vendor affiliation.

Front Matter

Syllabus

Syllabus — expert path for Nix & NixOS

Status: Canonical outline for this **standalone NixOS book**.

Baseline: Nix **2.34.x** · NixOS / nixpkgs **26.05** “Yarara”.

Voice: Topic-oriented journey notebook. **Ordering:** expert pedagogy by **theme**—not a day calendar and not a 90-day program.

! Important

Chapters follow **this** topic path. Source documents supply topics and exercises; they do not override the sequence below. There is **no** required joint schedule with any other book or series.

Why this order (expert rationale)

Nix rewards **tight feedback loops**. Long theory blocks before a real rebuild create fragile knowledge. Production NixOS work in 2026 is overwhelmingly **flake-pinned, module-composed, and generation-rollback based**. The path below matches that world.

Principle	What we do	What we avoid
Feedback early	Install Nix early; first NixOS host soon after	Weeks of pure language before any OS
Modern defaults	Flakes + <code>nix</code> CLI as default; classic CLI as literacy	Building muscle memory on <code>nix-env</code> / channels as primary
Host before packaging craft	Configure and consume packages on a real system first	Deep <code>stdenv</code> before you have felt closure size and rebuilds
System then user	NixOS host stable → Home Manager for the user layer	Dotfiles-only Nix that never owns the machine
Just-in-time depth	Module <i>types</i> catalog after you have written real modules	A full type catalog before <code>services.openssh.enable</code>
Patterns over product zoo	Proxy, secrets, one DB, CI, observability as <i>patterns</i>	Required chapters for every self-hosted app
Internals last	Evaluator, daemon, CA derivations after you can operate	Sandbox namespaces before you can roll back a bad config
Prove rebuildability	Capstone = wipe + rebuild from flake	“It worked once on my laptop”

How source notes were used

Source	Role
Personal lab notes	Mini-projects, 26.05 notes, deploy/secrets tool names
Depth checklists	Language, store, module types, HM, troubleshooting
Production syllabus notes	Stack defaults, hardening, multi-node capstone bar

Rejected as primary structure: day calendars; flakes-before-any-OS schedules; long language-only front loads; “install every service” tables; cert/exam framing; cross-links to series volumes.

Other languages (including Go)

This book does not teach Go or any application language. At packaging stages, any real binary you care about is a valid target. Do not stall NixOS waiting for another curriculum.

Modern stack (defaults for this book)

Layer	Default choice
Pinning	Flakes + <code>flake.lock</code>
CLI	Modern nix (<code>build</code> , <code>run</code> , <code>develop</code> , <code>flake</code> , <code>shell</code> , <code>profile</code>)
System	NixOS modules + <code>nixos-rebuild</code> / <code>nixos-rebuild --flake</code>
User	Home Manager as NixOS module (standalone when on non-NixOS)
Disks	Disko when reinstalling or provisioning
Secrets	sops-nix (primary); agenix compared once
Deploy	deploy-rs (single/simple) then Colmena (fleet)
Images	nixos-generators
Cache	cache.nixos.org + Cachix or self-hosted Attic / Harmonia
Boot security	Lanzaboote / Secure Boot when hardware allows
Tests	nixosTest + <code>flake checks</code>

Classic **nix-*** commands appear for reading older docs and mental models—not as the preferred daily driver.

Part map (topic book)

This **standalone NixOS book** is a **normal topic-oriented** manuscript. Parts and chapters are **not** a day calendar. Study pacing is yours.

Part	Directory	Theme	You can leave when...
Front matter	00-front-matter/	Syllabus + Lab 0	Lab topology is clear; Elitebook/KVM ready
Concepts	01-concepts/	Why Nix, store, language, CLI, flakes	You can explain a store path and use a flake devShell
Nix on Linux	02-nix-on-linux/	Fedora 44+ packaged Nix; dual-stack; HM standalone; bridge	You run flakes/HM on an existing distro and know what NixOS still adds
NixOS host	03-nixos-host/	Install, rebuild, users, net, modules	You rebuild and roll back a flake host
Home & layout	04-home-and-flake-layout/	Layout, locks, Home Manager	System + user config live cleanly in one flake
Services & security	05-services-and-security/	Services, hardening, proxy, containers	A small server slice is declarative and secret-safe
Packaging	06-packaging/	stdenv, overlays, caches, ROS 2 / nix-ros-overlay	You ship a package you use (robotics env optional)
Ops & fleet	07-ops-and-fleet/	Disko, deploy, tests, CI	You can deploy and test beyond one laptop
Internals	08-internals/	Evaluator, modules, upgrades	You can debug eval/store issues and plan upgrades
Capstone	09-capstone/	Rebuildability	Wipe/rebuild proof (+ optional multi-node bar)
Projects	99-projects/	Derivations + distro branding	Hands-on artifacts exist

Suggested mix when learning: ~25% concept / ~55% hands-on / ~20% review. Skip freely

when using the book as a reference.

Lab 0 (front matter) — before Concepts

Chapter	Purpose
Lab overview & roles	Host vs journey guest vs dev role
Journey single-server	KVM/libvirt (default) and optional Incus
NixOS dev machine	Editor, flakes, SSH, caches, drift discipline
Elitebook 2570p profile	Concrete 16 GB RAM/CPU/disk/BIOS plan
Ops cheatsheet	Daily commands

Exit criteria for Lab 0: spare machine boots; virt enabled; at least one NixOS VM; wipe boundaries known; git holds host/guest intent.

Concepts — Foundations

Goal: Correct mental model + modern toolchain before owning a machine.

Focus	Expert emphasis	Hands-on
Why Nix exists	Dependency hell, snowflake servers, honesty about limits; language vs package manager vs NixOS	Install Nix (multi-user); enable flakes + new CLI; first <code>nix repl</code>
Store & closures	Content addressing, store paths, GC roots at a glance	Inspect a package closure; find what keeps it alive
Language: values & attrsets	Types, <code>let</code> , nested sets, <code>inherit</code> — only what you need to read nixpkgs	Model a small config as an attrset
Language: functions	<code>{ a, b ? d }:</code> , <code>@</code> , ellipsis, composition	Write helpers used later in a flake
Laziness, imports, <code>lib</code>	Imports are eager, values lazy; <code>lib</code> as standard library	Multi-file expression; one <code>lib</code> pipeline
Derivations as idea	Evaluation vs realization; <code>.drv</code> vs outputs; sandbox <i>why</i>	<code>nix build</code> something tiny; read <code>nix log</code>
Modern CLI fluency	<code>build</code> / <code>run</code> / <code>develop</code> / <code>shell</code> / <code>flake show</code>	Prefer modern CLI for all tasks
First flake (project)	<code>inputs</code> / <code>outputs</code> / <code>lock</code> ; devShell before OS	Flake with <code>devShells.default</code> for a real project
Classic CLI literacy	Map old → new (<code>nix-build</code> , <code>nix-shell</code> , channels as history)	Same task once classic, once modern
Gate	Foundations review	Pin <code>nixpkgs</code> ; recreate shell on a clean machine/VM user

Depth to weave in (do not front-load a long language-only block): conditionals/operators, `with` pitfalls, string interpolation, type checks, fixed-point *awareness*, FOD concept, sandbox/substitute vocabulary.

Exit criteria: Explain store path + lock file; `nix develop` works offline after first populate; no dependency on channels for your project.

Nix on Linux — Fedora bridge (recommended before host cutover)

Goal: Master the **package manager on a normal Linux daily driver**—especially **Fedora 44+**, which ships Nix officially (`dnf install nix`, flakes OOTB, `/nix` + SELinux story)—so NixOS later is “machine modules + boot generations,” not a total reboot of your mental model.

Focus	Expert emphasis	Hands-on
Why Nix-on-Linux first Fedora 44 install	Split OS owner (dnf) vs Nix owner (store/flakes/HM) Multi-user daemon; single-user/rootless/Atomic notes	Write a dual-stack charter <code>dnf install nix;</code> <code>systemctl enable</code> <code>nix-daemon; nix run</code> <code>nixpkgs#hello</code>
Store, profiles, GC	Same roots model as NixOS minus bootloader	Thin profile; <code>nix store gc</code> ; inspect roots
CLI + devShells + direnv	Project flakes as SoT	Real repo with <code>flake.lock</code> + use <code>flake</code>
Home Manager standalone	User generations without NixOS	<code>home-manager switch --flake</code>
dnf coexistence	PATH policy; no dual services	Label tools dnf vs Nix
SELinux / Atomic / ops	Stay enforcing; bind-mount <code>/nix</code> when needed	Day-2 daemon + disk checklist
Bridge to NixOS	Port HM; add <code>nixosConfigurations</code>	VM plan + monorepo sketch
Gate	Fedora path solid	Ready for 03-nixos-host without relearning flakes

Exit criteria: Flakes + HM work on Fedora after reboot; you can explain what NixOS still adds (modules, system generations, boot).

Skip only if you already live on NixOS or will only use disposable NixOS VMs and never a foreign distro—still skim the bridge chapter for dual-machine layouts.

NixOS host

Goal: Generations, rollback, and “the module system as a user” before authoring a framework.



Tip

Expert default: Install and manage the first host **with flakes from the first NixOS install**. Do not invest weeks in a non-flake `configuration.nix` only to rewrite it.

Focus	Expert emphasis	Hands-on
Install	VM/spare disk; UEFI; Disko intro or manual once; generate hardware config	Bootable NixOS 26.05 lab
Flake host skeleton	<code>nixosConfigurations</code> ; <code>nixos-rebuild switch --flake .#host</code>	First flake-defined rebuild
Rebuild modes & generations	<code>switch</code> / <code>test</code> / <code>boot</code> / <code>build</code> ; boot menu rollback	Deliberate break → roll back
Users & access	Declarative users; SSH keys only for remote	Non-root admin pattern
Networking & firewall	<code>networkd</code> vs NM; open only what you need	Static or known DHCP + SSH
Packages & programs	<code>environment.systemPackages</code> vs <code>programs.*</code> modules	Install daily tools declaratively
systemd via Nix	One custom service + timer	Module snippet that generates units
Modules as consumer	Read <code>options</code> + source for 2–3 nixpkgs modules; <code>mkIf</code> / <code>mkDefault</code>	Change behavior via options, not forks
First custom module	<code>options</code> + <code>config</code> , imports, one assertion	Extract something into a module
Boot & hardware	<code>systemd-boot</code> vs GRUB; kernel packages; firmware	Document your boot path
Filesystems intro	Mounts; state vs store; impermanence <i>idea</i> only	Label what must persist
Gate	Minimal production-shaped host	Users, SSH, firewall, one service, one custom module — all in flake

Defer later: full Disko redesign, multi-host, secrets machinery, deep type catalogs.

Exit criteria: From cold boot you can roll back; flake is the only source of truth for the lab host.

Home Manager and flake layout

Goal: Structure that scales past one file; user environment without polluting system closure carelessly.

Focus	Expert emphasis	Hands-on
Flake layout	Split <code>hosts/</code> / <code>modules/</code> / <code>homes/</code> ; avoid 2k-line <code>flake.nix</code>	Refactor host work into layout
Lock discipline	Selective <code>flake update</code> ; pin strategy; registries lightly	Update one input only; explain lock diff
Home Manager on NixOS	HM as NixOS module first (one activation story)	User programs: shell, git, ssh
HM files & XDG	<code>home.file</code> , <code>xdg.configFile</code> ; theming only if you care	Migrate real dotfiles
HM services	User systemd timers/services	One user service you actually want
Dev shells + direnv	Project shells <i>outside</i> systemPackages	use flake on a repo
flake-parts (or discipline)	Reduce boilerplate or hand-roll cleanly — pick one style	Consistent multi-output flake
Checks	<code>nix flake check</code> ; format/lint as checks	Fail CI-shaped check locally
Overlays (light)	Pin or patch one package for the host	Overlay consumed by host + shell
Gate	Workstation/server skeleton	Host + HM + devShell + checks in one flake

HM depth (across HM chapters): programs modules, file permissions/templating, XDG, multi-machine user splits as *pattern*.

Exit criteria: New machine of same role = clone flake + hardware config + rebuild.

Services and security

Goal: Operate services like production without leaking secrets into the store.

Focus	Expert emphasis	Hands-on
Secret problem	Why strings in Nix are dangerous; store world-readable truth	Find a “secret” anti-pattern and remove it
sops-nix	Encrypted-in-repo, decrypt at activation	One secret end-to-end
agenix comparison	Age keys; trade-offs; pick a house default	Same secret second way once
Hardening baseline	SSH, sudo, unused services off, kernel defaults carefully	Apply a small hardening module
Firewall discipline	Per-service ports; deny by default	Audit open ports
TLS front door	ACME + Caddy or Nginx — one reverse-proxy pattern	HTTPS to a static or app backend
Data service pattern	PostgreSQL or other one DB — modules, state dirs, backups <i>thought</i>	Declarative DB + app user
nixos-containers / OCI	When containers help on NixOS; not “Docker first”	One declarative container or nspawn
Podman/Docker modules	Rootless notes; prefer OCI module over snowflake	Compare to NixOS containers
Observability pattern	Prometheus + Grafana or minimal logs first	Metrics for one service
Optional: k3s lite	Only if you need k8s; single-node	Skip cleanly if not
Gate	Lab slice	Proxy + secret + one app/data path + firewall story

Not required: Matrix, Nextcloud, Jitsi, full “homelab app of the day” lists. Revisit after Capstone if you want.

Exit criteria: Repo has no live secrets in plaintext; slice rebuilds on a fresh VM from flake + keys.

Packaging craft

Goal: Produce software, not only consume it—after you know how packages feel on a host.

Focus	Expert emphasis	Hands-on
stdenv phases	unpack→install→fixup; hooks; build vs native inputs	Package a small C/shell tool properly
Fetchers & FODs	Hash mismatch workflow; <code>fetchFromGitHub</code>	Package from a tag
Language ecosystem A	Your primary language helper (<code>buildGoModule</code> , etc.)	Package a tool you use
Language ecosystem B	Second ecosystem for pattern transfer	Smaller package
Patching & overrides	<code>overrideAttrs</code> , patches, overlays composition	Fix or pin something broken
Multiple outputs	Closure size discipline	Split out/dev/doc
Debug builds	<code>nix log</code> , <code>--keep-failed</code> , <code>breakpointHook</code>	Break and recover
nixpkgs layout	<code>by-name</code> , conventions, meta	PR-shaped package (submit optional)
Binary caches	Push/pull; trust/signatures awareness	Private or Cachix push
Remote builders	Offload when useful	Second VM as builder
ROS 2 / nix-ros-overlay (elective)	Overlay pin, <code>rosPackages.*</code> , underlay <code>buildEnv</code> , caches	Flake <code>devShell</code> + talker/listener; <code>colcon workspace</code>
Reproducibility honesty	What is and is not bit-for-bit	Clean rebuild comparison
Gate	Package + cache	Flake output used by your host or CI

Exit criteria: Someone else (or future you) builds your package from lock alone.

Ops and fleet

Goal: Provision and deploy like infra, not like a pet SSH session.

Focus	Expert emphasis	Hands-on
Disko for real	Declarative disks on reinstall/cloud	Install/reinstall with Disko
Impermanence (optional commit)	Persist deliberately	Prove reboot drops unpersisted state or document why you skip
deploy-rs	Simple push deploy	Deploy lab to remote VM
Colmena	Tags, multi-host, parallel	Two hosts (one may be local)
Shared modules	Roles vs hosts; no copy-paste drift	roles/web.nix pattern
Images & generators	Cloud image / custom ISO	Boot generated image
Terraform boundary	TF provisions, Nix configures	Sketch only is enough
nixosTest	VM tests for <i>your</i> modules	Test proxy or hardened SSH module
More tests	Expand suite; flake checks in CI shape	Several meaningful tests
CI pipeline	GHA/GitLab + Nix installer; cache write	PR build of flake
GC & store hygiene	Roots, retention, optimise	Policy + measured store size
Secure Boot / TPM path	Lanzaboote / LUKS unlock — hardware permitting	Document supported path or lab limits
Backup story	Borg/Restic/ZFS — pick one; declare it	Restore drill of <i>state</i>
Gate	Remote lab	Flake deploy + CI + cache + test green

Exit criteria: New host of an existing role is boring.

Internals and currency

Goal: Debug and upgrade without superstition.

Focus	Expert emphasis	Hands-on
Evaluator	Thunks, infinite recursion patterns	Trace a tricky eval
Module system deep	Full type catalog ; mkMerge / priorities	Redesign one module with proper types
Store & daemon	Substituters, signatures, sandbox namespaces	Explain roots + one sandbox log
Performance	max-jobs / cores / eval cost	Speed a full rebuild with measurement
CA derivations	Experimental awareness	Optional experiment or status memo
nixosTest mastery	Flaky tests, networking in tests	Harden test suite
Ecosystem map	devenv, Snowfall, nix-darwin — try one , don't collect all	Written comparison to your layout
Comparative landscape	Guix / image-based distros / Ansible — trade-offs	Honest scorecard + essay
26.05 release notes	Primary source, not blogs	Annotate impact on <i>your</i> flake
Upgrade planning	26.05 → next release dry-run	Written upgrade runbook
Troubleshooting drill	Boot, build, service, network matrix	Fix injected failures
Architecture review	Map prior parts onto design	Gap list before Capstone

Exit criteria: You can diagnose eval/store issues and produce IMPACT + upgrade artifacts.

Capstone and recovery

Capstone A — required

Build (or finish) a **flake-defined lab** that includes:

1. At least one NixOS host role you understand end-to-end
2. Home Manager for your user (if workstation) **or** minimal service user story (if server-only)
3. Secrets via sops-nix **or** agenix (house default)
4. One reverse-proxy or equivalent edge pattern
5. Deploy path (deploy-rs and/or Colmena)
6. At least one `nixosTest` or flake check that would catch a real regression
7. CI that builds and preferably pushes a cache
8. **Disaster recovery:** wipe lab disk/VM; rebuild from flake + keys alone
9. Rollback drill + full test suite + troubleshooting checklist
10. Retrospective: teach plan + roadmap (next release, nixpkgs PR, optional Capstone B)

Capstone chapter theme	Focus
Scope and plan	Close structural P0s; host skeleton
Implement	Secrets + proxy E2E
Harden and document	Deploy + CI + cache
Prove rebuildability	Tests + runbook + diagrams
Optional multi-node / freeze	Feature freeze tag; optional second node
Disaster recovery	Timed wipe + rebuild
Rollback and tests	Generation rollback + suite green
Retrospective	Teach plan + roadmap; no rewrite thrash

Capstone B — stretch

Only after A:

- 3 nodes, Colmena + Disko
 - LUKS (+ TPM if available)
 - Secure Boot / Lanzaboote if feasible
 - Fleet observability
 - CI → cache → deploy automation
-

Projects (optional depth track)

Track	Projects
Packaging	Hello CLI → script wrap → language package → overlay
Distro product	Identity → ISO/themes → complete multi-edition flake

Projects sit after packaging theory (or alongside late packaging). They force **demoable artifacts**, not only reading.

Review habit

Every substantial study block:

1. Refactor modules (DRY, naming, options)
 2. Re-read one official manual page you used
 3. Write 5 “gotchas” in your own words
 4. Delete or quarantine experiments that taught something but should not ship
-

Completeness audit (sources → parts)

Theme	Part(s)
Philosophy, store, GC, sandbox	Concepts, Internals
Language depth	Concepts (JIT), Internals types revisit
Modern + classic CLI	Concepts
Flakes, locks, checks	Concepts, Home & layout
First host, rebuild, boot, net, users	NixOS host
Module system practice → deep types	Host, Internals
Home Manager depth	Home & layout
Dev shells, direnv	Concepts, Home & layout
Secrets, hardening, proxy, DB pattern	Services & security
Containers / optional k3s	Services & security
Packaging, overlays, caches	Packaging
Disko, deploy, images, CI, tests	Ops & fleet
Secure Boot / TPM / Lanzaboote	Ops, Capstone stretch
Internals, upgrades, ecosystem	Internals
Capstone rebuild proof	Capstone
App product zoo	Out (optional post-book)
Cert tracks	Out
Day calendars / series cross-links	Out

Explicitly out of scope

- Go (or other language) full curriculum — separate books
 - Certification branding or exam prep
 - Exhaustive self-hosted app catalog as required chapters
 - Replacing nixos.org / nix.dev
 - Required pacing as “Day N”
-

Resources (primary, not blogs-of-2022)

- *Nix Pills* — store/derivation intuition (Concepts)
 - *nix.dev* — maintained tutorials (Concepts–layout)
 - Official NixOS & Nix manuals + **26.05 release notes**
 - *NixOS & Flakes Book* (community) — companion for flake layouts
 - Upstream READMEs: sops-nix, agenix, Disko, deploy-rs, Colmena, Lanzaboote
-

Writing rule for future chapters

1. Follow **parts in pedagogical order** when learning linearly.
 2. Each chapter: mental model → minimal working flake/config → break it → gotchas.
 3. Prefer the **default stack** table unless hardware forces a fork (document the fork).
 4. After content moves: `bash scripts/update-index.sh` only — never hand-edit `_quarto.yml`.
 5. Syllabus changes first if scope changes.
 6. **Topic book language only**—no day-calendar framing.
-

Lab hardware default (this book)

Item	Default
Primary lab chassis	HP Elitebook 2570p · 16 GB (or similar spare laptop)
Hypervisor	libvirt/KVM default; Incus optional
Journey guest	nixlab 4–6 GB
Deploy target	deploy-a ~2 GB optional
Host OS	NixOS 26.05 preferred

See Lab 0 chapters for concrete commands, failure modes, and acceptance criteria.

Lab 0 — Overview: roles before the Why Nix exists chapter

Lab 0 — Overview: roles before the Why Nix exists chapter

Before the Why Nix exists chapter. Do **not** skip this if you will follow the full journey on real hardware. The curriculum assumes you can **break and rebuild** machines without fear. That only works if lab roles are clear.

Why a lab host exists

NixOS learning fails when:

- The only machine is your daily driver and every `nixos-rebuild` is scary
- Disk fills because `/nix/store` and VM images share a tiny root
- Nested “VM inside VM inside laptop on battery” becomes the whole evening

A **single spare machine** (your case: HP Elitebook 2570p · 16 GB RAM) is enough for the entire path—if you treat it as a **lab server**, not a casual browser laptop.

Philosophy: host is not an appliance

Homelab hypervisors like Proxmox are excellent—and often **GUI-first**. You *can* automate them (CLI, API, Terraform, Ansible), but day-to-day debugging still tempts **one-off imperative fixes** that never return to git. That drift is annoying for humans and messy if an agent runs dozens of shell “fixes” that leave the machine unreproducible.

This curriculum optimizes for a different contract, in the spirit of operators who moved **Proxmox** → **NixOS + Incus** (see [Bas Nijholt, I’ve gone full Nix](#)):

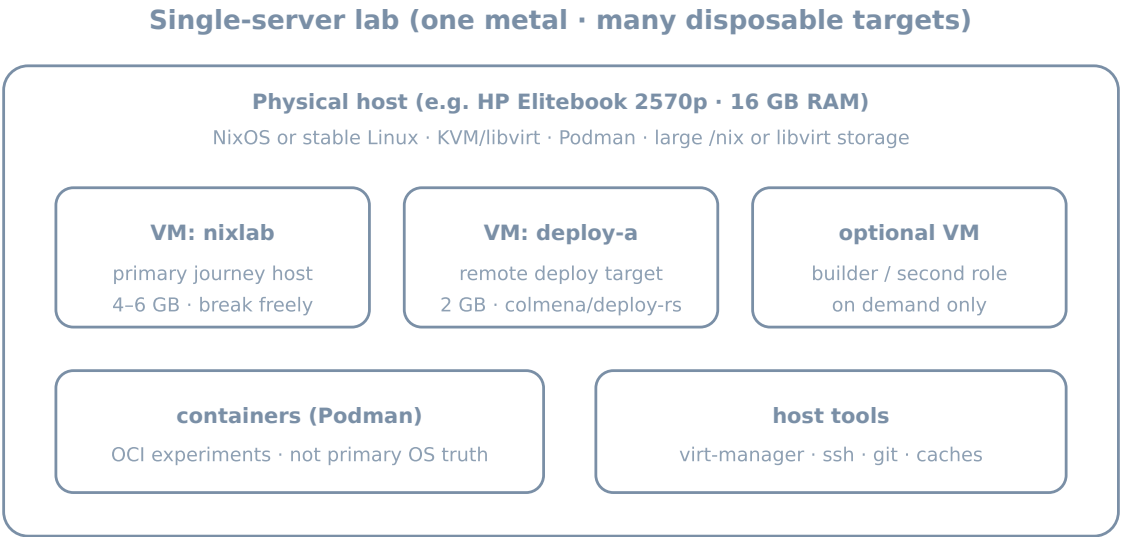
Idea	What it means for Lab 0
Git is the source of truth	Host networking quirks, hypervisor packages, firewall ports live in the flake—not only in shell history
Host is fully yours	NixOS-as-host is not a locked appliance: you <i>may</i> run tools on metal, but journey breakage stays in guests
CLI-first ops	Prefer <code>virsh</code> / <code>incus</code> / <code>nixos-rebuild</code> over click-only workflows so every step is copy-pasteable
Simulate before bare metal	Prove a config in a VM that mirrors the host role <i>before</i> you reinstall the Elitebook

Idea	What it means for Lab 0
Hardware fixes are modules	NIC offload bugs, firmware flags, microcode → commented Nix + systemd, not “I ran ethtool once in 2019”

Proxmox remains fine software. For **this** guide we default to **NixOS on metal** so one language describes host + guests.

Three roles (do not collapse them)

Role	What it is	Mutability
Lab host (hypervisor)	Metal OS: KVM/libvirt and/or Incus , storage, networking, maybe Podman	Stable; rare rebuilds; fully declarative
Journey guest(s)	NixOS VMs (or Incus VMs) you rebuild, break, roll back (the installing NixOS chapter+)	Disposable
Dev environment	Where you edit flakes, run <code>nix</code> CLI, commit git	Stable tools; can be host <i>or</i> a dedicated VM



Chapter	Purpose
Lab 0A — Journey single-server	One metal box: libvirt and/or Incus , containers, storage, simulate-before-metal
Lab 0B — NixOS dev machine	Editor, flakes, SSH, caches, drift discipline
Lab 0C — Elitebook 2570p profile	Concrete RAM/CPU/disk/BIOS plan for your spare HP
Lab 0D — Ops cheatsheet	Daily commands

You can run **0A** and **0B** on the same laptop (recommended for 16 GB): host = stable NixOS with hypervisor; guests = journey; host also has Nix + editor for flakes.

Recommended topology for this curriculum

```

Elitebook 2570p (16 GB)
  Host OS (NixOS 26.05 preferred)    ← fully yours; flake-managed
    Hypervisor (pick one primary)
      libvirt/KVM (default for this guide)
      and/or Incus (CLI-first VMs + system containers; optional)
        nixlab      4-6 GB  ← primary journey machine
        deploy-a    2 GB   ← deploy-rs / Colmena target
        (optional builder) on demand
    Podman (rootless)                ← OCI days
    ~/src/nix-config                  ← git = source of truth
    optional: Attic/Cachix
  Do NOT: run k3s multi-node + desktop + 3 heavy VMs all day

```

Decision: host OS

Host choice	Pros	Cons
NixOS host	One mental model; host reclaimable via generations; great for Lab 0B	Early days need care not to smash host
Debian/Ubuntu host	Familiar if you are not ready	Two worlds (apt host + NixOS guests); more drift risk

Expert default: NixOS as host with strong separation—host flake boring and pinned; guests are the playground.

Hypervisor choice (libvirt vs Incus)

Stack	Strength	Use when
libvirt + virt-manager Incus (LXD community fork)	Familiar, great snapshots, virt-install simple CLI-first VMs + system containers; strong for pets + automation	Default for this curriculum on 16 GB After host is solid, if you want one CLI surface
Both	Possible but heavy	Avoid on Elitebook unless you know why

Lab 0A documents **libvirt** fully and **Incus** as an optional path (aligned with Proxmox→NixOS+Incus style migrations).

What “ready for the Why Nix exists chapter” means

- ☐ Spare machine boots reliably; disk has free space (see Lab 0C)
- ☐ Virtualization enabled in BIOS; at least one NixOS VM can boot
- ☐ You can SSH into the VM from the host (or use console)
- ☐ You know which disk/VM you are allowed to wipe
- ☐ Dev tools path chosen (Lab 0B): editor + **nix** + git on host or dev VM
- ☐ Host (and guest) config lives in **git**, not only live state

Time budget

Task	Estimate
BIOS + disk + host install	2–4 h
KVM/Incus + first guest	1–2 h
Dev tooling (Lab 0B)	1–2 h
Elitebook tuning (Lab 0C)	30–60 min

Do this **once** before the Why Nix exists chapter; revisit when Stage II and Stage VI need second VMs.

Further reading

- [I've gone full Nix: Proxmox to NixOS + Incus](#) — Bas Nijholt (why declarative host + Incus; simulate-before-metal; hardware quirks in git)

Next

→ [Lab 0A — Journey single-server](#)

Lab 0A — Journey single-server (KVM + containers)

Lab 0A — Journey single-server (KVM + containers)

Goal: Turn one physical machine into a lab server that can run the entire Nix & NixOS journey: disposable NixOS VMs under **KVM/libvirt** (default) or **Incus** (optional CLI-first path), light **containers**, enough disk for `/nix` and disk images, and a network layout that makes deployments/Colmena feel real.

Audience: anyone on a single spare box (including HP Elitebook 2570p · 16 GB—see Lab 0C).

i Note

Several operational ideas below are aligned with [Bas Nijholt’s write-up of leaving Proxmox for NixOS + Incus](#) (2025–2026): git as source of truth, CLI-first hypervisor ops, simulate host configs in VMs before reinstalling metal, and capturing hardware quirks as Nix—not shell history. Proxmox itself remains fine software; the point is **reproducible host state**, not “GUI bad.”

What this lab optimizes for

Curriculum need	Lab feature
the Why Nix exists chapter–10 Nix on host or VM	nix available; flakes on
the installing NixOS chapter–22 break/rebuild NixOS	VM snapshots + generations
the secrets problem chapter–44 services	One primary guest with RAM for DB/proxy
the Disko chapter–70 deploy / multi-host	Second small VM on same lab network
Containers days	Podman and/or Incus system containers
Capstone wipe/rebuild	Guest disks you can delete without guilt
Host recoverability	Host flake + generations; optional “simulate host in VM” first

Architecture

```
libvirt default or lab bridge
192.168.122.0/24 (example NAT)
```

```

nixlab          ssh          deploy-a
4-6G RAM              2G RAM
journey primary      deploy target

edit flakes from host

host (metal)
KVM · git · nix

```

NAT is enough for the journey. Bridge to LAN only if you need LAN access from VMs and understand your network.

Part 1 — Host OS install (metal)

Option H1 (recommended): NixOS host

1. Backup anything on the Elitebook you still care about.
2. Install **NixOS 26.05** minimal or graphical ISO on metal.
3. Use **whole-disk** install on internal drive (or external SSD if you want zero risk to another OS).
4. Enable flakes from first config (Lab 0B expands this).

Minimal host modules (concept)—**all of this belongs in git**, including later hardware quirks:

```

{
  virtualisation.libvirtd.enable = true;
  virtualisation.libvirtd.qemu = {
    package = pkgs.qemu_kvm;
    swtpm.enable = true; # optional
  };
  programs.virt-manager.enable = true; # GUI optional; CLI remains primary
  users.users.YOU.extraGroups = [ "libvirtd" "kvm" "wheel" ];
  networking.firewall.allowedTCPPorts = [ 22 ];
  services.openssh.enable = true;

  # Disk: leave large room for /var/lib/libvirt and /nix
  nix.settings.experimental-features = [ "nix-command" "flakes" ];

  # Example pattern: document WHY a host tweak exists (see Part 9)

```

```
# systemd.services.fix-nic-offload = { ... };
}
```

Host is not a locked appliance. Unlike “don’t touch the hypervisor OS” products, NixOS-as-host *allows* you to run tools on metal (editor, caches, even a media app). Still: **curriculum breakage stays in guests**. If you change the host, change the **flake**, not only a live command.

Option H2: Debian/Ubuntu host

```
sudo apt update
sudo apt install -y qemu-kvm libvirt-daemon-system libvirt-clients \
    bridge-utils virt-manager
sudo usermod -aG libvirt,kvm "$USER"
# log out/in
virsh list --all
```

Install **Nix multi-user** on the host for Lab 0B (editing flakes without only working inside guests):

```
# official installer - follow nixos.org/download
nix --version
```

Part 2 — Storage plan (critical on 16 GB RAM laptops)

RAM is fine; **disk** is usually the failure mode.

Path	Role	Guidance
/nix	Host store (if NixOS/Nix on host)	40–80+ GB free if possible
/var/lib/libvirt/images	VM disks	40–100 GB free
~/src	Flake git repos	SSD strongly preferred

If the Elitebook still has a spinning HDD

- Prefer a **used SATA SSD** (even 256–512 GB) before investing more evenings.
- Or use **external USB3 SSD** for libvirt images + large store (acceptable for lab).

Thin provision

Use **qcow2** with modest virtual size (e.g. 32–40 GB per guest) so you do not preallocate everything.

Part 3 — KVM: create journey VMs

Resource template (16 GB host)

VM	vCPU	RAM	Disk	Purpose
nixlab	2	4096–6144 MiB	32–40 G	Main curriculum
deploy-a	1–2	2048 MiB	20–25 G	Remote deploy target
builder (optional)	2	2048	20 G	Offload builds (Stage V+)

Leave 4 GB for the host if you run a graphical desktop on metal.

Example: virt-install (adjust ISO path)

```
# Download NixOS 26.05 minimal ISO to ~/iso/
ISO=~/iso/nixos-minimal-26.05-x86_64-linux.iso

virt-install \
  --name nixlab \
  --memory 4096 \
  --vcpus 2 \
  --disk path=/var/lib/libvirt/images/nixlab.qcow2,size=40,format=qcow2 \
  --cdrom "$ISO" \
  --os-variant nixos-unknown \
  --network network=default \
  --graphics spice \
  --noautoconsole
```

Use virt-manager GUI if you prefer.

Snapshots = courage

Before risky Stage II/IV experiments:

```
virsh snapshot-create-as nixlab pre-stage-II "clean before break"
# restore:
# virsh snapshot-revert nixlab pre-stage-II
```

Also learn NixOS generations inside the guest—both matter.

Networking for “remote” deploy

On the host:

```
virsh net-dhcp-leases default
# note nixlab and deploy-a IPs
ssh YOU@192.168.122.XX
```

Put stable hostnames in host `/etc/hosts` or libvirt network XML if you get tired of changing IPs.

Part 4 — Containers (Podman)

Use containers for **OCI days**, not as a substitute for learning NixOS modules.

On host (NixOS):

```
virtualisation.podman.enable = true;
virtualisation.containers.enable = true;
```

Rootless Podman preferred for daily experiments.

```
podman run --rm -it docker.io/library/alpine:latest uname -a
```

When the curriculum says “declarative OCI on NixOS,” prefer defining that **inside nixlab**, so the journey guest remains the system under study.

Many operators eventually run **Compose-style “pets”** inside a VM while the **host** stays pure NixOS. That split (declarative host + stateful guest workloads) is valid later; early stages should still teach **NixOS modules** before outsourcing everything to containers.

Part 4b — Optional hypervisor: Incus (CLI-first VMs + system containers)

Incus (community fork of LXD) is a strong fit when you want **one CLI** for VMs and system containers, and when you care about agentic/automation workflows that hate opaque UIs. It is **optional** for this curriculum; libvirt remains the default on 16 GB laptops.

When to consider Incus on the lab host

Choose Incus if...	Prefer libvirt if...
You want incus launch / profiles as daily API	You already know virt-manager and want fastest start
You may import Proxmox LXC/VM images later	You only need 1–2 simple NixOS guests
You like image profiles + clean CLI	You want minimal host packages

Do not run full Incus + full libvirt stacks + desktop + heavy VMs on the Elitebook without a reason—pick a **primary**.

NixOS host sketch (Incus)

```
{
  virtualisation.incus.enable = true;
  # networking: follow current nixpkgs Incus docs for bridge (incusbr0) vs custom
  users.users.YOU.extraGroups = [ "incus-admin" ]; # group name may vary by package
  networking.firewall.trustedInterfaces = [ "incusbr0" ]; # only if you use default bridge
}
```

Verify against **nixpkgs 26.05** option names (`man configuration.nix` / search.nixos.org)—Incus module knobs evolve.

```
incus admin init    # first-time interactive; then encode stable choices into docs/flake notes
incus list
# VM example (image names change - use `incus image list images:`):
# incus launch images:nixos/unstable nixlab --vm -c limits.memory=4GiB
```

Simulate the host before you reinstall metal

Powerful idea from the Proxmox→NixOS migration write-up: run an Incus (or libvirt) VM that imports **almost the same modules** as the physical host, with a small **overrides** file (disk device names, interface names, disable real-metal-only services).

```
modules/host-common.nix    ← ssh, libvirt/incus, users, nix settings
hosts/elitebook.nix        ← real hardware-configuration + product
hosts/elitebook-sim.nix    ← imports common + overrides (virtio NICs, qcow root)
```

Workflow:

1. Get `elitebook-sim` healthy in a VM.
2. Only then install/reinstall the physical Elitebook from the same flake.
3. Keep `incus-overrides.nix`-style diffs tiny and commented.

This removes “nuking the only lab machine” fear—especially valuable once the host itself is NixOS.

If you already run Proxmox

You do **not** need to migrate mid-curriculum. If you later consolidate:

Workload	Typical move
Proxmox LXC	<code>vzdump</code> → import into Incus (community scripts exist; treat as advanced)
Proxmox VM disk	<code>qemu-img convert</code> → <code>qcow2</code> → Incus or libvirt import

Workload	Typical move
NixOS journey guests	Prefer fresh NixOS VMs for learning; migrate pets only

Keep migration notes in your flake repo (inventory of CT/VM IDs, disk backends). Optional reading: migration notes linked from [the same article’s references](#).

Part 5 — Nested virtualization?

Elitebook Ivy Bridge: **VT-x** yes (enable in BIOS). Nested KVM (VM inside VM) is **optional** and often slow on this generation.

Need nested?	When
No	Default path: guests are the NixOS systems under test
Maybe	You insist on running libvirt <i>inside</i> nixlab

Recommendation: **do not** nest until you have a specific reason.

Part 6 — Services the host should run (boring is good)

Service	Why
sshd	Access from your main laptop if Elitebook is headless-ish
libvirtd and/or incus	Hypervisor
chrony/systemd-timesyncd	TLS and logs make sense
Optional cockpit / virt-manager	GUI if local—still commit CLI equivalents

Do **not** turn the host into the Stage IV app zoo. Guests carry curriculum apps.

(Exception later: a *deliberate* host service that is in the flake—e.g. a binary cache—never “I apt installed it at 2am.”)

Part 7 — Daily workflow for the journey

1. Open host terminal / editor on `~/src/nix-config`
2. Edit flake modules for nixlab / deploy-a (and host/ only when intentional)
3. Prefer: `nix build / nixos-rebuild build-vm / --target-host`
4. Apply inside guest; avoid unexplained one-off guest shell config
5. Snapshot before scary changes
6. Commit flake.lock + config ← if it isn’t in git, it didn’t happen

Alternative: `nixos-rebuild --target-host / deploy-rs` from host into guests (Stage VI).

Debugging without drift

When something breaks, the order is:

1. Reproduce
2. Fix in **Nix** (or Incus profile / cloud-init you track)
3. `switch` / redeploy
4. Commit with a comment that cites *why* (forum link, hardware quirk)

Avoid: fix live → forget → six months of mystery. That is the same class of failure as “checkbox in a web UI three years ago.”

Part 8 — Declarative hardware quirks (host flake)

Older Intel NICs and laptops often need one-line kernel/driver workarounds. Example pattern (adapt; **do not** copy blindly):

```
# hosts/elitebook/networking.nix - illustrative
{
  # Document: link to bug/forum in a comment above the service.
  systemd.services.disable-nic-offload = {
    description = "Disable problematic NIC offloads on this hardware";
    wantedBy = [ "multi-user.target" ];
    after = [ "network-pre.target" ];
    serviceConfig.Type = "oneshot";
    serviceConfig.RemainAfterExit = true;
    script = ''
      # Example only - identify YOUR interface with `ip link`
      # ethtool -K enp0s25 tso off gso off gro off || true
    '';
    path = [ pkgs.ethtool ];
  };
}
```

On the Elitebook, when WiFi/Ethernet misbehaves: capture the fix **here**, not only in a successful interactive shell. Same for microcode, `boot.kernelParams`, and firmware.

Part 9 — Failure modes checklist

Symptom	Fix
VMs won't start	BIOS VT-x; user in <code>libvirtd/kvm</code> / <code>incus-admin</code> groups
Host OOM killer	Lower guest RAM; shut deploy-a when idle
Disk full	GC carefully; delete old snapshots; move images to external SSD
Nested rebuild hell	You nested by accident—flatten topology
WiFi-only flaky SSH	Prefer Ethernet dock/USB ethernet for lab sessions
“Works after I typed stuff” unreproducible	Re-apply from flake; delete untracked live changes
Incus + libvirt both fighting bridges	Pick one primary network stack

Checkpoint — Lab 0A complete

- ☐ Host installed and updated; **host flake** exists in git
- ☐ `virsh list --all` or `incus list` works (primary stack chosen)
- ☐ **nixlab** VM created (ISO install can finish on the installing NixOS chapter if needed)
- ☐ Snapshot (libvirt) or publish/restore strategy (Incus) tested once
- ☐ Optional: host-sim VM validates host modules before metal reinstall
- ☐ Podman (or Incus container) smoke test optional
- ☐ Written note: IPs/hostnames of guests
- ☐ Know wipe boundary: **guests + lab disks**; host only via generations + git
- ☐ At least one hardware or network quirk documented **in Nix** if you hit it

Next

- [Lab 0B — NixOS dev machine](#) (tooling on this same host)
- [Lab 0C — Elitebook 2570p](#) (your hardware numbers)

Lab 0B — NixOS development machine

Lab 0B — NixOS development machine

Goal: Configure a **comfortable, stable** environment for writing flakes, reviewing nixpkgs, running builds, and pushing configs to lab VMs—without turning your editor session into the system under test.

On a single Elitebook, **Lab 0B usually lives on the host** (same metal as Lab 0A). The split is **role**, not necessarily second hardware.

💡 Tip

Text-first infrastructure (flake, modules, `virsh/incus CLI`) is also what makes **human + AI** co-maintenance safe: agents can read and propose diffs against git. GUI-only state and untracked shell fixes cannot. Keep Lab 0B disciplined: **if it matters, it is in the repo.**

Dev machine vs journey guest

	Dev (this lab)	Journey guest (nixlab)
Purpose	Author configs, run <code>nix</code> tools	Be the OS you break
Rebuild frequency	Low	High
Secrets	Your age keys, git ssh	Lab secrets only
Editor	Yes	Optional
“Does this need to be pretty?”	Yes (you live here)	No

Part 1 — Install Nix (if host is not NixOS)

Multi-user install; enable flakes:

```
# ~/.config/nix/nix.conf
experimental-features = nix-command flakes
```

On **NixOS host**, put the same under `nix.settings.experimental-features`.

Part 2 — Essential packages

NixOS host module sketch

```
{ pkgs, ... }: {
  environment.systemPackages = with pkgs; [
    git
    git-lfs
    vim
    neovim
    ripgrep
    fd
    jq
    tree
    htop
    tmux
    direnv
    nix-direnv
    nixfmt-rfc-style    # or nixfmt - pick one and stick to it
    nil                 # LSP for Nix
    # optional:
    # vscode / emacs - your preference
  ];

  programs.direnv.enable = true;
  programs.direnv.nix-direnv.enable = true;

  # nice for rebuilds against VMs
  environment.systemPackages = with pkgs; [
    # already have git etc.
  ];
}
```

Non-NixOS host

```
nix profile install nixpkgs#git nixpkgs#neovim nixpkgs#direnv nixpkgs#nix-direnv nixpkgs#nil
```

Part 3 — Editor + LSP

Editor	Nix LSP
Neovim/VS Code	nil or nixd
Emacs	same + envrc

Minimal checks:

```
nil --help    # or nixd
rg "mkOption" ~/src/nix-config -n | head
```

Part 4 — Git layout for the whole journey

```
~/src/nix-config/          # or ~/lab/nixos/nixos-config
flake.nix
flake.lock
hosts/
  nixlab/
  deploy-a/
modules/
homes/                    # from Stage III
secrets/                  # encrypted only (Stage IV)
.envrc                    # use flake
README.md
```

```
cd ~/src/nix-config
git init
echo ".direnv" >> .gitignore
echo "result" >> .gitignore
echo "result-*" >> .gitignore
```

.envrc:

```
use flake

direnv allow
```

Part 5 — Build ergonomics

Keep host builds from melting a 2012 laptop

```
nix.settings = {
  max-jobs = 2;           # Elitebook: start low
  cores = 2;
  # auto-optimise-store = true; # optional
};
```

Substituters

Default `cache.nixos.org` is fine. Later stages may add Cachix/Attic—document tokens only in sops, never in world-readable nix.

Remote / guest as builder (optional, Stage V+)

When `nixlab` is stronger or you want isolation:

```
# conceptual - configure nix.buildMachines later in curriculum
```

Early on: **build on host, deploy to guest** is simplest.

Part 6 — Access to journey VMs

```
# ssh config ~/.ssh/config
Host nixlab
  HostName 192.168.122.10
  User YOU
  IdentityFile ~/.ssh/id_ed25519_lab

Host deploy-a
  HostName 192.168.122.11
  User YOU
  IdentityFile ~/.ssh/id_ed25519_lab
```

Generate a **lab-only** key:

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_lab -C "lab-elitebook"
```

Do not reuse your primary GitHub key as the only lab key if you prefer blast-radius control.

Part 7 — Workflow commands you will live with

```
# evaluate / build a guest config from the flake (host)
nix build .#nixosConfigurations.nixlab.config.system.build.toplevel

# apply (from host, once SSH works)
nixos-rebuild switch --flake .#nixlab --target-host YOU@nixlab --use-remote-sudo

# or: ssh and rebuild inside guest
ssh nixlab 'sudo nixos-rebuild switch --flake /path/to/config#nixlab'
```

```
# format
nix fmt # if configured
```

Part 8 — Home Manager on the dev role

Optional but nice on the host:

- Shell (fish/zsh), git, ssh config
- Keep HM on host **separate** from guest HM experiments

When Stage III teaches HM, practice on nixlab first if you want isolation; port habits back to host later.

Part 9 — Binary cache & secrets (preview only)

Later stage	Dev machine prep
sops-nix	Install sops , age ; create age key backup offline
CI	GitHub/GitLab SSH deploy key for the flake repo
Cachix	Account ready; don't put signing key in git

Create age key early if you want:

```
mkdir -p ~/.config/sops/age
age-keygen -o ~/.config/sops/age/keys.txt
# back up keys.txt to password manager / encrypted USB
```

Part 10 — Quality of life on old hardware

Tip	Why
External monitor + keyboard	2570p is a server that happens to fold
tmux/zellij	Long builds survive SSH drops
Disable heavy desktop effects on host	Free RAM for VMs
nix-store --optimise occasionally	Disk
Don't run browser + 2 VMs + big compile	Sequentialize

Dev machine checklist

- ☐ `nix flake metadata` works in `~/src/nix-config` (even empty flake)
- ☐ `direnv` loads a simple `devShell`
- ☐ Editor can jump/lint Nix files
- ☐ SSH to `nixlab` (or console) documented
- ☐ Lab SSH key created; age key backup plan written
- ☐ `max-jobs/cores` set conservatively
- ☐ `.gitignore` includes `result` and `.direnv`

Anti-patterns

Anti-pattern	Prefer
Editing production-like host to “try modules” One 2k-line <code>configuration.nix</code> on host	Try on <code>nixlab</code> (or host-sim VM) Layout from Stage III early for guests
Storing secrets in world-readable nix Building with <code>max-jobs = auto</code> until OOM “YOLO” imperative fix left uncommitted	<code>sops/agenix</code> later Cap jobs on Elitebook Encode fix in flake; commit with <i>why</i>
Relying on virt-manager clicks you cannot replay	Keep a <code>virt-install</code> / <code>incus</code> snippet in README

Next

- [Lab 0C — HP Elitebook 2570p profile](#)
- Then [Why Nix exists](#) (first topic chapter)

Lab 0C — HP Elitebook 2570p (16 GB) profile

Lab 0C — HP Elitebook 2570p · 16 GB RAM

lab profile

Concrete profile for a spare **HP Elitebook 2570p** with **16 GB RAM**, used as the single-server lab for this NixOS volume (Labs 0A + 0B on the same chassis).

Note

The 2570p is ~2012 Ivy Bridge hardware. It is **excellent** as a quiet lab server for learning NixOS. It is **not** a modern workstation: respect thermals, disk, and CPU age.

Hardware snapshot (typical 2570p)

Component	Typical	Lab implication
CPU	Intel Core i5/i7-3xxxM (Ivy Bridge)	2 cores / 4 threads class; VT-x yes
RAM	Up to 16 GB DDR3 (you have 16 GB)	Comfortable for 1 heavy + 1 light VM
GPU	Intel HD 4000	Fine for console/light desktop host; not GPU passthrough playground
Disk	Factory HDD or aftermarket SATA SSD	SSD strongly recommended
Display	12.5"	Treat as headless-ish; external monitor helps
NIC	Intel wired + WiFi	Prefer Ethernet for long SSH sessions
Firmware	HP BIOS	Enable Virtualization Technology

Exact CPU SKU varies (`lscpu` after boot). Confirm:

```
lscpu | rg -i 'Model name|Virtualization|CPU\\(s\\)|Thread'
free -h
lsblk
```

BIOS checklist (do this first)

1. **Virtualization Technology (VTx)** — Enabled
2. **VT-d** — Enable if present (not required for basic KVM)
3. Boot mode — UEFI preferred if disk allows; legacy OK for lab
4. Disable unused boot devices if they slow POST
5. Set fans/thermal to a sane default; keep vents clear

Save & reboot.

Disk strategy for *this* laptop

Best

Disk	Use
Internal SATA SSD 512 GB+	Host root + <code>/nix</code> + libvirt images
Optional USB3 SSD	Overflow images / backups

Acceptable

Disk	Use
Internal SSD 256 GB	Host + one VM; external for second VM images
HDD only	Usable but painful; prioritize SSD upgrade

Partition sketch (NixOS host, single disk)

ESP	512 MB	FAT32
root	rest	ext4 or btrfs
/nix		(same FS is fine; optional subvol on btrfs)

If dual-booting is tempting: **don't** for this spare. Dedicate the machine to lab.

RAM budget (16 GB fixed)

Consumer	Allocation	Notes
Host OS + desktop/ssh	4–5 GB	Prefer lightweight DE or none
nixlab VM	4–6 GB	Primary journey
deploy-a VM	2 GB	Stop when idle
Headroom / browser	2–3 GB	Close browser during big builds
Total	16 GB	Never overcommit hard on this chassis

Suggested virsh memory

```
# examples
# nixlab: 4096 or 6144 MiB
# deploy-a: 2048 MiB
```

During Stage IV (DB + proxy + observability): prefer **6 GB nixlab** and **powered-off deploy-a** unless needed.

CPU budget

Setting	Value
nixlab vCPUs	2
deploy-a vCPUs	1–2
Host <code>nix.settings.max-jobs</code>	1–2
Host <code>nix.settings.cores</code>	2

Ivy Bridge will thermal-throttle under sustained compile. That is normal—not a broken Nix.

What this machine is good for (curriculum map)

Stage	On 2570p?
I Foundations	Yes
II One NixOS host (in VM)	Yes
III HM + layout	Yes
IV Secrets + small services	Yes (watch RAM)
V Packaging	Yes (be patient; use caches)
VI Deploy two VMs	Yes
VI k8s / heavy observability	Optional/light only
VII Internals	Yes
VIII Capstone	Yes (scope to hardware)
Nested libvirt	Avoid
Multi-node k8s + desktop	No

Networking on the Elitebook

Mode	Recommendation
WiFi only	OK for browsing; flaky for long rebuilds
USB Ethernet / dock	Preferred for lab evenings
Bridged VM to LAN	Optional; NAT default is safer

Port-forwarding example (if you must reach a guest service from another PC): use host SSH tunnel rather than exposing VMs broadly.

```
ssh -L 8080:192.168.122.10:80 user@elitebook-host
```

Power & thermals

- Use **AC power** for builds and VM sessions
- Elevate rear for airflow if temps high
- `tlp` / power-profiles: on host, prefer performance when labbing on AC
- Expect fan noise under `nix build`—plan quiet hours

Firmware & drivers notes

Area	Note
WiFi	Prefer linux-firmware ; some cards need allowUnfree for redistributable firmware
CPU microcode	<code>hardware.cpu.intel.updateMicrocode = true</code> ; on NixOS host
GPU	Modesetting is enough; no NVIDIA complexity on stock 2570p
Fingerprint / WWAN	Ignore for lab
Ethernet	Older Intel NICs sometimes need offload disabled (<code>ethtool -K ...</code>); put that in the host flake with a comment (pattern in Lab 0A Part 8), not only in a one-off root shell

Host is fully yours (but still a lab)

On appliance hypervisors you are often discouraged from “messing with the host.” On **NixOS host**, you *can* install an editor, run a local cache, or attach a display—and still recover via generations.

That does **not** mean the Elitebook should become your Stage IV app zoo. Keep metal useful; keep **journey risk in VMs**.

Concrete “day zero” procedure (you)

1. **Backup** anything left on the 2570p.
2. Install **SSD** if still on HDD (worth it).
3. BIOS: enable VT-x.
4. (*Optional but wise*) On another machine or live USB workflow: draft `hosts/elitebook-sim.nix` and boot it as a VM first (Lab 0A “simulate host”).
5. Install **NixOS 26.05** as host (Lab 0A Option H1).
6. Apply host config from git: `libvirt` **or** `Incus` (pick one primary), `ssh`, `flakes`, `microcode`, your user.
7. Create **nixlab** (4–6 GB) + **deploy-a** (2 GB).
8. Snapshot blank/fresh installs.
9. Clone/create `~/src/nix-config` flake (Lab 0B); commit host + guest stubs.
10. SSH from your daily laptop into Elitebook; from Elitebook into VMs.
11. Only then start **the Why Nix exists chapter**.

Optional extras that fit this chassis

Extra	Why
Automatic libvirt start	<code>nixlab</code> available after host boot
External USB SSD labeled <code>nix-lab</code>	Images + <code>nix</code> copy backups
UPS not required	Laptop battery is a mini-UPS for short blips
Second-hand 8 GB→16 GB already done	Good—don’t bother 32 GB (max 16 on this platform)
Replace thermal paste	Only if throttling is extreme

What not to run on the 2570p

- Desktop + gaming + three 8 GB VMs
- Full Observability stack + k3s + browser on host simultaneously
- Treating host as wipe-target for Stage II installs (use **nixlab**)
- Expecting modern Secure Boot/TPM2 demos (Stage VI Secure Boot day may be **document-only** on this hardware)

For Lanzaboote/TPM chapters: complete theory + skip or use a newer machine if you have one later.

Elitebook readiness checklist

- ☐ 16 GB confirmed in `free -h`
- ☐ VT-x enabled; `kvm-ok` or equivalent happy
- ☐ SSD or external fast disk for images
- ☐ Host NixOS (or Debian+Nix) installed
- ☐ libvirt works; **nixlab** defined
- ☐ RAM split written down and enforced
- ☐ `max-jobs 2`
- ☐ Ethernet or stable WiFi plan
- ☐ Lab 0A + 0B checklists done
- ☐ Wipe boundary: guests only

After this page

You are ready for [Why Nix exists](#) (first topic chapter after Lab 0).

When the installing NixOS chapter says “install NixOS,” install into **nixlab** (or replace its disk), not by reinstalling the Elitebook host unless you intentionally redesign Lab 0A.

Lab 0D — Ops cheatsheet

Lab 0D — Ops cheatsheet (single-server lab)

Quick reference while studying. Details live in Labs 0A–0C.

Daily

```
# host - libvirt (default path)
virsh list --all
virsh start nixlab
virsh snapshot-list nixlab

# host - Incus (if that is your primary)
incus list
incus start nixlab
incus info nixlab

# network (libvirt NAT example)
virsh net-dhcp-leases default

# ssh
ssh nixlab
ssh deploy-a
```

Snapshots / restore points

```
# libvirt
virsh snapshot-create-as nixlab "label" "notes"
virsh snapshot-revert nixlab "label"
virsh snapshot-delete nixlab "label"

# Incus (publish image / snapshot features - see `incus snapshot` help for your version)
incus snapshot create nixlab pre-change
incus snapshot restore nixlab pre-change
```

Disk pressure

```
df -h / /nix /var/lib/libvirt/images
nix store gc          # careful: know generations
nix store optimise
du -sh /var/lib/libvirt/images/*
```

Guest rebuild (from host flake)

```
cd ~/src/nix-config
nix build .#nixosConfigurations.nixlab.config.system.build.toplevel
nixos-rebuild switch --flake .#nixlab --target-host USER@nixlab --use-remote-sudo
```

When the Elitebook sweats

1. Shut down deploy-a
2. Set max-jobs = 1
3. Close browser
4. Prefer substitutes over local compiles when possible
5. Pause nested experiments

Drift control

```
cd ~/src/nix-config
git status      # uncommitted host/guest edits?
git diff
# After a live debug: encode the fix in *.nix, rebuild, then commit
```

Emergency

Problem	Action
Host unresponsive	REISUB if enabled; else hard power; check disk full

Problem	Action
Guest won't boot	libvirt/Incus snapshot or NixOS generation from boot menu
Locked out of guest	virt-manager / virsh console / incus console
Store full	Delete old snapshots; nix store gc with eyes open
"Fixed it in the shell, lost it on reboot"	That was drift—put it in the flake

Inventory template (fill once)

```

Host hostname: _____
Host IP: _____
nixlab IP: _____
deploy-a IP: _____
Flake path: _____
Lab SSH key: _____
Age key backup: _____
Wipe-allowed disks: _____

```

Concepts

Concepts — Nix language, store, CLI, flakes

Concepts — Nix language, store, CLI, flakes

This part of the **standalone NixOS book** is organized by **topic** (normal book chapters), not by calendar day.

Read chapters in order within the part when learning; skip or skim freely when using the book as a reference.

What this part is for

Concepts builds the mental models every later part assumes: what Nix is, how the store works, how the language composes config-shaped data, how derivations become store paths, and how the modern flakes CLI pins work.

You will not install NixOS here. You will install the **package manager**, enable **flakes**, and practice evaluation, builds, and project shells on any supported host (Linux or macOS for most drills; some store demos prefer Linux).

This part	Next part
Language + store + flakes literacy	Nix on Linux (02-nix-on-linux/) — Fedora 44+ dual-stack bridge (recommended)
Disposable experiments on any Nix install	Then NixOS host — machine as code, generations, modules, boot

Baseline (this book)

Piece	Baseline
Nix	2.34.x (modern CLI + flakes)
nixpkgs channel pin	nixos-26.05 (Yarara) when you pin flakes
Default workflow	Flakes + nix build / nix run / nix develop
Classic CLI	Literacy only (last chapter of this part)

Version numbers drift in the wild; when in doubt, check **nix --version** and pin **26.05** in your own flakes so examples stay comparable.

Learning outcomes

By the end of this part you should be able to:

1. Separate the **three faces** of Nix: language, package manager, NixOS
2. Read a `/nix/store/...` path and explain **closures** and **GC roots**
3. Write nested **attrsets**, **let/inherit**, and config-shaped data
4. Write **functions** with attr-pattern args, defaults, ellipsis, and `@`
5. Split multi-file Nix with **laziness** and a few **lib** pipelines
6. Explain a **derivation** as a plan $\rightarrow$ `.drv` $\rightarrow$ realized output (including FODs at awareness level)
7. Use the **modern CLI** for build/run/shell/develop/eval/path-info
8. Author a **project flake** with lockfile + `devShells.default`
9. Translate **classic** commands when reading old docs—without living there

Prerequisites

You should have	Nice to have
Comfortable shell (bash/zsh)	Prior package-manager pain (apt/pip/npm)
Text editor + git basics	A disposable machine or VM for disk-heavy store labs
~10–20 GB free for store growth	Linux (closer to later NixOS host work)

No prior NixOS install required for this part.

Chapter map

Chapter	Title	Focus
Why Nix exists	Problem statement, three faces, install Nix, first REPL	
Store and closures	Path anatomy, references, closures, GC roots	
Language: attrsets	Values, strings, lists, sets, let , inherit , //	
Language: functions	Lambdas, pattern args, ? , ... , @ , call shapes	
Laziness, imports, lib	Lazy force, multi-file, infinite recursion, lib	
Derivations as an idea	Plans, sandbox, FODs, trivial builds, logs	
Modern Nix CLI	Installables, build/run/shell/develop, flake subcommands	
Flakes and devShells	inputs/outputs/lock, mkShell , git discipline	
Classic CLI literacy	Classic modern map, channels vs locks	

Suggested learning paths

Path	Order	Notes
Linear (recommended first pass)	01 → 09	Builds one model on the next
Language-first	03 → 05, then 01–02, 06–09	If you already installed Nix
Ops-curious skim	01, 02, 07, 08	Then return for language depth
Reference	Any chapter standalone	Use checkpoints + gotchas tables

How each chapter is structured

Topic chapters in this book generally share:

1. **Goal** — what competence you leave with
2. **Why it matters** — systems/ops motivation

3. **Theory** — tables, mental models, diagrams when present
 4. **Worked examples** — adaptablenippets (Nix 2.34 / flakes / 26.05 baseline)
 5. **Labs / exercises** — hands-on practice
 6. **Common pitfalls** — symptom → theory table
 7. **Checkpoint** — self-check before moving on
 8. **Further depth** — related chapters **in this book only**
-

Lab workspace convention

Prefer a single lab tree on your workstation (not tied to any calendar series):

```
mkdir -p ~/lab/nixos-book/concepts
cd ~/lab/nixos-book/concepts
```

Per-chapter subdirs (`why-nix`, `store`, `attrsets`, ...) keep experiments reproducible. Commit flakes and notes; do not commit private keys or machine secrets.

Mental model map (part-level)

```
Expression (Nix language)
  evaluate
```

```
Derivation plan (.drv)    realize    Store path(s)
```

```
references → closure
GC roots keep live
```

```
Modern CLI / flake outputs name installables
```

```
devShell / package / (later) nixosConfigurations
```

Keep this sketch in mind when a command “does nothing” (eval) versus “downloads forever” (realize) versus “fills the disk” (roots).

What you will *not* do in this part

Out of scope here	Covered later in this book
Install NixOS / UEFI / generations	Part NixOS host
Home Manager, fleet, Disko	Later parts
Deep <code>stdenv</code> packaging	Packaging part
Module system authoring	Host + Internals
Cross-links to any 90-day series	Not used in this book

Suggested checkpoints (part gate)

Before starting **Nix on Linux** (or NixOS host, if you skip the bridge), confirm:

- ☐ `nix --version` reports a 2.34.x-class Nix with flakes enabled
 - ☐ You can explain language vs package manager vs NixOS in one sentence each
 - ☐ You can define store path, closure, and GC root
 - ☐ You can evaluate multi-file Nix with functions and attrsets
 - ☐ You can `nix build` / `nix run` / `nix develop` against a personal flake
 - ☐ `flake.lock` is committed for that project
 - ☐ You can translate at least five classic commands to modern ones
-

Further depth (this book)

After Concepts...	Go to
Fedora / foreign distro daily driver	Part Nix on Linux (02-nix-on-linux) — official Fedora Nix, HM standalone, dual-stack
Own a machine as code	Part NixOS host (03-nixos-host)
Project layout, locks, HM	Part Home and flake layout

After Concepts...	Go to
Secrets, services, TLS	Part Services and security
Real packaging	Part Packaging
Evaluator / module deep dive	Part Internals

How to use this part as a reference

- **Sidebar titles** come from each chapter’s `title:` frontmatter—search by topic name
- **Gotchas tables** at chapter ends are optimized for “I hit this error” jumps
- **Worked examples** are meant to be copied and adapted, not treated as sacred
- Prefer **pinning nixpkgs to 26.05** in examples so your lock matches the book’s baseline

When a later chapter assumes “you know closures,” it means this part—not a calendar day number.

Why Nix exists

Why Nix exists

Goal: Separate the three meanings of “Nix,” understand the *problem statement* honestly, install the package manager with **flakes + modern CLI**, and evaluate first expressions in a REPL.

i Note

This chapter installs **Nix language + package manager**, not NixOS. The OS host arrives in the **NixOS host** part of this book.

On Fedora 44+: prefer the **official distro package** (`sudo dnf install nix`) and the full dual-stack bridge in **Nix on Linux** rather than only a generic upstream installer—SELinux and `/nix` integration are why Fedora packaged it.

Why this chapter matters

People fail Nix for social reasons as much as technical ones: they read a blog that mixes **language**, **CLI**, and **NixOS modules**, then paste commands from three eras of tooling. Theory first reduces that thrash.

Ops motivation: every later skill—closures, flakes, rebuilds, deployments—assumes you know *what problem Nix claims to solve* and *what it deliberately does not*. Without that frame you will treat Nix as “yet another package manager with weird paths” and fight the model.

Theory 1 — Three faces of the same ecosystem

Layer	Question it answers	You touch it in this chapter?
Language	How do we <i>describe</i> packages and configs?	Yes — REPL
Package manager	How do we <i>build/store/compose</i> software?	Yes — <code>install + nix run</code>
NixOS	How do we describe an entire <i>OS</i> ?	No — host part later

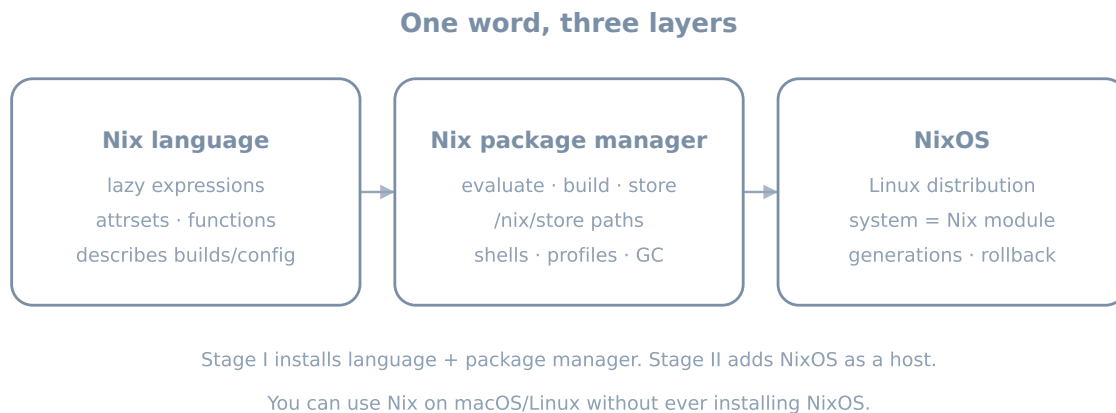


Figure 1: Language, package manager, and NixOS

Analogy (imperfect but useful)

Idea	Rough analogy
Nix expression	A pure function recipe
Derivation	A build plan (inputs → output path)
Store path	The baked loaf with a serial number
Profile / shell	A lunch tray pointing at selected loaves
NixOS system	The whole kitchen layout as code

Analogies break; the table is orientation, not law.

Naming traps in the wild

Someone says...	They might mean...
“Install Nix”	Package manager only
“Install NixOS”	Full Linux distribution
“Write some Nix”	Language expressions
“Nix config”	Could be any of the three

When reading docs, force yourself to label which face is speaking.

Theory 2 — Failure modes of traditional systems

Dependency hell

Classic package managers often assume **one** version of a library in a global prefix (`/usr/lib/...`). Applications A and B that need different versions conflict.

Example story

- App A needs `libssl.so.1.1`
- App B needs `libssl.so.3`
- Distro upgrade breaks A, or pinning freezes security updates for B

Snowflake servers

Imperative drift:

```
ssh prod
apt install foo
vim /etc/foo.conf
# six months later: nobody knows the full story
```

Language-level global installs

```
pip install --user something
npm install -g other
cargo install another
```

Each ecosystem invents isolation. None composes cleanly with the others or with the OS.

CI “works on my machine” graphs

Symptom	Typical cause
Green CI, red laptop	Different tool versions
Green laptop, red prod	Hidden system libraries
Two developers disagree	Floating “latest” pins

Nix’s bet is: **make the graph explicit and hashed**, then share it.

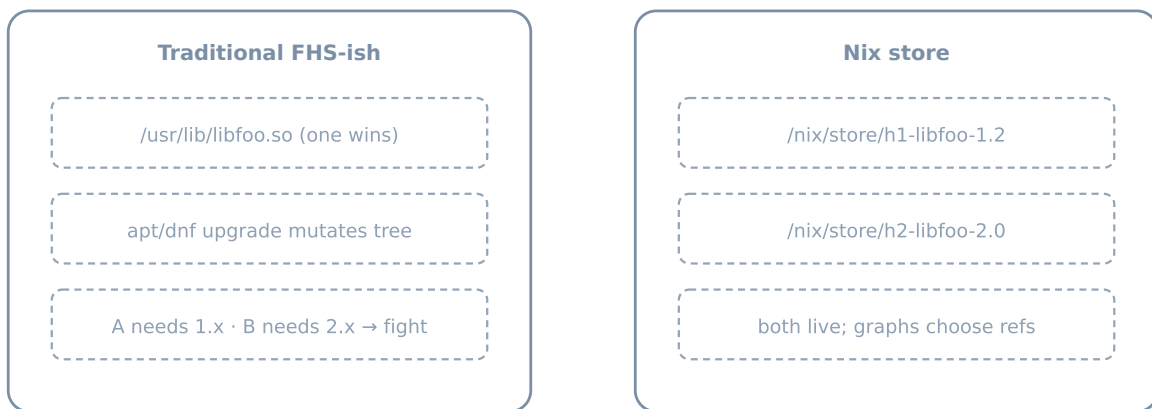
What “reproducible” can mean (honesty ladder)

Claim	Strength
“Same machine rebuilds the same store paths”	Often achievable with locks
“Same flake lock on two Linux machines → same paths”	Common with pure fixed-output inputs
“Bit-for-bit identical across OS/CPU forever”	Strong; not always true
“My <i>data</i> is reproducible”	Not Nix’s job unless you design state

Nix optimizes for **controlled, rebuildable software graphs**. It does not delete ops.

Theory 3 — Nix’s core bet: pure builds + hashed store

Dependency placement: one slot vs many hashes



Environments are compositions of store paths, not a single mutable /usr.

Figure 2: Traditional FHS vs Nix store coexistence

Content-addressed intuition

If a build is a function of its inputs, then informally:

$$\text{output path} = f(\text{build recipe, input paths, system, ...})$$

In the common **input-addressed** world, the hash reflects the derivation inputs. (Content-addressed derivations are an advanced evolution—see the **Internals** part of this book.)

Coexistence

```
/nix/store/hAAAAA-libfoo-1.2.3
/nix/store/hBBBBB-libfoo-2.0.0
```

Both can exist. Package A references hAAAAA; package B references hBBBBB. No single /usr/lib/libfoo.so gladiator match.

Environments as compositions

A “shell” or “profile” is not a pile of files dumped into /usr. It is a **set of store paths** arranged on PATH, LIB, etc.

Example mental model

```
devShell = { go_1_26, git, hello }
          → PATH entries under those store paths
```

What purity buys operations

Property	Operational win
Immutable store paths	Roll forward/back by re-pointing roots
Explicit references	Copy a complete closure to another host
Sandboxed builds	Fewer “works on my machine” build deps
Binary caches	Realize often means download, not compile

Theory 4 — Evaluation vs realization

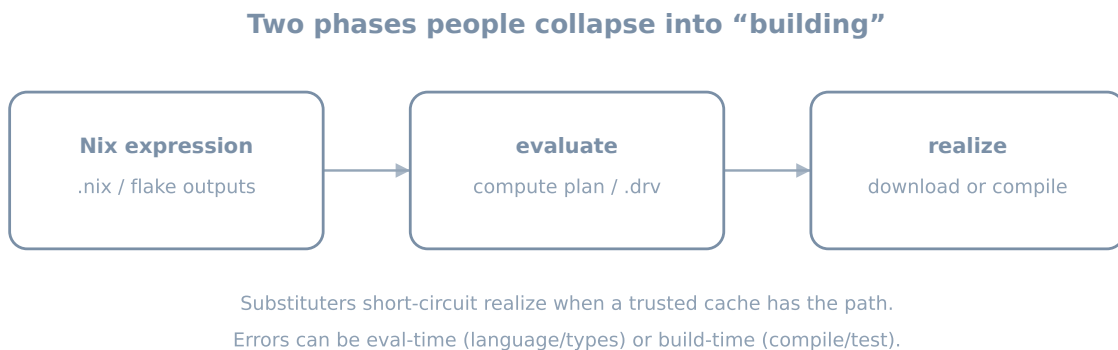


Figure 3: Evaluate then realize

Phase	What happens	Failure examples
Evaluate	Nix language runs; produces derivation plans	Infinite recursion, missing attr, type errors
Realize	Build or download outputs into the store	Compile error, hash mismatch, network/cache miss

Substituters (binary caches)

If a trusted cache already has the output path, realization becomes **download**, not compile. That is why first `nix run nixpkgs#hello` can be fast *or* slow depending on cache/network.

Example

```
eval:  nixpkgs.hello → derivation → planned out path P
realize: P exists on cache.nixos.org? download : build
```

Why the split matters for debugging

You see...	Likely phase
<code>error: undefined variable</code>	Evaluate
<code>error: attribute 'foo' missing</code>	Evaluate
<code>builder failed with exit code</code>	Realize
<code>hash mismatch in fixed-output</code>	Realize (fetcher)
Long silence then download progress	Realize via substituter

Theory 5 — Installer shapes and multi-user daemon

Shape	Idea
Multi-user + nix-daemon	Builds run as daemon; shared store; recommended default
Single-user	Store owned by your user; simpler, less isolation
Determinate installer	UX wrapper; still Nix— record that you used it

Why a daemon?

Sandbox builds need privileges to set up namespaces/mounts cleanly. Multi-user installs also protect the store from accidental mutation.

Config locations (theory)

File	Role
<code>~/.config/nix/nix.conf</code>	User Nix settings
<code>/etc/nix/nix.conf</code>	System-wide (multi-user)

Experimental features (`nix-command`, `flakes`) must be visible to the Nix that actually runs.

Disk and trust

Leave **tens of GB** free. The store grows with every experiment. Trust binary caches only when signatures match configured public keys—random HTTP mirrors are not enough.

Theory 6 — Flakes as the modern pinning model (preview)

You enable flakes in this chapter; deep flake authoring is the **Flakes and project devShells** chapter.

Concept	Meaning
<code>flake.nix</code>	Declarative inputs + outputs
<code>flake.lock</code>	Pinned revisions/hashes of inputs
<code>nixpkgs#hello</code>	Indirect flake reference to package <code>hello</code>

Contrast with channels

Channels	Flakes
Mutable “what is <code>nixpkgs</code> today?”	Lock file pins <i>exactly</i>
<code>NIX_PATH</code> / <code><nixpkgs></code>	Explicit inputs
Easy to drift	Designed for collaboration

This book: **flakes default**, channels as literacy only (classic CLI chapter).

Baseline pin you will reuse

```
inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
```

Theory 7 — CLI eras (so docs don't gaslight you)

Era	Examples	Status in this book
Classic	<code>nix-env</code> , <code>nix-build</code> , <code>nix-shell</code>	Literacy later (classic CLI chapter)
Modern	<code>nix build</code> , <code>nix run</code> , <code>nix develop</code> , <code>nix flake</code>	Default

Same underlying store; different UX and defaults. Prefer modern unless you are translating an old post.

Theory 8 — What Nix is *not*

Myth	Reality
“Nix replaces backups”	State (<code>/var</code> , DBs) needs its own policy
“Nix encrypts secrets”	Store is often world-readable; secrets need tools later
“One hash forever everywhere”	OS/CPU/impure inputs still matter
“No ops needed”	You still design networks, access, upgrades

Honesty keeps adoption durable.

Worked examples

Example A — “Install” without polluting a global profile

```
nix run nixpkgs#hello
```

Runs `hello` from a realized store path without committing to a permanent user profile.

Example B — Why two machines diverge without locks

```
Machine A: nixpkgs as of Monday
Machine B: nixpkgs as of Friday
→ different hashes for "hello", different closures
```

Flake locks exist to end that story.

Example C — What Nix does *not* do

```
Declarative Postgres service    automatic migration of your prod data
Encrypted secrets in git        "Nix encrypts the store"
```

Example D — Eval error vs build error (feel the phases)

```
# Eval failure (missing attribute) - no build starts
nix eval --expr 'let pkgs = {}; in pkgs.doesNotExist'

# Realization path for a known package (may download)
nix build nixpkgs#hello --print-out-paths
```

Example E — Minimal REPL tour

```
nix repl

1 + 1
{ a = 1; b = "x"; }.a
builtins.typeOf { x = 1; }
builtins.typeOf (x: x + 1)
```

Example F — Inspect config after enabling features

```
nix config show | grep -i experimental
nix --version
```

Lab 1 — Install Nix

Use current docs: <https://nixos.org/download/>
Or Determine installer if you choose—**write which**.

```
# new shell after install
nix --version
```

Disk: leave tens of GB free for later chapters and parts.

Record in notes:

- Installer name/version
 - Multi-user or single-user
 - OS and CPU (`uname -m`)
-

Lab 2 — Enable modern features

`~/.config/nix/nix.conf` (or `system nix.conf` if that is what your install uses):

```
experimental-features = nix-command flakes
```

Verify:

```
nix config show | grep -i experimental
nix flake --help | head
```

If settings do not apply, check whether the daemon needs a restart and whether user vs system config is the one being read.

Lab 3 — First modern commands

```
nix search nixpkgs hello | head
nix run nixpkgs#hello
```

REPL

```
nix repl
```

```
1 + 1
{ a = 1; b = "x"; }.a
builtins.typeOf { x = 1; }
```

Load `nixpkgs` when available (`:lf nixpkgs` or version-appropriate equivalent) and inspect `pkgs.hello`.

Lab 4 — Write the model

In your journal, answer without looking up:

1. Language vs PM vs OS—one sentence each
 2. Why two `libfoo` versions can coexist
 3. Eval failure vs build failure—one example each
 4. Which installer you used
-

Exercises

1. Draw the three faces of Nix from memory; check against the diagram.
 2. Name three traditional failure modes Nix targets and one it does not.
 3. Run `nix run nixpkgs#hello` twice; note whether the second is faster (cache).
 4. Force an eval error and a realize-related error; label each.
 5. Locate which `nix.conf` your install actually reads; quote the path.
 6. Enable flakes only in user config vs system config—document which worked.
 7. Compare `nix --version` with a teammate; note drift risk without locks.
 8. Write a 5-line “why Nix” pitch for a skeptic who loves apt.
 9. List five blog phrases that mix language/PM/OS; rewrite them precisely.
 10. Create `~/lab/nixos-book/concepts/why-nix/NOTES.md` with installer + baseline.
 11. Run `nix repl` and evaluate nested attrsets until muscle memory sticks.
 12. Explain input-addressed hashing in one paragraph without the word “magic.”
-

Common pitfalls

Symptom	Theory link
<code>nix</code> not found	Install/PATH; shell not reloaded
experimental feature errors	<code>nix.conf</code> not read by that Nix
Slow first command	Cold substituter / eval of <code>nixpkgs</code>
Permission errors	Single vs multi-user mismatch
Old blog uses only <code>nix-env -i</code>	Classic era; map later
“Nix broke my OS” after PM install	You installed PM, not NixOS—different surface
Expecting data reproducibility	Wrong layer of the honesty ladder
Mixing three installers	Record and stick to one story

Checkpoint

- ☐ Can explain three faces of Nix
- ☐ Can state two problems Nix targets and two it does not
- ☐ `nix run nixpkgs#hello` works
- ☐ Flakes + `nix-command` enabled
- ☐ REPL evaluated an `attrset`
- ☐ Installer name recorded
- ☐ Eval vs realize distinction written in own words

Further depth (this book)

Topic	Chapter / part
Store paths, closures, GC roots	The Nix store and closures
Language data model	Nix language: values and attrsets
Flake authoring	Flakes and project devShells
Classic CLI translation	Classic CLI literacy
Full OS as modules	Part NixOS host
Content-addressed derivations	Part Internals

The Nix store and closures

The Nix store and closures

Goal: Read `/nix/store` paths as a graph database of software, define **closures** precisely, and explain **GC roots**—with labs that only *observe* (no aggressive GC yet).

Why this chapter matters

Every advanced Nix topic—binary caches, deploy, Docker-from-Nix, NixOS activations—is a story about **store paths and references**. Without that theory, later tools feel like superstition.

Ops motivation: disk fills, deploys miss a library, “I deleted the project but used 40 GB”—all are root/closure mistakes. If you can *see* the graph, you can operate it.

Theory 1 — The store is the database

The directory `/nix/store` is not a random cache folder. It is the **authoritative object store** for:

- Package outputs
- Derivation files (`.drv`)
- Fixed-output download results
- Sometimes source mirrors

Invariants (practical)

Invariant	Meaning
Paths are immutable	You do not <code>vim</code> a store path to “fix prod”
Names include hashes	Collision-resistant identity
References are explicit	Runtime deps encoded as string refs inside outputs
Mutation is via new paths	“Upgrade” means new hash, not overwrite

Hand-editing the store voids the model (and often permissions prevent it).

What lives beside the store

Path (typical)	Role
<code>/nix/var/nix/db</code>	Nix database (references, validity)
<code>/nix/var/nix/profiles</code>	Profile generations (roots)
<code>/nix/var/nix/gcroots</code>	Explicit GC roots
<code>/nix/var/nix/daemon-socket</code>	Multi-user daemon

You rarely edit these by hand; you query them.

Theory 2 — Path anatomy



Figure 1: Store path anatomy

```
/nix/store/<hash>-<name>-<version>
```

```
convenience label (not trust)
cryptographic fingerprint of inputs (typical case)
global namespace of this Nix installation
```

Examples (illustrative shapes)

```
/nix/store/abc...-hello-2.12.1
/nix/store/def...-glibc-2.40-36
/nix/store/ghi...-bash-5.2-p37
```

Theory punchline: two paths with the same pretty name and different hashes are **different objects**. Tools that only show names are lying by omission.

Input-addressed vs content-addressed (awareness)

Model	Hash depends on	Note
Input-addressed (classic default)	Build inputs / derivation	Rebuild if inputs change
Content-addressed (experimental/advanced)	Output content	Same bytes → same hash potential

You will live mostly in input-addressed land until the **Internals** part.

Name segment is not a version pin

The `-2.12.1` label is human hinting. Trust the **hash**. Two `hello-2.12.1` builds from different nixpkgs revisions are different paths.

Theory 3 — References and the closure

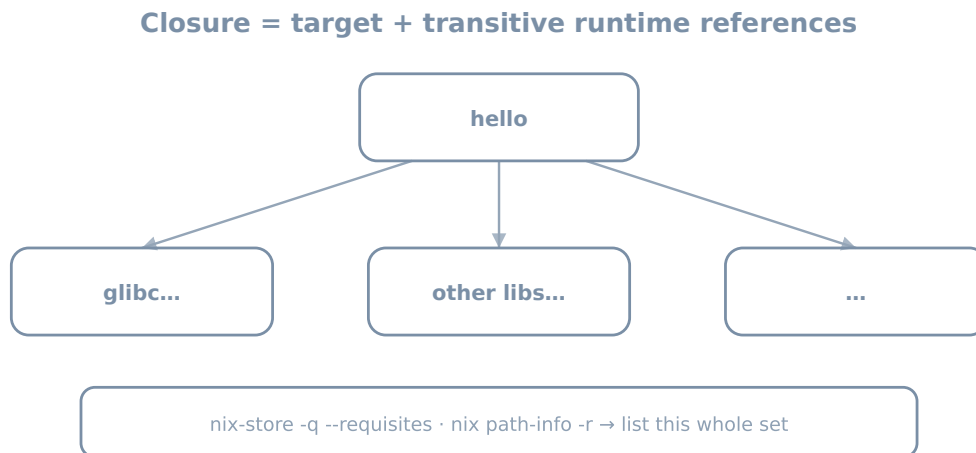


Figure 2: Closure as a reference graph

Direct references

A store path's **references** are other store paths it mentions (dynamic linker paths, wrapped scripts, metadata). Query:

```
nix-store -q --references "$P"
```

Closure (requisites)

The **closure** of a path **P** is the smallest set **S** such that:

1. **P** is in **S**
2. If **x** is in **S** and **x** references **y**, then **y** is in **S**

Operationally:

```
nix-store -q --requisites "$P"  
nix path-info -r "$P"
```

Why closures matter — worked scenarios

Scenario	Why closure is the unit
Copy software to another host	Need all runtime deps
Binary cache upload	Serve complete graphs
Disk usage of “hello”	Sum of requisites, not one path
Container image from Nix	Pack a closure
NixOS system activation	Whole system is a giant closure

Example narrative

```
hello  
  glibc  
  ...  
  (other runtime inputs)
```

hello is a thin node; **glibc** often dominates size. That is normal.

Closure size theory

```
nix path-info -S "$P"      # approx size of one path  
nix path-info -rS "$P"    # recursive / closure-oriented sizing
```

Compare single vs recursive. The gap is the theory lesson.

Referrers (reverse edges)

```
nix-store -q --referrers "$P"
```

Useful when asking: “who still holds this library?”

Theory 4 — Evaluation vs realization (store-centric)

Recall the Why Nix exists chapter:

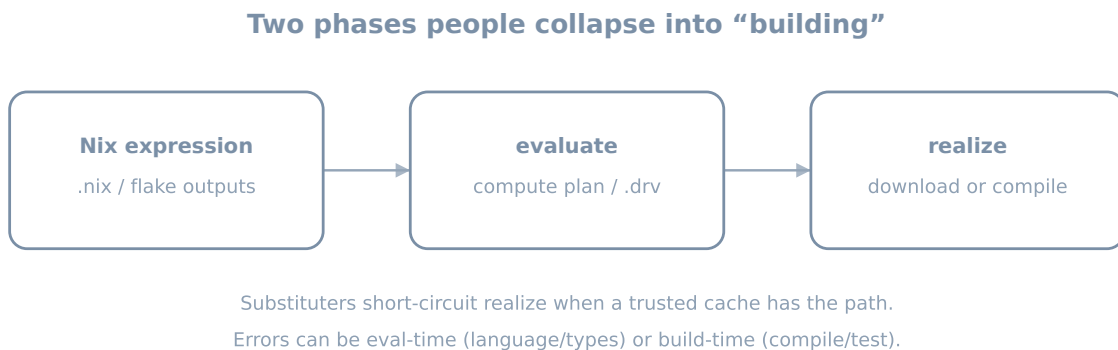


Figure 3: Evaluate then realize

Artifact	Typical location / form
Expression	.nix / flake
Derivation	.drv in store (build plan)
Output path	realized directory in store

Example commands

```
# realize outputs (modern)
nix build nixpkgs#hello --print-out-paths

# classic query tools still educational
nix-store -q --tree "$HELLO" | head
```

You can evaluate without realizing in some workflows; beginners usually realize first because they want a runnable binary.

Theory 5 — Profiles, generations, and roots

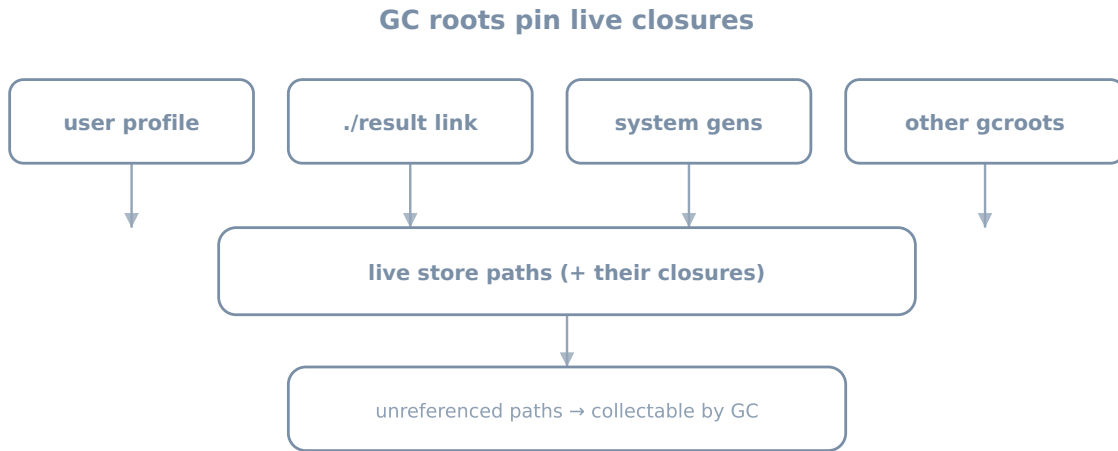


Figure 4: GC roots keep paths live

Why GC exists

Every experiment can add paths. Without collection, disks fill. Collection must not delete paths still **needed**.

Root types (non-exhaustive)

Root kind	Example
User profile generations	~/.nix-profile, state profiles
Result symlinks	./result from nix build
NixOS system profiles	/nix/var/nix/profiles/system* (host part)
Indirect roots under gcroots	Registered links
Running processes	Open files can keep paths live temporarily

The ./result trap (critical theory)

```
nix build nixpkgs#hello
# creates ./result -> /nix/store/...-hello-...
```

That symlink is a **GC root**. Leave hundreds of **result** links in temp dirs → permanent pins → “mystery” disk use.

Discipline: `rm result` when finished experimenting, or use `--no-link`.

Generations theory (preview)

Profiles are versioned. Rolling back a profile generation re-points the user environment. Deleting old generations **makes previously live closures collectable** if nothing else roots them.

! Important

Do **not** run `nix-collect-garbage -d` on a machine you care about until you understand generations. Observation only in this chapter.

Theory 6 — Substituters, signatures, trust (preview)

When realizing path **P**:

1. Ask configured **substituters** (e.g. `cache.nixos.org`)
2. Verify **signatures** against trusted keys
3. Else build locally

Trust model punchline: a random HTTP server offering store paths is not enough; signatures matter. Self-hosted caches (later packaging/ops chapters) require key management.

```
nix config show | grep -i substituter || true
nix path-info --json nixpkgs#hello 2>/dev/null | head -c 400
```

Theory 7 — What “uninstall” means

Action	Effect
Remove package from profile	Drop root edge; paths may remain
Delete project directory	Does not delete store
<code>rm result</code>	Drop one root
GC	Delete unreferenced paths

Example misconception

```
"I deleted ~/code/foo, so Nix freed 2GB"
→ False unless GC ran and nothing else rooted those paths
```

Operational uninstall checklist

1. Remove declarative/profile references
 2. Remove `result` and other roots
 3. Optionally drop old generations
 4. Run GC deliberately
-

Theory 8 — Copying closures (ops preview)

```
# conceptual modern/classic tools
nix copy --to ssh://host "$P"
# or nix-copy-closure in classic literacy
```

Deploy systems (later parts) are sophisticated closure movers. The unit of transfer is still the `closure`.

Worked examples bank

Example A — Build and print path

```
HELLO=$(nix build nixpkgs#hello --print-out-paths --no-link)
echo "$HELLO"
"$HELLO/bin/hello"
```

Example B — Direct vs full graph

```
nix-store -q --references "$HELLO"
nix-store -q --requisites "$HELLO" | wc -l
```

Expect `requisites count` `references count`.

Example C — Tree view

```
nix-store -q --tree "$HELLO" | head -60
```

Example D — Result root lifecycle

```
cd /tmp
rm -f result
nix build nixpkgs#hello
readlink result
# inspect roots (command variants exist)
nix-store --gc --print-roots 2>/dev/null | grep -i result || true
rm result
```

Example E — Path info metadata

```
nix path-info -sh "$HELLO"
nix path-info -rS "$HELLO" | tail
```

Example F — Compare two packages

```
H=$(nix build nixpkgs#hello --print-out-paths --no-link)
G=$(nix build nixpkgs#git --print-out-paths --no-link)
echo "hello reqs: $(nix-store -q --requisites "$H" | wc -l)"
echo "git reqs:   $(nix-store -q --requisites "$G" | wc -l)"
```

Lab 1 — Realize hello and dissect the path

```
mkdir -p ~/lab/nixos-book/concepts/store
cd ~/lab/nixos-book/concepts/store
HELLO=$(nix build nixpkgs#hello --print-out-paths --no-link)
echo "$HELLO"
ls -la "$HELLO"
ls -la "$HELLO/bin"
```

Journal: full path, name segment, hash length (approx).

Lab 2 — Closure measurements

```
echo "direct refs: $(nix-store -q --references "$HELLO" | wc -l)"
echo "requisites: $(nix-store -q --requisites "$HELLO" | wc -l)"
nix path-info -S "$HELLO"
nix path-info -rS "$HELLO" | tail -5
```

Write one sentence: *What dominates the closure?*

Lab 3 — Follow one dependency

```
nix-store -q --references "$HELLO"
# pick a non-hello path, ls it
```

Observe: another complete package prefix, not a classic `/usr` layout.

Lab 4 — Roots observation

```
ls -la /nix/var/nix/gcroots 2>/dev/null | head
ls -la ~/.nix-profile 2>/dev/null
nix profile list 2>/dev/null || true
```

Perform Example D (result create/remove). Note difference.

Lab 5 — Teach-back

Without notes, define:

1. Store path
2. Reference
3. Closure
4. GC root

5. Why `rm -rf project` free disk
-

Exercises

1. Parse five real store paths into hash + name; ignore the pretty version for trust decisions.
 2. Compute requisites count for `hello` and one larger package (`git` or `python3`).
 3. Find the single largest path in `hello`'s closure via `path-info`.
 4. Create three `result` links in different temp dirs; list them via `print-roots`; clean up.
 5. Explain why binary cache upload granularity is closure-shaped.
 6. Diagram references vs referrers for a tiny graph (3 nodes).
 7. Measure disk before/after realizing a package **without** GC (observe growth).
 8. Document every GC root kind you can observe on your machine.
 9. Use `nix-store -q --tree` and annotate three levels of the tree.
 10. Write a one-page “store operator card” for future-you.
 11. Compare `--no-link` vs default `result` workflow for lab hygiene.
 12. Predict whether deleting `~/.cache` frees store space (then verify: it does not).
-

Common pitfalls

Symptom	Theory
Disk full after demos	Roots + caches + no GC
<code>result</code> broken link	GC collected after root removed
“Uninstalled but still huge”	Roots remain; GC not run
Different hash on friend machine	Different nixpkgs revision
Permission denied writing store	Correct — immutability
Trusting package <i>name</i> only	Hash is identity
Blind <code>nix-collect-garbage -d</code>	Drops generations/roots you may need

Symptom	Theory
Thinking <code>/nix/store</code> is a cache you can wipe casually	It is the object database

Checkpoint

- ☐ Parse a store path into hash + name
- ☐ Define closure as a transitive reference set
- ☐ Show requisites count for `hello`
- ☐ Explain `./result` as a GC root
- ☐ List 2 root kinds
- ☐ Agree not to blind `-d` GC yet
- ☐ Single-path vs closure size surprise written down

Journal prompts

1. Closure vs single-path size surprise
2. Where *you* will accidentally create roots
3. One deploy implication of closures

Further depth (this book)

Topic	Chapter / part
Derivations producing paths	Derivations as an idea
Modern queries (<code>path-info</code>)	Modern Nix CLI
System generations as roots	Host: Rebuild and generations
GC hygiene in ops	Part Ops and fleet
Store daemon internals	Part Internals

Nix language: values and attrsets

Nix language: values and attrsets

Goal: Read and write Nix **values**, **attrsets**, **let...in**, and **inherit** well enough to model config-shaped data and follow nixpkgs snippets.

i Note

This chapter is language only—no flakes, no NixOS rebuilds. Everything runs in `nix repl` or `nix eval`.

Why this chapter matters

NixOS configuration *is* nested attribute sets with types and merge rules layered on top. If attrsets feel alien, every later chapter becomes copy-paste. Master the data model once; modules will make sense.

Ops motivation: reading an error about a missing attribute, writing a host model, and reviewing a teammate’s module all require fluent set/list/string literacy—not memorizing every option.

Theory 1 — Values are few; composition is everything

Nix has a small value inventory:

Kind	Examples	Notes
Integer	1, -3	Arbitrary precision integers
Float	1.5	Prefer integers unless you need floats
Boolean	true, false	
String	"hello", ''multi''	Two quoting styles
Path	./foo.nix, /etc/hosts	Distinct from strings
Null	null	Often “unset” in APIs
List	[1 2 3]	Heterogeneous allowed; spaces separate elements
Attrset	{ a = 1; b = "x"; }	The backbone
Function	x: x + 1	the language functions chapter

Type inspection

```
builtins.typeOf 1          # "int"
builtins.typeOf "x"        # "string"
builtins.typeOf ./foo      # "path"
builtins.typeOf { a = 1; } # "set"
builtins.typeOf [ 1 2 ]    # "list"
builtins.typeOf (x: x)     # "lambda"
```

Punchline: almost all “config” is **attrsets of attrsets**, with lists of packages/strings hanging off leaves.

Theory 2 — Strings: interpolation and two quote forms

Double quotes

```
let name = "yarara"; in "NixOS ${name}"
# → "NixOS yarara"
```

`${...}` interpolates an expression that stringifies cleanly (strings, paths, numbers, ...).

Indented strings (`'' ... ''`)

```
''
  line one
  line two
''
```

Common for shell scripts and multi-line text. Leading indentation is stripped relative to the least-indented line.

Escaping notes

Need	Approach
Literal <code>\${</code> in <code>''</code> strings	<code>''\${</code>
Nested quotes	Prefer indented strings for scripts
Path into string	<code>"\${./file}"</code> copies/context rules apply carefully

Paths vs path-looking strings

```
./config.nix    # path - can be imported, copied into store
"./config.nix"  # string - just characters
```

Import expects a **path** (or something that resolves to one). Confusing the two is a classic beginner bug.

Theory 3 — Lists

```
[ "git" "vim" "curl" ]
[ 1 2 (3 + 4) ]
```

- No commas between elements
- Parenthesize subexpressions when needed: [(f x) y]
- Concatenate with ++: [1 2] ++ [3]

Lists are **ordered**. Attrsets are **keyed**. Use the right shape:

Need	Prefer
Ordered packages on PATH	list
Named options / config tree	attrset
Set of users by name	attrset of attrsets

List builtins you will reuse

```
builtins.length [ 1 2 3 ]
builtins.head [ 1 2 3 ]
builtins.tail [ 1 2 3 ]
builtins.elemAt [ 10 20 30 ] 1
builtins.concatLists [ [ 1 ] [ 2 3 ] ]
builtins.filter (x: x > 1) [ 0 1 2 ]
builtins.map (x: x * 2) [ 1 2 3 ]
```

Theory 4 — Attribute sets (the main event)

```
{
  hostName = "lab";
  enable = true;
  ports = [ 22 80 ];
}
```

Access

```
let cfg = { hostName = "lab"; enable = true; };
in cfg.hostName
# or
cfg.${"hostName"}
```

Nested sets

```
{
  networking = {
    hostName = "lab";
    firewall = {
      enable = true;
      allowedTCPPorts = [ 22 ];
    };
  };
}
```

This is the *shape* of NixOS config, even before the module system:

```
config
  networking
    hostName
  firewall
    enable
    allowedTCPPorts
```

Dynamic keys

```
let key = "hostName";
in { ${key} = "lab"; }
```

Quoted keys for unusual names: { "my-key" = 1; }.

Update / merge intuition

```
# shallow "override" via //  
{ a = 1; b = 2; } // { b = 9; c = 3; }  
# → { a = 1; b = 9; c = 3; }
```

// is **shallow**. Nested sets are replaced wholesale unless you merge carefully (later: `lib.recursiveUpdate`, module `merge`).

```
{ n = { x = 1; }; } // { n = { y = 2; }; }  
# → { n = { y = 2; }; } # x is gone!
```

Attrset builtins

```
builtins.attrNames { z = 1; a = 2; }  
builtins.attrValues { a = 10; b = 20; }  
builtins.hasAttr "a" { a = 1; }  
builtins.getAttr "a" { a = 1; }  
builtins.removeAttrs { a = 1; b = 2; } [ "a" ]
```

Theory 5 — `let ... in` (local bindings)

```
let  
  a = 1;  
  b = a + 2;  
in  
  a + b  
# → 4
```

Rules that matter

1. Bindings in one `let` can refer to each other (order-independent among pure values).
2. Scope ends at the `in` expression.
3. Prefer `let` over repeating huge expressions.
4. Infinite recursion is possible if a binding depends on itself without a fixed point.

```
let x = x + 1; in x # boom: infinite recursion
```

Nested lets vs one flat let

Either works; flat is often clearer for small configs:

```
let
  host = "lab";
  user = "alice";
  packages = [ "git" "vim" ];
in {
  networking.hostName = host;
  users.users.${user}.isNormalUser = true;
  environment.systemPackages = packages; # shape only - not real pkgs yet
}
```

rec attrsets

```
rec {
  a = 1;
  b = a + 1; # refers to a in the same set
}
```

Useful; easy to create cycles. Prefer let when clarity wins.

Theory 6 — inherit (pull names into sets or lets)

Into an attrset

```
let
  pkgs = { hello = "h"; git = "g"; };
  system = "x86_64-linux";
in {
  inherit pkgs system;
  # equivalent to: pkgs = pkgs; system = system;
}
```

Rename form

```
let hello = "world"; in { inherit ( { x = hello; } ) x; }
```

Common real form:

```
{ inherit (pkgs) git vim curl; }
# → { git = pkgs.git; vim = pkgs.vim; curl = pkgs.curl; }
```

Into let

```
let
  inherit (builtins) typeOf attrNames;
in
  typeOf { a = 1; }
```

When to inherit: the name in the outer scope *is* the name you want. When names differ, write the binding explicitly.

Theory 7 — with (know it; use sparingly)

```
with builtins; typeOf [ 1 2 ]
```

with `set`; `expr` injects `set`'s attributes into scope. Powerful and **easy to shadow silently**.

```
let
  lib = { id = x: x; };
  id = "oops";
in
  with lib; id    # which id?
```

House rule for this book: prefer `let inherit (lib) ...` or qualified `lib.foo` over broad `with lib`; . You will still *read* with in older code.

Theory 8 — Conditionals and operators (minimal set)

```
if true then "a" else "b"

true && false
true || false
!true

1 == 1
1 != 2
```

```
# ordering on numbers/strings exists; avoid over-cleverness
```

`assert cond`; value fails evaluation if `cond` is false—useful later for module assertions; try once in this chapter:

```
let x = 2; in assert x > 0; x
```

Theory 9 — Config-shaped modeling (bridge to NixOS)

You are not writing a full module yet. You are practicing the **data shape**:

```
let
  hostName = "lab-vm";
  admin = "alice";
  sshPort = 22;
in {
  networking = {
    inherit hostName;
    firewall = {
      enable = true;
      allowedTCPPorts = [ sshPort ];
    };
  };
  users.users.${admin} = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];
  };
  environment.systemPackages = [
    "git"    # stand-ins; real pkgs are derivations later
    "vim"
  ];
}
```

Notice:

- Nested attrsets mirror option paths
 - `${admin}` dynamic key for username
 - `inherit hostName` avoids repetition
 - Lists for multi-valued options
-

Theory 10 — Destructuring patterns you will see

```
let
  set = { a = 1; b = 2; };
  inherit (set) a b;
in
  a + b
```

```
# nested access sugar people write in modules
config.networking.hostName
```

Reading real nixpkgs becomes pattern matching on these shapes—not learning a second language.

Worked examples bank

Example A — REPL tour

```
nix repl

1 + 1
{ a = 1; b = { c = 2; }; }.b.c
let x = "nix"; in "hello ${x}"
[ 1 2 ] ++ [ 3 ]
{ a = 1; } // { b = 2; }
builtins.attrNames { z = 1; a = 2; }
```

Example B — Deep access and missing attrs

```
let
  cfg = { networking.firewall.enable = true; };
in
  cfg.networking.firewall.enable
```

Missing attribute → evaluation error. That is good: fail early.

Example C — inherit package list pattern

```
let
  tools = {
    git = "git-placeholder";
    vim = "vim-placeholder";
    curl = "curl-placeholder";
  };
in {
  environment.systemPackages = with tools; [ git vim curl ];
  # or better without with:
  # environment.systemPackages = [ tools.git tools.vim tools.curl ];
}
```

Example D — Shallow merge trap

```
let
  base = { services.foo = { enable = true; port = 80; }; };
  overlay = { services.foo = { port = 8080; }; };
in
  base // overlay
# services.foo.enable disappeared - shallow //
```

Journal this; module system merges differently later (host modules chapters).

Example E — Dynamic user keys

```
let
  mkUser = name: {
    users.users.${name} = {
      isNormalUser = true;
    };
  };
in
  mkUser "alice"
```

(Functions deepen in the next chapter; the attrset shape is the point here.)

Lab 1 — Workspace and file evaluation

```
mkdir -p ~/lab/nixos-book/concepts/attrsets
cd ~/lab/nixos-book/concepts/attrsets
```

Create `host-model.nix`:

```
let
  hostName = "lab";
  adminUser = "alice";
  packages = [ "git" "vim" "htop" ];
in {
  networking = {
    inherit hostName;
    firewall.enable = true;
    firewall.allowedTCPPorts = [ 22 ];
  };
  users.users.${adminUser} = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];
  };
  environment.systemPackages = packages;
}
```

Evaluate:

```
nix eval --file ./host-model.nix
nix eval --file ./host-model.nix --apply 'cfg: cfg.networking.hostName'
```

Lab 2 — Nested merge experiments

In `merge-trap.nix`, reproduce Example D. Also try nested manual merge:

```
let
  base = { services.foo = { enable = true; port = 80; }; };
  overlay = { services.foo = { port = 8080; }; };
  deep =
    base
    // {
      services = base.services // {
        foo = base.services.foo // overlay.services.foo;
      };
    };
};
```

```
in
  deep
```

Confirm `enable` survives. Write one sentence on why `//` alone is not “config merge.”

Lab 3 — Attrset utilities in the REPL

```
builtins.attrNames { c = 1; a = 2; b = 3; }
builtins.attrValues { a = 10; b = 20; }
builtins.hasAttr "a" { a = 1; }
builtins.getAttr "a" { a = 1; }
```

Build a tiny “user directory”:

```
let
  users = {
    alice = { uid = 1000; shell = "bash"; };
    bob = { uid = 1001; shell = "zsh"; };
  };
in {
  names = builtins.attrNames users;
  aliceShell = users.alice.shell;
}
```

Lab 4 — Model something real from your life

Expand `host-model.nix` (or a second file) to include *fictional but realistic* keys for:

1. Hostname + domain
2. Two users (one admin)
3. Three “packages” as strings
4. Firewall ports for SSH + one app
5. A `services.myapp = { enable = true; ... }` nest

Use `let` + `inherit` at least twice. No need for real NixOS options names to validate—accuracy of *shape* matters.

Lab 5 — Teach-back

Without looking up, write:

1. Difference between `path` and `string`
 2. Why lists use spaces not commas
 3. What `//` does to nested sets
 4. When you would use `inherit`
 5. Why broad `with` is risky
-

Exercises

1. Write attrsets for: web service config, user map, package list—choose list vs set deliberately.
2. Convert a JSON-like mental model of your laptop hostname/firewall into Nix.
3. Break evaluation with a missing attribute; fix with a default in `let`.
4. Demonstrate `rec` vs `let` for the same binding graph.
5. Use indented strings to draft a tiny shell script body (string only).
6. Build a deep nest 4 levels deep; access the leaf two ways.
7. Implement shallow vs manual deep merge; document the difference.
8. Prefer `inherit (builtins) ...` in a file and evaluate it.
9. Find three places `with` would hurt readability; rewrite without it.
10. Model two hosts in one attrset keyed by hostname.
11. Use `assert` to enforce `sshPort > 0`.
12. Read a random small snippet from `nixpkgs` (online) and label each value kind.

13. Explain why NixOS options look like nested attrsets even though types exist.
14. Commit your `~/lab/nixos-book/concepts/attrsets` experiments.

Common pitfalls

Symptom	Theory
<code>error: undefined variable</code>	Scope / missing <code>let</code> binding
<code>unexpected ','</code> in list	No commas in Nix lists
Path vs string import confusion	Paths are a distinct type
Nested keys “disappeared” after <code>//</code>	Shallow merge
<code>with</code> silently shadows	Prefer explicit / <code>inherit</code>
Dynamic key without <code>\${}</code>	Use <code>{ \${name} = ...; }</code>
Infinite recursion in <code>let</code>	Self-reference without base
Semicolon missing in attrset	Every binding ends with <code>;</code>
Treating lists as sets	Wrong shape for named config

Checkpoint

- ☐ Named all major value kinds
 - ☐ Wrote nested attrset for host-shaped config
 - ☐ Used `let...in` and `inherit` deliberately
 - ☐ Demonstrated shallow `//` pitfall
 - ☐ Evaluated a `.nix` file with `nix eval`
 - ☐ Personal notes include 3 gotchas
 - ☐ Can choose list vs attrset for a new field
-

Further depth (this book)

Topic	Chapter
Functions over these values	Nix language: functions
Multi-file + lib helpers	Laziness, imports, and lib
Module merge (not //)	Host: Modules as a consumer
Real host config trees	Host: Flake-defined hosts

Nix language: functions

Nix language: functions

Goal: Write and call Nix functions fluently—especially **attribute-set pattern arguments**, defaults, @-binding, and ellipsis—so flake and module snippets stop looking like magic.

Why this chapter matters

In Nix, packages, shells, overlays, and modules are **functions returning values**. If you only know `x: x + 1`, you cannot read `nixpkgs`. This chapter is the real functional core.

Ops motivation: every flake `outputs = { self, nixpkgs, ... }: ...` and every module `{ config, lib, pkgs, ... }: ...` is the same skill. Fluency here turns “paste and pray” into “read and adapt.”

Theory 1 — Everything callable is a lambda

```
let
  inc = x: x + 1;
  add = x: y: x + y;    # curried
in
  add 2 3              # 5
```

Application is juxtaposition

```
f x      # call f with x
f x y     # (f x) y if curried
f { a = 1; } # call with one attrset arg
```

Parenthesize when needed: `(f x) + 1`, `map (x: x + 1) list`.

Functions are values

```
let
  ops = {
    inc = x: x + 1;
    double = x: x * 2;
  };
in
  ops.double (ops.inc 3) # 8
```

Identity and composition (lightweight)

```
let
  id = x: x;
  compose = f: g: x: f (g x);
in
  compose (x: x + 1) (x: x * 2) 3 # 7
```

Theory 2 — Attrset pattern arguments (the Nix idiom)

Almost every public Nix API looks like:

```
{ a, b }: a + b
```

Call site:

```
({ a, b }: a + b) { a = 1; b = 2; }
```

Destructuring is the point

```
{ hostname, user }: {
  networking.hostname = hostname;
  users.users.${user}.isNormalUser = true;
}
```

You **name** the fields you need. Extra fields are an error unless you allow them (ellipsis—below).

Why not positional for public APIs?

Positional	Attr pattern
Order fragile	Names document call sites
Hard to default	? defaults natural
Hard to extend	Ellipsis / optional fields

Theory 3 — Default arguments with ?

```
{ force ? false, retries ? 3 }: {
  inherit force retries;
}
```

```
f { }                # force=false, retries=3
f { force = true; }   # force=true, retries=3
f { retries = 10; }   # force=false, retries=10
```

Order of attributes in the pattern does not matter at the call site; names bind by key.

Defaults can be expressions

```
{ pkgs, system ? builtins.currentSystem }:
  # system falls back if omitted
  { inherit pkgs system; }
```

Be careful: defaults that force heavy evaluation run when the binding is **used** (laziness—the next chapter), but bad defaults can still surprise you.

Explicit null vs missing key

```
# if you pass force = null, the default is NOT used
# defaults apply only when the attribute is absent
```

Theory 4 — Ellipsis ... (allow unknown attributes)

```
{ a, b, ... }: a + b
```

Call with extras:

```
({ a, b, ... }: a + b) { a = 1; b = 2; c = 99; } # 3 - c ignored
```

Without ...:

```
({ a, b }: a + b) { a = 1; b = 2; c = 99; }  
# error: unsupported argument 'c'
```

When to use ellipsis

Situation	Ellipsis?
Strict public API	Often no —catch typos
Pass-through / forward kwargs	Yes
Module-style “rest of config”	Sometimes

House rule: use ... when forwarding; avoid it when you want misspelled option names to fail loudly.

Theory 5 — @ binding (name the whole set)

```
args@{ a, b, ... }: {  
  inherit a b;  
  allKeys = builtins.attrNames args;  
}
```

Or:

```
{ a, b, ... }@args: ... # same idea, different sugar position
```

Why @ exists

1. Forward the full attrset to another function
2. Inspect extras
3. Merge overrides

```
let
  f = { a, b ? 0, ... }@args:
    g (args // { b = b + 1; });
in
  f
```

Classic flake-shaped pattern:

```
outputs = inputs@{ self, nixpkgs, ... }:
  let
    system = "x86_64-linux";
    pkgs = import nixpkgs { inherit system; };
  in {
    packages.${system}.default = pkgs.hello;
  };
```

You will write this shape in the flakes chapter; understand the `inputs@{ ... }` part **now**.

Theory 6 — Composition and higher-order helpers

map / filter (builtins)

```
map (x: x * 2) [ 1 2 3 ]
# [ 2 4 6 ]

builtins.filter (x: x > 1) [ 0 1 2 3 ]
# [ 2 3 ]
```

Function returning function

```
let
  multiply = n: x: n * x;
  triple = multiply 3;
in
```

```
triple 4 # 12
```

enableIf-style helper (preview of module style)

```
let
  enableIf = cond: value: if cond then value else { };
in
  enableIf true { services.foo.enable = true; }
```

Real modules use `lib.mkIf`—same *idea*, better merge semantics (host modules chapters).

Theory 7 — Fixed-point awareness (not mastery)

You will see:

```
lib.fixpoint (self: {
  a = 1;
  b = self.a + 1;
})
```

Idea: define an attrset in terms of its final form. Overlays and package sets use this heavily. In this chapter: recognize it; do not invent fixed points for homework.

Informal:

```
self → { a = 1; b = self.a + 1; } → { a = 1; b = 2; }
```

Theory 8 — Call patterns you will see in the wild

1. Package “callPackage” shape

```
{ lib, stdenv, fetchurl }:
stdenv.mkDerivation { ... }
```

Someone injects dependencies by calling the function with an attrset of packages.

2. Overlay shape

```
final: prev: {  
  myPkg = prev.hello.overrideAttrs (old: { ... });  
}
```

Two arguments, both attrsets of packages (final = after overlays, prev = before this overlay).

3. Module shape (preview)

```
{ config, lib, pkgs, ... }: {  
  options = { ... };  
  config = { ... };  
}
```

Ellipsis is standard—modules receive many arguments.

4. override / overrideAttrs as functions

Packages often expose functions that take functions:

```
pkgs.hello.overrideAttrs (old: {  
  pname = old.pname;  
})
```

You are passing a function that receives old attrs—same language skill.

Theory 9 — Error messages as teachers

Error fragment	Likely cause
called without required argument 'x'	Missing key in attr call
unsupported argument 'y'	Extra key; need ... or remove
cannot coerce a function to ...	Forgot to call f / f args
value is a function while a set was expected	Passed lambda where attrset needed
Infinite recursion	Self application without base

Worked examples bank

Example A — Defaults + ellipsis

```
let
  mkService = { name, port ? 8080, enable ? true, ... }: {
    inherit name port enable;
  };
in
  mkService { name = "api"; host = "ignored-extra"; }
```

Example B — @ forward

```
let
  core = { a, b }: a + b;
  wrap = args@{ a, b ? 0, ... }:
    core { inherit a; b = b + 1; };
in
  wrap { a = 10; b = 1; extra = true; }
# → 12
```

Example C — User skeleton helper

```
let
  mkUser = {
    name,
    groups ? [ "wheel" ],
    keys ? [ ],
  }: {
    users.users.${name} = {
      isNormalUser = true;
      extraGroups = groups;
      openssh.authorizedKeys.keys = keys;
    };
  };
in
  mkUser {
    name = "alice";
    keys = [ "ssh-ed25519 AAAA... alice@lab" ];
  }
```

Example D — Package list merge helper

```
let
  mergePkgs = lists: concatLists lists;
in
  mergePkgs [
    [ "git" "vim" ]
    [ "curl" ]
    [ "htop" ]
  ]
```

Example E — Strict API for typos

```
let
  # no ellipsis - typo "hostNme" fails loudly
  mkHost = { hostName, user }: {
    inherit hostName user;
  };
in
  mkHost { hostName = "lab"; user = "alice"; }
```

Lab 1 — Workspace

```
mkdir -p ~/lab/nixos-book/concepts/functions
cd ~/lab/nixos-book/concepts/functions
```

Create `helpers.nix`:

```
rec {
  inc = x: x + 1;

  add = { a, b ? 0 }: a + b;

  mkHost = { hostName, user, sshPort ? 22, ... }: {
    networking = {
      inherit hostName;
      firewall.allowedTCPPorts = [ sshPort ];
    };
    users.users.${user}.isNormalUser = true;
  };
};
```



```

    enableIf = cond: attrs: if cond then attrs else { };

    mergeLists = builtins.concatLists;
}

```

Evaluate pieces:

```

nix eval --file ./helpers.nix --apply 'h: h.add { a = 2; b = 3; }'
nix eval --file ./helpers.nix --apply 'h: h.mkHost { hostName = "lab"; user = "alice"; }'

```

Lab 2 — Strict vs ellipsis

Create `strict.nix` and `loose.nix`:

```

# strict.nix
{ a, b }: a + b

# loose.nix
{ a, b, ... }: a + b

nix eval --file ./strict.nix --apply 'f: f { a = 1; b = 2; c = 3; }' # should fail
nix eval --file ./loose.nix --apply 'f: f { a = 1; b = 2; c = 3; }' # 3

```

Journal: when would failure be *better*?

Lab 3 — @ args inventory

```

# inspect.nix
args@{ a, b ? 1, ... }: {
  sum = a + b;
  keys = builtins.attrNames args;
  raw = args;
}

nix eval --file ./inspect.nix --apply 'f: f { a = 5; c = "extra"; }'

```

Confirm `keys` includes `c` and default `b` behavior matches your mental model.

Lab 4 — Compose three helpers into a “mini host”

In `mini-host.nix`, import/use helpers to produce one attrset that includes:

1. Hostname + admin user (via `mkHost`)
2. Conditionally enabled `services.demo` with `enableIf`
3. Merged package string lists

```
nix eval --file ./mini-host.nix
```

Lab 5 — Read real shapes (optional but high value)

If network/cache available:

```
nix repl

:lf nixpkgs
# skim types only - do not deep dive packaging yet
builtins.typeOf pkgs.hello
```

Open any small module online or local `nixpkgs` checkout later—note the header:

```
{ config, lib, pkgs, ... }:
```

Map each binder to this chapter’s theory.

Exercises

1. Write curried `add` and attr-pattern `add`; call both.
2. Implement `mkUser` with defaults for shell and groups.
3. Show that passing `retries = null` does not use the `?` default.
4. Build a strict validator function that rejects unknown keys (no `...`).
5. Use `@` to log (via returned attrset) all keys passed to a function.

6. Write `map/filter` pipeline over a list of host attrsets.
 7. Sketch an overlay `final: prev: ...` that renames nothing—just returns `{}`.
 8. Translate a Python `def f(a, b=1, **kw)` mental model into Nix patterns.
 9. Break and fix parentheses around lambdas in a list: `[ (x: x) ]`.
 10. Read `outputs = { self, nixpkgs, ... }` and rewrite with `@inputs`.
 11. Implement `enableIf` and contrast with raw `if` for empty attrsets.
 12. Document three error messages you triggered and their causes.
 13. Commit the functions lab directory.
 14. Preview: write the module header line from memory.
-

Common pitfalls

Symptom	Theory
called without required argument	Missing pattern field
unsupported argument '...'	Need <code>...</code> or remove extra
Defaults not applied	You passed <code>null</code> explicitly—defaults only for <i>missing</i> keys
<code>f x y</code> vs <code>f {...}</code> confusion	Know arity/shape of <code>f</code>
Forgetting parentheses around lambdas in lists	<code>[(x: x)]</code> not <code>[x: x]</code>
Trying defaults with positional args	Defaults are for attr patterns
Infinite recursion	Self-call without base case
Ellipsis hides typos forever	Prefer strict public APIs

Checkpoint

- ☐ Write a curried and an attr-pattern function
- ☐ Use `?` defaults correctly
- ☐ Explain ellipsis trade-off (strict vs loose)

- ☐ Use `@` to inspect or forward args
 - ☐ Build `mkHost` / `enableIf` style helpers
 - ☐ Recognize module/overlay/flake call shapes
-

Further depth (this book)

Topic	Chapter
Lazy evaluation of args	Laziness, imports, and lib
Flake <code>outputs</code> function	Flakes and project devShells
Module functions in production	Host: Modules as a consumer , Writing custom modules
<code>callPackage</code> packaging	Part Packaging

Laziness, imports, and lib

Laziness, imports, and lib

Goal: Split expressions across files, understand **lazy values** vs **eager imports**, and run useful **lib** pipelines without memorizing the whole standard library.

Why this chapter matters

Real Nix is multi-file. People hit “infinite recursion,” mysteriously slow eval, or “why did that download?” without a model of **when** work happens. This chapter installs that model and introduces **lib** as the toolbox you will touch forever.

Ops motivation: large host configs, shared modules, and nixpkgs itself only evaluate tractably because of laziness. Knowing what *forces* a value is how you debug slow evals and cycles.

Theory 1 — Laziness: values are promises

Nix does not compute a binding until something **needs** its value.

```
let
  a = builtins.trace "forcing a" (1 + 1);
  b = builtins.trace "forcing b" (2 + 2);
in
  a
```

Only "forcing a" prints—**b** is never forced.

Why it matters

Benefit	Example
Huge package sets	Unused packages not fully evaluated
Optional config	Disabled modules can avoid heavy work
Recursive structures	Attrsets can refer to each other carefully

Forcing

Anything that **inspects** the value forces it: arithmetic, string interpolation of it, `builtins.length`, taking an attribute that needs it, building a derivation that lists it as input, etc.

```
let
  expensive = builtins.trace "boom" (import ./big.nix);
in
  if false then expensive else 0
# "boom" should not appear - branch not taken
```

Mental model

```
binding = thunk (unevaluated)
use in expr → force thunk → value (cached for later forces)
```

Theory 2 — Imports are different: loading is eager (once forced)

```
import ./other.nix
```

`import` reads and parses the file and evaluates the root expression of that file **when the import expression is forced**. People say “imports are eager” meaning: **once you import, that file’s top-level expression is evaluated** (subject to laziness *inside* that file’s bindings).

Mental model

```
import path
  → read file
  → parse
  → evaluate root expression of that file
  → return the resulting value (function, attrset, ...)
```

Common patterns

```
# other.nix returns an attrset
import ./other.nix

# other.nix returns a function
import ./mkHost.nix { hostName = "lab"; user = "alice"; }
```

```
# relative to current file
import ./modules/ssh.nix
```

Paths

Prefer path literals so imports resolve relative to the **file**, not the process cwd:

```
import ./lib.nix          # good
import "./lib.nix"        # string - different rules; avoid casually
```

Import caching

Nix caches imports of the same path in an evaluation—knowing this helps reason about “why didn’t my edit apply?” (usually: different evaluation / flake copy / not saved).

Theory 3 — Multi-file composition patterns

Pattern A — Data module

users.nix:

```
{
  alice = { groups = [ "wheel" ]; };
  bob = { groups = [ ]; };
}
```

default.nix:

```
let
  users = import ./users.nix;
in {
  names = builtins.attrNames users;
  inherit users;
}
```

Pattern B — Function module

mkUser.nix:


```
{ name, groups ? [ ] }: {
  users.users.${name} = {
    isNormalUser = true;
    extraGroups = groups;
  };
}
```

default.nix:

```
import ./mkUser.nix { name = "alice"; groups = [ "wheel" ]; }
```

Pattern C — Directory + default.nix

Nix allows `import ./dir` when `./dir/default.nix` exists. Useful for tiny libraries.

Pattern D — Layered host-shaped tree (preview)

```
host.nix
imports data/users.nix
imports lib/mkFirewall.nix
returns nested config attrset
```

Same idea later as NixOS `imports = [ ... ]`, with module merge on top.

Theory 4 — Infinite recursion (how you will meet it)

```
let
  x = y;
  y = x;
in
  x
```

```
let
  pkgs = import nixpkgs { inherit system overlays; };
  overlays = [ (final: prev: { x = pkgs.hello; }) ]; # pkgs not ready
in
  pkgs
```

Debugging habits

1. Shrink the expression
2. `builtins.trace "here" value`
3. Break cycles by not referencing the final fixed point too early
4. Remember: printing an attrset forces attributes (in some UIs)
5. Bisect files: comment imports until eval succeeds

Fixed points (recognition only)

```
lib.fixpoint (self: {  
  a = 1;  
  b = self.a + 1;  
})
```

Cycles are intentional *only* via structured fixed points—not accidental `let` loops.

Theory 5 — lib as the standard library (taste, not catalog)

When you have `nixpkgs`:

```
lib = pkgs.lib;  
# or  
lib = import <nixpkgs/lib>; # classic; flakes prefer pkgs.lib
```

Attrsets

Function	Role
<code>lib.mapAttrs</code>	Map over name/value
<code>lib.filterAttrs</code>	Filter by predicate
<code>lib.attrNames</code> / <code>lib.attrValues</code>	Keys / values
<code>lib.recursiveUpdate</code>	Deep-ish merge
<code>lib.optionalAttrs</code>	Conditionally include an attrset
<code>lib.nameValuePair</code>	Build { <code>name</code> , <code>value</code> } for <code>listToAttrs</code>

Lists

Function	Role
<code>lib.concatLists</code>	Flatten one level
<code>lib.unique</code>	Dedup
<code>lib.toList</code>	Wrap non-list
<code>lib.optional</code>	[x] or [] from bool
<code>lib.optionals</code>	list or [] from bool
<code>lib.flatten</code>	Deeper flatten

Strings

Function	Role
<code>lib.concatStringsSep</code>	Join
<code>lib.hasPrefix</code> / <code>hasSuffix</code>	Tests
<code>lib.toLower</code>	Case
<code>lib.splitString</code>	Split

Example pipeline — users list → attrset

```
{ lib }:  
let  
  userList = [  
    { name = "alice"; admin = true; }  
    { name = "bob"; admin = false; }  
  ];  
in  
  lib.listToAttrs (  
    map (u: {  
      name = u.name;  
      value = {  
        isNormalUser = true;  
        extraGroups = lib.optional u.admin "wheel";  
      };  
    }) userList  
  )
```

Why not memorize all of lib?

Learn **10–15** functions deeply. Search `nixpkgs/lib` when you need more. Module authors live in `lib.mkOption`, `lib.mkIf`, `lib.types.*` (host modules chapters).

Theory 6 — NIX_PATH and angle-bracket paths (literacy)

```
import <nixpkgs> { }
```

<nixpkgs> looks up NIX_PATH. This book’s **default** is flakes—not channels—but you must recognize angle brackets in docs.

Style	Pinning
<nixpkgs>	Ambient, drift-prone
import ./pinned.nix	File-relative
Flake inputs.nixpkgs	Lock file (preferred)

Labs in this chapter should use **relative imports** between your files; if you pull nixpkgs, prefer a flake or a known pin over bare <nixpkgs> when you can.

Theory 7 — Evaluation vs runtime (again, file-centric)

```
let
  data = import ./data.nix;          # eval time: load nix
  # building a package would realize store paths - the derivations chapter
in
  data
```

Importing a Nix file does **not** compile C. It only produces Nix values (which might *describe* builds).

When does “work” happen?

Action	Phase
import ./x.nix forced	Eval
lib.mapAttrs ...	Eval
mkDerivation resulting build	Realize
fetchurl network	Realize (FOD)

Theory 8 — Project layout habits

Habit	Why
One concern per file	Diffs and reuse
<code>default.nix</code> as facade	Clean imports
Data vs functions separated	Test data without logic
Avoid 2k-line single files	Reviewability

Later flake layouts expand this (Home and flake layout part).

Worked examples bank

Example A — Lazy skip

```
let
  boom = builtins.abort "should not run";
in
  if true then 1 else boom
```

Example B — Import function

`add.nix`:

```
{ a, b }: a + b

import ./add.nix { a = 2; b = 40; }
```

Example C — `lib.optional` / `optionals`

```
lib.optional true "wheel"      # [ "wheel" ]
lib.optional false "wheel"    # [ ]
lib.optionals true [ "a" "b" ] # [ "a" "b" ]
```

Example D — `mapAttrs`

```
lib.mapAttrs (name: cfg: cfg // { inherit name; }) {
  alice = { uid = 1000; };
  bob = { uid = 1001; };
}
```

Example E — recursiveUpdate vs //

```
lib.recursiveUpdate
  { services.foo = { enable = true; port = 80; }; }
  { services.foo = { port = 8080; }; }
# enable preserved (deep merge behavior of recursiveUpdate)
```

Compare to shallow // from the attrsets chapter.

Lab 1 — Multi-file project skeleton

```
mkdir -p ~/lab/nixos-book/concepts/laziness/{lib,data}
cd ~/lab/nixos-book/concepts/laziness
```

data/users.nix:

```
[
  { name = "alice"; admin = true; shell = "bash"; }
  { name = "bob"; admin = false; shell = "zsh"; }
  { name = "carol"; admin = true; shell = "bash"; }
]
```

lib/users.nix:

```
{ lib }:
userList:
lib.listToAttrs (
  map (u: {
    name = u.name;
    value = {
      isNormalUser = true;
      extraGroups = lib.optional u.admin "wheel";
      shellName = u.shell;
    };
  }) userList
)
```

default.nix (standalone lib stub so you need no nixpkgs for this lab):

```
let
  lib = {
    optional = cond: x: if cond then [ x ] else [ ];
    listToAttrs = builtins.listToAttrs;
```

```

};
userList = import ./data/users.nix;
toUsers = import ./lib/users.nix { inherit lib; };
users = toUsers userList;
in {
  inherit users;
  admins =
    map (u: u.name) (builtins.filter (u: u.admin) userList);
}

```

```

nix eval --file ./default.nix
nix eval --file ./default.nix --apply 'v: v.admins'

```

Lab 2 — Trace forcing

Create `trace.nix`:

```

let
  a = builtins.trace "force-a" 1;
  b = builtins.trace "force-b" 2;
  c = a + 10;
in
  c

```

```

nix eval --file ./trace.nix

```

Edit so the result uses `b` instead; confirm traces change. Record observations.

Lab 3 — Infinite recursion (safe sandbox)

```

# cycle.nix - expected to fail
let x = y; y = x; in x

```

```

nix eval --file ./cycle.nix || true

```

Write a one-line description of the error. Then fix a *near-cycle* by breaking the dependency.

Lab 4 — Real lib if nixpkgs available

```
nix repl

:lf nixpkgs
lib = pkgs.lib
lib.id 42
lib.concatStringsSep "," [ "a" "b" "c" ]
lib.filterAttrs (n: v: v > 1) { a = 1; b = 2; c = 3; }
lib.recursiveUpdate { a.b = 1; } { a.c = 2; }
```

Port Lab 1's `toUsers` to use `pkgs.lib` instead of the stub (optional stretch).

Lab 5 — Teach-back

1. Lazy value vs import of a file
 2. Why multi-file beats one 2k-line expression
 3. One `lib` function you will actually reuse
 4. Why `<nixpkgs>` is ambient state
-

Exercises

1. Prove with `trace` that an unused `let` binding is not forced.
2. Force a binding via string interpolation and via `builtins.length`.
3. Split a host model into `data.nix` + `logic.nix` + `default.nix`.
4. Import a directory via `default.nix` facade.
5. Rebuild the shallow merge trap using `lib.recursiveUpdate`.
6. Write `lib.optionalAttrs` equivalents by hand; then use real `lib`.
7. Trigger and explain one infinite recursion from overlays-shaped code.

8. Map a list of services into an attrset with `listToAttrs`.
9. Document five force points you hit while exploring.
10. Compare eval time of a tiny file vs importing a heavy stub (subjective notes).
11. Replace all angle-bracket mental habits with relative imports in your lab.
12. List ten `lib.*` functions and one-line each (search as needed).
13. Commit the multi-file lab; ensure paths are file-relative.
14. Preview module `imports = [ ./a.nix ./b.nix ]` as the same composition idea.
15. Explain why disabled flakes inputs still may or may not evaluate (depends on forcing).

Common pitfalls

Symptom	Theory
Infinite recursion	Cycle in let/fixed-point
Import path not found	Relative to file vs cwd
Unexpected work during eval	Something forced a heavy binding
<code>cannot coerce a function</code>	Forgot to <i>call</i> imported function
Angle-bracket surprise versions	Unpinned <code>NIX_PATH</code>
<code>lib</code> undefined	Never imported <code>pkgs/lib</code>
Edit not picked up	Evaluating different file/copy
<code>//</code> where recursive merge needed	Wrong merge tool

Checkpoint

- ☐ Split data + logic across 2 files
- ☐ Explained laziness with a trace experiment
- ☐ Imported a function file and called it
- ☐ Used `map/filter` or `lib` pipeline on users

- ☐ Documented one recursion footgun
- ☐ Prefer relative imports over `<nixpkgs>` in new work

Further depth (this book)

Topic	Chapter / part
Derivations (when eval becomes build)	Derivations as an idea
Flake multi-file projects	Flakes and project dev Shells
Module imports + mkIf	Host modules chapters
Evaluator performance	Part Internals
Shared module libraries	Part Ops and fleet

Derivations as an idea

Derivations as an idea

Goal: Treat a **derivation** as a pure build plan: relate evaluation $\rightarrow$ `.drv` $\rightarrow$ store outputs; run a tiny build; read logs; explain sandbox and **fixed-output derivations (FODs)** at a conceptual level.

i Note

Deep `stdenv` phase mastery lives in the **Packaging** part of this book. This chapter is the **idea** and a successful build loop.

Why this chapter matters

Without “derivation,” the store is a pile of directories and flakes are just JSON with extra steps. Derivations are the **contracts** that produce store paths.

Ops motivation: cache misses, hash mismatches, sandbox failures, and “why did CI compile?” all sit on this abstraction. You cannot operate Nix without naming the plan.

Theory 1 — Derivation = build recipe with a promised output path

A derivation answers:

1. What **inputs** (other store paths, sources)?
2. What **builder** runs (usually `bash + stdenv`)?
3. What **environment** (env vars, sandbox)?
4. What **output path(s)** will hold results?

Informally:

```
derivation D
  inputs:  I1, I2, ...
  builder: B
  → output path P = hash(inputs, builder, args, system, ...)
```

In the common **input-addressed** model, the hash is about the plan, not the bytes of the result (contrast content-addressed—Internals part).

Vocabulary table

Term	Meaning
Derivation	Build plan object / <code>.drv</code>
Output	Realized store directory
Builder	Program that performs the build
Fixed-output derivation	Special case: hash of <i>content</i> declared
Dependent derivation	Normal build; hash of <i>inputs</i>

Theory 2 — Eval vs realize (operational)

Phase	Artifact	Command vibes
Evaluate	Nix values + derivation objects	<code>nix eval</code> , instantiate
Low-level plan	<code>.drv</code> file in the store	<code>nix derivation show</code>
Realize	Output path directories	<code>nix build</code>
Run	Binaries inside outputs	<code>nix run</code> , <code>./result/bin/...</code>

`.drv` is not the package

```
/nix/store/xxxx-hello-2.12.drv  ← plan
/nix/store/yyyy-hello-2.12      ← output (realized)
```

You ship and cache **outputs**; `.drv` explains how they were supposed to be built.

Pipeline sketch

```
flake/expression
  eval

derivation graph (.drv files)
  realize (build or substitute)

output paths in /nix/store
  references
```

Theory 3 — mkDerivation mental model (lite)

Most software uses `stdenv.mkDerivation`:

```
stdenv.mkDerivation {
  pname = "mytool";
  version = "0.1.0";
  src = ./src;
  buildInputs = [ /* runtime/build deps */ ];
  # phases: unpack, patch, configure, build, install, fixup, ...
}
```

For learning in this chapter, prefer **trivial builders** that avoid compilers:

```
pkgs.writeText "hello.txt" "hello from nix\n"

pkgs.writeShellScriptBin "greet" ''
  echo "hello from store script"
''
```

These still produce **real store paths** and teach the loop without a C toolchain fight.

Other trivial helpers (awareness)

Helper	Result
<code>writeText</code>	File in store
<code>writeTextFile</code>	More options
<code>writeShellScript</code>	Script path
<code>writeShellScriptBin</code>	<code>\$out/bin/name</code>
<code>runCommand</code>	Tiny custom builder

Theory 4 — Sandbox: why builds are fenced

The build sandbox aims to make builds **pure**:

Restriction (typical Linux sandbox)	Why
Limited filesystem view	No sneaking <code>/usr/lib</code> deps
No network (usually)	Downloads must be fixed-output derivations
Clean env	Fewer “works on my machine” env vars
Only declared inputs visible	Graph stays honest

Fixed-output derivations (FODs)

Fetchers (`fetchurl`, `fetchFromGitHub`) are special: they **may** use network but must declare a **hash** of the result. Wrong hash → fail. That is how purity reconciles with downloading sources.

```
fetchurl {
  url = "https://example.com/foo.tar.gz";
  sha256 = "..."; # wrong → error
}
```

Property	Normal derivation	FOD
Network	Usually blocked	Allowed
Output hash	From inputs (plan)	Declared (content)
Failure mode	Build error	Hash mismatch / fetch fail

Why FODs matter to operators

- Pinning sources makes rebuilds reproducible
- Hash mismatch is a security *and* integrity signal
- “Works with wrong hash” is not a feature

Theory 5 — Substituters vs local build

Realization of path P:

1. Is P already in the local store? → done
2. Can a trusted substituter provide P? → download
3. Else build from `.drv` locally

Implications

- First build can be “download a lot from `cache.nixos.org`”

- Changing one input changes the hash → cache miss → compile
- Trust is about **signatures**, not “HTTP has a file”

Observation	Interpretation
Instant hello	Substituted
Long compile after one-line change	Input hash changed
Hash mismatch on fetch	FOD integrity failure

Theory 6 — Outputs and multiple outputs (preview)

A derivation can have several outputs (out, dev, doc, ...):

```
P-out, P-dev, P-man
```

Closure size discipline later depends on not pulling dev into runtime. In this chapter: know out is the default.

```
nix derivation show nixpkgs#hello | head -c 1500
# look for output names in the JSON-ish structure
```

Theory 7 — What you inspect after a build

Tool	Use
<code>nix build ... --print-out-paths</code>	Output path
<code>ls result</code>	Symlink to output
<code>nix log <path or installable></code>	Build log
<code>nix derivation show</code>	JSON-ish plan
<code>nix path-info -rS</code>	Closure size

Reading a log like an operator

1. Was it a substitute or a local build?
2. Which phase failed (if stdenv)?
3. Missing dependency vs sandbox network vs hash?

Theory 8 — Impure vs pure evaluation (awareness)

Mode	Risk
Pure flake eval	Preferred; limited filesystem
<code>--impure</code>	Can see ambient state; less reproducible
<code>builtins.currentSystem</code>	Convenient; pin <code>system</code> when sharing

Prefer pure flakes in project work (next chapters).

Worked examples bank

Example A — Build hello from nixpkgs

```
nix build nixpkgs#hello --print-out-paths
./result/bin/hello
```

Example B — Trivial package via flake

See Lab 1. Prefer flakes over one-off impure exprs for cleanliness.

Example C — Read derivation metadata

```
OUT=$(nix build nixpkgs#hello --print-out-paths --no-link)
nix derivation show nixpkgs#hello | head -c 2000
nix log "$OUT" 2>/dev/null || nix log nixpkgs#hello | tail
```

Example D — Sandbox failure story (conceptual)

```
Builder does: curl http://example.com/dep.so
→ network blocked (non-FOD)
→ build fails
→ declare dep as input or use fetchurl with hash
```

Example E — runCommand sketch

```
pkgs.runCommand "note" { } ''
  mkdir -p $out
  echo "built at eval-defined plan" > $out/note.txt
''
```

Example F — Wrong FOD hash

Expect failure when sha256 does not match downloaded bytes—note the error class for packaging later.

Lab 1 — Workspace + flake for trivial builds

```
mkdir -p ~/lab/nixos-book/concepts/derivations
cd ~/lab/nixos-book/concepts/derivations
git init
```

Create `flake.nix`:

```
{
  description = "trivial derivations lab";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux"; # change if needed: aarch64-linux, aarch64-darwin, ...
      pkgs = import nixpkgs { inherit system; };
    in {
      packages.${system} = {
        greet = pkgs.writeShellScriptBin "greet" ''
          echo "hello from a derivation"
          '';
        note = pkgs.writeText "note.txt" ''
          Derivations plan store paths.
          '';
        default = self.packages.${system}.greet;
      };
    };
}
```

If your system is not `x86_64-linux`, edit `system`. Discover with:

```
nix eval --impure --expr 'builtins.currentSystem'

nix flake lock
nix build .#greet
./result/bin/greet
nix build .#note
cat result
```

Commit `flake.lock` when generated.

Lab 2 — Inspect plan and log

```
nix build .#greet --print-out-paths
nix derivation show .#greet | head -n 80
nix log .#greet | tail -n 40
```

Journal:

1. Output path hash/name
 2. Whether the “build” was essentially a write to the store
 3. One field you saw in `derivation show` that looks like an input
-

Lab 3 — Compare to `nixpkgs` hello closure

```
H=$(nix build nixpkgs#hello --print-out-paths --no-link)
G=$(nix build .#greet --print-out-paths --no-link)
echo "hello requisites: $(nix-store -q --requisites "$H" | wc -l)"
echo "greet requisites: $(nix-store -q --requisites "$G" | wc -l)"
nix path-info -rS "$H" | tail -3
nix path-info -rS "$G" | tail -3
```

Explain size difference in one sentence (runtime interpreter/libs vs tiny script).

Lab 4 — Break a FOD on purpose (optional)

```
nix build --impure --expr 'let pkgs = import (builtins.getFlake "nixpkgs") { system = builtins.currentSystem; } in pkgs.hello'
```

You want a **hash mismatch** or fetch failure—not a success. Note the error class for later packaging chapters.

Lab 5 — Teach-back

Define without notes:

1. Derivation
 2. `.drv` vs output path
 3. Why sandbox blocks network by default
 4. Substituter vs local build
 5. Why `writeShellScriptBin` is still a “real” package
-

Exercises

1. Add a second script package to the flake; make it **default**.
2. Use `--no-link` and explain GC-root impact.
3. Diff `derivation show` for `greet` vs `hello` (structural notes).
4. Force a rebuild (when applicable) and compare to substitute path.
5. Write `runCommand` output containing your username string (careful: purity).
6. Map each `stdenv` phase name to one sentence (even if you skip implementing).
7. Document sandbox vs FOD in a two-column table from memory.
8. Measure closure size before/after adding `pkgs.git` to a wrapper (optional).

9. Explain multi-output **dev** risk for runtime closures.
 10. Commit flake + lock; clone to a second directory and rebuild offline if possible.
 11. Trigger **system** mismatch by asking for the wrong triple; fix it.
 12. List three realize failures and which layer they belong to.
-

Common pitfalls

Symptom	Theory
Wrong system in flake	Outputs empty / wrong attr path
result pins GC root	rm result when done experimenting
Edited files in store	Immutable—change expression, rebuild
“Build” always compiles	Substituters often download
Impure path surprises	Unqualified imports / <code>currentSystem</code>
Darwin vs Linux scripts	shebangs and system must match host
FOD wrong hash ignored	Never; must match
Confusing .drv with runnable output	Plan vs product

Checkpoint

- ☐ Built a local trivial package via flake output
 - ☐ Ran the resulting binary/script
 - ☐ Inspected derivation show + log
 - ☐ Compared closure size to **hello**
 - ☐ Explained sandbox + FOD at awareness level
 - ☐ **flake.lock** present
 - ☐ Eval → drv → output story told in own words
-

Further depth (this book)

Topic	Chapter / part
Modern CLI around builds	Modern Nix CLI
Project shells	Flakes and project devShells
stdenv phases, FODs deep	Part Packaging
CA derivations	Part Internals
Remote builders / caches	Packaging + Ops parts

Modern Nix CLI

Modern Nix CLI

Goal: Prefer the **modern nix CLI** for all daily tasks: build, run, develop, shell, flake inspection, store queries—and know which command matches which intent.

Why this chapter matters

Docs and blog posts mix eras. Muscle memory on the wrong CLI makes flakes feel harder than they are. This chapter is deliberate fluency so flake authoring and host work feel like product work, not archaeology.

Ops motivation: incident response on a Nix machine is mostly “pick the right verb.” **build** vs **run** vs **develop** vs **eval** is how you avoid thrash under pressure.

Theory 1 — One CLI, many intents

Intent	Modern command	Rough meaning
Build installable → result	nix build	Realize outputs
Run app without installing	nix run	Build/realize + exec
Project dev environment	nix develop	Shell with build inputs (flake devShell)
Ad-hoc package shell	nix shell	Temporary PATH with packages
Explore flake outputs	nix flake show	Schema of outputs
Update lock	nix flake update	Refresh inputs (careful)
Metadata / eval	nix eval	Evaluate expressions
Store facts	nix path-info	Size, refs, sigs
Search	nix search	Find packages
Profile (user env)	nix profile	Generational user installs
Format / check (later)	nix fmt , nix flake check	Quality gates
Store GC	nix store gc	Collect unreferenced paths

Classic cousins (`nix-build`, `nix-shell`, ...) wait until the classic CLI chapter.

Theory 2 — Installables: how you *name* what you want

Modern CLI takes **installables**:

Form	Example	Meaning
Flake output	<code>.#greet</code>	Current flake, package <code>greet</code>
Flake + attr	<code>.#packages.x86_64-linux.greet</code>	Fully spelled
Indirect flake	<code>nixpkgs#hello</code>	Registry/indirect nixpkgs
Git flake	<code>github:NixOS/nixpkgs/nixos-26.05#hello</code>	Explicit
Path flake	<code>./myproj#default</code>	Local
File / expr	<code>-f file.nix, --expr '...'</code>	Non-flake (still useful)

splits flake ref from attribute

```
nixpkgs#hello

      attribute path inside outputs (with defaults)
flake reference
```

Ambiguity and defaults

`nix build .#` often means default package. If show is empty, check `system` keys under `packages.<system>`.

Theory 3 — `nix build` deep enough for daily use

```
nix build .#greet
nix build .#greet --print-out-paths
nix build .#greet -o greet-result
nix build nixpkgs#hello
```

Flag	Role
<code>--print-out-paths</code>	Print store paths
<code>-o name</code>	Name the result symlink

Flag	Role
<code>--no-link</code>	Realize without result
<code>-L / --print-build-logs</code>	Stream logs
<code>--rebuild</code>	Force rebuild (when applicable)
<code>--show-trace</code>	Eval error detail

Discipline: temporary builds → `--no-link` or `rm result` to avoid GC root clutter (store chapter).

Theory 4 — `nix run` vs `nix shell` vs `nix develop`

`nix run`

```
nix run nixpkgs#hello
nix run .#greet
nix run nixpkgs#cowsay -- "moo"
```

Builds/downloads if needed, runs the package’s main program, does **not** require a permanent profile entry. Args after `--` go to the program.

`nix shell`

```
nix shell nixpkgs#git nixpkgs#jq
# temporary shell with git + jq on PATH
nix shell nixpkgs#jq -c 'jq --version'
```

Ad-hoc tools. Great for “I need jq once.”

`nix develop`

```
nix develop                # .#devShells.default or similar
nix develop .#name
nix develop -c cargo --version
```

Enters the flake’s **devShell**—meant for project dependencies, compilers, env vars. The flakes chapter makes one.

Command	Sticky?	Typical use
<code>run</code>	No	One-shot CLI tools
<code>shell</code>	Until exit	Ad-hoc package mix

Command	Sticky?	Typical use
develop	Until exit	Project toolchain
profile install	Yes	User-global tools (use sparingly early)

Theory 5 — Flake subcommands

```
nix flake init           # scaffold (optional)
nix flake new mod        # template
nix flake show           # outputs tree
nix flake metadata       # resolved inputs, locks
nix flake check          # run checks (later quality)
nix flake update         # update all inputs
nix flake update nixpkgs # selective update
nix flake lock           # ensure lock matches urls
```

show vs metadata

Command	Answers
show	What outputs exist?
metadata	What are inputs pinned to?

JSON for scripting

```
nix flake metadata --json | head -c 400
nix flake show --json 2>/dev/null | head -c 400
```

Theory 6 — Eval and debugging eval

```
nix eval .#packages.x86_64-linux.greet.name
nix eval --raw .#packages.x86_64-linux.greet
nix eval --json nixpkgs#hello.meta | head
nix eval --show-trace --expr '1 + "x"' # intentional fail
```

```
nix repl
:lf .
:lf nixpkgs
```

REPL remains the best theory laboratory.

Goal	Tool
Print a string attr	<code>nix eval --raw</code>
Explore interactively	<code>nix repl</code>
Deep stack on error	<code>--show-trace</code>

Theory 7 — Store queries you will reuse

```
nix path-info -Sh ./result
nix path-info -rS ./result | tail
nix store gc --dry-run    # careful; prefer dry concepts early
# nix store delete ...    # advanced; skip casually
```

`nix-store -q` still works; modern `path-info` is friendlier.

Query	Command family
Size	<code>path-info -S / -rS</code>
Closure	<code>path-info -r</code>
Roots (classic)	<code>nix-store --gc --print-roots</code>

Theory 8 — Profiles without making them a lifestyle

```
nix profile list
nix profile install nixpkgs#jq
nix profile remove ...
```

This book's preference: project flakes + devShells over growing a huge imperative profile. Profiles are fine for a few personal tools; they are not a substitute for declarative hosts (NixOS host part).

Theory 9 — Search and discovery

```
nix search nixpkgs ripgrep
nix search nixpkgs#legacyPackages.x86_64-linux hello # more explicit forms exist
```

First search can be slow (indexing/eval). Prefer known attribute names when you have them.

Online companion: <https://search.nixos.org/packages> (select 26.05 when available).

Theory 10 — Error class → command checklist

Error class	First commands
experimental features	<code>nix config show</code>
attr missing	<code>nix flake show</code>
pure eval path missing	<code>git status</code> / flake files
build failed	<code>nix build -L</code> / <code>nix log</code>
wrong system	<code>builtins.currentSystem</code> / show packages

Worked examples bank

Example A — Same package, three ways

```
nix run nixpkgs#hello
nix shell nixpkgs#hello -c hello
nix build nixpkgs#hello --print-out-paths --no-link
```

Example B — Flake introspection

```
cd ~/lab/nixos-book/concepts/derivations # or your flake
nix flake show
nix flake metadata
```

Example C — Logs streaming

```
nix build nixpkgs#hello -L
```

Example D — JSON for scripting

```
nix flake metadata --json | head -c 400  
nix path-info --json nixpkgs#hello | head -c 400
```

Example E — Non-interactive develop

```
nix develop -c bash -lc 'git --version; echo $name'
```

Lab 1 — Cheat-sheet construction

```
mkdir -p ~/lab/nixos-book/concepts/modern-cli  
cd ~/lab/nixos-book/concepts/modern-cli
```

Create `CHEATSHEET.md` and fill by *running* each command at least once:

```
build:      ...  
run:        ...  
shell:      ...  
develop:    ... (may wait until flakes chapter if no shell yet)  
show:       ...  
metadata:   ...  
eval:       ...  
path-info:  ...  
search:     ...
```

Paste the exact successful command lines you used.

Lab 2 — Operate a local flake with modern CLI only

Reuse the derivations flake (or recreate a minimal package flake).

Tasks:

1. `nix flake show`
 2. `nix build .#greet --print-out-paths`
 3. `nix run .#greet`
 4. `nix path-info -rS ./result | tail` (if result exists)
 5. `rm -f result` and rebuild with `--no-link`
-

Lab 3 — Ad-hoc nix shell workflow

```
nix shell nixpkgs#jq nixpkgs#ripgrep -c bash -c 'jq --version; rg --version'
```

Solve a tiny task: query a JSON blob with `jq` **without** installing `jq` into a profile permanently.

```
echo '{"ok":true}' | nix shell nixpkgs#jq -c 'jq .'
```

Lab 4 — Search + run loop

```
nix search nixpkgs cowsay | head  
nix run nixpkgs#cowsay -- "modern cli"
```

Record the attribute name you used.

Lab 5 — Intent mapping drill (closed book)

Write the best command for each:

1. Temporarily need `ffmpeg`
 2. See outputs of current flake
 3. Build package without leaving `result`
 4. Run project's default app
 5. See what `nixpkgs` revision lock pins
-

Exercises

1. Build with `-o custom-result` and remove it; confirm root behavior.
 2. Compare `nix run` vs `nix shell -c` for a multi-arg program.
 3. Export `nix flake metadata --json` and extract locked rev (manual or `jq`).
 4. Use `nix eval` to print `hello.name` from `nixpkgs`.
 5. Intentionally pass a bad attr; read the error; fix via `flake show`.
 6. Time a cold vs warm `nix run nixpkgs#hello` (subjective notes OK).
 7. List profile generations; avoid installing anything permanent if possible.
 8. Write a 15-line operator runbook: “package missing on PATH.”
 9. Use `--show-trace` on a deliberate eval error.
 10. Document registry vs project lock difference for `nixpkgs#`.
 11. Practice `nix build + path-info -rS` closure report for two packages.
 12. Map five classic commands (from memory) to modern ones; verify next chapter.
-

Common pitfalls

Symptom	Theory
<code>experimental features</code> errors	<code>nix-command</code> / <code>flakes</code> not enabled
Attribute not found	Wrong system triple / output layout
<code>nix run</code> can't find binary	Package has no main program meta
Mixing <code>nix-env -i</code> habits	Prefer <code>run</code> / <code>shell</code> / <code>develop</code>
<code>flake show</code> empty packages	system key mismatch
Slow <code>nix search</code>	First-time indexing/eval
Sudo changes flake path cwd	Use absolute flake refs when needed
Profile becomes SoT	Move tools into flakes/HM later

Checkpoint

- ☐ Cheat-sheet written from real command output
- ☐ Can choose `run` vs `shell` vs `develop` correctly
- ☐ Built with and without `result` link
- ☐ Used `flake show` + `metadata`
- ☐ Used `path-info` on a real path
- ☐ No new dependency on classic CLI for daily work

Further depth (this book)

Topic	Chapter
Author flakes + devShells	Flakes and project devShells
Classic translation table	Classic CLI literacy
Host rebuild CLI	Host: Rebuild and generations
<code>nix flake check</code> quality	Home/flake layout part

Flakes and project devShells

Flakes and project devShells

Goal: Author a real **project flake** with pinned **inputs**, clear **outputs**, committed **flake.lock**, and a working **devShells.default**—before any NixOS host work.

Why this chapter matters

Flakes are the modern collaboration unit: reproducible inputs + named outputs. A **devShell** is the highest-value first output—shared tools without polluting a global profile. OS configuration comes in the NixOS host part; project pinning comes **now**.

Ops motivation: “works on my machine” dies when **flake.lock** is the shared truth. Teams that skip locks reinvent channel drift with extra steps.

Theory 1 — What a flake is

A flake is a directory (usually a git tree) with:

File	Role
<code>flake.nix</code>	Declarative inputs + outputs function
<code>flake.lock</code>	Exact pinned revisions/hashes of inputs

Evaluation model

```
flake.nix outputs = inputs: { ... }  
    ↓  
lock pins input trees  
    ↓  
outputs provide packages, shells, apps, nixosConfigurations, ...
```

Punchline: without the lock file, “same flake.nix” is not the same system.

Baseline for this book

```
inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
```

Nix **2.34.x** + flakes experimental features enabled (Why Nix exists chapter).

Theory 2 — inputs

```
{
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    # optional:
    # flake-utils.url = "github:numtide/flake-utils";
  };
}
```

Common URL forms

URL	Meaning
github:Owner/Repo	Default branch (avoid for pins—use lock)
github:Owner/Repo/ref	Branch/tag/rev
git+https://...	Arbitrary git
path:./vendor/x	Local path input

Following transitive inputs

```
inputs.foo.inputs.nixpkgs.follows = "nixpkgs";
```

Keeps a single nixpkgs when dependencies would otherwise multiply.

Input types (awareness)

Flakes can depend on other flakes, non-flake sources (with care), and path inputs. Prefer flake-native inputs for clarity.

Theory 3 — outputs

```
outputs = { self, nixpkgs, ... }@inputs:
  let
    system = "x86_64-linux";
    pkgs = import nixpkgs { inherit system; };
  in {
    packages.${system}.default = pkgs.hello;
    devShells.${system}.default = pkgs.mkShell {
      packages = [ pkgs.git pkgs.hello pkgs.jq ];
    };
  };
```

Output schema (common)

Output	Purpose
<code>packages.<system>.<name></code>	Buildable packages
<code>devShells.<system>.<name></code>	<code>nix develop</code> environments
<code>apps.<system>.<name></code>	<code>nix run</code> apps
<code>formatter.<system></code>	<code>nix fmt</code>
<code>checks.<system>.<name></code>	<code>nix flake check</code>
<code>nixosConfigurations.<name></code>	NixOS hosts (host part)
<code>overlays.default</code>	Package overlays (later)

`self`

`self` is the flake's own source (after copy to store). Used for `src = self;` packaging and referring to other outputs: `self.packages.${system}.foo`.

Naming default

Output	Default behavior
<code>packages.<sys>.default</code>	<code>nix build</code> / <code>nix run</code> without attr
<code>devShells.<sys>.default</code>	<code>nix develop</code> without attr

Theory 4 — Systems and portability

Hard-coding one `system` is fine for a first lab. Better patterns later: `flake-utils`, `flake-parts`, or explicit map over systems.

```
systems = [ "x86_64-linux" "aarch64-linux" "aarch64-darwin" "x86_64-darwin" ];
```

In this chapter: pick **your** system correctly or multi-define the same shell for two systems if you dual-boot.

Discover:

```
nix eval --impure --expr 'builtins.currentSystem'
```

genAttrs pattern

```
forAllSystems = f: nixpkgs.lib.genAttrs systems (system: f system);
```

Theory 5 — mkShell / mkShellNoCC

```
pkgs.mkShell {  
  packages = [ pkgs.go pkgs.git ];  
  shellHook = ''  
    echo "welcome to the dev shell"  
    export DEMO=1  
  '';  
}
```

Field	Role
<code>packages</code> / <code>nativeBuildInputs</code>	Tools on PATH
<code>buildInputs</code>	Libraries (more package-oriented)
<code>shellHook</code>	Runs on enter

Prefer putting **project** tools here, not every CLI you like globally.

mkShellNoCC

When you do not need a C toolchain, `mkShellNoCC` can shrink closure—useful for pure scripting shells.

Theory 6 — Lock discipline (intro)

```
nix flake lock          # ensure lock exists/up to date with urls
nix flake update        # bump inputs (all)
nix flake update nixpkgs # bump one input
```

Rule	Why
Commit <code>flake.lock</code>	Reproducibility
Review lock diffs	Security + surprise upgrades
Selective update	Smaller change sets
CI uses locked revs	Bit-identical intent

The lock discipline chapter (Home and flake layout part) goes deeper; here: **lock must exist and be committed**.

Theory 7 — Flakes and git

Flakes generally want a **git** repository. Untracked files may be invisible to evaluation depending on setup.

```
git init
git add flake.nix
# after first lock:
git add flake.lock
```

If something “missing,” check `git status`—not only the filesystem.

Dirty tree semantics

Flakes may copy a clean view of tracked files. Untracked `extra.nix` will not exist in eval—by design for purity.

Theory 8 — Registries (light awareness)

```
nix registry list
```

`nixpkgs#hello` resolves via registry → often a floating ref **until** you pin your project's own input. Project flakes should **not** rely on registry drift; they declare `inputs.nixpkgs.url`.

Reference	Pin quality
Registry <code>nixpkgs#</code>	Convenience; can drift
Project input + lock	Collaboration default

Theory 9 — Apps vs packages (optional clarity)

```
apps.${system}.default = {  
  type = "app";  
  program = "${self.packages.${system}.greet}/bin/greet";  
};
```

Often `nix run` on a package works via `meta.mainProgram`. Apps make the entrypoint explicit.

Theory 10 — What not to put in a first flake

Avoid early	Why
Entire NixOS config	Host part
Secrets	Store readability
Unrelated global tools dump	Closure bloat
Unlocked floating inputs	Drift

Worked examples bank

Example A — Minimal full flake

```
{
  description = "project shell";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      systems = [ "x86_64-linux" "aarch64-linux" "aarch64-darwin" "x86_64-darwin" ];
      forAllSystems = f: nixpkgs.lib.genAttrs systems (system: f system);
    in {
      devShells = forAllSystems (system:
        let pkgs = import nixpkgs { inherit system; };
        in {
          default = pkgs.mkShell {
            packages = with pkgs; [ git hello jq ];
          };
        });
    };
}
```

Example B — Enter and verify

```
nix develop
hello
git --version
jq --version
exit
```

Example C — Metadata

```
nix flake metadata
nix flake show
```

Example D — Non-interactive CI style

```
nix develop -c bash -lc 'git --version && jq --version'
```

Example E — Package + shell together

```
packages.${system}.greet = pkgs.writeShellScriptBin "greet" ''echo hi'';
devShells.${system}.default = pkgs.mkShell {
  packages = [ self.packages.${system}.greet pkgs.git ];
};
```

Lab 1 — Create the project

```
mkdir -p ~/lab/nixos-book/concepts/flakes/myproject
cd ~/lab/nixos-book/concepts/flakes/myproject
git init
```

Write `flake.nix` (start from Example A; trim systems if you want).

```
nix flake lock
nix flake show
nix develop -c hello
```

```
git add flake.nix flake.lock
git commit -m "initial project flake with devShell"
```

Lab 2 — Make it *your* project

Add tools you actually want for a small project (pick a lane):

Lane	Example packages
Go	go, gopls, git
Rust	rustc, cargo, rust-analyzer
Python	python3, uv or poetry
Ops	jq, ripgrep, kubectl (if desired)

Edit `packages = [ ... ]`, re-enter shell, verify versions.

Add a trivial project file `README.md` describing how to enter the shell.

Lab 3 — shellHook and env

```
shellHook = ''
  export PROJECT_ROOT="$PWD"
  echo "devShell ready for $PROJECT_ROOT"
'';

nix develop -c bash -c 'echo $PROJECT_ROOT'
```

Lab 4 — Offline-capable after populate (preview)

1. Enter `nix develop` once online (populate store).
2. Note `nix flake metadata` resolved revisions.
3. Optional: airplane mode / disconnect and `nix develop --offline` (or equivalent) if supported in your setup.

Journal what still requires network (usually nothing if all paths realized).

Lab 5 — Teach-back

1. Why `flake.lock` is mandatory in git
 2. Difference between `packages` and `devShells`
 3. What `nix develop` uses by default
 4. Why registry `nixpkgs#` your flake's pinned `nixpkgs`
-

Exercises

1. Add a second named devShell (`devShells.${system}.tiny`) with fewer packages.
2. Use `follows` with a second input (even a path stub) to practice wiring.

3. Bump only `nixpkgs` with selective update; read the lock diff.
 4. Break purity: leave a needed file untracked; observe failure; `git add`.
 5. Export `metadata --json` and record locked `narHash/rev`.
 6. Add a trivial `packages.default` and `nix run`.
 7. Document system portability: does a friend on another arch work?
 8. Compare closure of fat vs thin shell (`path-info`).
 9. Write `CONTRIBUTING.md`: “always commit lock.”
 10. Replace registry experiments with project inputs only.
 11. Try `nix flake check` if you add a simple check later (optional).
 12. Sketch how `nixosConfigurations` will hang off the same outputs function (host part).
-

Common pitfalls

Symptom	Theory
File not found in flake eval	Not git-added
Wrong system attr	<code>devShells.<system></code> mismatch
<code>nix develop</code> empty	No <code>default</code> shell
Divergent tools across machines	Lock not committed / not pulled
Huge shell closure	Too many packages; split shells later
Dirty tree warnings	Commit or understand copy semantics
Floating input without lock	Non-reproducible
Secrets in flake source	Store may expose them

Checkpoint

- ☐ `flake.nix` with `nixpkgs 26.05` input
- ☐ `flake.lock` committed

- ☐ `nix develop` provides 2 real tools
- ☐ `nix flake show` lists devShell
- ☐ README explains entry
- ☐ Understand lock vs registry
- ☐ Multi-system or correctly single-system for your machine

Further depth (this book)

Topic	Chapter / part
Classic modern map	Classic CLI literacy
Host flakes	Host: Flake-defined hosts
Lock discipline deep	Home and flake layout part
flake-parts, checks	Home and flake layout part
Packaging <code>src = self</code>	Packaging part

Classic CLI literacy

Classic CLI literacy

Goal: Map **classic** Nix commands and channel habits to the **modern** CLI—once—so legacy docs are readable without making classic tools your daily driver.

! Important

This chapter is **literacy**, not a lifestyle change. After it, default back to modern **nix** + flakes.

Why this chapter matters

Half the internet still says `nix-env -iA nixpkgs.hello` and `nix-shell -p`. If you cannot translate, you thrash. If you only learn classic, you fight 2026 defaults. Translation table + one dual run is enough.

Ops motivation: production runbooks, ancient Stack Overflow answers, and coworker muscle memory will keep showing classic commands. Translation skill is cheaper than rewriting the internet.

Theory 1 — Same store, two UIs

Layer	Shared	Differs
Store	<code>/nix/store</code> , derivations, GC roots	—
Eval	Nix language	Entry points, purity defaults
UX	—	Flag names, flake awareness, UX polish

Classic tools predate flakes. They still work. They are not the pedagogy default of this book.

`/nix/store`

classic CLI
(`nix-build`, ...)

modern ``nix``
(`nix build`, ...)

Theory 2 — Command map (core)

Classic	Modern analogue	Notes
<code>nix-build -A hello</code>	<code>nix build nixpkgs#hello</code>	Attr selection differs
<code>nix-build</code> on <code>default.nix</code>	<code>nix build -f . / flake packages</code>	
<code>nix-shell -p hello</code>	<code>nix shell nixpkgs#hello</code>	Ad-hoc tools
<code>nix-shell (dev shell.nix)</code>	<code>nix develop (flake) / nix-shell still</code>	Project shells
<code>nix-env -iA nixpkgs.hello</code>	<code>nix profile install nixpkgs#hello</code>	Prefer not to
<code>nix-env -q</code>	<code>nix profile list</code>	
<code>nix-env -e</code>	<code>nix profile remove</code>	
<code>nix-instantiate</code>	<code>nix eval / derivation show</code>	Different shapes
<code>nix-store -q --requisites</code>	<code>nix path-info -r</code>	Both fine
<code>nix-store -q --tree</code>	(still useful) / <code>path-info</code>	Keep classic for trees
<code>nix-channel --update</code>	<code>nix flake update (project)</code>	Different model
<code>nix-collect-garbage</code>	<code>nix store gc / nix-collect-garbage</code>	Still common
<code>nix-prefetch-url</code>	<code>nix store prefetch-file etc.</code>	Variants evolve

Intent-first map

Intent	Prefer modern	Classic you may see
Run once	<code>nix run</code>	<code>nix-shell -p ... --run</code>
Project tools	<code>nix develop</code>	<code>nix-shell</code>
Realize output	<code>nix build</code>	<code>nix-build</code>
Pin project	<code>flake.lock</code>	<code>channel + NIX_PATH</code>
Query closure	<code>nix path-info -r</code>	<code>nix-store -q --requisites</code>

Theory 3 — Channels as history

Channel model

```
nix-channel --add https://nixos.org/channels/nixos-26.05 nixpkgs
nix-channel --update
nix-env -iA nixpkgs.hello
```


Property	Effect
Mutable “latest on channel”	Drift between machines/days
NIX_PATH / <nixpkgs>	Ambient dependency
User or root channels	Confusing multi-user stories

Flake model (preferred)

```
inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
# flake.lock pins exact rev
```

Property	Effect
Lock file	Shared truth
Explicit inputs	No ambient channel required
Per-project pins	Different projects can differ safely

This book: channels = read-only literacy. Do not build new projects on channels.

When channels still appear

Context	Why
Old tutorials	Written pre-flakes default
Some NixOS install defaults	Historical paths
Angle-bracket imports	<code>import <nixpkgs> {}</code>

Theory 4 — shell.nix / default.nix legacy layouts

Classic project shell

shell.nix:

```
{ pkgs ? import <nixpkgs> { } } :
pkgs.mkShell {
  packages = [ pkgs.hello pkgs.git ];
}
```

```
nix-shell # enters shell.nix
```

Modern

`flake.nix` → `devShells.default` → `nix develop`

You may keep a thin `shell.nix` for non-flake users, but **your** path is flakes.

Dual support pattern (optional)

Some repos provide both: `flake` for modern users, `shell.nix` that re-exports for classic. Only do this if you must support both audiences.

Theory 5 — `nix-env` and imperative profiles

```
nix-env -iA nixpkgs.hello
nix-env -e hello
nix-env --list-generations
```

Problems as a primary workflow:

1. Not recorded in project git
2. Easy to create unreproducible laptops
3. Diverges from NixOS declarative model

`nix profile` is the modern imperative cousin—same caution.

Imperative tool	Declarative alternative
<code>nix-env -i</code>	flake packages / NixOS <code>systemPackages</code> / HM
Random <code>nix-shell -p</code> forever	<code>devShells</code>
Channel update as SoT	<code>flake.lock</code> review

Theory 6 — Purity and flakes

Classic ad-hoc eval often sees your `cwd` and `NIX_PATH` freely. Flake evaluation is **more pure** by default (limited access to paths outside the flake). That is why git-tracked files matter (flakes chapter).

When classic examples “just import `./.`”, flake equivalents may need **self** or explicit file inclusion.

Classic habit	Flake analogue
<code>import ./default.nix</code>	<code>outputs / packages</code>
Ambient <code><nixpkgs></code>	<code>inputs.nixpkgs</code>
Untracked helper file	Must be added to git

Theory 7 — Reading old error messages

Old-doc symptom	Translation
<code>attribute 'foo' missing in -A</code>	Wrong attr path / channel content
<code>file 'nixpkgs' was not found</code>	<code>NIX_PATH</code> / channel not set
<code>nix-env</code> permission errors	multi-user profile ownership
Blog says enable flakes differently	Check current Nix 2.34 docs
<code>error: experimental Nix feature</code>	modern CLI without features

Theory 8 — GC and store tools (both eras)

Task	Classic	Modern
GC	<code>nix-collect-garbage</code>	<code>nix store gc</code>
Delete gens aggressively	<code>nix-collect-garbage -d</code>	same caution
Query refs	<code>nix-store -q --references</code>	<code>nix path-info</code>
Roots	<code>nix-store --gc --print-roots</code>	still common

Do not GC aggressively on machines with important generations until the host generations chapter.

Theory 9 — Decision policy for this book

Situation	Policy
New personal project	Flakes + modern CLI only
Reading a 2019 blog	Translate; do not adopt channels
Coworker pastes <code>nix-env</code>	Offer modern equivalent once

Situation	Policy
Debugging store graphs	Either store tool family is fine
NixOS host management	<code>nixos-rebuild --flake</code> (host part)

Worked examples bank

Example A — Build hello both ways

```
# modern
nix build nixpkgs#hello --print-out-paths --no-link

# classic-ish
nix-build -E 'with import <nixpkgs> { }; hello' --no-out-link
```

(If `<nixpkgs>` missing, skip classic path or install channel only for the drill.)

Example B — Shell both ways

```
nix shell nixpkgs#hello -c hello
nix-shell -p hello --run hello
```

Example C — Query store both ways

```
P=$(nix build nixpkgs#hello --print-out-paths --no-link)
nix-store -q --references "$P"
nix path-info -r "$P" | head
```

Example D — Profile vs run

```
# avoid if possible
# nix-env -iA nixpkgs.jq
# prefer
nix run nixpkgs#jq -- --version
```

Example E — Channel list observation

```
nix-channel --list || true
```

Empty can be good on a flakes-first machine.

Lab 1 — Build the translation table

```
mkdir -p ~/lab/nixos-book/concepts/classic-literacy
cd ~/lab/nixos-book/concepts/classic-literacy
```

Create CLASSIC-MODERN.md with at least **10 rows** you personally verified (or attempted):

Task	Classic command	Modern command	Result notes
run hello			
...			

Lab 2 — Dual path for a project shell

Take your flakes chapter project (or recreate):

Modern (already works):

```
cd ~/lab/nixos-book/concepts/flakes/myproject
nix develop -c git --version
```

Classic parallel: write `shell.nix` that mirrors the tool list using `<nixpkgs>` *or* a pinned fetch if you know how—only for the drill:

```
{ pkgs ? import <nixpkgs> { } }:
pkgs.mkShell {
  packages = [ pkgs.git pkgs.hello pkgs.jq ];
}
```

```
nix-shell --run 'git --version'
```

Journal: which is easier to share with a teammate on another machine?

Lab 3 — Channel inspection (optional, disposable)

Only if you are willing to touch channels on a **lab account**:

```
nix-channel --list
# do not --update production machines casually
```

If empty: good—note “no channels; flakes only.”

Lab 4 — Doc archaeology

Find one external tutorial that uses classic CLI (search your history or web). Rewrite its first three commands into modern equivalents in your notes **without running random untrusted scripts as root**.

Lab 5 — Commitment statement

Write three sentences:

1. When I will still use classic tools
 2. When I will refuse to (prefer modern)
 3. How I will handle future blog posts that only show `nix-env`
-

Exercises

1. Expand your table to 15 rows including GC and store queries.
2. Translate `nix-env -iA nixpkgs.hello` into three modern alternatives (`run/shell/profile`).
3. Explain why `nix-shell -p` is not a project SoT.
4. Convert a classic `default.nix` package build to a flake `packages` output (sketch).
5. Document `NIX_PATH` on your machine; plan to ignore it for new work.

6. Pair-read an old tutorial with a highlighter: classic vs still-valid concepts.
7. Practice `nix-store -q --tree` vs modern path-info on the same path.
8. Write a “please use flakes” snippet for your team README.
9. List risks of `nix-collect-garbage -d` in classic blogs.
10. Map `nixos-rebuild` classic channel workflows to `--flake` (preview host part).
11. Deliberately avoid installing channels even if a blog demands it—note the workaround.
12. Quiz yourself: flashcard classic → modern for one week of reading.

Common pitfalls

Symptom	Theory
Following classic blog on NixOS host	May fight flakes/module setup
Installing channels “to fix” flakes	Usually wrong direction
<code>nix-shell</code> vs <code>nix develop</code> confusion	Different project conventions
Profile pollution from drills	Remove packages / use disposable user
Assuming channels == 26.05 pin	Channel lag vs flake lock rev
Mixing both as SoT	Pick flakes for new work
Treating literacy chapter as daily driver	Revert to modern defaults

Checkpoint

- ☐ Personal 10-row translation table
- ☐ Same task run modern + classic once
- ☐ Can explain channels vs flake locks
- ☐ Can explain why `nix-env` is not the goal
- ☐ Default workflow remains modern
- ☐ Commitment statement written

Concepts part — exit criteria

Before the **NixOS host** part:

- ☐ Three faces of Nix clear
 - ☐ Closures and GC roots clear
 - ☐ Attrsets + functions + multi-file Nix workable
 - ☐ Derivation idea + trivial build done
 - ☐ Modern CLI fluent
 - ☐ Project flake + lock + devShell done
 - ☐ Classic docs are readable, not authoritative
-

Further depth (this book)

Topic	Chapter / part
Host management CLI	Part NixOS host
Packaging classic attrs	Part Packaging
Deeper flake layout	Part Home and flake layout
Restart Concepts map	Concepts index

Nix on Linux

Nix on Linux — Fedora bridge path

Nix on Linux — Fedora bridge path

This part is the **on-ramp**: run the full **Nix package manager** on a normal Linux distribution—especially **Fedora 44+**, which now ships Nix officially—so you learn store, flakes, profiles, Home Manager, and project shells **without** replacing your OS yet.

When you later install **NixOS**, almost nothing you practice here is wasted. You already speak store paths, locks, **nix develop**, GC roots, and declarative user config. NixOS then “only” adds: module system for the *machine*, generations for the *bootloader*, and system-wide rebuilds.

💡 Tip

Who this part is for

- Daily driver is **Fedora 44** (or nearby: F43+, EPEL packaging where available)
- You want Nix **skills** before a full distro switch
- You already finished (or skimmed) **Concepts** (01-concepts/)
- You are *not* ready to wipe the host for NixOS this week

If you only have a spare disk/VM and want the OS immediately, you can jump to **NixOS host**—but this bridge still pays off for multi-machine workflows (Fedora laptop + NixOS server is a common end state).

Why Fedora 44 matters

Historically, Nix on Fedora meant the upstream multi-user installer, SELinux friction, and awkward ownership of `/nix`.

Fedora Change: **Nix package tool** (feature for Fedora 44) plus approval of `/nix` for the distro package changed that:

Before (typical)	Fedora 44+ packaged path
<code>curl \</code> <code>sh</code> Determinate/upstream installer	<code>sudo dnf install nix</code>
Manual SELinux babysitting	Policy-integrated Fedora build
Flakes often left experimental	Flakes enabled out of the box (Fedora packaging)

Before (typical)	Fedora 44+ packaged path
Atomic desktops painful	Documented rootless / bind-mount patterns

Upstream discourse and Fedora packaging notes also describe **nix-daemon multi-user, single-user**, and **pure rootless** modes (especially useful on Atomic/bootc-style systems). Always confirm with `/usr/share/doc/nix/README.fedora.md` on your machine—Fedora packaging details evolve faster than this notebook.

Baseline for this book still applies: treat **Nix 2.34-class** tooling and **nixpkgs 26.05** pins as the comparison point when you write flakes.

What you will learn here

Chapter	You leave able to...
Why Nix on Linux	Map package manager vs OS; design a dual-stack life (dnf + Nix)
Fedora 44 install	Install daemon or single-user; verify; read Fedora docs on disk
Store, profiles, GC	Own profiles without snowflaking; GC safely; see roots
CLI + devShells	Daily nix build/run/develop/shell ; flake projects; direnv
Home Manager standalone	Declarative user environment on Fedora
dnf + Nix coexistence	Decide what belongs where; avoid PATH wars
SELinux, Atomic, ops	Survive enforcing SELinux; Atomic /nix bind ; disk ops
Bridge to NixOS	Port habits into first nixosConfigurations host

Suggested study path

Concepts (store, language, flakes)

Nix on Linux (you are here:
Fedora daily driver)

same flake skills

NixOS host (machine as code)

Home Manager as NixOS module
(you already know standalone HM)

Path	Order
Recommended	Concepts → this part → NixOS host → Home layout part (skim what you already know)
Already installed Nix via curl	Start at install chapter for Fedora packaging migration , then continue
Atomic desktop only	Install + SELinux/Atomic chapter early

Lab workspace

```
mkdir -p ~/lab/nixos-book/nix-on-linux
cd ~/lab/nixos-book/nix-on-linux
git init    # optional but recommended
```

Keep experiments **outside** critical personal data. The store under `/nix` will grow—budget **20–50+ GB** if you build many shells.

Exit gate (before NixOS host)

- ☐ `nix --version` works after reboot
- ☐ Multi-user **or** intentional single-user/rootless mode documented in your notes
- ☐ Flakes work: `nix flake metadata nixpkgs` (or a personal flake)
- ☐ You can enter a project `devShell` and have teammates reproduce it via `flake.lock`
- ☐ You understand **profile** vs **devShell** vs **system packages** (NixOS later)
- ☐ Home Manager standalone rebuild succeeds for at least shell + git + editor snippet
- ☐ You can explain what moves to NixOS later (modules, generations, boot) without panic

Further depth (this book)

Next	Part
Install and own a machine	03-nixos-host/
Flake layout + HM as system module	04-home-and-flake-layout/
Packaging craft	06-packaging/
ROS 2 robotics env (overlay + flakes)	ROS 2 with nix-ros-overlay

Official docs win when notes disagree: [nix.dev](#), [NixOS manual](#), Fedora [/usr/share/doc/nix/](#), [Fedora Nix Change](#).

Why Nix on Linux first

Why Nix on Linux first

Goal: Decide *why* to run the Nix package manager on Fedora (or another Linux) **before** NixOS, and design a dual-stack workflow that makes the later OS transition easy—not confusing.

Why this chapter matters

Switching distros and learning Nix **at the same time** multiplies failure modes: bootloader issues look like “Nix is hard,” and store GC looks like “Fedora broke.”

Nix-on-Linux separates concerns:

Layer	Who owns it on Fedora + Nix	Who owns it on NixOS
Kernel, bootloader, base OS	Fedora (dnf, rpm-ostree, bootc...)	NixOS modules + generations
/nix/store software	Nix	Nix (same mental model)
Project toolchains	Nix flakes / devShells	Same
User dotfiles	Home Manager standalone <i>or</i> imperative	Home Manager (often as NixOS module)
System services (sshd, nginx)	systemd units / Fedora packages	NixOS services.*

Everything in the middle rows **transfers**. Only the “OS owner” row changes later.

Three faces again (bridge framing)

From Concepts you know:

1. **Language** — expressions
2. **Package manager** — store, builds, profiles
3. **NixOS** — OS as a module system

On Fedora you run **(1)** + **(2)** every day. **(3)** waits until you want the machine itself as a flake output (`nixosConfigurations`).

```
Today (Fedora 44 + Nix)
dnf          OS packages, kernel, desktop, SELinux policy
nix          toolchains, CLIs, project shells, optional HM

Later (NixOS)
nixos-rebuild OS + services + kernel modules as code
nix          same store / same flakes for packages & shells
home-manager often folded into the host flake
```

What Nix is good at *on* an existing distro

Strength	Concrete use on Fedora
Reproducible project envs	<code>flake.nix</code> + <code>direnv</code> → same Node/Go/Rust/Python as CI
Version coexistence	Old <code>glibc</code> -sensitive tools without breaking desktop
Non-root software	Install tools without waiting for RPM or COPR
Atomic user rollback (HM)	Generations for <i>your</i> home config
Cross-machine sameness	Same flake on Fedora laptop and NixOS server
Try packages safely	<code>nix shell nixpkgs#foo</code> — no global pollution

What you should still leave to Fedora

Keep on dnf / Atomic image	Why
Kernel, firmware, mesa, GPU stacks	Closest to hardware; SELinux-integrated
Desktop session (GNOME/KDE), PipeWire	Distro integration and portals
SELinux policy, firewalld defaults	Security baseline of the OS
Flatpaks for sandboxed GUI apps (optional)	Complements Nix; different trust model
System daemons you didn't declare in Nix	Avoid two sources of truth for sshd/nginx

Rule of thumb: OS and *machine identity* → Fedora for now. *Development surface and portable CLI life* → Nix.

Dual-stack anti-patterns

Anti-pattern	What goes wrong	Better
Install <i>everything</i> with <code>nix profile</code>	Huge profile, slow GC, PATH chaos	Flakes per project + small profile
Mix <code>nix-env -i</code> and flakes	Two eras of tooling	Modern CLI + flakes only
Put secrets in the store via HM	World-readable store paths	sops/age later; never commit secrets
Disable SELinux “to make Nix work”	Trains bad ops habits	Use Fedora packaging + docs
Skip locks	“Works on my machine” returns	Commit <code>flake.lock</code>
Expect Nix to replace dnf for Firefox+kernel	Wrong layer	Keep browser/kernel on Fedora

Skills map: bridge → NixOS

Skill on Fedora + Nix	Same skill on NixOS	New on NixOS
<code>nix build</code> / <code>nix run</code>	Yes	—
Flake inputs + lock	Yes	<code>nixosConfigurations</code> output
<code>nix develop</code> + <code>direnv</code>	Yes	Same
Store, closures, GC roots	Yes	System profile is also a root
<code>nix profile</code>	Optional	Prefer declarative system/HM
Home Manager standalone	Yes	Often <code>home-manager.users.*</code> module
systemd user services via HM	Partial	Full system units via modules
Roll back user env	HM generations	Boot generations + HM

You are not “wasting time” on Fedora—you are doing **package-manager literacy** with a normal desktop underneath.

Mental model: evaluation vs realization (ops view)

```
flake.nix + flake.lock
  nix develop / nix build
```

```
evaluate (pure-ish)
```

```
derivation plans (.drv)
  nix-daemon realizes
```

```
/nix/store/<hash>-name
```

```
ephemeral shell (devShell) - dies when shell exits (roots may linger with direnv)
profile generation - GC root until removed
```

On NixOS, add another root: **system generation** → bootloader entry.

Worked decision: “Should this be Nix or dnf?”

Want...	Choose
ripgrep, fd, latest helix for <i>you</i>	Nix profile or HM
Project needs Go 1.26 + exact lib versions	Flake devShell
System upgrade of GNOME	dnf / Software
Kernel module for VPN	Fedora packages
nginx on a lab VM you will reinstall as	Either; prefer learning NixOS module on a VM,
NixOS soon	not production Fedora host
CI-identical toolchain	Flake (Nix)

Lab

1. Write a short **charter** in ~/lab/nixos-book/nix-on-linux/CHARTER.md:

```
# Nix on Fedora charter

## OS owner
Fedora 44 - dnf handles: ...

## Nix owner
Nix handles: ...

## Out of scope for Nix (for 90 days)
...

## Success metrics
- [ ] One real project uses nix develop + direnv
- [ ] HM manages shell + git
- [ ] Disk budget for /nix: __ GB
```

2. List five tools you currently install via raw binaries or language version managers; mark each **Nix candidate** or **keep distro**.

Checkpoint

- ☐ You can explain package manager vs NixOS in one breath

- ☐ You have a written split of responsibilities (dnf vs Nix)
 - ☐ You know which skills transfer to NixOS unchanged
-

Further depth

- Install on Fedora 44: next chapter
- Concepts recap: [Why Nix exists](#), [Store and closures](#)
- Eventually: [NixOS install](#)

Install Nix on Fedora 44

Install Nix on Fedora 44

Goal: Install Nix from **Fedora's official packages**, enable the daemon (or choose single-user/rootless deliberately), verify flakes, and know where Fedora documents live on disk.

! Important

Prefer `dnf install nix` on Fedora 44+ over the classic `curl | sh` upstream installer unless you have a specific reason (custom builds, non-Fedora policy). The distro package is the supported path for SELinux and `/nix` integration.

Prerequisites

Need	Check
Fedora 44 (or current release with <code>nix</code> in repos)	<code>cat /etc/os-release</code>
<code>sudo</code>	Wheel user
Disk	Prefer 20 GB free on the filesystem that will hold <code>/nix</code>
Network	For first substitutes from <code>cache.nixos.org</code>

```
free -h
df -h /
# or wherever /nix will live
```

Install modes (choose one)

Fedora packaging supports several setups (see package docs):

Mode	Install sketch	Who builds	Best for
Multi-user + daemon (recommended)	<code>dnf install nix + enable nix-daemon</code>	nixbld users via daemon	Desktops, shared machines, matches NixOS mental model
Single-user	install excluding daemon / singleuser subpackage	Your UID	Solo laptops if daemon undesired
Rootless / Atomic	special /nix storage (bind mount)	Your UID / constrained	Silverblue, Kinoite, bootc images

This chapter emphasizes **multi-user daemon** on classic Workstation/Server, then notes Atomic.

Multi-user install (Workstation / Server)

1. Install packages

```
sudo dnf install -y nix
```

On some releases you may see related packages (`nix-daemon`, `docs`). Let `dnf` pull the recommended set.

```
rpm -q nix
rpm -ql nix | head
ls /usr/share/doc/nix/ 2>/dev/null || true
```

Read the Fedora-specific README if present:

```
less /usr/share/doc/nix/README.fedora.md
# or
rpm -qd nix
```

2. Enable the daemon

```
sudo systemctl enable --now nix-daemon.service
systemctl status nix-daemon --no-pager
```

3. Groups and shell environment

Multi-user installs typically put builders in `nixbld` and may require your user in a group to talk to the daemon. Fedora's README is authoritative; common pattern:


```
# If docs say so:
getent group nixbld
groups    # ensure you can use nix after re-login
```

Log out/in or open a **new** login shell so profile snippets from `/etc/profile.d/` or `/nix/...` apply.

```
# Common: bash/zsh pick up nix from profile.d
bash -l -c 'command -v nix && nix --version'
```

4. First smoke test

```
nix --version
nix doctor || true

# Classic hello (works even if you still learn modern CLI)
nix-shell -p hello --run hello

# Modern CLI + flakes (Fedora enables flakes in packaging - verify):
nix show-config | rg -n 'experimental-features|flakes|nix-command' || true
nix run nixpkgs#hello
```

If `nix run nixpkgs#hello` works, flakes + substitutes are alive.

Configure system vs user

Daemon / system config

Multi-user security options often live in `/etc/nix/nix.conf` (read by the daemon). Examples operators care about:

```
# Inspect effective config
nix show-config | rg -n 'sandbox|experimental|substituter|trusted'

# Example: ensure sandbox (security) - only if not already set by package defaults
# sudo tee -a /etc/nix/nix.conf <<'EOF'
# sandbox = true
# EOF
# sudo systemctl restart nix-daemon
```

Warning

Do not casually set `trusted-users = *` or disable sandbox “to go faster” on a daily driver. Prefer package defaults; change with intent.

User config

```
mkdir -p ~/.config/nix
```

Fedora may already enable flakes system-wide. If `nix flake` fails, set:

```
cat >> ~/.config/nix/nix.conf <<'EOF'
experimental-features = nix-command flakes
EOF
```

Re-check:

```
nix show-config | rg experimental-features
```

Single-user mode (optional)

When you intentionally avoid the daemon (read Fedora docs for exact package names):

```
# Illustrative - confirm package set on your release:
# sudo dnf install nix --exclude nix-daemon
# or: sudo dnf install nix-singleuser
```

Typical consequences:

- Builds run as your user
- Socket/daemon workflow differs
- Closer to “my laptop only,” farther from NixOS’s always-daemon world

If you plan NixOS soon, **prefer multi-user** so daemon habits match.

Atomic / immutable desktops (Silverblue, Kinoite, bootc)

Immutable systems often make `/nix` **not writable** as a normal directory. Community patterns:

1. Store data under `/var/nix` (writable)
2. **Bind-mount** `/var/nix` $\rightarrow$ `/nix` (Nix rejects a mere symlink for the store root in many setups)

Example systemd mount unit (adapt paths; verify against current Atomic guidance):

```
# /etc/systemd/system/nix.mount (illustrative)
[Unit]
Description=Bind mount /var/nix to /nix

[Mount]
What=/var/nix
Where=/nix
Type=none
Options=bind

[Install]
WantedBy=local-fs.target

sudo mkdir -p /var/nix /nix
sudo systemctl daemon-reload
sudo systemctl enable --now nix.mount
# then install/enable nix packages per Atomic docs
```

Also expect **SELinux** labeling steps on some images. Prefer Fedora documentation and the `nix` package README over blog posts when they disagree.

Verify deeply (acceptance checklist)

```
# Binary + version
nix --version

# Daemon (multi-user)
systemctl is-active nix-daemon

# Store exists and is a directory (not a broken symlink)
findmnt /nix || ls -ld /nix
df -h /nix
```

```
# Flake metadata (network)
nix flake metadata nixpkgs --json | head -c 200; echo

# Build something small and inspect store path
nix build nixpkgs#hello --print-out-paths
nix path-info -r $(nix build nixpkgs#hello --print-out-paths) | head
```

Record versions in your lab notes:

```
mkdir -p ~/lab/nixos-book/nix-on-linux
{
  echo "date: $(date -Iseconds)"
  echo "os: $(. /etc/os-release; echo $PRETTY_NAME)"
  nix --version
  rpm -q nix
  systemctl is-active nix-daemon || echo "daemon inactive"
} | tee ~/lab/nixos-book/nix-on-linux/install-record.txt
```

Upstream installer vs Fedora package

Fedora dnf package	Upstream multi-user script
SELinux integrated for Fedora	Often needs extra steps
Updates via dnf upgrade	Separate Nix upgrade story
Flakes Packaged defaults (enabled OOTB historically)	You enable experimental features
Support for Fedora + Nix communities	Nix upstream
Custom Nix builds	Easier

You can still use **nixpkgs** the same way either way. This book assumes **Fedora packaging** on F44+.

Uninstall / reset notes

If you need a clean slate (lab machine):

```
# Stop daemon first
sudo systemctl disable --now nix-daemon 2>/dev/null || true
# Remove packages
sudo dnf remove nix
# WARNING: deleting the store removes all Nix software
# sudo rm -rf /nix
```

Only wipe `/nix` if you accept redownloading everything.

Common failures

Symptom	Likely cause	Direction
<code>nix: command not found</code>	No login shell / PATH	New login; check <code>profile.d</code>
Cannot connect to daemon	<code>nix-daemon</code> down; permissions	<code>systemctl status nix-daemon</code> ; groups
Flakes experimental error	features not enabled	<code>~/.config/nix/nix.conf</code> or <code>/etc/nix/nix.conf</code>
SELinux denials	Mislabel / non-Fedora install	<code>ausearch -m avc -ts recent</code> ; prefer <code>dnf</code> package
<code>/nix</code> read-only	Atomic without bind mount	Atomic section above
Disk full during build	Store on small root	Move store / free space / GC later chapter

Lab

1. Complete multi-user install on Fedora 44.
2. Capture `install-record.txt` as above.
3. Run `nix run nixpkgs#cowsay -- "fedora+nix"`.
4. Skim `/usr/share/doc/nix/` and note three Fedora-specific warnings in your lab journal.

Checkpoint

- ☐ `nix run nixpkgs#hello` succeeds
 - ☐ Daemon mode intentional (or single-user/rootless documented)
 - ☐ You know where **system** vs **user** `nix.conf` live
 - ☐ Free space plan for `/nix` written down
-

Further depth

- [Store, profiles, GC](#)
- Concepts: [Modern CLI](#), [Flakes](#)

Store, profiles, and GC on Linux

Store, profiles, and GC on Linux

Goal: Operate `/nix/store` confidently on Fedora: see what a **profile** is, what keeps paths alive (**GC roots**), and how to garbage-collect without deleting tomorrow’s work.

This is the same store model as NixOS—only the set of roots differs (no bootloader generations yet).

Why this chapter matters

On Fedora, nothing forces you to be declarative. The fastest way to recreate “npm global hell” is:

```
nix profile install nixpkgs#tool-a
nix profile install nixpkgs#tool-b
# ...six months, no locks, no notes...
```

Learn **roots** early and you will treat profiles as a **small** personal toolkit and flakes as the real project SoT—exactly the habit NixOS rewards.

Store anatomy (quick ops view)

```
ls /nix/store | head
nix path-info nixpkgs#hello
nix path-info -r $(nix build nixpkgs#hello --print-out-paths) | wc -l
```

Idea	Meaning
Store path	<code>/nix/store/<hash>-name</code> content-addressed build result
Closure	Path + all runtime references
GC root	A pointer that says “do not delete this closure”
Profile	User (or system) generation of linked apps—a GC root

Diagram:


```
~/.nix-profile    /nix/var/nix/profiles/per-user/$USER/profile
```

```
generation-42-link    store paths
```

Profiles: what they are for

Inspect

```
nix profile list
readlink -f ~/.nix-profile
ls -l /nix/var/nix/profiles/per-user/$USER/ | tail
```

Install / remove (modern)

```
nix profile install nixpkgs#ripgrep nixpkgs#fd
rg --version
fd --version

nix profile list
nix profile remove ...    # use the index/name from list - check `nix profile remove --help`
```

Upgrade

```
nix profile upgrade
# or reinstall from a pinned flake input instead of floating nixpkgs#
```

Tip

Bridge habit: keep the imperative profile **thin** (editor, rg, direnv). Put project tools in devShells. On NixOS you will barely need `nix profile` at all.

Other roots you will meet

Root kind	Example	Created by
User profile	<code>~/.nix-profile</code>	<code>nix profile</code>
Result symlink	<code>./result</code>	<code>nix build</code> in a directory
direnv	<code>nix-direnv GC roots</code>	<code>use flake + allow</code>
Home Manager	<code>HM profile generations</code>	<code>home-manager switch</code>
NixOS system	<code>/run/current-system</code>	<code>nixos-rebuild (later)</code>

```
# List roots (classic tool still useful)
sudo nix-store --gc --print-roots 2>/dev/null | head -50
# or
nix-store --gc --print-roots 2>/dev/null | head -50
```

Find who pins a path:

```
# After building hello to ./result
nix build nixpkgs#hello
nix-store --query --roots $(readlink -f result) || true
```

Garbage collection

Safe everyday

```
# Delete unreachable paths
nix store gc
# or
nix-collect-garbage
```

Delete old profile generations

```
# User profiles / HM generations (read manpages for exact flags on your version)
nix-collect-garbage -d
# Older alias many still use:
# nix-env --delete-generations old
```

Warning

`nix-collect-garbage -d` is aggressive on **generations**. Fine on a lab account; think twice if you rely on rolling back HM quickly.

Disk pressure playbook

```
df -h /nix
nix path-info -Sh /nix/var/nix/profiles/per-user/$USER/profile 2>/dev/null || true
# Find large paths (nix-tree if installed)
nix run nixpkgs#nix-tree
```

Optional: set free-space goals in `nix.conf` (daemon may need system file):

```
min-free = 1073741824
max-free = 5368709120
```

Channels vs flakes (on Fedora)

Fedora users sometimes enable channels by habit. **This book’s default is flakes + locks.**

Approach	Pinning	Team repro
Channels + <code>nix-env</code>	Floating	Weak
<code>nix profile install</code>	Floating nixpkgs	Weak
<code>nixpkgs#foo</code> without lock		
Project <code>flake.lock</code>	Strong	Strong
HM flake input lock	Strong	Strong

```
# Prefer explicit flake refs in projects
nix flake metadata .
```

You may still see `nix-channel` on old blog posts—literacy only ([Classic CLI](#)).

Binary caches

Default substitutes (`cache.nixos.org`) keep Fedora+Nix usable.

```
nix show-config | rg substituter
# First download of a big stdenv is slow; later hits cache
```

Later: Cachix/Attic in packaging/ops parts. On a laptop, don’t disable substitutes “for purity cosplay” unless studying FODs.

Worked lab: profile vs shell

```
cd ~/lab/nixos-book/nix-on-linux
mkdir -p store-lab && cd store-lab

# A) Ephemeral - disappears when shell exits (no profile pollution)
nix shell nixpkgs#jq --command jq --version

# B) Build root in cwd
nix build nixpkgs#jq
ls -l result
./result/bin/jq --version

# C) Profile (persistent)
nix profile install nixpkgs#jq
command -v jq

# Clean lab build root
rm -f result
nix store gc
```

Write three sentences in your journal: what remained after each of A/B/C.

Checkpoint

- ☐ You can define store path, closure, GC root
 - ☐ `nix profile list` is intentional and small
 - ☐ You can find roots and run `nix store gc` safely
 - ☐ You prefer flake locks over channels for projects
-

Further depth

- [Store and closures](#)
- [Daily CLI and devShells](#)
- NixOS-side roots: [Rebuild and generations](#)

Daily CLI, flakes, and devShells

Daily CLI, flakes, and devShells

Goal: Make Nix on Fedora your **default way to enter project toolchains**—with flakes, locks, `nix develop`, and `direnv`—so the same workflow continues unchanged on NixOS.

Modern CLI map (muscle memory)

Task	Command
Build package	<code>nix build .#pkg / nix build nixpkgs#hello</code>
Run without installing	<code>nix run nixpkgs#cowsay -- hi</code>
Temporary tools	<code>nix shell nixpkgs#jq nixpkgs#yq-go</code>
Project shell	<code>nix develop</code>
Search	<code>nix search nixpkgs ripgrep</code>
Eval	<code>nix eval .#foo</code>
Flake show	<code>nix flake show</code>
Update inputs	<code>nix flake update</code>
Metadata	<code>nix flake metadata</code>

Always prefer **nix** over `nix-env` / `nix-channel` for new work.

First project flake on Fedora

```
mkdir -p ~/lab/nixos-book/nix-on-linux/demo-flake
cd ~/lab/nixos-book/nix-on-linux/demo-flake
git init
```

`flake.nix` (pin philosophy: track **nixos-26.05** to match this book, or `nixpkgs-unstable` if you accept drift—**commit the lock either way**):

```
{
  description = "Fedora bridge demo flake";
```

```

inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

outputs = { self, nixpkgs }:
  let
    system = "x86_64-linux"; # or: builtins.currentSystem cautiously; prefer explicit
    pkgs = import nixpkgs { inherit system; };
  in {
    packages.${system}.default = pkgs.hello;

    devShells.${system}.default = pkgs.mkShell {
      packages = with pkgs; [
        git
        jq
        ripgrep
        # language example - pick what you actually use:
        go_1_24 # adjust attribute to what exists on your pin
        # python3
        # nodejs_22
      ];
      shellHook = ''
        echo "devShell ready on $(uname -sr) - Nix on Linux bridge"
      '';
    };
  };
}

```

i Note

Attribute names like `go_1_24` change across `nixpkgs` pins. Use `nix search nixpkgs go` or `nix repl` to confirm on **your** lock. The pattern matters more than the exact attribute.

```

git add flake.nix
nix flake lock
git add flake.lock
nix develop
# inside shell:
jq --version
rg --version
exit

```

Reproducibility check (second machine / second user):

```
nix develop --offline # after first populate, often works if store warm
```

nix develop vs nix shell vs profile

	Lifetime	Pinned by	Use
nix shell nixpkgs#...	Shell only	Floating unless flake ref	Quick experiments
nix develop	Shell only	flake.lock	Real projects
nix profile install	Until removed	Weak unless you pin	Global personal CLIs

direnv + nix-direnv (high leverage)

Auto-load shells when you `cd` into a project—same on Fedora and NixOS.

Install tools (profile or HM later)

```
nix profile install nixpkgs#direnv nixpkgs#nix-direnv
```

direnvrc

```
mkdir -p ~/.config/direnv
# Prefer path from nix-direnv package; common pattern:
echo 'source $HOME/.nix-profile/share/nix-direnv/direnvrc' >> ~/.config/direnv/direnvrc
```

Shell hook

bash (~/.bashrc):

```
eval "$(direnv hook bash)"
```

zsh (~/.zshrc):

```
eval "$(direnv hook zsh)"
```


Project

```
cd ~/lab/nixos-book/nix-on-linux/demo-flake
echo 'use flake' > .envrc
direnv allow
cd ..
cd demo-flake    # should load shellHook / PATH
```

nix-direnv creates **GC roots** so `nix store gc` won't delete your active shell deps mid-week. Review roots periodically.

Ergonomic extras (optional)

```
nix profile install \
  nixpkgs#nix-output-monitor \
  nixpkgs#nix-tree \
  nixpkgs#nix-your-shell
```

Tool	Role
nom (nix-output-monitor)	Readable build logs (nom develop, etc.)
nix-tree	Why is the store huge?
nix-your-shell	Keep zsh/fish inside nix develop

Example zsh after omz/init:

```
# if nix-your-shell installed
command -v nix-your-shell >/dev/null && nix-your-shell zsh | source /dev/stdin
```

Multi-system flakes (preview)

Laptops vary (x86_64-linux, aarch64-linux). Pattern:

```
outputs = { self, nixpkgs }:
  let
    systems = [ "x86_64-linux" "aarch64-linux" ];
    forAllSystems = f: nixpkgs.lib.genAttrs systems (system: f system);
  in {
```

```
devShells = forAllSystems (system:
  let pkgs = nixpkgs.legacyPackages.${system};
  in {
    default = pkgs.mkShell { packages = [ pkgs.jq ]; };
  });
};
```

Same flake later grows `nixosConfigurations.myhost = ...` without changing how `devShells` work.

CI-shaped local check

```
# Fail if lock missing or eval broken
nix flake check || nix develop -c true
```

When you add packaging later, `checks` become richer ([flake checks](#)).

Lab

1. Convert **one real project** you maintain to `flake.nix` + use `flake direnv`.
 2. Commit `flake.lock`.
 3. On a clean shell, `cd` into the project and confirm tools appear via `direnv`.
 4. Run `nix flake metadata` and paste the lock refreshed time into your journal.
-

Checkpoint

- ☐ You use `nix develop` daily for at least one repo
- ☐ `flake.lock` is committed
- ☐ `direnv` loads without manual `nix develop`
- ☐ Profile remains small

Further depth

- [Flakes and devShells](#)
- [Home Manager standalone](#)
- [direnv chapter in Home layout](#)

Home Manager on Fedora (standalone)

Home Manager on Fedora (standalone)

Goal: Manage your **user** environment declaratively with Home Manager **without NixOS**, so later you only change *how* HM is attached (standalone → NixOS module), not the idea of declarative dots.

Why HM on Fedora

Imperative home	Home Manager
Edit ~/.bashrc in place	Generate from Nix; generations
“Which machine has my aliases?”	Same flake → same home
Drift	home-manager switch is the write path

You still use **Fedora** for the desktop session; HM owns *your* shell, git, editor configs, user packages, and optional user systemd units.

Standalone bootstrap (flake-based)

There are multiple install methods; prefer a **flake** so locks exist.

1. Minimal home flake skeleton

```
mkdir -p ~/src/home-config
cd ~/src/home-config
git init
```

Example flake.nix:

```
{
  description = "Standalone Home Manager on Fedora";

  inputs = {
```

```

nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
home-manager.url = "github:nix-community/home-manager/release-26.05";
home-manager.inputs.nixpkgs.follows = "nixpkgs";
};

outputs = { self, nixpkgs, home-manager }:
  let
    system = "x86_64-linux";
    pkgs = nixpkgs.legacyPackages.${system};
  in {
    homeConfigurations."alice" = home-manager.lib.homeManagerConfiguration {
      inherit pkgs;
      modules = [ ./home.nix ];
    };
  };
}

```

Replace `alice` with your username (whoami). Match **home-manager release** to nixpkgs branch (26.05 with 26.05).

2. home.nix

```

{ pkgs, ... }: {
  home.username = "alice";
  home.homeDirectory = "/home/alice";
  home.stateVersion = "26.05"; # read HM release notes when bumping

  home.packages = with pkgs; [
    ripgrep
    fd
    bat
  ];

  programs.home-manager.enable = true;

  programs.git = {
    enable = true;
    userName = "Alice Example";
    userEmail = "alice@example.com";
  };

  programs.bash = {
    enable = true;
    enableCompletion = true;
    shellAliases = {
      ll = "ls -la";
    };
  };
}

```

```
};
};

# programs.zsh.enable = true; # if you live in zsh
}
```

3. First switch

```
nix flake lock
# Modern invocation (HM versions vary slightly - check `home-manager --help`):
nix run home-manager/release-26.05 -- switch --flake .#alice

# After HM is on PATH via profile:
home-manager switch --flake .#$(whoami)
```

If the attribute is `homeConfigurations.alice`, the flake ref is `.#alice`.

4. Verify

```
home-manager generations
rg --version
git config --get user.email
```

What to put in HM vs project flakes

HM (user global)	Project flake
Shell, prompt, aliases	Language SDKs for <i>that</i> repo
git, gh, editor user config	Formatters/linters pinned to project
Personal CLI (rg, fd, bat)	Heavy toolchains (JDK, Android SDK)
User systemd timers	CI-only tools

Generations and rollback

```
home-manager generations
# switch to older generation if a bad config lands
home-manager switch --flake .#alice -n # dry-run when supported
```

This is the **user-level** analogue of NixOS boot generations. Learning it now removes fear later.

Secrets warning

HM can write files into the store. **Never:**

```
home.file.".ssh/id_rsa".text = "-----BEGIN ..."; # DO NOT
```

Use live files outside the store, or sops/age patterns (later book parts). Prefer `home.file.*.source` only for non-secret text you accept in the store.

Coexistence with Fedora's bashrc

When `programs.bash.enable = true`, HM manages your bash config. Avoid fighting both:

- Either HM owns `~/.bashrc`
- Or use `programs.bash.initExtra` to append and stop hand-editing

Same for `zsh`.

Lab

1. Create `~/src/home-config` with flake + `home.nix`.
 2. Manage git + three CLI tools via HM.
 3. Break an alias on purpose, `switch`, then roll back a generation.
 4. Write a note: “On NixOS I will attach this module via `home-manager.users....`”
-

Checkpoint

- ☐ `home-manager switch --flake ...` succeeds on Fedora
 - ☐ Generations list is non-empty
 - ☐ No secrets in the store from HM
 - ☐ You can explain standalone vs NixOS module HM
-

Further depth

- [Home Manager \(book part\)](#)
- [HM files and XDG](#)
- [Bridge to NixOS](#)

dnf and Nix coexistence

dnf and Nix coexistence

Goal: Run a calm dual-stack system: Fedora’s **dnf** (or rpm-ostree) owns the OS; Nix owns portable toolchains—without PATH wars, duplicate daemons, or split-brain config.

PATH reality

Order matters:

```
echo "$PATH" | tr ':' '\n' | head -20
type -a rg
type -a git
```

Typical conflict:

Tool	Fedora provides	Nix provides	Risk
git	/usr/bin/git	profile/HM	Different con-figs/behavior
python3	system Python	devShell Python	Shebangs
podman	OS integration	rare via nix	Prefer OS for containers on Fedora

Policy examples:

```
# Conservative dual-stack
OS: kernel, desktop, browsers (flatpak/dnf), podman, firmware
Nix: language toolchains, specialized CLIs, project shells
HM: dotfiles + personal CLIs that don't need OS integration
```

Decision matrix (printable)

Question	If yes →
Needs kernel module / udev / SELinux policy?	dnf
Needed identically on CI and laptop?	Nix flake
GUI app with portals/Flatpak better?	Flatpak or dnf
Only you need it in \$HOME?	HM or profile
System service on this Fedora host long-term?	dnf/systemd for now; NixOS modules later
Experiment for a weekend?	nix shell

Avoid dual service managers for the same app

Bad: nginx from dnf **and** experiments with Nix-built nginx on same ports.

```
ss -lntp | rg ':80|:443' || true
systemctl status nginx 2>/dev/null || true
```

Pick **one** owner per service on a given host.

Language version managers vs Nix

Tool	Overlap with Nix
nvm / fnm	devShell nodejs
pyenv	devShell python3
sdkman	devShell JDK
rustup	Often keep rustup <i>or</i> use nixpkgs rust—pick one story per project
goenv	devShell go

Migration pattern per repo:

1. Add flake **devShell** with tools
2. Document in README: **direnv allow**
3. Remove project-local **.nvmrc** only when team agrees

Graphics / Electron / browsers

Nix can build browsers; on Fedora daily drivers many people still use **Fedora/Flatpak** browsers for VA-API, Widevine, and portal integration. That’s fine. Use Nix where reproducibility beats integration.

Disk layout awareness

```
df -h / /nix /home
```

If / is small (Atomic, small root), ensure /nix lives on a large filesystem (bind mount from /var or secondary disk). dnf and Nix both cache—budget both.

Updates: two clocks

Stream	Command
Fedora	<code>sudo dnf upgrade</code>
Nix profile	<code>nix profile upgrade</code> / rebuild from flake
HM	<code>home-manager switch --flake ...</code>
Flake inputs	<code>nix flake update</code> + PR/review lock

Stagger upgrades when debugging (“did dnf or flake break me?”).

Lab

1. Produce a table of 15 tools on your PATH and label each **dnf** / **flatpak** / **nix** / **other**.
 2. Move two development tools from global installers into a project flake.
 3. Ensure `type -a` shows the intended binary first.
-

Checkpoint

- ☐ Written ownership policy for OS vs Nix
 - ☐ No accidental duplicate core services
 - ☐ PATH inspected and understood
 - ☐ Update rituals documented
-

Further depth

- [SELinux, Atomic, ops](#)
- [Packages on NixOS](#) (system layer later)

SELinux, Atomic, and day-2 ops

SELinux, Atomic, and day-2 ops

Goal: Keep Fedora’s **security and immutability** model while running Nix—without disabling SELinux—and operate the store like an adult (disk, daemon, upgrades).

SELinux: do not “fix” Nix by disabling it

Fedora’s packaged Nix is built to work under **enforcing** SELinux. Prefer:

```
getenforce
sudo ausearch -m avc -ts recent 2>/dev/null | tail -20
```

If you see denials after a non-Fedora install method, **switch to the Fedora package** before opening policy holes.

Do	Don't
<code>dnf install nix</code>	<code>setenforce 0</code> as permanent fix
Read <code>/usr/share/doc/nix/</code>	Copy random <code>chcon -R</code> from old blog posts without understanding
File bugs against Fedora nix package when policy is wrong	Disable SELinux system-wide

Atomic desktops and bootc

Problem

Image-based systems often make `/usr` immutable and restrict creating `/nix` on root.

Pattern A — bind mount

```
sudo mkdir -p /var/nix
# ensure /nix exists as mount point
sudo mkdir -p /nix
```

systemd `.mount` unit bind `/var/nix` → `/nix` (see [Install chapter](#)). Then install Nix packages and enable daemon **if** supported on that image.

Pattern B — rootless / pure user Nix

Fedora packaging notes mention **pure rootless** mode useful on Atomic. Follow current `README.fedora.md`—flags and subpackages change.

Pattern C — Nix in a toolbox/distrobox

Some Atomic users keep the host minimal and run a **Fedora toolbox** with classic `/nix` for heavy dev. Tradeoff: another layer; still teaches Nix well.

```
# illustrative
toolbox create -i fedora-toolbox:44 nixdev
toolbox enter nixdev
# then dnf install nix inside toolbox if appropriate
```

Day-2 operations checklist

Daemon health

```
systemctl status nix-daemon --no-pager
journalctl -u nix-daemon -b --no-pager | tail -40
```

Disk

```
df -h /nix
nix store gc
# optional deep clean of old generations
# nix-collect-garbage -d
```

Trust and config

```
nix show-config | rg -n 'substituter|trusted|sandbox|experimental'
```

Upgrade Nix itself (Fedora)

```
sudo dnf upgrade nix
sudo systemctl restart nix-daemon
nix --version
```

Flake-locked **nixpkgs** upgrades are separate from upgrading the **Nix binary**.

Networking / proxies / corporate TLS

If substitutes fail:

```
curl -I https://cache.nixos.org
# corporate MITM: may need custom CA trust for Nix - advanced; document carefully
```

Backup thoughts

Path	Back up?
flake.nix / flake.lock / HM repo	Yes — this is the real config
/nix/store	Optional; can rebuild from locks (slow)
Profile generation list	Nice; not required if flakes exist

On NixOS you will also back up **host flake** + secrets—not the entire store.

Performance tips (laptop)

- Keep direnv projects few and GC occasionally
- Use `nix-output-monitor` for readability, not speed
- Prefer substitutes; local compiles of browsers hurt battery
- `nix settings` / `max-jobs` and `cores` can be tuned in `nix.conf`

```
# example user conf - adjust to CPU
# max-jobs = auto
# cores = 0
```

Daemon may need matching system conf for multi-user.

Lab

1. With SELinux enforcing, run `nix run nixpkgs#hello` and confirm no new AVCs.
 2. Measure `df -h /nix` before/after `nix develop` on a heavy shell; run `nix store gc`.
 3. If on Atomic, document your `/nix` strategy in the lab charter.
-

Checkpoint

- ☐ SELinux remains enforcing
 - ☐ Daemon healthy after reboot
 - ☐ Disk budget monitored
 - ☐ Atomic path documented if applicable
-

Further depth

- [GC hygiene \(fleet part\)](#)
- [Troubleshooting](#)

Bridge to NixOS

Bridge to NixOS

Goal: Convert Fedora+Nix habits into a first **NixOS** host with minimal conceptual shock—reuse flakes, Home Manager knowledge, and store literacy.

What stays the same

You already do on Fedora	On NixOS
<code>/nix/store</code> , closures, GC roots	Identical store model
<code>nix build / run / develop</code>	Identical
<code>flake.lock</code> discipline	Identical (+ host config in same monorepo often)
Home Manager modules for git/shell	Same options; different wiring
<code>direnv</code> + project flakes	Identical

What is new

New concept	Meaning
<code>nixosConfigurations</code>	Flake output describing whole machines
Module system	<code>services.openssh.enable = true;</code> merges options
<code>nixos-rebuild switch --flake</code>	Activate system generation
Bootloader generations	Roll back from GRUB/systemd-boot
<code>environment.systemPackages</code>	System-wide packages (not just profile)
Systemd system units via Nix	Beyond user HM services
Disko / fileSystems	Disk layout as code



<code>dnf upgrade</code>	<code>nixos-rebuild (flake pin)</code>
<code>/etc</code> hand edits	modules in git
<code>~/.config</code> via HM standalone	HM standalone OR nixos module
nix profile thin toolkit	systemPackages + HM

Recommended transition shapes

A — Keep Fedora laptop; NixOS server/lab (common)

- Laptop: this whole **Nix on Linux** part forever
- Lab VM/metal: NixOS host part
- Share one git repo:

```
flake.nix
  devShells
  homeConfigurations.alice    # Fedora laptop
  nixosConfigurations.labbox  # NixOS
```

B — Dual boot / spare disk NixOS

- Practice rebuild/rollback risk-free
- Port HM modules first; add system modules gradually

C — Full cutover

- Only after: flake host builds in VM, secrets story exists, backup restore tested
-

Porting checklist (HM → NixOS)

1. **Keep** `programs.*` and `home.packages` modules almost as-is.
2. **Move** OS concerns out of HM: sound, printing, kernel, networking—use NixOS modules.
3. **Wire** HM as NixOS module:

```
# sketch - details in Home layout / host parts
{
  inputs.home-manager.nixosModules.home-manager
  {
    home-manager.useGlobalPkgs = true;
    home-manager.useUserPackages = true;
    home-manager.users.alice = import ./home.nix;
  }
}
```

4. **Replace** nix profile tools with `environment.systemPackages` or HM packages deliberately.
5. **Retire** dnf habits for software you now declare—keep mental note of firmware/kernel still “special.”

First NixOS learning loop (preview)

Do this on a **VM** (Lab 0), not your only laptop:

1. Install NixOS (graphical or minimal) — [Install](#)
2. Put host in a flake — [Flake host](#)
3. `nixos-rebuild switch --flake .#hostname`
4. Break sshd/firewall intentionally; roll back a generation — [Rebuild and generations](#)
5. Import your HM config

Success metric: **wipe VM disk, rebuild from flake + backup secrets, services return.**

Dual-stack monorepo sketch

```
{
  description = "Machines and homes";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    home-manager.url = "github:nix-community/home-manager/release-26.05";
```



```

    home-manager.inputs.nixpkgs.follows = "nixpkgs";
  };

  outputs = { self, nixpkgs, home-manager, ... }: {
    homeConfigurations.alice = /* Fedora path */;
    nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      modules = [ ./hosts/lab/configuration.nix ];
    };
    devShells.x86_64-linux.default = /* shared */;
  };
}

```

One mental model: **outputs** are different products from the same factory.

Skills you should already have (gate)

Before NixOS host part, from this bridge path:

- ☐ Nix installed via Fedora (or documented equivalent)
 - ☐ Flakes + lock for a real project
 - ☐ direnv workflow
 - ☐ HM standalone generation/rollback
 - ☐ Store GC literacy
 - ☐ Written dnf vs Nix policy
-

Lab

1. Draft `hosts/lab/configuration.nix` on paper or in git with only: hostname, user, ssh, firewall—**without** installing yet.
 2. List HM modules you will import unchanged.
 3. Schedule a VM install using Lab 0 chapters.
-

Checkpoint

- ☐ You can list what changes and what does not when moving to NixOS
 - ☐ You have a transition shape (A/B/C) written down
 - ☐ You are ready for part 03-nixos-host/ without relearning store/flakes
-

Further depth

- [NixOS host index](#)
- [Flake layout](#)
- [Home Manager](#)
- Capstone rebuild proof later: 09-capstone/

NixOS Host

NixOS host — install, configure, modules

NixOS host — install, configure, modules

This part of the **standalone NixOS book** is organized by **topic** (normal book chapters), not by calendar day.

Read chapters in order within the part when learning; skip or skim freely when using the book as a reference.

What this part is for

NixOS host turns the Concepts toolkit into **machine ownership**: install a disposable NixOS system, manage it with a flake (**nixosConfigurations**), rebuild and roll back safely, then configure users, network, packages, systemd, modules, boot, and filesystems.

You leave this part with a lab host whose **source of truth is a git flake**, not ad-hoc edits under `/etc/nixos` alone.

Previous part	This part	Next part
Concepts — language, store, flakes	Install + configure one host	Home Manager, flake layout, lock discipline

Baseline (this book)

Piece	Baseline
NixOS / nixpkgs	26.05 “Yarara”
Nix	2.34.x with <code>nix-command</code> + flakes
Host management	<code>sudo nixos-rebuild switch --flake .#<host></code>
Install target	Disposable VM preferred (not daily-driver only disk)

Learning outcomes

By the end of this part you should be able to:

1. Install NixOS 26.05 (UEFI-oriented) and document hardware/boot layout
2. Define `nixosConfigurations.<name>` and rebuild from a flake repo
3. Choose `switch` / `test` / `boot` / `build` and **rollback** generations
4. Declare users, SSH keys, and safe remote access habits
5. Configure networking and an explicit firewall that still allows admin access
6. Prefer `environment.systemPackages` vs `programs.*` deliberately
7. Ship custom **systemd** services and timers via Nix
8. Consume modules via options (`mkIf` / `mkDefault` literacy)
9. Write a small **custom module** with options, `mkIf`, and assertions
10. Document boot loader, kernel, and firmware choices
11. Inventory filesystems and classify **store vs state** (impermanence *idea*)

Prerequisites

Required	Strongly recommended
Concepts part (or equivalent fluency)	Working flake + modern CLI habits
Hypervisor or spare machine	VM snapshots before boot/network experiments
Git for the host config repo	Console access for lockout recovery drills

If you skipped Concepts, at least complete: store/closures, functions, flakes + devShell, modern CLI.

Chapter map

Chapter	Title	Focus
Installing NixOS	ISO, UEFI, partition, first boot, snapshots	
Flake-defined hosts	<code>nixosConfigurations</code> , repo layout, first switch	
Rebuild and generations	Modes, profile gens, rollback card	
Users and access	wheel, SSH keys, password hygiene	
Networking and firewall	networkd/NM, ports, DNS, safety patterns	
Packages and programs	systemPackages vs programs.*, closure discipline	
systemd via Nix	units, timers, absolute paths, inspect	
Modules as consumer	options, merge, mkIf/mkDefault, reading source	
Custom modules	options + config, assertions, namespaces	
Boot and hardware	loader, kernel, firmware, recovery	
Filesystems and state	mounts, persist lists, store data	

Suggested learning paths

Path	Order	Notes
Linear (recommended)	01 → 11	Each chapter assumes the flake host exists
Already installed	Start at 02; skim 01 for inventory	Still document hardware
Ops recovery focus	03, 04, 05, 10, 11	After a minimal flake host works
Module author track	08 → 09 after 02–07	Needs a living host config

Architecture sketch (part-level)

```
git repo: ~/nixos-config
flake.nix → nixosConfigurations.lab = nixosSystem { modules = [ ... ] }
hosts/lab/configuration.nix
hosts/lab/hardware-configuration.nix
modules/*.nix

sudo nixos-rebuild switch --flake .#lab

evaluate modules → system derivation → realize → activate

/nix/var/nix/profiles/system (generations + boot entries)
```

Safety rules for this part

1. Prefer a **VM** with snapshots before firewall, boot, or SSH lockout drills
 2. Keep **console** access until key-based SSH is proven
 3. Use `nixos-rebuild **test**` for risky network/firewall changes, then `switch`
 4. Never put private keys or live passwords in the flake
 5. Treat `hardware-configuration.nix` UUIDs as machine-specific truth
 6. Remember: **rollback restores system closure, not /var databases**
-

Lab workspace convention

On the lab host (or mirrored on your workstation):

```
mkdir -p ~/nixos-config/{hosts/lab,modules,docs}
cd ~/nixos-config
git init
```

Rebuild command of record:


```
sudo nixos-rebuild switch --flake .#lab
```

Document hostname, flake attr, and hypervisor network mode in `README.md`.

Part gate (before Home / flake layout)

- ☐ Disposable NixOS **26.05** lab boots
 - ☐ Flake `nixosConfigurations.<host>` is source of truth
 - ☐ Successful `switch --flake` and a documented **rollback** path
 - ☐ SSH key admin user; no private keys in git
 - ☐ Firewall enabled with explicit SSH allow
 - ☐ Daily tools declared (not only `nix profile`)
 - ☐ At least one custom systemd unit and one custom module
 - ☐ Boot + filesystem docs exist; store vs state explained
-

Further depth (this book only)

After this part...	Go to
Home Manager, direnv, flake-parts	Part Home and flake layout
sops, hardening, services, TLS	Part Services and security
Disko, impermanence, deploy, tests	Part Ops and fleet
Module system internals	Part Internals
Capstone multi-service host	Part Capstone / Projects

How to use this part as a reference

- Jump via chapter titles when debugging (“firewall”, “generations”, “custom modules”)
- Keep a **rollback card** (`docs/rollback.md`) updated as you learn boot and SSH
- Prefer options on search.nixos.org with **26.05** selected
- Do not link or depend on any external day-numbered curriculum; this book is self-contained

When later parts say “on your lab host,” they mean the machine and flake built here.

Installing NixOS

Installing NixOS

Goal: Install a **disposable** NixOS **26.05 “Yarara”** lab system (VM strongly preferred), boot it, capture hardware config notes, and prepare for flake-based management in the next chapter.

! Important

Do **not** experiment on your only daily-driver disk. Use a VM, spare machine, or spare disk. Reinstall without drama is a feature.

Why this chapter matters

Generations, rollback, modules, and firewall only become real on a host. Concepts was the package manager + language. This part is **owning a machine** as code—starting with a clean install.

Ops motivation: every production NixOS story begins with a first generation that boots. Install literacy (UEFI, ESP, generate-config, credentials) is how you recover when experiments go wrong.

Theory 1 — What you are installing

Piece	Role
NixOS	Linux distribution built from Nix modules
nixpkgs 26.05 “Yarara”	Package + module set baseline for this book
Installer image	Live environment to partition + install
First generation	Initial system closure after install

After install you have a bootable system. **Flake ownership** is the flake hosts chapter—this chapter can use installer defaults, but mentally prepare to migrate immediately.

Release identity

```
# after install
nixos-version
cat /etc/os-release
```

Confirm **26.05**, not accidental unstable, when following this book’s baseline.

Theory 2 — Disposable VM emphasis

Host choice	Recommendation
VM (QEMU/KVM, UTM, VirtualBox, VMware, ...)	Preferred
Spare laptop/NUC	Good
Dual-boot daily driver	Risky for learning
Cloud VM	Fine if you accept networking/SSH from the start

VM resources (practical)

Resource	Starting point
RAM	4–8 GB
Disk	40–64 GB+
CPU	2+ cores
Firmware	UEFI if available
Snapshot	Take one after first successful boot

Snapshots turn “I broke GRUB” into “restore 30 seconds.”

Networking modes (hypervisor)

Mode	Use
NAT	Simple outbound; port-forward for SSH
Bridged	LAN IP like a physical host
Host-only	Isolated lab segment

Record the mode in notes—the networking chapter needs it.

Theory 3 — UEFI vs legacy BIOS

Mode	Common loader on NixOS
UEFI	systemd-boot (often default) or GRUB EFI
BIOS/CSM	GRUB

Prefer UEFI for new labs. Note which you used—the boot and hardware chapter returns to boot loaders.

ESP requirements

Need	Typical
ESP size	512 MB–1 GB FAT32
Mount during install	<code>/mnt/boot</code>
Variables	<code>canTouchEfiVariables</code> often true on real UEFI

Theory 4 — Disk layout options

Manual (classic installer path)

1. Partition (ESP + root, optional swap)
2. Format
3. Mount at `/mnt`, ESP at `/mnt/boot`
4. `nixos-generate-config --root /mnt`
5. Edit `configuration.nix`
6. `nixos-install`

Disko (intro only)

Disko declares disks as code. Full Disko redesign is in the **Ops and fleet** part. In this chapter:

- Know Disko exists
- Optionally follow a **simple** Disko example if you already know it

- Otherwise **manual once** is fine and educational

Do not block install on learning Disko deeply here.

Suggested simple layout (UEFI)

```
ESP (vfat) 512MB-1GB → /boot
root (ext4 or btrfs) → /
swap (optional)
```

Why not exotic FS on first install?

ZFS/btrfs fancy layouts are powerful and easy to misconfigure. Prefer **boring** for the first bootable lab; redesign later with Disko when you understand state.

Theory 5 — nixos-generate-config outputs

File	Role
hardware-configuration.nix	Detected devices, fileSystems, initrd modules
configuration.nix	High-level system config (you own this)

Rules of thumb

- Treat hardware-configuration.nix as machine-specific; rarely hand-merge casually
- Put policy (users, services, packages) in configuration.nix / modules
- Next chapter both get imported from a flake

What generate-config is good at

Captures well	Does not invent for you
Disk UUIDs	Hostname policy
Initrd modules for disks	Firewall policy
Basic boot bits	SSH keys

Theory 6 — Installer networking and time

Install needs:

- Network for substitutes (unless offline media strategy)
- Correct clock helps TLS

```
# live environment examples (names vary)
ip a
ping -c 2 1.1.1.1
timedatectl 2>/dev/null || date
```

Broken DNS in the live environment looks like “NixOS install is broken.”

Theory 7 — First boot mental model

After `nixos-install` and reboot:

```
bootloader → kernel+initrd → stage-1/2 → systemd → multi-user
system profile: /nix/var/nix/profiles/system
generation N is current
```

You can log in as root or the user you created. **Prefer SSH key setup ASAP** (users chapter formalizes); avoid building habits around password-only remote root.

Activation vs “files on disk”

The installed system is a **closure** activated into place—not a traditional unpack of a rootfs tarball only. Generations will make that concrete in the rebuild chapter.

Theory 8 — Credentials hygiene on a lab

Practice	Why
Unique lab password	Compromise isolation
Note in password manager	Labs still get attacked if exposed
Prefer host-only networking until ready	Reduce drive-by SSH noise
Snapshot before experiments	Fast recovery
Temporary <code>initialPassword</code> only	Rotate; prefer keys

Practice	Why
----------	-----

Theory 9 — `system.stateVersion`

```
system.stateVersion = "26.05";
```

This is **not** “current release tracking.” It tells NixOS which stateful defaults to preserve for compatibility. Set it to the release you **installed**; read release notes before ever changing it.

Theory 10 — Minimal install config knobs

Focus during first install:

Knob	Why
Boot loader enable	Must boot
Hostname	Identity
User + wheel	Admin path
Optional SSH	Remote lab convenience
Timezone	Sanity
Experimental features	Smoother flake migration

Defer fancy services until the flake is source of truth.

Worked examples bank

Example A — Download image

Use official docs for current ISO:

- <https://nixos.org/download/>

Pick **26.05** graphical or minimal ISO matching your arch (`x86_64`, `aarch64`).

Verify checksums when provided.

Example B — Partition sketch (sfdisk/parted-level ideas)

```
/dev/vda1  EFI System  FAT32  /boot
/dev/vda2  Linux root   ext4   /
```

Device names: vda (virtio), sda, nvme0n1—use what the live system sees (lsblk).

Example C — Generate + install skeleton

```
# after mounts ready
sudo nixos-generate-config --root /mnt
sudo nano /mnt/etc/nixos/configuration.nix
sudo nixos-install
# set root password when prompted
```

Example D — Minimal configuration knobs to set *during* install

```
{
  imports = [ ./hardware-configuration.nix ];

  boot.loader.systemd-boot.enable = true;
  boot.loader.efi.canTouchEfiVariables = true;

  networking.hostName = "lab";
  time.timeZone = "UTC"; # set yours

  users.users.alice = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];
    # initialPassword only for offline lab convenience - rotate later
    initialPassword = "changeme";
  };

  services.openssh.enable = true; # if you want SSH into VM
  nix.settings.experimental-features = [ "nix-command" "flakes" ];

  system.stateVersion = "26.05"; # match release; do not casually change later
}
```

Exact options must match your installer generation—adapt from generated file rather than blind paste.

Example E — Mount checklist before generate-config

```
lsblk -f
findmnt /mnt
findmnt /mnt/boot
# ESP must be mounted or boot will fail later
```

Lab 1 — Plan and snapshot strategy

Write in `~/lab/nixos-book/host/install/NOTES.md` (on the **workstation**):

1. Hypervisor / hardware choice
 2. UEFI or BIOS
 3. Disk size
 4. Hostname
 5. Username
 6. Networking mode (NAT/bridge)
 7. Snapshot plan
-

Lab 2 — Install NixOS 26.05

1. Boot installer ISO
2. Partition + format + mount
3. `nixos-generate-config --root /mnt`
4. Edit config: hostname, user, boot loader, optional SSH
5. `nixos-install`
6. Reboot into installed system
7. Take a **VM snapshot** labeled `nixos-26.05-fresh-install`

If install fails, journal the error class (bootloader, mount, network, disk).

Lab 3 — Post-install inventory

On the new system:

```
nixos-version
uname -a
lsblk -f
cat /etc/os-release
ls /etc/nixos/
```

Copy off-box (screenshot/scp) or retype into host NOTES.md:

- nixos-version output
 - Disk layout
 - Boot loader in use
 - IP address if any
-

Lab 4 — Preserve hardware config

```
# on NixOS lab
sudo cp -a /etc/nixos /root/nixos-etc-backup-$(date +%F)
```

On workstation notes, record that the flake hosts chapter will create a **flake** and import `hardware-configuration.nix`.

Optional: already enable flakes in system config for a smoother next chapter:

```
nix.settings.experimental-features = [ "nix-command" "flakes" ];
```

Then `sudo nixos-rebuild switch` once if you edit on the running system.

Lab 5 — Break-glass familiarity

Without destroying the snapshot casually:

1. Locate boot menu / how to pick previous generation (may only have gen 1)
 2. Confirm you can log in on console
 3. If SSH enabled, log in from host once
-

Exercises

1. Compare minimal vs graphical ISO trade-offs for your lab.
 2. Document exact `lsblk` before and after partitioning.
 3. Practice mounting ESP incorrectly in a throwaway VM; see the failure mode (snapshot first).
 4. List every password/key created; store only in a password manager.
 5. Write a 10-step reinstall runbook from your real commands.
 6. Note substitute traffic during install (why network mattered).
 7. Identify `system.stateVersion` in your config; explain it to a peer.
 8. Capture `nixos-generate-config` diffs if you re-run after disk changes (careful).
 9. Time a full install; note where you waited on downloads.
 10. Sketch Disko-shaped layout for the *same* partitions (no implement required).
 11. Verify checksum of the ISO you downloaded.
 12. Confirm experimental features enabled before the flake chapter.
-

Common pitfalls

Symptom	Theory
No bootable device	ESP not mounted at <code>/mnt/boot</code> or wrong firmware mode
Install can't reach cache	Network/DNS in live env
Forgot root password	Live ISO recovery / reinstall—why snapshots matter
<code>hardware-configuration</code> wrong root UUID	Generated before final mounts
Using unstable ISO by mistake	Check 26.05 label
Daily driver install “for fun”	Restore from backup; prefer VM next time
Both GRUB and systemd-boot enabled	Loader conflict
Tiny disk	Store fills during first real use

Checkpoint

- ☐ Disposable NixOS 26.05 lab boots
- ☐ User + privilege path known (root/wheel)
- ☐ Disk + boot mode documented
- ☐ `/etc/nixos` inspected
- ☐ VM snapshot taken (if VM)
- ☐ Notes stored on workstation
- ☐ Ready to migrate to a flake repo next

Further depth (this book)

Topic	Chapter
Flake as SoT	Flake-defined hosts
Generations	Rebuild modes and generations
Boot details	Boot and hardware
Disks as code	Ops part: Disko
Filesystems / state	Filesystems and state

Flake-defined hosts

Flake-defined hosts

Goal: Manage the lab host with a **flake** (`nixosConfigurations`) from first ownership—`nixos-rebuild switch --flake .#<host>` is the source of truth.

💡 Tip

Do not invest weeks in non-flake `/etc/nixos` only to rewrite. Migrate now while the config is small.

Why this chapter matters

A NixOS system without a flake pin drifts like a channel laptop. Flakes give you:

- Locked nixpkgs (and later inputs)
- Named hosts in one repo
- A rebuild command you can document and automate

Ops motivation: the host becomes a git review unit. Incident response is “what changed in the flake?” not “what did someone type on the box?”

Theory 1 — `nixosConfigurations` output

```
outputs = { self, nixpkgs, ... }: {
  nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
    system = "x86_64-linux";
    modules = [
      ./hosts/lab/configuration.nix
      ./hosts/lab/hardware-configuration.nix
    ];
  };
};
```


Piece	Role
<code>nixosSystem</code>	Module system entry for a host
<code>system</code>	CPU-OS triple for packages
<code>modules</code>	List of NixOS modules (files/functions)
<code>attr name lab</code>	Host id in <code>.#lab</code>

Rebuild:

```
sudo nixos-rebuild switch --flake /path/to/repo#lab
# or from repo:
sudo nixos-rebuild switch --flake .#lab
```

Evaluation sketch

```
flake inputs (locked nixpkgs)

nixosSystem { modules }

merged config → system derivation (toplevel)

realize + activate → new generation
```

Theory 2 — Where the flake lives

Location	Pros	Cons
<code>/etc/nixos/flake.nix</code>	Simple path	Mixed with generated hardware; git harder
<code>/home/you/nixos-config</code> git repo	Proper history	Need path on rebuild
Separate git + copy	Flexible	Sync discipline

Recommended for this book: git repo in your home (or shared path), rebuild with explicit flake path. Optionally symlink strategy later.

Sudo and paths

Under `sudo`, `.` may not be where you think. Prefer:

```
sudo nixos-rebuild switch --flake /home/alice/nixos-config#lab
```

or always `cd` to the repo with a documented absolute path.

Theory 3 — Module list composition

```
modules = [
  ./hardware-configuration.nix
  ./configuration.nix
  ./modules/ssh.nix      # later
];
```

Each module is typically:

```
{ config, lib, pkgs, ... }: {
  # options / config
}
```

Or a plain attrset in simple files. Modules-as-consumer and custom-modules chapters deepen this; here: **imports work**.

imports inside configuration.nix

```
{
  imports = [
    ./hardware-configuration.nix
    ../../modules/common.nix
  ];
}
```

Either list modules in `flake.nix` or nest via `imports`—pick a convention and stick to it.

Theory 4 — Minimal flake host contents

flake.nix

```
{
  description = "lab flake host";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }: {
    nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux"; # match your VM
      modules = [
        ./hosts/lab/configuration.nix
        ./hosts/lab/hardware-configuration.nix
      ];
    };
  };
}
```

hosts/lab/configuration.nix

```
{ config, lib, pkgs, ... }: {
  networking.hostName = "lab";

  nix.settings.experimental-features = [ "nix-command" "flakes" ];

  boot.loader.systemd-boot.enable = true;
  boot.loader.efi.canTouchEfiVariables = true;

  time.timeZone = "UTC";

  users.users.alice = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];
  };

  environment.systemPackages = with pkgs; [ vim git ];

  services.openssh.enable = true;

  system.stateVersion = "26.05";
}
```

`hardware-configuration.nix`

Copy from the machine's generated file—**do not invent UUIDs**.

Theory 5 — First rebuild flow

```
edit flake modules
↓
nixos-rebuild switch --flake .#lab
↓
eval modules → system derivation → realize → activate
↓
new system generation
```

Outcome	Meaning
Success	New generation is current
Eval error	Fix Nix before build
Build error	Package/build failure
Activation error	System may be partially switched—read output carefully

Safe preview builds

```
nixos-rebuild build --flake .#lab
nix build .#nixosConfigurations.lab.config.system.build.toplevel
```

Build without activating when you want a cheap check.

Theory 6 — `specialArgs` / `extraSpecialArgs` (preview)

```
nixpkgs.lib.nixosSystem {
  specialArgs = { inherit inputs; };
  modules = [ ... ];
}
```

Used later to pass flake inputs into modules. Not required for the minimal host.

Theory 7 — Hostname vs flake attribute

Name	Where
<code>networking.hostName</code>	OS hostname
<code>nixosConfigurations.lab</code>	Flake attribute

They **may** match but are not required to. Document both in README.

Theory 8 — Git discipline for hosts

Commit	Why
<code>flake.nix</code> + modules	Policy history
<code>flake.lock</code>	Pin truth
<code>hardware-configuration.nix</code>	Machine facts (lab OK to commit)

Do **not** commit secrets. Public SSH keys are fine; private keys are not.

Flake purity reminder

Untracked module files may not appear in evaluation—`git add` them.

Theory 9 — Multiple hosts (preview)

```
nixosConfigurations = {  
  lab = nixpkgs.lib.nixosSystem { ... };  
  # laptop = nixpkgs.lib.nixosSystem { ... };  
};
```

Share modules via `modules/common.nix`. Fleet tooling comes in Ops part; structure starts now.

Theory 10 — Leaving installer `/etc/nixos` behind

Approach	Notes
Ignore <code>/etc/nixos</code> after migration	Flake repo is SoT
README warning	Prevent future-you editing wrong tree
Optional pointer file	Comment-only reminder

Avoid maintaining two competing configs.

Worked examples bank

Example A — Build without switching (safe)

```
nixos-rebuild build --flake .#lab
# or
nix build .#nixosConfigurations.lab.config.system.build.toplevel
```

Example B — Switch

```
sudo nixos-rebuild switch --flake .#lab
```

Example C — Show config value

```
nix eval .#nixosConfigurations.lab.config.networking.hostName
```

Example D — Show system path

```
nix eval --raw .#nixosConfigurations.lab.config.system.build.toplevel
```

Example E — Trace eval errors

```
nixos-rebuild build --flake .#lab --show-trace
```

Lab 1 — Repository layout on the lab host

```
mkdir -p ~/nixos-config/hosts/lab
cd ~/nixos-config
git init
```

```
sudo cp /etc/nixos/hardware-configuration.nix ~/nixos-config/hosts/lab/
sudo chown "$USER":"$USER" ~/nixos-config/hosts/lab/hardware-configuration.nix
```

Write `flake.nix` + `hosts/lab/configuration.nix` (adapt user, system, boot).

```
git add .
nix flake lock
git add flake.lock
git commit -m "initial nixosConfigurations.lab"
```

Lab 2 — First flake rebuild

```
cd ~/nixos-config
sudo nixos-rebuild switch --flake .#lab
```

Verify:

```
nixos-version
hostname
nix eval .#nixosConfigurations.lab.config.networking.hostName
```

Lab 3 — Prove flake is authority

1. Change a harmless package in `environment.systemPackages` (add `htop`).
2. Rebuild switch.
3. Confirm `htop` available.
4. Commit.

Avoid editing `/etc/nixos/configuration.nix` as the long-term path—if installer files remain, leave a comment in README: **flake repo is SoT**.

Lab 4 — README for future you

README.md:

```
# nixos-config

## Rebuild
sudo nixos-rebuild switch --flake .#lab

## Host
- flake attr: lab
- hostname: lab
- nixpkgs: 26.05 (see flake.lock)
```

Lab 5 — Optional: match /etc/nixos to flake

Some prefer keeping only a stub under /etc/nixos. Simplest reliable approach:

```
cd ~/nixos-config && sudo nixos-rebuild switch --flake .#lab
```

Do **not** create circular confusion between two live configs.

Exercises

1. Eval three config values with `nix eval` (hostname, stateVersion, openssh enable).
2. Break a module path on purpose; fix via `--show-trace`.
3. Add a second module file for packages only; import it.
4. Record locked nixpkgs rev in README from `nix flake metadata`.
5. Rebuild with absolute flake path under sudo.
6. Compare installer config vs flake config—list deltas.

7. Sketch multi-host layout for lab + future laptop.
8. Measure toplevel path size with `nix path-info -Sh` after build.
9. Ensure `flake.lock` is never gitignored.
10. Write a one-line alias `nrs` for rebuild (shell config later via HM).
11. Deliberately leave a file untracked; observe flake missing-file behavior.
12. Document `system` triple and why it must match the VM.

Common pitfalls

Symptom	Theory
Absolute import paths broken	Prefer relative module paths in repo
Wrong <code>system</code>	Eval/build target mismatch
hardware config not updated after disk change	Regenerate carefully
<code>experimental-features</code> missing	Rebuild cannot use flakes cleanly
Dirty git / missing files	Flake file inclusion
<code>stateVersion</code> folklore	Set to install release; read release notes before changing
Sudo cwd confusion	Absolute flake URI
Editing <code>/etc/nixos</code> by habit	Flake is SoT

Checkpoint

- ☐ `nixosConfigurations.<host>` exists
 - ☐ `hardware-configuration.nix` imported from flake repo
 - ☐ `switch --flake .#host` succeeded
 - ☐ Lock committed
 - ☐ README documents rebuild
 - ☐ Small declarative change proven
-

Further depth (this book)

Topic	Chapter
Rebuild modes / rollback	Rebuild modes and generations
Users/SSH	Users and access
Module literacy	Modules as a consumer
Custom modules	Writing custom modules
Advanced flake layout	Home and flake layout part

Rebuild modes and generations

Rebuild modes and generations

Goal: Master `nixos-rebuild modes`, `system generations`, and `rollback`—including a deliberate break and recovery on the disposable lab.

Why this chapter matters

NixOS’s famous safety is not automatic—it is **generational profiles + boot entries + knowing which rebuild verb you used**. Without that theory, “rollback” is folklore.

Ops motivation: production incidents need a rehearsed rollback card. This chapter builds that muscle on a disposable VM so production is not your first practice.

Theory 1 — System profile and generations

NixOS points the running system at a profile:

```
/nix/var/nix/profiles/system → system-N-link → store path
```

Each successful deployment can add a **generation N**. Boot loaders list generations so you can boot an older one when the new world is wrong.

Concept	Meaning
Generation	A complete system closure you can activate/boot
Current	What the profile points at now
Rollback	Point profile (and/or boot default) at older generation

Inspect

```
sudo nix-env -p /nix/var/nix/profiles/system --list-generations
readlink -f /nix/var/nix/profiles/system
ls -l /nix/var/nix/profiles/system*
```

Theory 2 — Rebuild modes

```
sudo nixos-rebuild <mode> --flake .#lab
```

Mode	Effect
switch	Build + activate now + set boot default to new gen
boot	Build + set boot default; do not activate until reboot
test	Build + activate now ; do not change boot default
build	Build only; no activation
dry-activate	Show activation plan (when supported/useful)
build-vm	Build a VM of the config (extra workflows)

Choosing modes

Situation	Prefer
Normal daily change	switch
Risky change; want easy reboot escape	test first or boot
Remote SSH-only host	Extra care—bad switch can lock you out
CI / check	build
Kernel/boot experiments	boot then reboot deliberately

Remote note

If you break networking/SSH on a remote machine with **switch**, you may need console. Lab VMs have consoles—use them.

Theory 3 — Activation vs boot default

```
test:  running config changes; reboot may return to previous default
switch: running + default updated
boot:  default updated; running stays old until reboot
```

Worked story

1. **test** a firewall change → SSH still works →
2. **switch** to commit it to boot default

Or:

1. boot a kernel change → reboot when ready →
2. If unbootable, select previous generation in boot menu

Decision flowchart

```
Is the change network/SSH sensitive?  
  yes → prefer test (with console available) → switch  
Is the change kernel/bootloader?  
  yes → consider boot → reboot → verify  
Otherwise → switch  
Want no activation? → build
```

Theory 4 — Listing and switching generations

```
sudo nix-env -p /nix/var/nix/profiles/system --list-generations  
# or  
sudo nixos-rebuild list-generations # when available in your version
```

Rollback helpers:

```
sudo nixos-rebuild switch --rollback  
# or boot older generation from boot menu
```

Also:

```
sudo /nix/var/nix/profiles/system-<N>-link/bin/switch-to-configuration switch
```

Prefer documented `nixos-rebuild` interfaces when possible.

Boot menu recovery

When userland is too broken to roll back from a shell:

1. Reboot
2. Open boot loader menu (systemd-boot/GRUB key)
3. Select previous generation
4. Fix flake; `switch` a good config

Document **your** hypervisor key chords in the rollback card.

Theory 5 — What is *not* rolled back

Rolled back with system gen	Not rolled back
Packages, units, users declared in config	Your data in /home , /var
Most /etc files managed by NixOS	Databases, untracked files
Kernel (for that gen)	Out-of-band disk edits
Firewall rules from config	Manual live nft edits as SoT (don't)

Punchline: rollback restores **system closure**, not time-travel for application data.

Theory 6 — GC and generations

Old generations **root** large closures. Cleaning:

```
sudo nix-collect-garbage -d # deletes old profile gens - careful
sudo nixos-rebuild ...     # after understanding
```

! Important

Do not delete generations until you have a known good current system and understand you are dropping rollback points.

Action	Risk
Keep last N gens	Disk vs safety trade
GC after known-good deploy	Frees space
GC before verifying	Removes escape hatches

Theory 7 — Flake + rebuild flags worth knowing

```
sudo nixos-rebuild switch --flake .#lab
sudo nixos-rebuild switch --flake .#lab --show-trace
sudo nixos-rebuild switch --flake .#lab -L
sudo nixos-rebuild switch --flake path:/home/you/nixos-config#lab
```

Flag	Use
<code>--show-trace</code>	Eval errors
<code>-L</code>	Build logs
<code>--impure</code>	Avoid unless you know why
<code>--upgrade</code> / channel habits	Prefer flake lock updates instead

Lock updates are a separate concern (Home and flake layout part).

Theory 8 — Activation scripts and failure modes

Activation runs scripts that update bootloader entries, restart units, rewrite `/etc`, etc.

Failure class	Response
Eval	No new gen; fix Nix
Build	No activation; fix package
Activation partial	Read output; may need rollback/console
Unit failed	<code>systemctl</code> , journal; gen may still be current

`test` is valuable because reboot can abandon a bad experiment if boot default never moved.

Theory 9 — Naming generations in ops notes

Track in `docs/rollback.md`:

Field	Example
Last known good gen	12
Last change summary	“opened ssh + http”
Snapshot name	<code>pre-firewall</code>

Generations numbers alone are not memory—annotate them.

Worked examples bank

Example A — Safe build

```
cd ~/nixos-config
nixos-rebuild build --flake .#lab
```

Example B — test then switch

```
sudo nixos-rebuild test --flake .#lab
# exercise the change
sudo nixos-rebuild switch --flake .#lab
```

Example C — rollback

```
sudo nixos-rebuild switch --rollback
```

Example D — Compare generations (conceptual)

```
ls -l /nix/var/nix/profiles/system-*link | tail
# note store paths differ per generation
```

Example E — Logs during rebuild

```
sudo nixos-rebuild switch --flake .#lab -L --show-trace
```

Lab 1 — Inventory generations before chaos

```
sudo nix-env -p /nix/var/nix/profiles/system --list-generations
readlink -f /nix/var/nix/profiles/system
```

Record current generation number in notes.

Lab 2 — Mode drills (harmless package)

1. Add a tiny package with `boot` mode only.
2. Confirm it is **not** on `PATH` until reboot (or document actual behavior you observe).
3. Reboot (VM snapshot first).
4. Confirm package present.

Then:

1. Use `test` for another package.
 2. Confirm available immediately.
 3. Reboot without `switch` and observe whether it persists as default—**write what happened**.
-

Lab 3 — Deliberate break → rollback

VM snapshot first.

Pedagogical break:

1. `switch` a config that installs a known package `cowsay`.
2. Then `switch` a config that removes it and sets a harmless env marker.
3. `rollback` and show package/env difference.

Document generation numbers before/after.

Safer annoying-but-not-fatal changes beat eval failures (eval never creates a “bad generation”).

Lab 4 — Boot menu literacy

On next reboot:

1. Enter boot loader menu (often hold space / show menu for `systemd-boot`).
2. Identify generation entries.
3. Boot current known-good.

Write how **your** hypervisor + loader expose this.

Lab 5 — Incident card

Create `~/nixos-config/docs/rollback.md`:

```
# Rollback card
1. Console access method: ...
2. List generations: ...
3. Rollback command: ...
4. Boot menu steps: ...
5. Last known good generation: ...
```

Exercises

1. Fill a table: mode × (activates now?) × (changes boot default?).
 2. Perform `build` then inspect `toplevel` path without switching.
 3. Simulate remote-only caution: list changes you would only `test` first.
 4. After three switches, list gens and mark which you could GC later.
 5. Break SSH in a snapshotted VM; recover via console + rollback.
 6. Time a no-op rebuild vs a package-adding rebuild.
 7. Document difference between profile rollback and boot-menu boot.
 8. Add a “last good gen” field to your git commit message habit.
 9. Explain to a peer why DB data survives rollback (or doesn’t get restored).
 10. Practice absolute flake path rebuild under `sudo`.
 11. Read activation output for one switch; annotate three steps.
 12. Update rollback card after the boot chapter for loader keys.
-

Common pitfalls

Symptom	Theory
Rollback “did nothing” for app data	Data system gen
Locked out via SSH	Use console; prefer test for net changes
No older generations	GC’d or first install
test change vanished after reboot	Expected if never switched
Flake path wrong under sudo	Use absolute path or root’s cwd discipline
Eval error mistaken for rollback drill	Need a successful bad generation
GC before verification	Lost escape hatches

Checkpoint

- ☐ Explain switch/test/boot/build
- ☐ List system generations
- ☐ Performed a rollback successfully
- ☐ Boot menu generation selection documented
- ☐ Incident card written
- ☐ VM snapshot restored option still available

Further depth (this book)

Topic	Chapter
Firewall safety with test	Networking and firewall
Boot loader entries	Boot and hardware
State vs generations	Filesystems and state
GC hygiene fleet-wide	Ops part

Users and access

Users and access

Goal: Declare users and remote access properly—**SSH keys** as the default remote path, non-root admin via `wheel`, and no long-lived plaintext password habits in the flake.

Why this chapter matters

Most early NixOS grief is lockouts and sloppy auth: root password only, keys never deployed, secrets pasted into world-readable store paths. Fix access patterns before opening the host to a network.

Ops motivation: access policy is production policy. A lab that practices key-only SSH and console recovery trains the same habits you need on real hosts.

Theory 1 — Declarative users

```
users.users.alice = {
  isNormalUser = true;
  extraGroups = [ "wheel" "networkmanager" ];
  openssh.authorizedKeys.keys = [
    "ssh-ed25519 AAAA... comment"
  ];
};
```

Field	Role
<code>isNormalUser</code>	Normal account defaults (home, shell group, ...)
<code>isSystemUser</code>	Service accounts
<code>extraGroups</code>	Group membership
<code>uid / group</code>	Pin when needed
<code>shell</code>	e.g. <code>pkgs.zsh</code>
<code>hashedPassword / initialHashedPassword</code>	Password auth
<code>openssh.authorizedKeys.keys</code>	SSH public keys
<code>description</code>	GECOS / human label

System users (preview)

```
users.users.labbot = {
  isSystemUser = true;
  group = "labbot";
};
users.groups.labbot = {};
```

Service bots do not belong in `wheel`.

Theory 2 — Root vs wheel admin

Pattern	Notes
Root login over SSH	Discourage; often disable
User in <code>wheel</code> + <code>sudo</code>	Preferred admin path
Passwordless <code>sudo</code>	Convenient lab; tighten later

```
security.sudo.wheelNeedsPassword = true; # or false for lab convenience
```

```
services.openssh.settings = {
  PermitRootLogin = "no";
  PasswordAuthentication = false;
};
```

Exact option paths can vary slightly by release—prefer `services.openssh.settings.*` on modern NixOS; verify with `docs/man configuration.nix` / `nixos-option` / search.nixos.org **26.05**.

Order of hardening

1. Key login works for wheel user
2. Disable password SSH
3. Disable root SSH
4. Consider `mutableUsers = false`

Never reverse the order on a remote-only host without console.

Theory 3 — Passwords and the store

Approach	Store risk
<code>initialPassword = "secret"</code>	Password may land in store/world-readable eval — lab only, rotate
<code>hashedPassword = "..."</code>	Hash still visible in store; better than plaintext but not a secret manager
SSH keys only	Public keys are public; private keys never in flake
sops-nix / agenix later	Services and security part

House rule now: SSH public keys in git are fine; **private keys and live passwords are not.**

Generate hash on the machine if you need password console login:

```
mkpasswd -m sha-512
# or openssl passwd -6
```

Why the store leaks matter

NixOS evaluation materializes values into the store. Anything in the flake that is a secret string is a secret management failure—even if file mode looks fine in git.

Theory 4 — SSH service basics

```
services.openssh = {
  enable = true;
  ports = [ 22 ];
  settings = {
    PasswordAuthentication = false;
    KbdInteractiveAuthentication = false;
    PermitRootLogin = "no";
  };
};
```

Firewall must allow the port (networking chapter)—if SSH dies after firewall work, check both.

Host keys vs user keys

Key type	Purpose
SSH host keys	Identify the machine to clients
User authorized keys	Authenticate users to the machine
Your laptop private key	Stays on client

Host keys live under `/etc/ssh` and are part of machine identity—backup/restore stories later (filesystems chapter).

Theory 5 — Authorized keys on NixOS

```
users.users.alice.openssh.authorizedKeys.keys = [ "ssh-ed25519 AAAA... alice@laptop" ];
# or
# users.users.alice.openssh.authorizedKeys.keyFiles = [ ./keys/alice.pub ];
```

Prefer **one key per line**, full public key material, no wrapping. A truncated key fails as “Permission denied (publickey).”

NixOS often manages authorized keys under `/etc/ssh/authorized_keys.d` rather than only `~/.ssh/authorized_keys`—verify on your host.

Client config hygiene

```
Host nixlab
  HostName 192.0.2.10
  User alice
  IdentityFile ~/.ssh/id_ed25519_nixlab
  IdentitiesOnly yes
```

`IdentitiesOnly yes` avoids offering the wrong key and hitting `MaxAuthTries`.

Theory 6 — Groups you will actually use

Group	Common reason
<code>wheel</code>	<code>sudo</code>
<code>networkmanager</code>	NM control (if NM)
<code>docker</code> / <code>podman</code>	Containers later—grant carefully
<code>video</code> / <code>input</code>	Desktop labs

Group	Common reason
-------	---------------

Least privilege: do not dump every group on the admin user “just in case.”

Theory 7 — Home directories and mutability

Declarative users create homes per policy. Files inside homes are **state**. Rolling back system generations does not reset `~/.ssh` edits. Home Manager (later part) manages dotfiles; this chapter is raw user access only.

mutableUsers

```
users.mutableUsers = false; # strong: only flake defines users/passwords
# users.mutableUsers = true; # imperative passwd may persist
```

Setting	Effect
<code>true</code>	Imperative drift possible
<code>false</code>	Users/passwords from config (hashed)

For a lab that teaches flakes as SoT, consider `false` once SSH keys work—know that password changes then require a rebuild.

Theory 8 — Sudo and elevation patterns

Pattern	Lab use
<code>wheelNeedsPassword = true</code>	Safer habit
passwordless sudo	Faster drills; note risk
<code>sudo -u service users</code>	Later service chapters

Prefer proving `sudo -v` works before disabling other access methods.

Theory 9 — Debugging SSH failures

From client:

```
ssh -vvv -i ~/.ssh/id_ed25519_nixlab alice@<vm-ip> 2>&1 | tail -40
```

On host:

```
sudo journalctl -u sshd -n 50 --no-pager  
ss -ltn | grep 22
```

Symptom	Checks
Connection refused	sshd down / wrong IP / firewall
Permission denied (publickey)	key material, user, identities
Auth succeeds, sudo fails	wheel membership

Theory 10 — Audit checklist for flakes

```
cd ~/nixos-config  
rg -n "initialPassword|password\s*=" || true  
rg -n "BEGIN OPENSSSH PRIVATE KEY" || true  
rg -n "PRIVATE KEY" || true
```

Rotate anything that ever landed in git history.

Worked examples bank

Example A — Admin user with key

```
users.users.alice = {  
  isNormalUser = true;  
  extraGroups = [ "wheel" ];  
  openssh.authorizedKeys.keys = [  
    "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIExampleKeyMaterial alice@laptop"  
  ];  
};
```

Example B — Disable password SSH

```
services.openssh.enable = true;  
services.openssh.settings.PasswordAuthentication = false;
```

Example C — Console password without storing plaintext

```
users.users.alice.hashedException = "$6$..."; # from mkpasswd
```

Example D — System user for a service (preview)

```
users.users.labbot = {  
  isSystemUser = true;  
  group = "labbot";  
  description = "Lab automation user";  
};  
users.groups.labbot = {};
```

Example E — Key file in repo

```
users.users.alice.openssh.authorizedKeys.keyFiles = [  
  ./keys/alice.pub  
];
```

Ensure `./keys/alice.pub` is git-tracked (public only).

Lab 1 — Generate a lab keypair on the client

On your workstation (not necessarily the VM):

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_nixlab -C "nixlab"  
cat ~/.ssh/id_ed25519_nixlab.pub
```

Copy the **public** key into the flake. Never commit the private key.

Lab 2 — Wire user + SSH into flake

Edit `hosts/lab/configuration.nix`:

1. Admin user with `wheel`
2. `openssh.authorizedKeys.keys`
3. SSH enabled
4. Prefer `PasswordAuthentication = false` once key login works
5. `PermitRootLogin = "no"` when ready

```
cd ~/nixos-config
sudo nixos-rebuild switch --flake .#lab
```

From client:

```
ssh -i ~/.ssh/id_ed25519_nixlab alice@<vm-ip>
```

Lab 3 — Verify identity paths

On host:

```
whoami
groups
sudo -v
ss -ltn | grep -E ' :22'
```

On client: confirm you did **not** need a password for SSH (if disabled).

Lab 4 — Lockout drill (console required)

Snapshot first.

1. Misconfigure authorized key (truncate key).
2. Rebuild.

3. Confirm SSH fails.
4. Fix via **console**, rebuild.

Write the recovery path on your rollback card.

Lab 5 — Audit flake for secret smells

```
cd ~/nixos-config
rg -n "initialPassword|password\s*=" || true
rg -n "BEGIN OPENSSSH PRIVATE KEY" || true
```

Remove plaintext habits. Document any temporary lab exceptions.

Lab 6 — `users.mutableUsers` experiment (optional, snapshot)

1. Note current `mutableUsers`
2. Set explicitly in flake
3. Rebuild
4. Document whether `passwd` changes survive the next rebuild

Revert if the policy is too strict for console-only recovery yet.

Lab 7 — SSH hard fail checklist

Run the verbose client + `journalctl` flow from Theory 9. Write the actual `authorized_keys` path you observed.

Exercises

1. Add a second non-admin user with a different key.
2. Move public keys to `keyFiles` and rebuild.
3. Toggle `wheelNeedsPassword` and document UX impact.
4. Attempt root SSH after `PermitRootLogin = "no"`; confirm failure.
5. Create a system user; prove it has no login shell access as intended.
6. Document host key fingerprints for your SSH `known_hosts` process.
7. Practice recovery: break sudo group membership; fix on console.
8. Write an access policy paragraph for this lab (who, how, exceptions).
9. Scan git history for accidental secrets if you ever committed them.
10. Align firewall chapter: ensure port 22 allowed after hardening.
11. Compare password hash approaches; choose one documented method.
12. Update `docs/rollback.md` with SSH lockout recovery steps.
13. Explain store visibility of `hashedPassword` to a peer.
14. Prepare for Home Manager later: list dotfiles you will *not* put in the system flake.

Common pitfalls

Symptom	Theory
Key in flake but SSH password still works	PasswordAuthentication still on
Permission denied (publickey)	Wrong user, wrong key, typo in key line
Locked out	Console + rollback
Password in git history	Rotate; rewrite history if needed; prefer never
Root SSH only	Add wheel user before disabling root
Private key committed	Emergency rotation
Wrong <code>authorized_keys</code> path assumptions	NixOS managed paths
<code>mutableUsers</code> surprise	Policy mismatch with <code>passwd</code>

Checkpoint

- ☐ Declarative non-root admin user
 - ☐ SSH key login works
 - ☐ Password SSH disabled (or explicitly justified)
 - ☐ Root SSH disabled or justified
 - ☐ No private keys in repo
 - ☐ Lockout recovery tested on console
 - ☐ Secret audit clean
-

Further depth (this book)

Topic	Chapter / part
Firewall for SSH	Networking and firewall
Host keys as state	Filesystems and state
Secrets management	Services and security: sops/agenix
User dotfiles	Home Manager chapters

Networking and firewall

Networking and firewall

Goal: Configure lab networking deliberately—pick **networkd** or **NetworkManager**, know your address, and run a **default-deny-ish** firewall that still allows SSH.

Why this chapter matters

A flake host that “has SSH in config” but blocked by firewall, or random DHCP with no notes, is not operable. Networking is part of the system closure story.

Ops motivation: most “NixOS is down” tickets are network, DNS, or firewall mistakes. Declare them, test them, document them.

Theory 1 — Two common network stacks on NixOS

Stack	Typical use	Module vibes
systemd-networkd	Servers, VMs, simple static	<code>systemd.network</code> , <code>networking.useNetworkd</code>
NetworkManager	Laptops, Wi-Fi, desktop	<code>networking.networkmanager.enable</code>

Do not enable both casually without understanding conflicts. For a **server-like VM**, `networkd` or simple DHCP via `networking.*` is enough.

Classic DHCP simplicity

```
networking.useDHCP = false;  
networking.interfaces.ens3.useDHCP = true; # interface name from `ip a`
```

Or global patterns depending on release defaults—**inspect generated hardware and `ip a`**.

networkd file shape (server VM)

```
{
  networking.useNetworkd = true;
  networking.useDHCP = false;
  systemd.network.networks."10-wan" = {
    matchConfig.Name = "en*";
    networkConfig.DHCP = "yes";
  };
}
```

Interface match patterns reduce rename pain vs hard-coding `ens3` only—verify on **your** virt NIC names.

Theory 2 — Host identity

```
networking.hostName = "lab";
# networking.domain = "example.local";
```

Hostname affects prompts, certificates later, and mental ops. Keep flake attr and hostname documented (flake hosts chapter).

Extra hosts

```
networking.extraHosts = ''
  127.0.0.1 lab.local
  10.0.0.5 sibling.lab
'';
```

Useful for multi-VM labs; real DNS arrives with TLS/ACME later.

Theory 3 — Firewall model

NixOS firewall (iptables/nftables backend depending on config) is commonly:

```
networking.firewall = {
  enable = true;
  allowedTCPPorts = [ 22 ];
  allowedUDPPorts = [ ];
}
```

```
# allowedTCPPortRanges = [ { from = 8000; to = 8010; } ];
};
```

Idea	Practice
Default deny inbound	Enable firewall
Open only needed ports	List explicitly
Per-interface rules	Advanced; later
Service modules open ports	Some set firewall rules when enabled

Always ensure SSH port is allowed before testing remote-only access.

nftables awareness (26.05)

Modern NixOS often prefers **nftables** as the firewall backend. You rarely write raw rules by hand; you set module options and **inspect**:

```
sudo nft list ruleset | head -80
sudo iptables -L -n 2>/dev/null | head || true
```

Practice	Avoid
Module options + rebuild	Hand-editing live nft rules as SoT
Document backend in docs/network.md	Assuming iptables forever

Theory 4 — Interface names

Predictable names (`enp0s3`, `ens18`, `eth0`) depend on virt hardware.

```
ip -br a
networkctl status # if networkd
nmcli device      # if NM
```

Hard-coding the wrong interface → no network after switch. Prefer DHCP-on-all or verified names.

Theory 5 — Static addressing (optional lab)

```
networking.interfaces.ens3.ipv4.addresses = [{  
    address = "192.168.56.10";  
    prefixLength = 24;  
}];  
networking.defaultGateway = "192.168.56.1";  
networking.nameservers = [ "1.1.1.1" "9.9.9.9" ];
```

Use only when you understand the hypervisor network. Wrong gateway = silence.

Theory 6 — DNS and resolution

```
resolvectl status # systemd-resolved often in play  
getent hosts cache.nixos.org
```

Broken DNS looks like “Nix is broken” during rebuilds (substituters).

Failure	Symptom
No route	Timeouts
Bad DNS	NXDOMAIN / resolve fail mid-rebuild
Captive / filtered net	Partial substituter failures

Theory 7 — Firewall vs service modules

Example: enabling `services.openssh` does not always mean you thought about firewall—on many configs you still list port 22 (or rely on module defaults). **Verify:**

```
sudo nft list ruleset | head  
ss -ltn
```

Hardening deep-dives continue in the Services and security part; this chapter is correctness + minimal surface.

Theory 8 — Rebuild safety pattern for remote operators

```
sudo nixos-rebuild test --flake .#lab
# new shell: ssh still works?
sudo nixos-rebuild switch --flake .#lab
```

`test` activates without making the generation the boot default—slightly safer while iterating firewall. Console access remains the real safety net.

Lockout drill ethics

Only lock yourself out on a VM with console + snapshot. Production requires out-of-band access plans.

Theory 9 — Port audit as continuous practice

Listener	Firewall	Config source	Keep?
:22	open	<code>allowedTCPPorts</code> / ssh	yes
localhost-only	n/a	bind address	ok
mystery	?	?	investigate

Any public listener without a story is a future ticket.

Theory 10 — Hypervisor port forwards

NAT mode often needs host port → guest 22 forward. Document:

```
host localhost:2222 → guest :22
ssh -p 2222 alice@127.0.0.1
```

Firewall on guest still applies to forwarded traffic.

Worked examples bank

Example A — Server-like DHCP + SSH

```
{
  networking.hostName = "lab";
  networking.firewall.enable = true;
  networking.firewall.allowedTCPPorts = [ 22 ];
  services.openssh.enable = true;
}
```

Example B — Explicit interface DHCP

```
{
  networking.useDHCP = false;
  networking.interfaces.ens3.useDHCP = true;
}
```

Example C — Temporary debug open (lab only)

```
# networking.firewall.allowedTCPPorts = [ 22 80 443 ];
```

Remove ports you do not need.

Example D — networkd DHCP match

See Theory 1 networkd snippet.

Example E — DNS pin

```
networking.nameservers = [ "1.1.1.1" "9.9.9.9" ];
```

Lab 1 — Discover ground truth

On the lab host:

```
ip -br a
ip route
cat /etc/hostname
resolvectl status 2>/dev/null | head
```

Record interface name(s), IPv4, default route.

Lab 2 — Declare firewall explicitly

In flake config:

1. `networking.firewall.enable = true`
2. Allow SSH port only
3. Rebuild (test recommended if remote)

```
sudo nixos-rebuild test --flake .#lab
# verify ssh still works
sudo nixos-rebuild switch --flake .#lab
```

Lab 3 — Port audit

```
ss -ltnup
# map each listener to a reason
```

Fill table in docs/network.md.

Lab 4 — Controlled break (console ready)

Snapshot.

1. Remove 22 from `allowedTCPPorts` while SSH is your only remote path.
2. `switch` (expect lockout if no console).
3. Fix via console; restore port.

If you are console-only already, simulate by checking `nft` counters and documenting the mistake.

Lab 5 — Network README snippet

Add to `docs/network.md`:

- Hypervisor network mode (NAT/bridge/host-only)
 - How to find VM IP from host
 - Firewall policy summary
 - DNS notes
 - Backend (nftables/iptables observation)
-

Lab 6 — Firewall rule listener map

Build a three-column table:

Listener (<code>ss</code>)	Firewall open?	Config source
:22	yes	<code>allowedTCPPorts</code> / <code>ssh</code> module

Lab 7 — DNS failure drill (safe)

```
getent hosts cache.nixos.org
```

Document: rebuilds that need substitutes fail hard when DNS dies mid-flight—order network fixes carefully.

Exercises

1. Draw your hypervisor network path (host → NAT → guest).
2. Switch from interface-hardcoded DHCP to a match pattern (if networkd).
3. Add a second allowed port for a localhost service only if intentional.
4. Compare `test` vs `switch` after a firewall edit.
5. Capture `nft list ruleset` before/after enabling firewall.
6. Break DNS with a bad nameserver in a snapshotted VM; recover.
7. Document IPv6 status on your lab (on/off/partial).
8. Map each `ss` listener to a Nix option or “unknown.”
9. Write a “no inbound except SSH” policy in one paragraph.
10. Practice reconnect scripts from the workstation.
11. Note whether openssh module opened ports for you automatically.
12. Prepare for TLS chapter: pick a future lab hostname.

Common pitfalls

Symptom	Theory
Rebuild can't fetch	DNS/route broken mid-change
SSH timeout after firewall	Port not allowed
Interface rename after hardware change	Update config
Both NM and networkd fighting	Pick one model
<code>useDHCP</code> defaults surprise	Read options for 26.05
Live <code>nft</code> edits as SoT	Lost on rebuild
NAT without port forward	“VM has no SSH” false alarm

Checkpoint

- ☐ Interface + IP documented
 - ☐ Firewall enabled with explicit SSH allow
 - ☐ Port audit table written
 - ☐ Recovery from firewall mistake understood
 - ☐ Committed network docs
 - ☐ Prefer `test` for risky net changes
-

Further depth (this book)

Topic	Chapter / part
SSH users	Users and access
Service ports	systemd via Nix
Hardening / TLS	Services and security part
Firewall discipline deep	Services and security part

Packages and programs

Packages and programs

Goal: Install software **declaratively** on the host—know when to use `environment.systemPackages` versus `programs.*` modules, and keep ad-hoc profiles from becoming the real SoT.

Why this chapter matters

New NixOS users `nix-env` / `nix profile install` a zoo, then wonder why rebuilds do not reproduce the machine. Declarative packages are how the host stays honest.

Ops motivation: reproducible admin tooling is part of incident response. The box you SSH into should carry `jq` and friends because the flake says so—not because someone remembered to install them once.

Theory 1 — `environment.systemPackages`

```
environment.systemPackages = with pkgs; [  
  vim  
  git  
  curl  
  htop  
  jq  
];
```

Property	Meaning
On PATH system-wide	For all users (typical)
Closure in system gen	Rolled back with system
No service config	Just binaries/libs

This is the blunt instrument—and it is correct for many CLI tools.

Organization pattern

```
environment.systemPackages = with pkgs; [  
  # editors  
  vim  
  # net  
  curl  
  # observability  
  htop  
];
```

Readable policy beats an unsorted dump.

Theory 2 — `programs.*` modules

Many packages have first-class modules:

```
programs.git.enable = true;  
programs.zsh.enable = true;  
programs.tmux.enable = true;
```

Modules may:

- Install the package
- Drop config snippets
- Set shells in `/etc/shells`
- Enable related services

Prefer `programs.*` when you need integration, not only a binary.

Discovery

```
man configuration.nix # huge  
nixos-option programs.git 2>/dev/null
```

Online: <https://search.nixos.org/options> (select **26.05** when available).

Theory 3 — Decision table

Need	Prefer
Just a CLI on PATH	<code>environment.systemPackages</code>
Shell as login shell + vendor config	<code>programs.zsh</code> / <code>programs.fish</code> / <code>programs.bash</code>
Git system-wide with config hooks	<code>programs.git</code> and/or HM later
Language toolchains for <i>one project</i>	devShell , not <code>systemPackages</code>
Desktop app	packages or specialized modules
Always-on service	<code>services.*</code> / systemd chapter

Theory 4 — Closure discipline (light)

Every package on the system increases:

- Download size
- Eval/build time
- Attack surface
- Cognitive load

Smell	Fix
Entire language toolchains system-wide	<code>devShell</code> / HM
Three editors “just in case”	Pick one for system
Random <code>nix profile</code> duplicates	Remove; declare in flake
Debug tools forever	Keep a small admin set

```
nix path-info -Sh /run/current-system
nix path-info -rS /run/current-system | sort -n | tail
```

Theory 5 — Overlays and pins (awareness only)

You might see:

```
nixpkgs.overlays = [ ... ];
```

or pinned packages. Later parts cover overlays properly. Here: stick to nixpkgs **26.05** attribute names from your lock.

Theory 6 — Unfree and allow flags

Some packages need:

```
nixpkgs.config.allowUnfree = true;
```

Or per-package predicates:

```
nixpkgs.config.allowUnfreePredicate = pkg:
  builtins.elem (pkgs.lib.getName pkg) [ "vscode" ]; # example only
```

Only enable when you understand licensing implications for **your** lab. Narrow predicates beat global allow when possible.

Theory 7 — System vs user packages

Layer	Mechanism
System	<code>environment.systemPackages</code> , <code>programs.*</code>
User (later)	Home Manager <code>home.packages</code>
Project	<code>nix develop</code> shells
Imperative	<code>nix profile</code> — minimize

Stage-shaped later work moves dotfiles and user CLIs to HM; this chapter is system layer only.

Theory 8 — `environment.defaultPackages` trap

NixOS may install a small default package set. Know it exists so you do not double-declare or fight removals:

```
# advanced / intentional only
# environment.defaultPackages = [ ];
```


Prefer adding what you need over emptying defaults on day one unless you understand the fallout (e.g. `nano/perl` expectations in scripts).

Theory 9 — Package name binary name

pkgs attr	Binary might be
<code>ripgrep</code>	<code>rg</code>
<code>neovim</code>	<code>nvim</code>
some wrappers	different <code>meta.mainProgram</code>

```
nix search nixpkgs ripgrep
command -v rg
```

Theory 10 — Layers interaction with generations

Adding/removing system packages creates new generations. Rollback brings the old package set back—another reason not to rely on imperative profiles for “real” tools.

Worked examples bank

Example A — Baseline admin tools

```
{ pkgs, ... }: {
  environment.systemPackages = with pkgs; [
    vim
    git
    curl
    wget
    htop
    tmux
    jq
    ripgrep
    tree
  ];
}
```

```
}
```

Example B — programs integration

```
{  
  programs.zsh.enable = true;  
  users.users.alice.shell = pkgs.zsh;  
}
```

Example C — Search package attr

```
nix search nixpkgs ripgrep
```

Example D — Conditional unfree single package

```
{  
  nixpkgs.config.allowUnfreePredicate = pkg:  
    builtins.elem (pkgs.lib.getName pkg) [ "vscode" ];  
}
```

Example E — Avoid double install

If `programs.git.enable = true` already installs `git`, do not also list `git` unless you have a reason—collapse to one path.

Lab 1 — Declare your daily CLI set

List 8–15 tools you truly want on the `lab` host. Add them via `environment.systemPackages`. Rebuild switch. Verify each:

```
command -v rg  
command -v jq  
# ...
```

Lab 2 — Convert one package to `programs.*`

Pick something with a module (git, zsh, tmux, ...):

1. Remove from `systemPackages` if redundant
 2. Enable `programs.<name>`
 3. Rebuild
 4. Note extra behavior (completions, config files)
-

Lab 3 — Anti-profile audit

```
nix profile list 2>/dev/null || true
```

If tools exist only imperatively, either migrate into flake or remove them. Document: **flake owns system tools**.

Lab 4 — Size awareness

```
nix path-info -Sh /run/current-system  
nix path-info -rS /run/current-system | sort -n | tail
```

Write one sentence on the cost of your package list.

Lab 5 — Split comments in config

Organize packages with section comments. Commit as readable policy.

Lab 6 — `programs.*` option archaeology

```
nixos-option programs.zsh 2>/dev/null | head -40
```

List three options under a `programs.*` module that are **not** just “install the binary.”

Lab 7 — Remove one redundant package

Find a package present both via `systemPackages` and a `programs.*` enable (or profile). Collapse to one path. Rebuild. Confirm still on `PATH`.

Exercises

1. Build a “minimal rescue” package set vs “comfortable admin” set; compare sizes.
 2. Move a language toolchain out of `systemPackages` into a project `devShell`.
 3. Enable `programs.tmux` and note config paths created.
 4. Document unfree policy for the lab in `README`.
 5. Find three packages where `attr` is binary name.
 6. Use `nix-tree` or `path-info` to find heavy dependencies (if available).
 7. Prove rollback restores a removed package.
 8. Write a PR-style commit message for package policy changes.
 9. Avoid `with pkgs`; for a section; use explicit `pkgs.` and compare readability.
 10. List tools that should never be system-wide on a shared host.
 11. Prepare HM split: which packages will leave system later?
 12. Search options for a program you use; enable one non-obvious toggle.
-

Common pitfalls

Symptom	Theory
Package name binary name	Check <code>pkgs</code> attr vs <code>meta.mainProgram</code>
Unfree errors	<code>allowUnfree</code> policy
Tool missing after rebuild	Not in config; was profile-only
Giant system closure	Too many packages / wrong layer
Shell not changing	<code>users.users.*.shell</code> not set
Double-declared tools	<code>programs</code> + <code>systemPackages</code>
Emptying <code>defaultPackages</code> casually	Surprising script breakage

Checkpoint

- ☐ Daily tools declared in flake
- ☐ At least one `programs.*` used
- ☐ Imperative profile not required for lab tools
- ☐ Rough system path size noted
- ☐ Config comments organized
- ☐ Layering (system vs project) explained

Further depth (this book)

Topic	Chapter / part
Custom services using <code>pkgs</code>	systemd via Nix
Module options	Modules as a consumer
Home packages	Home Manager chapters
Overlays / packaging	Packaging part

systemd via Nix

systemd via Nix

Goal: Express **systemd services and timers** in NixOS modules—ship one custom service + one timer, and verify units with standard systemd tools.

Why this day exists

NixOS does not replace systemd; it **generates** units. If you only know **systemctl** on mutable distros, you will fight activation. Today connects module options → unit files → runtime.

Theory 1 — Module options generate units

```
systemd.services.hello-lab = {
  description = "Day17 hello service";
  wantedBy = [ "multi-user.target" ];
  serviceConfig = {
    Type = "oneshot";
    ExecStart = "${pkgs.coreutils}/bin/echo hello-from-nix";
  };
};
```

On rebuild, NixOS materializes unit files under the system closure and activates them.

You write	systemd sees
<code>systemd.services.<name></code>	<code>name.service</code>
<code>systemd.timers.<name></code>	<code>name.timer</code>
<code>systemd.user.services...</code>	user units (later/HM)

Theory 2 — Service essentials

Field	Role
<code>description</code>	Human text
<code>wantedBy</code>	Targets that pull this in
<code>after / requires / wants</code>	Ordering/deps
<code>serviceConfig.Type</code>	simple, oneshot, notify, ...
<code>serviceConfig.ExecStart</code>	Command (prefer absolute store paths)
<code>path</code>	Extra PATH packages for the unit
<code>script</code>	Convenience instead of ExecStart in some cases
<code>enable / wantedBy</code>	Whether it starts

Absolute paths matter

```
ExecStart = "${pkgs.hello}/bin/hello";
```

Not bare `hello`—units may not share your interactive PATH.

Theory 3 — Timers

```
systemd.services.lab-tick = {
  serviceConfig.Type = "oneshot";
  script = ''
    echo "tick $(date -Is)" >> /var/log/lab-tick.log
  '';
};

systemd.timers.lab-tick = {
  wantedBy = [ "timers.target" ];
  timerConfig = {
    OnBootSec = "1min";
    OnUnitActiveSec = "5min";
    Unit = "lab-tick.service";
  };
};
```

Timer idea	Example
Calendar	<code>OnCalendar = "daily"</code>
After boot	<code>OnBootSec</code>
Periodic	<code>OnUnitActiveSec</code>

Prefer timers over classic cron on NixOS unless you have a reason.

Theory 4 — State directories and permissions

Services that write state need:

- Writable paths (`/var/lib/...`)
- Correct `User=` / `Group=`
- Sometimes `StateDirectory=` in `serviceConfig`

```
serviceConfig = {  
  User = "labbot";  
  StateDirectory = "labbot"; # → /var/lib/labbot  
};
```

Do not write into the Nix store.

Theory 5 — Inspecting generated units

```
systemctl status lab-tick.timer  
systemctl cat lab-tick.service  
journalctl -u lab-tick.service -n 50  
systemctl list-timers --all | head
```

`systemctl cat` shows the unit text—great for verifying Nix rendered what you meant.

Theory 6 — Relationship to `services.*` high-level modules

High-level modules (`services.nginx`, `services.postgresql`) **wrap** `systemd` units with options, hardening, and firewall hooks. Custom `systemd.services` is the escape hatch and learning tool.

Pattern:

```
prefer services.<known> when it exists  
else systemd.services.<custom>
```

Theory 7 — Failure and rollback

A bad unit can fail activation. Generations still help:

1. Fix config
2. Or rollback generation
3. `journalctl -xe` for clues

`test` mode is your friend for unit experiments.

Worked examples bank

Example A — oneshot + timer writing a log

See Theory 3.

Example B — long-running simple service

```
systemd.services.lab-http-static = {
  wantedBy = [ "multi-user.target" ];
  after = [ "network.target" ];
  serviceConfig = {
    Type = "simple";
    ExecStart = "${pkgs.python3}/bin/python3 -m http.server 8000 --bind 127.0.0.1";
    Restart = "on-failure";
  };
};
# open firewall only if you intentionally expose it
```

Bind to localhost for lab safety unless you mean to expose.

Example C — path for tools in script

```
systemd.services.lab-path-demo = {
  path = [ pkgs.jq pkgs.coreutils ];
  script = ''
    echo '{"ok":true}' | jq .
```

```
'';  
serviceConfig.Type = "oneshot";  
};
```

Lab 1 — Ship lab-tick timer

Add service+timer from Theory 3 (adjust log path permissions—use `StateDirectory` or a path you own).

```
sudo nixos-rebuild switch --flake .#lab  
systemctl status lab-tick.timer  
systemctl start lab-tick.service  
journalctl -u lab-tick.service -n 20
```

Lab 2 — systemctl cat literacy

```
systemctl cat lab-tick.service  
systemctl cat lab-tick.timer
```

Annotate in notes: which lines came from your Nix options.

Lab 3 — One long-running service (localhost)

Deploy Example B or simpler. Verify:

```
curl -sS http://127.0.0.1:8000/ | head  
systemctl status lab-http-static
```

Ensure firewall does **not** expose 8000 unless intentional.

Lab 4 — Break ExecStart on purpose

Point `ExecStart` at a missing path, rebuild, watch failure, fix, rebuild. Note whether generation advanced.

Lab 5 — Document units in README

```
## Custom units
- lab-tick.timer: ...
- lab-http-static.service: ...
```

Theory 8 — Hardening starters for custom units

Even lab units benefit from a few defaults:

```
systemd.services.lab-http-static.serviceConfig = {
  Type = "simple";
  ExecStart = "${pkgs.python3}/bin/python3 -m http.server 8000 --bind 127.0.0.1";
  Restart = "on-failure";
  # hardening starters (tune per workload)
  NoNewPrivileges = true;
  PrivateTmp = true;
  ProtectSystem = "strict";
  ProtectHome = true;
  # ReadWritePaths = [ "/var/lib/lab-http" ]; # if needed
};
```

Do not cargo-cult every sandbox flag onto a unit that must write state—pair with `StateDirectory` / `ReadWritePaths`.

Theory 9 — systemd.tmpfiles for paths units need

```
systemd.tmpfiles.rules = [  
  "d /var/log/lab-tick 0755 root root -"  
];
```

Creates directories at boot/activation. Prefer `StateDirectory=` when the unit's runtime user owns the state under `/var/lib`.

Theory 10 — Activation vs live systemctl edit

Path	SoT?
Flake <code>systemd.services.*</code>	Yes
<code>systemctl edit --full drop-ins</code>	Drift; vanishes from mental model
Hand unit in <code>/etc/systemd/system</code>	Fights NixOS activation

If you must debug live, capture the working unit into the flake before the next rebuild.

Worked example D — oneshot with RemainAfterExit

```
systemd.services.lab-seed = {  
  wantedBy = [ "multi-user.target" ];  
  serviceConfig = {  
    Type = "oneshot";  
    RemainAfterExit = true;  
    ExecStart = "${pkgs.coreutils}/bin/touch /var/lib/lab-seed/ok";  
    StateDirectory = "lab-seed";  
  };  
};
```

Useful for “run once to prepare state” patterns without a long-running daemon.

Lab 6 — Timer calendar form

Replace interval timer with:

```
timerConfig.OnCalendar = "*-*-* *:0/15:00"; # every 15 minutes - verify syntax
```

```
systemctl cat lab-tick.timer
systemd-analyze calendar '*-*-* *:0/15:00'
```

Document which calendar expression you settled on.

Lab 7 — Unit dependency map

For your long-running service, write:

```
network.target → lab-http-static.service → (curl clients)
```

Add `after` / `wants` only when real. Over-`requires` makes boot fragile.

Common gotchas

Symptom	Theory
Executable not found	Need store absolute path or <code>path</code>
Timer never fires	not <code>wantedBy</code> <code>timers.target</code>
Permission denied writing log	state dir / user
Service works manually not under systemd	<code>PATH</code> / env differences
Forgotten firewall	the networking chapter + service ports

Checkpoint

- ☐ Custom service in flake
- ☐ Custom timer in flake
- ☐ `systemctl cat` inspected
- ☐ `journalctl` read for your unit
- ☐ Localhost service verified

Commit

```
cd ~/nixos-config  
git add .  
git commit -m "day17: systemd services and timers"
```

Modules as a consumer

Modules as a consumer

Goal: Use the NixOS module system **as a consumer**—read options and source for real modules, apply `mkIf` / `mkDefault` concepts correctly, and change behavior via **options** rather than forking `nixpkgs`.

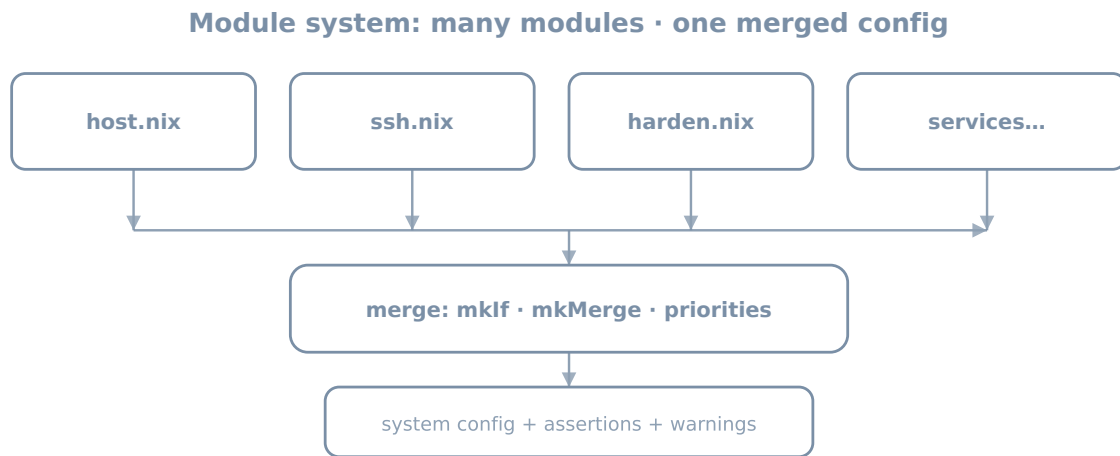


Figure 1: Module merge diagram

Why this day exists

Copy-pasting entire module files into your repo is a maintenance trap. Power users set options, import small local modules, and only override when necessary. That skill starts with **reading**.

Theory 1 — A module is a function (usually)

```
{ config, lib, pkgs, ... }: {
  options = { /* declarations */ };
  config = { /* implementations */ };
}
```

Argument	Role
<code>config</code>	Final merged configuration (lazy)
<code>lib</code>	Library + <code>mkOption</code> , <code>mkIf</code> , ...
<code>pkgs</code>	Package set for this system
...	Other module args

Some files are plain attrsets; most nixpkgs modules use the function form.

Theory 2 — Options vs config

Side	Meaning
options	Schema: what <i>may</i> be set, types, defaults, descriptions
config	Values and implementations that produce system artifacts

As a consumer you mostly **set option values**:

```
{
  services.openssh.enable = true;
  networking.firewall.allowedTCPPorts = [ 22 ];
}
```

You are writing **config** fragments that merge into the global config.

Theory 3 — Merge semantics (intuition)

Multiple modules set the same option; the module system **merges** by type:

Type vibe	Merge idea
Boolean	Priority rules / any true patterns depend on option
List	Concatenate
Attrset	Deep merge of attributes
Exclusive values	Conflict error or priority

This is why shallow `//` (the language attrsets chapter) is not how NixOS config works.

Theory 4 — lib.mkIf

```
{ config, lib, ... }: {
  config = lib.mkIf config.services.foo.enable {
    environment.systemPackages = [ /* ... */ ];
    networking.firewall.allowedTCPPorts = [ 8080 ];
  };
}
```

mkIf cond value includes value only when cond is true—**without** breaking merge when false (unlike a raw if that might remove structure incorrectly in some patterns).

Consumer pattern: you rarely write mkIf until the custom modules chapter; you **benefit** from modules that use it when you set enable = true.

Theory 5 — lib.mkDefault and priority

```
services.openssh.enable = lib.mkDefault true;
```

Helper	Idea
mkDefault	Low priority default; easy to override
mkForce	High priority; stomps others
mkOverride n	Numeric priority control

Consumer rule: prefer ordinary assignment first. Use mkForce only when you understand why a default wins.

```
# last resort
networking.hostName = lib.mkForce "lab";
```

Theory 6 — How to read a nixpkgs module

Step path

1. Find option on search.nixos.org/options

2. Note default and type
3. Open “Declared in” source file
4. Skim options then `config = mkIf cfg.enable`

Local source on a NixOS system

```
# approximate - paths vary by version
nix eval --raw 'nixpkgs#path'
# then browse nixos/modules/...
```

Or:

```
nix repl
:lf nixpkgs
# explore; or use ripgrep in a checkout
```

What to extract mentally

Question	Why
What does enable turn on?	Side effects
Which ports open?	Firewall
Which user created?	Permissions
What state dir?	Backup/persist
Hardening defaults?	Security

Theory 7 — Consumer anti-patterns

Anti-pattern	Prefer
Vendor-copy whole module	Set options; small local module
Patch store files	Declarative options / overlays later
mkForce everything	Understand merge
Ignore option descriptions	Read them
Edit generated unit files in /etc	Change Nix; rebuild

Theory 8 — imports as composition

```
{
  imports = [
    ./hardware-configuration.nix
    ./modules/ssh.nix
    ./modules/packages.nix
  ];
}
```

Each import is a module. Order rarely matters for merges (priorities do). the custom modules chapter authors one.

Worked examples bank

Example A — Read openssh options

Conceptually you will find:

```
options.services.openssh.enable = mkEnableOption "...";
config = mkIf cfg.enable { ... };
```

Set as consumer:

```
services.openssh.enable = true;
services.openssh.settings.PasswordAuthentication = false;
```

Example B — Override a default carefully

```
# module might mkDefault some package set
# you add:
environment.systemPackages = [ pkgs.htop ];
# lists merge - usually no mkForce needed
```

Example C — When mkForce appears in blogs

Often silencing a conflict:

```
boot.kernelPackages = lib.mkForce pkgs.linuxPackages_latest;
```

Use sparingly; document why.

Lab 1 — Study three modules

Pick **three** from:

- `services.openssh`
- `services.timesyncd` or `time`
- `programs.git`
- `networking.firewall`
- `services.nginx` (even if unused)

For each, fill:

Module	enable option	2 interesting options	One side effect

Keep notes in `~/nixos-config/docs/modules-study.md`.

Lab 2 — Change behavior via options only

Without forking module source:

1. Tighten or loosen one SSH setting
2. Change timezone or NTP-related option
3. Adjust firewall ports

Rebuild; verify with `systemctl / ss / ssh`.

Lab 3 — Trace an option to source

Using search.nixos.org or local `nixpkgs`:

1. Choose `networking.firewall.enable`
2. Find declaring file
3. Read how firewall backend is selected

Write five lines on what `enable = true` actually does.

Lab 4 — Priority experiment (safe)

In a throwaway module snippet:

```
{ lib, ... }: {  
  networking.hostName = lib.mkDefault "default-name";  
}
```

And in main config:

```
networking.hostName = "lab";
```

Confirm final hostname is `lab`. Then reverse priorities with `mkForce` experimentally and reset to clean state.

Lab 5 — Consumer checklist

Before inventing a custom module tomorrow, ask:

1. Does an option already exist?
 2. Can two options combine?
 3. Am I fighting a default that `mkDefault` set?
-

Common gotchas

Symptom	Theory
Option does not exist	Typos / wrong release
Infinite recursion	config referring badly in custom modules
Silent override	Priorities / mkForce elsewhere
Copied outdated module	Diverges from nixpkgs
mkIf confusion	Study enable patterns first

Checkpoint

- ☐ Three modules studied with notes
- ☐ Behavior changed via options only
- ☐ Can explain options vs config
- ☐ Can explain mkIf / mkDefault roles
- ☐ Avoided forking nixpkgs modules

Commit

```
cd ~/nixos-config
git add .
git commit -m "day18: modules as consumer mkIf mkDefault literacy"
```

Writing custom modules

Writing custom modules

Goal: Author a **small local module** with **options + config**, import it into the host, gate behavior with **mkIf**, and fail early with an **assertion**.

Why this day exists

Stage II ends with a production-shaped host—not a 400-line `configuration.nix`. Custom modules are how you name intent (`myLab.monitoring.enable`) instead of scattering unrelated assignments.

Theory 1 — Minimal custom module shape

modules/hello-lab.nix:

```
{ config, lib, pkgs, ... }:
let
  cfg = config.myLab.hello;
in {
  options.myLab.hello = {
    enable = lib.mkEnableOption "hello lab demo module";

    package = lib.mkOption {
      type = lib.types.package;
      default = pkgs.hello;
      description = "Package providing the hello binary";
    };

    greetInterval = lib.mkOption {
      type = lib.types.str;
      default = "hourly";
      description = "systemd OnCalendar expression for greeting timer";
    };
  };

  config = lib.mkIf cfg.enable {
    environment.systemPackages = [ cfg.package ];
  };
}
```

```

systemd.services.my-lab-hello = {
  serviceConfig = {
    Type = "oneshot";
    ExecStart = "${cfg.package}/bin/hello";
  };
};

systemd.timers.my-lab-hello = {
  wantedBy = [ "timers.target" ];
  timerConfig = {
    OnCalendar = cfg.greetInterval;
    Persistent = true;
    Unit = "my-lab-hello.service";
  };
};
};
}

```

Theory 2 — mkOption and types (starter set)

Type	Use
lib.types.bool	Flags
lib.types.str	Strings
lib.types.int	Integers
lib.types.package	Packages
lib.types.listOf lib.types.str	Lists
lib.types.attrsOf ...	Maps
lib.types.port	Ports
lib.types.nullOr ...	Optional

Deep type catalogs come later. Today: **bool** + **package** + **str** is enough.

`mkEnableOption "doc" → boolean option with standard description pattern.`

Theory 3 — cfg = config... pattern

Always bind:

```
let cfg = config.myLab.hello; in ...
```

Then `mkIf cfg.enable` and read `cfg.package`. Avoid long repeated paths; avoid accidental recursion by not defining options in terms of themselves carelessly.

Theory 4 — Assertions

```
config = lib.mkIf cfg.enable {
  assertions = [
    {
      assertion = cfg.greetInterval != "";
      message = "myLab.hello.greetInterval must be non-empty";
    }
  ];
};
```

Assertions fail **evaluation** with a clear message—better than a mysterious empty timer.

Also available: `warnings = [ "..." ]`; for non-fatal nags.

Theory 5 — Import into the host

hosts/lab/configuration.nix:

```
{
  imports = [
    ./hardware-configuration.nix
    ../../modules/hello-lab.nix
  ];

  myLab.hello.enable = true;
  # myLab.hello.greetInterval = "daily";
}
```

Or list modules in `flake.nix` `modules = [ ... ]`.

Theory 6 — Namespace your options

Good	Risky
<code>myLab.* / myOrg.*</code> Clear enable flags	Top-level inventing <code>services.foo</code> that collides Global options without prefix

Use a personal/org prefix until you contribute upstream.

Theory 7 — What to extract first

Extract **cohesive features**, not random lines:

Good extraction	Weak extraction
“lab SSH hardening bundle”	One unrelated package
“tick timer feature”	Half a firewall + half a user
“admin tools profile”	Entire host config as one module forever

God modules become tomorrow’s spaghetti.

Worked examples bank

Example A — Enable site

```
myLab.hello.enable = true;
```

Example B — Assertion trip

Set an invalid option deliberately; rebuild; read error; fix.

Example C — Package override

```
myLab.hello.package = pkgs.hello; # or another pkg with similar interface carefully
```

Lab 1 — Create module file

```
mkdir -p ~/nixos-config/modules
# write modules/hello-lab.nix (Theory 1)
```

Import + enable. Rebuild:

```
cd ~/nixos-config
sudo nixos-rebuild switch --flake .#lab
hello
systemctl list-timers | grep my-lab-hello || true
```

Lab 2 — Add a real assertion

Extend options with something checkable, e.g.:

```
port = lib.mkOption {
  type = lib.types.port;
  default = 22;
};
```

Assert `port != 0` or that firewall contains the port when a hypothetical service is enabled—keep it simple but real.

Lab 3 — Extract something from your host config

Identify 10–40 lines in `configuration.nix` (packages bundle, SSH settings, timer) and move into `modules/<feature>.nix` with an enable flag.

Before/after: host file should get shorter and more intentional.

Lab 4 — Disable path

```
myLab.hello.enable = false;
```

Rebuild; confirm packages/units from the module disappear (or stop being enabled). Re-enable.

Lab 5 — Document the module

modules/README.md:

```
# modules
## hello-lab
Options: myLab.hello.*
Purpose: ...
```

Theory 8 — mkDefault vs bare assignment (producer side)

Inside your module's config, prefer leaving room for host overrides:

```
config = lib.mkIf cfg.enable {
  environment.systemPackages = [ cfg.package ];
  # If you set a global-ish value, consider mkDefault so hosts can override:
  # networking.firewall.allowPing = lib.mkDefault true;
};
```

Helper	Intent
bare =	Force (subject to merge priority rules)
mkDefault	Soft default
mkForce	Rare; win conflicts deliberately

the modules as consumer chapter covered consumer-side `mkDefault`; today you **emit** values carefully.

Theory 9 — Option descriptions are UX

```
greetInterval = lib.mkOption {
  type = lib.types.str;
  default = "hourly";
  example = "daily";
  description = ''
    systemd OnCalendar expression for the greeting timer.
    Prefer named calendars like hourly/daily while learning.
  '';
};
```

Future you reads `nixos-option` output under pressure—write for that moment.

Theory 10 — Module args you receive

```
{ config, lib, pkgs, modulesPath, ... }:
```

Arg	Use
<code>config</code>	Read final/merged values (<code>cfg = config...</code>)
<code>lib</code>	<code>mkOption</code> , <code>mkIf</code> , <code>types</code> , ...
<code>pkgs</code>	Packages for defaults and units
<code>modulesPath</code>	Import nixpkgs NixOS modules by path (advanced)

Do not add custom top-level args unless you wire `specialArgs` / `_module.args` intentionally (later).

Worked example D — warnings for soft policy

```
config = lib.mkIf cfg.enable {
  warnings =
    lib.optional (cfg.greetInterval == "minutely")
      "myLab.hello.greetInterval = minutely will spam logs on the lab host";
};
```


Lab 6 — nixos-option on your option

```
nixos-option myLab.hello.enable
nixos-option myLab.hello.greetInterval
```

If the option is missing, the module is not imported into the evaluated system.

Lab 7 — Priority conflict drill

Set the same option in host config and inside the module (with/without `mkDefault`). Rebuild and note which wins. Write one sentence on the result in `modules/README.md`.

Common gotchas

Symptom	Theory
Infinite recursion	options/config cycle
Option not found	Not imported
Type error	Wrong types.*
Module always on	Forgot <code>mkIf</code>
Colliding option names	Namespace
Assertion silent	Not inside merged config properly

Checkpoint

- ☐ Custom module with options + config
- ☐ `mkIf` on enable
- ☐ At least one assertion
- ☐ Imported into host flake
- ☐ Extracted real config into a feature module
- ☐ Enable/disable verified

Commit

```
cd ~/nixos-config  
git add .  
git commit -m "day19: first custom module options assertions"
```

Boot and hardware

Boot and hardware

Goal: Document and intentionally configure the lab’s **boot loader**, **kernel packages**, and **firmware**—so recovery and hardware quirks are not mysteries.

! Important

Boot experiments can brick a bad disk layout. Prefer the **disposable VM**, snapshot first, and change one variable at a time.

Why this day exists

When generations fail to boot, the difference between “NixOS is broken” and “I know how to pick last generation / fix EFI” is this literacy. Hardware config is part of ownership.

Theory 1 — Boot chain (UEFI lab)

```
UEFI firmware
→ EFI system partition (ESP) files
→ boot loader (systemd-boot or GRUB)
→ kernel + initrd (from a generation)
→ stage-1 (find root, unlock, ...)
→ stage-2 / systemd
```

NixOS generations typically install loader entries pointing at store paths for kernel/initrd.

Theory 2 — systemd-boot vs GRUB

Loader	Common on	Notes
systemd-boot	UEFI VMs/laptops	Simple, generation entries
GRUB	BIOS + UEFI	More knobs; legacy friendly

```
# systemd-boot (UEFI)
boot.loader.systemd-boot.enable = true;
boot.loader.efi.canTouchEfiVariables = true;

# GRUB example (do not enable both casually)
# boot.loader.grub.enable = true;
# boot.loader.grub.device = "/dev/vda";
```

Pick one model. Mixing without care is a classic footgun.

Generation UI

- systemd-boot: machine-id entries, often need key to show menu
- GRUB: menu with generation list

Document how **your** VM displays the menu (ESC, hold space, etc.).

Theory 3 — Kernel packages

```
boot.kernelPackages = pkgs.linuxPackages;           # default channel kernel
# boot.kernelPackages = pkgs.linuxPackages_latest;
# boot.kernelPackages = pkgs.linuxPackages_6_12; # example versioned attr - check 26.05
```

Choice	Trade-off
Default	Matches nixpkgs expectations
<code>_latest</code>	Newer hardware; more churn
Pinned series	Stability / specific needs

Kernel modules:

```
boot.kernelModules = [ ];
boot.initrd.availableKernelModules = [ /* often from hardware-config */ ];
boot.extraModulePackages = [ ];
```

`hardware-configuration.nix` usually fills `initrd` modules for disks/filesystems—do not delete blindly.

Theory 4 — Firmware

```
hardware.enableRedistributableFirmware = true;
# hardware.enableAllFirmware = true; # broader; licensing awareness
```

VMs often need little firmware; bare metal Wi-Fi/GPU might need redistributable firmware enabled.

Theory 5 — CPU / virt hardware bits

Common NixOS knobs:

```
# virtualisation-friendly examples (enable only if true for you)
# services.qemuGuest.enable = true;
# virtualisation.hypervGuest.enable = true;
```

Also: microcode updates on real Intel/AMD machines (`hardware.cpu.*.updateMicrocode`)—relevant on bare metal, less so on many VMs.

Theory 6 — `hardware-configuration.nix` discipline

Do	Don't
Import it	Hand-edit UUIDs casually
Regenerate after disk changes	Copy another machine's hardware file
Review <code>initrd</code> modules	Assume it never needs refresh

Regenerate carefully (live media or docs)—wrong root UUID → unbootable.

Theory 7 — Temporary boot parameters

Kernel params:

```
boot.kernelParams = [ "console=ttyS0" ]; # example serial console for some VMs
```

Useful for debugging; remove noise when done. One change per rebuild when experimenting.

Worked examples bank

Example A — Document current loader

```
bootctl status 2>/dev/null | head -40
ls /boot 2>/dev/null | head
ls /boot/loader/entries 2>/dev/null | head
```

Example B — Kernel identity

```
uname -r
nixos-version
ls /run/current-system/kernel
```

Example C — Safe param experiment

Add a harmless `boot.kernelParams` comment-level change only after snapshot; or set a documented console param if your hypervisor uses serial.

Lab 1 — Boot path document

Create `docs/boot.md` in the flake repo:

```
# Boot path - lab
- Firmware: UEFI / BIOS
- Loader: systemd-boot / GRUB
- ESP mount: /boot (options)
- Kernel package option: ...
- uname -r: ...
```

- How to open generation menu: ...
- Snapshot name: ...

Fill with real commands' output.

Lab 2 — Verify loader config matches reality

Compare flake `boot.loader.*` with `bootctl status` / GRUB presence. Fix mismatches.

Lab 3 — Kernel package awareness

1. Record current `boot.kernelPackages` (implicit default if unset).
2. Optionally try `linuxPackages_latest` in a VM with snapshot.
3. Reboot; `uname -r`; note difference.
4. Revert if not needed.

If you skip the change, still write *how* you would do it.

Lab 4 — Firmware decision

State in docs:

- VM only → firmware settings chosen
- Bare metal → enable redistributable firmware if devices need it

Rebuild if you change it.

Lab 5 — Recovery dry-run (no panic)

Without breaking the system:

1. List generations in bootloader docs
2. Rehearse verbally: “If black screen after upgrade, I will...”
3. Update the rebuild and generations chapter rollback card with boot-menu specifics

Theory 8 — `boot.loader.timeout` and generation menu

```
boot.loader.timeout = 3; # seconds; 0 may hide menu unless key held
```

For labs that practice rollback, a short non-zero timeout is friendlier than zero. Document the key to force the menu on your firmware (often Esc or hold Space for systemd-boot).

Theory 9 — Initrd and LUKS awareness (preview)

Encrypted roots need initrd unlock configuration. Even if your VM is plaintext:

Concept	Why note it
<code>boot.initrd.luks.devices</code>	Maps UUID → mapped name
Prompt vs keyfile	Physical vs automated unlock
Wrong UUID	Emergency shell at stage-1

Disko/impermanence (Stage VI) will revisit layout; today only name the risk.

Theory 10 — hardware-configuration.nix vs flake purity

hardware-configuration.nix is **machine-specific generated state**. It belongs in git for the lab host **or** is generated on first install and kept private per machine.

Approach	Trade-off
Commit lab VM hardware file	Easy clone of <i>this</i> VM
Generate per machine	Correct for fleet; clone drill needs care

Do not copy UUIDs between bare metal and VM casually.

Worked example D — QEMU guest agent (virt)

```
services.qemuGuest.enable = true; # if hypervisor is QEMU/KVM
```

Helps shutdown/reboots from host tooling. Enable only when accurate.

Lab 6 — Loader entries inventory

```
bootctl list 2>/dev/null | head -60
ls -la /boot/loader/entries 2>/dev/null | head
```

Map one entry to a NixOS generation number in docs/boot.md.

Lab 7 — Kernel param one-shot (snapshot)

With a VM snapshot:

```
boot.kernelParams = [ "loglevel=7" ]; # noisier logs; lab only
```

Rebuild, reboot, confirm cat /proc/cmdline, revert.

Common gotchas

Symptom	Theory
Two boot loaders enabled	Conflicts
<code>canTouchEfiVariables</code> false in some virt	May need alternate install path
Removed initrd modules	Disk not found
Latest kernel + guest tools mismatch	Virt drivers
Editing <code>/boot</code> by hand	Lost on rebuild

Checkpoint

- ☐ `docs/boot.md` complete
- ☐ Loader in flake matches reality
- ☐ Kernel identity recorded
- ☐ Firmware policy chosen
- ☐ Rollback card updated with boot menu
- ☐ Snapshot available before any kernel play

Commit

```
cd ~/nixos-config
git add .
git commit -m "day20: boot hardware documentation"
```

Filesystems and state

Filesystems and state

Goal: Inventory mounts and partitions; separate **store**, **state**, and **ephemeral** paths; understand the **impermanence idea** (persist deliberately) without implementing full impermanence yet.

Why this day exists

Nix makes the system closure rebuildable; it does **not** automatically make your data policy sane. Ops failures often come from “I rolled back the system but the database is gone” or the reverse. Label what must persist.

Theory 1 — Three layers of “the disk”

Layer	Examples	Rebuild replaces?
Store	<code>/nix/store</code>	Via new paths; GC collects dead
Config activation	much of <code>/etc</code>	Yes, from generation
State	<code>/var</code> , <code>/home</code> , databases	No (unless you design it)

```
/home /var (state)      ← backup & persist policy
/etc (mixed; mostly managed)
/nix/store (immutable pkgs)
```

Theory 2 — fileSystems options

From `hardware-configuration.nix` typically:

```
fileSystems."/ = {
  device = "/dev/disk/by-uuid/...";
  fsType = "ext4";
};

fileSystems."/boot" = {
  device = "/dev/disk/by-uuid/...";
  fsType = "vfat";
};
```

Field	Role
device	UUID/LABEL preferred over raw <code>/dev/sdX</code>
fsType	ext4, btrfs, xfs, vfat, zfs, ...
options	mount flags (<code>noatime</code> , ...)
neededForBoot	initrd critical mounts

Swap:

```
swapDevices = [{ device = "/dev/disk/by-uuid/..."; }];
```

Theory 3 — Why UUID/LABEL

Device names (`vda2`, `sda1`) can reorder. By-uuid is stable:

```
ls -l /dev/disk/by-uuid
lsblk -f
```

Theory 4 — State directories that matter

Path	Typical content
<code>/var/lib</code>	Service state (postgres, docker, ...)
<code>/var/log</code>	Logs (sometimes ephemeral OK)
<code>/var/cache</code>	Regenerable caches
<code>/home</code>	User data, secrets, keys
<code>/root</code>	Root's home
<code>/etc/ssh</code> host keys	Machine identity (special)

Path	Typical content
/nix	Store + nix state database

When you enable services (Stage IV+), each service’s state dir is part of your **persist list**.

Theory 5 — Impermanence idea (concept only)

Impermanence (pattern/tooling, the impermanence chapter deep dive): treat root as mostly ephemeral; **bind-mount** or **symlink** only chosen paths from a persistent volume.

```
boot → empty-ish root
      → restore/persist: /home, /var/lib/postgres, /etc/ssh, ...
      → everything else regenerated by NixOS activation
```

Why people want it

Benefit	Meaning
Force declarative truth	Untracked snowflakes disappear
Cleaner systems	Less drift
Clear backup targets	Persist volume = backup unit

Costs

Cost	Meaning
Design work	Must list persist paths correctly
Host keys / secrets	Easy to forget; painful
Debugging	“Where did my file go?”

Today: you do **not** need to enable impermanence. You **do** need a written list of what *would* be persisted.

Theory 6 — Store is not a backup of state

```
nixos-rebuild switch
→ new system closure
  snapshot of /var/lib/postgresql
```

Backups are a separate story (the backups chapter). Generations help **software**; not **data**.

Theory 7 — tmpfs and ephemeral mounts (awareness)

```
# conceptual examples - do not apply casually
# boot.tmp.useTmpfs = true;
# fileSystems."/tmp" = { device = "tmpfs"; fsType = "tmpfs"; };
```

Useful later with impermanence. Understanding: not every path needs disk durability.

Worked examples bank

Example A — Mount inventory

```
findmnt -R /
lsblk -f
df -hT
```

Example B — What's big?

```
sudo du -xhd1 / 2>/dev/null | sort -h
sudo du -sh /nix/store /var /home 2>/dev/null
```

Example C — SSH host key paths

```
ls -l /etc/ssh/ssh_host_*
```

These are state even though under `/etc`.

Lab 1 — Full inventory table

In docs/filesystems.md:

Mountpoint	device/UUID	fsType	options	role (root/boot/state/...)
------------	-------------	--------	---------	----------------------------

Fill from `findmnt + lsblk -f`.

Lab 2 — Persist classification

Label each:

Path	Persist?	Why	Backup?
/nix/store	rebuildable		no (or cache strategy)
/home/alice	yes		yes
/var/log	maybe		optional
/var/lib	mostly yes		yes
/etc/ssh host keys	yes	identity	yes
/etc/nixos if still used	prefer git flake		git

Be specific to **your** lab services.

Lab 3 — State created by your Stage II work

List state produced so far:

- user homes
- SSH host keys
- timer logs you wrote
- any DB? (probably not yet)

Lab 4 — Impermanence reading (light)

Skim what the impermanence chapter will cover (impermanence module concepts) from memory of Theory 5. Write five sentences: *Would I want impermanence on this lab VM? Why/why not?*

No implementation required.

Lab 5 — Danger checklist

Confirm you will **never**:

- `rm -rf /nix/store` as “cleanup”
 - Format ESP casually
 - Delete `/var/lib` because “Nix will regenerate” (it will not for app data)
-
-

Theory 8 — Btrfs / ZFS awareness without deploying them

You may see labs using btrfs snapshots or ZFS datasets. For Stage II:

FS	Persist story
ext4 default VM	Simple; backups = copy/dump
btrfs	Subvolumes + snapshots possible
ZFS	Datasets; often external to vanilla hardware-config

Do not switch root FS casually mid-lab. Understand that snapshot tools are about **state**, not store purity.

Theory 9 — `/nix` on its own mount (optional design)

Some installs put `/nix` on a dedicated volume:

Benefit	Cost
Reinstall root without losing store cache	More mount points to get right
Clear size accounting	Hardware-config complexity

Record whether your lab is single-root or split.

Theory 10 — Bind mounts as proto-impermanence

```
# conceptual - only if you knowingly add a data disk
# fileSystems."/var/lib/postgres" = {
#   device = "/data/postgres";
#   options = [ "bind" ];
# };
```

Bind mounts are how many impermanence setups pin state onto a durable dataset. Note the idea; implement later (the impermanence chapter).

Worked example D — neededForBoot meaning

```
fileSystems."/" = {
  device = "/dev/disk/by-uuid/...";
  fsType = "ext4";
  neededForBoot = true; # default true for /
};
```

Extra mounts required to reach multi-user (e.g. separate `/var`) may need `neededForBoot = true` or the system reaches emergency targets.

Lab 6 — What survives nixos-rebuild switch?

Create a marker:

```
echo "state-$(date -Is)" | sudo tee /var/tmp/day21-marker
echo "etc-try" | sudo tee /etc/day21-hand-edit 2>/dev/null || true
```

Rebuild switch. Check which markers remain. Note: hand-edited `/etc` files may be replaced by activation—another reason flake is SoT.

Lab 7 — Size budget sentence

From Lab 2 `du` output, write one budget sentence:

“If I wipe the VM disk, I lose X (state) but can rebuild Y (system) from `git+flake.lock`.”

Common gotchas

Symptom	Theory
Rolled back OS, data weird	Data generation
Copied hardware-config UUIDs wrong	Unbootable
Thought <code>/etc</code> fully ephemeral	Host keys, some static files
No backup of home	Lab loss on disk wipe
Impermanence without persist list	Instant footgun

Checkpoint

- ☐ Mount inventory table complete
- ☐ Persist classification written
- ☐ Store vs state explained in own words
- ☐ Impermanence idea summarized
- ☐ Host keys noted as state
- ☐ Docs committed

Commit

```
cd ~/nixos-config  
git add .  
git commit -m "day21: filesystems inventory persist vs store"
```

Home and Flake Layout

Home Manager and flake layout

Home Manager and flake layout

This part of the **standalone NixOS book** is organized by **topic** (normal book chapters), not by calendar day.

Read chapters in order within the part when learning; skip or skim freely when using the book as a reference.

Baseline: NixOS / nixpkgs **26.05**, flakes, modern **nix** CLI, Home Manager as a **NixOS module**.

What this part is for

You already own a host from a flake (**nixosConfigurations**). That is necessary and insufficient for a maintainable personal platform. This part covers the **shape** of a multi-concern flake and the **user plane**:

Concern	Chapters
Repo layout that scales past one file	Flake repository layout
Pins you can explain and revert	Lock file discipline
User environment as NixOS module	Home Manager on NixOS
Dotfiles, XDG, store-backed files	Home files and XDG
User systemd units and timers	Home Manager user services
Project toolchains without profile bloat	direnv and project shells
Multi-output flake structure	flake-parts (optional structure)
Local quality gates	Flake checks
Shared package tweaks	Overlays (light introduction)

After this part you should be able to hand a second host or a second user a **predictable import graph**, activate system + home with one rebuild, and keep project compilers out of the always-on profile.

Prerequisites

From earlier parts of this book you should already be comfortable with:

- Store paths, closures, and why the store is not private
- Attrsets, modules, and `nixosSystem`
- A working flake host: `sudo nixos-rebuild switch --flake .#<host>`
- Generations and rollback
- Basic SSH/user config on the host

If those are shaky, return to **concepts** and **NixOS host** before deepening layout and Home Manager.

Chapter map

Chapter	Focus	Outcome
Flake repository layout	<code>hosts/ · modules/ · homes/</code> DAG	Thin <code>flake.nix</code> ; rebuild-equivalent split
Lock file discipline	Selective updates, <code>follows</code> , registries	Written lock policy; revert skill
Home Manager on NixOS	HM as NixOS module, 26.05 pin	One rebuild activates system + user
Home files and XDG	<code>home.file</code> , <code>xdg.configFile</code>	Real configs migrated; secrets out
Home Manager user services	<code>systemd.user.*</code>	Timers/services that survive reinstall
<code>direnv</code> and project shells	<code>devShells</code> + <code>use flake</code>	Toolchains load on <code>cd</code>
flake-parts (optional structure)	Framework vs hand-rolled helpers	House style documented
Flake checks	<code>checks</code> + <code>nix flake check</code>	Format/shell/host gates
Overlays (light introduction)	Small wrappers / overrides	Same overlay on host + shell

Mental model — three planes

```
flake.nix (wiring only)
  inputs → nixosConfigurations / shells
```

```
hosts/      modules/    homes/
(machine)   (policy)    (user / HM)
```

Plane	Truth lives in	Activated by
Host	hosts/<name>/, hardware	nixos-rebuild
Policy	modules/common, roles, services	imported by hosts
User	homes/<user>/ via Home Manager	same rebuild (module mode)

Rule: host never imported by a common module; user config does not replace system daemons; project compilers do not live forever in `home.packages`.

Dependency direction (import DAG)

```
flake.nix
→ hosts/lab
  → modules/common/*
  → modules/roles/*
  → modules/services/*
  → hardware-configuration
  → home-manager.users.<name> → homes/<name>
```

Allowed	Forbidden
Host imports modules	Module imports a specific host
Role imports common	Common imports role
Service module self-contained	Service reaches into <code>hosts/lab</code>
Flake wires HM users	Homes import host hardware

Circular graphs fail as infinite recursion or missing attributes. Design the DAG **before** splitting files.

Suggested repo tree (target shape)

```
flake.nix
flake.lock
hosts/
  lab/
    default.nix
    hardware-configuration.nix
modules/
  common/
    default.nix
    nix.nix
    users.nix
    ssh.nix
  roles/
    server.nix
    workstation.nix
  services/
homes/
  alice/
    default.nix
    programs.nix
    files.nix
    services.nix
overlays/
  default.nix
lib/
docs/
  layout.md
  lock-policy.md
  flake-style.md
```

Names are conventions; **layering** is the invariant.

How to work this part

1. **Layout first** — behavior-preserving split of an existing host flake.
2. **Lock policy** — before adding HM, sops, or flake-parts inputs casually.
3. **HM module mode** — one activation path with the host.
4. **Files and services** — migrate real user state deliberately.
5. **direnv** — project shells, not profile bloat.
6. **Structure + checks + light overlays** — multi-output hygiene.

Prefer small commits: move-only, then feature. Do not mix layout refactors with hardening or new network services.

Cross-cutting conventions (26.05)

Topic	Convention in this book
nixpkgs	<code>github:NixOS/nixpkgs/nixos-26.05</code>
Home Manager	<code>github:nix-community/home-manager/release-26.05</code> + follows
HM activation	NixOS module (<code>home-manager.nixosModules.home-manager</code>)
<code>useGlobalPkgs</code>	<code>true</code> so overlays reach HM
Secrets in homes	Never plaintext tokens in <code>home.file</code> / <code>text</code>
Project tools	flake <code>devShells</code> + <code>direnv</code> , not eternal <code>home.packages</code>

What this part does *not* cover

Topic	Where it lives
sops-nix / agenix	Services and security patterns
Fleet deploy, Disko, CI	Ops and fleet
Full packaging / overlays mastery	Packaging
Day-calendar curriculum	Not this book's model

Skills checklist (end of part)

- ☐ Thin flake wiring; `hosts` / `modules` / `homes` tree
- ☐ Import DAG has no reverse host deps
- ☐ Lock policy written; selective update + revert practiced
- ☐ HM module mode: `git` + shell for at least one user
- ☐ Two real configs via `XDG` / `home.file` or `programs.*`
- ☐ One user timer or service via HM
- ☐ One project with `use flake` + `direnv`
- ☐ House flake style (hand-rolled or `flake-parts`) documented
- ☐ `nix flake check` fails on intentional format break
- ☐ One light overlay or wrapper visible on host and shell

How chapters are structured

Each topic chapter typically includes:

1. **Goal** and callouts (baseline, pitfalls)
2. **Why this chapter exists** — problem framing
3. **Theory** sections with tables and full Nix snippets
4. **Worked examples** you can paste and adapt
5. **Exercises** (8–15) with verify commands
6. **Common gotchas**
7. **Checkpoint** for self-assessment

Indexes (this file and part openers) orient; topic chapters go deep.

Reading strategies

Mode	Approach
First pass	Layout → lock → HM → files → services → direnv; then style/checks/overlays
Reference	Jump to gotchas + worked example for the tool you need
Audit existing flake	Layout + lock + HM boundary chapters first
Laptop vs server	Roles chapter patterns; lighter HM on servers

Lab host assumptions

Examples use:

- Host attr / hostname: `lab`
- User: `alice` (replace with yours)
- System: `x86_64-linux` (swap for `aarch64-linux` as needed)
- Rebuild: `sudo nixos-rebuild switch --flake .#lab`

Paths under `~/lab/...` are suggestions only—use your real flake repo.

Next part

Services and security patterns builds on this shape: secrets (sops-nix), hardening, firewall, TLS front door, data services, containers, observability. Layout and HM must be boring before those topics get loud.

Flake repository layout

Flake repository layout

Goal: Refactor a single-host flake into a **hosts** / **modules** / **homes** layout that stays rebuildable, readable, and ready for multi-host growth—without a 2 000-line `flake.nix`.

i Note

Baseline: NixOS / nixpkgs **26.05**, flakes, modern `nix` CLI. Home Manager is wired in the Home Manager chapter; in this chapter only **create the homes/ slot** so later imports have a home.

Why this chapter exists

Owning one machine from a flake proves control. **Shape** is what keeps that control after a second host, a second user, or the first “extract this SSH policy” refactor.

A common failure mode:

```
flake.nix          # 1800 lines of mixed host, packages, users, networking
hardware-configuration.nix
```

That works until the import graph becomes a ball of yarn. Layout is not aesthetics; it is **dependency direction and change-surface control**.

Theory 1 — What “layout” means in a flake monorepo

A flake has two layers:

Layer	Role
flake.nix outputs	Thin wiring: inputs → <code>nixosConfigurations</code> , later <code>devShells</code> / <code>checks</code> / <code>packages</code>
Module tree	Almost all real config: options + config fragments imported by hosts

Flake monorepo shape (hosts · modules · homes)

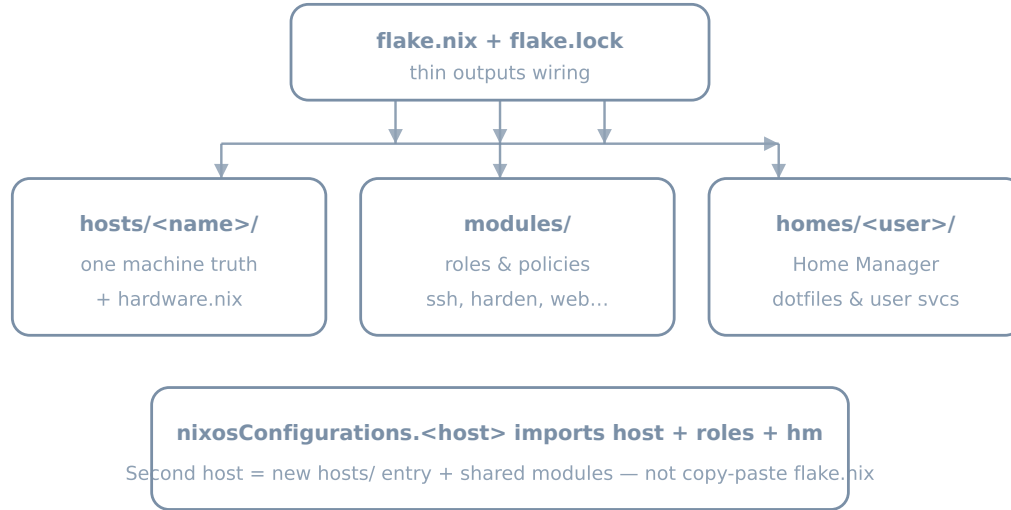


Figure 1: Flake hosts modules homes layout

Rule of thumb: if a file is only true for *one machine*, it lives under **hosts/<name>/**. If it is true for *a role or policy*, it lives under **modules/**. If it is true for *a user*, it will live under **homes/** (Home Manager starts in the next chapters).

Target tree

```

flake.nix
flake.lock
hosts/
  lab/
    default.nix          # host entry: imports modules + hardware
    hardware-configuration.nix
modules/
  common/
    default.nix          # baseline shared by all hosts
    nix.nix
    users.nix
    ssh.nix
    networking.nix
  roles/
    workstation.nix
    server.nix
  services/
    # empty or one tiny service extracted from the monolith
homes/

```

```

    alice/
      default.nix          # stub until HM chapter
  lib/
    default.nix          # optional helpers (mkHost, ...)
  docs/
    layout.md

```

Names are conventions, not law. **The invariant is layering**, not exact folder names.

Theory 2 — Dependency direction (prevent circular imports)

Think of imports as a DAG:

```

flake.nix
→ hosts/lab
    → modules/common/*
    → modules/roles/*
    → modules/services/*
    → hardware-configuration
    → (later) homes via HM module

```

Allowed	Forbidden
Host imports modules	Module imports a <i>specific</i> host
Role imports common modules	Common imports role
Service module is self-contained	Service module reaches into <code>hosts/lab</code> paths
<code>lib</code> pure helpers used by modules	<code>lib</code> that evaluates full NixOS configs

Circular imports fail evaluation with confusing “infinite recursion” or “attribute missing” symptoms. Design the graph **before** you split files blindly.

What belongs where

Concern	Place
<code>networking.hostName</code> , hardware, disk labels	<code>hosts/<name>/</code>
SSH policy, firewall defaults, time zone, locale	<code>modules/common/</code>
“This is a headless server” package set	<code>modules/roles/server.nix</code>
One service option bundle	<code>modules/services/<name>.nix</code>
User shell, git, editors	<code>homes/<user>/</code>
<code>specialArgs</code> , input plumbing	<code>flake.nix</code> only

Theory 3 — Host entry pattern

A host file is a **module**, not a second flake:

```
# hosts/lab/default.nix
{ config, pkgs, lib, ... }:
{
  imports = [
    ./hardware-configuration.nix
    ../../modules/common
    ../../modules/roles/server.nix
    # ../../modules/services/my-timer.nix
  ];

  networking.hostName = "lab";

  # Host-only overrides (mkForce / mkDefault as needed)
  # services.openssh.openFirewall = true;
}
```

And the flake stays thin:

```
# flake.nix
{
  description = "NixOS lab - hosts/modules/homes layout";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  };

  outputs = { self, nixpkgs, ... }:
  let
    system = "x86_64-linux"; # or aarch64-linux
  in {
    nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
      inherit system;
      modules = [
        ./hosts/lab
      ];
      # specialArgs = { inherit inputs; }; # when modules need inputs
    };
  };
}
```

default.nix vs explicit path

Importing `./hosts/lab` loads `./hosts/lab/default.nix`. Prefer that over listing every leaf in `flake.nix`.

specialArgs vs module arguments

Mechanism	Use
<code>specialArgs = { inherit inputs; }</code>	Pass flake inputs into all host modules
<code>_module.args</code>	More advanced; avoid until needed
<code>import of pure lib/</code>	Helpers that need no <code>config</code>

Do not thread half the world through `specialArgs` “just in case.” Add when a module actually needs an input (Home Manager, `sops-nix`, ...).

Theory 4 — Module “barrel” files

`modules/common/default.nix` re-exports fragments:

```
# modules/common/default.nix
{
  imports = [
    ./nix.nix
    ./users.nix
    ./ssh.nix
    ./networking.nix
  ];
}
```

Benefits:

- Hosts import **one** common entry
- You can still open a single policy file for review
- Diffs stay small when only SSH changes

Refactor **behavior-preserving** first: `cut` → `import` → `rebuild` → only then change options. Never mix layout moves with hardening or new services in one commit.

Optional deeper split

```
modules/common/  
  default.nix  
  nix.nix  
  users.nix  
  ssh.nix  
  networking.nix  
  locale.nix  
  time.nix
```

Split when a file exceeds ~150 lines or two people edit conflicting concerns—not on principle alone.

Theory 5 — Hardware config and purity

`hardware-configuration.nix` is **host-local** (UUIDs, firmware). Do **not** share it across machines.

Practice	Why
Keep under <code>hosts/<name>/</code>	Machine-specific
Generated by installer / <code>nixos-generate-config</code>	Not hand-invented blindly
Import only from that host	Avoid wrong disk UUIDs

Optional `lib/mkHost` helpers can wait until you have three hosts—prefer an obvious explicit `nixosSystem` until then.

Before → after: a monolith `configuration.nix` pointed from `flake.nix`; after, `flake.nix` points at `./hosts/lab`, which imports `hardware` + `modules/common` (+ `roles/services`). Same behavior; smaller change surface.

Theory 6 — Roles vs common vs services

Kind	Semantics	Example
common role	True for nearly every host Class of machine	flakes enabled, base SSH policy <code>server.nix</code> disables Bluetooth; <code>workstation.nix</code> enables NetworkManager
service	One workload option bundle	<code>modules/services/caddy.nix</code>

```
# modules/roles/server.nix
{
  imports = [ ../common ];
  # server-lean defaults
  # hardware.bluetooth.enable = false;
  # services.printing.enable = false;
}
```

Hosts compose roles; roles compose common. Services are imported by hosts or roles when that workload is on.

Theory 7 — Multi-host readiness (without building the second machine)

Sketch without wiring a non-bootable config:

```
hosts/
  lab/
  spare/      # optional empty skeleton
  default.nix # imports common + own hardware later

# flake.nix (future)
nixosConfigurations = {
  lab = nixpkgs.lib.nixosSystem { ... modules = [ ../hosts/lab ]; };
  # spare = nixpkgs.lib.nixosSystem { ... modules = [ ../hosts/spare ]; };
};
```

Do not invent fake hardware UUIDs “for symmetry.” Document the intended second host in docs/layout.md instead.

Theory 8 — Relative paths and flake root

After moves, relative imports break silently until eval:

From	Import
hosts/lab/default.nix	../../modules/common
modules/common/ssh.nix	no host paths
flake.nix	./hosts/lab only

Avoid absolute `/home/you/...` imports. Flake evaluation should work from any clone path.

Worked example — Minimal SSH + users + nix split

```
# modules/common/ssh.nix
{ lib, ... }:
{
  services.openssh = {
    enable = true;
    settings = {
      PasswordAuthentication = false;
      KbdInteractiveAuthentication = false;
      PermitRootLogin = "no";
    };
  };
}

# modules/common/users.nix
{ pkgs, ... }:
{
  users.users.alice = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];
    openssh.authorizedKeys.keys = [
      "ssh-ed25519 AAAA...REPLACE_WITH_YOUR_PUBLIC_KEY alice@lab"
    ];
  };
}

# modules/common/nix.nix
{
  nix.settings = {
    experimental-features = [ "nix-command" "flakes" ];
  };
  nixpkgs.config.allowUnfree = false;
}

# modules/common/networking.nix
{
  # hostName stays on the host; put only shared defaults here
  networking.firewall.enable = true;
  time.timeZone = "UTC"; # or your zone
}
```

```

}

# hosts/lab/default.nix
{ ... }:
{
  imports = [
    ./hardware-configuration.nix
    ../../modules/common
  ];

  networking.hostName = "lab";
  system.stateVersion = "26.05"; # keep your real stateVersion
}

```

Worked example — Thin flake with optional lib

```

# lib/systems.nix
{
  defaultSystem = "x86_64-linux";
}

# flake.nix
{
  description = "NixOS lab";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs, ... }:
  let
    inherit (import ./lib/systems.nix) defaultSystem;
  in {
    nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
      system = defaultSystem;
      modules = [ ./hosts/lab ];
    };
  };
}

```

Keep lib/ free of full system evaluation until you know you need it.

Exercises

Exercise 1 — Inventory the monolith

```
cd /path/to/your-flake
find . -name '*.nix' | sort
wc -l flake.nix configuration.nix hosts/*/*.nix 2>/dev/null || true
```

Journal:

1. List every *concern* in the big file (users, SSH, packages, networking, ...)
2. Mark each as **host** / **common** / **role** / **service** / **user**
3. Note any absolute paths or secrets (flag secrets for the secrets chapters—do not leave plaintext passwords in git)

Exercise 2 — Create the directory skeleton

```
mkdir -p hosts/lab modules/common modules/roles modules/services homes/alice lib docs
```

Move hardware config:

```
# adjust names to your layout
mv hardware-configuration.nix hosts/lab/ 2>/dev/null || true
cp configuration.nix /tmp/configuration.nix.bak 2>/dev/null || true
```

Exercise 3 — Split common modules

Create `ssh.nix`, `users.nix`, `nix.nix`, `networking.nix`, and the barrel `default.nix` from the worked examples. Replace keys and usernames with yours—**public keys only**.

Exercise 4 — Host entry + thin flake

Write `hosts/lab/default.nix`. Point `flake.nix` at `./hosts/lab` only. Remove or stop importing the old monolith path.

Exercise 5 — Build before switch

```
nixos-rebuild build --flake .#lab
sudo nixos-rebuild switch --flake .#lab
```

Verify:

```
hostname
systemctl is-active sshd
nixos-version
```

If something broke: roll back, fix one module, retry.

Exercise 6 — Document the tree

Write docs/layout.md:

```
flake.nix          → outputs only
hosts/lab          → hostname, hardware, host overrides
modules/common     → shared policy
modules/roles      → (empty or one role)
modules/services   → extracted service if any
homes/alice        → Home Manager chapters
```

Exercise 7 — Role extract (optional stretch)

Move 2–3 server-lean settings into modules/roles/server.nix and import from the host. Rebuild. Confirm behavior unchanged.

Exercise 8 — Path-break drill

Temporarily wrong a relative import (../common instead of ../../modules/common). Run nixos-rebuild build, read the error, fix. Note the message for future you.

Exercise 9 — Diff against backup

```
# conceptual: compare options that matter
diff -u /tmp/configuration.nix.bak hosts/lab/default.nix | head -80 || true
```

List any intentional deltas; there should be none for a pure layout move.

Exercise 10 — Second-host skeleton (docs only)

Write five lines in docs/layout.md about what a second host would import. Do not invent hardware.

Exercise 11 — Secrets flag list

From Exercise 1, list every sensitive-looking string. Either remove now or ticket for secrets chapters. No live tokens in git.

Exercise 12 — Commit hygiene

```
git add hosts modules homes lib docs flake.nix flake.lock
git status
git commit -m "flake: hosts/modules/homes layout (behavior-preserving)"
```

Prefer one commit for layout-only moves.

Common gotchas

Symptom / mistake	What to do
Infinite recursion after split	Module imports a host or config cycle; straighten the DAG
<code>error: path '...' does not exist</code>	Relative paths wrong after move; re-check <code>../../modules</code>
Flake still points at old <code>configuration.nix</code>	Update <code>modules = [../hosts/lab]</code> ;
Hardware UUIDs missing	Moved hardware file but forgot to import it
“Works on eval, fails on boot”	Bootloader/filesystem only in hardware host file—verify
Mixed absolute <code>/home/you/...</code> imports	Use flake-relative paths only
Giant PR of layout + features	Split commits: move-only first
<code>stateVersion</code> changed “to match release”	Do not casual-bump; keep original
Barrel forgets a file	Missing policy after split—add to imports
Role imports host	Invert: host imports role

Checkpoint

- ☐ Directory tree matches `hosts / modules / homes` (homes may be stub)
- ☐ `flake.nix` is thin wiring, not the bulk of config
- ☐ `nixos-rebuild switch --flake .#lab` (or your hostname) succeeds
- ☐ Import graph has no host→module reverse deps
- ☐ Layout documented in `docs/layout.md`
- ☐ Behavior roughly matches pre-refactor host
- ☐ No plaintext secrets introduced during the move
- ☐ `stateVersion` unchanged unless you know why

Journal (optional)

Write three personal gotchas before continuing—especially path errors and anything you almost put in the wrong plane.

Lock file discipline

Lock file discipline

Goal: Treat `flake.lock` as a **deliberate pin**, not a mysterious generated blob—update **one input at a time**, read diffs, and explain how locks interact with registries and multi-machine rebuilds.

! Important

Blind `nix flake update` on a production host is how “nothing changed except everything.” Prefer **staged, selective updates**.

Why this chapter exists

Flakes solve channel drift **only if** you respect the lock file. Teams fail when:

- Someone updates all inputs “because CI was green last week”
- Lock files are gitignored (never do this for apps/hosts)
- Two machines use different locks and argue about “Nix being unreproducible”
- Registries silently redirect `nixpkgs` away from the flake’s pin

Lock hygiene is foundational for every later input (Home Manager, `sops-nix`, `flake-parts`, ...).

Theory 1 — What `flake.lock` actually pins

For each flake input, the lock records roughly:

Field	Meaning
<code>type / owner / repo / url</code>	Where the input came from
<code>rev</code>	Exact git commit (or equivalent)
<code>narHash</code>	Content hash of the fetched tree
<code>lastModified</code>	Metadata for humans/tools

Implication: two clones with the **same lock** should fetch the **same input trees** (modulo substituted vs rebuilt outputs later). Without the lock, `nixos-26.05` is a **moving branch tip**.

Inputs vs follow-ons

```
inputs = {
  nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  home-manager.url = "github:nix-community/home-manager/release-26.05";
  home-manager.inputs.nixpkgs.follows = "nixpkgs";
};
```

`follows` means Home Manager reuses **your** nixpkgs pin instead of its own nested lock entry as a separate copy. That reduces “two nixpkgs versions in one eval” confusion and often saves eval work.

Anatomy of a lock node (conceptual)

```
"nixpkgs": {
  "locked": {
    "lastModified": 1700000000,
    "narHash": "sha256-...",
    "owner": "NixOS",
    "repo": "nixpkgs",
    "rev": "abcdef...",
    "type": "github"
  },
  "original": {
    "owner": "NixOS",
    "ref": "nixos-26.05",
    "repo": "nixpkgs",
    "type": "github"
  }
}
```

`original` is policy (track nixos-26.05); `locked` is the decision (this commit).

Theory 2 — Update operations (precision toolkit)

Command	Effect
<code>nix flake lock</code>	Refresh lock metadata without changing intended refs (often no-op if already locked)
<code>nix flake update</code>	Update all inputs to latest matching their url refs
<code>nix flake update nixpkgs</code>	Update only the nixpkgs input

Command	Effect
<code>nix flake update home-manager</code>	Update only that input
<code>nix flake metadata</code>	Show resolved pins / last modified

Modern CLI also supports updating multiple named inputs; check `nix flake update --help` on your Nix version.

Mental model

```
flake.nix  →  policy ("track nixos-26.05")
flake.lock →  decision ("this commit of that policy")
```

Changing `flake.nix` URLs without updating lock can error or keep stale locks depending on change type—always re-lock intentionally and commit both files together when inputs change.

Theory 3 — How to read a lock diff

```
git diff flake.lock | head -200
```

Look for:

1. **Which input** changed (`"nixpkgs": { block`)
2. **Old rev vs new rev**
3. **narHash change** (expected when rev changes)
4. Nested inputs (if not follows)

Optional deep inspection:

```
nix flake metadata
nix flake metadata --json | jq '.locks.nodes.nixpkgs'
```

Map `rev` to a GitHub compare URL when investigating regressions:

```
https://github.com/NixOS/nixpkgs/compare/<old>...<new>
```


Nested duplicates

If you see `nixpkgs_2` style nodes, a dependency brought its own `nixpkgs`. Prefer:

```
inputs.foo.inputs.nixpkgs.follows = "nixpkgs";
```

Then re-lock and confirm the duplicate disappears or shrinks.

Theory 4 — Pin strategies (pick a house rule)

Strategy	When	Risk
Track release branch (<code>nixos-26.05</code>) + lock commits	Default for this book	Branch tip moves; you choose when to update
Pin exact rev in <code>flake.nix</code>	Freeze hard for a milestone	Manual bumps; easy to forget security updates
Track <code>nixos-unstable</code>	Bleeding edge lab	Breakage rate higher
Multiple flakes (host vs tools)	Advanced separation	Operational complexity

Recommended for lab hosts: track `nixos-26.05` (and matching HM release), **commit lock**, update **one input per change window**, rebuild + smoke-test before updating the next.

Security vs stability

Updating `nixpkgs` is how you receive package and module fixes. Never freeze a public-facing host forever “because it works.” Discipline is **when and how**, not **whether**.

Cadence idea	Fit
Weekly selective <code>nixpkgs</code> on lab	Learning hosts
PR-based lock bumps in CI	Teams
Point release freezes for demos	Talks / workshops
Unattended full <code>flake update</code> nightly	Usually a bad default for hosts

Theory 5 — Registries (light literacy)

Flake **registries** map short names (`nixpkgs`) to URLs for ad-hoc commands like `nix run nixpkgs#hello`.

Concern	Practice
Global registry	Convenient for exploration
Your host flake	Should use explicit inputs , not hope the registry matches
CI	Prefer locked flake in repo over registry luck

```
nix registry list
```

If global `nixpkgs` points at unstable while your flake pins 26.05, **that is fine** for one-off `nix run`—as long as you never assume those are the same pin.

Registry vs input (summary)

```
nix run nixpkgs#hello      → registry resolution (may float)
nix build .#packages...    → your flake.lock
nixos-rebuild --flake .#   → your flake.lock
```

Theory 6 — Multi-machine and “reproducible”

Same lock + same system (`x86_64-linux`) → same **evaluation** of inputs. Realization may still:

- Download substitutes vs build locally
- Differ if impure eval sneaks in (`builtins.currentSystem` misuse, unpinned `fetchTarball`, IFD surprises)
- Differ across architectures

Lock discipline is **necessary, not always sufficient**, for bit-identical everything—but it is the main lever you control daily.

Clone checklist

```
git clone <repo>
cd <repo>
git status    # flake.lock must be present
nix flake metadata
```

```
sudo nixos-rebuild switch --flake .#lab
```

Missing lock → everyone invents their own “26.05 tip.”

Theory 7 — Commit policy for locks

Do	Don't
Commit <code>flake.lock</code> with the change that needs it	Gitignore lock for NixOS host flakes
One logical update per commit when possible	Mix lock bumps with huge unrelated refactors
Note <i>why</i> you updated in commit body	“chore: update” with no smoke test
Keep <code>flake.nix</code> + <code>flake.lock</code> paired in reverts	Restore only one of the pair

Example commit body:

```
flake: update nixpkgs (26.05 tip)

Smoke: sshd active, rebuild switch OK, hello from devShell OK.
```

Revert pattern

```
git checkout HEAD~1 -- flake.lock
# if flake.nix inputs also changed, restore that too:
# git checkout HEAD~1 -- flake.nix flake.lock
sudo nixos-rebuild switch --flake .#lab
```

Theory 8 — Inputs that should co-move

Pair	Rule of thumb
nixpkgs nixos-26.05 + HM release-26.05	Keep major/minor aligned
nixpkgs + sops-nix	follows nixpkgs; update sops after nixpkgs if needed

Pair	Rule of thumb
flake-parts + nixpkgs	Framework usually tracks independently; still follows when offered

Updating HM without nixpkgs (or the reverse) is a common source of obscure option errors. Prefer: update nixpkgs → rebuild → update HM → rebuild.

Worked example — Selective update narrative

```
Session A:  update nixpkgs only → rebuild lab → OK
Session B:  add home-manager input → lock appears → rebuild
Session C:  update home-manager only → HM activation issue → revert lock hunk
```

Reverting a bad lock is often faster than debugging three simultaneous input moves.

Worked commands

```
cp flake.lock /tmp/flake.lock.bak
nix flake update nixpkgs
git diff --stat flake.lock
nixos-rebuild build --flake .#lab
# if OK:
sudo nixos-rebuild switch --flake .#lab
# if bad:
cp /tmp/flake.lock.bak flake.lock
sudo nixos-rebuild switch --flake .#lab
```

Exercises

Exercise 1 — Snapshot current pins

```
cd /path/to/your-flake
nix flake metadata
cp flake.lock /tmp/flake.lock.bak
git status
```

Journal:

1. List every input name and its locked `rev` (short hash is fine)
2. Confirm `flake.lock` is tracked by git
3. Note whether any input uses `follows`

Exercise 2 — Read the lock structure

```
nix flake metadata
nix flake metadata --json | jq '.locks.nodes | keys' 2>/dev/null || true
```

Open `flake.lock` and identify:

- The root node's inputs
- The `nixpkgs` node's `rev` and `narHash`
- Any nested `nixpkgs_2` style duplicates (candidate for `follows`)

Exercise 3 — Selective update of one input

Only if you accept rebuild churn. On a disposable lab host:

```
nix flake update nixpkgs
git diff --stat flake.lock
git diff flake.lock | head -120
```

Build before switch:

```
nixos-rebuild build --flake .#lab
sudo nixos-rebuild switch --flake .#lab
```

Smoke checklist:

```
systemctl is-system-running || true
systemctl is-active sshd
nix --version
```

If break: restore backup lock and re-switch.

Exercise 4 — Prove “update all” is louder

On a throwaway branch:

```
git checkout -b experiment/flake-update-all
nix flake update
git diff --stat flake.lock
```

Count how many inputs / revs moved. Then restore:

```
git checkout main # or your real branch name
git branch -D experiment/flake-update-all
```

Write one sentence: *why selective update is the default for hosts*.

Exercise 5 — Document house rules

Create docs/lock-policy.md:

```
# Lock policy (lab)

- Track nixpkgs nixos-26.05
- Commit flake.lock always
- Update one input per change window
- Smoke: rebuild + SSH + one app path
- Revert lock on failure before deep debug
- Align HM release-* with nixos-*
```

Exercise 6 — Registry awareness (read-only)

```
nix registry list
nix flake metadata
```

Note if the registry's nixpkgs rev differs from the flake's. No need to change registries.

Exercise 7 — follows fix (if needed)

If you have (or add) a second input with its own nixpkgs:

```
inputs.foo.inputs.nixpkgs.follows = "nixpkgs";

nix flake lock
nix flake metadata --json | jq '.locks.nodes | keys'
```

Exercise 8 — Compare URL construction

Write the GitHub compare URL for your last nixpkgs bump (from Exercise 3) or a synthetic old→new pair from metadata history. Bookmark the pattern.

Exercise 9 — Impurity awareness list

List three things that can still differ with the same lock (substituters, arch, impure builtins). One sentence each.

Exercise 10 — CI anti-pattern note

In docs/lock-policy.md, add: “CI must not run `nix flake update` on every job; lock bumps are deliberate PRs.”

Exercise 11 — Paired restore drill

```
# simulate bad partial restore understanding
git show HEAD:flake.lock >/dev/null
# document: always restore flake.nix + flake.lock together when input URLs change
```

Exercise 12 — Multi-input update plan

Write the order you will use when adding Home Manager (next chapters): lock HM → follows → rebuild → only then update nixpkgs again.

Common gotchas

Symptom / mistake	What to do
Everyone’s host differs after clone follows not set → two nixpkgs	Missing or uncommitted <code>flake.lock</code> Add <code>inputs.foo.inputs.nixpkgs.follows = "nixpkgs"</code>
Updated lock, forgot rebuild CI uses <code>nix flake update</code> every run	<code>switch</code> / <code>boot</code> to activate Pin lock in repo; update in deliberate PRs
“I’m on 26.05” but lock is months old	Expected until you update; schedule updates
Restored <code>flake.nix</code> but not lock	Always restore the pair when debugging inputs
Private input auth fails after update	Token/SSH access issue, not lock format
HM release nixpkgs release	Align <code>release-26.05</code> with <code>nixos-26.05</code>
Reviewed only <code>flake.nix</code> in PR Massive lock + feature PR	Always review lock diff too Split: lock bump PR vs feature PR

Checkpoint

- ☐ Can explain what `rev + narHash` buy you
- ☐ Can update **one** named input and read the lock diff
- ☐ Have a written house lock policy
- ☐ Know how to revert `flake.lock` quickly
- ☐ Understand registry `flake pin`
- ☐ `follows` planned or applied for companion inputs
- ☐ Know why full `nix flake update` is loud on hosts
- ☐ Commit policy treats lock as first-class

Journal (optional)

Write three personal gotchas before continuing—especially any surprise nested `nixpkgs` or registry mismatch.

Home Manager on NixOS

Home Manager on NixOS

Goal: Wire **Home Manager as a NixOS module** (not standalone-first), manage one user's shell/git/ssh client config, and rebuild with a single `nixos-rebuild switch --flake` activation story.

i Note

House default for this book: Home Manager **NixOS module** on NixOS hosts. Standalone HM is for non-NixOS machines or special cases—know it exists; do not make it your lab's primary path.

Why this chapter exists

Host modules put packages on the **system** (`environment.systemPackages`). That is correct for daemons and multi-user tools—and wrong as the only place for **personal** developer config. Home Manager (HM) owns:

- User programs (`programs.git`, `programs.bash` / `zsh` / `fish`, ...)
- Dotfiles and XDG (next chapter)
- User systemd services (HM services chapter)

Doing HM as a NixOS module means: **one flake, one rebuild, one generation story** for system + user.

Theory 1 — Three ways people run Home Manager

Mode	How it activates	When to use
NixOS module	Part of <code>nixos-rebuild</code>	Default on NixOS (this book)
Standalone	<code>home-manager switch --flake ...</code>	Other distros; or isolated user experiments
nix-darwin module	macOS analog	Out of scope here

Why module mode first

- Aligns HM with system generations (easier mental model for labs)
- `users.users.<name>` and `home-manager.users.<name>` stay in one repo
- Avoids “system switched but HM forgotten” dual-track ops

Standalone still matters if you develop on Ubuntu with HM-only—but your **NixOS lab** should not require two tools for every change.

Theory 2 — Input wiring (26.05-aligned)

```
# flake.nix (merge with your layout)
{
  description = "NixOS lab with Home Manager";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

    home-manager.url = "github:nix-community/home-manager/release-26.05";
    home-manager.inputs.nixpkgs.follows = "nixpkgs";
  };

  outputs = { self, nixpkgs, home-manager, ... }@inputs: {
    nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      specialArgs = { inherit inputs; };
      modules = [
        ./hosts/lab
        home-manager.nixosModules.home-manager
      ]
      {
        home-manager.useGlobalPkgs = true;
        home-manager.useUserPackages = true;
        # home-manager.extraSpecialArgs = { inherit inputs; };
        home-manager.users.alice = import ./homes/alice;
      };
    };
  };
}
```

Option	Meaning
<code>useGlobalPkgs = true</code>	HM uses the same <code>pkgs</code> as the system (consistent overlays later)
<code>useUserPackages = true</code>	User packages install into user profile via HM
<code>home-manager.users.alice = ...</code>	That user's HM module tree
<code>extraSpecialArgs</code>	Pass flake inputs into HM modules when needed

Version match: HM release-26.05 with nixpkgs nixos-26.05. Mismatched releases are a common source of obscure option errors.

Where to put the HM block

Style	Pros
Inline in <code>flake.nix</code> as above	Obvious for one host
Dedicated module <code>modules/common/home-manager.nix</code>	Cleaner multi-host
Per-host in <code>hosts/lab/default.nix</code>	Different users per machine

All are valid; pick one and document it.

Theory 3 — System user vs HM user

You still need a **NixOS user**:

```
# modules/common/users.nix
{
  users.users.alice = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];
    # shell can be set here OR via HM programs.* - pick one source of truth
  };
}
```

HM configures that user's home:

```
# homes/alice/default.nix
{ pkgs, ... }:
{
  home.username = "alice";
  home.homeDirectory = "/home/alice";
}
```

```

home.stateVersion = "26.05"; # set once; do not casually bump

programs.home-manager.enable = true;

programs.git = {
  enable = true;
  userName = "Alice Example";
  userEmail = "alice@example.invalid";
};

programs.bash = {
  enable = true;
  enableCompletion = true;
};

home.packages = with pkgs; [
  ripgrep
  fd
];
}

```

home.stateVersion

Like NixOS `system.stateVersion`, this is a **compatibility pin** for HM stateful defaults—not “current release marketing.” Set it to the HM release you first adopt for that user; read release notes before bumping.

Name and path must match

Field	Must agree with
<code>home-manager.users.alice</code>	attr name
<code>home.username</code>	"alice"
<code>home.homeDirectory</code>	actual home, usually <code>/home/alice</code>
<code>users.users.alice</code>	system user exists

Mismatches produce activation failures or configs landing in the wrong place.

Theory 4 — What belongs in HM vs NixOS

Concern	Prefer
sshd, firewall, kernel, system users	NixOS modules
Interactive shell, prompt, git, editor config	HM
CLI tools only you use	HM <code>home.packages</code>
Libraries/services shared by system units	NixOS
Secrets for user apps	Secrets chapters (not plaintext in HM)
Network daemons (nginx, postgres)	NixOS
Project compilers	Project <code>devShells</code> + <code>direnv</code> (later chapter)

Anti-pattern: dumping everything into `environment.systemPackages` because HM felt optional.
Opposite anti-pattern: putting `nginx` only in HM.

Theory 5 — Activation model (what rebuild does)

On `nixos-rebuild switch` with HM module:

1. System config builds
2. HM config for each defined user builds
3. Activation scripts run: system switch + HM activation for users

Failures can be **system** or **HM**. Read the traceback: `home-manager-alice.service` style units often appear in logs.

```
journalctl -u home-manager-alice.service -b --no-pager | tail -80
```

(Username substitution as appropriate.)

Standalone commands (awareness)

Even in module mode you may see:

```
home-manager generations 2>/dev/null || true
ls -la ~/.local/state/nix/profiles 2>/dev/null || true
```

You should **not** need `home-manager switch` for normal changes if module mode is correct.

Theory 6 — Programs modules vs raw packages

HM `programs.*` modules often:

- Install the package
- Generate config files
- Wire shell integration

Example: `programs.git.enable = true` vs only `home.packages = [ pkgs.git ]` (no managed config). Prefer **programs modules** when they exist for tools you configure.

Discover options:

- [Home Manager option search](#)
- Or `eval/docs` from your locked HM input

Shell source of truth

Approach	Note
<code>users.users.alice.shell = pkgs.bashInteractive</code>	System login shell
<code>programs.bash.enable = true</code>	HM manages bash config
Both fighting	Pick login shell once; let HM own rc files

Theory 7 — Splitting the home tree

```
homes/alice/
  default.nix      # imports + identity (username, stateVersion)
  programs.nix     # git, bash, ssh client
  files.nix        # later: xdg / home.file
  services.nix     # later: user systemd
```

```
# homes/alice/default.nix
{
  imports = [
    ./programs.nix
    # ./files.nix
    # ./services.nix
  ];

  home.username = "alice";
  home.homeDirectory = "/home/alice";
```

```

home.stateVersion = "26.05";
programs.home-manager.enable = true;
}

```

Theory 8 — Backup and collision with existing dots

HM activation may refuse to overwrite an existing regular file with a symlink. Before first activation of a path:

```

ls -la ~/.bashrc ~/.config/git 2>/dev/null || true
# backup and move aside if needed

```

Situation	Approach
Imperative bashrc	Backup; let HM own it
Mixed partial HM	Finish migration; avoid dual management
One-off experiment file	Keep outside HM paths

Worked example — Minimal useful user

```

# homes/alice/default.nix
{ pkgs, lib, ... }:
{
  home.username = "alice";
  home.homeDirectory = "/home/alice";
  home.stateVersion = "26.05";

  programs.home-manager.enable = true;

  programs.git = {
    enable = true;
    userName = "Alice";
    userEmail = "alice@example.invalid";
    extraConfig = {
      init.defaultBranch = "main";
      pull.rebase = true;
    };
  };
};

```



```

programs.ssh = {
  enable = true;
  # Do not put private keys in the flake. Config only:
  matchBlocks = {
    "lab-remote" = {
      hostname = "192.0.2.10"; # documentation range example
      user = "alice";
    };
  };
};

programs.bash = {
  enable = true;
  shellAliases = {
    ll = "ls -la";
    ".." = "cd ..";
  };
};

home.packages = with pkgs; [ htop ];
}

```

Private keys stay in `~/.ssh/` with correct permissions—**never** in git, never as world-readable store strings (secrets chapters).

Worked example — HM wiring module (multi-host friendly)

```

# modules/common/home-manager.nix
{ inputs, ... }:
{
  imports = [
    inputs.home-manager.nixosModules.home-manager
  ];

  home-manager = {
    useGlobalPkgs = true;
    useUserPackages = true;
    users.alice = import ../../homes/alice;
  };
}

```

Requires `specialArgs = { inherit inputs; };` on `nixosSystem` (or equivalent) so `inputs` is available.

Exercises

Exercise 1 — Add the input and lock

```
cd /path/to/your-flake
# edit flake.nix inputs + outputs as in Theory 2
nix flake lock
# or: nix flake update home-manager
git diff flake.lock | head -80
```

Confirm follows for nixpkgs.

Exercise 2 — Stub → real home module

```
mkdir -p homes/alice
```

Write `homes/alice/default.nix` with `git` + `bash` + one package. Wire HM NixOS module and `home-manager.users.<you>`.

Ensure `users.users.<you>` exists on the system side with matching name and home directory.

Exercise 3 — Rebuild and verify

```
sudo nixos-rebuild switch --flake .#lab
```

As the target user:

```
git config --global --list | head
which rg || which htop
echo "$HOME"
home-manager generations 2>/dev/null || true
```

Exercise 4 — Change only HM, rebuild once

Edit `programs.git.userName`, rebuild, confirm new value. You should **not** need a separate `home-manager switch` if module mode is correct.

Exercise 5 — Deliberate failure drill

```
programs.git.enabl = true; # intentional typo
```

Rebuild, read the error, fix. Note whether the failure is clearly HM evaluation.

Exercise 6 — Journal: boundary decisions

List 10 tools you use daily. Mark each **NixOS** / **HM** / **either** / **project shell**. Resolve at least the ambiguous ones with a one-line reason.

Exercise 7 — Activation log literacy

```
journalctl -u home-manager-$USER.service -b --no-pager | tail -100
```

Save one successful and (from Exercise 5) one failed snippet in notes (redact secrets).

Exercise 8 — Split programs.nix

Move `git/bash/ssh` into `homes/alice/programs.nix` and import it. Rebuild; behavior unchanged.

Exercise 9 — Package placement check

Add a tool to `home.packages`, rebuild, verify `command -v`. Confirm it is **not** required for other users of the host.

Exercise 10 — Shell source of truth

Document in `docs/layout.md` (or home notes): where login shell is set, where aliases live.

Exercise 11 — Backup existing dots

Before enabling a `programs.*` that manages a file you already have, backup:

```
cp -a ~/.gitconfig /tmp/gitconfig.bak 2>/dev/null || true
```

Exercise 12 — Version alignment proof

```
nix flake metadata | tee /tmp/flake-meta.txt
```

Confirm HM input tracks `release-26.05` and `nixpkgs nixos-26.05`.

Exercise 13 — Negative case: standalone not required

Write two sentences: when you would use standalone HM; why the lab does not.

Common gotchas

Symptom / mistake	What to do
HM release nixpkgs release	Align <code>release-26.05</code> with <code>nixos-26.05</code>
User name mismatch	<code>users.users.x</code> must match <code>home-manager.users.x</code>
<code>home.homeDirectory</code> wrong	Must match actual home path
Infinite recursion with <code>useGlobalPkgs</code>	Check for pkgs/HM cycles; simplify modules
Expecting standalone CLI only	Module mode activates via <code>nixos-rebuild</code>
Private keys in flake	Remove immediately; rotate if committed
Bumping <code>home.stateVersion</code> casually	Read HM release notes first
Shell set in three places	One source of truth (NixOS <i>or</i> HM for login shell)
Activation blocked by existing file	Backup/remove blocker
Forgot <code>follows</code>	Two <code>nixpkgs</code> ; fix and re-lock
HM user without system user	Create <code>users.users.*</code> first
Putting <code>sshd</code> in HM	System service → NixOS module

Checkpoint

- ☐ HM input pinned with `follows nixpkgs`
- ☐ HM NixOS module imported in flake
- ☐ At least one user has git + shell via HM
- ☐ Single `nixos-rebuild switch --flake` activates system + HM

- ☐ No private key material in the repo
- ☐ `home.stateVersion` set consciously
- ☐ Boundary list (NixOS vs HM vs project) started
- ☐ Know where to read HM activation failures

Journal (optional)

Write three personal gotchas before continuing.

Home files and XDG

Home files and XDG

Goal: Manage real user files with Home Manager—`home.file`, `xdg.configFile`, and XDG directories—while understanding **store-backed** links, permissions, and what must **never** be checked into the flake as plaintext secrets.

Why this chapter exists

The Home Manager chapter enabled programs modules. This chapter is the **file layer**: most of a personal environment is text under `~/.config`, `~/.local`, and a few classic dotfiles. HM makes those files **declarative and rebuildable**—or, if misused, makes secrets **world-readable in the Nix store**.

Theory 1 — How HM places files

Typical mechanism:

1. File contents (or a path) enter the Nix store as part of the HM generation
2. Activation creates **symlinks** from the home directory into the store (or manages mutable copies depending on options)
3. Next activation may replace links when content changes

Implications:

Property	Consequence
Store paths are world-readable	No passwords, tokens, private keys in HM file content
Links point at immutable content	Editing the live file in <code>~</code> may be wrong layer—edit the flake
Rebuild required for changes	Dotfile tweak → rebuild (or temporary escape hatch)

Escape hatches (know, use sparingly)

- Manual files outside HM for truly mutable scratch
- `mkOutOfStoreSymlink` patterns (advanced; can reintroduce impurity)
- Secrets tooling (services/security part) for sensitive values

```
# advanced awareness only - impure path outside store
# home.file."somewhere".source =
#   config.lib.file.mkOutOfStoreSymlink "/home/alice/mutable/config";
```

Prefer store-backed management for anything you want rebuildable and reviewable in git.

Theory 2 — home.file vs xdg.configFile

API	Default target idea	Example use
home.file.<Path>	Relative to \$HOME	home.file.".npmrc"
xdg.configFile.<Path>	Under XDG config home (usually ~/.config)	xdg.configFile."foot/foot.ini"
xdg.dataFile	Under data home	Desktop entries, shared data
xdg.mimeApps	MIME defaults	Default apps

Prefer `xdg.configFile` for modern tools that honor XDG.

Minimal examples

```
# homes/alice/files.nix
{ pkgs, ... }:
{
  xdg.enable = true;

  xdg.configFile."demo/hello.txt".text = ''
    managed by home-manager
    rebuilt with the host flake
  '';

  home.file.".config/demo/from-home-file.txt".text = ''
    same idea via home.file
  '';
}
```

Import from `homes/alice/default.nix`:

```
{
  imports = [ ./files.nix ];
}
```


Theory 3 — Content sources: text, source, templates

Field	Use
<code>.text</code>	Inline small configs
<code>.source</code>	Path to a file in the repo (<code>./config/gitconfig</code> sibling)
<code>.source = pkgs.writeText ...</code>	Generated in Nix
<code>.executable = true</code>	Scripts that need <code>+x</code>

Repo layout option:

```
homes/alice/  
  default.nix  
  files.nix  
  config/  
    starship.toml  
    htoprc  
    nvim/init.lua
```

```
xdg.configFile."starship.toml".source = ./config/starship.toml;  
xdg.configFile."nvim/init.lua".source = ./config/nvim/init.lua;
```

Benefit of `source`: normal editor diffs on real files; Nix only wires them.

Recursive directories

```
xdg.configFile."nvim" = {  
  source = ./config/nvim;  
  recursive = true;  
};
```

Use when a tool expects a tree. Keep secrets out of that tree.

Theory 4 — XDG directories

```
xdg.enable = true;  
# xdg.configHome / dataHome / cacheHome / stateHome can be customized  
# defaults are usually correct - only override with a reason
```

Variable	Typical path
XDG_CONFIG_HOME	~/.config
XDG_DATA_HOME	~/.local/share
XDG_CACHE_HOME	~/.cache
XDG_STATE_HOME	~/.local/state

Tools differ in XDG discipline. Your job: **put managed files where the tool reads them**, not where tradition says.

Classic vs XDG

Classic path	Prefer if tool supports XDG
~/.gitconfig	programs.git (HM) or XDG git config
~/.bashrc	programs.bash
~/.config/foo	xdg.configFile."foo/..."

Theory 5 — Permissions and clobber behavior

HM will not always happily overwrite unexpected pre-existing files—activation may fail if a real file blocks a symlink.

Situation	Approach
Old imperative dotfile in the way	Backup and remove, then rebuild
You need a secret file mode 0600	Do not put secret content in store; use runtime secret install
Public config	Store-backed symlink is fine

Check links:

```
ls -la ~/.config/demo
readlink -f ~/.config/demo/hello.txt
stat "$(readlink -f ~/.config/demo/hello.txt)"
```

Mode options

```
home.file.".local/bin/lab-tool" = {
  text = ''
    #!/usr/bin/env bash
    echo lab
  '';
  executable = true;
};
```

Remember: store object readability still applies to content; `executable` does not make content private.

Theory 6 — Theming (optional, bounded)

Theming (GTK, fonts, pointer cursors) is an **infinite rabbit hole**. If you care:

- Prefer HM modules that exist (gtk, qt, fonts at system level, etc.)
- Bound the experiment: one terminal theme + one font—not a full desktop redesign

If you run a server-only lab: **skip theming**; migrate shell/git configs instead.

Theory 7 — Multi-machine user splits

Same user, different hosts:

```
# homes/alice/default.nix
{
  imports = [
    ./common.nix
    # ./hosts/lab.nix    # optional per-host HM slice
  ];
}
```

Or select in the flake:

```
home-manager.users.alice = {
  imports = [
    ./homes/alice/common.nix
    ./homes/alice/hosts/lab.nix
  ];
}
```

```
};
```

Do not copy entire homes per host if 90% is shared—**common** + **overlay modules**.

Theory 8 — `programs.*` vs raw files for the same tool

Prefer	When
<code>programs.git</code> <code>xdg.configFile</code>	Structured options exist Tool has no HM module or you want raw file fidelity
Both for same path	Conflict risk—pick one

```
# Good: structured
programs.git.ignores = [ "result" ".direnv/" ];

# Also good: raw when complex
# xdg.configFile."git/ignore".source = ./config/git-ignore-global;
```

Theory 9 — What never goes in managed files

Never in flake/HM text	Why
API tokens	Store world-readable
Private SSH keys	Same + git leak
Cloud provider secrets	Same
DB passwords	Same
<code>.env</code> production values	History forever if committed

Gray area: non-secret feature flags, public keys, preferred editor settings—OK.

Worked example — Migrate git ignores + editor config

```
# homes/alice/git.nix
{
  programs.git = {
    enable = true;
    userName = "Alice";
    userEmail = "alice@example.invalid";
    ignores = [
      "*.env"
      ".direnv/"
      "result"
      "result-*"
    ];
  };

  xdg.configFile."nvim/init.lua".text = ''
    -- minimal managed nvim config
    vim.opt.number = true
    vim.opt.relativenumber = true
  '';
}
```

No API tokens. No extraConfig with github.token.

Worked example — Structured home tree

```
homes/alice/
  default.nix
  programs.nix
  files.nix
  config/
    starship.toml
    htop/htoprc

# homes/alice/files.nix
{
  xdg.enable = true;
  xdg.configFile."starship.toml".source = ./config/starship.toml;
  xdg.configFile."htop/htoprc".source = ./config/htop/htoprc;
}
```

Exercises

Exercise 1 — Inventory live dotfiles

As your lab user:

```
ls -la ~ | head -50
ls -la ~/.config 2>/dev/null | head -50
```

Pick **3–5** real configs to migrate (git, shell rc fragments, one tool). Skip secrets and browser profiles.

Journal: path → HM attribute choice (programs.* vs xdg.configFile vs home.file).

Exercise 2 — First managed XDG file

Add `xdg.enable = true` and a demo `xdg.configFile`. Rebuild. Verify symlink into store:

```
ls -la ~/.config/demo
readlink -f ~/.config/demo/hello.txt
stat "$(readlink -f ~/.config/demo/hello.txt)"
```

Note permissions on the store path (world-readable). **That observation is the point.**

Exercise 3 — Migrate via source

Place a real config file next to the HM module and set `.source`. Rebuild. Confirm tool behavior.

Exercise 4 — Conflict drill

While HM manages a path: backup, replace symlink with a regular file, rebuild, observe activation error. Restore by removing the blocker and rebuilding. Document the error text.

Exercise 5 — Secret anti-pattern (fake only)

On a local throwaway branch (do not push real secrets):

```
# BAD - educational only, use fake value
# home.file.".config/acme/token".text = "super-secret-placeholder";
```

If you activate a fake value, show yourself:

```
# cat "$(readlink -f ~/.config/acme/token)"
```

Then **delete** the pattern, rebuild, and write: “secrets need secrets tooling, not HM file content.”
Never commit real tokens. If you did: rotate them.

Exercise 6 — Structure the home tree

```
homes/alice/  
  default.nix  
  programs.nix  
  files.nix  
  config/
```

Rebuild once more to prove the split.

Exercise 7 — Recursive config tree

If you use nvim or similar, try `recursive = true` on a config directory. Confirm links.

Exercise 8 — Executable home file

Install a tiny script under `~/.local/bin` via `home.file + executable = true`. Ensure `~/.local/bin` is on PATH (HM `home.sessionPath` or shell config).

Exercise 9 — programs vs file for one tool

For git or another tool, write two sentences: why you chose `programs.*` vs raw file.

Exercise 10 — Multi-host plan

Sketch `common.nix` + optional `hosts/lab.nix` even if you only have one machine.

Exercise 11 — Cache vs config

List one path that should stay **unmanaged** (cache, browser profile). Confirm it is not in the flake.

Exercise 12 — Activation after migrate

```
sudo nixos-rebuild switch --flake .#lab
journalctl -u home-manager-$USER.service -b --no-pager | tail -40
```

Exercise 13 — Documentation

Add a short “Home files” section to `docs/layout.md`: which configs are managed, which are intentionally not.

Common gotchas

Symptom / mistake	What to do
Activation fails on existing file	Backup/remove blocker; rebuild
Edited ~ file, change vanished	You edited a symlink or lost it on rebuild—edit flake
Secret in store	Remove from Nix; rotate credential
Wrong path (~/.foo vs XDG)	Check tool docs; fix attribute
Huge binary blobs in HM	Don't; keep media out of the flake
text with nested quotes hell	Prefer .source files
Assuming 0600 store file is private	Store is not a private vault
Dual management of same path	Pick programs.* or file API
Recursive source includes .git junk	Clean source tree
Theming rabbit hole on server	Skip; ship shell/git first

Checkpoint

- ☐ `xdg.enable` (or conscious decision not to)
- ☐ At least two real configs managed via HM files or programs
- ☐ Can show a managed path is store-backed
- ☐ Home tree split into programs/files modules
- ☐ Written note: no secrets in HM file content
- ☐ Conflict + recovery once
- ☐ At least one .source migration
- ☐ Know when to use escape hatches vs store links

Journal (optional)

Write three personal gotchas before continuing.

Home Manager user services

Home Manager user services

Goal: Run **user-level** systemd services and timers through Home Manager—work that belongs to a login user, not to the system-wide daemon set.

Why this chapter exists

System systemd via NixOS covers host daemons. Not everything should be system-wide:

Better as user service	Better as system service
Personal backup of ~/docs	PostgreSQL, Caddy, sshd
Wait-for-graphical clipboard tool	Network-facing APIs
User-specific sync agent	Multi-user shared daemons
Periodic <code>git fetch</code> of notes	CI runners for a team

HM exposes a structured way to declare user units so they survive reinstalls and rebuilds.

Theory 1 — Systemd user vs system

Bus	Unit location idea	Starts when
System	/etc/systemd/system (generated)	Boot / system targets
User	~/.config/systemd/user (generated by HM)	User session / lingering

Lingering

If you need user services **without** an interactive login (headless user timers):

```
sudo loginctl enable-linger alice
loginctl show-user alice | grep Linger
```

Without linger, user services may only run while logged in (depending on setup). For a server lab admin user, lingering is often desirable; for a laptop, maybe not—**document the choice**.

Theory 2 — HM service options

Home Manager provides:

- `systemd.user.services.<name>`
- `systemd.user.timers.<name>`
- `services.<hm-module>` for popular user apps (where modules exist)

Raw unit example:

```
# homes/alice/services.nix
{ pkgs, ... }:
{
  systemd.user.services.lab-heartbeat = {
    Unit = {
      Description = "Write a heartbeat timestamp";
    };
    Service = {
      Type = "oneshot";
      ExecStart = "${pkgs.bash}/bin/bash -c 'date -Is >> %h/lab-heartbeat.log'";
    };
  };

  systemd.user.timers.lab-heartbeat = {
    Unit = {
      Description = "Run lab-heartbeat every 15 minutes";
    };
    Timer = {
      OnBootSec = "2min";
      OnUnitActiveSec = "15min";
      Persistent = true;
    };
    Install = {
      WantedBy = [ "timers.target" ];
    };
  };
}
```

`%h` is `systemd` specifier for the user home directory.

Import from `homes/alice/default.nix`.

HM attrset shape

HM's user unit options often mirror unit file sections as nested attrsets (`Unit`, `Service`, `Install`). Exact schema can vary slightly by HM version—when in doubt, check HM option docs for your **26.05-aligned** pin.

Theory 3 — Mapping system knowledge to user bus

Concept	System (NixOS)	User (HM)
Service attrset	<code>systemd.services.*</code>	<code>systemd.user.services.*</code>
Timer	<code>systemd.timers.*</code>	<code>systemd.user.timers.*</code>
journal	<code>journalctl -u foo</code>	<code>journalctl --user -u foo</code>
restart	<code>systemctl restart</code>	<code>systemctl --user restart</code>
list timers	<code>systemctl list-timers</code>	<code>systemctl --user list-timers</code>

Permissions: user services **cannot** bind privileged ports (<1024) without capabilities setup; they should not replace system network daemons.

Theory 4 — Simple vs oneshot vs long-running

Type	Use
<code>oneshot</code>	Task that exits; good with timers
<code>simple</code> / default	Long-running process
<code>exec</code>	systemd newer behavior; fine when you know it

For a periodic script: **oneshot service + timer**.

For a personal agent: long-running service with `Restart = "on-failure"`.

```
systemd.user.services.notes-agent = {
  Unit.Description = "Long-running personal agent";
  Service = {
    ExecStart = "${pkgs.coreutils}/bin/sleep infinity"; # placeholder
    Restart = "on-failure";
    RestartSec = "5s";
  };
  Install.WantedBy = [ "default.target" ];
};
```

(Replace the placeholder with a real agent binary from `pkgs`.)

Theory 5 — Paths and packages in units

Always use **store paths** from **pkgs**, not assumed global binaries:

```
ExecStart = "${pkgs.restic}/bin/restic snapshots";  
# not: ExecStart = "restic snapshots";
```

Environment:

```
Service = {  
  Environment = [  
    "FOO=bar"  
  ];  
  # EnvironmentFile must not point at secrets in the flake/store  
};
```

Sensitive env files belong to secret deployment paths with tight permissions—not **home.file** with tokens.

WorkingDirectory and paths

```
Service = {  
  WorkingDirectory = "%h/notes";  
  ExecStart = "${pkgs.git}/bin/git fetch --all --prune";  
};
```

Ensure the directory exists (create once imperatively or with a oneshot).

Theory 6 — Prefer HM modules when they exist

Examples (availability depends on HM version):

- **services.syncthing** (user)
- **services.gpg-agent**
- Various desktop helpers

Pattern: search HM options first; fall back to raw **systemd.user.services** for custom scripts.

```
services.gpg-agent = {  
  enable = true;  
  enableSshSupport = true;  
};
```

Theory 7 — Targets, wants, and session lifecycle

Target	Role
default.target	User session default
timers.target	Timers
graphical-session.target	Needs graphical session

```
Install.WantedBy = [ "default.target" ];  
# or timers.target for timers
```

Graphical-only tools should not use headless linger assumptions without testing.

Theory 8 — Debugging workflow

```
systemctl --user daemon-reload # usually activation handles this  
systemctl --user status lab-heartbeat.service  
systemctl --user start lab-heartbeat.service  
journalctl --user -u lab-heartbeat.service -n 50 --no-pager  
systemctl --user list-timers --all
```

Break → read journal → fix flake → rebuild → retest. Avoid editing generated unit files under `~/.config/systemd/user` as source of truth.

Worked example — Notes fetcher timer

```
{ pkgs, ... }:  
{  
  home.packages = [ pkgs.git ];  
  
  systemd.user.services.fetch-notes = {  
    Unit.Description = "Fetch personal notes repo";  
    Service = {  
      Type = "oneshot";  
      WorkingDirectory = "%h/notes";  
      ExecStart = "${pkgs.git}/bin/git fetch --all --prune";  
    };  
  };  
};
```

```
systemd.user.timers.fetch-notes = {
    Unit.Description = "Timer for fetch-notes";
    Timer = {
        OnCalendar = "hourly";
        Persistent = true;
    };
    Install.WantedBy = [ "timers.target" ];
};
}
```

Requires ~/notes to be a git repo. Create it if missing (empty repo is fine).

Worked example — Calendar timer variants

```
Timer = {
    OnCalendar = "daily";
    # OnCalendar = "Mon..Fri *--* 09:00:00";
    Persistent = true;
    RandomizedDelaySec = "10m";
};
```

Persistent = true catches up missed runs after downtime (within reason).

Exercises

Exercise 1 — User systemd baseline

```
systemctl --user status
systemd-analyze --user blame 2>/dev/null | head || true
loginctl show-user "$USER" | grep -E 'Linger|State'
```

Decide whether to enable linger for your lab user; document the choice.

Exercise 2 — Heartbeat oneshot + timer

Add the heartbeat service from Theory 2. Rebuild:

```
sudo nixos-rebuild switch --flake .#lab
```

As user:

```
systemctl --user list-timers --all | grep -i heart || systemctl --user list-timers --all
systemctl --user start lab-heartbeat.service
systemctl --user status lab-heartbeat.service
cat ~/lab-heartbeat.log
systemctl --user enable --now lab-heartbeat.timer
```

Exercise 3 — Journal debugging

```
journalctl --user -u lab-heartbeat.service -n 50 --no-pager
```

Break ExecStart deliberately, rebuild, read the failure, fix.

Exercise 4 — One service you actually want

Ideas:

- `git fetch` on a notes repo
- `rsync` dry-run of a directory to a backup path (no secrets in unit)
- Generate a dated file in `~/lab/`

Implement, enable, prove timer or manual start works twice.

Exercise 5 — Boundary check

Write four lines:

1. Why this is not a system service
2. What user it runs as
3. Whether `linger` is required
4. What happens on logout without `linger`

Exercise 6 — Import `services.nix`

Ensure `homes/alice/services.nix` is imported from `default.nix`. Rebuild once after the split.

Exercise 7 — list-unit-files literacy

```
systemctl --user list-unit-files | head -40
systemctl --user cat lab-heartbeat.service
```

Confirm the unit content matches your flake (store paths visible).

Exercise 8 — Failure restart policy

Add a long-running service with `Restart = "on-failure"` (even a simple script). Kill it, observe restart behavior, then disable if you do not need it.

Exercise 9 — OnCalendar experiment

Change heartbeat to `OnCalendar = "*/0/10"` (every 10 minutes) or similar; reload; `list-timers`.

Exercise 10 — Wrong bus drill

Run `systemctl status lab-heartbeat` **without** `--user`. Note the confusion. Always use `--user` for HM units.

Exercise 11 — Secret audit of unit text

Read your service modules: no passwords in `Environment=`. Plan secrets paths if needed later.

Exercise 12 — Graphical vs headless note

If this host is headless, list any user services that would need a desktop session—and do not enable them.

Exercise 13 — Comparison table in docs

Add to `docs/layout.md`: one system service vs one user service you run, with rationale.

Exercise 14 — Clean disable

Disable and remove a throwaway timer after testing:

```
systemctl --user disable --now lab-heartbeat.timer 2>/dev/null || true
# remove from flake; rebuild
```

Common gotchas

Symptom / mistake	What to do
Timer never fires while logged out	Enable linger or accept session-bound life
<code>systemctl</code> without <code>--user</code>	You queried the system bus
Binary not found	Use <code>\${pkgs.foo}/bin/foo</code>
Service works manually, not on timer	Check <code>WantedBy</code> , enable timer, <code>list-timers</code>
Putting <code>nginx</code> in user <code>systemd</code>	Use NixOS system module instead
Secrets in <code>Environment=</code> in flake	Secrets chapters; remove from git
Activation OK but unit missing	Confirm HM user matches logged-in user
Edited generated unit by hand	Revert; change flake only
<code>WorkingDirectory</code> missing	Create dir or unit fails
Port 80 in user service	Use <code>system.proxy/service</code>
Linger enabled globally without thought	Document per-user choice

Checkpoint

- ☐ At least one user service defined via HM
- ☐ At least one timer or a long-running user service
- ☐ Can use `systemctl --user` and `journalctl --user`
- ☐ Linger decision documented
- ☐ No secrets in unit text
- ☐ Clear system-vs-user boundary write-up
- ☐ Units imported from `services.nix` (or equivalent split)
- ☐ Know how to read a failed user unit journal

Journal (optional)

Write three personal gotchas before continuing.

direnv and project shells

direnv and project shells

Goal: Keep project toolchains in **flake devShells**, load them automatically with **direnv** (use **flake**), and stop stuffing every language toolchain into system or HM profiles.

Why this chapter exists

Two failure modes dominate Nix workstations:

1. **Profile bloat:** Go, Node, Rust, Python, JDK all in `home.packages` forever
2. **Snowball shells:** random `nix-shell -p` with no lock, unreproducible on Monday

The modern fix:

Layer	Holds
NixOS / HM	OS, editors, generic CLI, user services
Project flake devShells	Compilers, linters, project CLIs
direnv	Enters/exits the shell when you <code>cd</code>

Project pinning is a first-class skill—independent of host configuration.

Theory 1 — devShells in a flake

Minimal project flake (can be **separate** from the NixOS flake):

```
# ~/lab/demo-app/flake.nix
{
  description = "demo app toolchain";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
  let
    system = "x86_64-linux";
    pkgs = nixpkgs.legacyPackages.${system};
  in {
```

```

devShells.${system}.default = pkgs.mkShell {
  packages = with pkgs; [
    go_1_24 # adjust to versions available on your pin
    gopls
    git
  ];
  shellHook = ''
    export DEMO_APP=1
    echo "entered demo-app devShell"
  '';
};
};
}

```

Enter manually:

```

cd ~/lab/demo-app
nix develop

```

Multi-system sketch

```

let
  systems = [ "x86_64-linux" "aarch64-linux" ];
  forAllSystems = nixpkgs.lib.genAttrs systems;
in {
  devShells = forAllSystems (system:
    let pkgs = nixpkgs.legacyPackages.${system}; in {
      default = pkgs.mkShell {
        packages = with pkgs; [ git jq ];
      };
    });
}

```

Why not always put shells in the NixOS flake?

You *can* expose devShells from the host flake, but **project repos** usually own their own flake so clones on other machines work without your entire OS config. Host flake remains for nixosConfigurations (+ checks). Host-level shells for ops tools (nil, nixpkgs-fmt) are still fine.

Theory 2 — direnv + nix-direnv

direnv loads environment variables when you enter a directory with `.envrc`.

nix-direnv (recommended) caches Nix flake shells efficiently so every `cd` is not a full re-eval disaster.

HM installation pattern

```
# homes/alice/direnv.nix
{ pkgs, ... }:
{
  programs.direnv = {
    enable = true;
    nix-direnv.enable = true;
    enableBashIntegration = true;
    # enableZshIntegration / enableFishIntegration as needed
  };
}
```

Rebuild host/HM, then new shells pick up the hook (`eval "$(direnv hook bash)"` is usually wired by the module).

Verify hook

```
command -v direnv
type direnv
# open a new login shell if needed
```

Theory 3 — .envrc with use flake

In the project:

```
# ~/lab/demo-app/.envrc
use flake
```

Allow once:

```
direnv allow
```

Behavior:

Action	Result
cd into project	Loads <code>devShell</code> env
cd out	Unloads
flake/lock change	direnv reloads (may rebuild)

Named shells

```
# use flake .#ci
use flake
```

If you define `devShells.${system}.ci`, you can select it—check current nix-direnv / direnv-stdlib docs for exact syntax on your pin.

`.envrc` is not for secrets

Do not put API tokens in `.envrc` committed to git. Use:

- Untracked `.envrc.local` patterns (with care)
- Secret managers / sops (services/security part)
- Runtime private files excluded by git

```
.envrc.local
.env
```

Theory 4 — System packages vs project shells

Put in HM/system	Put in devShell
git, neovim, htop	go, cargo, node, python venv tools
direnv itself	project linters pinned to repo
SSH client	DB client only for one service's repo

Test: if two projects need different major versions, it belongs in the project shell.

Theory 5 — Pinning discipline for project flakes

Project `flake.lock` should be committed for apps you ship or share. For private throwaways, still locking saves future you.

Update project `nixpkgs` **independently** of the host flake—on purpose. Host OS pin every app pin.

Host flake	Project flake
Stability for OS modules	May need newer compiler
Selective update cadence	Can track unstable carefully
Shared by all services	Local to one repo

Theory 6 — `nix develop` vs `nix shell` vs HM packages

Command	Role
<code>nix develop</code>	Flake devShell (hooks, packages, inputsFrom)
<code>nix shell nixpkgs#foo</code>	Ad-hoc package on PATH
<code>home.packages</code>	Always-on user profile
<code>direnv use flake</code>	Automates develop-class env

`inputsFrom` (awareness)

```
pkgs.mkShell {  
  inputsFrom = [ self.packages.${system}.default ];  
  packages = [ pkgs.jq ];  
};
```

Pulls build inputs from a package—useful when packaging and developing the same project.

Theory 7 — `shellHook` hygiene

Do	Don't
Export needed <code>PROJECT_ROOT</code>	Print banners that annoy CI
Set <code>LANG</code> if required	Embed secrets
Keep hooks idempotent	Assume interactive-only blindly

```
shellHook = ''
  export PROJECT_ROOT="$PWD"
  # echo only if interactive:
  # [ -n "$PS1" ] && echo "devShell ready"
'';
```

Theory 8 — Host flake devShell for Nix ops

Optional host-level shell (not a substitute for project shells):

```
# in host flake outputs
devShells.${system}.default = pkgs.mkShell {
  packages = with pkgs; [ nixpkgs-fmt nil git jq ];
};
```

```
cd /path/to/nixos-config
nix develop
```

Use for editing the OS flake; keep language toolchains in app repos.

Worked example — Multi-tool Python shell

```
devShells.${system}.default = pkgs.mkShell {
  packages = with pkgs; [
    python3
    python3Packages.pytest
    ruff
  ];
  shellHook = ''
    export PYTHONPATH="$PWD/src" '${PYTHONPATH:+:$PYTHONPATH}'
  '';
};

# .envrc
use flake
```

Worked example — Isolation proof layout

```
~/lab/proj-a/    # flake with cowsay only
~/lab/proj-b/    # flake with hello only

cd ~/lab/proj-a && command -v cowsay
cd ~/lab/proj-b && command -v hello
cd ~ && command -v cowsay; command -v hello # both missing if thinned
```

Exercises

Exercise 1 — Install direnv via HM

Add `programs.direnv` with `nix-direnv.enable = true`. Rebuild. Verify:

```
command -v direnv
direnv version
```

Open a new login shell if hooks are missing.

Exercise 2 — Create a project flake

```
mkdir -p ~/lab/demo-app
cd ~/lab/demo-app
git init
# write flake.nix with mkShell and 2-3 tools
nix flake lock
nix develop -c which git
```

Exercise 3 — Wire direnv

```
echo 'use flake' > .envrc
direnv allow
cd ..
cd demo-app
echo "PATH head:"; echo "$PATH" | tr ':' '\n' | head
type python3 2>/dev/null || type go 2>/dev/null || type hello 2>/dev/null || true
```

Confirm leaving the directory drops the env.

Exercise 4 — Prove version isolation

Make **two** tiny projects with different tools (e.g. one with `hello` only, one with `cowsay` only). `cd` between them; show different `PATH` entries.

Exercise 5 — Thin the profile (careful)

List `home.packages` / system packages. Move **one** language toolchain from always-on profile into a project shell. Rebuild HM. Confirm the tool is gone from bare shells but present under `direnv`.

Do not remove your editor without a plan.

Exercise 6 — Document the workflow

Journal template:

```
cd project → direnv loads flake lock pin → tools available
change flake → direnv reload → maybe download/build
secrets → never in committed .envrc
```

Exercise 7 — Commit project lock

```
cd ~/lab/demo-app
git add flake.nix flake.lock .envrc
git status
# ensure .env / secrets not staged
```

Exercise 8 — Reload after lock bump

```
nix flake update nixpkgs
# leave and re-enter directory or: direnv reload
```

Exercise 9 — Quiet shellHook

If your hook is noisy, gate prints on interactive shells. Re-enter and confirm.

Exercise 10 — Host ops shell

Add a minimal `devShells` to the `host` flake with `nixpkgs-fmt` or `nil`. `nix develop` from the host repo.

Exercise 11 — Ad-hoc vs project

```
nix shell nixpkgs#jq -c jq --version
```

Write one sentence: when ad-hoc is OK vs when a project flake is required.

Exercise 12 — gitignore hygiene

Ensure `.direnv/` is ignored in the project (and ideally global gitignore).

```
.direnv/  
result  
result-*  
.env  
.envrc.local
```

Exercise 13 — Failure: allow missing

```
# new clone simulation  
rm -f .envrc # careful  
echo 'use flake' > .envrc  
# without allow, direnv blocks - observe message  
direnv allow
```

Exercise 14 — Boundary table

Fill a 6-row table: tool → HM / system / project shell. Commit to notes.

Common gotchas

Symptom / mistake	What to do
direnv not hooking	New shell; enable integration for your shell
use flake blocked	<code>direnv allow</code>
Slow every <code>cd</code>	Ensure <code>nix-direnv</code> enabled
Wrong system arch in flake	Match <code>system</code> to the machine
Secrets in <code>.envrc</code> committed	Rotate; use gitignored local files / sops

Symptom / mistake	What to do
Host flake-only shells	Fine for ops; projects should still travel
<code>shellHook</code> prints annoyingly	Keep hooks quiet in shared repos
Forgotten <code>flake.lock</code> in project	Commit it
Tool still in <code>home.packages</code>	Thinned incompletely
<code>.direnv</code> not ignored	Add to gitignore
Expecting macOS path on Linux CI	Pin systems explicitly

Checkpoint

- ☐ `direnv` + `nix-direnv` via HM
- ☐ At least one project with `devShells.default` + lock
- ☐ `.envrc` with `use flake` allowed
- ☐ Enter/leave directory toggles tools
- ☐ One toolchain removed from always-on profile (or plan written)
- ☐ Secret policy for env files stated
- ☐ Project lock committed
- ☐ Understand host ops shell vs project shell

Journal (optional)

Write three personal gotchas before continuing.

flake-parts (optional structure)

flake-parts (optional structure)

Goal: Choose a **house style** for multi-output flakes—either adopt **flake-parts** to reduce boilerplate, or enforce a **hand-rolled** structure with equal consistency—and apply it to your host flake.

i Note

There is no moral victory in either choice. There is only **consistency**. Mixing half flake-parts and half ad-hoc `outputs = args: ...` is the failure mode.

Why this chapter exists

By now your flake may export:

- `nixosConfigurations.*`
- `devShells.*` (host-level tooling)
- Soon: `checks.*`, maybe `packages.*`, `formatter`

Raw `outputs` functions grow nested `let` blocks and copy-pasted `system` lists. Two responses exist in the ecosystem:

Style	Idea
flake-parts	Framework: modules for flakes; per-system helper; community modules
Hand-rolled discipline	Small local helpers (<code>forAllSystems</code>), strict file layout, no new dependency

You **pick one** and document it.

Theory 1 — Pain that motivates structure

Typical messy outputs:

```
outputs = { self, nixpkgs, home-manager, ... }@inputs:
let
```



```

    system = "x86_64-linux";
    pkgs = import nixpkgs { inherit system; };
in {
    nixosConfigurations.lab = /* ... */;
    devShells.${system}.default = /* ... */;
    # forgot aarch64; forgot checks; formatter missing
};

```

Problems:

- Single-system assumptions
- Repeated pkgs construction
- Hard to share patterns across repos
- Easy to forget an output class

Theory 2 — Hand-rolled discipline (no flake-parts)

Pattern used widely:

```

{
    inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    # home-manager ...

    outputs = { self, nixpkgs, ... }@inputs:
    let
        systems = [ "x86_64-linux" "aarch64-linux" ];
        forAllSystems = nixpkgs.lib.genAttrs systems;
        pkgsFor = system: import nixpkgs {
            inherit system;
            # overlays = [ self.overlays.default ];
        };
    in {
        nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
            system = "x86_64-linux";
            specialArgs = { inherit inputs; };
            modules = [ ./hosts/lab /* hm module wiring */ ];
        };

        devShells = forAllSystems (system:
            let pkgs = pkgsFor system; in {
                default = pkgs.mkShell {
                    packages = with pkgs; [ nixpkgs-fmt nil ];
                };
            });
    };
}

```

```

    formatter = forAllSystems (system: (pkgsFor system).nixpkgs-fmt);
  };
}

```

House rules for hand-rolled

1. One `systems` list
2. One `pkgsFor` / overlay injection point
3. Host modules stay under `hosts/`; flake stays wiring
4. New output types get a comment in `docs/flake-style.md`
5. No deep logic in `flake.nix`—call into `./lib` or `./flake/` if needed

Theory 3 — flake-parts approach

`flake-parts` turns outputs into a **module system** for flakes.

```

{
  description = "lab with flake-parts";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    flake-parts.url = "github:hercules-ci/flake-parts";
    home-manager.url = "github:nix-community/home-manager/release-26.05";
    home-manager.inputs.nixpkgs.follows = "nixpkgs";
  };

  outputs = inputs@{ flake-parts, ... }:
    flake-parts.lib.mkFlake { inherit inputs; } {
      systems = [ "x86_64-linux" "aarch64-linux" ];

      perSystem = { pkgs, system, ... }: {
        devShells.default = pkgs.mkShell {
          packages = with pkgs; [ nixpkgs-fmt nil ];
        };
        formatter = pkgs.nixpkgs-fmt;
      };

      flake = {
        nixosConfigurations.lab = inputs.nixpkgs.lib.nixosSystem {
          system = "x86_64-linux";
          specialArgs = { inherit inputs; };
          modules = [

```

```

        ./hosts/lab
    inputs.home-manager.nixosModules.home-manager
    {
        home-manager.useGlobalPkgs = true;
        home-manager.useUserPackages = true;
        home-manager.users.alice = import ./homes/alice;
    }
];
};
};
}

```

Concept	Role
<code>perSystem</code>	Outputs that vary by system (<code>packages</code> , <code>devShells</code> , <code>checks</code> , <code>formatter</code>)
<code>flake = { ... }</code>	Top-level flake attrs like <code>nixosConfigurations</code>
Modules	You can split flake-parts config across files

Costs

- Another input to pin and learn
- Stack traces include framework layers
- Overkill for a single `nixosConfigurations` only—but multi-output flakes benefit

Split flake-parts modules (optional)

```

# flake.nix
outputs = inputs@{ flake-parts, ... }:
  flake-parts.lib.mkFlake { inherit inputs; } {
    imports = [
      ./flake-modules/devshell.nix
      ./flake-modules/hosts.nix
    ];
    systems = [ "x86_64-linux" ];
  };

```

Theory 4 — Decision rubric

Choose flake-parts if...	Choose hand-rolled if...
You expect many per-system outputs	You want minimal dependencies
You like module-shaped config everywhere	You already understand raw flakes well
You will copy patterns across many repos	One lab flake is enough for months
You accept framework upgrades	You prefer total local control

Book stance: either is valid. What matters is that you **have a style** and stick to it.

Theory 5 — What not to do

- Rewrite layout (hosts/modules/homes) and adopt flake-parts in the same panicked hour without rebuild
- Import flake-parts and still paste a second full **outputs** tree beside it
- Put NixOS module logic inside **perSystem** incorrectly (hosts are usually under **flake.nixosConfigurations**)
- Enable every community flake-parts module on day one
- Change house style every weekend

Theory 6 — Formatter as a style signal

Regardless of framework, expose a formatter:

```
nix fmt
```

With **formatter** output set, this becomes a team habit before checks enforce it.

Formatter	Notes
nixpkgs-fmt	Common, stable
alejandra	Opinionated alternative
nixfmt	Ecosystem evolving—pin deliberately

Pick one; put it in **formatter** and in the devShell.

Theory 7 — Overlay injection point

Whichever style you pick, define **one** place overlays are applied to `pkgs`:

```
pkgsFor = system: import nixpkgs {
  inherit system;
  overlays = [ (import ./overlays) ];
};
```

flake-parts: configure `perSystem` `pkgs` via its `nixpkgs` module patterns, or construct consistently—document how.

Worked example — Split hand-rolled helpers

```
# lib/flake-utils.nix
{ nixpkgs }:
rec {
  systems = [ "x86_64-linux" "aarch64-linux" ];
  forAllSystems = nixpkgs.lib.genAttrs systems;
  pkgsFor = system: import nixpkgs { inherit system; };
}

# flake.nix uses:
# let u = import ./lib/flake-utils.nix { inherit nixpkgs; }; in ...
```

Keeps `flake.nix` readable without `flake-parts`.

Worked example — Decision record template

```
# Flake style

Decision: hand-rolled | flake-parts
Date: YYYY-MM-DD
Rationale: ...

Rules:
- systems = ...
- overlays applied in ...
- hosts defined under ...
```

- how to add a new `nixosConfiguration`
 - how to add a `devShell`
-

Exercises

Exercise 1 — Audit current outputs

```
cd /path/to/your-flake
nix flake show
```

Journal:

1. Which outputs exist today?
2. Which will exist after checks/overlays chapters?
3. Is `flake.nix` already painful?

Exercise 2 — Pick and write the decision

Create `docs/flake-style.md` with the template above. Commit this even before code changes—forces intentionality.

Exercise 3A — If flake-parts: adopt on a branch

```
git checkout -b experiment/flake-parts
# add input, rewrite outputs via mkFlake
nix flake lock
nix flake show
nixos-rebuild build --flake .#lab
sudo nixos-rebuild switch --flake .#lab # if build OK
```

Prove HM + host still activate.

Exercise 3B — If hand-rolled: extract helpers

```
git checkout -b experiment/flake-discipline
# add systems + forAllSystems + formatter + devShells
nix flake show
nix fmt 2>/dev/null || true
nix develop -c true
nixos-rebuild build --flake .#lab
```

Exercise 4 — Merge the winner

Merge the experiment branch to your main lab branch. Delete half-finished alternatives. One style in `flake.nix`.

Exercise 5 — Teach the style to “future you”

In `docs/flake-style.md`, document how to:

1. Add a new `nixosConfigurations` host
2. Add a per-system `devShell`
3. Update `flake-parts` (if used) or `helpers` (if not)

Exercise 6 — Formatter end-to-end

```
nix fmt
git diff
```

Format should be deterministic. Fix any files the formatter rewrites.

Exercise 7 — `specialArgs` survival

After refactor, confirm modules that need `inputs` still receive them (HM, future sops).

Exercise 8 — Show vs build

```
nix flake show
nixos-rebuild build --flake .#lab -L
```

If `show` looks empty-ish, `build` often has the real trace.

Exercise 9 — `follows` still holds

```
nix flake metadata
```

Confirm companion `inputs` still `follows` `nixpkgs` after the rewrite.

Exercise 10 — Negative hybrid cleanup

Search for leftover dual patterns (`outputs` = dead code, unused `flake-parts` import). Remove.

Exercise 11 — Systems honesty

If you only ever build `x86_64-linux`, either keep one system or document why two systems exist without CI.

Exercise 12 — Style PR description

Write a short paragraph as if reviewing your own style adoption: risks, rollback, smoke tests.

Common gotchas

Symptom / mistake	What to do
<code>perSystem</code> vs <code>flake</code> confusion	Hosts go under <code>flake</code> ; shells under <code>perSystem</code>
Lost <code>specialArgs</code> / inputs	Thread <code>inputs</code> explicitly after rewrite
HM module dropped in refactor	Re-check modules list post-migration
Double <code>nixpkgs</code> without follows	Keep follows on all companion inputs
<code>nix flake show</code> empty-ish	Eval error—run build to see full trace
Framework without docs	Write house docs the same day
Layout + framework same commit	Split; prove rebuild between
Formatter wars in one PR	Pick one tool; migrate separately
Community modules kitchen-sink	Add one optional module at a time
Infinite recursion after pkgs hook	Simplify pkgs construction

Checkpoint

- ☐ Explicit house style documented
- ☐ `nix flake show` reflects multi-output intent
- ☐ Host rebuild still works
- ☐ `formatter` and/or `devShells` exist in the chosen style
- ☐ No hybrid mess left on main branch
- ☐ How-to for adding a host written
- ☐ `specialArgs`/inputs still correct for HM
- ☐ follows discipline intact

Journal (optional)

Write three personal gotchas before continuing.

Flake checks

Flake checks

Goal: Add flake **checks** so `nix flake check` exercises eval/format/build assumptions locally—the same shape CI will use later.

Why this chapter exists

A flake that only **switches** on your laptop is a private ritual. Checks turn “I hope this still evaluates” into a **command**:

```
nix flake check
```

This chapter is not full **nixosTest** mastery (ops/testing chapters). It is **lightweight gates**: format, build a shell, evaluate a host, maybe build a package.

Theory 1 — What checks are

Flake schema: `checks.<system>.<name>` → derivations. `nix flake check` builds them (with some special cases).

Check type	Purpose
Format / lint	Style consistency
<code>nixosSystem</code> eval/build	Config still evaluates
<code>mkShell</code> build	DevShell inputs resolve
Small package build	Packaging sanity
<code>nixosTest</code>	VM integration (later parts)

Failed check → non-zero exit → CI red.

Theory 2 — Hand-rolled checks skeleton

```
# inside outputs, with forAllSystems / pkgsFor from the flake-parts chapter
checks = forAllSystems (system:
  let
    pkgs = pkgsFor system;
  in {
    # 1) formatter check
    format = pkgs.runCommand "check-format" {
      nativeBuildInputs = [ pkgs.nixpkgs-fmt ];
      src = pkgs.lib.cleanSource ./.;
    } ''
      set -euo pipefail
      cp -r $src tree
      chmod -R u+w tree
      cd tree
      find . -name '*.nix' -print0 | xargs -0 nixpkgs-fmt --check
      mkdir -p $out
      touch $out/ok
    '';

    # 2) devShell builds
    devshell = self.devShells.${system}.default;
  });
```

Building the full host `toplevel` is an excellent check but can be **slow** and architecture-bound. For a laptop-only `x86_64-linux` lab, pinning checks to that system is fine.

Host toplevel (strong, heavy)

```
// nixpkgs.lib.optionalAttrs (system == "x86_64-linux") {
  lab-toplevel = self.nixosConfigurations.lab.config.system.build.toplevel;
}
```

Gate by system so aarch64 eval of an x86-only host is not nonsense.

Lighter host check — eval-oriented

Some teams use CI scripts (`nixos-rebuild build`) without putting `toplevel` in `checks`. Either works if **documented**.

Theory 3 — flake-parts checks

Under `perSystem`:

```
perSystem = { self', pkgs, ... }: {
  checks = {
    devshell = self'.devShells.default;
    # treefmt / git-hooks modules exist in the ecosystem-optional
  };
};
```

Community modules (`treefmt-nix`, `git-hooks.nix`) are optional accelerators—do not require them if time is short.

Theory 4 — Format check that actually fails

A useful format check must **see your sources** and use the formatter’s **check** mode.

Notes:

- `--check` must exist for your formatter CLI (`nixpkgs-fmt` supports it; other formatters differ)
- Copying entire flake including `.git` can be heavy—trim with `lib.cleanSource`
- IFD and huge contexts slow checks—keep them lean

```
src = pkgs.lib.cleanSource ./.;
```

Alternative: `treefmt` (awareness)

`treefmt` + `treefmt-nix` can unify formatters. Optional complexity—only adopt if you want multi-language format in one gate.

Theory 5 — What belongs in check vs push-time CI only

Local nix flake check	CI later
Format	Same + matrix systems
Shell / small builds	Cache push
Optional host toplevel	Full VM tests
Fast enough to run pre-commit	Longer integration

Keep local checks **fast enough that you run them**. A 40-minute check nobody runs is theater.

Budget (guideline)	Contents
< 1–2 min warm	format + shell
< 10–15 min	+ host toplevel on one system
CI nightly	tests, multi-system, slow builds

Theory 6 — nix flake check UX

```
nix flake check
nix flake check -L          # logs
nix build .#checks.x86_64-linux.format -L
```

Individual check attrs help debug without running the full set.

Common failure modes

Failure	Meaning
format	Style drift
devshell	Missing package / bad mkShell
lab-toplevel	Host module error
Eval error before build	checks attrset itself broken

Theory 7 — Checks and secrets

Checks often copy sources into the builder. **Do not** place plaintext secrets in the flake tree. Encrypted sops files are ciphertext (OK); private keys and `.env` must stay out.

```
.env
*.pem
keys.txt
```

Theory 8 — Wiring consistency with pkgs/overlays

If overlays affect `hello-lab` or `formatters`, `pkgsFor` in `checks` must match `host/devShell` construction—or document intentional differences.

```
pkgsFor = system: import nixpkgs {  
  inherit system;  
  overlays = import ./overlays;  
};
```

Worked example — Practical minimal set

```
checks = forAllSystems (system:  
  let pkgs = pkgsFor system; in {  
    shell = self.devShells.${system}.default;  
  } // nixpkgs.lib.optionalAttrs (system == "x86_64-linux") {  
    # lab-toplevel = self.nixosConfigurations.lab.config.system.build.toplevel;  
  });
```

Uncomment `toplevel` when you accept the build cost.

Worked example — Script wrapper

```
# tools/check.sh  
#!/usr/bin/env bash  
set -euo pipefail  
nix flake check -L  
  
chmod +x tools/check.sh  
./tools/check.sh
```

Exercises

Exercise 1 — Baseline without checks

```
cd /path/to/your-flake
nix flake show
nix flake check || true
```

Note current behavior (may pass with zero checks or warn).

Exercise 2 — Add devShell as a check

Wire `checks.<system>.shell = self.devShells.${system}.default` (or `flake-parts` equivalent).

```
nix build .#checks.x86_64-linux.shell -L
```

Exercise 3 — Add a format check that can fail

1. Implement format check on `*.nix`
2. Run `nix flake check`
3. Break formatting in a temp edit
4. Confirm check fails
5. Restore format

Exercise 4 — Optional host toplevel check

If disk/time allow:

```
nix build .#checks.x86_64-linux.lab-toplevel -L
```

Record wall-clock time. If over your patience budget, document “CI-only toplevel” and keep `shell+format` local.

Exercise 5 — Developer loop script (optional)

Add `tools/check.sh` or a Makefile target. Not required if you prefer raw commands.

Exercise 6 — Journal: CI shape

Write the future CI job in five lines:


```
checkout
install nix
nix flake check
# later: deploy job separate
```

Exercise 7 — Single-attr debug

```
nix build .#checks.x86_64-linux.format -L
```

Practice isolating one check.

Exercise 8 — cleanSource impact

If format check is slow or includes junk, switch to `lib.cleanSource` and remeasure.

Exercise 9 — System gate

If you list multiple systems, ensure host toplevel is only on the host architecture.

Exercise 10 — Pre-commit habit

Run `nix flake check` before a real commit. Note anything surprising (downloads, rebuilds).

Exercise 11 — Document check inventory

In `docs/flake-style.md` or `docs/checks.md`, list each check and what failure means.

Exercise 12 — Secret scan of check context

Confirm no private key paths are required for checks to pass.

Exercise 13 — Fail-closed demo

Leave a format break on a throwaway branch; show `nix flake check` non-zero; fix on mainline branch only.

Exercise 14 — Overlay consistency

If you have a wrapper package from overlays, add a tiny check that builds it—or document why not.

Common gotchas

Symptom / mistake	What to do
Checks empty / not discovered	Must be under checks output correctly
IFD surprises slow checks	Avoid unnecessary import-from-derivation
Building host on wrong system	optionalAttrs gate
Format check always succeeds	You didn't pass real sources
Check context includes secrets	Don't; keep secrets out of flake source
nix flake check downloads the world	First run cold; then cached—note substituters
Top-level too slow → abandoned	Split local vs CI checks
flake-parts checks in wrong section	Use perSystem.checks
Formatter mismatch vs nix fmt	Same package in formatter + format check
Broken self reference	Ensure self.devShells exists before check

Checkpoint

- ☐ At least two meaningful checks (e.g. shell + format)
- ☐ **nix flake check** fails when format intentionally broken
- ☐ Know how to build one check attribute
- ☐ Host toplevel check added **or** explicitly deferred with reason
- ☐ Future CI sketch written
- ☐ Checks are fast enough to run before commit
- ☐ Documented inventory of checks
- ☐ No secrets required to evaluate checks

Journal (optional)

Write three personal gotchas before continuing.

Overlays (light introduction)

Overlays (light introduction)

Goal: Apply a **small overlay** so the same package change is visible to the NixOS host and to devShells—without forking nixpkgs or cargo-culting overlay stacks.

! Important

Overlays are sharp tools. Prefer **options**, **package overrides in a single module**, or **upstream pins** when they suffice. This chapter teaches the mechanism so you are not helpless—not so every problem becomes an overlay.

Why this chapter exists

Sooner or later you need:

- A newer/older minor of one tool than nixpkgs 26.05 ships
- A one-line patch to a package you use
- A renamed or wrapped package for your lab

Overlays let you modify the **pkgs** fixed-point. Done lightly, they are elegant. Done heavily, they make upgrades impossible.

Large third-party overlays (for example [nix-ros-overlay](#) for ROS 2) use the same mechanism at ecosystem scale: you import *their* overlay and usually **nixpkgs.follows** their pin so the world stays coherent.

Theory 1 — What an overlay is

An overlay is a function:

```
final: prev: {  
  # new or overridden attrs  
}
```

Argument	Meaning
<code>prev</code>	Package set before this overlay

Argument	Meaning
<code>final</code>	Package set after all overlays (fixed-point)—use for deps on other overridden pkgs

```
final: prev: {
  hello = prev.hello.overrideAttrs (old: {
    # example: add a patch or change configure flags
  });
}
```

Fixed-point intuition

```
pkgs = fix (self: vanilla // overlay1 self vanilla // overlay2 self ...)
```

You rarely write `fix` yourself; Nixpkgs applies overlays for you.

Theory 2 — Wiring overlays on NixOS

```
# modules/common/nixpkgs.nix
{ ... }:
{
  nixpkgs.overlays = [
    (import ../../overlays/hello-lab.nix)
    # or inline:
    # (final: prev: { ... })
  ];
}
```

With `home-manager.useGlobalPkgs = true`, HM sees the same overlaid `pkgs`. That is why the Home Manager chapter set that flag.

Flake-level pkgs for shells/checks

```
pkgsFor = system: import nixpkgs {
  inherit system;
  overlays = [
    (import ../overlays/hello-lab.nix)
  ];
}
```

```
};
```

Consistency rule: the overlay list used for `nixosConfigurations` should match the list used for `devShells` / checks when you care about the same tools—or document intentional differences.

Theory 3 — `overrideAttrs` vs `override` vs new package

Tool	When
<code>pkg.override { ... }</code>	Change <code>callPackage</code> parameters
<code>pkg.overrideAttrs</code>	Change derivation attributes (patches, version, src)
New attr <code>final.mytool = ...</code>	Add a package name without clobbering

Example patch (illustrative):

```
# overlays/hello.nix
final: prev: {
  hello = prev.hello.overrideAttrs (old: {
    patches = (old.patches or []) ++ [
      # ./patches/hello-example.patch
    ];
  });
}
```

If you do not have a real patch, use a **harmless visible change**—or prefer adding a **wrapper** package instead of mutating `hello`:

```
final: prev: {
  hello-lab = prev.writeShellScriptBin "hello-lab" ''
    echo "lab overlay wrapper"
    exec ${prev.hello}/bin/hello "$@"
  '';
}
```

Wrappers are often safer than overriding core packages.

Theory 4 — Composition order

```
nixpkgs.overlays = [ overlayA overlayB ];
```

Later overlays see earlier results via **prev** chain. Conflicts: last writer wins for the same attr. Keep **one file per concern** and a short list.

```
# overlays/default.nix
[
  (import ./hello-lab.nix)
  # (import ./mypackage.nix)
]
```

```
nixpkgs.overlays = import ../../overlays/default.nix;
```

Theory 5 — When not to overlay

Need	Prefer
Enable a service option	NixOS module option
Package only for one project	Project <code>devShell</code> / <code>packages</code>
Temporary try	<code>nix shell</code> / branch pin
Huge fork of desktop stack	Separate <code>nixpkgs</code> input (advanced)
One CLI flag	Wrapper script in module

Overlays that rewrite half of `python3Packages` are packaging-part territory—and often a mistake for hosts.

Theory 6 — Pinning a package from another `nixpkgs` (light)

Sometimes you want one package from a newer pin without moving the whole OS:

```
# advanced sketch - use sparingly
# inputs.nixpkgs-unstable.url = "github:NixOS/nixpkgs/nixos-unstable";
# final: prev: {
#   fancy = inputs.nixpkgs-unstable.legacyPackages.${prev.system}.fancy;
# }
```


Costs: two package graphs, closure size, ABI surprises. In this chapter, a **wrapper** or small `overrideAttrs` is enough.

Theory 7 — Overlays vs `packageOverrides` (classic)

Older NixOS configs used `nixpkgs.config.packageOverrides`. Prefer `nixpkgs.overlays` in modern flakes. If you find both, consolidate.

Theory 8 — Infinite recursion patterns

Anti-pattern	Why
<code>final.foo = final.foo.override...</code> carelessly	Wrong fixed-point use
Overlay imports a module that imports pkgs that need the overlay	Cycles
Using <code>final</code> where <code>prev</code> suffices	Accidental recursion

Rule of thumb: take base package from `prev`; reach for `final` only when depending on another **new** attr you defined in the same overlay set.

```
final: prev: {
  myLib = prev.callPackage ./mylib { };
  myApp = prev.callPackage ./myapp { inherit (final) myLib; };
}
```

Worked example — Shared overlay module

```
overlays/
  default.nix      # composes list
  hello-lab.nix
modules/common/nixpkgs.nix

# overlays/default.nix
[
  (import ./hello-lab.nix)
]
```

```
# overlays/hello-lab.nix
final: prev: {
  hello-lab = prev.writeShellScriptBin "hello-lab" ''
    echo "hello from lab overlay"
    exec ${prev.hello}/bin/hello "$@"
  '';
}

# modules/common/nixpkgs.nix
{ ... }:
{
  nixpkgs.overlays = import ../../overlays/default.nix;
}
```

In a module that receives `pkgs`:

```
{ pkgs, ... }:
{
  environment.systemPackages = [ pkgs.hello-lab ];
}
```

And in flake `pkgsFor`, import the same overlay list for shells/checks.

Worked example — HM sees the overlay

```
# homes/alice/programs.nix fragment
{ pkgs, ... }:
{
  home.packages = [ pkgs.hello-lab ];
}
```

Requires `home-manager.useGlobalPkgs = true`.

Exercises

Exercise 1 — Create overlay file

```
mkdir -p overlays
# add hello-lab.nix and default.nix as above
```

Exercise 2 — Attach to NixOS

Import overlay list via `nixpkgs.overlays`. Add `pkgs.hello-lab` to system or HM packages. Rebuild:

```
sudo nixos-rebuild switch --flake .#lab
hello-lab
```

Exercise 3 — Attach to devShell / checks pkgs

Ensure `pkgsFor` (or `flake-parts pkgs`) uses the same overlays.

```
nix develop -c hello-lab
```

If only system has it, fix the flake `pkgs` construction.

Exercise 4 — Prove HM sees it (if useGlobalPkgs)

```
home.packages = [ pkgs.hello-lab ];
```

Rebuild; as user command `-v hello-lab`.

Exercise 5 — Intentional override discipline

Journal:

1. What you overlaid/wrapped and why
2. How you will remove it when upstream catches up
3. Whether a project-only shell would have been enough

Exercise 6 — Negative space

Find one change you *could* do with an overlay but will do with a module option instead. Write it down—builds judgment.

Exercise 7 — Composition list

Add a second tiny wrapper (`cowsay-lab` or similar). Confirm both appear. Practice ordering in `overlays/default.nix`.

Exercise 8 — Failure: overlay not wired

Temporarily remove overlay from `pkgsFor` only; observe shell missing package while system has it. Restore consistency.

Exercise 9 — Document overlays

Add `docs/overlays.md`: list each overlay, purpose, removal criteria.

Exercise 10 — Check integration

If you have flake checks, add a check that builds `pkgs.hello-lab` or run it in the shell check path.

Exercise 11 — Prefer wrapper over override

If you mutated `hello`, convert to `hello-lab` wrapper instead. Rebuild.

Exercise 12 — Search before overlay

For one real need, search `nixpkgs` options / HM options first. Only overlay if no option exists.

Exercise 13 — Recursion caution note

Write two sentences in `docs/overlays.md` about `final` vs `prev`.

Exercise 14 — Upgrade readiness

When `nixpkgs` updates, re-run `hello-lab` and note if still needed.

Symptom / mistake	What to do
-------------------	------------

Common gotchas

Symptom / mistake	What to do
<code>pkgs.hello-lab</code> missing	Overlay not wired into that <code>pkgs</code> import
HM doesn't see overlay	<code>useGlobalPkgs</code> false or separate <code>pkgs</code>
Infinite recursion in overlay	Using <code>final</code> incorrectly; simplify
Overlay only on system	Wire flake <code>pkgsFor</code> too
Overlaying all of <code>nixpkgs</code> for fun	Revert; minimize surface
Patch path wrong	Path relative to overlay file / flake root
Two competing overlay lists	One source of truth
Clobbering upstream attr accidentally	Prefer new attr names
Project-only need in host overlay	Move to project flake
Forgot removal plan	Document in <code>docs/overlays.md</code>

Checkpoint

- ☐ Overlay or wrapper package defined in repo
- ☐ Visible on NixOS host packages
- ☐ Visible in `devShell` **or** documented intentional difference
- ☐ HM consistency verified if applicable
- ☐ Removal plan written
- ☐ Judgment note: when not to overlay
- ☐ Single overlay composition list
- ☐ Docs list active overlays

Journal (optional)

Write three personal gotchas before continuing.

Part close-out

You now have:

Capability	Chapter
Scalable repo layout	Flake repository layout
Deliberate pins	Lock file discipline
User plane via HM module	Home Manager + files + services
Project toolchains	direnv + devShells
Multi-output style	flake-parts or hand-rolled
Quality gates	Flake checks
Light package tweaks	Overlays

Next part: **Services and security patterns**—secrets, hardening, firewall, TLS, data services, containers, observability—on top of this shape.

Services and Security

Services and security patterns

Services and security patterns

This part of the **standalone NixOS book** is organized by **topic** (normal book chapters), not by calendar day.

Read chapters in order within the part when learning; skip or skim freely when using the book as a reference.

Baseline: NixOS / nixpkgs **26.05**, flakes, Home Manager as NixOS module (from the previous part), deliberate firewall and secrets discipline.

What this part is for

Layout and Home Manager made the repo operable. This part makes a **service slice** operable **and** defensible:

Theme	Chapters
Secrets threat model	The secrets problem
Primary secrets tooling	sops-nix
Comparison alternative	agenix
Host attack surface	Host hardening patterns
Network policy	Firewall discipline
HTTPS front door	TLS and reverse-proxy
Durable state	A data service pattern (PostgreSQL)
Isolation literacy	NixOS containers
OCI industry path	Podman and Docker patterns
See what is running	Observability patterns
Optional k8s awareness	k3s (optional)

After this part you should be able to run a small app path—proxy → backend → database—with encrypted secrets, documented ports, and a monitoring or logs pattern—without secrets in the Nix store.

Prerequisites

- Working flake host with `hosts/` · `modules/` · `homes/` layout (or equivalent discipline)
- Lock policy and selective updates
- Comfort with `systemd` units and `journalctl`
- Firewall basics from the host networking chapter
- No reliance on plaintext passwords in the flake

Chapter map

Chapter	Focus	Outcome
The secrets problem	Store vault	Audit + remove anti-patterns
sops-nix	Encrypt in git; path at runtime	End-to-end secret consumer
agenix (comparison)	Same job, different UX	House default declared
Host hardening	SSH, sudo, surface	Hardening module + recovery notes
Firewall discipline	Deny by default	<code>docs/ports.md</code> living matrix
TLS front door	Caddy or Nginx + ACME/lab TLS	443 front; localhost backends
Data service	PostgreSQL pattern	State, auth, backup story
NixOS containers	<code>nspawn containers.*</code>	Isolation decision record
Podman/Docker	Declarative OCI	Localhost publish + comparison
Observability	Metrics and/or logs	Runbook or Prometheus path
k3s optional	Skip or lite install	Explicit decision artifact

Mental model — service slice

`clients`

`:443 reverse proxy (TLS)` `modules/services/proxy-*.nix`

`127.0.0.1:app` `backend unit / OCI / nspawn`

`127.0.0.1:5432` `PostgreSQL (localhost)`

secrets via sops-nix	/run/secrets/... paths only
firewall	22, 80, 443 (documented)
observability	journald and/or Prometheus localhost

Layer	Default posture
Edge	TLS terminate; minimal open ports
App	Bind localhost; no public DB
Secrets	Ciphertext in git; decrypt at activation
State	Mutable dirs outside store; backup plan
Observe	Localhost metrics or excellent logs

Suggested module layout (extends previous part)

```
modules/
  common/
    hardening/
    firewall.nix
    sops.nix
  services/
    proxy-caddy.nix  # or proxy-nginx.nix
    postgres.nix
    observability.nix
    podman.nix
    oci-hello.nix
  secrets/
    lab.yaml          # sops-encrypted
    .sops.yaml
  docs/
    secrets.md
    ports.md
    proxy.md
    data-service.md
    isolation.md
    observability.md
```

How to work this part

1. Threat model and cleanup before tooling

2. **sops-nix end-to-end** before real service passwords
3. **agenix once** for comparison; pick a house default
4. **Hardening + firewall** before public exposure
5. **Proxy + one data service** as the spine
6. **Containers literacy** without containerizing everything
7. **Observability** so the slice is inspectable
8. **k3s** only if justified—or skip with a written decision

Prefer small commits: secrets infra, then hardening, then each service.

Cross-cutting conventions (26.05)

Topic	Convention
Secrets primary	sops-nix (agenix compared once)
Secret consumption	Runtime paths , never Nix strings of secret values
DB / metrics / backends	Listen on 127.0.0.1 unless documented otherwise
Proxy	One of Caddy or Nginx
OCI	Prefer Podman ; localhost port publishes
Ports	Living docs/ ports.md
ACME tokens / DNS API	sops only

Security non-goals (this part)

Non-goal	Note
Full CIS benchmark automation	Patterns over checklists
Multi-AZ HA databases	Single-node honesty
Corporate SSO platform	Mentions only
“Containers make it secure”	Kernel shared; be honest
Required Kubernetes	Optional chapter

Skills checklist (end of part)

- ☐ Can explain why store vault in one minute
 - ☐ sops-nix: encrypt, activate, consume path
 - ☐ House secrets default documented; dual-stack cleaned up
 - ☐ Hardening module: no password SSH, recovery path written
 - ☐ Firewall on; every open port in `docs/ports.md`
 - ☐ TLS (or documented lab alternative) reverse proxy
 - ☐ PostgreSQL (or one DB) with backup/restore story
 - ☐ Isolation decision (native vs nspawn vs OCI)
 - ☐ One declarative OCI container **or** justified refusal
 - ☐ Observability path: metrics and/or logs runbook
 - ☐ k3s decision recorded (skip is first-class)
-

Reading strategies

Mode	Approach
Spine first	secrets → sops → harden → firewall → TLS → postgres → observability
Compare tools	sops vs agenix; nspawn vs OCI; metrics vs logs
Audit existing host	secrets problem + firewall + ports.md first
Resource-constrained	skip k3s; logs-first observability; native modules over k8s

Relationship to other parts

Earlier	This part uses
Host networking / firewall intro	Deepens discipline
Systemd via Nix	Service units, ordering
Flake layout / HM / locks	Where modules and inputs live
Later: ops, packaging, capstone	Deploy, tests, multi-node, wipe proofs

Lab host assumptions

Examples use host `lab`, user `alice`, system `x86_64-linux`, domains like `app.example.invalid` (replace with real DNS or lab TLS). **Never** paste production secrets into the book examples or your git history.

Next parts

Packaging, ops and fleet, and **capstone** assume you can keep secrets out of the store, open only justified ports, and rebuild a service slice from the flake. If this part is thin, those parts become theater.

The secrets problem

The secrets problem

Goal: Internalize why **secrets-as-Nix-strings** fail on NixOS: the store is a world-readable object database. Find and remove secret anti-patterns from your lab flake before wiring sops-nix.

! Important

Never put live passwords, API tokens, or private keys in `flake.nix`, modules, or HM text files. Examples below use obvious placeholders only.

Why this chapter exists

Real services need credentials. The naive NixOS instinct:

```
# CATASTROPHIC PATTERN - do not use
services.someApp.password = "hunter2";
```

or:

```
# ALSO BAD
environment.variables.API_TOKEN = "sk-live-...";
```

Those values enter the **Nix store**, typically **world-readable**, often copied to binary caches if you ever push paths, and immortalized in old generations until GC.

This chapter is pure threat model + cleanup. Implementation tools: **sops-nix** (next) and **agenix** (comparison).

Theory 1 — The store is not a vault

Recall the store and closures chapter: store paths are immutable objects. Permissions are generally:

```
/nix/store/... readable by all users on the machine
```

So:

Belief	Reality
“It’s in my private git repo”	Still world-readable on every machine that builds/activates
“Only root rebuilds”	Any local user can <code>cat</code> the store path
“I’ll delete the file later”	Old generations may retain the path
“I’ll push to my cache”	You may distribute the secret to the cache’s consumers
“HM mode 600 symlink”	Target in store is still world-readable

Evaluation vs runtime

Nix evaluation **sees strings**. If a module option takes a string password, that string becomes part of a derivation input or config file in the store.

```
Nix string secret
→ appears in .drv / generated config
→ realized under /nix/store
→ readable by local users (and caches if pushed)
```

Good secrets appear as **paths to runtime files** (`tmpfs /run/secrets/...`) that are not Nix-string content of the secret itself.

Theory 2 — Common anti-patterns catalog

Pattern	Why it fails
Password options in nixpkgs modules set to plaintext	Store + process listing risk
<code>writeText "secret" "value"</code>	Explicit store file with secret
HM <code>home.file</code> with tokens	Store-backed home links
Embedding TLS private keys in module <code>text</code>	Private key in store
<code>builtins.readFile ./secret.key</code> into config	File content becomes string in eval graph
Committing <code>.env</code> with production secrets	Git history forever
<code>environment.sessionVariables</code> with tokens	Often ends up visible
Embedding secrets in <code>shellHook</code>	Project flakes leak too

`builtins.readFile` trap

```
# STILL BAD if the result is interpolated into world-readable config
services.foo.token = builtins.readFile ./token.txt;
```

Reading a non-store private file at eval time can also break purity and CI; when it “works,” it often still **copies content into the store** via generated configs.

Theory 3 — What “good” looks like (preview)

Approach	Idea
sops-nix (primary this book)	Encrypted files in git; decrypt on activation to root-owned paths
agenix	Age-encrypted secrets; similar activation story
External secret inject	Cloud SM, Vault—out of band at runtime
Password hashes only	<code>hashedPassword</code> for users—not reversible plaintext
SSH keys	Public in git; private only on disk/agent
LoadCredential / EnvironmentFile	systemd reads path at start

Good secrets:

1. Appear in git **only encrypted** (or not at all)
2. Land on disk at **activation/runtime** with tight permissions
3. Are **not** Nix string inputs to public derivations

```
# GOOD shape (conceptual)
# service reads config.sops.secrets."app/db_password".path
# NEVER builtins.readFile that path into another derivation
```

Theory 4 — Generations and GC

Even after you remove a secret from the flake:

- Previous system generations may still reference store paths containing it

- `nix-collect-garbage` eventually drops unreachable paths—but not if a generation roots them

Incident response sketch:

1. **Rotate** the credential (assume compromise)
2. Remove from config
3. Delete old generations once new system confirmed
4. GC
5. If paths were pushed to a cache: treat cache as compromised for that secret

```
sudo nix-env --list-generations -p /nix/var/nix/profiles/system
# sudo nix-collect-garbage -d # only when you accept dropping old gens
```

Theory 5 — Multi-user threat on a “lab laptop”

If your lab has only one human user, risk feels abstract—until:

- You enable a network service
- A second local account exists
- A container escapes
- You copy a store path off-machine
- A backup tarball includes `/nix/store`

Design as if **local multi-user** and **backup tarballs of `/nix/store`** exist.

Theory 6 — Distinguishing config from secret

Config (OK in git)	Secret (not OK plaintext)
Port numbers	DB passwords
Public hostnames	API tokens
Public SSH keys	Private SSH keys
Non-secret feature flags	Session signing keys
ACME email (usually)	ACME account private material (tool-managed)
Package names	Registry pull tokens

Gray areas (document decisions): internal-only hostnames, non-production lab passwords that still teach bad habits—**prefer good habits even in lab**.

Theory 7 — Process listings and logs

Even outside the store:

Risk	Mitigation
ps shows command-line secrets	Prefer env files / credentials; never <code>cmd --password x</code>
journald logs echo secrets	Never <code>echo "\$TOKEN"</code> in units
Core dumps	Limit where appropriate; avoid secrets in argv

Theory 8 — Git history is forever (enough)

Force-push rewriting history is incomplete if:

- Forks and clones exist
- CI caches retained logs
- Chat paste included the secret

Rotation is mandatory after exposure. History scrubbing is extra, not a substitute.

Worked example — Finding secrets with search

```
cd /path/to/your-flake
rg -n -i 'password\s*=\s*\s*' .
rg -n -i 'token\s*=\s*\s*' .
rg -n -i 'api[_-]?key' .
rg -n -i 'BEGIN OPENSSH PRIVATE KEY|BEGIN RSA PRIVATE KEY|BEGIN SECRET' .
rg -n 'hashedPassword|initialHashedPassword' .
rg -n -i 'secret|passwd|private_key' --glob '*.nix' .
```

hashedPassword is OK if it is a **hash**, not a plaintext password field.

Worked example — Safe store readability demo

```
# DO NOT MERGE - educational package in flake outputs temporarily
# packages.x86_64-linux.fakeSecretDemo =
#   pkgs.writeText "fake-secret" "PLACEHOLDER_ONLY";

# nix build .#fakeSecretDemo
# cat result
# ls -la result
```

Then **delete** the demo. Point: content is just a file in the store.

Exercises

Exercise 1 — Threat model paragraph

Write 10–15 lines:

- Who can read `/nix/store` on your lab host?
- Who can read your git remote?
- What is the worst secret currently at risk (even if lab-only)?

Exercise 2 — Audit the flake for anti-patterns

Run searches from the worked example. Open every hit. Classify:

Hit	OK / Bad / Investigate
-----	------------------------

Fix **Bad** by removing or replacing with placeholders and a comment `// TODO sops`.

Exercise 3 — Prove store readability (safe demo)

Build a **fake** `writeText` or temporary package; show world-readable content; delete.

Exercise 4 — User password hygiene check

If any user still has plaintext password configuration, migrate to:

- SSH keys only for remote, and/or
- `hashedPassword` generated via `mkpasswd`

```
mkpasswd -m yescrypt
```

Do not commit the interactive password itself.

Exercise 5 — Remove anti-patterns and rebuild

```
sudo nixos-rebuild switch --flake .#lab
```

Confirm services you care about still run **without** embedding secrets yet.

Exercise 6 — Incident checklist card

Write a 5-step card for “secret committed to git”:

1. Rotate
2. Rewrite history or accept leak + rotate (policy)
3. Remove from flake
4. Drop generations / GC
5. Audit caches

Exercise 7 — HM file audit

```
rg -n 'home\.file|xdg\.configFile' homes/ || true
```

Ensure no token-like `.text` values.

Exercise 8 — Environment variable audit

```
rg -n 'environment\.(variables|sessionVariables)|Environment\s*=' .
```

Exercise 9 — Generation awareness

```
sudo nix-env --list-generations -p /nix/var/nix/profiles/system | tail
```

Note how many generations might hold old secrets if any ever existed.

Exercise 10 — Backup story note

If you back up the host, does the backup include store paths? Write one sentence on secret risk in backups.

Exercise 11 — Public vs private key check

Confirm only **public** SSH keys live in the flake; private keys only on disk/agent.

Exercise 12 — Document TODO list for sops

List every secret the next service chapters will need (DB password, optional Grafana admin, DNS token). No values—names only.

Exercise 13 — Peer review simulation

Read your threat model as if onboarding a coworker. Fix any “lab only, nobody cares” handwaves.

Exercise 14 — CI log risk

If CI evaluates the flake, confirm logs would not print secrets (they should not be in eval). Note any impure `readFile` risks.

Common gotchas

Symptom / mistake	What to do
“Lab only, nobody cares”	Habits transfer to prod; practice correctly
Secret in old generation	Rotate + GC old gens
HM file “mode 600” but store-backed	Still world-readable store object
<code>readFile</code> feels private	Content still often ends up in store configs
Binary cache push after leak	Rotate; scrub if possible; assume leak
Focusing on encryption theater without rotation	Rotation is mandatory after exposure
hashedPassword confused with password	Hash only in git
Secrets in commit messages	Rotate; amend is not enough if pushed
Demo package left merged	Remove educational <code>writeText</code>
Thinking sops alone without path discipline	Still don’t <code>readFile</code> into store

Checkpoint

- ☐ Can explain why store vault in one minute
- ☐ Repo audited with secret heuristics
- ☐ Anti-patterns removed or ticketed for sops-nix
- ☐ Fake store demo understood and deleted
- ☐ User access uses keys/hashes, not plaintext passwords in git
- ☐ Incident rotation checklist written
- ☐ Named list of future secrets (no values)
- ☐ HM and environment audits clean

Journal (optional)

Write three personal gotchas before continuing.

sops-nix

sops-nix

Goal: Wire **sops-nix** end-to-end: encrypt a secret in-repo, decrypt at activation, consume it from a NixOS service or timed script—**without** plaintext secrets in the Nix store evaluation graph.

i Note

House primary: sops-nix. The agenix chapter compares agenix; do not run both as permanent dual systems unless you have a reason.

! Important

Use **placeholder values** and lab-only credentials. Never paste production tokens into examples or commits.

Why this chapter exists

The secrets problem chapter established the threat model. This chapter installs the **default tooling** for this book:

Property	sops-nix approach
At rest in git	Encrypted YAML/JSON/ENV (SOPS)
Encryption keys	Age and/or GPG; age recommended
On host	Decrypted at activation to a path like <code>/run/secrets/...</code>
Nix modules	Reference secret paths , not secret strings

Theory 1 — Components

Piece	Role
SOPS	Encrypts/decrypts structured files
age	Modern encryption tool; identities (private) + recipients (public)
sops-nix	NixOS/HM modules: decrypt during activation; systemd credentials patterns

Piece	Role
<code>.sops.yaml</code>	Creation rules: which keys encrypt which files

Flow

```
edit plaintext locally (editor via `sops file`)
  → encrypted file committed
  → nixos-rebuild
  → sops-nix activation decrypts with host/user age key
  → secret file on tmpfs/run with tight perms
  → service reads path
```

Theory 2 — Age keys (host-centric pattern)

Common NixOS pattern: **host age key** on the machine, public key in `.sops.yaml` creation rules.

Generate key (on the lab host; protect private key):

```
nix shell nixpkgs#age -c age-keygen -o "$HOME/.config/sops/age/keys.txt"
# private key file - NEVER commit
# public key printed as age1...
```

For unattended host decryption, sops-nix docs describe placing keys where the activation can read them (often `/var/lib/sops-nix/key.txt` or configured path)—**follow current sops-nix README for 26.05-compatible setup**. Private key stays on host disk with root-only permissions—not in git.

Tip

Also encrypt to **your admin age key** so you can decrypt on a workstation for editing. Creation rules can list multiple recipients.

Key backup

Losing all private keys without other recipients = **cannot decrypt**. Keep admin recipient + offline backup of identities.

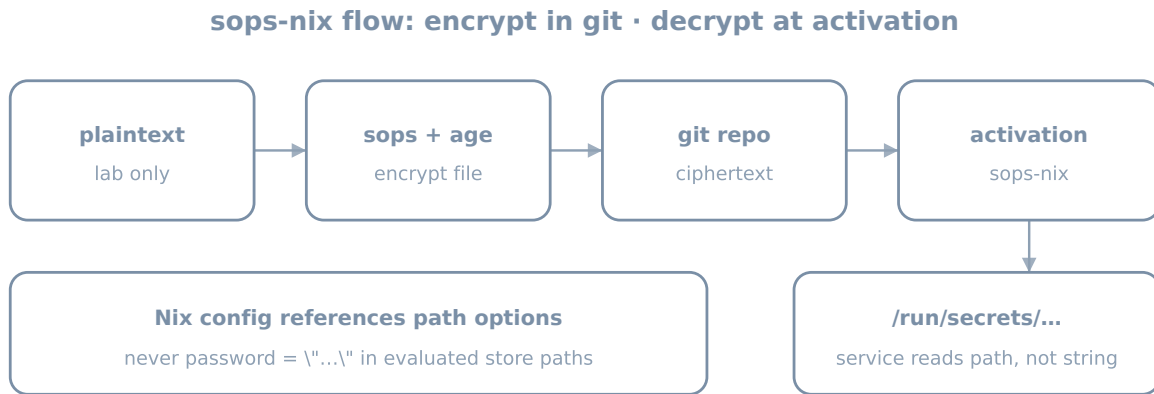


Figure 1: sops-nix encrypt decrypt flow

Theory 3 — Flake input

```

inputs = {
  nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  sops-nix.url = "github:Mic92/sops-nix";
  sops-nix.inputs.nixpkgs.follows = "nixpkgs";
};

# modules list includes:
# inputs.sops-nix.nixosModules.sops

# flake.nix sketch
outputs = { self, nixpkgs, sops-nix, ... }@inputs: {
  nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
    system = "x86_64-linux";
    specialArgs = { inherit inputs; };
    modules = [
      ./hosts/lab
      sops-nix.nixosModules.sops
      # or import modules/common/sops.nix that uses inputs
    ];
  };
};

```

Update only this input when bumping:

```
nix flake update sops-nix
```

Theory 4 — Minimal NixOS sops config

```
# modules/common/sops.nix
{ config, ... }:
{
  sops.defaultSopsFile = ../../secrets/lab.yaml;
  # sops.age.keyFile = "/var/lib/sops-nix/key.txt"; # per upstream guidance
  # sops.age.sshKeyPaths = [ ... ]; # alternative: derive from SSH host key

  sops.secrets."lab/demo_token" = {
    mode = "0400";
    owner = "root";
    # restartUnits = [ "some-service.service" ];
  };
}
```

Consume **path**:

```
systemd.services.sops-demo-check = {
  wantedBy = [ "multi-user.target" ];
  serviceConfig = {
    Type = "oneshot";
    RemainAfterExit = true;
    ExecStart = "/bin/sh -c 'test -f /run/secrets/lab/demo_token && echo ok > /var/lib/sops-";
  };
};
```

Exact secret path layout depends on sops-nix version and naming—inspect after activation:

```
sudo ls -la /run/secrets
```

Never do `builtins.readFile config.sops.secrets."lab/demo_token".path` into another world-readable derivation if that re-embeds content. Prefer `systemd LoadCredential`, `EnvironmentFile` pointing at the secret path, or scripts that read at runtime.

Theory 5 — Creating encrypted files

`.sops.yaml` example shape:

```
keys:
- &admin age1AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
- &host age1BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB
creation_rules:
```

```
- path_regex: secrets/.*\.yaml$
  key_groups:
    - age:
      - *admin
      - *host
```

Edit:

```
nix shell nixpkgs#sops nixpkgs#age -c sops secrets/lab.yaml
```

Example structure **before** encryption (editor buffer only):

```
lab:
  demo_token: PLACEHOLDER_LAB_TOKEN_ROTATE_ME
```

After save, file on disk is encrypted ciphertext suitable for git.

Adding a secret later

1. `sops secrets/lab.yaml`
2. Add key under structured YAML
3. Declare `sops.secrets."` in Nix
4. Rebuild
5. Point service at `.path`

Theory 6 — HM secrets (awareness)

`sops-nix` also integrates with Home Manager for user secrets. Prefer **system** secrets for host services first; user secrets when a user-only app needs them.

Theory 7 — Bootstrapping chicken-and-egg

First host key setup is the awkward part:

1. Generate age key on host
2. Put **public** key in `.sops.yaml`
3. Encrypt secrets to that recipient
4. Place **private** key where `sops-nix` expects
5. Rebuild

Document bootstrap in `docs/secrets.md`.

SSH host key method (awareness)

Some setups convert or use SSH host keys as age identities. Follow upstream docs if you choose that—document which method you used.

Theory 8 — systemd integration patterns

Pattern	Use
<code>config.sops.secrets.*.path</code> in <code>ExecStart</code> script	Read file at runtime
<code>LoadCredential=</code>	systemd credentials API
<code>EnvironmentFile=</code>	If file is <code>KEY=value</code> format
<code>restartUnits</code>	Restart service when secret changes

```
sops.secrets."lab/demo_token" = {  
  mode = "0400";  
  restartUnits = [ "lab-token-touch.service" ];  
};
```

Theory 9 — Multi-host recipients

```
creation_rules:  
- path_regex: secrets/lab\.yaml$  
  key_groups:  
    - age: [*admin, *host_lab]  
- path_regex: secrets/prod\.yaml$  
  key_groups:  
    - age: [*admin, *host_prod]
```

Do not encrypt prod secrets only to a laptop key if the server must decrypt unattended.

Worked example — Secret for a custom script service

```
{ config, pkgs, ... }:
{
  sops.secrets."lab/demo_token" = {
    mode = "0400";
    owner = "root";
  };

  systemd.services.lab-token-touch = {
    description = "Demonstrate runtime read of sops secret path";
    wantedBy = [ "multi-user.target" ];
    serviceConfig = {
      Type = "oneshot";
      RemainAfterExit = true;
      ExecStart = pkgs.writeShellScript "lab-token-touch" ''
        set -euo pipefail
        token_path="${config.sops.secrets."lab/demo_token".path}"
        test -r "$token_path"
        # Do NOT echo the token to logs
        wc -c < "$token_path" > /var/lib/lab-token-bytes
      '';
    };
  };
}
```

Note: `writeShellScript` embeds the **path string**, not the secret content—good.

Exercises

Exercise 1 — Add input and module

```
# edit flake inputs; nix flake update sops-nix
# import sops-nix.nixosModules.sops
nix flake metadata
```

Exercise 2 — Age keys and `.sops.yaml`

1. Generate admin age identity (laptop)
2. Generate or configure host identity per sops-nix docs
3. Write `.sops.yaml` with both public keys
4. Confirm private keys are gitignored


```
keys.txt
*.agekey
```

Exercise 3 — First encrypted secret file

```
mkdir -p secrets
# create encrypted secrets/lab.yaml via sops
git add secrets/lab.yaml .sops.yaml
# ensure plaintext never staged
```

Exercise 4 — Declare sops.secrets and rebuild

```
sudo nixos-rebuild switch --flake .#lab
sudo ls -la /run/secrets
sudo wc -c /run/secrets/... # do not cat into scrollbar you paste publicly
```

Exercise 5 — Runtime consumer

Add the oneshot that **reads path** without logging secret contents. Prove `/var/lib/lab-token-bytes` or similar.

Exercise 6 — Failure drill

1. Break key access (rename key file temporarily)
2. Rebuild or restart activation path
3. Observe failure mode
4. Restore key

Journal: how the host fails closed.

Exercise 7 — Docs

`docs/secrets.md`:

- Key locations
- How to add a secret
- How to add a recipient
- Rotation sketch

Exercise 8 — Selective lock update

```
nix flake update sops-nix
git diff flake.lock | head -80
```

Exercise 9 — Permission check

```
sudo ls -la /run/secrets
# confirm mode 0400 (or intended) and owner
```

Exercise 10 — Second secret

Add another key in the YAML; declare second `sops.secrets` entry; rebuild; list paths.

Exercise 11 — No readFile audit

```
rg -n 'builtins.readFile.*sops|sops\.secrets' .
```

Ensure no anti-pattern re-embedding.

Exercise 12 — Journal hygiene

```
journalctl -u lab-token-touch.service -b --no-pager | tail
```

Confirm secret values never appear.

Exercise 13 — Recipient add rehearsal (docs)

Write steps to add a second admin age public key without performing a risky prod op.

Exercise 14 — Bootstrap checklist in git

Commit `docs/secrets.md` with bootstrap steps a future you can follow on a blank host.

Common gotchas

Symptom / mistake	What to do
Cannot decrypt on host	Host public key not in creation rules / wrong private key path
Cannot edit on laptop	Admin age key missing from recipients
Secret path empty	Activation order; sops config; check journal
Plaintext committed	Rotate; purge if needed; fix
<code>readFile</code> secret into store	Use runtime path only
Logged secret via <code>echo \$TOKEN</code>	Never log secrets
GPG-only complexity	Prefer age for new labs
Private key in git	Rotate immediately; purge
Wrong defaultSopsFile path	Fix relative path from module file
Dual sops+agenix forever	Pick default after comparison
follows missing on sops-nix	chapter Align nixpkgs

Checkpoint

- ☐ sops-nix input pinned with follows
- ☐ `.sops.yaml` with host + admin recipients
- ☐ Encrypted secret file in git
- ☐ Private keys not in git
- ☐ `sops.secrets` declared; appears under `/run/secrets`
- ☐ Service/script consumes **path** only
- ☐ Bootstrap docs written
- ☐ Failure-closed drill done once

Journal (optional)

Write three personal gotchas before continuing.

agenix (comparison)

agenix (comparison)

Goal: Implement the **same lab secret** once with **agenix**, compare trade-offs to sops-nix, and declare a **house default** (this book: **sops-nix** primary).

i Note

Comparison chapter dual-stack forever. After the lab, remove or disable the non-default tool so you maintain one system.

Why this chapter exists

Both sops-nix and agenix solve “encrypted secrets in git + decrypt on activation.” Teams pick based on:

Factor	Why it matters
File format UX	YAML multi-secret files vs one age file per secret
Tooling	sops editor vs agenix CLI
Ecosystem fit	Docs you read; modules you copy
Complexity	Bootstrap and key distribution

You should **not** choose from social-media vibes alone. Touch both once.

Theory 1 — agenix mental model

[agenix](#) (and compatible community usage) roughly:

Concept	Meaning
secrets.nix	Maps secret files → public keys allowed to decrypt
*.age files	Ciphertext in repo
age identities	Private keys on machines/admins
NixOS module	<code>age.secrets.<name>.file = ./secret.age;</code> → runtime path

Flow

```
echo 'PLACEHOLDER' | agenix -e secret.age
git add secret.age secrets.nix
nixos-rebuild → decrypt to /run/agenix/... (path scheme per version)
service reads path
```

Theory 2 — Flake input sketch

```
inputs = {
  nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  agenix.url = "github:ryantm/agenix";
  agenix.inputs.nixpkgs.follows = "nixpkgs";
};

# modules list:
# inputs.agenix.nixosModules.default
```

Pin and follows like any other input (lock discipline).

```
nix flake update agenix
```

Theory 3 — secrets.nix shape

```
# secrets/secrets.nix - public keys only
let
  # replace with your real age public keys
  admin = "age1AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA";
  host  = "age1BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB";
in
{
  "lab-demo-token.age".publicKeys = [ admin host ];
}
```

No private keys in this file.

Identity reuse

Reuse **public** age keys from the sops-nix setup when possible so you are not juggling unrelated key material during comparison.

Theory 4 — NixOS declaration

```
{ config, inputs, ... }:
{
  imports = [ inputs.agenix.nixosModules.default ];

  age.secrets.lab-demo-token = {
    file = ../../secrets/lab-demo-token.age;
    # mode / owner as needed
  };

  # config.age.secrets.lab-demo-token.path → runtime path
}
```

Consumer pattern matches sops-nix: **path at runtime**, never embed content in derivations.

```
systemd.services.agenix-demo = {
  wantedBy = [ "multi-user.target" ];
  serviceConfig = {
    Type = "oneshot";
    RemainAfterExit = true;
    ExecStart = "/bin/sh -c 'test -r ${config.age.secrets.lab-demo-token.path}'";
  };
};
```

Theory 5 — Side-by-side comparison

Dimension	sops-nix	agenix
Multi-key structured file	Natural (YAML tree)	Usually one file per secret
Edit UX	sops file in-place decrypt editor	agenix -e
Creation rules	.sops.yaml	secrets.nix publicKeys
HM integration	Strong story	Available patterns
Learning transfer	SOPS used outside Nix too	Age-centric, Nix-flavored

Dimension	sops-nix	agenix
House default (this book)	Primary	Compare once

When agenix might still win for you

- You want the simplest possible “one secret, one file” model
- Your team already standardized on agenix
- You dislike SOPS YAML tooling

When sops-nix wins for this curriculum

- Multiple secrets per environment file
- Same SOPS skills in non-Nix contexts
- Book consistency for later examples

Theory 6 — Dual-running risks

Running both modules on one host:

- Two key bootstraps
- Two runtime directories
- Cognitive load on incident response

Acceptable **for one comparison lab**. Not acceptable as permanent undefined state—pick a default before the next service chapters pile on.

Theory 7 — Declaring house default

Write in docs/secrets.md:

```
## House default

**sops-nix** for all new secrets.

agenix: lab comparison only; module removed after comparison
(or: we chose agenix because ... - only if you override the book).
```

If you override the book, do so **explicitly** so later sops-shaped examples are translated consciously.

Theory 8 — Re-encryption and rotation

Event	sops-nix	agenix
Add recipient	update <code>.sops.yaml</code> , re-encrypt	update <code>secrets.nix</code> , re-encrypt
Rotate value	edit via tool	re-encrypt file
Rotate keys	re-encrypt all files	re-encrypt all <code>.age</code>

Practice rotation on a **placeholder** secret, not production.

Theory 9 — Path API discipline (both tools)

```
# GOOD: path reference
environment.etc."demo-path".text = config.age.secrets.lab-demo-token.path;
# (still careful: only the path string)

# BAD: content into store
# environment.etc."demo".text = builtins.readFile config.age.secrets....path;
```

Same rule as sops: runtime consumers, not eval-time content embedding.

Worked example — Same placeholder secret both ways

Tool	Artifact
sops	<code>secrets/lab.yaml</code> key <code>lab.demo_token</code>
agenix	<code>secrets/lab-demo-token.age</code>

Same placeholder value during local edit; different ciphertext formats in git. Runtime paths differ; consumer scripts should use module-provided `.path`.

Exercises

Exercise 1 — Add agenix input

```
# flake input + lock update agenix only
nix flake update agenix
nix flake metadata
```

Exercise 2 — Public keys and secrets.nix

Reuse **public** age keys from sops-nix if possible. Write `secrets/secrets.nix` with `publicKeys` only.

Exercise 3 — Encrypt one age secret

Using current agenix/age CLI docs for your pin:

```
nix shell github:ryantm/agenix#agenix nixpkgs#age
# agenix -e secrets/lab-demo-token.age
```

Put only a **lab placeholder** value inside.

Exercise 4 — NixOS module + rebuild

Declare `age.secrets....`, rebuild, list runtime secrets dir for agenix:

```
sudo nixos-rebuild switch --flake .#lab
sudo ls -la /run/agenix 2>/dev/null || sudo ls -la /run/secrets 2>/dev/null || true
```

Confirm path exists; do not paste secret contents into notes.

Exercise 5 — Consumer parity

Point a oneshot at `config.age.secrets.lab-demo-token.path` similar to sops-nix. Prove readable by the intended user.

Exercise 6 — Comparison write-up

One page in the journal:

1. Bootstrap friction: sops vs agenix
2. Day-2 secret add friction (ongoing maintenance)
3. Multi-secret ergonomics
4. Failure modes observed
5. **Decision:** house default = ...

Exercise 7 — Decommission the non-default

If house default is sops-nix:

1. Remove agenix module from active host imports
2. Optionally leave encrypted `.age` file as historical artifact or delete
3. Remove agenix input **or** keep input unused (prefer remove to simplify lock)
4. Rebuild clean

Opposite steps if you chose agenix (then translate later chapters).

Exercise 8 — docs/secrets.md house default section

Commit the decision explicitly.

Exercise 9 — follows check

Confirm agenix input used `follows` while present; clean lock after removal.

Exercise 10 — Incident note

Add one paragraph: how rotation differs if dual tools had been left enabled.

Exercise 11 — Team rationale

Write two sentences you would tell a teammate who prefers the other tool.

Exercise 12 — Path-only audit

```
rg -n 'age\.secrets|sops\.secrets' modules/ || true
```

Ensure consumers use `.path`.

Exercise 13 — Rebuild with single stack

```
sudo nixos-rebuild switch --flake .#lab
nix flake metadata # no unexpected dual inputs if removed
```

Exercise 14 — Future secret list

Map upcoming secrets (DB, proxy DNS token, Grafana) to the **chosen** tool only.

Common gotchas

Symptom / mistake	What to do
Private key in secrets.nix	Remove; rotate
Wrong public key list	Re-encrypt with correct recipients
Both tools half-configured	Finish decommission exercise
Assuming identical paths	Always use <code>.path</code> from the module
Editing ciphertext as plaintext	Use tool endpoints only
Forgetting follows on agenix input	Fix lock discipline
Leaving dual forever “for flexibility”	Pick one; document override
Comparison without write-up	Decision is the deliverable
Production secrets in comparison	Placeholders only

Checkpoint

- ☐ agenix encrypt + decrypt-on-activate worked once
- ☐ Comparison table completed in your words
- ☐ House default declared in `docs/secrets.md`
- ☐ Non-default tool removed or clearly disabled
- ☐ Still no plaintext secrets in git
- ☐ Rebuild succeeds with chosen stack only
- ☐ Future secrets mapped to one tool
- ☐ follows/lock cleaned up

Journal (optional)

Write three personal gotchas before continuing.

Host hardening patterns

Host hardening patterns

Goal: Apply a **small, documented hardening module** for a lab server: SSH posture, sudo policy, disable unused services, and careful sysctl/kernel defaults—without cargo-culting every CIS line item.

Warning

Hardening can lock you out of a remote VM. Keep a **console/VNC path** or test with `nixos-rebuild test` / boot generation rollback ready.

Why this chapter exists

Secrets tooling protects credentials. Hardening reduces **attack surface** and enforces least privilege. Production-shaped labs should not run with:

- Password SSH
- Root SSH login
- Unnecessary network services
- Empty sudo policies that surprise you later

This chapter builds a **module you can import** on server roles—not a security theater checklist of 200 toggles.

Theory 1 — Threat model for this lab

Assume:

Asset	Threat
SSH on a public IP	Password spray, key theft
Local users	Accidental privilege escalation
Services you enable next	Extra open ports
Laptop multi-boot lab	Lower remote risk, still practice

Non-goals: full SELinux policy authoring, corporate MDM, compliance certifications.

Role split

Role	Hardening intensity
server	Strict SSH, fewer conveniences
workstation	Keys still; keep desktop UX

Theory 2 — SSH baseline (NixOS)

```
# modules/common/hardening/ssh.nix
{
  services.openssh = {
    enable = true;
    settings = {
      PasswordAuthentication = false;
      KbdInteractiveAuthentication = false;
      PermitRootLogin = "no";
      X11Forwarding = false;
      # MaxAuthTries = 4;
      # AllowUsers = [ "alice" ];
    };
  };
}
```

Setting	Intent
No passwords	Keys only
No root login	Use <code>sudo</code> from admin user
No X11 forward	Reduce attack surface on servers

Ensure your user has `openssh.authorizedKeys.keys` **before** applying password-off if you ever had passwords.

Optional allowlists

```
services.openssh.settings.AllowUsers = [ "alice" ];
# services.openssh.listenAddresses = [ { addr = "0.0.0.0"; port = 22; } ];
```


Theory 3 — sudo and wheel

```
# modules/common/hardening/sudo.nix
{
  security.sudo = {
    enable = true;
    wheelNeedsPassword = true; # or false for lab convenience - document choice
    execWheelOnly = true;
  };
  # users.users.alice.extraGroups = [ "wheel" ];
}
```

Policy	Trade-off
<code>wheelNeedsPassword = true</code>	Safer; needs password/TTY
<code>= false</code>	Convenient lab; weaker if key stolen + local

Prefer **true** on anything network-exposed.

Theory 4 — Disable what you do not use

Examples (only if present/enabled):

```
{
  services.avahi.enable = false;
  services.printing.enable = false;
  hardware.bluetooth.enable = false;
}
```

Do not randomly disable things you depend on (NetworkManager on a laptop role). Use **roles**:

```
# modules/roles/server.nix
{
  imports = [ ../common/hardening ];
}

# modules/roles/workstation.nix
{
  # lighter hardening; keep user conveniences
}
```

Theory 5 — Kernel / sysctl care

NixOS exposes many `boot.kernel.sysctl` knobs. Apply **few**, understand each:

```
{
  boot.kernel.sysctl = {
    "net.ipv4.conf.all.rp_filter" = 1;
    "net.ipv4.tcp_syncookies" = 1;
    # Do not paste a 50-line blog dump without reading
  };
}
```

Kernel package pinning and hardened profiles exist in the ecosystem; treat full “hardened kernel” as optional advanced work—not required for this part.

Theory 6 — Automatic updates? (judgment)

Unattended upgrades on NixOS are a product decision:

- Flake pins mean “latest” is not automatic unless you automate lock bumps + rebuilds
- Auto-reboot policies can surprise

For this book: **manual, deliberate updates** (lock discipline) unless you later build CI-driven update PRs.

Theory 7 — Module shape

```
modules/common/hardening/
  default.nix    # imports ssh.nix sudo.nix misc.nix
  ssh.nix
  sudo.nix
  misc.nix

# default.nix
{
  imports = [ ./ssh.nix ./sudo.nix ./misc.nix ];
}
```

Import from server role or common—**document** if workstations share it.

Theory 8 — Assertions as guardrails

```
{ config, lib, ... }:  
{  
  assertions = [  
    {  
      assertion = config.services.openssh.settings.PasswordAuthentication == false;  
      message = "PasswordAuthentication must be false on this host role";  
    }  
  ];  
}
```

Assertions fail evaluation early—better than silent drift.

Theory 9 — Safe apply order

1. Confirm console access
2. Confirm admin key SSH works
3. `nixos-rebuild test or boot`
4. Open new SSH session before closing old
5. Only then `switch` if using `switch` and tests pass

```
sudo nixos-rebuild test --flake .#lab  
# new ssh session:  
ssh lab  
sudo -v
```

Worked example — Recovery card

```
# Recovery if SSH rejects  
  
1. Hypervisor / cloud serial console  
2. Login locally  
3. sudo nixos-rebuild switch --rollback  
   # or pick previous generation at bootloader  
4. Fix flake; re-apply carefully
```

Keep this in docs/hardening.md.

Exercises

Exercise 1 — Inventory surface

```
ss -tulpn | sort
systemctl list-unit-files --state=enabled | head -80
```

Journal: what is listening, what you do not recognize.

Exercise 2 — Write hardening module

Create `modules/common/hardening/` with SSH + sudo + 2–3 sensible disables for **your** role.

Exercise 3 — Apply carefully

```
sudo nixos-rebuild test --flake .#lab
# new ssh session
ssh lab
sudo -v
```

Then switch if satisfied.

Exercise 4 — Lockout recovery drill (document)

Write exact steps to recover if SSH rejects you. Do not skip writing this.

Exercise 5 — Assertions (optional polish)

Add an assertion for `PasswordAuthentication false` (or your critical policy).

Exercise 6 — Diff and explain

```
ss -tulpn
```

Explain each remaining listener in one line (`sshd`, etc.).

Exercise 7 — Key-before-password-off proof

Confirm `authorizedKeys` present for your admin user before disabling passwords.

Exercise 8 — wheel policy document

Write one sentence in `docs/hardening.md` for `wheelNeedsPassword` choice.

Exercise 9 — Role wiring

Import `hardening` from `modules/roles/server.nix` (or equivalent). Rebuild.

Exercise 10 — PermitRootLogin check

```
sudo sshd -T | grep -i -E 'passwordauthentication|permitrootlogin'
```

Exercise 11 — Disable list rationale

For each disabled service, one-line reason in `docs`.

Exercise 12 — Sysctl minimalism

If you added `sysctls`, write what each does. Remove any you cannot explain.

Exercise 13 — Workstation exception note

If you also use this flake on a laptop role, document what differs.

Exercise 14 — Generation rollback rehearsal (console available)

From console, practice listing generations; do not necessarily roll back if healthy.

Symptom / mistake	What to do
-------------------	------------

Common gotchas

Symptom / mistake	What to do
Locked out via SSH	Console + previous generation
PasswordAuth off before keys	Fix via console; add keys
Hardening module on laptop breaks UX	Split server/workstation roles
Blind sysctl paste	Remove unknowns
PermitRootLogin yes “for debug” left on	Remove before network exposure
No recovery notes	Write recovery card before remote travel
test vs switch confusion	Prefer test when changing SSH
AllowUsers typo lockout	Console fix
Firewall not yet disciplined	Next chapters—but do not open random ports here

Checkpoint

- ☐ Hardening module exists and is imported on the lab role
- ☐ Password SSH off; root SSH off
- ☐ sudo policy explicit
- ☐ Unnecessary services disabled with reasons
- ☐ Recovery path documented
- ☐ Rebuild + new SSH session verified
- ☐ Listeners explained
- ☐ Assertions or checklist for critical SSH settings

Journal (optional)

Write three personal gotchas before continuing.

Firewall discipline

Firewall discipline

Goal: Treat the NixOS firewall as a **deny-by-default** policy: open only ports you can name, tie openings to services, and audit with tools—not hope.

Why this chapter exists

Host networking introduced firewall basics. Services (proxy, DB, metrics) tempt:

```
networking.firewall.enable = false; # "lab"
```

or:

```
networking.firewall.allowedTCPPorts = [ 22 80 443 5432 3000 9090 9100 /* ... */ ];
```

Without a story. This chapter is **discipline**: every open port has an owner service and a reason.

Theory 1 — NixOS firewall model

With `networking.firewall.enable = true` (default on many installs):

Mechanism	Role
nftables/iptables backend	Implementation (prefer not hand-editing)
<code>allowedTCPPorts</code> / UDP	Host-wide opens
<code>allowedTCPPortRanges</code>	Ranges when unavoidable
Interface-specific options	Limit exposure to LAN interfaces
Service modules	Often open ports via <code>openFirewall</code> options

Prefer service options

```
services.openssh.enable = true;
# many services offer:
# services.foo.openFirewall = true;
```

That keeps port + service definition adjacent. Global lists become a last resort.

Theory 2 — Deny by default

```
{
  networking.firewall = {
    enable = true;
    allowPing = true; # or false on hardened internet hosts
    allowedTCPPorts = [ ];
    allowedUDPPorts = [ ];
  };
}
```

Then add ports **only** with services. SSH often opened by `services.openssh` + related settings depending on config—**verify** rather than assume.

Check:

```
sudo nixos-option networking.firewall.allowedTCPPorts
sudo nft list ruleset 2>/dev/null | head -100
# or iptables-save depending on backend
```

Theory 3 — Per-service matrix (living document)

Maintain `docs/ports.md`:

Port	Proto	Service	Source scope	Module
22	TCP	sshd	admin IPs / world (lab)	hardening/ssh
443	TCP	caddy/nginx	world	reverse-proxy
5432	TCP	postgres	localhost only	data service

If Postgres is localhost-only, **do not** open 5432 on the firewall.

Theory 4 — Localhost vs LAN vs world

Scope	Pattern
Localhost only	Bind service to 127.0.0.1; no firewall open
LAN only	Firewall interface restrictions / not exposing WAN NIC
World	80/443 typically; auth/TLS required

```
# postgres listen addresses - keep lab default local
services.postgresql.enable = true;
# ensure listen_addresses = 'localhost' for lab default
```

Exact options depend on module—read search.nixos.org for your pin.

Theory 5 — Temporary debug holes

Bad habit: leave `allowedTCPPorts = [ 3000 ]` forever for a dev server.

Better:

- SSH tunnel: `ssh -L 3000:127.0.0.1:3000 lab`
- Or short-lived config on a branch
- Or VPN/Tailscale patterns (document if you use them)

Theory 6 — Interaction with containers (preview)

Containers and port publishes can bypass your mental model if you open host ports carelessly. Treat published ports as first-class firewall entries in `docs/ports.md`.

Theory 7 — Interfaces and trusted nets

```
networking.firewall = {
  enable = true;
  # trustedInterfaces = [ "tailscale0" ]; # only if real
  # interfaces."eth0".allowedTCPPorts = [ 22 ]; # verify option shape on 26.05
};
```

Prefer **least exposure**: admin SSH on a management interface if you have one; never open DB ports “because the app is in the same VPC narrative” without proof.

Cloud security groups

NixOS firewall is **not** a substitute for cloud SG/NSG rules—and vice versa. Document both layers in `docs/ports.md` when the lab is a VPS.

Theory 8 — Connection tracking

Allowing TCP 22 does not require opening random high ports for SSH. Related/established traffic is handled by the firewall state machine. Do not “open 1024–65535 for good measure.”

Theory 9 — Co-locate opens with services

```
# modules/services/proxy.nix
{
  networking.firewall.allowedTCPPorts = [ 80 443 ];
  services.caddy.enable = true;
  # ...
}
```

Deleting the service module should remove the hole—review PRs for orphaned ports.

Worked example — Tight SSH + HTTPS owned by proxy

```
# modules/common/firewall.nix
{ lib, ... }:
{
  networking.firewall = {
    enable = true;
    allowPing = true;
    allowedTCPPorts = [
      # 80 443 added with the proxy module, not here forever empty-handed
    ];
  };
}
```

When proxy lands, open 80/443 in the **proxy** module file.

Worked example — Service openFirewall pattern

```
services.nginx.enable = true;
# services.nginx... openFirewall-style options - check 26.05 module

# Fallback explicit ownership in the same module file:
# networking.firewall.allowedTCPPorts = [ 80 443 ];
```

Exercises

Exercise 1 — Audit current opens

```
ss -tulpn
sudo nft list ruleset 2>/dev/null | head -150
```

Fill the ports table for **current** state.

Exercise 2 — Enable firewall explicitly

Ensure `networking.firewall.enable = true`. Rebuild carefully (console ready).

```
sudo nixos-rebuild test --flake .#lab
```

Confirm SSH still works from a **new** session.

Exercise 3 — Remove unjustified ports

Delete any “temporary” ports. Rebuild. Re-audit `ss` and `ruleset`.

Exercise 4 — Service-tied open (stub)

```
# modules/services/proxy-firewall.nix
# TLS chapter will open 80/443 here when proxy is enabled
```

Wire the idea that **proxy owns those ports**.

Exercise 5 — Localhost service experiment

Run a quick local listener **or** use an existing local-only service; confirm it is not reachable from another machine without a firewall hole and public bind.

```
# on lab: nc -l 127.0.0.1 9999
# from another host: should fail unless you opened the port AND bound 0.0.0.0
```

Exercise 6 — ports.md committed

Commit docs/ports.md with the matrix. Update as services arrive.

Exercise 7 — Diff firewall after change

```
sudo nft list ruleset > /tmp/fw-after.txt
# compare mentally to docs/ports.md
```

Exercise 8 — Ping policy decision

```
networking.firewall.allowPing = true; # or false
```

Write one sentence: who needs ICMP on this host?

Exercise 9 — SSH tunnel habit

Practice one tunnel instead of opening a port:

```
ssh -L 9090:127.0.0.1:9090 lab
```

Exercise 10 — Cloud SG note

If on a VPS, paste (redacted) SG rules into ports.md as a second table.

Exercise 11 — Orphan port hunt

Find any allowedTCPPorts entry without a matching enabled service. Remove or justify.

Exercise 12 — UDP inventory

```
ss -ulpn
```

Document any unexpected UDP listeners.

Exercise 13 — test generation safety

Use `nixos-rebuild test` when tightening rules so reboot returns to previous if you disconnect.

Exercise 14 — Pre-proxy checklist

List ports you expect after TLS + postgres chapters; mark which are localhost-only.

Common gotchas

Symptom / mistake	What to do
Locked out of SSH	Console; previous generation; re-open 22
Firewall on but service bound wrong	Fix listen address
Docker-ish publishes confuse audit	Inventory publish flags (Podman chapter)
<code>enable = false</code> “for lab”	Re-enable; use tunnels for debug
Open 5432 to world	Bind localhost; proxy/app only
Ruleset empty surprise	Backend differ; verify with <code>ss</code> too
Orphan ports after service delete	Review ports.md + module
Opened high port ranges	Close; use tunnels
Trusted interface too broad	Narrow trust

Checkpoint

- ☐ Firewall enabled
- ☐ Every open port documented in `docs/ports.md`
- ☐ No unjustified temporary holes
- ☐ SSH verified after firewall changes
- ☐ Localhost vs world distinction understood
- ☐ Plan for proxy ports owned by proxy module

- ☐ Ping policy decided
- ☐ Tunnel practiced once

Journal (optional)

Write three personal gotchas before continuing.

TLS and reverse-proxy front door

TLS and reverse-proxy front door

Goal: Put a **TLS reverse proxy** in front of a simple backend using **ACME** (Let's Encrypt or lab alternative)—pick **Caddy or Nginx**, not both, as the house reverse-proxy pattern.

i Note

Public ACME needs a public DNS name pointing at your lab. If you cannot, use **internal TLS**, **mkcert**, or Caddy/Nginx self-signed lab modes—and document the deviation. The pattern still counts.

Why this chapter exists

Services should not all expose their own TLS stacks. The production pattern:

```
Internet/LAN clients
  → :443 reverse proxy (TLS terminates)
    → 127.0.0.1:app-port (HTTP plain on localhost)
```

One certificate story, one cipher/policy place, one set of firewall holes (80/443).

Theory 1 — Choose one proxy

Proxy	Strengths	Notes
Caddy	Automatic HTTPS UX, simple config	Great default for labs
Nginx	Ubiquitous, explicit	ACME via security.acme + nginx module

Pick one for the rest of this part. Write the choice in **docs/proxy.md**.

Do not run both on 443.

Theory 2 — ACME on NixOS

NixOS `security.acme` (used heavily with `nginx`) or Caddy’s built-in ACME:

Concern	Practice
Email	Real email for Let’s Encrypt notices
Domains	You control DNS
HTTP-01	Port 80 reachable from internet
DNS-01	Provider token via sops (not plaintext)
Staging	Use LE staging while debugging rate limits

Secrets

ACME account keys are managed on disk by the tooling—do not invent plaintext private keys in the flake. DNS provider tokens: **sops-nix**.

Theory 3 — Caddy pattern (NixOS)

```
# modules/services/proxy-caddy.nix
{ pkgs, ... }:
{
  networking.firewall.allowedTCPPorts = [ 80 443 ];

  services.caddy = {
    enable = true;
    virtualHosts."app.example.invalid".extraConfig = ''
      encode gzip
      reverse_proxy 127.0.0.1:8080
    '';
  };
}
```

Caddy often obtains certs automatically when DNS is real. For `invalid/lab` domains, configure accordingly or use a local-only site block for HTTP lab testing.

Static site alternative:

```
services.caddy.virtualHosts."static.example.invalid".extraConfig = ''
  root * ${pkgs.writeTextDir "index.html" "<h1>lab</h1>"}
  file_server
'';
```

(Adjust to current Caddy module options on 26.05 if attribute names differ—verify with option search.)

Theory 4 — Nginx + ACME pattern

```
# modules/services/proxy-nginx.nix
{
  networking.firewall.allowedTCPPorts = [ 80 443 ];

  security.acme = {
    acceptTerms = true;
    defaults.email = "you@example.invalid";
  };

  services.nginx = {
    enable = true;
    recommendedProxySettings = true;
    recommendedTlsSettings = true;
    virtualHosts."app.example.invalid" = {
      enableACME = true;
      forceSSL = true;
      locations."/" = {
        proxyPass = "http://127.0.0.1:8080";
        proxyWebsockets = true;
      };
    };
  };
};
}
```

Theory 5 — Backend for the lab

Minimal backend options:

1. **Static files** via the proxy itself
2. **Tiny declarative service:**

```
systemd.services.lab-backend = {
  wantedBy = [ "multi-user.target" ];
  serviceConfig = {
    DynamicUser = true;
  };
}
```

```

    ExecStart = "${pkgs.python3}/bin/python3 -m http.server 8080 --bind 127.0.0.1";
    WorkingDirectory = "${pkgs.writeTextDir "index.html" "<h1>backend</h1>"}";
  };
};

```

Bind **127.0.0.1** so the firewall need not open 8080.

Theory 6 — Failure modes

Symptom	Checks
ACME fail	DNS A/AAAA, port 80, rate limits, email/terms
502 bad gateway	Backend down; wrong port
Cert works, app not	<code>proxyPass</code> URL
Works on host, not remote	Firewall 80/443; cloud SG

```

curl -vI https://app.example.invalid
journalctl -u caddy -b --no-pager | tail -50
# or nginx / acme services

```

Theory 7 — Certificate storage and renewal

ACME clients store account/certs under state dirs (often under `/var/lib`). That state is **not** in the Nix store.

Event	Result
System generation rollback	Software/config may roll back; certs usually remain
Wipe <code>/var</code>	Certificates gone; re-issue (rate limits!)
Domain change	New cert path; update proxy vhost

Back up ACME state on anything beyond a disposable lab, or accept re-issuance cost.

Theory 8 — HTTP → HTTPS redirect

```
curl -I http://app.example.invalid
curl -I https://app.example.invalid
```

Expect redirect or secure connection—not a permanent plain HTTP production path.

Theory 9 — Internal TLS without public ACME

```
# Conceptual lab path - exact options differ by proxy:
# - generate mkcert/local CA on admin machine
# - place cert/key via sops-nix into /run/secrets
# - point proxy ssl_certificate paths at those files
```

Document production delta: “In prod we would use ACME DNS-01 / HTTP-01; lab uses internal CA.”

Worked example — ports.md dual-write

Port	Service
80	ACME HTTP-01 / redirect
443	HTTPS proxy
8080	localhost backend only

docs/proxy.md and docs/ports.md must agree.

Exercises

Exercise 1 — Pick proxy and document

Write docs/proxy.md with Caddy or Nginx and domain plan (public / LAN / self-signed).

Exercise 2 — Localhost backend

Add declarative backend on 127.0.0.1:8080. Prove:

```
curl -sS http://127.0.0.1:8080/ | head
```

From another machine, connection should fail if not firewalled open and not public-bound.

Exercise 3 — Enable proxy module

Import proxy module; open 80/443; rebuild.

```
sudo nixos-rebuild switch --flake .#lab
```

Exercise 4 — TLS path

If public DNS: complete ACME; `curl -I https://your.domain`.

If not: configure self-signed or internal CA approach; `curl -k` for lab proof; document how production would differ.

Exercise 5 — Headers and hygiene (light)

Enable recommended TLS/proxy settings (nginx flags above, or Caddy defaults). Confirm you do not open backend port publicly.

Exercise 6 — Break/fix

Stop backend; observe 502; start backend; recover. Journal the unit names.

Exercise 7 — ports.md + proxy.md dual-write

Ensure both docs agree on engine, public ports, backend, TLS mode.

Exercise 8 — Certificate expiry awareness

```
echo | openssl s_client -connect 127.0.0.1:443 -servername app.example.invalid 2>/dev/null |
```

Exercise 9 — ss audit

```
ss -tlnp | grep -E ':80|:443|:8080'
```

Confirm 8080 is 127.0.0.1 only.

Exercise 10 — ACME journal

```
journalctl -u acme-app.example.invalid.service -b --no-pager 2>/dev/null | tail -40 || true  
journalctl -u caddy -b --no-pager | tail -40
```

Exercise 11 — Staging note

If you hit LE rate limits, document staging usage for next attempt.

Exercise 12 — WebSocket flag (if needed)

If your app needs websockets, confirm proxy config (`proxyWebsockets` or Caddy defaults).

Exercise 13 — Single proxy enforcement

```
# ensure only one of caddy/nginx enabled  
systemctl is-active caddy nginx 2>/dev/null || true
```

Exercise 14 — Secret path for DNS-01 (plan)

If using DNS-01, name the sops secret key in docs (no value).

Exercise 15 — Smoke from remote

From another machine (or phone on data): hit `https://...` if public. Record result.

Symptom / mistake	What to do
-------------------	------------

Common gotchas

Symptom / mistake	What to do
LE rate limited	Use staging; wait; fix DNS first
Both Caddy and Nginx on 443	Pick one
Backend on 0.0.0.0:8080	Bind localhost
Secrets for DNS-01 in git	sops
IPv6 DNS mismatch	Align AAAA or disable
Forgot port 80 for HTTP-01	Open 80 or use DNS-01
502 ignored	Fix backend unit first
Certs wiped with <code>/var</code>	Backup ACME state
docs disagree	Dual-write exercise

Checkpoint

- ☐ One proxy chosen and documented
- ☐ Backend on localhost
- ☐ 80/443 in firewall intentionally
- ☐ HTTPS or documented lab TLS alternative works
- ☐ `docs/ports.md` updated
- ☐ 502 break/fix once
- ☐ 8080 not world-open
- ☐ Cert dates inspected once

Journal (optional)

Write three personal gotchas before continuing.

A data service pattern

A data service pattern

Goal: Run **one** data service—**PostgreSQL** by default—declaratively on NixOS: module options, state directories, app role/auth via secrets paths, and a written backup story (even if automation is minimal).

! Important

DB passwords go through **sops-nix paths**, not Nix strings. Examples use placeholders and path references only.

Why this chapter exists

Application servers without state are easy. Real systems need **durable data**:

Concern	NixOS angle
Package + unit	<code>services.postgresql</code> module
State location	<code>services.postgresql.dataDir</code> / ensure on persistent disk
Auth	peer/scram; password files from secrets
Network exposure	localhost first (firewall discipline)
Backups	Not automatic just because Nix is declarative

Nix rebuilds the **software and config**; it does not magically version your rows.

Theory 1 — One DB only (pattern depth)

Depth > zoo. We use PostgreSQL. If you already run another engine at work, the pattern transfers: module → state dir → credentials → firewall → backups.

Theory 2 — Minimal PostgreSQL module

```
# modules/services/postgres.nix
{ config, pkgs, ... }:
{
  services.postgresql = {
    enable = true;
    package = pkgs.postgresql_16; # pick a version available on 26.05 pin
    ensureDatabases = [ "labapp" ];
    ensureUsers = [
      {
        name = "labapp";
        ensureDBOwnership = true;
      }
    ];
    settings = {
      # listen_addresses = "localhost"; # confirm option name on your version
    };
  };

  # Do NOT open 5432 on the host firewall for world access in the default lab.
}
```

Verify option names against search.nixos.org for 26.05.

Theory 3 — State directories and disk

PostgreSQL data lives **outside** the Nix store (mutable).

Question	Answer you must know
Where is <code>dataDir</code> ?	Document path
Is it on the disk that survives reinstall?	Ops/impermanence later
What happens on rollback of <i>system</i> generation?	Software rolls back; data may not

Impermanence hosts need explicit persistence for DB dirs—note the footgun now.

```
sudo -u postgres psql -c 'SHOW data_directory;'
```

Theory 4 — Credentials

Patterns:

1. **Peer auth** for local system users (good for admin)
2. **SCRAM password** for app users—password from sops at runtime
3. **Socket-only** access for local apps

Sketch with sops (illustrative—align with module capabilities):

```
sops.secrets."postgres/labapp_password" = {  
  owner = "postgres";  
  mode = "0400";  
};  
  
# Prefer module-supported passwordFile style options when available.  
# If you script role passwords, run a oneshot that reads the secret path  
# and does not print it.
```

Never:

```
# BAD  
# services.postgresql.ensureUsers = [{ name = "labapp"; password = "secret"; }];
```

Theory 5 — App connection story

App config should receive:

- Host: 127.0.0.1 or socket
- DB name: labapp
- User: labapp
- Password: from env file populated at activation from sops

Your TLS front door proxy does **not** replace DB auth. Backend talks to DB on localhost; users talk to proxy on 443.

Theory 6 — Backup thought (required cognitive work)

Even a one-node lab needs a plan:

Element	Example
What	<code>pg_dump</code> of <code>labapp</code>
When	nightly timer (system or user)
Where	<code>/var/backups/postgres/</code> on another disk or remote
Encryption	restic/age to remote (secrets via sops)
Restore test	once per quarter / before capstone

```
sudo -u postgres pg_dump labapp > /tmp/labapp-dump.sql
# store off-box in real life; scrub /tmp later
```

Theory 7 — Upgrades

Postgres major upgrades are **not** “change package attr and forget.” They need dump/restore or `pg_upgrade` procedures. Pin a major version intentionally; schedule upgrades.

Theory 8 — `ensureDatabases` vs migrations

`ensureDatabases` / `ensureUsers` create **empty** roles/DBs. Application schema is **not** Nix’s job by default.

Layer	Owner
Package + unit + listen policy	NixOS module
Roles/DBs existence	<code>ensure*</code> options / oneshots
Schema / migrations	App tooling (flyway, django, ...)
Data	Backups

Theory 9 — Socket vs TCP local connections

Postgres often exposes a Unix socket under `/run/postgresql`. Local admin via `sudo -u postgres psql` uses peer auth. Apps may use:

```
host=127.0.0.1 port=5432
# or host=/run/postgresql
```

Pick one, document in `docs/data-service.md`, match app secret env.

Worked example — Service + docs

```
modules/services/postgres.nix
docs/data-service.md
- version
- dataDir
- DB/user names
- auth method
- backup sketch
- restore sketch
```

Worked example — Timer sketch for dumps (optional)

```
{ pkgs, ... }:
{
  systemd.services.labapp-pg-dump = {
    serviceConfig = {
      Type = "oneshot";
      User = "postgres";
      ExecStart = "${pkgs.bash}/bin/bash -c '${pkgs.postgresql}/bin/pg_dump labapp | ${pkgs.
    };
  };
  # + timer + StateDirectory/tmpfiles for /var/backups
}
```

Wire carefully (permissions, disk space). Even a commented sketch in the flake beats “we’ll back up later.”

Exercises

Exercise 1 — Enable PostgreSQL

Add module; rebuild; verify unit:

```
sudo systemctl status postgresql
sudo -u postgres psql -c '\l'
```

Exercise 2 — Database and role

Confirm labapp DB and user via `ensure*` options. Connect:

```
sudo -u postgres psql -d labapp -c 'SELECT 1;'
```

Exercise 3 — Wire a secret password path

Create sops secret placeholder for app password. Configure the safest supported method for your module version. Prove app user can authenticate **without** password living in the flake as plain-text.

Exercise 4 — Network posture

```
ss -tulpn | grep 5432 || true
```

Confirm listening on localhost (or document intentional LAN bind). Ensure firewall does not expose 5432 publicly.

Update `docs/ports.md`.

Exercise 5 — Manual dump

```
sudo -u postgres pg_dump labapp | head
```

Write restore commands in `docs/data-service.md`.

Exercise 6 — Failure narrative

Answer:

1. System generation rollback after a bad app deploy—does data revert?
2. Disk failure—what is lost without backups?
3. Secret rotation steps for DB password

Exercise 7 — Schema independence proof

```
sudo -u postgres psql -d labapp -c "CREATE TABLE lab_demo(id int);"  
sudo -u postgres psql -d labapp -c '\dt'  
sudo nixos-rebuild switch --flake .#lab  
sudo -u postgres psql -d labapp -c '\dt'
```

Table should still exist—state generation.

Exercise 8 — Document restore dry-run

```
# restore sketch
# sudo -u postgres psql -d labapp < labapp-dump.sql
```

Exercise 9 — dataDir documentation

```
sudo -u postgres psql -c 'SHOW data_directory;'
```

Record in docs.

Exercise 10 — Peer auth admin path

Document how you admin without password in git (`sudo -u postgres psql`).

Exercise 11 — Package pin note

Write which `postgresql_*` package attr you chose and why you will not casual-bump majors.

Exercise 12 — Backup location permissions

```
sudo mkdir -p /var/backups/postgres
# ownership suitable for dump user
```

Exercise 13 — App connection string template

Write a template with `path` reference to password file—not the password.

Exercise 14 — ports.md final for DB

Confirm 5432 is localhost-only in docs and reality.

Symptom / mistake	What to do
-------------------	------------

Common gotchas

Symptom / mistake	What to do
Data in store myth	Data is on mutable disk
Open 5432 to world	Bind localhost; close firewall
Password in nix	sops path / peer auth
Major version bump casual	Plan dump/restore
No backup thought	Exercises 5–6 required
Proxying Postgres over HTTP	Wrong layer; don't
ensure* expected to migrate schema	Use app migrations
Rollback “restored” DB	Generations data
Dump committed with secrets	Scrub; don't commit dumps

Checkpoint

- ☐ PostgreSQL enabled declaratively
- ☐ App DB + user exist
- ☐ Auth without plaintext secret in git
- ☐ Localhost exposure default
- ☐ Manual dump performed once
- ☐ Backup/restore story written
- ☐ ports.md updated
- ☐ Schema-vs-generation understood

Journal (optional)

Write three personal gotchas before continuing.

NixOS containers

NixOS containers

Goal: Understand **isolation options** on NixOS—especially declarative **containers.* (nspawn-style)** and light OCI awareness—and run **one** declarative container (or clearly document why your lab uses a pure NixOS service instead).

i Note

This chapter is **not** “Docker first.” Prefer a native NixOS module when it exists. Containers help for upstream images, multi-version isolation, or hostile workloads—not as a default personality. Pin remains **NixOS 26.05**.

Why this chapter exists

Ecosystem pressure says “everything in containers.” NixOS offers stronger default stories for many services: one store, one firewall story, one secrets story. Still, you need literacy so you can refuse containers for good reasons—or adopt them deliberately.

Tool	Isolation style
NixOS systemd services	Same host, units, store
containers.<name>	systemd-nspawn guest with NixOS config
OCI (Podman/Docker)	Image-based; next chapter
VMs / k3s	Heavier; optional k3s chapter

Theory 1 — When containers help on NixOS

Use container	Prefer native module
Vendor only ships image	PostgreSQL, Caddy, Prometheus in nixpkgs
Need older/newer app than nixpkgs	Package it when feasible
Separate failure domain lightly	One more systemd service is enough
Multi-tenant sketch	Single-user lab
Hostile untrusted binary	Still not a full security boundary—see Theory 4

Decision micro-script

```
Is there a maintained NixOS module?
yes → use it
no  → Can I package it?
      yes → package + native service
      no  → OCI (next chapter) or nspawn NixOS guest (this chapter)
```

Theory 2 — containers.<name> (NixOS nspawn)

NixOS can define ephemeral/declarative containers. The guest is a **nested NixOS module graph**—not a Dockerfile.

```
# modules/services/demo-container.nix
{
  containers.demo = {
    autoStart = true;
    privateNetwork = true;
    hostAddress = "192.168.100.1";
    localAddress = "192.168.100.2";
    config = { config, pkgs, ... }: {
      system.stateVersion = "26.05";
      networking.firewall.allowedTCPPorts = [ 80 ];
      services.nginx = {
        enable = true;
        virtualHosts."localhost" = {
          root = pkgs.writeTextDir "index.html" "<h1>container</h1>";
        };
      };
    };
  };
};
```

Option idea	Meaning
<code>autoStart</code>	Start at boot
<code>privateNetwork</code>	Separate network namespace
<code>hostAddress</code> / <code>localAddress</code>	veth pair addressing
<code>bindMounts</code>	Host paths into guest
<code>config</code>	Nested NixOS module system
<code>ephemeral</code>	Lose rootfs state on stop (when supported)

Exact networking options evolve—verify against **26.05** options docs if attributes differ.

Pros / cons

Pros	Cons
Declarative guest config	Nested complexity / longer eval
Shares host store often	Not full VM isolation
Good teaching tool	Debug requires container tooling
Rebuild updates guest like host	Nested <code>stateVersion</code> still required

```
sudo machinectl list
sudo nixos-container status demo 2>/dev/null || true
sudo nixos-container root-login demo 2>/dev/null || true
```

Theory 3 — OCI awareness (without next-chapter depth)

OCI images: layers + config + entrypoint. On NixOS you may:

- Pull images with Podman/Docker (next chapter)
- Build images from Nix (`dockerTools`—packaging awareness later)
- Run rootless vs rootful

An **image is not a NixOS module**. It is a blob with its own update story, CVE surface, and entrypoint that may ignore your host hardening.

NixOS guest (<code>containers.*</code>)	OCI image
Config is Nix	Config is image + run flags
Updates via host rebuild	Updates via pull/tag discipline
Same module options as host	Ad-hoc env vars / volumes
Fits flake locks naturally	Needs digest pins for honesty

Theory 4 — Security boundaries (honest)

Boundary	Strength
Process on host	Weak isolation
systemd hardening (<code>ProtectSystem</code> , ...)	Better, still host kernel
nspawn container	Stronger namespaces
OCI with user namespaces	Better if configured; easy to misconfigure

Boundary	Strength
VM	Stronger (hardware virt)
Separate machine	Operational isolation

Do not claim “container = secure” without profile hardening and a patch process for images.

Hardening checklist (guest)

- Minimal packages in guest
- Firewall on guest *and* publish policy on host
- No unnecessary bind mounts of / or secret dirs
- No privileged flags unless justified in writing

Theory 5 — State and containers

Pattern	Use
Ephemeral guest	Stateless demos, rebuild-friendly
Bind <code>/var/lib/app</code> from host	Persist data; backup host path
Named volume (OCI)	Next chapter territory
Store-only content	Ideal for pure static sites

Mounting database dirs into random containers without care is a good way to corrupt data.

Theory 6 — Relation to proxy / firewall

```
[client] → host:443 (caddy/nginx)
          ↓
        192.168.100.2:80 (container)
```

Every publish updates `docs/ports.md`.

Mistake	Fix
Open guest port on public interface	Proxy only; firewall deny direct
Forget <code>ports.md</code>	Fix immediately
Double NAT confusion	Draw the path once

Theory 7 — Rebuild UX and nested eval

```
sudo nixos-rebuild build --flake .#lab -L
```

Friction vs native service	Why
Longer eval	Nested module system
Harder journal navigation	Machine names / nspawn units
Networking options maze	Worth a diagram

Theory 8 — Feature flag for demos

```
{ config, lib, ... }:
let cfg = config.myLab.demoContainer; in {
  options.myLab.demoContainer.enable = lib.mkEnableOption "demo nspawn container";
  config = lib.mkIf cfg.enable {
    containers.demo = { /* ... */ };
  };
}
```

Default off on constrained hosts.

Worked example — Prefer native, demo container optional

Path A (valid): skip long-lived containers; write a decision record “native modules suffice for the service slice.”

Path B: run the `containers.demo` nginx guest and curl via host networking setup you chose.

Both paths require **written judgment**. Path A is not a skip of the chapter—it is an evidence-backed refusal.

Exercises

Exercise 1 — Decision record

In `docs/isolation.md`:

```
# Isolation choices

Service slice:
- proxy: native module
- postgres: native module
- container: yes/no because ...

Boundary strength needed:
Threat model (1 paragraph):
When we would revisit containers:
```

Exercise 2 — Explore host tools

```
command -v nixos-container || true
command -v machinectl || true
systemctl list-units 'systemd-nspawn@*' 2>/dev/null | head || true
```

Exercise 3 — One declarative container or dry config

Implement `containers.demo` or write the full Nix config with `enable false` if you truly cannot run `nspawn`. Prefer running when possible.

```
sudo nixos-rebuild switch --flake .#lab
sudo nixos-container list 2>/dev/null || sudo machinectl list
```

Exercise 4 — Reach the guest service

Curl from host to guest IP or port. Document the path in `docs/isolation.md`.

```
curl -sS http://192.168.100.2/ | head
```

Exercise 5 — Compare rebuild UX

Change guest `index.html` content via Nix; rebuild; note friction vs native `services.nginx` on host.

Exercise 6 — When you would use OCI instead

Write three bullets for the Podman chapter: cases where an upstream image beats `nspawn` NixOS guest.

Exercise 7 — Firewall / ports doc

Update docs/ports.md for any published path. If Path A, document that no extra ports were opened.

Exercise 8 — Nested stateVersion

Confirm guest sets `system.stateVersion = "26.05"` (or your real pin).

Exercise 9 — Journal path

```
journalctl -M demo -b --no-pager 2>/dev/null | tail || true
```

Exercise 10 — Bind mount caution note

Write two sentences: what you would never bind-mount into a guest.

Exercise 11 — Disable cleanly

If demo only, disable container; rebuild; confirm resources freed.

Exercise 12 — Eval time measurement

```
time nixos-rebuild build --flake .#lab
```

Note cost of nested modules.

Exercise 13 — Diagram

ASCII diagram of client → proxy → container (or N/A for Path A).

Exercise 14 — Threat model honesty

One paragraph: nspawn is not a VM; when you would use a real VM instead.

Common gotchas

Symptom / mistake	What to do
Nested <code>stateVersion</code> missing	Set in guest config
Networking black hole	Start simpler; read options
Assuming Docker API	That's the Podman chapter
Container as default for Postgres	Prefer native module
Forgotten port documentation	Update ports.md
Nested eval errors	<code>nixos-rebuild build</code> ; read trace
Bind-mounting secrets world-readable	Fix modes; sops on host only
Expecting VM-level isolation	Adjust threat model
Path A without write-up	Decision record required

Checkpoint

- ☐ Isolation decision record written
- ☐ Understand nspawn-style `containers.*` vs OCI
- ☐ One container ran **or** justified skip with config draft
- ☐ Networking/ports documented
- ☐ Comparison note vs native services
- ☐ Preview of OCI use cases written
- ☐ Nested stateVersion correct if enabled
- ☐ Honest security boundary language

Journal (optional)

Write three personal gotchas before continuing.

Podman and Docker patterns

Podman and Docker patterns

Goal: Run **OCI containers** on NixOS via **declarative modules** (prefer **Podman**), understand rootless vs rootful trade-offs, and compare honestly to NixOS nspawn—without turning the host into an imperative `docker run` snowflake.

Why this chapter exists

The previous chapter covered NixOS-native containers. The industry default many colleagues know is **Docker/Podman + images**. On NixOS you should still:

1. Prefer a nixpkgs service module when it exists
2. When you need an image, declare it in the flake/host modules
3. Track ports, volumes, and update cadence

Theory 1 — Podman vs Docker on NixOS

Engine	Notes
Podman	Daemonless roots; good rootless story; often preferred on NixOS labs
Docker	Familiar Compose ecosystem; <code>virtualisation.docker</code> module

House suggestion: Podman unless you must match a Docker-only workflow.

```
# modules/services/podman.nix
{
  virtualisation.podman = {
    enable = true;
    dockerCompat = true; # optional: docker CLI alias - document if used
  };
}
```

Docker alternative sketch:

```
{
  virtualisation.docker.enable = true;
  # users.users.alice.extraGroups = [ "docker" ]; # rootful socket risk
}
```

`docker` group membership is effectively **near-root**. Prefer rootless or rootful with care.

Theory 2 — Rootless vs rootful

Mode	Pros	Cons
Rootless	Better default security	Port <1024 hard; storage quirks
Rootful	Simpler networking/ports	Higher blast radius

Rootless Podman for user workloads; rootful only when needed—or put proxy on host and map high ports.

Theory 3 — Declarative containers (OCI)

Common patterns include `virtualisation.oci-containers.containers` (backend `docker`/`podman`).

```
# modules/services/oci-hello.nix
{
  virtualisation.oci-containers = {
    backend = "podman";
    containers = {
      hello-lab = {
        image = "docker.io/library/nginx:alpine";
        ports = [ "127.0.0.1:8081:80" ];
        # volumes = [ "/var/lib/hello-lab:/usr/share/nginx/html:ro" ];
        autoStart = true;
      };
    };
  };
}
```

Field	Discipline
<code>image</code>	Pin tags carefully; digests better for prod

Field	Discipline
ports	Prefer 127.0.0.1: binds
volumes	Host paths must be managed (tmpfiles/persist)
environment	No secrets in plaintext —use secret files / sops-generated env

Exact option schema: confirm on search.nixos.org for 26.05.

Theory 4 — Images and supply chain (awareness)

Practice	Why
Pin by digest	Tags move
Prefer known registries	Reduce surprise
Update deliberately	Same as flake lock discipline
Scan when available	Optional tooling

Nix can *build* images from packages (`dockerTools`)—packaging mindset later. Here: consume one official image declaratively.

```
# Prefer tags you understand; better: digest pins when available
image = "docker.io/library/nginx:1.27";
# image = "nginx@sha256:...";
```

Floating `:latest` is the OCI equivalent of an unlocked flake input.

Theory 5 — Compare to nspawn

Dimension	nspawn <code>containers.*</code>	OCI Podman
Guest config	NixOS modules	Image + env/cmd
Updates	nixpkgs guest	Image pulls
Good for	Nix-first isolation	Vendor images
Store integration	Strong	Looser

Write your own two-sentence preference for the service slice.

Theory 6 — Firewall and proxy integration

```
clients → host :443 Caddy/Nginx → 127.0.0.1:8081 → OCI nginx
```

Do not publish OCI ports on 0.0.0.0 unless docs/ports.md says so and auth exists.

```
# proxy snippet idea
# reverse_proxy 127.0.0.1:8081
```

Theory 7 — Stateful volumes

If the container needs state:

1. Create host directory with tmpfiles or activation
 2. Bind mount
 3. Include in backup story
 4. Never put secrets into world-readable volume without mode care
-

Theory 8 — Rootless paths and user namespaces

Rootless Podman stores containers under the user data dir. On NixOS:

Concern	Note
subuid/subgid	User must be mapped
Lingering	User services may need lingering for boot-start
Ports <1024	Often need rootful or redirect

```
podman info | head
id alice # check subuids if rootless fails
```

Theory 9 — Imperative snowflakes vs flake

```
podman run --rm -p 127.0.0.1:8082:80 docker.io/library/nginx:alpine
# stop/remove; do NOT leave unmanaged containers as the real setup
```

Imperative is fine for five-minute experiments. The rebuildable truth lives in `virtualisation.oci-containers`.

Worked example — Full small stack slice

```
{ ... }:  
{  
  imports = [ ./podman.nix ];  
  
  virtualisation.oci-containers.backend = "podman";  
  virtualisation.oci-containers.containers.webish = {  
    image = "docker.io/library/nginx:1.27";  
    ports = [ "127.0.0.1:8081:80" ];  
    autoStart = true;  
  };  
}
```

```
curl -sS http://127.0.0.1:8081/ | head  
podman ps
```

Exercises

Exercise 1 — Enable Podman (or Docker)

```
# add module, rebuild  
podman info 2>/dev/null || docker info 2>/dev/null
```

Document rootless vs rootful choice.

Exercise 2 — Declarative OCI container

Define one container with `localhost` port publish. Rebuild. Verify:


```
podman ps  
curl -sS http://127.0.0.1:8081/ | head
```

Exercise 3 — Imperative contrast (then undo)

Short imperative run; stop/remove; journal why the flake definition wins.

Exercise 4 — Wire to proxy (optional but valuable)

Add a reverse_proxy path to the OCI port. Update ports.md.

Exercise 5 — Secret anti-pattern check

Confirm no `environment = { PASSWORD = "..."} in nix`. If the image needs a password, plan sops-backed env file mount.

Exercise 6 — Comparison table filled

Complete nspawn vs OCI table in docs/isolation.md with *your* results.

Exercise 7 — Volume inventory

```
podman volume ls 2>/dev/null || docker volume ls 2>/dev/null || true
```

Map each volume to a backup/persist decision.

Exercise 8 — Update story one-pager

Write five lines: how you will refresh images, who tests, whether flake lock/image digest changes ship together.

Exercise 9 — autoStart reboot proof

Reboot or restart the oci unit; confirm container returns.

Exercise 10 — Port conflict check

```
ss -tlnp | grep 8081
```

Avoid clashing with TLS chapter backend on 8080.

Exercise 11 — docker group risk note

If you used Docker + group, write why it is near-root.

Exercise 12 — Image pin upgrade

Change tag deliberately; rebuild; observe pull; document.

Exercise 13 — Disable cleanly

Remove container from config; rebuild; `podman ps clean`.

Exercise 14 — Isolation.md final

State house preference: native / nspawn / OCI for each service in the slice.

Common gotchas

Symptom / mistake	What to do
Port already in use	Conflict with TLS backend—choose free port
Image pull fails	Network/registry; mirror; pin
Rootless port 80 bind fails	Use high ports + host proxy
<code>docker</code> group casual	Avoid; understand privilege
State lost on recreate	Bind volume on host
Forgot <code>autoStart</code>	Container missing after reboot
Secrets in <code>environment=</code>	sops file mounts
<code>:latest</code> forever	Pin tag/digest
Imperative left running	Remove snowflake

Checkpoint

- ☐ Podman or Docker enabled via NixOS module
- ☐ One OCI container declarative in flake
- ☐ Localhost publish; ports documented
- ☐ Imperative snowflake cleaned up
- ☐ Isolation docs updated vs nspawn
- ☐ No plaintext secrets in container env

- ☐ Image update story written
- ☐ autoStart verified or documented

Journal (optional)

Write three personal gotchas before continuing.

Observability patterns

Observability patterns

Goal: Add an **observability pattern** for the lab slice—either **Prometheus + Grafana** (or Prometheus alone) **or** a deliberate **logs-first** minimal stack—so one service is inspectable without SSH archaeology alone.

Why this chapter exists

Services without observability fail quietly:

- Disk fills
- Proxy 502s for hours
- Postgres connections exhaust
- ACME renewals fail

You do not need a full SRE platform. You need a **repeatable pattern**: what you measure, where it lives, who can access it, and how it is firewalled.

Theory 1 — Three pillars (light)

Pillar	Lab approach
Logs	journald (journalctl) first-class
Metrics	Prometheus node exporter + app exporters optional
Traces	Optional later; not required

Pick a **primary**: metrics stack **or** logs discipline. Doing both shallowly is fine; doing neither is not.

Theory 2 — Logs-first pattern (valid)

If resources are tight:

```
journalctl -u caddy -b --no-pager | tail
journalctl -u postgresql -b --no-pager | tail
journalctl -p err -b --no-pager | tail -50
```

Declarative helpers:

```
{
  services.journald.extraConfig = ''
    SystemMaxUse=500M
  '';
}
```

Document a **debug runbook** for proxy, DB, and sops activation units. That can satisfy the pattern if you explicitly choose logs-first in `docs/observability.md`.

Theory 3 — Prometheus pattern

```
# modules/services/prometheus.nix
{
  services.prometheus = {
    enable = true;
    port = 9090;
    listenAddress = "127.0.0.1";
    scrapeConfigs = [
      {
        job_name = "node";
        static_configs = [{
          targets = [ "127.0.0.1:9100" ];
        }];
      }
    ];
  };

  services.prometheus.exporters.node = {
    enable = true;
    port = 9100;
    listenAddress = "127.0.0.1";
    openFirewall = false;
  };
}
```

Verify option names for 26.05. Access via SSH tunnel:

```
ssh -L 9090:127.0.0.1:9090 lab
# browse localhost:9090
```

Or reverse-proxy with **authentication**—never raw public Prometheus without auth.

Theory 4 — Grafana (optional layer)

```
{
  services.grafana = {
    enable = true;
    settings = {
      server = {
        http_addr = "127.0.0.1";
        http_port = 3000;
      };
    };
  };
}
```

Initial admin password: use sops / first-boot procedures—**not** plaintext in git. Prefer password-file options when available.

Theory 5 — What to scrape for *your* slice

Target	Why
node exporter	Host health
Caddy/Nginx metrics	If exporter/module available
Postgres exporter	Optional; more moving parts
Blackbox / probes	Optional

Minimum bar: node exporter + Prometheus **or** excellent journald runbooks + one automated log check.

Sample queries:

```
up
node_memory_MemAvailable_bytes
node_load1
```

Theory 6 — Firewall and multi-tenant risk

Metrics endpoints often expose sensitive operational detail. Defaults:

- Listen on 127.0.0.1
 - SSH tunnel or authenticated proxy
 - Update docs/ports.md
-

Theory 7 — Alerting (awareness only)

Alertmanager is real production work. Optional: record “what would page me.” Do not block on paging infrastructure.

Theory 8 — Retention and disk

Metrics and journal logs fill disks.

```
services.journald.extraConfig = ''  
    SystemMaxUse=500M  
'';
```

Prometheus local TSDB also needs a size budget. A lab that fills the disk mid-capstone is a self-own.

```
journalctl --disk-usage  
du -sh /var/lib/prometheus2 2>/dev/null || du -sh /var/lib/prometheus 2>/dev/null || true
```

Theory 9 — Authn in front of Grafana

Never expose Grafana 3000 to the world bare. Patterns:

1. Localhost + SSH tunnel
 2. Reverse proxy + SSO/basic auth
 3. VPN-only interface
-

Worked example — Minimal metrics module set

```
modules/services/observability.nix
- prometheus listen localhost
- node exporter localhost
docs/observability.md
- how to tunnel
- example PromQL: up, node_load1
- log commands for proxy/db
```

Worked example — Runbook skeleton

```
# docs/runbooks/service-down.md

## Proxy 502
journalctl -u caddy -b --no-pager | tail -100
systemctl status lab-backend

## Postgres
journalctl -u postgresql -b --no-pager | tail -100
sudo -u postgres psql -c 'SELECT 1;'

## Secrets activation
journalctl -b --no-pager | rg -i sops | tail
```

Exercises

Exercise 1 — Choose path

Write top of docs/observability.md:

```
Primary path: metrics | logs-first | both
Rationale: ...
```

Exercise 2A — Metrics path

Enable Prometheus + node exporter (Grafana optional). Rebuild.

```
curl -sS http://127.0.0.1:9090/-/ready
curl -sS http://127.0.0.1:9100/metrics | head
```

Exercise 2B — Logs-first path

Create a runbook with copy-pasteable `journalctl` lines for:

1. Proxy
2. PostgreSQL
3. sops-related activation
4. OCI container unit if any

Add `journal` retention config if useful.

Exercise 3 — One service deep dive

Pick proxy **or** postgres **or** node. Answer:

- How do I know it is up?
- How do I see last error?
- What metric or log line changes when I break it deliberately?

Perform a controlled break/fix (stop unit; observe; start).

Exercise 4 — Access path

Document SSH tunnel **or** authenticated proxy. Prove you can view metrics/logs without opening world-readable admin UIs.

Exercise 5 — Update ports and isolation docs

Mark 9090/9100/3000 as localhost-only (if used).

Exercise 6 — Capstone foreshadow

List three signals you would want before a wipe/rebuild proof (e.g. free disk, proxy up, DB accepting connections).

Exercise 7 — One dashboard or one journal recipe

Either export a minimal Grafana dashboard path **or** ship `docs/runbooks/service-down.md`.

Exercise 8 — Disk use snapshot

```
journalctl --disk-usage  
du -sh /var/lib/prometheus2 2>/dev/null || true
```

Record numbers in observability notes.

Exercise 9 — PromQL basics (metrics path)

In Prometheus UI: run up and one node metric. Screenshot optional; notes required.

Exercise 10 — Password hygiene for Grafana

If Grafana enabled, confirm no admin password in git.

Exercise 11 — Unit list for the slice

```
systemctl list-units --type=service --state=running | rg -i 'caddy|nginx|postgres|prometheus'
```

Exercise 12 — Error priority journal

```
journalctl -p err -b --no-pager | tail -30
```

Exercise 13 — Tunnel muscle memory

```
ssh -L 9090:127.0.0.1:9090 lab
```

Exercise 14 — Docs cross-link

From docs/observability.md, link to ports.md and proxy.md paths (relative notes).

Symptom / mistake	What to do
-------------------	------------

Common gotchas

Symptom / mistake	What to do
Grafana/Prometheus on 0.0.0.0 public	Bind localhost; add auth
Scrape target wrong	Check listen addresses
Huge cardinality later	Keep lab exporters few
Logs-only with no runbook	Write runbook
Password in flake for Grafana	sops / setup procedure
Thinking Nix replaces monitoring	It does not
Disk full of journals/TSDB	Cap retention
No break/fix	Do Exercise 3
Metrics without ports.md	Update docs

Checkpoint

- ☐ Observability path chosen and documented
- ☐ Metrics stack running **or** logs runbook complete
- ☐ One controlled break/fix observed
- ☐ Admin interfaces not world-open
- ☐ ports.md updated
- ☐ Signals list for future ops
- ☐ Disk usage noted
- ☐ Access path (tunnel/auth) documented

Journal (optional)

Write three personal gotchas before continuing.

k3s (optional awareness)

k3s (optional awareness)

Goal: Either stand up a **single-node k3s** for literacy **or skip cleanly** with a written decision—Kubernetes is not required for this part’s service slice.

! Important

If you do not need k8s for work or curiosity right now, **skip**. A forced k3s install can destabilize a small lab (CPU, disk, conceptual overload).

Why this chapter exists

Some learners finish a proxy + Postgres slice and ask: “Should everything move to Kubernetes?” On NixOS the honest answer is usually **no**—not for a one-node lab with Caddy + Postgres.

This chapter exists to:

1. Prevent FOMO-driven architecture
2. Give a **safe optional on-ramp** if you truly need k8s APIs
3. Practice **skipping** as a professional skill

Theory 1 — Decision rubric

Choose skip if...	Choose k3s lite if...
Service slice already works natively	You must learn kubectl for a job soon
Host is small on RAM/disk	You have 8–16 GB RAM lab resources
You have not finished secrets/proxy/DB solidly	Those foundations are already solid
Curiosity only, no time	You will actually deploy one manifest

Default recommendation: skip, write the decision, move on.

Theory 2 — What k3s is

k3s is a lightweight Kubernetes distribution. On NixOS:

```
# modules/services/k3s.nix - only if you opt in
{
  services.k3s = {
    enable = true;
    role = "server";
    # extraFlags = [ "--write-kubeconfig-mode=0644" ]; # understand security implications
  };
}
```

Options and defaults change—read current module docs for 26.05 before enabling.

Implications:

Area	Effect
Ports	API server, kubelet, etc.—firewall carefully
CPU/RAM	Non-trivial idle cost
Storage	Container images + etcd/sqlite data
Mental model	YAML/Helm vs NixOS modules

Theory 3 — NixOS vs k8s configuration

Concern	NixOS native	k3s
System packages	flake modules	images inside cluster
Host firewall	networking.firewall	+ k8s Services/NodePorts
Secrets	sops-nix on host	also k8s Secrets (different!)
Rollback	generations	cluster state is separate

Running k3s **does not replace** host flake discipline. You now have **two** control planes: NixOS + cluster.

Theory 4 — Single-node honesty

Single-node k3s is fine for learning APIs. It is **not**:

- Multi-AZ HA
 - A reason to abandon PostgreSQL module for “cloud native” on one VPS without backups
 - Automatically more secure
-

Theory 5 — Minimal literacy tasks (if enabled)

1. `kubectl get nodes`
 2. Deploy a tiny `Deployment` + `Service`
 3. Expose via host proxy or port-forward—not random `NodePort` to the world
 4. Delete the workload
 5. Document how to disable `services.k3s` and reclaim resources
-

Theory 6 — Skip path is first-class

Skipping is documented output:

```
# docs/k3s-decision.md
Decision: SKIP
Date: ...
Reasons:
- Service slice does not require k8s
- Host resources: ...
- Will revisit if: job requirement / multi-service mesh need
```

That file **passes** the chapter.

Workload	Prefer
----------	--------

Theory 7 — When NixOS modules beat k8s on a single node

Workload	Prefer
One reverse proxy + one DB	NixOS modules
Many identical microservices with k8s API dependency	k3s/k8s
GPU training cluster	Often specialized schedulers
Homelab “because CNCF”	Usually modules + compose/podman

NixOS is already a declarative control plane for **one machine**. k3s adds a second control plane—justify it.

Theory 8 — Resource floor honesty

Single-node k3s wants real RAM/CPU. On a 2 GB VM:

```
k3s + postgres + caddy + prometheus = thrash
```

Skip is the professional choice when resources are tight.

Theory 9 — Feature flag module

```
# modules/services/k3s.nix
{ config, lib, ... }:
let cfg = config.myLab.k3s; in {
  options.myLab.k3s.enable = lib.mkEnableOption "optional single-node k3s";
  config = lib.mkIf cfg.enable {
    services.k3s = {
      enable = true;
      role = "server";
      # extraFlags = [ "--disable traefik" ]; # if you keep host proxy as edge
    };
    # networking.firewall.allowedTCPPorts = [ 6443 ]; # admin API - lock down
  };
}
```

Default `enable = false` on the lab host.

Worked example — Disable / enable toggles

Prefer a role flag or simply do not import k3s module. Avoid half-enabled clusters.

Worked example — Tiny workload (if enabled)

```
# manifests/hello.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello
spec:
  replicas: 1
  selector:
    matchLabels: { app: hello }
  template:
    metadata:
      labels: { app: hello }
    spec:
      containers:
        - name: hello
          image: docker.io/library/nginx:alpine
          ports:
            - containerPort: 80
```

```
kubectl apply -f manifests/hello.yaml
kubectl get pods
kubectl port-forward deploy/hello 8088:80
curl -sS http://127.0.0.1:8088/ | head
```

Exercises

Exercise S — Skip path (most readers)

1. Write `docs/k3s-decision.md` with SKIP + reasons
2. Skim what k3s would open (ports, disk) from docs—no install
3. List two future triggers to revisit
4. Commit and proceed

Time box: ~30 minutes. Do not guilt-install.

Exercise S2 — Skip artifact quality bar

Your skip note must include:

1. Decision (skip)
2. Resource or complexity reason
3. What you use instead (NixOS modules / podman)
4. Revisit criteria (“if job requires CKA lab...”)

A one-word “skipped” is not enough.

Exercise K1 — Opt-in: enable k3s

Only if rubric says go:

```
sudo nixos-rebuild switch --flake .#lab
sudo systemctl status k3s
sudo k3s kubectl get nodes
```

Kubeconfig location per module—follow module notes. Restrict access modes.

Exercise K2 — Tiny workload

Apply the hello Deployment; port-forward; curl; delete or keep intentionally.

Exercise K3 — Tear-down plan

Document:

```
kubectl delete -f manifests/hello.yaml
# disable services.k3s; rebuild; confirm resources freed
```

Update firewall if you opened API ports. Prefer API on localhost / VPN only.

Exercise K4 — Compare control planes

Write a short note: what still belongs in the NixOS flake vs what you put in the cluster.

Exercise K5 — API surface inventory (if enabled)

```
sudo k3s kubectl get nodes
sudo k3s kubectl get pods -A | head
ss -tlnp | grep -E '6443|10250' || true
```

Add every k3s port to docs/ports.md with “cluster-internal / admin-only” labels.

Exercise 6 — Resource estimate

Write RAM/disk numbers before and (if enabled) after k3s.

Exercise 7 — Secrets boundary note

One paragraph: host sops vs k8s Secrets—both needed if cluster is on.

Exercise 8 — Proxy interaction

If Traefik conflicts with Caddy/Nginx, disable one; document edge story.

Exercise 9 — Decision commit

```
git add docs/k3s-decision.md
git commit -m "docs: k3s optional decision recorded"
```

Exercise 10 — FOMO check

Write two sentences: what pressure almost made you install k3s, and why you did or did not.

Exercise 11 — Revisit calendar

If skip: pick a future review date or trigger (job change, multi-node need).

Exercise 12 — ports.md consistency

Whether skip or enable, ports.md should not claim k3s ports that do not exist.

Exercise 13 — Disable half-install

If you started and regret: disable module, rebuild, free disk images.

Exercise 14 — Part retrospective

List the service slice components you trust without k8s: proxy, DB, secrets, firewall, observability.

Common gotchas

Symptom / mistake	What to do
Guilt install before readiness	Use skip path
Opened k8s API to world	Close; bind/firewall
Disk full of images	Prune; disable k3s
Secrets only in k8s, host ignored	Host sops still matters
Two proxies fighting	One ingress story
Undocumented cluster	Decision doc mandatory either way
Thrashing small VM	Skip; free resources
“Everything in cluster”	Push back; host still owns kernel/disks

Checkpoint

If skipped

- ☐ docs/k3s-decision.md with SKIP + reasons
- ☐ Future revisit triggers listed
- ☐ No half-installed k3s units left
- ☐ ports.md consistent

If enabled

- ☐ Single-node ready; `kubect1 get nodes` works
- ☐ One workload applied and removed or still intentional
- ☐ API not casually public
- ☐ Disable/tear-down plan written
- ☐ ports.md updated
- ☐ Resource impact noted

Journal (optional)

Write three personal gotchas before continuing.

Part close-out

You should now have a **service and security spine**:

Capability	Chapter
Store vault	Secrets problem
Encrypt-in-git secrets	sops-nix (+ agenix compared)
Reduced attack surface	Hardening
Deny-by-default network	Firewall + ports.md
TLS edge	Reverse proxy
Durable data	PostgreSQL pattern
Isolation literacy	nspawn + OCI
Inspectability	Observability
k8s FOMO managed	Optional k3s decision

Later parts (packaging, ops/fleet, capstone) assume this discipline. If secrets still live as Nix strings or 5432 is world-open, fix that before scaling out.

Packaging

Packaging craft

Packaging craft

This part of the **standalone NixOS book** is organized by **topic** (normal book chapters), not by calendar day.

Read chapters in order within the part when learning; skip or skim freely when using the book as a reference.

Baseline: Nix **2.34.x**, flakes-first workflows, nixpkgs / NixOS **26.05**. Language builders, FOD hash fields, and CLI flags evolve—prefer patterns that match current nixpkgs examples when a helper renames an attribute.

What this part is for

You already run hosts, compose modules, and pin flakes. **Packaging craft** is how you:

1. Turn source into store paths with deliberate **stdenv** phases
2. Pin bytes with fixed-output derivations (FODs)
3. Use language ecosystems without cargo-culting random blogs
4. Patch and override without forking nixpkgs forever
5. Split outputs, debug sandboxes, navigate nixpkgs layout
6. Feed fleets with **binary caches** and **remote builders**
7. Speak honestly about **reproducibility**
8. (Elective) Run **ROS 2** via **nix-ros-overlay** without Ubuntu-deb lock-in

Without this part, “we use Nix” means only consuming **environment.systemPackages**. With it, you **produce** packages, own supply-chain pins, and debug CI the same way you debug a laptop.

Prerequisites

Skill	Where it came from
Derivations, store paths, closures	Concepts part
Flakes, <code>callPackage</code> , overlays (light)	Home and flake layout
Host consumption of packages	NixOS host part
Optional: secrets for cache tokens	Services and security

You do **not** need prior C packaging expertise. You do need willingness to read build logs.

Chapter map

Chapter	File	Focus
stdenv and build phases	01-stdenv-phases	genericBuild , hooks, inputs kinds
Fetchers and FODs	02-fetchers-fod	Hash workflow, trust, flake inputs vs src
Language ecosystems A	03-language-ecosystems-A	One deep language builder
Language ecosystems B	04-language-ecosystems-B	Second ecosystem; pattern transfer
Patching and overrides	05-patching-overrides	overrideAttrs , overlays, patches
Multiple outputs	06-multiple-outputs	out/dev/man , closure size
Debugging builds	07-debug-builds	Logs, keep-failed, breakpointHook
Navigating nixpkgs	08-nixpkgs-layout	by-name , PR-shaped packages
Binary caches	09-binary-caches	Substituters, Cachix/Attic/Harmonia, trust
Remote builders	10-remote-builders	Offload realization; features
Reproducibility	11-reproducibility	L0–L5 honesty, measurement
ROS 2 / nix-ros-overlay	12-ros2-nix-ros-overlay	Robotics underlays, flakes, caches, workspaces

Suggested linear path: **01** → **02** → **03** → **04** → **05** → **06** → **07** → **08**, then **09** **10** as a pair, finish with **11**.

Robotics elective: after overlays + caches literacy, take **12** (can also run from a Fedora dual-stack install).

Mental model — three layers of “a package”

```
Meta & policy (license, mainProgram, ...)
```

```
Build plan (phases, hooks, language helper)
```

```
Fixed inputs (src FOD, vendor/cargo/npm ...)
```

```
/nix/store/...-pname-version
```

Layer	Failure class	First tool
Inputs	Hash mismatch, floating tag	Prefetch / fakeHash
Build	Compile, link, install	-L, nix log
Meta / consume	Wrong bin name, missing wrap	nix run, path-info

stdenv is the spine

Almost every language helper is still `stdenv.mkDerivation` plus setup hooks:

```
buildGoModule / buildRustPackage / buildPythonApplication / buildNpmPackage
```

```
stdenv.mkDerivation (+ setup hooks)
```

```
unpack → patch → configure → build → check → install → fixup
```

If you cannot name the default phase order, every “mysterious” failure later is slower than it needs to be. The `stdenv` chapter is non-skippable for practitioners who only ever used `nix profile install`.

FODs vs normal builds

Kind	Network in sandbox?	Output path depends on
Normal	No (default)	Derivation inputs
Fixed-output	Yes, limited	Declared output hash

Package **src** and language dependency graphs are almost always FODs. Updating a hash is a **trust decision**, not a clerical annoyance—covered in the fetchers chapter and again under reproducibility.

Caches vs builders (preview)

These two chapters are confused constantly:

```
[dev] --build--> [builder] --push--> [cache] --substitute--> [fleet]
```

Mechanism	Moves	Needs existing path?
Binary cache	Results (NARs)	Yes
Remote builder	Work (realize drv)	No

Packaging craft produces the paths; ops and CI **distribute** them. Read both packaging chapters before wiring fleet CI.

Lab workspace conventions

Suggested roots (adjust freely):

```
~/lab/nixos/packaging/
  stdenv/      # shell + C toys
  fod/         # fetchurl / GitHub drills
  eco-a/       # primary language tool
  eco-b/       # second ecosystem
  overrides/   # patches + overlays
  multi-out/   # lib + app
  debug/       # intentional breaks
  by-name/     # nixpkgs-shaped layout
  cache/       # push/pull notes
  builders/    # remote build topology
  repro/       # REPRO.md + measurements
```

Each chapter’s lab can be a **separate flake** or packages inside one monorepo flake—prefer one flake with `packages.*` attrs if you already have a lab flake.

```
# Recurring commands
nix build .#attr -L
nix run .#attr
nix flake check
nix path-info -rS ./result | sort -k2 -n | tail
nix log .#attr
```

How chapters are structured

Most chapters use the same spine so you can scan as a reference:

1. **Goal** — what “done” looks like
 2. **Why it matters** — ops/packaging motivation
 3. **Theory** — models, tables, complete Nix snippets
 4. **Worked examples** — copyable, 26.05-shaped
 5. **Lab** — multi-step exercises
 6. **Pitfalls** — symptom → fix tables
 7. **Checkpoint** — self-review list
 8. **Commit** — suggested message (optional discipline)
-

Reading paths

Path A — “I need one internal CLI in nixpkgs style”

1. stdenv phases
2. Fetchers / FODs
3. Language ecosystems A (your language)

4. Debugging builds
5. nixpkgs layout (meta + by-name shape)
6. Reproducibility (write claims)

Path B — “CI is compiling the world”

1. Binary caches (trust + push)
2. Remote builders
3. Debugging builds (cache miss vs compile fail)
4. Reproducibility (L1/L2 measurements)

Path C — “We forked half of nixpkgs”

1. Patching and overrides
2. Multiple outputs (if closures bloated)
3. nixpkgs layout (upstream-shaped expressions)
4. Language ecosystems B (stop reinventing builders)

Glossary (part-local)

Term	Meaning
stdenv	Standard environment: toolchain + <code>genericBuild</code> driver
Phase	Named step (<code>buildPhase</code> , <code>installPhase</code> , ...)
Hook	<code>preInstall</code> / <code>postFixup</code> style glue around phases
FOD	Fixed-output derivation; hash pins content
SRI hash	<code>sha256-...</code> form preferred in modern nixpkgs
vendorHash / cargoHash / npmDepsHash	Language-dep FODs
overrideAttrs	Change derivation attributes after the fact
Overlay	Function <code>final: prev: { ... }</code> reshaping pkgs
Multiple outputs	Separate store paths (<code>out</code> , <code>dev</code> , ...)
Substituter	Binary cache endpoint Nix may pull from
Remote builder	Machine that realizes derivations for you
L1 / L2 repro	Same path on rebuild / across similar hosts

Quality bar for “done packaging”

A package is not “done” when it builds once on your laptop. Aim for:

Criterion	Check
Builds from flake lock	<code>nix build .#pkg -L</code>
Runnable entrypoint	<code>meta.mainProgram + nix run</code>
Hashes intentional	No <code>fakeHash</code> left committed
Logs readable	You can name the failing phase
Meta honest	license, description, platforms
Closure sane	<code>nix path-info -Sh</code> ; multi-out if library
Claim measured	REPRO note for anything you ship to others

Relationship to other parts

Part	Connection
Concepts	Derivations and store theory become muscle memory here
NixOS host	<code>environment.systemPackages</code> and modules <i>consume</i> packages
Home / flakes	Overlays, checks, lock discipline
Services / security	Cache tokens and signing keys are secrets
Ops and fleet	Deploy tools ship closures you learned to build
Internals	Evaluator, CA derivations, store daemon deepen later
Projects	Hands-on packaging drills reuse this part’s patterns

This book does **not** cross-link a parallel day-calendar series. Everything here is topic-organized library material.

Anti-patterns to avoid while studying

Anti-pattern	Prefer
Disable sandbox to “make it work”	Find undeclared dep or FOD
Branch name as <code>rev</code> forever	Tag or full commit SHA
Global overlay for a one-off flag	Local <code>overrideAttrs</code>

Anti-pattern	Prefer
Single \$out with headers + docs + bins for shared libs	Multiple outputs
“Fully reproducible” in README without measurement	L1/L2 claims with lock + system
Copying random default.nix without reading phases	stdenv + official language helper

Part checkpoint (before you leave packaging)

- ☐ Can list default stdenv phase order from memory
- ☐ Shipped at least one package with a real FOD hash workflow
- ☐ One language-builder package end-to-end (ecosystem A)
- ☐ Second ecosystem or transfer table completed (ecosystem B)
- ☐ Patch + overlay composition demonstrated
- ☐ Multi-output or closure inspection done deliberately
- ☐ Personal debug runbook exists
- ☐ Package shaped like nixpkgs (meta, preferably by-name layout)
- ☐ Cache trust model written (even if only cache.nixos.org)
- ☐ Know cache vs remote builder in one paragraph
- ☐ Honesty note on reproducibility for one attr

Suggested lab commit series

```
git commit -m "packaging: stdenv greet + addone"
git commit -m "packaging: FOD hash workflow"
git commit -m "packaging: language ecosystem A"
git commit -m "packaging: language ecosystem B"
git commit -m "packaging: patches and overlays"
git commit -m "packaging: multi-output lib + app"
git commit -m "packaging: debug runbook"
git commit -m "packaging: by-name shape + meta"
git commit -m "packaging: cache trust notes"
git commit -m "packaging: remote builder topology"
git commit -m "packaging: REPRO.md measurements"
```

How to use this part as reference

When something breaks in six months:

Symptom	Jump to
Unknown phase failure	Debugging builds; stdenv phases
hash mismatch	Fetchers and FODs
Language dep network in build	Ecosystems A/B
Need a CVE pin	Patching and overrides
Runtime closure huge	Multiple outputs
Where do I put this for upstream?	nixpkgs layout
CI rebuilds everything	Binary caches; remote builders
Manager asks “is it reproducible?”	Reproducibility

Next

Start with **stdenv and build phases**: name every default phase, package a shell tool and a tiny C tool, and break install on purpose so logs become familiar. Then pin bytes with FODs before you trust any “reproducible” claim.

stdenv and build phases

stdenv and build phases

Goal: Package a small C or shell tool with `stdenv.mkDerivation`, name every default phase, and know when to override hooks vs rewrite the whole build.

i Note

Baseline: Nix 2.34 / nixpkgs **26.05**, flakes-first. Language helpers (`buildGoModule`, ...) wrap the same lifecycle—master phases here first.

Why it matters

Most “how do I package X?” questions collapse into one skill: **drive `stdenv` phases deliberately**. Language helpers are thin wrappers on the same lifecycle. If you cannot read a failing phase log, every later packaging chapter will feel random.

You have already *consumed* packages on a host. Packaging craft is producing them: pure plans, sandboxed builds, store outputs you can reason about.

Theory 1 — What `stdenv.mkDerivation` actually is

A derivation is a pure build plan: inputs → sandbox → outputs under `/nix/store`. `stdenv.mkDerivation` fills that plan with a **bash driver** (`genericBuild`) and a default toolchain (compiler, coreutils, patch, make, ...).

```
{ stdenv, fetchurl, lib }:  
stdenv.mkDerivation rec {  
  pname = "hello";  
  version = "2.12.1";  
  src = fetchurl {  
    url = "mirror://gnu/hello/hello-${version}.tar.gz";  
    hash = "sha256-jZkUKv2SV28wsM18tCqNxoCZmLxdYH2Idh9RLibH2yA=";  
  };  
  meta = {  
    description = "GNU Hello prints a friendly greeting";  
    license = lib.licenses.gpl3Plus;  
    mainProgram = "hello";  
  };  
}
```

```
};
}
```

Concept	Meaning
pname / version	Human labels; feed name as <code>\${pname}-\${version}</code> by default
src	Input path (usually a fixed-output fetcher)
buildInputs	Host libraries linked into the result (runtime + link time on native)
nativeBuildInputs	Tools that run on the build machine (compilers, cmake, pkg-config)
propagated*	Rare; pollutes dependents—prefer explicit deps
meta	Description, license, platforms, mainProgram

buildInputs vs nativeBuildInputs (cross-aware)

Attr	Role	Cross example
nativeBuildInputs	Execute during build	pkg-config , cmake , host gcc when cross-compiling
buildInputs	Link / use as target libraries	openssl , zlib for the target system
depsBuildBuild	Advanced bootstrap	Tools for the build→build hop
depsBuildTarget	Cross edge cases	See nixpkgs cross docs

For native-only packaging you can get away with sloppy placement; **cross** and **static** punish it. Prefer correct classification from the first package you write.

Evaluation vs realization

```
nix eval / callPackage → derivation (.drv)
nix build / realize      → /nix/store/... outputs
```

Phase scripts run at **realization**. A typo in a phase string fails the build; a typo in attribute structure may fail at eval. Know which side you are on before staring at compiler errors that never ran.

Theory 2 — Default phase order

`genericBuild` runs roughly:

```

unpackPhase
patchPhase
updateAutotoolsGnuConfigScriptsPhase  # often invisible
configurePhase
buildPhase
checkPhase          # if doCheck = true
installPhase
fixupPhase
installCheckPhase   # if doInstallCheck = true
distPhase           # rare for binary packages

```

Hooks fire **before** and **after** many phases (`preConfigure`, `postInstall`, ...). Prefer hooks over replacing an entire phase when you only need a few lines.

Phase responsibilities

Phase	Default intent	Common overrides
unpack	Extract <code>src</code> into <code>\$sourceRoot</code>	<code>sourceRoot</code> , <code>setSourceRoot</code> , custom <code>unpackPhase</code> for flat files
patch	Apply patches list	<code>prePatch</code> to fix line endings
configure	<code>./configure --prefix=\$out</code> if script exists	<code>cmakeConfigurePhase</code> , <code>meson</code> , or <code>dontConfigure</code>
build	<code>make</code>	Custom compile; prefer language helpers for Go/Rust/...
check	<code>make check</code>	Gate with <code>doCheck = true</code>
install	<code>make install</code>	Copy binaries into <code>\$out/bin</code>
fixup	Strip, compress man, move docs, patch shebangs	<code>dontStrip</code> , <code>separateDebugInfo</code>

Hooks API (muscle memory)

When you **replace** a phase, call hooks so setup hooks from dependencies still run:

```

runHook preBuild
# your commands
runHook postBuild

```

Hook	Fires
<code>preUnpack</code> / <code>postUnpack</code>	Around unpack
<code>prePatch</code> / <code>postPatch</code>	Around patches
<code>preConfigure</code> / <code>postConfigure</code>	Around configure
<code>preBuild</code> / <code>postBuild</code>	Around build
<code>preInstall</code> / <code>postInstall</code>	Around install
<code>preFixup</code> / <code>postFixup</code>	Around fixup

Hook	Fires
------	-------

Skipping `runHook` is a silent footgun: `wrapProgram` hooks, Python path setup, and many language helpers never run.

Theory 3 — Important environment variables in the sandbox

During the build, the driver exports paths you should treat as API:

```
echo "$out"           # primary output
echo "$src"           # path to source input
echo "$NIX_BUILD_TOP" # temp build directory
echo "$sourceRoot"    # directory after unpack
echo "$PWD"           # usually $NIX_BUILD_TOP/$sourceRoot during build
echo "$CC" "$CXX"     # stdenv compilers
echo "$NIX_CFLAGS_COMPILE" # injected flags
```

Multiple outputs (later chapter) add `$dev`, `$man`, `$doc`, In this chapter, stick to `$out`.

Shebang and RPATH fixups happen in `fixupPhase` / `patchShebangs`. If a script still points at `/usr/bin/env` incorrectly, you usually forgot `nativeBuildInputs` for the interpreter **or** skipped fixup (`dontFixup = true` is rarely what you want).

Flags that skip work

Attr	Effect
<code>dontUnpack</code>	Skip unpack (inline sources / copy yourself)
<code>dontConfigure</code>	Skip configure
<code>dontBuild</code>	Skip build
<code>dontInstall</code>	Almost never—you need <code>\$out</code> populated
<code>dontFixup</code>	Skip strip/shebang; debug only
<code>doCheck</code>	Enable <code>checkPhase</code>
<code>enableParallelBuilding</code>	<code>make -j</code> style parallelism

Theory 4 — Minimal patterns you will reuse forever

Shell-only “package” (no compile)

```
{ stdenv, lib, bash }:
stdenv.mkDerivation {
  pname = "greet";
  version = "0.1.0";
  dontUnpack = true;
  installPhase = ''
    runHook preInstall
    mkdir -p $out/bin
    cat > $out/bin/greet <<'EOF'
    #!${bash}/bin/bash
    echo "hello from packaging craft"
    EOF
    chmod +x $out/bin/greet
    runHook postInstall
  '';
  meta = {
    description = "Trivial greet script packaged with stdenv";
    license = lib.licenses.mit;
    mainProgram = "greet";
  };
}
```

Note the `${bash}/bin/bash` in the shebang: absolute store path, pure and fixup-friendly.

Autotools / simple C (upstream tarball)

```
{ stdenv, fetchurl, lib }:
stdenv.mkDerivation rec {
  pname = "hello";
  version = "2.12.1";
  src = fetchurl {
    url = "https://ftp.gnu.org/gnu/hello/hello-${version}.tar.gz";
    hash = "sha256-jZkUKv2SV28wsM18tCqNxoCZmLxdYH2Idh9RLibH2yA=";
  };
  doCheck = true;
  meta = {
    homepage = "https://www.gnu.org/software/hello/";
    license = lib.licenses.gpl3Plus;
    mainProgram = "hello";
  };
}
```

Single C file without autotools

```
{ stdenv, lib }:  
stdenv.mkDerivation {  
  pname = "addone";  
  version = "0.1.0";  
  src = ./src; # directory with addone.c  
  buildPhase = ''  
    runHook preBuild  
    $CC addone.c -O2 -o addone  
    runHook postBuild  
  '';  
  installPhase = ''  
    runHook preInstall  
    install -Dm755 addone $out/bin/addone  
    runHook postInstall  
  '';  
  meta = {  
    description = "Adds one to an integer argument";  
    license = lib.licenses.mit;  
    mainProgram = "addone";  
  };  
}
```

Flat single-file source (unpack quirks)

```
{ stdenv, lib }:  
stdenv.mkDerivation {  
  pname = "note";  
  version = "0.1.0";  
  src = ./note.sh;  
  dontUnpack = true;  
  installPhase = ''  
    runHook preInstall  
    install -Dm755 $src $out/bin/note  
    runHook postInstall  
  '';  
  meta = {  
    mainProgram = "note";  
    license = lib.licenses.mit;  
  };  
}
```

Or keep unpack and set:


```
unpackPhase = ''
  cp $src ./note.sh
  chmod +x ./note.sh
  sourceRoot=.
'';
```

Theory 5 — finalAttrs vs rec

Modern nixpkgs prefers **finalAttrs** over **rec** for self-reference:

```
stdenv.mkDerivation (finalAttrs: {
  pname = "cooltool";
  version = "1.2.3";
  src = fetchurl {
    url = "https://example.com/${finalAttrs.pname}-${finalAttrs.version}.tar.gz";
    hash = "sha256-...";
  };
})
```

Style	Risk
<code>rec { version = ...; name = "\${pname}-\${version}"; }</code>	Easy to shadow; override surprises
<code>finalAttrs:</code>	Overrides of <code>version</code> update dependent attrs cleanly

Use either in lab; prefer **finalAttrs** when shaping packages for upstream later.

Theory 6 — What fixup actually does (short list)

`fixupPhase` (simplified mental model):

1. Move docs/man if multi-out helpers apply
2. Strip binaries (unless `dontStrip`)
3. `patchShebangs` on scripts under `$out`

4. Compress man pages
5. Record references for the runtime closure

If a binary works in the build directory but fails after install, check:

- Did you install into \$out?
- Did RPATH / interpreter need `patchelf` (usually automatic for stdenv ELF)?
- Did you set `dontFixup`?

```
file result/bin/addone
ldd result/bin/addone    # Linux
```

Worked example — flake that exports packages

```
# flake.nix
{
  description = "stdenv packaging lab";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      systems = [ "x86_64-linux" "aarch64-linux" "aarch64-darwin" "x86_64-darwin" ];
      forAllSystems = f: nixpkgs.lib.genAttrs systems (system: f (
        import nixpkgs { inherit system; }
      ));
    in {
      packages = forAllSystems (pkgs: {
        default = pkgs.callPackage ./pkgs/greet.nix { };
        greet = pkgs.callPackage ./pkgs/greet.nix { };
        addone = pkgs.callPackage ./pkgs/addone.nix { };
        hello = pkgs.hello; # control sample from nixpkgs
      });

      checks = forAllSystems (pkgs: {
        greet-runs = pkgs.runCommand "greet-runs" {
          nativeBuildInputs = [ self.packages.${pkgs.system}.greet ];
        } ''
          greet | grep -q "packaging craft"
          touch $out
        '';
      });
    }
}
```

```

    addone-runs = pkgs.runCommand "addone-runs" {
      nativeBuildInputs = [ self.packages.${pkgs.system}.addone ];
    } ''
      test "$(addone 41)" = "42"
      touch $out
    '';
  });

  devShells = forAllSystems (pkgs: {
    default = pkgs.mkShell {
      packages = [
        self.packages.${pkgs.system}.greet
        self.packages.${pkgs.system}.addone
      ];
    };
  });
};
}

```

```

nix build .#greet
./result/bin/greet
nix build .#addone
./result/bin/addone 41
nix flake check
nix log ./result      # after a failed build: nix log .#attr

```

Host consumption (optional)

```

# on a NixOS host module
{ pkgs, inputs, ... }:
{
  environment.systemPackages = [
    inputs.self.packages.${pkgs.system}.greet
  ];
}

```

Or without rebuilding the host: `nix shell .#greet`.

Worked example — intentional phase map document

For every package, keep a one-pager:

```
## addone
- unpack: directory src (default)
- patch: none
- configure: skipped (no configure script / default no-op)
- build: $CC addone.c
- check: none (doCheck false)
- install: install -Dm755 → $out/bin/addone
- fixup: strip + shebang (n/a for pure ELF)
```

This document is gold when something fails six months later.

Lab — multi-step

Suggested workspace: `~/lab/nixos/packaging/stdenv`

Step 1 — Scaffold

```
mkdir -p ~/lab/nixos/packaging/stdenv/{pkgs,src}
cd ~/lab/nixos/packaging/stdenv
```

Create `src/addone.c`:

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char **argv) {
    if (argc != 2) {
        fprintf(stderr, "usage: %s N\n", argv[0]);
        return 1;
    }
    printf("%d\n", atoi(argv[1]) + 1);
    return 0;
}
```

Step 2 — Write three packages

1. `pkgs/greet.nix` — shell script package
2. `pkgs/addone.nix` — single-file C
3. Optional: package `hello` yourself with `fetchurl` (compare to `pkgs.hello`)

Wire the flake above. Confirm:

```
nix build .#greet -L
nix build .#addone -L
nix run .#greet
```

Step 3 — Break phases on purpose

In `addone.nix`, set temporarily:

```
installPhase = "false"; # fail install
```

Build, read the log, note which phase failed:

```
nix build .#addone --print-build-logs || true
# restore installPhase with runHook wrappers
```

Step 4 — Flip input kinds

Move a tool between `buildInputs` and `nativeBuildInputs` (e.g. add `pkgs.pkg-config` wrongly then correctly). On native Linux both may “work”; write one sentence on **why** native is still correct for compilers and `pkg-config`.

Step 5 — Phase introspection

```
nix build .#greet -L
nix path-info -r ./result | head
nix derivation show .#greet | head -c 2000
```

Document: `unpack` → `build` → `install` → `fixup` for each of your packages (what actually ran).

Step 6 — Drop `runHook` once

Remove `runHook preInstall` / `postInstall` from `greet`, rebuild. Note: may still succeed for trivial packages—document why hooks still matter for non-trivial trees (wrappers, setup hooks).

Step 7 — Parallel build flag

For a make-based package (`hello`), set `enableParallelBuilding = true` and rebuild with `-L`. Note make job messages.

Step 8 — Host or shell consumption

Wire `systemPackages` or use `nix develop` / `nix shell`. Prove the binary is on `PATH`.

Exercises

1. **Empty \$out:** Write a derivation that “succeeds” but leaves `$out` empty—does Nix allow it? What does `nix path-info` show? Fix by installing a marker file.
 2. **sourceRoot:** Unpack a nested tarball layout (or fake one) and set `sourceRoot` correctly.
 3. **Check phase:** Enable `doCheck` with a trivial `checkPhase` that greps your binary’s help text.
 4. **meta.mainProgram:** Break `mainProgram`, run `nix run .#greet`, fix it.
 5. **Compare to nixpkgs:** `nix edit nixpkgs#hello` (or equivalent) and list three differences from your toy expressions.
-

Common gotchas

Symptom / mistake	What to do
<code>builder for '...' failed with exit code 1 with no clue</code>	Rebuild with <code>-L / --print-build-logs</code> ; jump to last phase name in log
<code>install: missing destination</code>	Create <code>\$out/bin</code> before install; use <code>install -Dm755</code>
Binary runs in build sandbox but not after install	Forgot install into <code>\$out</code> ; or <code>RPATH/interpreter</code> wrong—check <code>ldd / file</code>
<code>patchShebangs</code> left bad paths	Ensure interpreter package is a build input; call <code>patchShebangs \$out</code>
Overrode <code>buildPhase</code> without <code>runHook</code>	Add <code>runHook preBuild / postBuild</code>
Used <code>buildInputs</code> for <code>cmake/pkg-config</code> <code>src</code> is a single file, unpack fails	Prefer <code>nativeBuildInputs</code> <code>dontUnpack + copy</code> , or custom <code>unpackPhase</code>
Name collisions / impure timestamps in C	Avoid embedding timestamps; prefer <code>-D</code> defines from version only
<code>rec</code> + override surprises	Prefer <code>finalAttrs</code> pattern

Symptom / mistake	What to do
Darwin vs Linux <code>\$CC</code> assumptions	Use <code>stdenv</code> 's <code>\$CC</code> , not hard-coded <code>gcc</code>
Forgetting <code>meta.license</code>	Set it; habit for <code>nixpkgs</code> -shaped work
Parallel race in custom <code>buildPhase</code>	Document; fix races or disable parallel for that package

Checkpoint

- ☐ Can list default phase order from memory
- ☐ `greet` and `addone` build with `nix build`
- ☐ At least one intentional phase failure diagnosed from logs
- ☐ `nativeBuildInputs` vs `buildInputs` explained in notes
- ☐ Phases documented for each package
- ☐ Hooks (`runHook`) used in overridden phases
- ☐ Flake `checks` smoke-test at least one package
- ☐ Optional: compared personal `hello` packaging to `pkgs.hello`

Commit

```
git add .
git commit -m "packaging: stdenv phases - greet + addone packages"
```

Write three personal gotchas before continuing to fetchers and FODs.

Fetchers and fixed-output derivations

Fetchers and fixed-output derivations

Goal: Use fixed-output derivations (FODs) correctly—`fetchurl`, `fetchFromGitHub`, `vendor` hashes—and debug hash mismatches without turning off purity.

i Note

Baseline: Nix 2.34 / nixpkgs **26.05**. Prefer SRI `hash = "sha256-..."` over legacy `sha256 = "..."` attrs (both still appear in the wild).

Why it matters

Every package starts as **bytes from somewhere**. If those bytes are not fixed-output and hashed, your “reproducible” package is a network role-play. FODs are the contract: *given this hash, the store path is determined by content, and the build may touch the network only to produce those exact bytes*.

Without FOD discipline you cannot trust CI, caches, or offline rebuilds. With it, hash mismatches become a clear signal: either you meant to bump content, or something untrustworthy changed.

Theory 1 — Fixed-output vs normal derivations

Kind	Network in sandbox?	Output path depends on	Failure mode
Normal build	No (default)	Inputs (input-addressed)	Compile errors, missing tools
Fixed-output (FOD)	Yes, limited	Declared output hash	Hash mismatch if bytes change

FODs power almost every `src = fetch...` you see in nixpkgs.

```
src = fetchurl {  
  url = "https://example.com/foo-1.2.3.tar.gz";  
  hash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";  
};
```

Why the store path is special

For FODs, the output path is a function of the **hash** (and related fixed-output metadata), not of the builder script details the same way normal derivations are. That is why two packages fetching the same bytes can share a store path, and why a wrong hash never “quietly” accepts different content.

```
Normal:  out = f(name, builder, input paths, env, system, ...)
FOD:     out = f(outputHash, outputHashMode, ...) # content-addressed flavor
```

(Exact hashing rules are an internals topic; operationally: **trust the hash field.**)

Theory 2 — Common fetchers (2026 mental map)

Fetcher	Use when	Notes
<code>fetchurl</code>	Direct tarball/file URL	Mirrors, <code>curlOpts</code> , <code>name</code>
<code>fetchTarball</code>	Eval-time tarball (flakes/inputs style)	Different API; not always for package <code>src</code>
<code>fetchFromGitHub</code>	GitHub repo + rev/tag	Sets <code>owner</code> , <code>repo</code> , <code>rev</code> , <code>hash</code>
<code>fetchFromGitLab</code>	GitLab	Similar shape
<code>fetchgit</code>	Arbitrary git	Needs <code>hash</code> ; <code>leaveDotGit</code> carefully
<code>fetchzip</code>	Zip that should unpack	Unpacks in <code>fetcher</code>
<code>fetchpatch</code> / <code>fetchpatch2</code>	Single patch files	Great for <code>patches = [...]</code>
<code>fetchPypi</code>	Python sdist/wheel helpers	Via <code>python</code> infra
Language-specific vendor hashes	Go modules, Cargo, npm	See language ecosystem chapters

`fetchFromGitHub` pattern

```
{ stdenv, fetchFromGitHub, lib }:  
stdenv.mkDerivation rec {  
  pname = "example-src-only";  
  version = "14.1.1";  
  src = fetchFromGitHub {  
    owner = "BurntSushi";  
    repo = "ripgrep";  
    rev = version;           # tag or full commit  
    hash = "sha256-...";     # FOD for the archive Nix downloads  
  }
```

```

};
# real packaging would use rustPlatform; this is fetcher-focused
meta = {
  description = "Fetcher demo only";
  license = lib.licenses.unfreeRedistributable; # demo-set real license for real pkgs
};
}

```

Prefer **tags** or **full commit SHAs** over branch names (**main**). Branches move; FODs must not.

Submodules and deep clones

```

src = fetchFromGitHub {
  owner = "org";
  repo = "repo";
  rev = "v1.0.0";
  hash = "sha256-...";
  fetchSubmodules = true; # changes the hash vs without
};

```

Sparse / partial checkouts are advanced—prefer release tarballs when upstream publishes them.

Theory 3 — The hash mismatch workflow

You will see this forever:

```

error: hash mismatch in fixed-output derivation '...':
      specified: sha256-AAAA...
      got:      sha256-BBBB...

```

Correct response

1. **Trust “got” only after you trust the URL/rev.** If the project was compromised, blindly updating the hash pins malware.
2. Confirm you intended that **rev** / version.
3. Replace **hash** with the **got** value (or use `lib.fakeHash` once to force a fail).
4. Rebuild; lock the known-good hash in git.
5. Never leave **hash = ""**; or `lib.fakeHash` in committed production flakes.

Prefetch helpers

```
# URL
nix-prefetch-url https://ftp.gnu.org/gnu/hello/hello-2.12.1.tar.gz
nix store prefetch-file https://ftp.gnu.org/gnu/hello/hello-2.12.1.tar.gz

# Flake-style prefetch (metadata + hash)
nix flake prefetch github:BurntSushi/ripgrep/14.1.1

# Generic: let the build fail once
# hash = lib.fakeHash;

# Temporary during packaging
hash = lib.fakeHash; # forces mismatch with a clear "got"
```

SRI vs base32

Modern nixpkgs prints and prefers **SRI** (`sha256-...`). Older docs show `base32`. Both can refer to the same bytes; do not mix formats carelessly when comparing strings by eye—use the `got:` line Nix prints.

Theory 4 — Impure traps and purity

Trap	Why it hurts	Fix
<code>builtins.fetchGit</code> without flake lock discipline	Eval-time drift	Prefer flake inputs or FOD fetchers with hash
Branch <code>rev = "main"</code>	Hash churn	Pin commit
<code>curl</code> in a normal <code>buildPhase</code>	Sandbox network blocked	Use FOD or <code>pkgs.fetchurl</code> as input
Post-download patching inside FOD	Changes bytes → hash mismatch	Patch in normal derivation <code>patches</code>
Leaving <code>.git</code> without care	Non-determinism	Default archives omit <code>.git</code> ; <code>leaveDotGit</code> is advanced
Mirror that rewrites tarballs	Same URL, new bytes	Pin mirrors; notice mismatch as signal

Trap	Why it hurts	Fix
LFS pointers instead of files	Tiny “source”, broken build	Ensure LFS materialization in fetch

Rule: Download and fix bytes in the FOD **or** patch after unpack in the normal build—do not mix casually.

```
[FOD: pure download → exact bytes]
```

```
[Normal derivation: patch / compile / install]
```

Theory 5 — Flake inputs vs package fetchers

Mechanism	Pinned by	Good for
<code>inputs.foo.url = "github:..."</code>	<code>flake.lock</code>	Nix code, whole repos as flakes
<code>fetchFromGitHub</code> in a package	hash on the fetcher	Upstream source of a package
<code>pkgs.fetchgit</code> of a monorepo path	hash + sparse options	Subdir projects
<code>path: / ./src</code>	Your git tree	Lab-local code

Do not confuse “my flake input updated” with “my package `src` hash updated.” Both can change store paths; only one is your app’s source tarball.

```
# flake input - for Nix modules / another flake
inputs.tooling.url = "github:acme/tooling/v1.2.3";

# package src - for building C/Go/... sources
src = fetchFromGitHub {
  owner = "acme";
  repo = "tooling";
  rev = "v1.2.3";
  hash = "sha256-...";
};
```

Sometimes you use **both**: flake input for Nix files, FOD for the language project’s release tarball. That is normal—document which is which.

Theory 6 — outputHashMode (awareness)

FODs declare how content is hashed:

Mode	Typical use
<code>flat</code>	Single file
<code>recursive</code>	Directory tree (default for many fetchers)

You rarely set this by hand when using `fetchurl` / `fetchFromGitHub`. When writing custom FODs (`stdenv.mkDerivation` with `outputHash`), wrong mode causes confusing mismatches. Prefer high-level fetchers.

```
# Conceptual custom FOD - prefer fetchurl in real packages
stdenv.mkDerivation {
  name = "my-fod";
  builder = ...;
  outputHashMode = "flat";
  outputHashAlgo = "sha256";
  outputHash = "sha256-...";
}
```

Theory 7 — Private sources and credentials

Approach	Notes
CI token in netrc / access tokens	Keep out of world-readable store; prefer FODs that do not embed secrets in drv env
SSH <code>fetchgit</code>	Agent/daemon SSH complexity; document who can build
Vendor a release tarball internally	Mirror to your cache/object store + <code>fetchurl</code>
<code>builtins.fetchGit</code> with local credentials	Eval-time; flake lock discipline still required

Never put passwords in `hash` or URL query strings committed to git. Prefer: internal mirror + FOD hash of the mirrored artifact.

Worked examples

Example A — tarball package skeleton

```
# pkgs/hello-fod.nix
{ stdenv, fetchurl, lib }:
stdenv.mkDerivation rec {
  pname = "hello";
  version = "2.12.1";
  src = fetchurl {
    url = "https://ftp.gnu.org/gnu/hello/hello-${version}.tar.gz";
    hash = "sha256-jZkUKv2SV28wsM18tCqNxoCZmLxdYH2Idh9RLibH2yA=";
  };
  doCheck = true;
  meta = {
    description = "GNU Hello via explicit FOD";
    license = lib.licenses.gpl3Plus;
    mainProgram = "hello";
  };
}
```

Example B — GitHub + patch FOD

```
{ stdenv, fetchFromGitHub, fetchpatch, lib }:
stdenv.mkDerivation {
  pname = "example";
  version = "1.0.0";
  src = fetchFromGitHub {
    owner = "example-org";
    repo = "example";
    rev = "v1.0.0";
    hash = "sha256-SRCSOURCESOURCESOURCESOURCESOURCESOURCESO=";
  };
  patches = [
    (fetchpatch {
      url = "https://github.com/example-org/example/commit/abcdef.patch";
      hash = "sha256-PATCHPATCHPATCHPATCHPATCHPATCHPATCHPA=";
    })
  ];
  meta = {
    license = lib.licenses.mit;
  };
}
```

Local patches (`./patches/foo.patch`) are **path inputs**, not FODs. Remote `fetchpatch` is a FOD—second hash to maintain.

Example C — intentional mismatch

```
src = fetchurl {
  url = "https://ftp.gnu.org/gnu/hello/hello-2.12.1.tar.gz";
  hash = lib.fakeHash;
};
```

First build prints the real hash; paste it back.

Example D — flake wiring

```
{
  description = "FOD lab";
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = import nixpkgs { inherit system; };
    in {
      packages.${system}.hello-fod = pkgs.callPackage ./pkgs/hello-fod.nix { };
      packages.${system}.default = self.packages.${system}.hello-fod;
    };
}
```

Lab — multi-step

Suggested workspace: `~/lab/nixos/packaging/fod`

Step 1 — Prefetch a known tarball

```
mkdir -p ~/lab/nixos/packaging/fod/pkgs
cd ~/lab/nixos/packaging/fod
nix store prefetch-file \
  https://ftp.gnu.org/gnu/hello/hello-2.12.1.tar.gz
```

Record the SRI hash.

Step 2 — Package from that tarball

Write `pkgs/hello-fod.nix` with `fetchurl` + your hash. Export from a flake:

```
nix build .#hello-fod -L
./result/bin/hello
```

Step 3 — Force a mismatch

Change one character of the hash; rebuild; capture the error. Restore the correct hash.

Step 4 — GitHub fetcher

Pick a **tiny** tagged GitHub project (or a single-file tool). Package with `fetchFromGitHub`:

```
src = fetchFromGitHub {
  owner = "...";
  repo = "...";
  rev = "vX.Y.Z";
  hash = lib.fakeHash;
};
```

Iterate until the hash stabilizes. Prefer a project with a clear install (copy binary/script).

Step 5 — Patch as FOD and as path

Add a local patch first:

```
patches = [ ./patches/fix-typo.patch ];
```

Then replace with `fetchpatch` for one upstream commit if available—note the second hash.

Step 6 — Document the trust boundary

In `NOTES.md`:

1. Where did bytes come from?
2. Who could change them?
3. What does updating `hash` mean operationally?
4. Difference between updating `flake.lock` and updating `src.hash`.

Step 7 — Offline check

After populate:

```
nix build .#hello-fod --offline
```

If it works, your FOD is already in the store; purity is about **definition**, not “never used network once.”

Step 8 — Submodules experiment (optional)

Find a tiny repo with a submodule **or** skip with a written note on why `fetchSubmodules` changes hashes.

Exercises

1. Prefetch the same URL twice; confirm hash identity.
2. Change only the **name** of a `fetchurl` (if supported) and observe whether the store path changes—document findings.
3. Write a one-paragraph runbook: “upstream re-tagged v1.2.3 with different bytes.”
4. Compare `nix flake prefetch` output to `fetchFromGitHub` hash for the same rev—are they interchangeable? (Often related but not always drop-in; note what you observe.)
5. List every FOD in one of your language packages (src + vendor/cargo/npm).

Common gotchas

Symptom / mistake	What to do
Hash mismatch after “nothing changed”	Upstream re-released same tag; pin commit SHA
<code>fetchFromGitHub</code> hash differs on Darwin vs Linux	Unusual for pure archives; check name , filters, or LFS
Using empty <code>sha256</code>	Prefer <code>lib.fakeHash</code> for clarity; never commit fakes
Network error inside normal derivation	Move download to fetcher

Symptom / mistake	What to do
Submodules missing	<code>fetchSubmodules = true</code> (+ hash changes)
Huge monorepo as <code>src</code>	Sparse checkout / release tarball / <code>sourceRoot</code>
Private repo fetch	Mirror + FOD; careful with credentials
Patched inside FOD builder	Move patches to normal derivation
Confused flake lock with <code>src</code> hash	Update the pin that actually feeds the package
<code>leaveDotGit = true</code> nondeterminism	Avoid unless required; pin carefully

Checkpoint

- ☐ Explain FOD vs normal derivation in one paragraph
- ☐ Packaged something with `fetchurl` and correct SRI hash
- ☐ Packaged something with `fetchFromGitHub`
- ☐ Demonstrated and fixed a hash mismatch deliberately
- ☐ Notes on trust when updating hashes
- ☐ Offline rebuild succeeded for a cached FOD
- ☐ Distinguishes flake input pins from package `src` hashes

Commit

```
git add .
git commit -m "packaging: fetchers and FOD hash workflow"
```

Write three personal gotchas before continuing to language ecosystems.

Language ecosystems (part A)

Language ecosystems (part A)

Goal: Package a real tool in **your primary language** using the nixpkgs language builder (`buildGoModule`, `buildRustPackage`, `buildPythonApplication`, ...)—not raw `stdenv` unless the ecosystem forces it.

💡 Tip

Pick **one** primary ecosystem in this chapter. Part B is a second, smaller transfer exercise. Depth beats collecting half-broken scaffolds for five languages.

Why it matters

`stdenv` taught phases. Production packaging is almost always **ecosystem helpers**: they know how to vendor deps, set `LDFLAGS`, wrap interpreters, and place modules. Mastering one helper end-to-end is how packaging craft becomes useful at work.

Raw `stdenv` for a Go module graph is possible and miserable. Use the builder; understand that it is still phases + FODs underneath.

Theory 1 — How language builders relate to `stdenv`

```
buildGoModule / buildRustPackage / buildPythonApplication / ...
```

```
stdenv.mkDerivation (+ setup hooks)
```

```
phases + FODs (module vendor hashes, cargoLock, ...)
```

You still have `pname`, `version`, `meta`, `patches`, and `hooks`—but **fetching language deps** is specialized.

Ecosystem	Primary builder (nixpkgs)	Extra hash / lock concept
Go	<code>buildGoModule</code>	<code>vendorHash</code>
Rust	<code>rustPlatform.buildRustPackage</code>	<code>cargoHash</code> / <code>cargoLock</code>

Ecosystem	Primary builder (nixpkgs)	Extra hash / lock concept
Python	<code>buildPythonApplication</code> / <code>buildPythonPackage</code>	<code>pyproject</code> / <code>fetchPypi</code> / overrides
Node	<code>buildNpmPackage</code> / <code>importNpmLock</code>	<code>npmDepsHash</code>
Zig	<code>buildZigPackage</code> (where available) / <code>stdenv</code>	Fetch deps carefully
C/C++	<code>stdenv</code> + <code>cmake</code> / <code>meson</code>	Usually no vendor hash

Shared skeleton

```
src FOD → language-dep FOD → build/install hooks → $out/bin → meta
```

When stuck, ask first: **which hash is wrong—source or language deps?**

Theory 2 — Go path (`buildGoModule`)

```
{ lib, buildGoModule, fetchFromGitHub }:
buildGoModule rec {
  pname = "hello-go";
  version = "0.1.0";

  src = fetchFromGitHub {
    owner = "you";
    repo = "hello-go";
    rev = "v${version}";
    hash = "sha256-...";
  };

  vendorHash = "sha256-..."; # FOD of the vendored module graph

  ldflags = [
    "-s" "-w"
    "-X main.version=${version}"
  ];

  meta = {
    description = "Tiny Go greeter";
    license = lib.licenses.mit;
    mainProgram = "hello-go";
  };
}
```

Local module (lab-friendly)

```
{ lib, buildGoModule }:
buildGoModule {
  pname = "hello-go";
  version = "0.1.0";
  src = ../src/hello-go;
  vendorHash = null; # only if no external modules
  meta = {
    mainProgram = "hello-go";
    license = lib.licenses.mit;
  };
}
```

If you import modules, use a real `vendorHash`.

vendorHash workflow

1. Set `vendorHash = lib.fakeHash`; (or `null` only if truly module-free).
2. Build; read `got`: hash.
3. Paste; rebuild.
4. When `go.mod` / `go.sum` change, **vendorHash changes**.

```
nix build .#hello-go -L
```

Useful knobs

Attr	Purpose
<code>subPackages</code>	Build only <code>./cmd/foo</code>
<code>proxyVendor</code>	Vendor via module proxy inside FOD
<code>doCheck = false</code>	Skip tests if network-bound (prefer fixing tests)
<code>env.CGO_ENABLED = "0"</code>	Pure Go when appropriate
<code>ldflags</code>	Version injection, strip
<code>tags</code>	Build tags
<code>modRoot</code>	Multi-module monorepos

Example src/hello-go

```
// src/hello-go/main.go
package main

import (
    "fmt"
    "os"
)

var version = "dev"

func main() {
    if len(os.Args) > 1 && os.Args[1] == "--version" {
        fmt.Println(version)
        return
    }
    fmt.Println("hello-go")
}

// src/hello-go/go.mod
module example.com/hello-go

go 1.22
```

Theory 3 — Rust path (buildRustPackage)

```
{ lib, rustPlatform, fetchFromGitHub }:
rustPlatform.buildRustPackage rec {
    pname = "hello-rs";
    version = "0.1.0";

    src = fetchFromGitHub {
        owner = "you";
        repo = "hello-rs";
        rev = "v${version}";
        hash = "sha256-...";
    };

    cargoHash = "sha256-...";

    meta = {
```



```

    description = "Tiny Rust greeter";
    license = lib.licenses.mit;
    mainProgram = "hello-rs";
  };
}

```

Alternatively lock from a committed `Cargo.lock`:

```

cargoLock = {
  lockFile = ./Cargo.lock;
  # outputHashes = { "foo-0.1.0" = "sha256-..."; }; # for git deps
};

```

Follow current nixpkgs examples for **26.05** if `cargoHash` vs `git deps` bit you.

Git dependencies in Cargo

Vendoring git crates often needs `cargoLock.outputHashes` for each git source. Expect deep debugging the first time—same skills as the `debug builds` chapter.

Useful knobs

Attr	Purpose
<code>cargoBuildFlags</code>	Extra flags
<code>cargoTestFlags</code>	Test control
<code>buildAndTestSubdir</code>	Workspace member
<code>doCheck</code>	Run tests
<code>auditable</code>	Supply-chain related (when available)

Theory 4 — Python path (application vs library)

```

{ lib, python3Packages, fetchPypi }:
python3Packages.buildPythonApplication rec {
  pname = "hello-py";
  version = "0.1.0";
  pyproject = true;

  src = fetchPypi {
    inherit pname version;
    hash = "sha256-...";
  };
};

```

```

build-system = with python3Packages; [ setuptools ];
dependencies = with python3Packages; [ click ];

meta = {
  description = "Tiny Click CLI";
  license = lib.licenses.mit;
  mainProgram = "hello-py";
};
}

```

Local app:

```

{ lib, python3Packages }:
python3Packages.buildPythonApplication {
  pname = "hello-py";
  version = "0.1.0";
  pyproject = true;
  src = ../src/hello-py;
  build-system = with python3Packages; [ setuptools ];
  meta = {
    mainProgram = "hello-py";
    license = lib.licenses.mit;
  };
}

```

Builder	When
buildPythonApplication	CLI / end-user tool on PATH
buildPythonPackage	Library for others to depend on

Prefer **existing** python3Packages.* deps over vendoring wheels by hand. Align interpreters (python311 vs python312) across the graph.

pytest pattern (sketch)

```

nativeCheckInputs = with python3Packages; [ pytestCheckHook ];
# or pytest in checkPhase via hooks the builder provides

```

Theory 5 — Node (buildNpmPackage) sketch

```
{ lib, buildNpmPackage, fetchFromGitHub }:  
buildNpmPackage rec {  
  pname = "hello-node";  
  version = "0.1.0";  
  src = fetchFromGitHub {  
    owner = "you";  
    repo = "hello-node";  
    rev = "v${version}";  
    hash = "sha256-...";  
  };  
  npmDepsHash = "sha256-...";  
  meta = {  
    mainProgram = "hello-node";  
    license = lib.licenses.mit;  
  };  
}
```

package-lock.json (or supported lock) must exist; hash tracks the npm dependency FOD. Lock format churn changes hashes—commit the lockfile.

Theory 6 — Version injection and meta.mainProgram

Ecosystem	Version injection idea
Go	ldflags = ["-X main.version=\${version}"];
Rust	env.VERGEN_* or clap long_version via build.rs / env
Python	single-source version in package metadata
Node	package.json version; avoid drift with Nix version

Always set:

```
meta.mainProgram = "hello-go";
```

so `nix run` finds the binary without guessing.

Theory 7 — Tests and purity

Situation	Approach
Unit tests offline	Keep <code>doCheck = true</code>
Tests need network	Patch, mock, or disable with a comment
Tests need golden files with timestamps	Fix tests, don't disable sandbox
Integration tests need services	Prefer <code>nixosTest</code> later; not every unit suite

Disabling checks forever without a ticket is how bitrot lands in production images.

Worked example — flake with Go tool + check

```
{
  description = "language ecosystem A lab";
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      systems = [ "x86_64-linux" "aarch64-linux" ];
      forAllSystems = nixpkgs.lib.genAttrs systems;
    in {
      packages = forAllSystems (system:
        let pkgs = import nixpkgs { inherit system; }; in {
          default = pkgs.callPackage ./pkgs/hello-go.nix { };
          hello-go = pkgs.callPackage ./pkgs/hello-go.nix { };
        });

      apps = forAllSystems (system: {
        default = {
          type = "app";
          program = "${self.packages.${system}.default}/bin/hello-go";
        };
      });

      checks = forAllSystems (system:
        let pkgs = import nixpkgs { inherit system; }; in {
          hello-go-version = pkgs.runCommand "hello-go-version" {
            nativeBuildInputs = [ self.packages.${system}.default ];
          } ''
            hello-go --version | tee "$out"
          '';
        });
    }
```

```

devShells = forAllSystems (system:
  let pkgs = import nixpkgs { inherit system; }; in {
    default = pkgs.mkShell {
      packages = [
        pkgs.go
        pkgs.gopls
        self.packages.${system}.default
      ];
    };
  });
};
}

```

Adapt builders and binary names if your primary language is not Go.

Lab — multi-step

Suggested workspace: `~/lab/nixos/packaging/eco-a`

Step 1 — Choose primary language

Write the choice in `NOTES.md`: Go / Rust / Python / Node / other. Stick to it for all steps.

Step 2 — Minimal upstream project

Either:

- Create a tiny local project under `src/`, or
- Fetch a small tagged tool you actually use.

Requirements: builds offline once vendored; one CLI entrypoint; license known.

Step 3 — First package expression

Use the matching builder. Start with `lib.fakeHash` / fake vendor hashes as appropriate.

```
nix build .#default -L
```

Iterate until green.

Step 4 — Wire `meta.mainProgram` and run

```
nix run .#  
# or  
./result/bin/<name> --help
```

Step 5 — Version injection

Pass version via `ldflags` (Go), `env/build.rs` (Rust), or package metadata (Python/Node). Prove `tool --version` matches derivation `version` where feasible.

Step 6 — Tests

Enable package checks if feasible. If tests need network, patch or disable **with a comment** explaining why—not silence.

Step 7 — Consume from a `devShell`

```
nix develop  
which <tool>
```

Optional: add to a NixOS `environment.systemPackages` via flake package output.

Step 8 — Break the `vendor/cargo/npm` hash

Edit a dependency version or lock line; rebuild; observe hash mismatch; restore or update intentionally.

Step 9 — Closure glance

```
nix path-info -rS ./result | sort -k2 -n | tail  
nix path-info -Sh ./result
```

Note interpreter vs pure binary closure differences.

Exercises

1. Map your builder's phases back to stdenv names in a table.
 2. Produce a second output binary with `subPackages` or workspace members (if applicable).
 3. Add a `passthru.tests` or flake `checks` that fails if `--version` drifts.
 4. Cross-compile awareness: read whether your builder sets `GOOS/CARGO_BUILD_TARGET`—document only.
 5. Find the same tool in nixpkgs (`nix edit nixpkgs#...`) and list three techniques they use that you skipped.
-

Common gotchas

Symptom / mistake	What to do
<code>vendorHash</code> / <code>cargoHash</code> / <code>npmDepsHash</code> mismatch	Update after intentional lock change; never ignore silently
Go build tries network in buildPhase	Modules must be vendored via FOD; check <code>proxyVendor</code> / vendor dir
Rust git dependency hash missing	Add <code>outputHashes</code> for the git crate
Python module import works in tests but CLI missing	Use <code>buildPythonApplication</code> + <code>meta.mainProgram</code>
Node package missing lockfile	Generate supported lock; don't hand-wave <code>node_modules</code>
<code>mainProgram</code> wrong	Fix meta; <code>nix run</code> depends on it for default bin
CGO / shared libs surprise	Set <code>CGO_ENABLED</code> , add <code>buildInputs</code> for libs
Building entire monorepo	<code>subPackages</code> / workspace members / flags
Mixed python3 versions	Align package sets
Disabled checks permanently	Ticket + comment or fix

Checkpoint

- ☐ Primary language chosen and recorded

- ☐ Package builds with official language helper
- ☐ Vendor/cargo/npm hash workflow demonstrated
- ☐ `nix run` or `./result/bin` works
- ☐ Version string controlled from Nix where possible
- ☐ Notes on how this maps back to stdenv phases
- ☐ At least one check or test story
- ☐ Closure size noted

Commit

```
git add .  
git commit -m "packaging: language ecosystem A package"
```

Write three personal gotchas before part B.

Language ecosystems (part B)

Language ecosystems (part B)

Goal: Package a **second** language ecosystem tool so the *patterns* transfer: FODs for language deps, helpers vs raw stdenv, and how overrides differ across builders.

Why it matters

At work you will not stay monoglot. The point is not fluency in five builders—it is recognizing the **same skeleton**:

```
src FOD → language-dep FOD → build/install hooks → $out/bin → meta
```

Part A was depth. This chapter is **transfer** with a deliberately smaller scope: one binary, clear version, known license.

Theory 1 — Pattern map across ecosystems

Concern	Go	Rust	Python	Node
Source pin	src.hash	src.hash	src.hash / fetchPypi	src.hash
Dep pin	vendorHash	cargoHash / lock	package set / frozen	npmDepsHash
Builder	buildGoModule	buildRustPackage	buildPython*	buildNpmPackage
Skip tests	doCheck	doCheck / cargo flags	pytest hooks	often build-only
Binary name	module main	crate bin	meta.mainProgram	meta.mainProgram
Cross	GOOS/GOARCH care	target triples	often pure	often pure JS
Override flavor	overrideAttrs	overrideAttrs	overridePythonAttrs	overrideAttrs

When stuck, ask: *which hash is wrong—source or language deps?*

Theory 2 — Choose B deliberately

If part A was...	Good part B pick	Why
Go	Python CLI or Rust	Different dep story
Rust	Go small tool	Faster vendor cycle
Python	Node or Go	Leaves interpreter wrapping
Node	Python or Go	Sees compiled vs interpreted

Avoid starting a second multi-crate workspace. **One binary, clear version, known license.**

Transfer questions (answer in notes)

1. What is the second FOD field named?
2. Where does the binary land under \$out?
3. What is the fastest hash-mismatch workflow for this builder?
4. Would I pick this ecosystem for internal CLIs at work? Why?

Theory 3 — Wrapping and runtime deps

Interpreted ecosystems often need **wrappers**:

```
{ stdenv, lib, python3, makeWrapper }:  
stdenv.mkDerivation {  
  pname = "tool";  
  version = "0.1.0";  
  dontUnpack = true;  
  nativeBuildInputs = [ makeWrapper ];  
  installPhase = ''  
    runHook preInstall  
    mkdir -p $out/bin  
    cat > $out/bin/tool <<'EOF'  
    #!/bin/sh  
    echo tool  
    EOF  
    chmod +x $out/bin/tool  
    wrapProgram $out/bin/tool \  
      --prefix PATH : ${lib.makeBinPath [ python3 ]}  
    runHook postInstall
```

```

'';
meta = {
  mainProgram = "tool";
  license = lib.licenses.mit;
};
}

```

Situation	Prefer
Official language builder exists	Use it
Need one env var or PATH sibling	wrapProgram / makeWrapper
Need whole tree of modules	Builder's dependency graph
Need different interpreter	Override package set / python version

Python applications usually wrap PYTHONPATH via the builder—do not reimplement unless you must.

Theory 4 — Overrides preview

Deep dive lives in the patching chapter; know these exist:

```

# Change flags without forking the whole expression
helloGo.overrideAttrs (old: {
  ldflags = (old.ldflags or []) ++ [ "-X main.extra=eco-b" ];
})

# Python package override inside python3.pkgs
python3.pkgs.requests.overridePythonAttrs (old: {
  doCheck = false;
})

# Node / generic
helloNode.overrideAttrs (old: {
  postPatch = (old.postPatch or "") + ''
    substituteInPlace package.json --replace-fail '"private": true' '"private": false'
  '';
})

```

Composition rules matter later—for now, confirm your ecosystem B package can be lightly tweaked without a full fork.

Theory 5 — Cross-ecosystem FOD discipline

Whatever language B you chose, fetchers obey the same rules as the fetchers chapter:

Rule	Why
Pin URL + hash	Reproducible
Vendor offline where possible	CI without network flakiness
Prefer language-native lock → Nix conversion carefully	Don't invent hashes by hand when tooling exists
Never commit <code>fakeHash</code>	CI will fail or worse, someone “fixes” blindly

```
# Always capture both hashes when documenting
echo "src.hash=..."
echo "langDep.hash=..." # vendorHash / cargoHash / npmDepsHash
```

Theory 6 — Runtime vs build-time closure

```
nativeBuildInputs → tools for build (compilers, make)
buildInputs       → linked/used at build; may propagate
propagated*       → infect dependents (use sparingly)
```

A package that “works in `nix develop`” but fails as `nix build` often mixed shell conveniences with missing `buildInputs`.

Ecosystem	Common closure weight
Pure Go (<code>CGO_ENABLED=0</code>)	Small
Rust release bin	Small-medium
Python app	Interpreter + site-packages
Node CLI	Node runtime + modules (or bundled)

```
nix path-info -Sh ./result
nix path-info -rS ./result | sort -k2 -n | tail -n 20
```

Theory 7 — Dual interface pattern

Make both A and B print something comparable so checks share structure:

```
eco-a: hello from ecosystem A
eco-b: hello from ecosystem B
```

Or both support `--version` with Nix-injected versions. Unified checks then look like:

```
both-run = pkgs.runCommand "both-run" {
  nativeBuildInputs = [ a b ];
} ''
  ${a}/bin/${a.meta.mainProgram} --version
  ${b}/bin/${b.meta.mainProgram} --version
  touch $out
'';
```

Worked example — dual packages in one flake

```
{
  description = "language ecosystems A+B lab";
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      systems = [ "x86_64-linux" "aarch64-linux" ];
      forAllSystems = nixpkgs.lib.genAttrs systems;
    in {
      packages = forAllSystems (system:
        let pkgs = import nixpkgs { inherit system; }; in {
          lang-a = pkgs.callPackage ./pkgs/lang-a.nix { };
          lang-b = pkgs.callPackage ./pkgs/lang-b.nix { };
          default = self.packages.${system}.lang-b;
        });

      checks = forAllSystems (system:
        let
          pkgs = import nixpkgs { inherit system; };
          a = self.packages.${system}.lang-a;
          b = self.packages.${system}.lang-b;
        in {
          both-run = pkgs.runCommand "both-run" {
            nativeBuildInputs = [ a b ];
          } ''
            "${a}/bin/${a.meta.mainProgram}" --help >/dev/null 2>&1 \
            || "${a}/bin/${a.meta.mainProgram}" >/dev/null
```

```

        "${b}/bin/${b.meta.mainProgram}" --help >/dev/null 2>&1 \
        || "${b}/bin/${b.meta.mainProgram}" >/dev/null
        touch $out
    '';
});
};
}

```

Tiny Python app (if A was compiled)

```

{ lib, python3Packages }:
python3Packages.buildPythonApplication {
  pname = "eco-b-greet";
  version = "0.1.0";
  pyproject = true;
  src = ../src/eco-b-greet;
  build-system = with python3Packages; [ setuptools ];
  meta = {
    description = "Ecosystem B greet";
    license = lib.licenses.mit;
    mainProgram = "eco-b-greet";
  };
}

```

```

# src/eco-b-greet/pyproject.toml
[project]
name = "eco-b-greet"
version = "0.1.0"
requires-python = ">=3.11"
[project.scripts]
eco-b-greet = "eco_b_greet:main"
[build-system]
requires = ["setuptools"]
build-backend = "setuptools.build_meta"

```

```

# src/eco-b-greet/eco_b_greet.py
def main():
    print("hello from ecosystem B")

if __name__ == "__main__":
    main()

```

Tiny Node CLI (alternative)

```
{ lib, buildNpmPackage }:  
buildNpmPackage {  
  pname = "eco-b-node";  
  version = "0.1.0";  
  src = ../src/eco-b-node;  
  npmDepsHash = "sha256-...";  
  meta = {  
    mainProgram = "eco-b-node";  
    license = lib.licenses.mit;  
  };  
}
```

Tiny Go (if A was Python)

```
{ lib, buildGoModule }:  
buildGoModule {  
  pname = "eco-b-go";  
  version = "0.1.0";  
  src = ../src/eco-b-go;  
  vendorHash = null;  
  meta = {  
    mainProgram = "eco-b-go";  
    license = lib.licenses.mit;  
  };  
}
```

Lab — multi-step

Suggested workspace: ~/lab/nixos/packaging/eco-b
(or extend the ecosystem A flake)

Step 1 — Import or copy the part A package

Keep lang-a buildable so you compare logs side by side.

Step 2 — Scaffold smaller B project

Budget: under ~50 lines of application code. No microservices.

Step 3 — Package with the B builder

```
nix build .#lang-b -L
nix run .#lang-b
```

Record every hash you had to set (src + language deps).

Step 4 — Comparison table (required)

In `NOTES.md`:

Question	Lang A	Lang B
Builder name		
Source hash field		
Dep hash field		
Time to first success		
Hardest error		
Where binary lands		
Closure (nar size)		
Would I use this at work for CLIs?		

Step 5 — Unified check

Add a flake `checks.*` that smoke-tests both tools. `nix flake check`.

Step 6 — Optional same interface

Make both print a comparable line so a shell script can assert either.

Step 7 — Closure glance

```
nix path-info -rS ./result | sort -k2 -n | tail
nix path-info -Sh ./result
```

Note which ecosystem pulled a heavier closure and why (interpreter? compiler runtime? libc?).

Step 8 — Hash bump drill

1. Intentionally change a src rev or use `lib.fakeHash`
2. Run build; capture the `got:` hash
3. Paste correct hash; rebuild green
4. Write the exact error class name in notes

Step 9 — Light override

Apply `overrideAttrs` or `overridePythonAttrs` to flip `doCheck` or add an `ldflags` entry. Rebuild once.

Exercises

1. Draw the FOD graph for A and B (boxes and arrows).
 2. Time cold build vs warm cache for each (rough wall clock).
 3. Document one override API difference between the two builders.
 4. Try `nix run` without `mainProgram` set—observe failure mode—then fix.
 5. Write a recommendation: “default internal CLI language for our team” with two bullets of rationale from your measurements.
-

Common gotchas

Symptom / mistake	What to do
Trying to “finish” a second huge app	Shrink scope; transfer patterns, not product
Mixing package sets (python311 vs python312)	Align interpreters
Assuming <code>vendorHash = null</code> always works	Only pure module-free trees
<code>npmDepsHash</code> churn from lock format	Commit lock; pin npm
Comparing wall times unfairly	Same machine, warm vs cold cache noted
Forgetting <code>meta.license</code>	Set it; habit for nixpkgs-shaped work

Symptom / mistake	What to do
Unified check hardcodes wrong bin names	Use <code>meta.mainProgram</code>
Porting Go ldflags habits to Python	Use each ecosystem's idioms
Skipping comparison table	Required—this chapter is about transfer

Checkpoint

- ☐ Second ecosystem package builds and runs
- ☐ Comparison table filled honestly
- ☐ Both A and B reachable from one flake
- ☐ At least one combined check
- ☐ Closure notes captured
- ☐ Hash bump drill completed
- ☐ Light override demonstrated

Commit

```
git add .
git commit -m "packaging: language ecosystem B + comparison"
```

Write three personal gotchas before patching and overrides.

Patching and overrides

Patching and overrides

Goal: Fix or pin packages with `overrideAttrs`, `patches`, and overlays—without maintaining a permanent hard fork of `nixpkgs`.

Why it matters

Upstream will be wrong for your window of time: a CVE pin, a broken release, a flag your hardware needs. The professional skill is **minimal, reviewable deltas** that compose, not copy-pasting half of `nixpkgs` into your flake.

Theory 1 — Three layers of change

Mechanism	Scope	Reuse	Maintenance
override / overrideAttrs	One package instance	Call-site	Low–medium
Overlay	Package set view for a flake/host	Shared modules	Medium
Fork / fetch your tree	Anything	High control	High—avoid until needed

```
nixpkgs → overlay(s) → pkgs.yourTool
          ↑
      overrideAttrs / patches
```

Overlays reshape `pkgs` for everyone who shares that package set. Call-site overrides affect one reference. Choose the smallest scope that works.

Theory 2 — overrideAttrs

```
pkgs.hello.overrideAttrs (old: {
  patches = (old.patches or []) ++ [ ./patches/hello-msg.patch ];
  postPatch = (old.postPatch or "") + ''
    echo "packaging-craft" >> README
  '';
})
```

Form	Notes
<code>overrideAttrs (old: { ... })</code>	Preferred; see previous <code>attrs</code>
<code>overrideAttrs { ... }</code>	Easy to drop old patches by mistake
<code>package.override { stdenv = ...; }</code>	Change callPackage args (dependencies), not phases

override vs overrideAttrs

```
# Change dependencies / function args
pkgs.ffmpeg.override { withXcb = true; }

# Change derivation attributes / phases
pkgs.ffmpeg.overrideAttrs (old: {
  configureFlags = (old.configureFlags or []) ++ [ "--enable-something" ];
})
```

Language builders often expose both (e.g. `overridePythonAttrs`).

Composing safely

```
old.patches or []           # never assume patches exists
(old.postPatch or "") + ''  # string concat for hooks
(old.nativeBuildInputs or []) ++ [ pkgs.foo ]
```

Dropping upstream patches by accident is a classic security and build break.

Theory 3 — Patches as first-class inputs

```
stdenv.mkDerivation {
  # ...
  patches = [
    ./patches/0001-fix-build-on-musl.patch
    (fetchpatch {
      url = "https://github.com/org/proj/commit/abc123.patch";
      hash = "sha256-...";
    })
  ];
}
```

Practice	Why
Numbered filenames	Apply order is list order
Keep patches small	Rebase pain scales with hunk size
Prefer upstream PRs	Local patches rot on every release
Test without patch once	Confirm the patch is still required
Record expiry	“Remove when nixpkgs rev ...”

`patchPhase` applies these automatically; fail logs show hunk rejects.

Generating a patch

```
# Against unpacked sources or a git checkout of upstream
diff -u a/file b/file > patches/0001-msg.patch
# or
git format-patch -1 HEAD --stdout > patches/0001-msg.patch
```

Refresh patches when you bump `src.rev`—do not force with weaker reject settings as policy.

Theory 4 — Overlay composition

```
# overlays/hello-patch.nix
final: prev: {
  hello = prev.hello.overrideAttrs (old: {
    patches = (old.patches or []) ++ [ ../patches/hello-msg.patch ];
  });
}
```

```
# flake.nix fragment
pkgs = import nixpkgs {
  inherit system;
  overlays = [
    (import ./overlays/hello-patch.nix)
    (import ./overlays/my-tool.nix)
  ];
};
```

Order matters

```
overlay1 final:prev → overlay2 final:prev → ...
```

Later overlays see earlier results via the fixed-point.

Use **final** when depending on other packages you also overlay; use **prev** to modify the incoming package without recursion traps.

```
final: prev: {
  myApp = prev.myApp.overrideAttrs (_: { /* ... */ });
  # depend on overlaid hello:
  myAppWrapped = final.hello;
}
```

Infinite recursion patterns

```
# BAD
final: prev: {
  hello = final.hello.overrideAttrs (...); # often loops
}

# GOOD
final: prev: {
  hello = prev.hello.overrideAttrs (...);
}
```

NixOS / HM consumption

```
nixpkgs.overlays = [
  (import ./overlays/hello-patch.nix)
];
```

Ensure the **same** `pkgs` instance that builds your host sees the overlay. A second `import nixpkgs { }` without overlays silently ignores your work.

Theory 5 — Pinning and “don’t rebuild the world”

Goal	Technique
One package newer than channel	Overlay with <code>fetchFromGitHub</code> src bump + hashes
One package older (pin)	Overlay src to known-good rev; note security debt
Only your app sees the change	Don’t put overlay in global NixOS; inject via <code>callPackage</code> scope
Temporary CI fix	Overlay in flake only; ticket to drop it

```
final: prev: {
  curl = prev.curl.overrideAttrs (old: rec {
    version = "8.11.1";
    src = prev.fetchurl {
      url = "https://curl.se/download/curl-${version}.tar.bz2";
      hash = "sha256-...";
    };
  });
}
```

Bumping shared libraries can rebuild huge closures—check before celebrating:

```
nix why-depends .#myService .#openssl
nix build .#myService --dry-run 2>&1 | head
```

Theory 6 — Scoped injection without global overlays

```
# Only this package sees a special dependency
pkgs.callPackage ./pkgs/mytool.nix {
  openssl = pkgs.openssl.override { ... };
}

# Overlay adds your package without changing world openssl
final: prev: {
  mytool = final.callPackage ../pkgs/mytool.nix { };
}
```

Prefer **narrow** scope for experimental flags; global overlays for org-wide pins with review.

Theory 7 — Language-specific override helpers

Ecosystem	Helper	Example use
Generic	<code>overrideAttrs</code>	phases, patches, env
Generic	<code>override</code>	<code>callPackage</code> parameters
Python	<code>overridePythonAttrs</code>	<code>doCheck</code> , <code>deps</code>
Haskell	<code>overrideCabal</code> (ecosystem)	when you live there
Node	often <code>overrideAttrs</code>	<code>npmDepsHash</code> , <code>patches</code>

```
python3.pkgs.numpy.overridePythonAttrs (old: {  
  doCheck = false;  
})
```

Worked examples

Example A — local patch on hello

```
# patches/hello-msg.patch  
--- a/src/hello.c  
+++ b/src/hello.c  
@@ -59,7 +59,7 @@  
- printf ("Hello, world!\n");  
+ printf ("Hello, packaging craft!\n");
```

(Exact hunks must match the version you patch—regenerate against real sources.)

```
# overlays/hello-craft.nix  
final: prev: {  
  hello = prev.hello.overrideAttrs (old: {  
    patches = (old.patches or []) ++ [ ../patches/hello-msg.patch ];  
  });  
}
```

Example B — overlay adding your ecosystem A package

```
final: prev: {  
  hello-go = prev.callPackage ../pkgs/hello-go.nix { };  
}
```

Example C — packageOverrides legacy note

Older blogs use `packageOverrides`. Prefer **overlays** in modern flakes; recognize the old name when reading.

Example D — flake export

```
{
  description = "overrides lab";
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = import nixpkgs {
        inherit system;
        overlays = [ (import ./overlays/hello-craft.nix) ];
      };
    in {
      packages.${system}.hello = pkgs.hello;
      packages.${system}.hello-callsite =
        (import nixpkgs { inherit system; }).hello.overrideAttrs (old: {
          patches = (old.patches or []) ++ [ ./patches/hello-msg.patch ];
        });
    };
}
```

Lab — multi-step

Suggested workspace: `~/lab/nixos/packaging/overrides`

Step 1 — Baseline

```
nix build nixpkgs#hello -o baseline-hello
./baseline-hello/bin/hello
```

Step 2 — Write a patch

Create a minimal patch (message change or README touch). If patching C is painful on your platform, patch a script package you own from the `stdenv` chapter instead.

Step 3 — overrideAttrs only (no overlay)

```
pkgs.hello.overrideAttrs (old: {  
  patches = (old.patches or []) ++ [ ./patches/hello-msg.patch ];  
})
```

```
nix build .#hello-patched -L  
./result/bin/hello
```

Step 4 — Promote to overlay

Move the same change to overlays/... Import overlay into flake pkgs. Confirm drv path differs from unpatched when appropriate.

Step 5 — Stack two overlays

1. Overlay patches `hello`
2. Overlay adds `myTool` depending on `final.hello`

Prove `myTool` sees the patched `hello` if it references it (even a trivial wrap).

Step 6 — Dependency override

Pick a package with a boolean flag (choose something not enormous if you can):

```
pkgs.ffmpeg_7.override { /* one flag - or smaller package */ }
```

Build **or** evaluate and document why you stopped short if closure is huge.

Step 7 — Maintenance note

In `NOTES.md`: expiry conditions for the patch (“remove when `nixpkgs rev ...`” / “upstream PR #”).

Step 8 — Intentional drop of upstream patches

Temporarily set `patches = [ ./only-ours.patch ]`; without `old.patches`. Observe risk; restore proper composition.

Exercises

1. Write a table: call-site override vs overlay vs nixpkgs PR—when each wins.
2. Refresh a patch against a newer `hello` version; document the conflict resolution.
3. Use `lib.optionals stdenv.isLinux` for a Linux-only patch.
4. Find a package in your flake that should **not** use a global overlay; refactor to call-site.
5. Draft a PR description as if upstreaming your patch (problem, approach, testing).

Common gotchas

Symptom / mistake	What to do
Patch rejects	Refresh patch against actual <code>src</code> version
Overlay silently not applied	Ensure same <code>pkgs</code> instance reaches the host/package
Infinite recursion in overlay	Use <code>prev</code> for the package you modify
Dropped upstream patches	Always <code>(old.patches or []) ++ [ours]</code>
Rebuilt half of nixpkgs	You overrode a low-level lib; narrow scope
Unmaintainable mega-patch	Split; upstream; or wait for release
Darwin vs Linux patch drift	Conditional patches with <code>lib.optionals</code>
Two nixpkgs imports, one overlay	Unify package set construction
Security pin without expiry note	Add ticket date and risk acceptance

Checkpoint

- ☐ `overrideAttrs` patch builds and changes behavior
- ☐ Overlay form works from flake
- ☐ Two overlays composed with known order

- ☐ Difference `override` vs `overrideAttrs` written down
- ☐ Patch expiry / upstream plan noted
- ☐ Avoided infinite recursion pattern
- ☐ Understood blast radius of low-level library overrides

Commit

```
git add .  
git commit -m "packaging: patches, overrideAttrs, overlays"
```

Write three personal gotchas before multiple outputs.

Multiple outputs

Multiple outputs

Goal: Use multi-output derivations (`out`, `dev`, `man`, `doc`, ...) to keep **runtime closures** small while preserving headers and debug assets for dependents.

Why it matters

A single `$out` that ships headers, man pages, static libs, and a 2 MB binary forces every dependent to download and GC the world. Multi-output packaging is how nixpkgs stays usable on laptops and CI runners.

For fleet images, multi-out is a **disk and bandwidth** control: servers need `out`, build machines need `dev`, humans sometimes need `man`.

Theory 1 — Outputs are separate store paths

```
/nix/store/hash-foo-1.0      ← out (runtime)
/nix/store/hash-foo-1.0-dev  ← dev (headers, pkg-config)
/nix/store/hash-foo-1.0-man  ← man
/nix/store/hash-foo-1.0-doc  ← html docs
```

Each is a different path. **References** between them are tracked so GC stays correct: runtime `out` should **not** reference `dev` if you did it right.

```
stdenv.mkDerivation {
  pname = "foo";
  version = "1.0";
  outputs = [ "out" "dev" "man" ];
  # ...
}
```

Default if omitted: `outputs = [ "out" ];`.

First output is special

The first element of `outputs` is the default when someone depends on the package without selecting an output. Convention: **out first**.

```
outputs = [ "out" "dev" "man" ]; # good
outputs = [ "dev" "out" ];      # surprising defaults
```

Theory 2 — Conventional output roles

Output	Typical content	Who needs it
out	Binaries, shared libs, runtime data	Everyone running the tool
dev	Headers, .pc, static libs, cmake files	Compiling dependents
lib	Sometimes shared libs split from bins	Rare patterns / legacy
man	Man pages	Humans; optional on servers
doc	HTML/PDF docs	Optional
info	Texinfo	Optional
devdoc	Heavy API docs	Dev machines
debug	Separate debuginfo (with separateDebugInfo)	Debuggers

Environment variables in the build

Var	Points at
\$out	First output (usually runtime)
\$dev	Dev output path
\$man	Man output
\$outputs	List of outputs

Install into the correct root:

```
install -Dm755 foo $out/bin/foo
install -Dm644 foo.h $dev/include/foo.h
install -Dm644 foo.pc $dev/lib/pkgconfig/foo.pc
install -Dm644 foo.1 $man/share/man/man1/foo.1
```

Theory 3 — Why dependents get smaller

```
app depends on libfoo
→ needs libfoo.out (and its runtime closure)
→ does NOT need libfoo.dev unless building against headers
```

`nix path-info -r` on an application should **not** pull `*-dev` for every library if packaging is correct.

```
nix path-info -r ./result | rg -- '-dev$' || echo "no dev outputs in closure (good for runtime)"
```

Reference leaks

If a file in `$out` contains the store path of `$dev` (e.g. a script hardcoding include paths), Nix records a **reference**, and GC / closure pull `dev` into runtime. That defeats multi-out.

```
# After build, hunt for leaks (illustrative)
nix path-info -r ./libgreet | rg dev || true
strings result/bin/* | rg 'dev' || true
```

Theory 4 — `moveToOutput` and helpers

`stdenv` provides helpers used heavily in `nixpkgs`:

```
moveToOutput share/man "$man"
moveToOutput include "$dev"
moveToOutput lib/pkgconfig "$dev"
```

Hazard	Result	Mitigation
Headers installed to <code>\$out</code> references <code>\$dev</code>	Runtime closure grows Forces <code>dev</code> into runtime closure	Move to <code>\$dev</code> Fix install paths / <code>moveToOutput</code>
Forgetting <code>outputs</code> but installing to <code>\$dev</code>	Empty/wrong paths	Declare <code>outputs</code> first
<code>pkg-config</code> file wrong prefix	Consumers cannot find libs	Check <code>.pc prefix/libdir</code>
Static libs in <code>out</code>	Heavier runtime	Prefer static in <code>dev</code>

Theory 5 — Consuming multi-output packages

```
nativeBuildInputs = [ pkg-config ];
buildInputs = [ foo ]; # pulls foo.dev for compilation via setup hooks usually
```

Setup hooks from dev outputs typically put headers on the search path when foo is in buildInputs.

Selecting outputs explicitly:

```
foo.dev # the dev store path attribute
foo.out
foo.man
```

```
nix build nixpkgs#openssl
nix build nixpkgs#openssl.dev -o o-dev
nix build nixpkgs#openssl.man -o o-man
du -sh result o-dev o-man
```

Multiple outputs and nix shell

Depending on tooling, you may need the package default output only—headers come when building other packages, not always when “shelling” into a library.

Theory 6 — separateDebugInfo

```
stdenv.mkDerivation {
  # ...
  separateDebugInfo = true;
}
```

When supported, debug symbols go to a separate output so runtime stays lean while GDB can still find debuginfo via standard mechanisms.

Goal	Approach
Small production closure	strip + multi-out; optional separate debug
Debug a crash on a server	install debug outputs temporarily; don't ship forever
Reproducible debug paths	Prefer separateDebugInfo over ad-hoc unstripped bins

Theory 7 — When single-output is fine

Package kind	Multi-out?
Tiny shell script CLI	Usually no
Go/Rust single static-ish binary	Often single out
Shared C library used by many pkgs	Yes (out+dev)
Huge docs package	Split doc/man
Firmware blobs	Case-by-case

Do not multi-out for fashion—multi-out when dependents would otherwise drag weight.

Worked example — toy multi-output C library + app

```
/* src/libgreet/greet.c */
#include "greet.h"
const char *greet_msg(void) { return "hello multi-out"; }
```

```
/* src/libgreet/greet.h */
#pragma once
const char *greet_msg(void);
```

```
/* src/greetapp/main.c */
#include <stdio.h>
#include "greet.h"
int main(void) { puts(greet_msg()); return 0; }
```

```
# pkgs/libgreet.nix
{ stdenv, lib }:
stdenv.mkDerivation {
  pname = "libgreet";
  version = "0.1.0";
  outputs = [ "out" "dev" "man" ];
  src = ../src/libgreet;

  buildPhase = ''
    runHook preBuild
    $CC -shared -fPIC greet.c -o libgreet.so
    runHook postBuild
  '';
```

```

installPhase = ''
  runHook preInstall
  install -Dm755 libgreet.so $out/lib/libgreet.so
  install -Dm644 greet.h $dev/include/greet.h
  mkdir -p $dev/lib/pkgconfig
  cat > $dev/lib/pkgconfig/libgreet.pc <<EOF
  prefix=$out
  libdir=$out/lib
  includedir=$dev/include
  Name: libgreet
  Version: 0.1.0
  Libs: -L\'${libdir} -lgreet
  Cflags: -I\'${includedir}
  EOF
  mkdir -p $man/share/man/man3
  echo ".TH GREET 3" > $man/share/man/man3/greet.3
  runHook postInstall
'';

meta = {
  description = "Tiny greet library (multi-output lab)";
  license = lib.licenses.mit;
};
}

# pkgs/greetapp.nix
{ stdenv, lib, libgreet, pkg-config }:
stdenv.mkDerivation {
  pname = "greetapp";
  version = "0.1.0";
  src = ../src/greetapp;
  nativeBuildInputs = [ pkg-config ];
  buildInputs = [ libgreet ];
  buildPhase = ''
    runHook preBuild
    $CC main.c -o greetapp $(pkg-config --cflags --libs libgreet)
    runHook postBuild
  '';
  installPhase = ''
    runHook preInstall
    install -Dm755 greetapp $out/bin/greetapp
    runHook postInstall
  '';
  meta = {
    mainProgram = "greetapp";
    license = lib.licenses.mit;
  };
}

```

```

    };
}

# flake fragment
{
  packages = forAllSystems (system:
    let
      pkgs = import nixpkgs { inherit system; };
      libgreet = pkgs.callPackage ./pkgs/libgreet.nix { };
    in {
      libgreet = libgreet;
      greetapp = pkgs.callPackage ./pkgs/greetapp.nix { inherit libgreet; };
      default = self.packages.${system}.greetapp;
    });
}

nix build .#libgreet
nix build .#libgreet.dev -o result-dev
nix build .#libgreet.man -o result-man
nix build .#greetapp
./result/bin/greetapp
nix path-info -r ./result | rg libgreet

```

Lab — multi-step

Suggested workspace: ~/lab/nixos/packaging/multi-out

Step 1 — Inspect an existing multi-out package

```

nix eval nixpkgs#openssl.outputs
nix build nixpkgs#openssl -o o-out
nix build nixpkgs#openssl.dev -o o-dev
du -sh o-out o-dev

```

Record sizes.

Step 2 — Build libgreet with three outputs

Implement the toy library (or split your stdenv tool into lib + bin).

Step 3 — Verify separate paths

```
nix build .#libgreet
nix build .#libgreet.dev
nix build .#libgreet.man
nix path-info .#libgreet .#libgreet.dev .#libgreet.man
```

Step 4 — Build greetapp against libgreet

Confirm the app runs and its runtime closure includes `libgreet out` but ideally not manpages.

```
nix path-info -r ./result | rg 'libgreet'
```

Step 5 — Deliberate mistake

Install a header into `$out/include` and rebuild app. Compare closure size / path-info. Fix it.

Step 6 — Document policy

In `NOTES.md`: when you will use `outputs = [ "out" "dev" ]` for internal packages vs single-output scripts.

Step 7 — Optional separateDebugInfo

```
separateDebugInfo = true;
```

Build and note the extra debug output if supported on your platform.

Step 8 — pkg-config break/fix

Corrupt the `.pc libdir`; observe consumer failure; restore.

Exercises

1. Compare `du -sh` for `openssl.out` vs `openssl.dev` on your system.
 2. Find three multi-out packages in `nixpkgs` used by your hosts; list their `outputs`.
 3. Explain in two sentences how GC roots interact with multi-out (each path separately).
 4. Design outputs for a hypothetical SDK: runtime daemon, headers, CLI, docs.
 5. Measure `greetapp` closure before/after the intentional header-in-out mistake.
-

Common gotchas

Symptom / mistake	What to do
<code>dev</code> empty	Not installing into <code>\$dev</code> ; check <code>outputs</code> list
Runtime needs <code>dev</code>	Cross-reference leak; fix files that embed <code>\$dev</code> paths
<code>pkg-config</code> cannot find package	Ensure <code>.pc</code> in <code>dev</code> and package in <code>buildInputs</code>
Only <code>result</code> symlink	Multi-out: use <code>nix build .#pkg.dev</code> or <code>-o</code> names
Broke library for dependents	Test a dependent package in the same flake
Docs in <code>out</code>	<code>moveToOutput</code> or install directly to <code>\$doc/\$man</code>
Wrong outputs order	Put <code>out</code> first
Static lib only in <code>out</code>	Move <code>.a</code> to <code>dev</code> when possible

Checkpoint

- ☐ Can explain multi-out as multiple store paths
- ☐ Toy (or real) package produces `out` + `dev`
- ☐ Dependent app builds against `dev`, runs against `out`

- ☐ Closure inspection done with `nix path-info`
- ☐ One deliberate mis-install fixed
- ☐ Policy note written for internal packages
- ☐ Understand reference leaks

Commit

```
git add .  
git commit -m "packaging: multiple outputs libgreet + greetapp"
```

Write three personal gotchas before debugging builds.

Debugging builds

Debugging builds

Goal: Recover from failed package builds using `nix log`, `-L`, `--keep-failed`, `breakpointHook`, and structured isolation of phase failures—without disabling the sandbox as a first resort.

Why it matters

Packaging skill is half writing expressions and half **reading sandboxes under stress**. Teams lose days because someone only knows “it failed” and rebuilds with random flags. This chapter builds a deliberate debug playbook.

Theory 1 — Where information lives

Tool	Shows	When
<code>nix build -L /</code>	Live phase output	First failure
<code>--print-build-logs</code>		
<code>nix log <drv\ path></code>	Stored build log	After failure/success
<code>nix derivation show</code>	JSON-ish drv env	Input/env inspection
<code>--keep-failed</code>	Leaves <code>tmp</code> build dir	Interactive inspect
<code>breakpointHook</code>	Drops into shell on failure	Deep phase debug
<code>nix why-depends</code>	Why a dep is in a closure	Bloat / unexpected refs
<code>--rebuild</code>	Force re-realization	Cache hiding issues

```
nix build .#broken -L
nix log .#broken
nix derivation show .#broken | less
```

Modern CLI accepts installables (`.#pkg`); classic `nix-store -l /nix/store/...drv` still works.

Theory 2 — Eval failure vs build failure

Separate these **immediately**:

```
nix eval .#packages.x86_64-linux.foo.drvPath
# works? → build issue. fails? → expression / module issue.
```

Class	Signals	Look at
Eval	error: before builder starts	Nix syntax, missing attrs, infinite recursion
Build	Phase logs, compiler errors	Sources, deps, phases
FOD	hash mismatch	URL/rev/hash trust
Substitute	download errors	Network, cache keys

Spending an hour in `config.log` for an eval error is pure waste.

Theory 3 — Keep failed trees

```
nix build .#broken --keep-failed -L
```

On failure, Nix prints a path under `/tmp/nix-build-...` (or configured temp). You can:

```
cd /tmp/nix-build-foo-*
ls -la
# inspect source tree after patchPhase, object files, config.log, etc.
```

Warning

The kept tree is **not** the sandbox re-entered with the same env. It is a corpse for forensics. Re-running `make` there may work or lie—prefer `breakpointHook` for a live env.

Never commit keep-failed trees. They are for diagnosis only.

Theory 4 — breakpointHook

```
{ stdenv, breakpointHook }:
stdenv.mkDerivation {
  pname = "explode";
  version = "0.0.1";
  dontUnpack = true;
  nativeBuildInputs = [ breakpointHook ];
```

```

    buildPhase = ''
        echo "about to fail"
        false
    '';
}

```

When the build fails, you get a shell **inside** the build environment (with caveats on platforms and interactive settings). Fix hypotheses, then exit and fix the Nix expression permanently.

Also related: temporary extra tools in `nativeBuildInputs` for inspection (`strace`, `file`, `busybox`)—**remove before merge**.

Theory 5 — Common failure classes

Class	Signals	First moves
Hash / FOD	hash mismatch	Fetchers chapter workflow
Configure	<code>config.log</code> , missing headers	<code>nativeBuildInputs</code> , <code>pkg-config</code>
Compile	<code>error:</code> , missing symbols	deps, flags, patches
Link	undefined reference	<code>buildInputs</code> , order, static/shared
Install	empty <code>\$out</code> , wrong paths	<code>installPhase</code> , multi-out
Check	test suite	<code>doCheck</code> , sandbox network, golden files
Patch	rejects	refresh patch
Purity	banned network/path	FOD or remove impurity
Eval	before build	fix Nix syntax/attrs

Reading a log quickly

1. Jump to the **last** phase banner before the error
 2. Read the first **error:** line, not the last warning
 3. Check whether the failing command is yours or a setup hook
 4. Re-run with `-L` if the stored log is truncated in your UI
-

Theory 6 — Sandbox: don't disable casually

Flag / knob	Effect	Use?
<code>--option sandbox false</code>	Allows host impurities	Last resort local diagnosis only
<code>--no-sandbox</code>	Same family	Never as CI policy
<code>require-sigs</code> / <code>substituters</code>	Cache trust	Unrelated to compile fails
<code>--noChroot</code> (legacy patterns)	Avoid	Prefer proper FODs

If it only builds with sandbox off, your package is **not done**. Undeclared `/usr` deps will break on the next machine.

```
nix show-config | rg -i sandbox
# or
nix config show | rg -i sandbox
```

Theory 7 — Reproducing CI failures locally

```
# Clean-ish rebuild of one attr
nix build .#foo --rebuild -L

# Ignore binary cache for this attr (force local compile)
nix build .#foo --option substitute false -L

# Show why a dep is present
nix why-depends .#foo nixpkgs#openssl
```

Match CI: same flake lock, same system, same Nix version band (book baseline: **2.34.x**, nixpkgs **26.05**).

Bisect mental model

```
old lock (green)  update input  > new lock (red)
                    narrow: which input?
                    then: which package rebuild?
                    then: which phase?
```

Lock discipline + phase logs = faster bisect.

Theory 8 — Save the drv path

Always save the **drv path** from a failure before GC:

```
nix eval .#packages.x86_64-linux.foo.drvPath
nix log /nix/store/...drv
```

Without it, debugging after garbage collection becomes archaeology.

Worked example — scripted break/fix series

```
# pkgs/debug-lab.nix
{ stdenv, lib }:
stdenv.mkDerivation {
  pname = "debug-lab";
  version = "0.1.0";
  src = ../src/debug-lab;

  buildPhase = ''
    runHook preBuild
    $CC broken.c -o debug-lab
    runHook postBuild
  '';

  installPhase = ''
    runHook preInstall
    install -Dm755 debug-lab $out/bin/debug-lab
    runHook postInstall
  '';

  meta = {
    mainProgram = "debug-lab";
    license = lib.licenses.mit;
  };
}

/* broken.c - missing semicolon */
#include <stdio.h>
int main(void) {
  puts("debug-lab")
  return 0;
}
```

Fix: add the semicolon. Re-run with `-L` until green.

Failure dossier template

```
## Failure dossier
- Command:
- Eval or build?
- Phase (if build):
- First error line:
- Fix:
- Verification command:
```

Worked example — personal runbook skeleton

```
# DEBUG-RUNBOOK

1. Eval or build? (`nix eval ...drvPath`)
2. `-L` first failure phase
3. `nix log`
4. Classify: hash vs compile vs link vs install vs check vs purity
5. `--keep-failed` if need tree
6. `breakpointHook` / extra tools if need live env
7. Only then consider sandbox experiments
8. Port fix into Nix sources; never only /tmp
9. Add a check so it cannot regress silently
```

Lab — multi-step

Suggested workspace: `~/lab/nixos/packaging/debug`

Step 1 — Baseline success package

Start from `stdenv addone` or `ecosystem A` tool. Confirm green build + log:

```
nix build .#addone -L
nix log .#addone | tail -n 50
```


Step 2 — Inject a compile failure

Break a source file. Capture:

```
nix build .#addone -L 2>&1 | tee fail-compile.txt
nix log .#addone | tee fail-compile-log.txt
```

Step 3 — --keep-failed

```
nix build .#addone --keep-failed -L || true
# locate printed /tmp/nix-build-* path; list files; find .o / sources
```

Document what you could see vs not see.

Step 4 — Inject an install failure

Fix compile; break `installPhase` (`mkdir` missing, wrong path). Repeat log + keep-failed.

Step 5 — FOD failure recall

Temporarily wrong `src.hash`; confirm you recognize the error class in under 10 seconds.

Step 6 — breakpointHook (Linux recommended)

Add hook; force `false` in `buildPhase`; enter the breakpoint if your environment supports it. Type `env | sort | head`, inspect `$CC`, exit.

Step 7 — Write a personal runbook

`DEBUG-RUNBOOK.md` with **your** ordered steps (max one page).

Step 8 — Force local rebuild

```
nix build .#addone --rebuild --option substitute false -L
```

Note time vs cached build.

Step 9 — Complete failure dossier

For one intentional break, fill the dossier template under `notes/debug-dossier.md`.

Step 10 — Sandbox proof note

```
nix show-config 2>/dev/null | rg -i sandbox || nix config show | rg -i sandbox
```

State whether you ever disabled sandbox and why that is dangerous (undeclared deps “work on my machine”).

Exercises

1. Given only a CI log snippet ending mid-compile, list three next commands you would run locally.
 2. Break `doCheck` with a failing test; identify `checkPhase` in logs.
 3. Use `nix derivation show` to find an env var you did not know was set.
 4. Compare `--keep-failed` corpse vs `breakpointHook` live shell for the same failure—write pros/cons.
 5. Time how long it takes you to classify five pasted error messages (FOD/eval/compile/link/install).
-

Common gotchas

Symptom / mistake	What to do
<code>nix log</code> says no build log	Build never started (eval) or GC'd; rebuild with <code>-L</code>
Kept temp already deleted	Re-run with <code>--keep-failed</code> immediately
<code>breakpointHook</code> “does nothing”	Platform/interactive limits; use <code>keep-failed</code> + better logs
Fixed tree by hand in <code>/tmp</code> only	Always port fix into Nix sources
Disabled sandbox “to move on”	Track impurity; fix properly
CI-only failure	Align system, lock, substituters, case-sensitive paths
Silent <code>doCheck</code> fails	Watch for <code>checkPhase</code> in log; set <code>doCheck</code> explicitly
Scrolled past first error	Search for error: from top of failure region

Checkpoint

- ☐ Used `-L` and `nix log` on a real failure
- ☐ Used `--keep-failed` and inspected the tree
- ☐ Classified at least three failure kinds
- ☐ Tried `breakpointHook` or documented why not
- ☐ Personal debug runbook committed
- ☐ Eval vs build separation practiced
- ☐ Sandbox policy note written

Commit

```
git add .  
git commit -m "packaging: debug builds runbook and failure drills"
```

Write three personal gotchas before navigating nixpkgs.

Navigating nixpkgs

Navigating nixpkgs

Goal: Navigate modern nixpkgs package layout—especially **pkgs/by-name**—and shape a package so it could become an upstream PR (submission optional).

Why it matters

Copy-pasting random `default.nix` from a blog creates private snowflakes. Knowing where packages live, how they are called, and what reviewers expect makes your flake packages **portable** and your future contributions reviewable.

Theory 1 — Mental map of nixpkgs (2026)

Path	Role
<code>pkgs/by-name/<p>/<pname>/package.nix</code>	Preferred home for many new single-package expressions
<code>pkgs/top-level/all-packages.nix</code>	Historic wiring; less hand-editing for new pkgs
<code>pkgs/development/...</code>	Older category tree still holds many packages
<code>pkgs/applications/...</code>	Same—legacy category layout
<code>lib/</code>	Nix library functions
<code>nixos/modules/</code>	NixOS modules (not package expressions)
<code>nixos/tests/</code>	nixosTest definitions
<code>maintainers/maintainer-list.nix</code>	Maintainer identities
<code>pkgs/build-support/</code>	Fetchers, stdenv pieces, helpers

by-name rules of thumb

```
pkgs/by-name/he/hello/package.nix
```

```
package directory = pname
first two lowercase letters of pname
```

Rule	Detail
File name	Usually <code>package.nix</code> (not always <code>default.nix</code>)
Shard	<code>toLower (substring 0 2 pname)</code>
Scope	One package per dir; keep it self-contained
Call	Auto-wired for eligible packages—follow current <code>nixpkgs</code> docs if adding upstream

Your **flake** can still use `callPackage ./pkgs/foo.nix { }` without sharding; by-name is the **upstream shape**.

Theory 2 — Anatomy of a PR-shaped `package.nix`

```
{
  lib,
  stdenv,
  fetchFromGitHub,
  # only real deps
}:

stdenv.mkDerivation (finalAttrs: {
  pname = "cooltool";
  version = "1.2.3";

  src = fetchFromGitHub {
    owner = "acme";
    repo = "cooltool";
    rev = "v${finalAttrs.version}";
    hash = "sha256-...";
  };

  nativeBuildInputs = [ /* ... */ ];
  buildInputs = [ /* ... */ ];

  meta = {
    description = "One-line description";
    homepage = "https://github.com/acme/cooltool";
    license = lib.licenses.mit;
    maintainers = with lib.maintainers; [ /* your handle if upstream */ ];
    platforms = lib.platforms.unix;
    mainProgram = "cooltool";
  };
})
```

Modern habits

Habit	Why
<code>finalAttrs</code> : pattern	Version/src self-reference without <code>rec</code> pitfalls
Explicit function head args	<code>callPackage</code> injects deps
<code>meta.mainProgram</code>	<code>nix run nixpkgs#cooltool</code>
Real license	Policy and binary caches
No unrelated formatting of huge trees	Reviewer happiness
Tests via <code>passthru</code> / <code>nixosTests</code>	“Done upstream” bar

Theory 3 — `callPackage` and fixed-points

```
pkgs.callPackage ./package.nix { }  
# package.nix is a function of deps
```

```
# Override only one dep  
pkgs.callPackage ./package.nix {  
  openssl = pkgs.openssl_3;  
}
```

Overlays often do:

```
final: prev: {  
  cooltool = final.callPackage ./cooltool/package.nix { };  
}
```

Use `final.callPackage` so deps see other overlay packages.

Why function heads matter

```
# callPackage binds by argument name  
{ stdenv, fetchurl, openssl }: # must match pkgs attributes  
stdenv.mkDerivation { ... }
```

Unknown args fail; unused args may still be required by callers for override ergonomics—follow local style.

Theory 4 — Reading existing packages as literature

```
# If you have a nixpkgs checkout:
rg -n 'pname = "ripgrep"' -S pkgs | head

# Query without full clone:
nix edit nixpkgs#ripgrep
# opens the file in $EDITOR when configured

nix eval nixpkgs#ripgrep.meta --json | jq .
```

Read three packages in **your** primary language:

1. Tiny CLI
2. One with patches
3. One with multiple outputs

Note patterns: hooks, `finalAttrs`, meta, tests.

What to look for

Signal	Meaning
<code>finalAttrs</code>	Modern self-reference
<code>passthru.tests</code>	Linked tests
<code>updateScript</code>	nix-update support
<code>meta.changelog</code>	Release notes link
<code>separateDebugInfo</code>	Debug hygiene
Language builder	Ecosystem-native packaging

Theory 5 — Tests and passthru (awareness)

Many nixpkgs packages expose:

```
passthru.tests = {
  version = testers.testVersion { package = finalAttrs.finalPackage; };
};
passthru.updateScript = nix-update-script { };
```

Your flake can mirror with `checks`. Upstream PRs often expect *some* test story for non-trivial packages.

Theory 6 — Category tree vs by-name coexistence

Not everything is migrated:

```
pkgs/applications/misc/hello/default.nix    # classic
pkgs/by-name/ri/ripgrep/package.nix        # by-name
```

Aspect	Category tree	by-name
Discovery	Human taxonomy	Automatic shard
New packages	Discouraged for many cases	Preferred
Review tooling	Mature	Mature + automation
Your flake	N/A	Copy the <i>shape</i>

Do not rewrite half of nixpkgs for fun. **New** private packages: by-name shape. **Reading** old packages: know both trees.

Theory 7 — Contribution hygiene (even if you never PR)

Practice	Why
One logical change per commit	Review
cooltool: init at 1.2.3 message style	Convention
No secrets in expressions	Security
Builds on at least one platform claimed	Honesty
nixpkgs-review awareness	Community expectation
License correct	Legal

```
# PR-CHECKLIST.md
- [ ] Builds on my system
- [ ] meta complete (description, license, mainProgram, platforms)
- [ ] Hashes pinned (no fakeHash)
- [ ] Naming matches pname
- [ ] No secrets
- [ ] Commit message style ok
- [ ] Optional: nixpkgs-review
```

Worked example — flake package laid out like by-name

```
packaging-by-name/  
  flake.nix  
  pkgs/  
    by-name/  
      co/cooltool/package.nix  
  overlays/default.nix  
  PR-CHECKLIST.md  
  NOTES.md  
  
# overlays/default.nix  
final: prev: {  
  cooltool = final.callPackage ../pkgs/by-name/co/cooltool/package.nix { };  
}  
  
# flake.nix  
{  
  description = "by-name shaped package lab";  
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";  
  outputs = { self, nixpkgs }:  
    let  
      system = "x86_64-linux";  
      pkgs = import nixpkgs {  
        inherit system;  
        overlays = [ (import ../overlays/default.nix) ];  
      };  
    in {  
      packages.${system}.default = pkgs.cooltool;  
      packages.${system}.cooltool = pkgs.cooltool;  
      checks.${system}.meta-license =  
        assert pkgs.cooltool.meta ? license;  
        pkgs.runCommand "meta-ok" { } "touch $out";  
    };  
}
```

Reuse your best package from earlier chapters as `cooltool`—reshape, don't invent a new product.

Lab — multi-step

Suggested workspace: `~/lab/nixos/packaging/by-name`

Step 1 — Locate packages

```
nix edit nixpkgs#hello || true
nix eval nixpkgs#hello.meta --json | jq .
```

Record: description, license, platforms, maintainers shape.

Step 2 — Reshape your best packaging package

Move it to:

```
pkgs/by-name/<shard>/<pname>/package.nix
```

Use `finalAttrs` if it fits.

Step 3 — Overlay + flake export

call `Package` through overlay; `nix build .#cooltool`.

Step 4 — Meta completeness pass

Ensure:

- ☐ description
- ☐ homepage (if any)
- ☐ license
- ☐ mainProgram
- ☐ platforms or sensible default

Step 5 — PR-shaped checklist

Write `PR-CHECKLIST.md` even if not submitting.

Step 6 — Optional: clone nixpkgs and dry-place the file

```
# shallow clone is large; optional
# place file in correct by-name path; do not push unless you intend a real PR
```

Step 7 — Compare category tree vs by-name

Find one package still under `pkgs/applications/...` and one under `by-name`. Note one reviewer-facing difference.

Step 8 — Literature notes

For three packages in your language, fill:

Package	Builder	Patches?	Multi-out?	Tests?

Exercises

1. Compute the by-name shard for `pname = "ripgrep"`, `pname = "Go"`, `pname = "q"`.
2. Convert a `rec` derivation to `finalAttrs`.
3. Add a flake check that fails if `meta.license` is missing.
4. Read `lib.licenses` docs mentally—pick correct license for a real tool you use.
5. Sketch what would block an upstream PR for your package (honest list).

Common gotchas

Symptom / mistake	What to do
Wrong two-letter shard	Use first two chars of pname , lowercase
<code>default.nix</code> in by-name	Prefer <code>package.nix</code> per current convention
Recursive <code>rec</code> version bugs	Prefer <code>finalAttrs</code>
Giant unrelated deps in head	Only list what the function uses
Meta missing <code>mainProgram</code>	Multi-bin packages: set correctly or omit carefully
Assuming auto-discovery in your flake	Flakes need explicit <code>callPackage</code> /overlay
Editing all-packages by habit	Prefer by-name for new pkgs

Symptom / mistake	What to do
Fake maintainers list entry	Only add yourself upstream if contributing for real

Checkpoint

- ☐ Explained `by-name` sharding
- ☐ Package lives in `by-name-shaped` path
- ☐ Overlay + `callPackage` works
- ☐ Meta filled to `nixpkgs`-ish bar
- ☐ PR checklist written (submit optional)
- ☐ Read three upstream packages as literature
- ☐ Know category tree still exists

Commit

```
git add .
git commit -m "packaging: by-name shaped package + meta"
```

Write three personal gotchas before binary caches.

Binary caches

Binary caches

Goal: Pull and push substitutes with **cache.nixos.org**, **Cachix**, and/or self-hosted **Attic** / **Harmonia**—with a clear model of trust and signatures.

Why it matters

Without substitutes, packaging and fleet deploys become compile farms. With careless substituters, you outsource trust to whoever can serve a path. Caches are a **performance and supply-chain** topic, not just “make CI fast.”

Theory 1 — Substituters vs builders

```
eval → wanted store path P
      trusted substituter has P? → download (+ verify sigs)
      else                       → build (local or remote builder)
```

Term	Meaning
Substituter	HTTP(S) NAR store endpoint (<code>cache.nixos.org</code>)
Binary cache	Popular name for substituter + signing
Signed NAR	Artifact with signature by a cache key
Trusted key	Public keys your Nix is willing to accept
Priority	Order among substituters

```
# NixOS example
nix.settings = {
  substituters = [
    "https://cache.nixos.org"
    "https://myorg.cachix.org"
  ];
  trusted-public-keys = [
    "cache.nixos.org-1:6NCHdD59X431oOgWypbMrAURkbJ16ZPMQFGspcDShjY="
    "myorg.cachix.org-1:BASE64KEY..."
  ];
};
```

Flake/`nix.conf` user settings similar under `~/.config/nix/nix.conf` or daemon config.

Remote builders (next chapter) move *work*. Caches move *results*. You usually want both.

Theory 2 — Trust model (non-negotiable)

Claim	Reality
“HTTPS means safe binaries”	No—HTTPS protects transport, not <i>who built</i> the bits
“I added a cache URL only”	Incomplete without matching public key (or explicit none)
“Private cache = trusted”	Only if builders and keys are controlled
“cache.nixos.org is magic”	Trusted by default for official paths; still a third party

Rule: Every substituter you add is part of your TCB (trusted computing base) for that machine class.

```
Who can push? → Who holds signing keys? → Which hosts trust the pubkey?
               ↓
           What is the revoke plan?
```

Theory 3 — Cachix (hosted, popular)

Cachix hosts per-project caches with simple push UX.

```
nix shell nixpkgs#cachix -c cachix --help

# Auth (token from UI; treat as secret)
export CACHIX_AUTH_TOKEN=... # prefer secret manager / CI secret

cachix use myorg # writes substituter + pubkey locally (daemon may need trust)
cachix push myorg ./result
```

CI sketch:

```
- uses: cachix/install-nix-action@v31
- uses: cachix/cachix-action@v16
  with:
    name: myorg
```



```

    authToken: ${ secrets.CACHIX_AUTH_TOKEN }}
- run: nix build .#default -L

```

Pin action versions to what you verify—follow current Cachix docs for 2026.

Push full runtime closure

```

nix-store -qR ./result | cachix push myorg
# or modern copy patterns when endpoint supports them

```

Theory 4 — Self-hosted: Attic & Harmonia

Project	Role (approx.)	When
Attic	Modern multi-tenant binary cache server	Teams wanting self-host push/pull
Harmonia	cache.nixos.org-compatible server in Rust	Serve local store as substituter
nix-serve / nix-serve-ng	Classic signing servers	Legacy blogs

Harmonia sketch (NixOS)

```

services.harmonia = {
  enable = true;
  # signKeyPath = "/var/lib/secrets/harmonia.secret"; # manage via sops-nix
  # settings...
};
# nginx TLS reverse proxy in front in production

```

Attic sketch (conceptual)

```

# server configured with tokens / buckets (see upstream Attic docs)
attic login local http://cache.example.internal:8080 "$TOKEN"
attic push local:lab ./result

```

Exact CLI flags evolve—**read current Attic docs** for 26.05-era packaging; labs may use Cachix **or** local Harmonia/Attic **or** file-store simulation.

Theory 5 — Push what matters

Push	Skip
Your packages & CI deps	Entire nixpkgs rebuilds by accident
Closure of deploy targets	Personal /tmp experiments
Signed artifacts from trusted CI	Unsigned ad-hoc laptop builds for prod (policy choice)

Policy question	Example house rule
Who may push?	CI on <code>main</code> only
Who may add a substituter?	Module PR review
Laptop pushes?	Dev cache only, not prod keys

Theory 6 — Substituter order and priority

```
nix.settings = {  
  substituters = [  
    "https://cache.nixos.org"  
    "https://myorg.cachix.org"  
  ];  
  trusted-public-keys = [  
    "cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFGspcDShjY="  
    "myorg.cachix.org-1:..."  
  ];  
};
```

Idea	Practice
Official cache first	Common default
Extra caches append	Org/project binaries
Keys must match	Failures if mismatch / unsigned

Never add a substituter without its **public key**. Trust is cryptographic, not vibes.

Theory 7 — What “cache miss” really means

```
eval → derive output path hash
      → query substituters for that path
        → hit: download NAR
        → miss: build locally (or remote builder)
```

A miss is normal for private packages. A miss on `hello` from `cache.nixos.org` means network, config, or exotic platform issues.

```
nix path-info --store https://cache.nixos.org nixpkgs#hello
nix store ping --store https://cache.nixos.org
```

Force local build (contrast)

```
nix build .#cooltool --option substitute false -L
```

Feel the difference; do not leave `substitute false` in user config.

Theory 8 — Daemon vs user config

On multi-user Nix (NixOS default), **daemon** settings govern builds:

Location	Affects
<code>nix.settings</code> in NixOS	Daemon (system)
<code>~/.config/nix/nix.conf</code>	User; may be ignored for daemon builds
<code>trusted-users</code>	Who may set extra substituters

Symptom: “I added a cache to my user conf and nothing happened.” Fix: system module or trusted user policy.

Worked examples

NixOS module fragment for team cache

```
{ lib, ... }:
{
  nix.settings = {
    substituters = lib.mkAfter [
      "https://myorg.cachix.org"
    ];
    trusted-public-keys = lib.mkAfter [
      "myorg.cachix.org-1:REPLACE_ME="
    ];
  };
}
```

Local file NAR cache (always available lab path)

```
nix build .#cooltool -o result
nix copy --to "file://$PWD/nar-cache" ./result
# elsewhere or after GC of path (careful):
nix copy --from "file://$PWD/nar-cache" --to "$NIX_STORE" ./result
```

Query whether a path would substitute

```
OUT=$(nix build nixpkgs#hello --print-out-paths --no-link)
nix path-info --store https://cache.nixos.org --json "$OUT" | jq .
```

Lab — multi-step

Suggested workspace: ~/lab/nixos/packaging/cache

Step 1 — Observe default cache

```
nix build nixpkgs#hello -L
# note download lines mentioning cache.nixos.org
nix config show 2>/dev/null | rg -i 'substituter|trusted-public' \
  || nix show-config | rg -i 'substituter|trusted-public'
```

Step 2 — Build your package cold-ish

```
nix build .#cooltool --rebuild -L
```

Time it roughly.

Step 3 — Choose push target

Path A — Cachix: create free/personal cache if allowed; `cachix use`; push `./result`.

Path B — Self-host lab: enable Harmonia or Attic on a lab VM; configure key; push/copy.

Path C — Document-only: if no account/VM, write full config you *would* apply and simulate with `nix copy --to file:///...`

Step 4 — Second machine / second user story

On another user or VM (or after careful path GC):

```
nix build .#cooltool -L
# should substitute if cache wired and path present
```

Step 5 — Trust write-up

TRUST.md:

1. Who can push?
2. Who holds signing keys?
3. What hosts trust the pubkey?
4. What is the revoke plan?

Step 6 — CI note

Add a short paragraph: push from CI only with secrets, not from random laptops, if that is your house rule. Ops CI chapter deepens this.

Step 7 — Negative test

Point at a cache **without** adding the public key (on a disposable Nix config if you can). Observe failure mode; restore.

Step 8 — Document lab cache policy

In `docs/cache.md`:

1. Which substituters are enabled
 2. Which keys
 3. Whether you push CI builds
 4. Who may add a new cache (code review rule)
-

Exercises

1. Explain why HTTPS alone is insufficient for binary trust (one paragraph).
 2. Map your org's (or lab's) TCB for substituters.
 3. Compare wall time: `substitute` vs `--option substitute false` for a medium package.
 4. Draft a revoke checklist for a leaked Cachix token.
 5. Find the public key string for `cache.nixos.org` on your machine; confirm it matches docs.
-

Common gotchas

Symptom / mistake	What to do
"don't know how to trust" / unsigned	Add correct trusted-public-keys or sign
User <code>nix.conf</code> ignored by daemon	Configure system/daemon NixOS settings
Pushed wrong path	Push derivation outputs you care about; verify with another client
Cache hit but old content	Paths are immutable; rebuild means new hash
Leaked Cachix token	Rotate; treat like deploy key
Priority fights	Order substituters; understand fallback to build
Assumed private URL = auth	Many caches need tokens separately from Nix substituter URL

Symptom / mistake	What to do
Pushed entire nixpkgs by accident	Scope push to your closures

Checkpoint

- ☐ Explain substituter + signature trust
- ☐ Built with and without substitutes
- ☐ Pushed **or** file-store simulated push
- ☐ Second client pull story demonstrated/documentated
- ☐ TRUST.md written
- ☐ Daemon vs user config understood
- ☐ Cache miss vs hit explained

Commit

```
git add .
git commit -m "packaging: binary caches push/pull and trust notes"
```

Write three personal gotchas before remote builders.

Remote builders

Remote builders

Goal: Offload realization to a remote Nix machine (Linux builder VM is ideal), understand **builders** / build machine config, and know when remote build beats “just use a cache.” Align lab with **nixos-26.05** clients and builders when possible.

i Note

Remote builders move **work**; binary caches move **results**. Confusing them wastes weeks. You usually want both: build once on a beefy box, push to Cachix/Attic, everyone substitutes.

Why it matters

Laptops should not compile Chromium. CI runners should not all rebuild the same uncached leaf. **Remote builders** are how Darwin talks to Linux, how small VMs ship large closures, and how fleets keep eval local while realization lives elsewhere.

Theory 1 — Cache vs remote builder

Binary cache	Remote builder
Needs existing path?	No
Uses your flake inputs	Evaluates plan, builds drv remotely
Good for repeat installs	First build, exotic systems
Trust in network	SSH + Nix auth + who runs builder
Pat-tern	Upload inputs + download outputs

```
[dev] --build--> [builder] --push--> [cache] --substitute--> [fleet]
```

Theory 2 — Configuration surfaces (2026)

NixOS (nix.buildMachines)

```
{
  nix.distributedBuilds = true;
  nix.buildMachines = [
    {
      hostName = "builder.lab.internal";
      system = "x86_64-linux";
      protocol = "ssh-ng"; # preferred modern protocol when available
      sshUser = "nixbuilder";
      sshKey = "/root/.ssh/id_build"; # or agent patterns
      maxJobs = 8;
      speedFactor = 4;
      supportedFeatures = [ "nixos-test" "big-parallel" "kvm" ];
      mandatoryFeatures = [ ];
    }
  ];
  nix.settings = {
    builders-use-substitutes = true; # remote may fetch deps itself
  };
}
```

Ad-hoc one-shot

```
nix build .#pkg --builders 'ssh-ng://nixbuilder@builder.lab.internal x86_64-linux' --max-job
# classic:
# --option builders 'ssh://...'
```

Features

Feature	Typical meaning
big-parallel	Allow heavy parallel builds
kvm	Nested VM tests (nixosTest)
nixos-test	OS tests scheduling
benchmark	Specialization (rare)

If a drv requires `kvm` and the builder lacks it, scheduling skips that machine—often looking like “mysterious local build” or hard failure depending on flags.

Theory 3 — SSH and store talk

Remote build roughly:

1. Local eval (usually) produces derivations
2. Upload input paths the remote lacks
3. Remote builds
4. Download result paths back (unless you only needed remote side effects)

```
# Test SSH as the Nix daemon user (often root on NixOS)
sudo ssh -i /root/.ssh/id_build nixbuilder@builder.lab.internal nix-store --version
```

Daemon context: On multi-user Nix, *root* (nix-daemon) must reach the builder—not only your user ssh config—unless using remote build modes that run as you (know your setup).

Check	Why
Non-interactive SSH	No password prompts in daemon
<code>known_hosts</code> stable	Automation vs MITM prompts
Dedicated build key	Least privilege vs your personal key
Builder disk free	Full disk mid-build is common
Builder Nix version band	Avoid obscure protocol skew

Theory 4 — Cross and multi-system

Goal	Approach
Build <code>aarch64-linux</code> from <code>x86_64</code> laptop	<code>aarch64</code> builder or <code>qemu-binfmt</code> (slower)
Build Linux from Darwin	Linux remote builder almost always
Same-arch offload only	Simplest lab
<code>nixosTest</code> needing <code>kvm</code>	Builder with <code>kvm</code> feature + hardware

```
# Prefer real hardware/VM matching system
system = "aarch64-linux";
```

Honest lab goal: **one successful remote same-arch build**. Cross is stretch, not gate.

Theory 5 — Security notes

Risk	Mitigation
Builder can see sources	Treat builder as trusted environment
SSH key sprawl	Dedicated build key; restrict <code>authorized_keys</code>
Untrusted builder returns malicious paths	Only use builders you control; CI policies
World-open builder	VPN/Tailscale; no public SSH
Builder as root playground	Separate user; trusted-users carefully

Remote builders are **not** a sandbox that makes untrusted code safe. A compromised builder is a supply-chain incident.

Decision card — do I need a builder?

Situation	Prefer
Package already on cache	Substituter only
First build of large leaf on weak laptop	Remote builder
Darwin needing Linux closure	Linux builder
CI rebuilds identical drv every job	Push cache after one builder run
Untrusted contributor PR	Do not give them your builder

Theory 6 — Scheduling knobs that matter

Knob	Effect
<code>max-jobs 0 (client)</code>	Force offload when supported
<code>speedFactor</code>	Prefer faster builders
<code>builders-use-substitutes</code>	Remote fetches deps itself
<code>supportedFeatures</code>	Eligibility filter
Local <code>max-jobs > 0</code>	Local may steal work

```
nix build .#cooltool -L \  
  --builders "ssh-ng://nixbuilder@builder x86_64-linux - 8 1 big-parallel" \  
  --max-jobs 0
```

Upload storms

If every client uploads the world to the builder, enable `builders-use-substitutes` and share the same binary cache with the builder so it pulls deps itself.

Theory 7 — Builder host minimal config

```
# on builder  
users.users.nixbuilder = {  
  isNormalUser = true;  
  # group membership per current wiki; keep tight  
};  
nix.settings.trusted-users = [ "root" "nixbuilder" ];  
# SSH key auth only; open firewall only to admin net  
# Ensure large /nix disk; enable flakes
```

Exact store write permissions and `nix.sshServe` patterns vary by version—verify against **26.05** docs when wiring production.

Topology diagram (keep in notes)

```
[dev laptop / host A] --ssh-ng--> [builder VM B: NixOS 26.05, multi-user Nix]  
    |                               |  
    +-----> Cachix/Attic <-----+ (optional push from B or A)
```

Worked example — two-VM lab checklist

1. Inventory local capacity (max-jobs, disk)
2. Provision builder VM with free /nix space
3. Create dedicated SSH key; install pubkey for nixbuilder
4. Prove non-interactive SSH as root (daemon) and as user
5. One-shot remote build with `--max-jobs 0`

6. Capture log evidence of remote host
7. Persist `nix.buildMachines`
8. Optional: push results to cache; rebuild substitute-only
9. Failure injection: stop `sshd`; observe client error; restore

Prove which machine built

```
nix path-info -Sh .#cooltool
# keep a snippet of the build log that mentions the remote host
```

If you cannot show evidence of remote work, you may have silently built local—re-check `--max-jobs 0` and builder SSH as daemon user.

Lab — multi-step

Suggested workspace: `~/lab/nixos/packaging/builders`

Step 1 — Inventory local capacity

```
nix config show 2>/dev/null | rg 'max-jobs|cores|builders' \
  || nix show-config | rg 'max-jobs|cores|builders'
nproc
df -h /nix
```

Step 2 — Provision builder

Options:

1. Second NixOS VM on LAN/VPN
2. Same machine via `ssh://localhost` (limited learning value but tests plumbing)
3. Cloud NixOS box

Install Nix multi-user; enable flakes; ensure free disk (tens of GB).

Step 3 — SSH from daemon-relevant user

```
ssh -i ~/.ssh/id_build nixbuilder@builder 'nix-store --version'
sudo ssh -i /root/.ssh/id_build nixbuilder@builder 'nix-store --version'
```

Document which user the daemon uses on your OS.

Step 4 — One-shot remote build

```
nix build .#cooltool -L \
  --builders "ssh-ng://nixbuilder@builder x86_64-linux - 8 1 big-parallel" \
  --max-jobs 0
```

Capture log proving remote participation.

Step 5 — Persist config

Add `nix.buildMachines` on the client host flake **or** user remote builders file. Rebuild; run without CLI flags.

Step 6 — Feature flag experiment

Request a bogus mandatory feature; observe scheduling failure; remove it. Write one sentence on what you saw.

Step 7 — Combine with cache (optional)

On success, push the result (binary caches chapter). Then disable builder and rebuild from substitute-only.

Step 8 — Write topology diagram

ASCII in `NOTES.md`: who evals, who builds, who signs, who deploys, trust boundaries.

Step 9 — Failure injection

1. Stop `sshd` on builder (or block with firewall).
2. Retry build; capture the error class (SSH vs scheduling).
3. Restore builder; confirm recovery.

Document: “client symptoms when builder is down.”

Step 10 — Disk pressure note

Fill or simulate low disk on builder (carefully); document monitoring need (`df -h /nix` in run-books).

Exercises

1. One paragraph: when cache alone is enough vs when you need a builder.
 2. Design features list for a “CI builder” vs a “laptop offload” builder.
 3. Explain why user SSH working is insufficient on multi-user Nix.
 4. Estimate upload size for a clean builder without shared cache (qualitative).
 5. Security review: what can a malicious builder do to your laptop store?
-

Common gotchas

Symptom / mistake	What to do
<code>cannot build on ...</code>	Features/system mismatch; check <code>supportedFeatures</code>
SSH works as user, not as root	Configure root’s key or daemon SSH
Huge upload times	Enable <code>builders-use-substitutes</code> ; share cache with builder
Darwin → Linux confusion	Need Linux builder; not just <code>--system</code>
Stale <code>known_hosts</code> / MITM prompts	Non-interactive SSH hardening for automation
Builder disk full	GC on builder; monitor
Nested flake eval differences	Same Nix version band; same inputs
Assuming cache = builder	Re-read Theory 1
Silent local build	<code>--max-jobs 0</code> ; verify logs

Checkpoint

- ☐ Distinguished cache vs remote builder
- ☐ Completed at least one remote (or localhost SSH) build
- ☐ Knew which SSH identity the builder used
- ☐ Documented persistent config approach
- ☐ Topology notes include trust
- ☐ Failure injection documented
- ☐ Features experiment observed

Commit

```
git add .  
git commit -m "packaging: remote builders lab and topology"
```

Write three personal gotchas before reproducibility.

Reproducibility (honest view)

Reproducibility (honest view)

Goal: Measure what “reproducible” means for **your** packages—bit-for-bit where possible, rebuild-equivalent where not—and document limits without marketing language.

i Note

Baseline: flakes + **nixpkgs 26.05**-class pins. This chapter is measurement and claims hygiene—not a promise that every package is bitwise identical forever.

Why it matters

Nix marketing slides say “reproducible builds.” Production engineers say “same flake lock gave the same store paths on two Linux boxes last Tuesday.” Both can be true at different strengths. Packaging craft ends with **honesty**, or fleet ops and disaster recovery false confidence will hurt.

Theory 1 — Strength ladder

Level	Claim	How to test
L0	Builds on my laptop once	<code>nix build</code>
L1	Rebuilds same path on same machine	<code>nix build --rebuild</code> → identical out path
L2	Same lock → same paths on two similar Linux hosts	Compare <code>nix build --print-out-paths</code>
L3	Bit-for-bit identical NAR contents	<code>diffoscope</code> / hash NARs
L4	Cross-OS / cross-vendor identical	Often false for complex pkgs
L5	Reproducible <i>data</i> and <i>uptime state</i>	Not Nix’s job alone

Most flake apps aim for **L1–L2**. Many packages are not fully L3 yet (timestamps, capture order, non-deterministic linking). **L5** is backups, migrations, and ops—not `stdenv`.

How to talk about this in a runbook

Weak claim	Stronger claim
“Fully reproducible”	“With lock X on x86_64-linux, attr Y yields path P”
“Same everywhere”	“Same path on two Linux builders; Darwin not claimed”
“Immutable forever”	“Unchanged until inputs change; updates are deliberate”

Theory 2 — Input-addressed sameness

In the default model, the **output path hash** is a function of the derivation inputs (names, other paths, builder, system, env). If two machines produce the **same out path**, Nix agrees the build plan matched—not always that every byte was independently audited by you.

Content-addressed derivations (internals part) change the story: output hash can track content. Know they exist; **do not** depend on CA here unless you already enabled it deliberately.

```
input-addressed: out path = f(derivation inputs...)
content-addressed: out path can reflect output hash (advanced)
```

Path sameness vs byte sameness

Observation	Means
Same out path	Same build plan (input-addressed)
Same NAR hash	Same bytes
Cache hit of path P	Someone trusted published P; not necessarily you built it twice

Theory 3 — Why bit-for-bit fails

Source	Example
Embedded timestamps	gzip headers, archives, man pages
Unordered maps / readdir	Parallel compression, zip member order
CPU-specific codegen	<code>-march=native</code> (avoid in shared packages)

Source	Example
Download nondeterminism	Unpinned floating tags (FODs)
Test data with time	Snapshot tests embedding dates
Build-id / path capture	Binaries recording build paths poorly
RNG in build	Unseeded UUIDs in artifacts

```
# Bad idea in shared packages:
NIX_CFLAGS_COMPILE = "-march=native";
```

Prefer portable flags; put optimized overlays in **explicit local** overlays, not silent defaults.

Fixed-output derivations (FODs)

FODs pin **output hash** of fetched content. They improve input stability but do not magically make compilers deterministic. A wrong hash fails the build; a floating **main** branch fetch is still an honesty bug at the pin layer.

Theory 4 — Tools for comparison

```
# Same path string?
nix build .#cooltool --print-out-paths | tee path-a.txt
# on machine B with same lock:
# nix build .#cooltool --print-out-paths | tee path-b.txt
diff path-a.txt path-b.txt

# Force rebuild and compare path (avoid silent substitute hiding local nondeterminism)
nix build .#cooltool --rebuild --print-out-paths

# Diff two store paths if they differ
nix store diff-closures /nix/store/...-a /nix/store/...-b

# Detailed binary diff
nix shell nixpkgs#diffoscope -c diffoscope ./result-a ./result-b

# Hash runtime closure (rough fingerprint)
nix path-info -rS ./result | sort | tee closure.txt
nix hash path ./result

# When you must ignore caches to test local build determinism
nix build .#cooltool --rebuild --option substitute false -L
```

Theory 5 — Operational reproducibility vs academic

Ops need	Nix contribution
Rebuild server from git	Flake lock + Disko + secrets keys
Same CI binary as prod	Shared cache + same installable attr
Legal/compliance bit-identical	Extra tooling; not free with Nix alone
Debug “works on my machine”	Locks + sandboxed builds + L1/L2 tests
Trust binary cache	Signatures + trusted keys

Write claims in runbooks as **testable sentences**, not adjectives.

Theory 6 — What Nix does *not* make reproducible

Domain	Why
Database contents	State; backups
TLS certificates from ACME	Time and CA interaction
VM disk drift	Imperative writes outside store
Hardware firmware	Out of band
“Same UI screenshot”	Fonts, GPU, timing
User home clutter	Not in the flake

Honesty: **system closure** rebuildable **product data** immortal.

Theory 7 — Cache vs reproducibility

```
## Cache vs reproducibility
- Same out path from cache I audited the build locally.
- Signatures answer "who published this path?" not "is the universe bit-identical?"
- Our claim: trusted substituters may supply P if signatures verify.
- Our non-claim: every path was built twice under our eyes.
```

Substitutes accelerate L2 distribution; they do not replace measurement.

Worked example — measurement script

```
#!/usr/bin/env bash
# scripts/measure-repro.sh
set -euo pipefail
ATTR="${1:-.#cooltool}"
OUT1=$(nix build "$ATTR" --print-out-paths --no-link)
OUT2=$(nix build "$ATTR" --rebuild --print-out-paths --no-link)
echo "first:    $OUT1"
echo "rebuild:  $OUT2"
if [[ "$OUT1" == "$OUT2" ]]; then
    echo "L1 path match: YES"
else
    echo "L1 path match: NO"
    nix store diff-closures "$OUT1" "$OUT2" || true
fi

chmod +x scripts/measure-repro.sh
./scripts/measure-repro.sh .#cooltool | tee ~/lab/nixos/packaging/repro/l1-cooltool.txt
```

Worked example — deliberate nondeterminism

```
# pkgs/bad-stamp.nix - lab only
{ stdenv }:
stdenv.mkDerivation {
    name = "bad-stamp";
    phases = [ "buildPhase" "installPhase" ];
    buildPhase = ''
        date > stamped.txt
    '';
    installPhase = ''
        mkdir -p $out
        cp stamped.txt $out/
        echo ok > $out/ok
    '';
}
```

Rebuild twice; discuss whether **out path** changed (input-addressed plan may still match if recipe identical while **contents** differ—observe carefully with `--rebuild` and content hashes). Then remove **date** and remeasure.

Worked example — REPRO.md template

```
## Claims we make
- With flake.lock revision ... on x86_64-linux, `nix build .#cooltool` yields path P.
- We push P to cache C signed by key K (if applicable).

## Claims we do not make
- Bit-for-bit identity on Darwin vs Linux
- Reproducible database contents
- Forever stability without lock updates
- L3 for all transitive nixpkgs deps

## Measured on
- date, nix version, systems, results table

## Residual risks
- ...
```

Two-machine log template

```
## L2 attempt
Lock: flake.lock nixpkgs rev ...
Attr: .#cooltool
Machine A: hostname, nix version, out path
Machine B: hostname, nix version, out path
Match: YES/NO
If NO: first nix store diff-closures lines / system difference notes
```

Even a failed L2 (no second machine) earns credit if you document the blocker and a next attempt plan.

Lab — multi-step

Suggested workspace: ~/lab/nixos/packaging/repro

Step 1 — Environment record

```
mkdir -p ~/lab/nixos/packaging/repro
cd /path/to/your/flake
nix flake metadata | tee ~/lab/nixos/packaging/repro/metadata.txt
```



```
nix --version | tee ~/lab/nixos/packaging/repro/nix-version.txt
uname -m | tee ~/lab/nixos/packaging/repro/uname.txt
```

Step 2 — Pick three artifacts

1. Your primary packaging package (e.g. `.#cooltool`)
2. `nixpkgs#hello`
3. One “fat” but sane package you use (e.g. `git`—not a browser)

Step 3 — L1 measurements

For each, run rebuild comparison; record path equality in a table:

Attr	Path1	Path2	L1 match?

Step 4 — L2 if you have two environments

Same `flake.lock`, two machines or two VMs. Compare printed out paths. If only one machine, document the blocker.

Step 5 — Substitute false control

For your package, once with substitutes allowed and once with `--option substitute false --rebuild`. Note time and path.

Step 6 — Deliberate nondeterminism (lab only)

Add `bad-stamp` package; rebuild; fix by removing `date`; note findings.

Step 7 — Closure drift story

Change a dependency pin (or overlay bump) while keeping app source identical. Show out path change. One paragraph on **reproducible inputs**.

Step 8 — Honesty statement

Write `REPRO.md` from the template.

Step 9 — Optional diffoscope

If two outputs differ, run diffoscope and paste the **top** finding into notes—not a 10k-line dump.

Step 10 — Cache trust paragraph

Link binary cache trust to honesty using the Theory 7 bullets.

Step 11 — README claim scrub

If your package README says “fully reproducible,” rewrite that sentence using L1/L2 language from REPRO.md. Paste before/after into notes.

Exercises

1. Classify five real packages you use into L0–L3 ambition.
2. Argue for or against requiring L3 for an internal Go CLI (one page max).
3. Show that floating `inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable"` without lock discipline breaks L2.
4. Measure L1 for a multi-output package’s `out` vs `dev` separately.
5. Write an executive one-liner that is true and useful about Nix reproducibility.

Common gotchas

Symptom / mistake	What to do
Assumed same path across Darwin/Linux	Different system → different paths normal
Compared without same lock	Update discipline first
Trusted cache hit hiding local nondeterminism	<code>--rebuild / --option substitute false</code>
Marketing “fully reproducible” in README	Replace with measured claims
Floating <code>main</code> input	Pin; FODs with correct hashes
Compared <code>result</code> symlinks without realizing GC	Paths may still exist; note roots
L3 required for every package	Prefer honest L1–L2 for apps

Symptom / mistake	What to do
Confused data durability with package repro	Backups chapter / ops

Checkpoint

- ☐ Ladder L0–L5 explained in own words
- ☐ L1 measured for own package
- ☐ Environment metadata captured
- ☐ Deliberate nondeterminism demo + fix
- ☐ REPRO.md honesty statement written
- ☐ Cache trust paragraph written
- ☐ README claim scrubbed if needed
- ☐ Know path-sameness vs byte-sameness
- ☐ Three personal gotchas

Commit

```
git add .  
git commit -m "packaging: reproducibility measurements and honesty notes"
```

You now have the packaging spine: phases, FODs, ecosystems, overrides, multi-out, debug, nixpkgs shape, caches, builders, honesty. Ops and fleet chapters ship the closures you learned to build.

ROS 2 robotics with nix-ros-overlay

ROS 2 robotics with nix-ros-overlay

Goal: Stand up a **reproducible ROS 2 development environment** with [nix-ros-overlay](#)—flakes, binary caches, distro selection (Humble/Jazzy/...), project **devShells**, and a path toward packaging your own nodes—without being locked to Ubuntu’s deb repos.

i Note

This chapter is about the **package manager + overlay**, not a full robotics curriculum. It assumes you know (or will learn elsewhere) ROS 2 concepts: nodes, topics, DDS, workspaces, **colcon**. Here you learn how Nix makes those tools **pinable and portable** on NixOS, Fedora, or any Linux with Nix.

Why this chapter exists

Classic ROS 2 setup:

```
Ubuntu LTS → apt ROS repo → /opt/ros/<distro> → source setup.bash
```

That works—and couples your **OS**, **ROS distro**, and **system Python** forever. Teams then invent Docker stacks so CI can still build.

nix-ros-overlay answers a different question:

Want	Approach
Same ROS graph on Fedora / NixOS / CI	Overlay + flake lock
Multiple ROS distros without dual-boot	Separate devShells
Honest dependency graph	Nix store + hydra status badges
Avoid apt pollution on daily driver	Project shell only

It is an **overlay** on a specific nixpkgs pin (the overlay *follows its own nixpkgs*). Treat it like a specialized ecosystem helper—related to [language ecosystems](#) and [overlays](#), not like `environment.systemPackages = [ ros-desktop-full ]` on every laptop forever.

Mental model

```
nix-ros-overlay flake
```

```
overlays.default    pinned nixpkgs    rosPackages.<distro>
```

```
pkgs = import nixpkgs { overlays = [ ... ]; }
```

```
buildEnv { underlay = true; paths = [ ros-core ... ]; }
```

```
mkShell / nix develop (+ colcon, clang, ...)
```

Piece	Role
Overlay	Injects <code>rosPackages.*</code> and ROS build helpers into <code>pkgs</code>
<code>rosPackages.<distro></code>	Attrset of packages for Humble, Jazzy, Kilted, Lyrical, Rolling, ...
<code>buildEnv { underlay = true; ... }</code>	Composes a ROS underlay environment (AMENT/COLCON paths) for shells
<code>colcon</code>	Workspace build tool (non-ROS nixpkgs package, usually)
Binary caches	Avoid compiling the world; still expect occasional local builds

ROS 1 / Gazebo Classic were removed from `master` after EOL (2025). Historical work lives on branches such as `ros1-25.05`—this chapter is **ROS 2 only**.

Status and expectations (be honest)

Upstream tracks build health per distro on Hydra (see the README badge table on [loposided98/nix-ros-overlay](https://github.com/loposided98/nix-ros-overlay)).

What generally works:

- Large fraction of distro packages on **x86_64-linux** / **aarch64-linux**
- Functional **dev environments** via `nix-shell` / `nix develop`

- Generated package defs via **superflore**

What still hurts:

- Not every package builds (thousands of packages; 80–90% targets are aspirational and drop as distros age)
- Free **ros.cachix.org** capacity is limited—packages may be missing or evicted
- Graphics (RViz, Gazebo/Gz) on non-NixOS needs **nixGL** or **nix-system-graphics**
- macOS is not a first-class story

Warning

Budget **time and disk**. First entry into a desktop-class shell can pull tens of gigabytes when substitutes miss. Prefer a slim **ros-core** shell before **desktop** / **desktop-full**.

Binary caches (do this first)

Official community cache (Cachix)

```
# Option A
cachix use ros

# Option B - nix.conf (user or system)
# substituters = https://cache.nixos.org https://ros.cachix.org
# trusted-public-keys = cache.nixos.org-1:... ros.cachix.org-1:dSyZxI8geDCJrwgvCOHDoAf0m5sV1wC
```

Public key commonly documented by the project:

```
ros.cachix.org-1:dSyZxI8geDCJrwgvCOHDoAf0m5sV1wCPjBkKL+38Rvo=
```

Experimental Attic cache (when Cachix is thin)

Upstream documents an experimental Hydra-fed cache (availability not guaranteed). Example from project docs (verify current values on the README before trusting):

```
extra-substituters = https://attic.iid.ciirc.cvut.cz/ros
extra-trusted-public-keys = ros:JR95vUYsShSqfA1VTYofT1Nz6uXasm5Qrc0sGry9f6Q=
```

On **NixOS**, prefer declarative:

```
# modules/common/nix-ros-cache.nix (sketch)
{
  nix.settings = {
    substituters = [
      "https://cache.nixos.org"
      "https://ros.cachix.org"
      # "https://attic.iid.ciirc.cvut.cz/ros" # optional experimental
    ];
    trusted-public-keys = [
      "cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFGspcDShjY="
      "ros.cachix.org-1:dSyZxI8geDCJrwgvCOHDoAfOm5sV1wCPjBkKL+38Rvo="
      # "ros:JR95vUYsShSqfA1VTYofT1Nz6uXasm5Qrc0sGry9f6Q="
    ];
  };
}
```

Multi-user: **trusted** substituters belong in the **daemon** config (system), not only `~/.config/nix/nix.conf`.

Fast path: try upstream examples (no project yet)

Flake develop (recommended)

```
# ROS 2 Jazzy desktop-class example (name may track upstream flake outputs)
nix develop \
  --option extra-substituters 'https://ros.cachix.org' \
  --option extra-trusted-public-keys 'ros.cachix.org-1:dSyZxI8geDCJrwgvCOHDoAfOm5sV1wCPjBkKL
  github:lopsided98/nix-ros-overlay/master#example-ros2-desktop-jazzy
```

Inside the shell:

```
ros2 --help
# Classic demo (if packages present in that env)
ros2 launch demo_nodes_cpp talker_listener_launch.xml
```

Classic nix-shell form

```
nix-shell \
  -I nix-ros-overlay=https://github.com/lopsided98/nix-ros-overlay/archive/master.tar.gz \
  --option extra-substituters 'https://ros.cachix.org' \
  --option extra-trusted-public-keys 'ros.cachix.org-1:dSyZxI8geDCJrwgvCOHDoAfOm5sV1wCPjBkKL
  '<nix-ros-overlay/examples/ros2-desktop.nix>' --argstr rosDistro jazzy
```


Prefer **flakes** + **lock** for anything you keep.

```
# Scaffold from upstream template
mkdir -p ~/lab/nixos-book/ros2-env && cd ~/lab/nixos-book/ros2-env
nix flake init --template github:lopsided98/nix-ros-overlay
```

Project flake: ROS 2 underlay in a devShell

Critical rule: `nixpkgs.follows = "nix-ros-overlay/nixpkgs"`.

If you pin book baseline `nixos-26.05` *independently*, you will fight ABI/version skew. The overlay chooses a `nixpkgs` it builds against—**follow it**.

Minimal flake.nix

```
{
  description = "ROS 2 lab environment via nix-ros-overlay";

  inputs = {
    nix-ros-overlay.url = "github:lopsided98/nix-ros-overlay/master";
    # IMPORTANT: do not replace this with a free-floating nixos-26.05 input
    nixpkgs.follows = "nix-ros-overlay/nixpkgs";
  };

  outputs = { self, nix-ros-overlay, nixpkgs }:
    nix-ros-overlay.inputs.flake-utils.lib.eachDefaultSystem (system:
      let
        pkgs = import nixpkgs {
          inherit system;
          overlays = [ nix-ros-overlay.overlays.default ];
        };

        # Pick ONE distro per shell (or define multiple shells)
        rosDistro = pkgs.rosPackages.jazzy; # or .humble / .kilted / .lyrical / .rolling

        rosEnv = rosDistro.buildEnv {
          underlay = true;
          paths = with rosDistro; [
            ros-core
            # Grow deliberately:
            # ros-base
            # desktop
            # demo-nodes-cpp
            # rviz2

```

```

        # teleop-twist-keyboard
    ];
};
in {
    devShells.default = pkgs.mkShell {
        name = "ros2-jazzy-lab";
        packages = [
            pkgs.colcon
            pkgs.cmake
            pkgs.gcc
            pkgs.python3
            rosEnv
            # optional quality-of-life
            # pkgs.rosPackages.jazzy.ros2cli # often already via env
        ];
        shellHook = ''
            echo "ROS 2 shell (nix-ros-overlay) - distro attrs under pkgs.rosPackages.*"
            # AMENT/COLCON hooks are provided by buildEnv underlay; if a var is missing,
            # check `env | rg -i 'ament|colcon|ros'` and upstream examples.
        '';
    };

    # Optional second shell: minimal CI-ish
    devShells.ci = pkgs.mkShell {
        name = "ros2-ci";
        packages = [
            pkgs.colcon
            (rosDistro.buildEnv {
                underlay = true;
                paths = with rosDistro; [ ros-core ];
            })
        ];
    };
});

# Flake-level nixConfig is honored by many CLIs; system trusted-keys still matter for mult
nixConfig = {
    extra-substituters = [ "https://ros.cachix.org" ];
    extra-trusted-public-keys = [
        "ros.cachix.org-1:dSyZxI8geDCJrwgvCOHDoAfOm5sV1wCPjBkKL+38Rvo="
    ];
};
}

cd ~/lab/nixos-book/ros2-env
git init

```

```
# paste flake.nix
nix flake lock
nix develop
ros2 --help
```

direnv

```
echo 'use flake' > .envrc
direnv allow
```

Same [direnv discipline](#) as any other project shell—plus larger GC roots. Prune unused roots when disk hurts.

Choosing a ROS 2 distro attr

Names track ROS releases exposed by the overlay (check `pkgs.rosPackages` in `nix repl` when unsure):

Distro attr (typical)	Role
<code>humble</code>	Older LTS-class; good for legacy robots
<code>jazzy</code>	Common current teaching/desktop target
<code>kilted</code> / <code>lyrical</code>	Newer releases as overlay adds them
<code>rolling</code>	Bleeding edge; expect breakage

```
nix develop -c nix eval --raw --impure --expr \
  'let pkgs = import (builtins.getFlake "git+file:///\"$PWD\"").inputs.nixpkgs { system = "x86_64-linux"; };
  # Simpler once inside a shell that has pkgs: use nix repl with the overlay loaded'
```

Practical approach:

```
nix repl
# :lf github:lopsided98/nix-ros-overlay
# then explore outputs / pkgs construction from the flake
```

Or open upstream Hydra badges before pinning a distro for a class.

Workspace workflow (overlay underlay + your packages)

Nix supplies the **underlay** (system ROS packages). Your robot code often still uses a **colcon workspace** on top:

```
~/lab/robot_ws/  
flake.nix          # nix-ros-overlay shell  
.envrc  
src/  
  my_robot_bringup/  
  my_robot_desc/
```

```
nix develop  
mkdir -p src && cd src  
# git clone your packages ...  
cd ..  
colcon build --symlink-install  
source install/setup.bash  
ros2 pkg list | head
```

Why not put *everything* in Nix immediately?

Phase	Reasonable default
Week 1	Overlay underlay + colcon for <i>your</i> packages
Later	Package stable internal libs with ROS Nix builders / overlays
CI	<code>nix develop -c colcon build</code> with lockfile

Building every teammate's WIP branch as pure Nix is optional maturity—not day-one tax.

Custom ROS packages (direction of travel)

Patterns used in the ecosystem (details evolve; check overlay helpers when you implement):

1. **Overlay on the overlay** — add your package set on top of `nix-ros-overlay.overlays.default`
2. **buildRosPackage / distro helpers** — when exported by the overlay version you pin
3. **Upstream first** — prefer nixpkgs for non-ROS deps; fix rosdep → nix name mappings when evaluation complains about `_unresolved_<dep>`

Evaluation error pattern from upstream FAQ:

```
Function called without required argument "_unresolved_<dependency>"
```

Usually means rosdep lacks a `nixos` key for that dependency—map it to a `nixpkgs` package or package the dep.

Do **not** start by forking thousands of generated expressions; override the few packages you need.

Graphics: RViz, Gz, tools on non-NixOS

On **NixOS**, OpenGL/Vulkan integration is usually straightforward if drivers are set up.

On **Fedora / Ubuntu + Nix**, Nix-built GL apps often need a wrapper:

Tool	Idea
nixGL	Wrap <code>rviz2</code> with host GL
nix-system-graphics	System-oriented graphics story

```
# Conceptual - install nixGL per its docs, then:
# nixGL rviz2
```

Put the wrapper in `shellHook` or a small script in the flake so students don't invent flags.

NixOS host integration (what belongs where)

Layer	Recommendation
devShell (this chapter)	Daily robot development
HM packages	Maybe <code>colcon/clangd</code> —not full desktop ROS
environment.systemPackages	Avoid multi-GB ROS desktops system-wide
Robot appliance image	Separate <code>nixosConfigurations.robot</code> with only runtime nodes

Runtime robot image sketch (conceptual):

```
# hosts/robot/configuration.nix - illustration only
{ pkgs, ... }: {
  # Use same overlaid pkgs in the flake's nixosSystem
  environment.systemPackages = [
    # minimal runtime env or specific nodes-not desktop-full
  ];
}
```

```
# networking, users, systemd services that launch ros2 launch ...
}
```

Wire the overlay once at flake level so **devShell** and **robot image** share the same pin.

Fedora dual-stack note

If you followed [Nix on Linux](#):

- Keep **desktop/GPU** on Fedora
- Keep **ROS** in a project flake shell
- Add ROS caches to the **daemon** config
- Do not `dnf install` ROS debs *and* Nix ROS for the same workspace—pick one story per project

Troubleshooting

Symptom	Direction
Massive local compiles	Caches not trusted by daemon; check <code>nix show-config \\\ rg substituter</code>
<code>nixpkgs</code> version chaos	Forgot <code>nixpkgs.follows = "nix-ros-overlay/nixpkgs"</code>
<code>_unresolved_*</code> eval error	Missing rosdep→nix mapping; package or map dependency
Package fails to build	Check Hydra; open issue; override; try another distro
<code>ros2</code> not found in shell	<code>buildEnv</code> paths incomplete; compare upstream example env
RViz black window / GL error	<code>nixGL</code> / drivers; try non-NixOS graphics wrappers
Disk full	<code>nix store gc</code> ; drop <code>desktop-full</code> shells; limit <code>direnv</code> roots
DDS / multicast weirdness	Host firewall/network; <code>ROS_DOMAIN_ID</code> ; not always a Nix bug

```
# Useful inspections
nix flake metadata
nix path-info -Sh .#devShells.x86_64-linux.default 2>/dev/null || true
```

```
env | rg -i 'ament|colcon|ros|rmw' | head
```

Security and supply chain

- Overlay + ROS sources are a **large** trust surface—pin **flake.lock** and review updates
 - Cachix/Attic keys: only add caches you intend to trust for substitutes
 - Robot network: treat DDS discovery like any open LAN service
 - Don't bake robot secrets into store paths (same rule as HM)
-

Lab A — Hello underlay

1. Enable `ros.cachix.org` on your Nix install.
2. `nix develop github:lopsided98/nix-ros-overlay/master#example-ros2-desktop-jazzy` (or current equivalent).
3. Run talker/listener launch; capture a screenshot or log snippet in lab notes.
4. Record closure size: how painful was the first download?

Lab B — Project flake

1. `nix flake init --template github:lopsided98/nix-ros-overlay` in `~/lab/nixos-book/ros2-env`.
2. Trim env to `ros-core` (+ one demo package if available).
3. Add `direnv` + commit `flake.lock`.
4. `colcon build` an empty package skeleton; `ros2 pkg list` shows it after source.

Lab C — Distro compare (optional)

1. Define `devShells.humble` and `devShells.jazzy`.
2. Document one package that exists in one and fails in the other.
3. Write a team policy: “default teaching distro = ...”.

Lab D — Robot appliance sketch (optional)

1. In a monorepo flake, add `nixosConfigurations.ros-bot` sharing the overlay.
 2. System packages only runtime nodes; develop still uses `devShells.default`.
 3. Explain why the robot image is smaller than the desktop shell.
-

Checkpoint

- ☐ You can explain why `nixpkgs.follows` the overlay
 - ☐ Caches configured for multi-user Nix
 - ☐ A project flake enters `ros2` successfully
 - ☐ You know underlay vs colcon workspace
 - ☐ Graphics plan for RViz on your host OS
 - ☐ You did **not** dump `desktop-full` into `environment.systemPackages` without reason
-

Further depth (this book)

Topic	Chapter
Overlays mechanism	Overlays (light)
Project shells	direnv and devShells
Language packaging patterns	Ecosystems A/B
Caches	Binary caches
Fedora dual-stack	Nix on Linux

Topic	Chapter
Reproducibility honesty	Reproducibility

Upstream

- [lopsided98/nix-ros-overlay](#) — README, examples, Hydra badges
- [ros.cachix.org](#) — community substitutes
- ROS 2 docs — nodes/launch/DDS semantics

When this notebook and upstream disagree, **trust the overlay README and your flake.lock.**

Ops and Fleet

Operations and fleet

Operations and fleet

This part of the **standalone NixOS book** is organized by **topic** (normal book chapters), not by calendar day.

Read chapters in order within the part when learning; skip or skim freely when using the book as a reference.

Baseline: Nix **2.34.x**, flakes-first, NixOS **26.05**. Deploy tool CLIs, Disko modes, and generator format names evolve—prefer current upstream READMEs when a flag renames.

What this part is for

Packaging craft produces store paths. **Operations and fleet** ships them safely:

1. **Disks as code** (Disko) so reinstalls are boring
2. **Impermanence** patterns so roots stop accumulating snowflakes
3. **Deploy tools** (deploy-rs, Colmena) for remote activation
4. **Shared modules** so ten hosts do not fork ten firewalls
5. **Images** (nixos-generators) for cloud and edge
6. **Terraform boundary** so cloud APIs and NixOS stay friends
7. **Tests and CI** so regressions die in PRs
8. **GC hygiene, Secure Boot/TPM awareness, backups** so the fleet survives Monday

Without this part, NixOS is a clever single host. With it, NixOS is an operations platform.

Prerequisites

Skill	Source
Flake host, rebuild, generations	NixOS host part
Modules as consumer and author	NixOS host part
Secrets (sops-nix or equivalent)	Services and security
Binary caches / builders vocabulary	Packaging craft
Optional: packaging your own modules' packages	Packaging craft

You need **at least one disposable machine or VM**. Disko and impermanence are unsafe on irreplaceable disks.

Chapter map

Chapter	File	Focus
Disko	01-disko	Declarative partitioning / filesystems
Impermanence	02-impermanence	Ephemeral root + persist lists
deploy-rs	03-deploy-rs	Profile deploy over SSH
Colmena	04-colmena	Hive, tags, multi-node
Shared modules	05-shared-modules	Roles, profiles, host thinness
Images / generators	06-images-generators	ISO, qcow, cloud images
Terraform boundary	07-terraform-boundary	TF provisions; Nix configures
nixosTest	08-nixos-tests	VM integration tests
More testing	09-more-tests	Check suite / pyramid
CI pipelines	10-ci-pipeline	Forge CI + flakes + cache
GC hygiene	11-gc-hygiene	Roots, generations, optimise
Secure Boot / TPM	12-secure-boot-tpm	Lanzaboote awareness, LUKS+TPM
Backups	13-backups	Restic/Borg/ZFS + restore drill

Suggested path: **Disko** → **Impermanence** → **deploy-rs** → **Colmena** → **shared modules**, then **images** → **terraform**, then **tests** → **CI**, finish with **GC** / **SB-TPM** / **backups**.

Mental model — fleet pipeline

```
modules + host data
```

```
nixosConfigurations / hive
```

```
deploy-rs / Colmena / nixos-rebuild  
nixos-generators images  
nixosTest VMs
```

```
Disko layouts + (optional) impermanence  
secrets activation (sops-nix / agenix)  
binary cache + remote builders
```

Stage	Failure mode	First fix locus
Eval	Module merge errors	Shared modules / host
Build	Compile / missing cache	Packaging / CI cache
Copy	SSH / disk	Deploy tool / network
Activate	Switch scripts, boot	Host modules
Prove	Tests / canaries	nixosTest / live probes
Survive	Disk full / no backup	GC / backups

Disk and identity

```
Disko describes blocks
```

```
Impermanence describes what must survive reboot
```

```
Backups describe how state returns after disaster
```

```
Secure Boot/TPM describes boot trust and unlock
```

These four chapters answer different questions. Confusing “declarative disk” with “backed up database” is a classic Capstone failure mode.

Deploy tools at a glance

Tool	Strength	Typical use
<code>nixos-rebuild</code> <code>--target-host</code>	Built-in	One-off / small
deploy-rs	Profiles, magic rollback patterns	Small-medium fleets
Colmena	Hive, tags, parallel apply	Multi-role fleets

You do not need both forever. Leave this part with a **deliberate choice** and one working path.

Lab topology suggestions

```
[admin laptop]
SSH
  lab-vm-1 (Disko reinstall target)
  lab-vm-2 (second Colmena node or remote builder)
optional: cache VM (Harmonia/Attic)
```

Cloud stretch:

```
[OpenTofu] → VM + disk + SG → [image from nixos-generators] → deploy-rs
```

Suggested workspace root:

```
~/lab/nixos/ops/
  disko/
  impermanence/
  deploy/
  colmena/
  modules/          # may be your main flake
  images/
  terraform/
  tests/
  ci/
  gc/
  secureboot/
  backups/
```

Many labs live **inside one monorepo flake** with `nixosConfigurations`, `deploy`, `colmena`, and `checks`—prefer that once you have more than two hosts.

How chapters are structured

1. **Goal** — done criteria
 2. **Why it matters** — ops motivation
 3. **Theory** — models + complete Nix fragments
 4. **Worked examples** — 26.05-shaped
 5. **Lab** — multi-step, disposable infrastructure
 6. **Pitfalls** — symptom → fix
 7. **Checkpoint** — self-review
 8. **Commit** — optional discipline
-

Reading paths

Path A — “Homelab single server → clean reinstall”

1. Disko
2. Impermanence (or conscious reject)
3. deploy-rs
4. Backups
5. GC hygiene

Path B — “Three nodes with roles”

1. Shared modules
2. Colmena (or deploy-rs multi-node)
3. nixosTest + more tests
4. CI pipeline
5. Binary cache (packaging part)

Path C — “Cloud image factory”

1. Shared modules
 2. Images / generators
 3. Terraform boundary
 4. CI pipeline
 5. Secure Boot awareness (if metal/edge)
-

Glossary (part-local)

Term	Meaning
Disko	Declarative disk partitioning/formatting for NixOS
Impermanence	Ephemeral root; explicit persist paths
Profile	Nix profile activation target (system generation)
Hive	Colmena multi-node definition
Tag	Colmena selection label (@web)
nixosTest	QEMU VM integration test harness
nixos-generators	Build disk/ISO/cloud images from modules
Boundary	Clear split TF vs Nix responsibilities
GC root	Reference that keeps store paths alive
Lanzaboote	Secure Boot integration for NixOS generations
Restore drill	Prove backups by restoring, not only by job logs

Safety rules (non-negotiable)

Rule	Why
Disposable disks for Disko experiments	Wipes are real
Recovery USB before Secure Boot enrollment	Bricks happen
Never paste production secrets into examples	Supply chain + git history
PR forks often must not get push tokens	Cache/signing TCB
GC without understanding roots	Can remove needed paths; rarely deletes running system if roots correct—but can remove rollback options

Quality bar for “fleet ready”

Criterion	Evidence
Storage is code	Disko config in git; reinstall notes
Hosts are thin	Roles/profiles, not copy-paste
Remote activate works	deploy-rs or Colmena success log
Regressions caught	<code>nix flake check</code> / CI green
State is honest	Backup + restore drill document
Disk won't fill silently	GC policy + monitoring note
Boot trust known	SB/TPM dossier or enablement notes

Relationship to other parts

Part	Connection
NixOS host	Single-host skills scale to fleets here
Services / security	Secrets, firewall, TLS feed deploy targets
Packaging	Caches/builders feed CI and deploy speed
Internals	Deeper module/eval performance later
Capstone	Combines ops chapters into a rebuildable system
Projects	Optional drills; this part is the ops spine

No day-calendar language; no parallel series cross-links. This is the standalone library's operations volume.

Anti-patterns

Anti-pattern	Prefer
Hand-edited <code>/etc</code> on prod nodes	Module + deploy
Unique firewall per host by copy-paste	Shared profile
Image modules forked from live hosts	One module graph → image + deploy
TF <code>user_data</code> installing Docker via curl	Golden NixOS image + Nix modules
Tests only on laptop	CI <code>nix flake check</code>
Backup job green without restore	Restore drill
“We'll enable Secure Boot later” with no notes	Written path + recovery plan

Part checkpoint (before you leave ops)

- ☐ Disko layout for at least one disposable host; reinstall or format proof
- ☐ Impermanence implemented **or** written reject with inventory of what must persist
- ☐ One successful remote deploy (deploy-rs and/or Colmena)
- ☐ Roles/profiles refactor started (hosts thinner than before)
- ☐ One image artifact built from shared modules **or** documented stretch plan
- ☐ Terraform/Nix boundary sketch written
- ☐ At least one nixosTest (or documented KVM blocker)
- ☐ Three-check suite shape in flake
- ☐ CI workflow runs checks on the forge you use
- ☐ GC policy + roots understanding demonstrated
- ☐ Secure Boot/TPM awareness dossier
- ☐ Backup tool chosen + **restore drill** documented

Suggested lab commit series

```
git commit -m "ops: disko layout and reinstall notes"
git commit -m "ops: impermanence or reject rationale"
git commit -m "ops: deploy-rs end-to-end"
git commit -m "ops: colmena hive and tags"
git commit -m "ops: shared roles and profiles"
git commit -m "ops: nixos-generators image"
git commit -m "ops: terraform boundary sketch"
git commit -m "ops: nixosTest for role module"
git commit -m "ops: flake check suite"
git commit -m "ops: CI workflow"
git commit -m "ops: GC policy"
git commit -m "ops: secure boot / TPM notes"
```

```
git commit -m "ops: backup + restore drill"
```

How to use this part as reference

Symptom	Jump to
Reinstall tribal knowledge	Disko
Root full of mystery state	Impermanence
Need SSH activate	deploy-rs
Need tags / many hosts	Colmena
Copy-paste hosts	Shared modules
Cloud boot drift	Images; Terraform boundary
“Works on my VM”	nixosTest; more tests; CI
Disk full mid-deploy	GC hygiene
Evil maid / datacenter unlock	Secure Boot / TPM
DB gone after rebuild	Backups

Next

Start with **Disko** on a disposable VM: declare ESP + root, format, install from a flake, and prove a second reinstall from the same code. Everything else in this part assumes storage is no longer folklore.

Disko

Disko

Goal: Describe disks declaratively with **Disko**, install or reinstall a lab host from that description, and wire Disko into your flake so storage stops being tribal knowledge.

! Important

Disko can **wipe disks**. Use a disposable VM, spare drive, or cloud instance. Never experiment on a disk with irreplaceable data.

Disko idea: disk layout as data · installer applies it



Figure 1: Disko layout

Why it matters

Installer GUIs and one-shot `hardware-configuration.nix` fragments do not scale. Fleet habits require **repeatable storage**: partitions, LVM, btrfs/ZFS layouts, LUKS—as code. Disko is the common 2026 answer in many NixOS fleets.

Without Disko (or equivalent), disaster recovery docs say “partition like last time” and nobody remembers last time.

Theory 1 — What Disko is

[Disko](#) turns a Nix attrset into:

1. Partitioning / formatting scripts
2. Mount configuration compatible with NixOS
3. Optionally integrated into `nixos-rebuild` / installer flows

```
disko.devices = { ... }

    disko script (destroy/create/mount)
    fileSystems / boot / swap NixOS options
```

Piece	Role
<code>disko.devices.disk.*</code>	Physical (or virt) disks
<code>content.type</code>	<code>gpt</code> , <code>luks</code> , <code>btrfs</code> , <code>filesystem</code> , ...
<code>mountpoint</code>	Becomes NixOS <code>fileSystems</code> entries
CLI modes	<code>destroy</code> / <code>format</code> / <code>mount</code> / <code>disown</code> (names evolve—check <code>--help</code>)

Disko is **not** a backup tool. It describes structure; data durability is the backups chapter.

Theory 2 — Minimal GPT + ext4 root (UEFI)

```
# modules/disko-lab.nix
{
  disko.devices = {
    disk = {
      main = {
        type = "disk";
        device = "/dev/vda"; # CHANGE per machine: nvme0n1, sda, ...
        content = {
          type = "gpt";
          partitions = {
            ESP = {
              size = "512M";
              type = "EF00";
              content = {
                type = "filesystem";
                format = "vfat";
                mountpoint = "/boot";
                mountOptions = [ "umask=0077" ];
              };
            };
          };
        };
      };
    };
  };
}
```

```

    root = {
      size = "100%";
      content = {
        type = "filesystem";
        format = "ext4";
        mountpoint = "/";
      };
    };
  };
};
};
};
};
};
}

```

Field	Notes
device	Host-specific; inject via host module
type = "gpt"	UEFI-friendly
ESP EF00	EFI system partition type
mountpoint	Becomes NixOS fileSystems

Prefer stable device paths

```
device = "/dev/disk/by-id/nvme-..."; # better than /dev/nvme0n1 when possible
```

Virt devices (/dev/vda) are fine in VMs; metal likes by-id.

Theory 3 — Flake integration

```

# flake.nix fragment
{
  inputs.disko.url = "github:nix-community/disko";
  inputs.disko.inputs.nixpkgs.follows = "nixpkgs";

  outputs = { self, nixpkgs, disko, ... }: {
    nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      modules = [
        disko.nixosModules.disko
        ./hosts/lab/disko.nix
        ./hosts/lab/configuration.nix

```



```

];
};
};
}

```

```

# From installer ISO or rescue, after networking + clone:
sudo nix --extra-experimental-features "nix-command flakes" run github:nix-community/disko -
  --mode destroy,format,mount \
  --flake .#lab
# modes/flags evolve-check current Disko CLI help

```

Then install:

```
sudo nixos-install --flake .#lab
```

Or use documented `disko-install` flows when available in your Disko version.

Evaluate mounts without wiping

```

nix eval .#nixosConfigurations.lab.config.fileSystems --json | jq 'keys'
# ensure / and /boot appear as expected after disk module merge

```

Theory 4 — LUKS sketch (common homelab)

```

root = {
  size = "100%";
  content = {
    type = "luks";
    name = "crypted";
    settings.allowDiscards = true; # SSDs; understand TRIM tradeoffs
    content = {
      type = "filesystem";
      format = "ext4";
      mountpoint = "/";
    };
  };
};

```

Unlock method	Notes
Passphrase	Simple lab; interactive boot

Unlock method	Notes
Key file	Automation; protect the key
TPM2	Secure Boot/TPM chapter

Password vs TPM unlock: passphrase in this chapter’s lab; TPM path appears later. **Store recovery keys offline** before production encryption.

Theory 5 — btrfs subvolumes (impermanence-ready)

```
content = {
  type = "btrfs";
  subvolumes = {
    "@" = {
      mountpoint = "/";
    };
    "@home" = {
      mountpoint = "/home";
    };
    "@nix" = {
      mountpoint = "/nix";
    };
    "@persist" = {
      mountpoint = "/persist";
    };
    "@log" = {
      mountpoint = "/var/log";
    };
  };
};
```

Impermanence loves `@persist` + wipe `@` on boot. You can create the layout even if you skip impermanence initially—future-you will thank you.

ZFS note

Disko can express ZFS layouts; complexity jumps (pools, datasets, encryption). If you already run ZFS operationally, Disko still helps **repeat** the layout; if you are learning both ZFS and Disko, do `ext4/btrfs` first.

Theory 6 — Host-specific devices without copy-paste hell

```
# modules/disko-btrfs-template.nix
{
  disko.devices.disk.main = {
    type = "disk";
    # device set per host
    content = { /* shared layout */ };
  };
}

# hosts/lab/disko.nix
{ lib, ... }:
{
  imports = [ ../../modules/disko-btrfs-template.nix ];
  disko.devices.disk.main.device = lib.mkDefault "/dev/vda";
}

# hosts/metal/disko.nix
{
  imports = [ ../../modules/disko-btrfs-template.nix ];
  disko.devices.disk.main.device = "/dev/disk/by-id/nvme-...";
}
```

Never commit a fleet-wide wrong device path.

Theory 7 — Swap, BIOS, and multi-disk

Swap

```
swap = {
  size = "8G";
  content = {
    type = "swap";
    resumeDevice = true; # if you want hibernation carefully
  };
};
```

Or prefer `zramSwap` in NixOS config and skip disk swap on small VMs—**decide intentionally**.

Legacy BIOS / hybrid

UEFI is the default story here. BIOS/GPT or MBR layouts exist in Disko examples—follow current docs if you must boot old hardware.

Multi-disk

```
disk.main → system
disk.data → /var/lib/postgres bulk (example)
```

Keep system vs data responsibilities clear in DISK.md.

Theory 8 — Interaction with hardware-configuration.nix

After Disko-driven install, generated hardware config may still detect NICs and kernel modules. **Filesystem mounts** should come from Disko's merge, not a conflicting hand-maintained `fileSystems` block.

Source of truth	For
Disko	partitions, formats, mountpoints
hardware-configuration / nixos-generate-config	often CPU microcode, kernel modules, NIC
Host module	hostname, networking, roles

Conflicts: two modules defining different `fileSystems."/"` → merge errors or surprise boots.

Worked example — VM lab checklist

1. Create empty qcow2 / virt disk
2. Boot NixOS 26.05 installer ISO
3. Git clone flake (or nix copy)
4. Identify disk: `lsblk / by-id`
5. Set device in disk config
6. Run disk (destructive)
7. `nixos-install --flake .#lab`
8. Reboot; verify `findmnt / lsblk`

9. Wipe once more; reinstall; time it

```
lsblk -f
findmnt /
# source of truth is Nix; /etc/fstab is generated view
cat /etc/fstab
```

DISK.md template

```
# DISK.md
## lab
- device: /dev/vda (virtio)
- layout: GPT ESP 512M + ext4 root
- LUKS: no
- disk command that saved us: ...
- recovery: installer ISO + flake URL + this file
```

Lab — multi-step

Suggested workspace: your main lab flake + `~/lab/nixos/ops/disko` notes

Step 1 — Snapshot safety

Confirm target disk is disposable. `lsblk` screenshot/paste into notes.

Step 2 — Add Disko input

Pin via flake; follows `nixpkgs`; update lock.

Step 3 — Write simplest UEFI layout

`ext4 root` + ESP. Parameterize device.

Step 4 — Dry evaluation

```
nix eval .#nixosConfigurations.lab.config.fileSystems --json | jq 'keys'
```

Step 5 — Apply on VM

Run `Disko + install` or rebuild path if reinstalling an already-Disko host per docs.

Step 6 — Reinstall proof (core)

Wipe once more (VM) and reinstall from same flake. Time it. Note pain points.

Step 7 — Optional LUKS or btrfs

Add one complexity only if Step 6 succeeded cleanly.

Step 8 — Document recovery

`DISK.md`: device names per host, LUKS key location, `disko` command that saved you.

Step 9 — Wrong-disk tabletop

Write a five-line “stop the line” checklist if `lsblk` shows unexpected serials.

Exercises

1. Convert your layout from `device = "/dev/vda"` to a `by-id` path in a metal story (even if hypothetical).
 2. Sketch a three-disk server (boot/system/data) as `Disko attrset stubs`.
 3. Explain why `Disko` does not replace backups (one paragraph).
 4. Diff `fileSystems` eval before and after adding a swap partition.
 5. Time: first install vs second reinstall—what was slow (download, format, copy)?
-

Common gotchas

Symptom / mistake	What to do
Wiped wrong disk	Device IDs; prefer <code>/dev/disk/by-id/...</code>
No EFI boot	Missing ESP / wrong bootloader module
Device path differs after hardware change	by-id; update host module
Forgot disk module import	<code>fileSystems</code> empty/wrong
Swap forgotten	Add swap partition or zram intentionally
Cloud images	Device often <code>/dev/vda</code> or <code>/dev/nvme0n1</code> ; check vendor docs
Conflicting <code>fileSystems</code>	One source of truth
LUKS without recovery key	Create and store offline before prod
Impermanence without <code>@persist</code>	Plan subvolumes early

Checkpoint

- ☐ Disko input wired in flake
- ☐ Declarative layout for lab host committed
- ☐ At least one real format/install or reinstall on disposable media
- ☐ `fileSystems` matches intent after boot
- ☐ `DISK.md` recovery notes written
- ☐ Device path strategy chosen (by-id vs virt path)
- ☐ Optional LUKS/btrfs attempted or deferred with reason

Commit

```
git add .
git commit -m "ops: disk layout and reinstall proof"
```

Write three personal gotchas before impermanence.

Impermanence (idea and patterns)

Impermanence (idea and patterns)

Goal: Implement **or** consciously reject root filesystem impermanence using **impermanence** patterns—proving that unpersisted state disappears on reboot while **/persist** (or equivalent) keeps what you declare.

💡 Tip

Impermanence is optional as a lifestyle. It is **not** optional to understand. Skipping still requires a written “why.” Disk layouts pair with Disko; secrets with sops-nix/agenix. Target: **NixOS 26.05**.

Why it matters

Snowflakes are not only packages—they are **mutated roots**: **/etc** edits, **/var** surprises, leftover packages. Impermanence makes the root a camping tent and **/persist** the shipping container. Combined with Disko btrfs/ZFS layouts, reinstalls become boring—and disaster recovery gets a cleaner story.

Even if you never wipe root on a daily driver, the **inventory habit** (what must survive reboot?) is permanent skill.

Theory 1 — Model

boot

```
tmpfs or fresh subvolume for /  
bind/mount persisted paths from /persist  
everything else is recreated by activation
```

Path class	Examples	Strategy
Store	<code>/nix</code>	Own subvolume/partition; never wipe carelessly
Persist	<code>/persist/var/lib/postgres</code> , ssh host keys	Explicit list
Ephemeral	<code>/var/cache</code> , junk in <code>/root</code>	Accept loss

Path class	Examples	Strategy
Secrets	sops age keys	Persist carefully; permissions matter
Boot	EFI, kernels, generations	Separate story; do not “wipe /boot” casually

What “disappears” means

On reboot, anything not in the persist list is gone *if* you actually wipe/reset root. Soft modes that only document policy without wipe are still useful—but do not claim impermanence until the reboot proof works.

Theory 2 — impermanence module (nix-community)

```
# flake input
inputs.impermanence.url = "github:nix-community/impermanence";

{
  imports = [ inputs.impermanence.nixosModules.impermanence ];

  environment.persistence."/persist" = {
    hideMounts = true;
    directories = [
      "/var/log"
      "/var/lib/nixos"
      "/var/lib/systemd/coredump"
      { directory = "/var/lib/tailscale"; mode = "0700"; }
    ];
    files = [
      "/etc/machine-id"
      "/etc/ssh/ssh_host_ed25519_key"
      "/etc/ssh/ssh_host_ed25519_key.pub"
    ];
    users.alice = {
      directories = [ "Downloads" ".ssh" ];
      # files = [ ".config/sops/age/keys.txt" ]; # only if you accept this layout
    };
  };
}
```

Exact option paths can track module versions—confirm against current impermanence README when options shift.

neededForBoot

```
fileSystems."/persist".neededForBoot = true;
```

If `/persist` is not available early enough, activation ordering fails in mysterious ways. Treat this as non-negotiable for real setups.

Theory 3 — btrfs rollback-on-boot pattern

Common production pattern:

1. Root subvolume `@`
2. On boot, optionally reset `@` to empty snapshot
3. Mount `@persist` at `/persist`
4. Impermanence binds declared paths

```
# Conceptual boot.initrd or systemd unit logic-use a maintained pattern
# Prefer copying a pattern you understand over opaque one-liners.
# Pair with Disko btrfs subvolumes from the Disko chapter.
```

Layout piece	Role
<code>@</code>	Ephemeral root
<code>@persist</code>	Declared state
<code>@nix</code>	Store (never auto-wipe)
EFI	Bootloader

ZFS native datasets with rollback are an alternative school—pick **one** for the lab. Mixing half-understood btrfs blog posts with half-understood ZFS is how you reinstall for the wrong reason.

Theory 4 — What breaks first (expect it)

Symptom	Cause	Fix
Cannot SSH after reboot	Host keys not persisted	Persist <code>/etc/ssh/ssh_host_*</code>

Symptom	Cause	Fix
Machine ID churn sops fails	<code>/etc/machine-id</code> ephemeral Age key path wiped	Persist or accept Persist key path; mode 600
Docker/podman volumes gone User linger/services odd	State under <code>/var/lib</code> systemd user state	Persist selected dirs Persist carefully or redesign
Tailscale identity reset	State dir wiped	Persist with tight mode
NetworkManager connections gone	Connection profiles ephemeral	Persist or declarative NM
ACME/certs confusion	Live cert state paths	Persist or use DNS-01 carefully

Iterate: wipe/reboot is a **test suite**, not a one-shot.

Theory 5 — Soft impermanence

Not ready to wipe root? Still gain discipline:

```
# Document-only policy
# - no manual /etc edits
# - all state dirs listed in inventory
# - weekly diff against expected paths
```

Or tmpfs for `/tmp` only—training wheels.

Level	What you prove
A Full	Wipe + persist list + reboot proof
B Partial	Persist mounts without wipe; inventory honest
C Skip	Written architecture why mutable root is OK

All three are valid **if documented**. Silent skip is not.

Theory 6 — Interaction with generations and secrets

Impermanence does **not** replace Nix generations. Store rollbacks and root wipes are different axes:

```

Generation rollback → previous system closure
Root wipe           → empty / except binds from /persist
Secrets             → must live on persist or offline media

```

Scenario	Tool
Bad module activation	Generation rollback
Unknown files on root	Impermanence wipe
Lost age key	Offline backup; real pain if missing

Theory 7 — Home Manager alignment

If HM manages `~/.config/...` on an impermanent home, either:

- Persist the HM-managed paths you care about, or
- Accept regenerate-from-flake behavior, or
- Keep home on a persistent dataset

Mismatch symptoms: “my HM files vanished” after reboot. Align Home Manager layouts with persist lists.

Services checklist (starting set)

Path	Usually
<code>/etc/ssh/ssh_host_*</code>	Persist
<code>/var/lib/nixos</code>	Persist
<code>/var/log</code>	Persist or ship logs off-box
<code>/var/lib/postgresql</code>	Persist + backups
<code>/var/cache</code>	Ephemeral
<code>/tmp</code>	Ephemeral
sops key	Persist, mode 600
<code>/var/lib/docker</code>	Prefer not to; redesign or persist knowingly

Worked example — minimal VM proof

```
# hosts/imperm-lab/configuration.nix fragments
{ inputs, ... }:
{
  imports = [
    ./disko-btrfs.nix
    inputs.impermanence.nixosModules.impermanence
    ./persist.nix
  ];

  fileSystems."/persist".neededForBoot = true;

  users.users.root.openssh.authorizedKeys.keys = [
    "ssh-ed25519 AAAA... lab"
  ];
  services.openssh.enable = true;
  system.stateVersion = "26.05";
}
```

```
# persist.nix
{
  environment.persistence."/persist" = {
    hideMounts = true;
    directories = [
      "/var/log"
      "/var/lib/nixos"
    ];
    files = [
      "/etc/machine-id"
      "/etc/ssh/ssh_host_ed25519_key"
      "/etc/ssh/ssh_host_ed25519_key.pub"
    ];
  };
}
```

Proof protocol:

```
echo "ephemeral-should-die" | sudo tee /root/SCRATCH
echo "persist-me" | sudo tee /persist/PROOF
sudo reboot
# after boot:
cat /root/SCRATCH      # should fail
cat /persist/PROOF     # should work
ssh lab true          # should still work if keys persisted
```

Capture both outcomes in notes. A proof that only tests the success path is incomplete.

Worked example — IMPERMANENCE.md template

```
# IMPERMANENCE.md
## Level
A / B / C

## Inventory (path → persist|regenerate|ignore)
- ...

## Wipe mechanism
btrfs reset @ / tmpfs / none

## Proof
- date:
- SCRATCH gone: yes/no
- PROOF present: yes/no
- SSH after reboot: yes/no

## Secrets
- age key path:
- offline backup location:

## Why not full wipe (if B/C)
- ...
```

Lab — multi-step

Suggested notes: ~/lab/nixos/ops/impermanence

Step 1 — Decide commit level

Level	Action
A Full	btrfs/ZFS + impermanence module + wipe proof
B Partial	Persist module without wipe; list future wipe
C Skip	Written architecture review why not (still do path inventory)

Record choice in IMPERMANENCE.md.

Step 2 — State inventory

On current lab host:

```
sudo du -xh /var/lib 2>/dev/null | sort -h | tail -n 30
ls /etc/ssh
systemctl --user list-units --failed 2>/dev/null || true
```

Classify each important path: **persist** / **regenerate** / **ignore**.

Step 3 — Implement level A or B

Wire flake input; persist SSH keys + one service state dir you actually use. Set modes deliberately.

Step 4 — Proof reboot

Create SCRATCH vs PROOF files; reboot; record outcomes with timestamps.

Step 5 — Secrets path

Confirm sops-nix/agenix key path survives (or document regenerate procedure and offline backup location).

Step 6 — Rollback story interaction

Write 5 lines: store generations vs root wipes.

Step 7 — If Level C

Write one page: threat model, backup story, why mutable root is acceptable for this host class, and what would trigger revisiting (e.g. multi-user, kiosk, fleet sameness).

Step 8 — Persist list review

Print effective list; remove anything that is “comfort mutable root.” Goal is intentional survival, not nostalgia.

Step 9 — Second surprise reboot

After a week of use (or a forced second cycle), reboot again; capture new paths that “should have been persisted.” Update the list.

Exercises

1. Map every path under `/var/lib` larger than 100 MB to persist/regenerate/ignore.
 2. Design persist modes for a postgres host vs a stateless web edge.
 3. Explain why wiping `/nix` is not impermanence (one paragraph).
 4. Write a failure story: age key on ephemeral root—detection and recovery.
 5. Compare tmpfs-root vs btrfs-reset-root (pros/cons table).
-

Common gotchas

Symptom / mistake	What to do
<code>/persist</code> not neededForBoot	Boot fails ordering; set flag
Persisted too little	Iterate after each surprise reboot
Persisted too much	You rebuilt a mutable root in disguise
Home Manager vs system paths	Align HM files with persist lists
LUKS + wipe scripts	Order of unlock vs mounts
Forgetting <code>/nix</code> separation	Do not wipe the store subvolume
SSH host key regenerate	Clients scream TOFU; persist keys
Age key mode 644	Tighten; treat as root-equivalent
Claiming impermanence without wipe	Call it policy, not impermanence

Checkpoint

- ☐ Level A/B/C chosen and documented
- ☐ State inventory exists
- ☐ Reboot proof (or skip rationale) recorded
- ☐ SSH still works after reboot if remote
- ☐ Secrets path understood

- ☐ Generations vs wipe note written
- ☐ Persist list reviewed for “too much”

Commit

```
git add .  
git commit -m "ops: impermanence level and reboot proof"
```

Write three personal gotchas before deploy-rs.

deploy-rs

deploy-rs

Goal: Deploy a flake-defined NixOS host to a remote machine with **deploy-rs**, understand activation profiles, and succeed at least once end-to-end over SSH. Target config: **NixOS 26.05**.

i Note

`nixos-rebuild --target-host` works; fleets still standardize on deploy tools for profiles, magic rollback, and multi-node later. **deploy-rs** is a common choice for small-to-medium flakes before (or beside) Colmena.

Deploy path: flake → build → push → switch



`deploy-rs` · colmena · `nixos-rebuild --target-host` · always pin `flake.lock`

Lab: host builds · nixlab/deploy-a receive · snapshots before scary switches

Figure 1: Deploy flow

Why it matters

Pushing a closure and activating it **without** sitting on the machine is the core of fleet ops. This chapter installs the boring muscle memory: SSH auth, profile activation, proof file, second delta, and a safe failure story.

Theory 1 — Deploy pipeline

```
local eval (flake) → build (local/remote) → copy closure → activate → health check → keep/ro
```

Step	Failure mode
Eval	Module errors
Build	Compile / missing cache
Copy	SSH/disk
Activate	Boot/switch scripts fail
Confirm	Custom checks fail → auto rollback (when configured)

Understanding *where* it failed is half the job—read the stage in the log before rewriting modules blindly.

```

deploy-rs
  builds profile path P (NixOS system)
  copies closure of P to target
  activates P (switch-to-configuration)
  optional confirm / magic rollback window

```

Theory 2 — Flake skeleton with deploy-rs

```

{
  description = "deploy-rs lab";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    deploy-rs.url = "github:serokell/deploy-rs";
    deploy-rs.inputs.nixpkgs.follows = "nixpkgs";
  };

  outputs = { self, nixpkgs, deploy-rs, ... }:
    let
      system = "x86_64-linux";
    in {
      nixosConfigurations.lab-vm = nixpkgs.lib.nixosSystem {
        inherit system;
        modules = [ ./hosts/lab-vm/configuration.nix ];
      };

      deploy.nodes.lab-vm = {
        hostname = "lab-vm.example.internal"; # or IP
        profiles.system = {
          user = "root";
          path = deploy-rs.lib.${system}.activate.nixos
            self.nixosConfigurations.lab-vm;
        };
      };
    };

```

```

    };
};

checks = builtins.mapAttrs
  (system: deployLib: deployLib.deployChecks self.deploy)
  deploy-rs.lib;
};
}

```

```

nix shell github:serokell/deploy-rs -c deploy .#lab-vm
# or after packaging deploy-rs in a shell:
deploy .#lab-vm

```

Pin via flake input so CI and laptop match. Exact CLI entrypoints: prefer current serokell docs for your pin.

Theory 3 — SSH prerequisites

```

ssh root@lab-vm.example.internal nix --version
# or deploy user with passwordless sudo / root activation per your policy

```

Requirement	Notes
Nix on target	NixOS has it; confirm version band
Flakes enabled	On target as needed for activation
Disk space	Closures are large first time
Network to substituters	Or pre-pushed cache
Non-interactive auth	Keys, not passwords
Same arch (or builder)	Arch mismatch fails painfully

```

# target snippets
services.openssh.enable = true;
users.users.root.openssh.authorizedKeys.keys = [
  "ssh-ed25519 AAAA... deploy"
];
# Prefer dedicated deploy key; hardening still applies
system.stateVersion = "26.05";

```

Theory 4 — Magic rollback / confirmation

deploy-rs can require confirmation that the new system is healthy; if SSH dies, it rolls back. Read current serokell docs for `confirmTimeout` / `magic-rollback` options for your version—**enable something** before testing firewall changes remotely.

```
deploy.nodes.lab-vm = {
  hostname = "192.0.2.50";
  # sshOpts = [ "-p" "22" ];
  profiles.system = {
    user = "root";
    path = deploy-rs.lib.x86_64-linux.activate.nixos
      self.nixosConfigurations.lab-vm;
    # magicRollback / confirmTimeout per current docs
  };
};
```

Always keep a **console/VNC** path on the first remote firewall experiment. Magic rollback is not a substitute for out-of-band access.

If this fails...	Look at
Eval on laptop	modules, flake inputs
Build	caches, builders, arch
Copy	SSH, disk, bandwidth
Activate	systemd, scripts, secrets paths
Confirm timeout	network cut by your own firewall

Theory 5 — Build locality

Mode	Idea
Build local, copy	Laptop/CI builds; target only activates
Build on target	Sometimes via remote builder / deploy options
Build on remote builder + cache	Packaging machinery

For lab: local build + copy is fine if architectures match. Cross-arch without a builder is pain—fix remote builders first if needed.

Cache interaction

Deploy once with cold cache (or after GC on target), once with warm substitutes. Record wall times. Deploy without cache is a social incident for fleets.

Theory 6 — Profiles beyond system

deploy-rs supports multiple **profiles** per node (system is the common first). Later you might activate user/home profiles; master **one** system profile end-to-end first.

Profile idea	Role
system	NixOS activation
Future custom	App-only activations (advanced)

Do not invent multi-profile complexity until single profile is boring.

Theory 7 — vs nixos-rebuild --target-host

```
nixos-rebuild switch --flake .#lab-vm --target-host root@lab-vm
```

Concern	rebuild --target-host	deploy-rs
Built-in	Yes	Flake input
Magic rollback ergonomics	Limited	Stronger story
Multi-node	Manual	deploy.nodes
Checks	Manual	deployChecks
Learning cost	Low	Medium

Write five lines on when you prefer each after you have used both once.

Worked example — trivial visible change

```
# hosts/lab-vm/configuration.nix
{ pkgs, ... }:
{
  networking.hostName = "lab-vm";
  environment.systemPackages = [ pkgs.hello pkgs.git ];
  environment.etc."deploy-proof".text = "deployed-by-deploy-rs\n";
  services.openssh.enable = true;
```



```
    system.stateVersion = "26.05";  
}
```

```
deploy .#lab-vm  
ssh lab-vm cat /etc/deploy-proof
```

Second deploy: change the text; redeploy; confirm update. Two successful deploys beat one lucky shot.

Generation awareness

```
ssh lab-vm -- sudo nix-env -p /nix/var/nix/profiles/system --list-generations | tail
```

Note generation number before/after deploy. You cannot rollback what you never listed.

Worked example — RUNBOOK seed

```
# Deploy lab-vm  
1. VPN/SSH path reachable  
2. nix flake lock status known  
3. deploy .#lab-vm  
4. health: cat /etc/deploy-proof && systemctl is-system-running  
5. if fail: console access; list generations; rollback
```

Lab — multi-step

Suggested notes: ~/lab/nixos/ops/deploy

Step 1 — Target ready

Disposable NixOS VM with SSH key auth. Snapshot the VM **before** first deploy.

Step 2 — Add deploy-rs input

follows nixpkgs; lock; commit.

Step 3 — Wire `deploy.nodes`

Match `nixosConfigurations` name; set hostname/IP. Prefer IP in lab DNS chaos.

Step 4 — First deploy

```
nix flake check    # includes deployChecks if wired
deploy .#lab-vm
# -s / skip-checks flags: know them; don't live on skip forever
```

Capture full terminal output in notes.

Step 5 — Prove activation

```
ssh lab-vm cat /etc/deploy-proof
ssh lab-vm nixos-version
ssh lab-vm readlink /run/current-system
```

Step 6 — Second deploy delta

Change `environment.etc` text; redeploy; confirm update. Note wall time vs first deploy (cache effects).

Step 7 — Failure drill (safe)

Deploy a config that adds a harmless broken oneshot **or** bad package, observe failure handling; restore from snapshot if needed. Do **not** lock yourself out on purpose without console.

Step 8 — Compare one-liner

Once, run classic `nixos-rebuild --target-host`. Write comparison notes.

Step 9 — Document RUNBOOK seed

As above.

Step 10 — Generation awareness after deploy

List generations; record numbers.

Step 11 — Cache interaction

Cold vs warm deploy times.

Step 12 — Multi-node sketch (optional)

Add a second node attr in `deploy.nodes` even if only one is real—prove eval shape.

Exercises

1. Draw the pipeline and mark where magic rollback acts.
 2. Design a deploy key policy (who has it, rotation).
 3. Estimate first-deploy closure size for your lab host (`nix path-info -Sh`).
 4. Write a “deploy failed at copy” runbook paragraph.
 5. Compare deploy-rs vs Colmena for a 2-node vs 20-node mental model (preview).
-

Common gotchas

Symptom / mistake	What to do
Arch mismatch	Build on same system or use remote builder
Permission denied	root keys / sudo deploy user
Out of disk on target	GC; grow disk
Hang on activation	Console; magic rollback timeouts
<code>deploy</code> not found	<code>nix shell</code> the tool
Hostname DNS	Use IP for lab
Checks fail locally	Fix <code>deployChecks</code> or understand impure needs
<code>follows</code> forgotten	Mixed nixpkgs between deploy-rs and host
Firewall lockout	Console first; enable magic rollback
Secrets missing on target	sops keys / paths must exist before service start

Checkpoint

- ☐ deploy-rs in flake
- ☐ Successful remote activation
- ☐ Visible file/package change verified over SSH
- ☐ Second delta deploy succeeded
- ☐ Comparison note vs `nixos-rebuild --target-host`
- ☐ RUNBOOK seed written
- ☐ Generation numbers recorded
- ☐ Magic rollback / confirm awareness

Commit

```
git add .  
git commit -m "ops: deploy-rs end-to-end"
```

Write three personal gotchas before Colmena.

Colmena

Colmena

Goal: Express a small hive with **Colmena**, deploy at least two nodes (one may be local), use tags, and know when Colmena wins over deploy-rs for fleet shape. Hosts pin **NixOS 26.05**.

i Note

deploy-rs is excellent node/profile oriented glue. **Colmena** shines when you think in **hives**: many hosts, tags (`@web`, `@db`), parallel apply, and a flake hive attribute. Real orgs often evaluate both; leave ops with a deliberate choice, not two half-wired tools forever.

Why it matters

One remote deploy proves the pipe. Fleets need **selection**, **defaults**, and **blast-radius control**. Colmena's mental model maps cleanly onto roles you will build in the shared modules chapter.

Theory 1 — Hive mental model

```
meta (nixpkgs, specialArgs)
  nodes
    host-a → NixOS module stack
    host-b → ...
  defaults → shared modules
```

Concept	Role
<code>meta.nixpkgs</code>	Package set for evaluation
<code>defaults</code>	Applied to every node
<code>nodes.<name></code>	Per-host config + deployment keys
<code>tags</code>	Selection subsets for apply
<code>specialArgs</code>	Inject libs/secrets helpers carefully

Theory 2 — Flake-based Colmena (modern)

```
{
  description = "colmena hive lab";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs, ... }: {
    colmena = {
      meta = {
        nixpkgs = import nixpkgs { system = "x86_64-linux"; };
        # specialArgs = { inherit inputs; };
      };

      defaults = { pkgs, ... }: {
        # shared baseline modules
        imports = [ ../modules/profiles/baseline.nix ];
        system.stateVersion = "26.05";
        deployment.tags = [ "all" ];
      };

      edge = { name, nodes, ... }: {
        deployment = {
          targetHost = "edge.lab.internal";
          targetUser = "root";
          tags = [ "web" "edge" ];
        };
        imports = [ ../hosts/edge/configuration.nix ];
      };

      db = {
        deployment = {
          targetHost = "db.lab.internal";
          targetUser = "root";
          tags = [ "db" ];
        };
        imports = [ ../hosts/db/configuration.nix ];
      };
    };
  };
}
```

CLI shapes evolve—typical patterns:

```
nix shell nixpkgs#colmena -c colmena --help
colmena build
```

```
colmena apply --on @web
colmena apply-local --sudo # local node stories
```

Confirm current Colmena flake attribute and CLI against upstream docs for your pin.

Theory 3 — Tags and blast radius

Tag pattern	Use
@all / all	Everything (dangerous)
@web	Edge nodes
@db	Data plane
@canary	Single host first

```
# Conceptual - flags vary by version
colmena apply --on @web
colmena apply --on edge
```

Rule: Never default to “apply entire hive” on Friday. Prefer tag + canary.

Parallelism

Colmena can apply many nodes in parallel. Great for speed; terrible for correlated mistakes. Start serial for scary changes (firewall, disk, boot).

Theory 4 — Evaluation vs activation

```
colmena build    → realize all selected system closures
colmena apply    → build + copy + switch on targets
```

Command family	Risk
build/eval only	Low—catches module errors
apply one tag	Medium
apply all	High

Wire flake checks that **eval** the hive even when you do not apply.

Theory 5 — Relationship to nixosConfigurations

Many flakes expose both:

```
nixosConfigurations.edge = nixpkgs.lib.nixosSystem { ... };  
# colmena.nodes.edge imports same modules
```

Approach	Pros
Single module entry per host	DRY
Generate Colmena from nixosConfigurations	Less drift
Separate trees	Fast way to diverge—avoid

Shared modules chapter makes this maintainable.

Theory 6 — When Colmena vs deploy-rs

Situation	Lean
1–3 nodes, profile/rollback focus	deploy-rs
Many nodes, tags, parallel ops	Colmena
Already invested in one	Don't dual-run forever
Need both temporarily	Document primary

Write a decision paragraph in notes after you deploy with both once (or one deeply).

Theory 7 — Secrets and specialArgs

```
meta = {  
  nixpkgs = import nixpkgs { system = "x86_64-linux"; };  
  specialArgs = { inherit inputs; };  
};
```

Inject	Careful of
inputs	Large eval; fine when needed
Secret paths	OK
Secret strings	Never—store contamination

Inject	Careful of
--------	------------

sops-nix still decrypts on the host; Colmena only delivers config that references paths.

Worked example — two-node hive (one local)

```
colmena = {
  meta.nixpkgs = import nixpkgs { system = "x86_64-linux"; };

  defaults = {
    deployment.tags = [ "all" ];
    services.openssh.enable = true;
    system.stateVersion = "26.05";
  };

  laptop = {
    deployment = {
      allowLocalDeployment = true;
      targetHost = null; # local patterns per docs
      tags = [ "local" ];
    };
    environment.etc."hive-proof".text = "laptop\n";
  };

  vm = {
    deployment = {
      targetHost = "192.0.2.50";
      targetUser = "root";
      tags = [ "vm" "web" ];
    };
    environment.etc."hive-proof".text = "vm\n";
    imports = [ ./hosts/vm.nix ];
  };
};

colmena apply --on @vm
ssh root@192.0.2.50 cat /etc/hive-proof
```

Lab — multi-step

Suggested notes: ~/lab/nixos/ops/colmena

Step 1 — Install Colmena in a shell

```
nix shell nixpkgs#colmena -c colmena --version
```

Step 2 — Define hive with defaults

Baseline SSH + stateVersion + one shared import.

Step 3 — Two nodes

One may be local; one remote VM. Distinct `hive-proof` texts.

Step 4 — Build only

```
colmena build  
# or build --on @vm
```

Fix eval errors before apply.

Step 5 — Apply tagged subset

Apply only @vm (or equivalent). Prove file on remote.

Step 6 — Tag experiment

Add @web; apply by tag; document selection syntax you used.

Step 7 — Blast-radius note

Write: “scary change procedure” = canary host → tag → all.

Step 8 — Decision card

Colmena vs deploy-rs for *your* lab size.

Step 9 — Optional parallel apply

Apply two nodes; note ordering/logs; decide if parallel is for you.

Step 10 — DRY modules

Point both nodes at a shared profile module (preview of shared modules chapter).

Exercises

1. Design tags for web/db/obs/canary.
 2. Eval-only CI job sketch for the hive.
 3. Failure mode: wrong `targetHost`—what do logs show?
 4. Convert a deploy-rs node list into a Colmena hive sketch.
 5. Security: who can run `colmena apply --on @all`?
-

Common gotchas

Symptom / mistake	What to do
Applied all nodes by accident	Tags; confirm prompts; canary
Hive eval huge/slow	Reduce specialArgs; modularize
Drift vs nixosConfigurations	Share modules
Local deploy confusion	Read allowLocalDeployment docs
SSH user wrong	targetUser + keys
Mixed nixpkgs	Single meta.nixpkgs
Parallel firewall change	Serial + console
Secrets in hive attrset	Paths only

Checkpoint

- ☐ Hive evaluates
- ☐ At least one remote (or local) apply succeeded
- ☐ Tags used for selection
- ☐ Build-only vs apply distinction clear
- ☐ Decision note vs deploy-rs
- ☐ Blast-radius procedure written

Commit

```
git add .  
git commit -m "ops: colmena hive and tags"
```

Write three personal gotchas before shared modules.

Shared modules across hosts

Shared modules across hosts

Goal: Refactor fleet config into **roles** and **profiles** so hosts become thin compositions—no copy-paste SSH/firewall/observability blocks.

Why it matters

Two Colmena nodes already tempt duplication. Ten nodes without roles guarantee drift: one host missing a hardening line is an incident. Modules are the compression algorithm of NixOS fleets.

Theory 1 — Layering

```
nixpkgs modules
  your library modules (roles/, profiles/, services/)
    host modules (hosts/edge)
      secrets / hardware / disk (host-specific)
```

Layer	Contains	Avoid
Role	web, db, observer — purpose	Hardware paths
Profile	baseline, hardened, workstation	One-off hacks
Host	hostname, addresses, disk, keys	Recreating baseline
Service module	Single app options	Global side effects silent

Theory 2 — Role module pattern

```
# modules/roles/web.nix
{ config, lib, pkgs, ... }:
let
  cfg = config.roles.web;
in {
  options.roles.web = {
```

```

    enable = lib.mkEnableOption "web edge role";
    domain = lib.mkOption {
      type = lib.types.str;
      example = "service.example.com";
    };
  };

  };

  config = lib.mkIf cfg.enable {
    networking.firewall.allowedTCPPorts = [ 80 443 ];
    services.nginx.enable = true;
    # services.nginx.virtualHosts.${cfg.domain} = { ... };
    environment.systemPackages = [ pkgs.curl ];
  };
}

# hosts/edge/configuration.nix
{
  imports = [
    ../../modules/profiles/baseline.nix
    ../../modules/roles/web.nix
  ];
  roles.web = {
    enable = true;
    domain = "edge.example.com";
  };
  networking.hostName = "edge";
}

```

Hosts should read like a **shopping list**, not a novel.

Theory 3 — Profile module pattern

```

# modules/profiles/baseline.nix
{ lib, pkgs, ... }:
{
  services.openssh = {
    enable = true;
    settings.PasswordAuthentication = false;
  };
  environment.systemPackages = [ pkgs.vim pkgs.git pkgs.htop ];
  time.timeZone = lib.mkDefault "UTC";
  i18n.defaultLocale = lib.mkDefault "en_US.UTF-8";
}

```



```
# modules/profiles/hardened.nix
{
  imports = [ ./baseline.nix ];
  # tighter ssh, sysctl, etc.-compose, don't fork baseline forever
}
```

Profile	Intent
baseline	Every host
hardened	Prod-ish
workstation	GUI/dev laptops
lab	Relaxed for learning

Theory 4 — Options API design

Practice	Why
<code>mkEnableOption</code>	Clear on/off
Typed options	Catch typos at eval
<code>mkDefault</code> in profiles	Hosts can override
<code>mkForce</code> sparingly	Fight club otherwise
Document example	Next human

```
options.roles.db = {
  enable = lib.mkEnableOption "database role";
  package = lib.mkPackageOption pkgs "postgresql_16" { };
  listenAddresses = lib.mkOption {
    type = lib.types.listOf lib.types.str;
    default = [ "127.0.0.1" ];
  };
};
```

Theory 5 — Directory layout that scales

```
flake.nix
modules/
  profiles/
    baseline.nix
```

```

    hardened.nix
roles/
  web.nix
  db.nix
services/
  myapp.nix
common/
  sops.nix
  networking-lab.nix
hosts/
  edge/
    configuration.nix
    disko.nix
    hardware.nix
  db/
    configuration.nix
    disko.nix

```

Rule	Detail
No <code>hosts/*</code> copying firewalls	Profiles/roles
Disko/hardware only under host	Physical truth
Secrets wiring shared; keys host-specific	sops creation rules

Theory 6 — SpecialArgs and lib helpers

```

nixpkgs.lib.nixosSystem {
  specialArgs = { inherit inputs; myLib = import ./lib; };
  modules = [ ./hosts/edge ];
};

```

Keep helpers pure and small. A 2k-line `lib/default.nix` becomes a second language.

Theory 7 — Testing shared modules

Roles without tests drift. Preview:

```

# tests/web-role.nix - nixosTest imports roles.web
nodes.machine = {

```

```
imports = [ ../modules/roles/web.nix ];
roles.web.enable = true;
roles.web.domain = "test.local";
};
```

Full harness in the `nixosTest` chapter—design modules so tests can import them without host hardware.

Worked example — before/after host thinness

Before (bad)

```
# hosts/edge/configuration.nix - 200 lines of ssh, firewall, nginx, users...
```

After (good)

```
{
  imports = [
    ./hardware.nix
    ./disko.nix
    ../../modules/profiles/hardened.nix
    ../../modules/roles/web.nix
    ../../modules/common/sops.nix
  ];
  networking.hostName = "edge";
  networking.hostId = "deadbeef"; # if needed
  roles.web = {
    enable = true;
    domain = "edge.example.com";
  };
  # host-only networking addresses, etc.
  system.stateVersion = "26.05";
}
```

Measure: host file line count before/after; aim for sharp drop.

Worked example — flake wiring both deploy tools

```
# same modules feed:
# - nixosConfigurations.edge
# - colmena.edge
# - deploy.nodes.edge
# - nixos-generators image for edge
```

One module graph is the entire point of this chapter.

Lab — multi-step

Suggested work: your fleet flake

Step 1 — Inventory duplication

```
rg -n "openssh|allowedTCPPorts|nginx" hosts modules | head
```

List copy-pasted blocks.

Step 2 — Extract profiles/baseline.nix

SSH + packages + stateVersion defaults. Import from two hosts.

Step 3 — Extract one role

roles.web or roles.lab-tooling with enable flag.

Step 4 — Thin hosts

Move host-specific only. Commit before/after line counts in notes.

Step 5 — Override test

Use lib.mkForce or host option once deliberately; document who wins.

Step 6 — Wire into Colmena/deploy-rs

Prove both nodes still deploy/eval.

Step 7 — Option docs

Add `example` and a one-line comment per role option.

Step 8 — Negative test

Enable role without required option; ensure eval fails clearly.

Step 9 — README map

```
# modules/README.md
profiles/ - ...
roles/ - ...
```

Exercises

1. Design options for a `roles.observer` (metrics scrapes only).
 2. Draw merge priority: baseline vs hardened vs host.
 3. Find three nixpkgs modules that are good style models; note patterns.
 4. Refactor a third host using only imports + three option sets.
 5. Write a “no mkForce without comment” team rule.
-

Common gotchas

Symptom / mistake	What to do
Role enables services globally without <code>mkIf</code>	Always gate on <code>cfg.enable</code>
Host still 500 lines	Keep extracting
Option name typos	Types + eval tests
Circular imports	Layer strictly
Hardware in roles	Move to host
Silent <code>mkForce</code> wars	Prefer defaults + host overrides
<code>specialArgs</code> secret strings	Paths only
Untested roles	<code>nixosTest</code> next

Checkpoint

- ☐ Baseline profile shared by 2 hosts
- ☐ At least one role module with options
- ☐ Hosts thinner (measured)
- ☐ Deploy/hive still evaluates
- ☐ modules README map
- ☐ Clear enable-option gating

Commit

```
git add .  
git commit -m "ops: shared roles and profiles"
```

Write three personal gotchas before images.

Images with nixos-generators

Images with nixos-generators

Goal: Build a bootable artifact (cloud image, ISO, or VM format) from your flake modules using **nixos-generators** (and related patterns), then boot it once and prove it is the *same* module graph as a live host—not a snowflake fork.

i Note

Images are **first-class deploy targets**, not a side quest. The win is one module tree → install path *and* golden image path. NixOS **26.05** pin stays consistent with the rest of this book.

Why it matters

Not every node is `nixos-install` by hand. Cloud and edge want **images**: qcow2, AMIs (via pipelines), raw disks, installer ISOs. Generating images from the **same modules** as deploy targets closes a huge consistency gap—and catches “works on my VM but not in the AMI” earlier.

Without an image story you either:

- Hand-maintain cloud user-data that reimplements half of NixOS, or
- Drift image hosts away from Colmena/deploy-rs nodes until nobody trusts either.

Theory 1 — Image = NixOS config + format

```
same modules as hosts/edge
```

```
nixos-generators / make-disk-image / sd-image
```

```
artifact (iso, qcow2, raw, gce, azure, ...)
```

Format (examples)	Use
iso	Installer / live recovery
qcow / qcow2 / vmware / virtualbox	Hypervisors

Format (examples)	Use
amazon / gce / azure	Cloud (tooling evolves)
sd-card / raw	SBC / dd

Check the **current** nixos-generators format list for your pin—names and defaults change. Prefer the project README over stale blog posts.

Mental model

An image build is still a NixOS evaluation: modules merge, activation scripts exist, store paths land on a disk layout the *format* owns. You are not “exporting files”; you are **realizing a system derivation into a disk picture**.

Theory 2 — nixos-generators integration shapes

Flake app / package patterns

Community patterns include:

```
# Conceptual - verify against current nixos-generators docs
inputs.nixos-generators.url = "github:nix-community/nixos-generators";
inputs.nixos-generators.inputs.nixpkgs.follows = "nixpkgs";

nix run github:nix-community/nixos-generators -- --help
# format flags and flake integration evolve
```

Module specializations

Images often need:

Concern	Image-friendly approach
Hostname	Generic or cloud-init later
SSH keys	Inject via cloud-init / metadata; or bake lab keys carefully
Disko	May differ from bare-metal Disko; know format’s disk story
Size	Minimal profiles; avoid workstation GUIs on cloud images
First boot	growpart, regenerate machine-id policies

Theory 3 — Same modules, different entrypoints

```
# modules shared
imports = [
  ../modules/profiles/baseline.nix
  ../modules/roles/web.nix
];

# hosts/edge - deploy target
# images/edge - generators entry with format-specific tweaks only
```

Allowed in image entry	Forbidden without reason
Format modules	Entirely different nginx config
Cloud agent packages	Hardcoded prod secrets
Temporary serial console	Divergent firewall policy forever

If image and live host diverge by 200 lines, you have two products.

Theory 4 — Cloud-init and metadata boundary

Terraform/cloud metadata often supplies:

- SSH keys
- Hostname
- Network

NixOS modules should **consume** those mechanisms or disable conflicting agents intentionally.

Smell	Fix
500-line bash user-data	Bake image; tiny user-data
Image with prod secrets	Inject at instance start via sops/metadata carefully
No SSH to new instance	Console + known image user/key story

Theory 5 — Build performance and caches

Image builds are **heavy**. Use:

- Binary caches for the system closure
- Remote builders with disk
- CI only on main or nightly for full images; PR builds eval/smaller checks

```
nix build .#images.edge-qcow -L
ls -lh result*
```

Theory 6 — ISO as recovery

A flake-built installer ISO that includes your SSH key and network helpers is a **recovery product**:

```
laptop dies → boot ISO → clone flake → Disko → install
```

Pair with Disko chapter reinstall proofs. Store ISO hashes in runbooks if you distribute them.

Worked example — qcow lab image (shape)

```
# Illustrative structure - adapt to current generators API
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  # inputs.nixos-generators = ...;

  outputs = { self, nixpkgs, ... }: {
    nixosConfigurations.edge-image = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      modules = [
        ./modules/profiles/baseline.nix
        ./modules/roles/web.nix
        ./images/edge-image.nix
      ];
    };
    # packages.x86_64-linux.edge-qcow = ... generators expression ...
  };
```

```

}

# images/edge-image.nix
{ lib, modulesPath, ... }:
{
  imports = [
    (modulesPath + "/profiles/qemu-guest.nix")
  ];
  services.openssh.enable = true;
  # lab only:
  users.users.root.openssh.authorizedKeys.keys = [ "ssh-ed25519 AAAA... lab" ];
  system.stateVersion = "26.05";
}

```

Boot under qemu:

```

qemu-system-x86_64 -m 2048 -enable-kvm -drive file=./result/nixos.qcow2,if=virtio
# paths vary by generator output layout

```

Prove:

```

ssh root@... cat /etc/os-release
# compare packages/options to live edge host where possible

```

Worked example — consistency checklist

```

# IMAGE-CONSISTENCY.md
- Module list shared with hosts/edge: yes/no
- Divergences (intentional): ...
- Format: qcow2 / iso / ...
- Build command:
- Boot proof date:
- SSH method:

```

Lab — multi-step

Suggested notes: ~/lab/nixos/ops/images

Step 1 — Pick a format

qcow2 for local hypervisor is the friendliest lab path. ISO if you care about recovery.

Step 2 — Wire generators input

follows nixpkgs; read current README for flake examples.

Step 3 — Image module entry

Import the **same** baseline/role as a live host. Only format-specific imports extra.

Step 4 — Build artifact

```
# per current docs
nix build ... -L
du -h result*
```

Step 5 — Boot once

QEMU/libvirt/VirtualBox—whatever you have. Reach SSH or console.

Step 6 — Consistency proof

Compare `nixos-version`, hostname policy, and one role-provided package/service to the live host story.

Step 7 — Size diet (optional)

Remove fat packages; rebuild; note image size delta.

Step 8 — Document CI policy

When do you rebuild images? Who may publish them?

Step 9 — Stretch cloud

If credentials exist, note AMI/GCE path as stretch—do not block on cloud.

Exercises

1. List five things that must never be baked into a published image.
 2. Design a versioning scheme for golden images (flake rev + date).
 3. Explain why Disko for metal and cloud image root disk may differ.
 4. Sketch a pipeline: CI builds qcow → uploads → TF launches.
 5. Recovery ISO contents checklist (network, git, ssh, disk).
-

Common gotchas

Symptom / mistake	What to do
Format name changed	Read current generators docs
Image huge	Minimal profile; multi-out not enough—fewer packages
Cannot SSH	Console; authorizedKeys; cloud-init race
Divergent modules	Re-unify with live host
Build OOM/disk	Builder sizing; GC
Secrets in image	Remove; inject later
Slow every PR	Cache; nightly images
Wrong system arch	Match hypervisor/cloud

Checkpoint

- ☐ Generators (or equivalent) wired
- ☐ Image built from shared modules
- ☐ Booted once with proof
- ☐ Consistency notes vs live host
- ☐ CI/publish policy sketched
- ☐ No secrets baked intentionally

Commit

```
git add .  
git commit -m "ops: nixos-generators image from shared modules"
```

Write three personal gotchas before Terraform boundary.

Terraform boundary

Terraform boundary

Goal: Draw a clean boundary—**Terraform (or OpenTofu) provisions cloud resources; NixOS owns system configuration**—and produce a sketch you could implement without turning TF into a second module system. Images from the generators chapter + deploy from deploy-rs/Colmena sit on the Nix side; pin OS stories to **26.05**.

i Note

Full multi-cloud automation is out of scope. A **sharp sketch** with one example resource graph is the bar. Implement only if you already have cloud credentials and want the stretch.

Why it matters

Nix people overreach into provisioning; TF people template snowflake user-data forever. The 2026 happy path for many teams:

Terraform/OpenTofu: VPC, VM, disks, DNS, IAM, object storage

outputs: IP, IDs, SSH target

Nix flake: Disko/image/deploy-rs/Colmena configure the OS

Without a boundary doc, every new hire reinvents the split—and usually badly.

Theory 1 — Responsibility split

Concern	Owner	Why
Cloud account / IAM	TF	Lifecycle & policy engines
VM size, subnet, SG	TF	Provider API native
Disk attach / snapshots API	TF	Cloud object model
Packages, services, users	Nix	Pure rebuild story
SSH hardening	Nix	Modules
App releases	Nix / CI	Closures
DNS record for service	Often TF	Or Nix + provider with care
Golden AMI/GCE image build	Nix + CI	Images chapter

Concern	Owner	Why
Instance launch from image	TF	Provider resource

Smell: 500-line `user_data` shell scripts installing docker by curl.

Smell: Nix trying to create AWS VPCs with ad-hoc `runCommand` and keys in the store.

Theory 2 — Handoff patterns

Pattern A — Image + launch

```
CI: nix build image → upload AMI/GCE image
TF: instance from image ID
Nix: optional second-phase deploy-rs for fast iteration
```

Pattern B — Generic distro image + first deploy

```
TF: stock NixOS or minimal image + SSH key
deploy-rs/Colmena: converge to desired closure
```

Pattern C — NixOS on metal (no TF)

```
Disko + install + deploy-rs
TF only for adjacent cloud DNS/objects if any
```

Pattern	Best when
A	Immutable cattle, slow-changing base
B	Fast module iteration
C	Homelab / colo

Theory 3 — What to put in TF state vs flake lock

Artifact	System of record
VPC ID, instance ID	TF state (remote backend)
nixpkgs rev, module graph	<code>flake.lock</code> + git
Deploy proof file	Nix modules
Object storage bucket for backups	TF create; Nix restic config consumes name
DNS API token	Secret manager; neither plaintext git

Never put TF state in the Nix store. Never put flake lock “truth” only in a random S3 object without git.

Theory 4 — OpenTofu vs Terraform

For this book, treat **OpenTofu** and **Terraform** as interchangeable at the boundary level. Choose based on org license/policy. Examples may say TF generically.

```
# versions.tf - illustrative
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    # provider pins...
  }
}
```

Theory 5 — Minimal resource graph (sketch)

```
[network]
  vpc
  subnet
  security_group (22 from admin CIDR; 80/443 public if web)
[compute]
  instance (ami/image from Nix pipeline or stock)
  eip / dns A record
[data]
  data_volume (optional)
  backup_bucket
```

```
# pseudo-HCL - not a full provider config
resource "aws_instance" "edge" {
  ami            = var.nixos_ami # produced by image pipeline
  instance_type  = "t3.small"
  subnet_id      = aws_subnet.main.id
  vpc_security_group_ids = [aws_security_group.edge.id]
  key_name       = var.ssh_key_name # or cloud-init only

  tags = {
    Role = "edge"
    Nix  = "flake-managed"
  }
}

output "edge_ip" {
  value = aws_instance.edge.public_ip
}

# after apply
deploy .#edge # hostname from TF output
```

Theory 6 — Secrets across the boundary

Secret	Where
Cloud provider credentials	TF env / CI OIDC—not flake
SSH deploy key	Agent / CI secret—not world-readable
sops age keys	Host persist / secure bootstrap
DB passwords	sops-nix on Nix side after instance exists

TF `sensitive` outputs still land in state—protect state backends.

Theory 7 — Anti-patterns gallery

Anti-pattern	Prefer
TF <code>remote-exec</code> apt-get	NixOS image + modules

Anti-pattern	Prefer
Nix <code>pkgs.writeScript</code> creating AWS resources with static keys	TF + IAM roles
Duplicating SG rules in nginx modules only	TF SG + Nix firewall both intentional
Mutating instance packages by hand post-TF	redeploy Nix
One mega root module for all envs	workspaces/stacks + clear envs

Worked example — BOUNDARY.md template

```
# BOUNDARY.md
## Owners
- Cloud resources: Terraform/OpenTofu in repo `infra/`
- OS/app: Nix flake in repo `nix/`

## Handoff
1. TF apply → outputs IP, instance id, bucket name
2. Update flake host inventory / sops if needed
3. deploy-rs/Colmena apply
4. Health checks

## Image pipeline
- Built by: CI job ...
- Consumed by: TF var nixos_ami

## Non-goals
- TF will not install packages
- Nix will not create VPCs

## Secrets
- ...
```

Worked example — inventory bridge

```
# hosts/edge/deployment-ip.nix - generated or hand-updated from TF output
{
  # Prefer automation later; honesty first
  # deployment target:
```

```
# networking...
}  
  
tofu output -raw edge_ip > /tmp/edge_ip  
# feed deploy hostname
```

Automating TF output → flake is optional stretch; **document the manual bridge** first.

Lab — multi-step

Suggested notes: ~/lab/nixos/ops/terraform

Step 1 — Write BOUNDARY.md

Fill the template even with zero cloud resources.

Step 2 — Draw resource graph

ASCII or mermaid for one environment (lab).

Step 3 — Choose pattern A/B/C

Record why for your context.

Step 4 — Optional implement

If credentials exist: one VM + SG + output IP; deploy Nix once.

Step 5 — User-data audit

If any user-data remains, cap it at <20 lines or delete.

Step 6 — State backend note

Where would state live? Who accesses it?

Step 7 — Failure story

“TF created VM, deploy failed”—recovery steps.

Step 8 — Stretch destroy

`tofu destroy` only for disposable lab; never prod without ceremony.

Exercises

1. Assign 15 concerns to TF vs Nix vs Shared in a table.
 2. Rewrite a fictional 80-line user-data into image+modules bullets.
 3. Design IAM least privilege for CI that builds AMIs.
 4. Explain why flake lock is not TF state (one paragraph).
 5. Multi-env: how do you stop lab AMI IDs leaking into prod TF?
-

Common gotchas

Symptom / mistake	What to do
Drift: hand-edited security group	TF is source; import or recreate
Deploy to wrong IP	Outputs + inventory discipline
Secrets in TF console output logs	sensitive + log hygiene
NixOS firewall vs cloud SG confusion	Both; document layers
AMI stale	Image pipeline cadence
Two teams, no BOUNDARY.md	Write it
<code>remote-exec</code> growth	Delete; bake image

Checkpoint

- ☐ BOUNDARY.md written
- ☐ Resource graph sketched
- ☐ Pattern A/B/C chosen

- ☐ Handoff steps listed
- ☐ Optional real TF apply or explicit skip
- ☐ User-data policy stated
- ☐ State backend thought through

Commit

```
git add .  
git commit -m "ops: terraform/nix boundary sketch"
```

Write three personal gotchas before nixosTest.

nixosTest

nixosTest

Goal: Write and run a **nixosTest** VM test that exercises **your** module (proxy, hardened SSH, or role from the shared modules chapter)—not only a copy-paste hello world.

Why this day exists

nixos-rebuild on a pet proves little about the next pet. **nixosTest** boots one or more QEMU VMs from your modules, runs shell Python/Perl test scripts, and fails CI when regressions land. It is the highest leverage test type in NixOS land for system behavior.

Theory 1 — What a nixosTest is

```
NixOS configs (nodes) → build VM images → boot under QEMU → testScript → pass/fail
```

Piece	Role
nodes	Attrset of NixOS module stacks
testScript	Python (modern) driving VMs
name	Test identity
Driver	nixos/lib/testing.nix machinery

Requires **KVM** (or slow emulation) and often builder feature **kvm** / **nixos-test**.

Theory 2 — Minimal single-node test

```
# tests/ssh-hardening.nix
{
  name = "ssh-hardening";
  nodes.machine = { pkgs, ... }: {
    imports = [ ../modules/profiles/hardened.nix ];
```

```

users.users.alice = {
  isNormalUser = true;
  openssh.authorizedKeys.keys = [
    "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIExampleKeyForTestOnly not-a-real-key"
  ];
};
services.openssh.enable = true;
};

testScript = ''
  machine.wait_for_unit("sshd.service")
  machine.succeed("grep -q 'PasswordAuthentication no' /etc/ssh/sshd_config")
  # or sshd -T output depending on version
  machine.succeed("sshd -T | grep -i 'passwordauthentication no'")
'';
}

```

Wire into flake:

```

checks.x86_64-linux.ssh-hardening =
  nixpkgs.lib.nixos.runTest {
    imports = [ ./tests/ssh-hardening.nix ];
    hostPkgs = import nixpkgs { system = "x86_64-linux"; };
    # API: prefer current nixpkgs `runTest` / `nixosTest` helper for 26.05
  };

```

i Note

nixpkgs has migrated helpers (`pkgs.testers.runNixOSTest`, `lib.nixos.runTest`, classic `nixosTest`). Use whatever your **26.05** pin documents; the **shape** (nodes + testScript) remains the skill.

Classic style still seen in blogs:

```

import nixpkgs {
  system = "x86_64-linux";
  config = { allowUnfree = false; };
} .lib.nixosSystem { } # not this

# classic:
pkgs.nixosTest {
  name = "example";
  nodes.machine = { ... };
  testScript = '...';
}

```

Theory 3 — testScript essentials

```
machine.wait_for_unit("multi-user.target")
machine.wait_for_open_port(22)
machine.succeed("true")
machine.fail("false")
machine.copy_from_host("file", "/tmp/file")
print(machine.succeed("uname -a"))
```

Multi-node:

```
nodes = {
    server = { ... };
    client = { ... };
};

server.wait_for_unit("caddy.service")
client.succeed("curl -f http://server/")
```

Network is usually a test VLAN; hostnames often equal node names.

Theory 4 — What to test (high value)

Good	Weak
Module assertions about sshd config	<code>true</code> only
Service reaches active + port open	Only that evaluation succeeds
Client→server HTTP 200	Full browser suites
Firewall rejects unexpected port	Entire Kubernetes

Test **your** contracts from the hardening chapter-38 / 61.

Theory 5 — Running tests

```
nix build .#checks.x86_64-linux.ssh-hardening -L
# or
nix flake check -L
```

Interactive debug (when supported by driver):

```
nix build .#checks... -L
# many setups: ./result/bin/nixos-test-driver with breakpoints in testScript

machine.shell_interact() # if available in your nixpkgs test driver
```

Worked example — web role smoke test

```
# tests/web-role.nix
{
  name = "web-role";
  nodes.server = { ... }: {
    imports = [
      ../modules/profiles/baseline.nix
      ../modules/roles/web.nix
    ];
    roles.web = {
      enable = true;
      domain = "server"; # or localhost style per caddy config
    };
  };
  nodes.client = { pkgs, ... }: {
    environment.systemPackages = [ pkgs.curl ];
  };
  testScript = ''
    server.wait_for_unit("caddy.service")
    server.wait_for_open_port(80)
    client.succeed("curl -f --retry 5 --retry-delay 1 http://server/ | grep -q ok")
  '';
}
```

Adapt to your actual reverse proxy module from Stage IV.

Lab — multi-step

Suggested workspace: fleet flake `tests/`; notes in `~/lab/nixos/ch64`

Step 1 — Confirm virtualization

```
ls /dev/kvm && echo kvm-ok || echo "need kvm or nested virt"
```

Step 2 — Hello test

Minimal machine with `environment.etc."t".text = "ok";` and `succeed("test -f /etc/t")`.

Step 3 — Port your module

SSH hardening **or** web role **or** custom Stage IV module.

Step 4 — Make it fail once

Break the module; observe red test; fix.

Step 5 — Wire checks

`nix flake check` runs it (may be heavy—ok).

Step 6 — Document runtime

Wall clock; note CI machine requirements (the CI pipelines chapter).

Step 7 — Multi-node stretch

Client/server only if single-node is green.

Theory 6 — Tests as flake checks

```
checks.${system}.lab-ssh = self.nixosConfigurations.lab.config.system.build.toplevel; # heavy
# better: dedicated nixosTest derivation
checks.${system}.web-smoke = pkgs.nixosTest { ... };
```

Wiring tests into `checks` makes `nix flake check` the Stage VI/CI gate. Keep at least one **fast** smoke and quarantine long tests behind explicit attrnames.

Theory 7 — Interactive vs pure test runs

```
nix build .#checks.x86_64-linux.web-smoke -L
# driver interactively (when supported by your setup):
# nix run .#checks....driverInteractive
```

Interactive drivers help author tests; CI should run non-interactive builds.

Worked example — wait for unit + port

```
machine.wait_for_unit("caddy.service")
machine.wait_for_open_port(443)
machine.succeed("curl -k -sS https://127.0.0.1 | grep -q lab")
```

Prefer `wait_for_unit` / `wait_for_open_port` over `sleep 30`.

Lab — one negative assertion

```
machine.fail("curl -sS --max-time 2 http://127.0.0.1:5432 || true")
# better: assert postgres not on 0.0.0.0 if that is policy
```

Encode a security/policy property, not only happy path.

Lab — time the test

```
time nix build .#checks.x86_64-linux.web-smoke -L
```

Record duration. Tests that take 20+ minutes will not be run and thus do not protect you—split or slim.

Common gotchas

Symptom / mistake	What to do
No <code>/dev/kvm</code>	Nested virt; remote builder with kvm; or accept qemu slow
Hang on boot	Serial logs; simplify modules; <code>wait_for_unit</code>
DNS names	Use node names as hostnames
Testing wrong module version	Same flake inputs as deploy
Over-large configs	Slim test nodes; don't import Desktop
Flaky timing	<code>wait_for_open_port</code> , retries

Checkpoint

- ☐ One `nixosTest` green
- ☐ Tests **your** module behavior
- ☐ Intentional red→green cycle
- ☐ Flake `checks` wiring
- ☐ KVM/runtime notes written

Commit

```
git add .
git commit -m "day64: nixosTest for own module"
```

Write three personal gotchas before continuing.

More testing patterns

More testing patterns

Goal: Grow a **suite** of at least three meaningful flake checks (including `nixosTest` and lighter `eval/build` checks) that mirror CI shape without becoming a flaky tar pit.

i Note

Builds on the `nixosTest` chapter (`nixosTest`). Baseline: flakes, **26.05** pins, modules you already own. Diversify **kinds** of checks; do not only clone one VM test three times.

Why this day exists

One demo test rots. A small suite with clear ownership becomes the merge gate for roles and packages. Today you diversify test layers, kill flakes early, and time the suite so the CI pipelines chapter CI has a budget.

Theory 1 — Test pyramid for NixOS fleets

Layer	Examples	Speed	Catches
Eval	<code>nix flake show</code> , option asserts, <code>nix eval</code>	Fast	Typos, missing options
Build	<code>nix build .#pkg</code> , host toplevel	Medium	Compile/package breaks
VM (<code>nixosTest</code>)	Service behavior	Slow	Integration
Deploy dry	<code>deploy-rs</code> checks, colmena eval	Medium	Hive wiring
Live canary	Real host probes	Manual/slow	Reality

Aim: **many fast, few slow**. Capstone and CI die when every PR boots three full VMs for a typo in a comment.

```
/live canary\  
/  nixosTest  \  
/ build / toplevel \  
/ eval / lint / fmt \  

```

Theory 2 — Three checks minimum (template)

```
# flake outputs fragment - illustrative
checks = forAllSystems (system:
  let
    pkgs = import nixpkgs {
      inherit system;
      overlays = [ self.overlays.default ];
    };
  in {
    # 1) package as check (build is the test)
    cooltool-build = self.packages.${system}.cooltool;

    # 2) pure command check
    cooltool-runs = pkgs.runCommand "cooltool-runs" {
      nativeBuildInputs = [ self.packages.${system}.cooltool ];
    } ''
      cooltool --help >/dev/null
      touch $out
    '';
  } // nixpkgs.lib.optionalAttrs (system == "x86_64-linux") {
    # 3+) VM tests - Linux only
    ssh-hardening = import ./tests/ssh-hardening.nix { inherit pkgs self; };
    web-role = import ./tests/web-role.nix { inherit pkgs self; };
  }
);

nix build .#checks.x86_64-linux.cooltool-runs -L
nix build .#checks.x86_64-linux.ssh-hardening -L
nix flake check -L
```

Theory 3 — Lightweight non-VM checks

Format / lint

```
# Prefer your Stage III treefmt/nixfmt wiring if present
fmt = pkgs.runCommand "fmt-check" {
  nativeBuildInputs = [ pkgs.nixfmt-rfc-style ];
} ''
```

```
# example only - real checks usually run treefmt --ci on src
touch $out
'';
```

Eval / build host configuration

```
edge-toplevel =
  self.nixosConfigurations.edge.config.system.build.toplevel;

nix build .#nixosConfigurations.edge.config.system.build.toplevel -L
```

Strong, but heavy—cache it; don’t duplicate five slight variants.

Structured eval assertions

```
nix eval .#nixosConfigurations.edge.config.networking.hostName
nix eval --json .#nixosConfigurations.edge.config.services.openssh.enable
```

Wrap in a `runCommand` with `jq` if you want it as a formal check:

```
host-name-ok = pkgs.runCommand "host-name-ok" {
  nativeBuildInputs = [ pkgs.jq ];
} ''
  echo ${pkgs.lib.escapeShellArg (
    builtins.toJSON self.nixosConfigurations.edge.config.networking.hostName
  )} | jq -e '. == "edge"'
  touch $out
'';
```

(Prefer cleaner patterns as your flake matures; the point is **fast** feedback.)

Theory 4 — Anti-flake tactics

Problem	Fix
Sleep-based waits	<code>wait_for_unit</code> / <code>wait_for_open_port</code>
Order-dependent multi-test	Isolated nodes per test file
Network to internet in tests	Forbid; vendor fixtures
Shared mutable state	Each test pure

Problem	Fix
Over-broad flake check on Darwin	<code>optionalAttrs</code> for Linux VM tests
30-minute suite	Split “CI fast” vs “nightly full”
Secret decryption in CI	Dummy modules / test keys only

```
# nixosTest script - good waits
machine.wait_for_unit("nginx.service")
machine.wait_for_open_port(80)
machine.succeed("curl -f http://127.0.0.1/")

# Run one check without the world
nix build .#checks.x86_64-linux.ssh-hardening -L
```

Theory 5 — Mapping tests to modules

Module / concern	Check ideas
<code>profiles.hardened</code>	<code>sshd -T</code> probes, package absences
<code>roles.web</code>	curl client→server via proxy
Stage V package	help/version smoke
Disko	eval <code>fileSystems</code> keys (not full wipe)
<code>sops-nix</code>	assert secret options configured; decrypt with test key only
firewall	multi-node: client cannot hit closed port
deploy hive	eval nodes exist / <code>--dry-activate</code> if available

Every check should name an **owner module** in a README table so rotting tests get a human.

Theory 6 — Suite layout and CI shape

```
tests/
  ssh-hardening.nix
  web-role.nix
  cooltool.nix
  fixtures/
```

```

    test-secrets.nix
    README.md          # table: check → layer → minutes → owner

# flake fragment
checks.x86_64-linux = {
  inherit (self.packages.x86_64-linux) cooltool;
  cooltool-runs = /* ... */;
  ssh-hardening = /* ... */;
  web-role = /* ... */;
  edge-toplevel =
    self.nixosConfigurations.edge.config.system.build.toplevel;
};

```

the CI pipelines chapter will run these on GitHub Actions. Design attrs you can select:

```

nix build -L \
  .#checks.x86_64-linux.cooltool-runs \
  .#checks.x86_64-linux.ssh-hardening

```

Worked example — fail-inject once

```

# 1) break intentionally
# e.g. change expected hostname assertion or stop nginx in test config
nix build .#checks.x86_64-linux.web-role -L # expect fail
# 2) restore
# 3) confirm green
# 4) confirm other checks still independent
nix build .#checks.x86_64-linux.cooltool-runs -L

```

If breaking test A always breaks unrelated B, your suite shares too much impure state.

Lab — multi-step

Suggested notes: ~/lab/nixos/ch65

Step 1 — Inventory today's checks

```
mkdir -p ~/lab/nixos/ch65
cd /path/to/your/flake
nix flake show 2>&1 | tee ~/lab/nixos/ch65/flake-show.txt
```

List current checks and gaps.

Step 2 — Add check #1 (package)

Build + run your Stage V package (`runCommand` or `package attr` as check).

Step 3 — Add check #2 (nixosTest)

Keep/improve the `nixosTest` chapter test; ensure waits are unit/port based.

Step 4 — Add check #3 (config)

Either second `nixosTest` **or** `toplevel build` for one host **or** eval assertion.

Step 5 — Fail inject matrix

Break each check once; confirm isolation; restore green.

Step 6 — Timing budget

```
/usr/bin/time -p nix flake check -L \
2>&1 | tee ~/lab/nixos/ch65/timing.txt
```

If over your CI budget, document split: `checks.ci` vs full suite.

Step 7 — README table

Check	Layer	Minutes	Owner module
-------	-------	---------	--------------

Step 8 — Stability

Rerun thrice (or twice if time-boxed); note flakiness tickets.

Step 9 — Secrets policy for tests

Write one paragraph: how CI avoids prod sops keys (fixtures, dummy tokens).

Common gotchas

Symptom / mistake	What to do
<code>flake check</code> builds everything forever	Select attrs; fix IFD explosions
Darwin CI without KVM	Gate VM tests on Linux
Secret decryption in checks	Dummy modules / mock
Duplicate heavy toplevels	Share less; accept one + cache
Tests not in merge gate	the CI pipelines chapter wires CI
Sleep-only “green”	Replace with readiness waits
Three identical VM tests	Diversify layers

Checkpoint

- ☐ 3 meaningful checks green
- ☐ At least one VM test and one non-VM test
- ☐ Timing measured
- ☐ README/table of suite with owners
- ☐ Fail-inject done once per check
- ☐ Flakiness notes empty or ticketed
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "day65: expanded flake check suite"
```

Write three personal gotchas before continuing.

CI pipelines for flakes

CI pipelines for flakes

Goal: Run your flake checks/builds on **GitHub Actions** (or GitLab) with a modern Nix installer, optional **Cachix/Attic** push, and a PR-shaped workflow that fails on regressions.

i Note

If tests only run on your laptop, they are hobbies. 2026 default shape: **forge CI + flakes + binary cache**. Baseline nixpkgs: **26.05**.

Why this day exists

Stage V packages and Stage VI modules become team infrastructure only when a clean runner reproduces them. CI also forces purity: “works on my machine” impurities die in public logs. Capstone the capstone harden chapter reuses this muscle.

Theory 1 — Minimal trustworthy pipeline

```
checkout → install Nix → (auth cache) → nix flake check / build → (push cache)
```

Job / step	Purpose
checkout	Source + lockfile
install Nix	Known installer; flakes enabled
cache auth	Pull/push substitutes
check	<code>nix flake check</code> or selected checks
build-pkg	Stage V package
build-hosts	Optional toplevel builds

Trust boundaries

Surface	Rule
PR from fork	Often no secret cache write tokens
main	May push cache

Surface	Rule
Logs	Never print age keys, Cachix tokens, SSH keys
trusted-users	Do not weaken developer laptops for CI

Theory 2 — GitHub Actions workflow (2026-shaped)

```
# .github/workflows/nix.yml
name: nix

on:
  push:
    branches: [ main ]
  pull_request:
  workflow_dispatch:

permissions:
  contents: read
  # id-token: write # if using OIDC for caches later

concurrency:
  group: nix-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install Nix
        uses: DeterminateSystems/nix-installer-action@main
        # pin to a reviewed tag in real repos

      - name: Magic Nix Cache (optional)
        uses: DeterminateSystems/magic-nix-cache-action@main
        # or Cachix below

      # - uses: cachix/cachix-action@v15
      #   with:
      #     name: myorg
      #     authToken: ${{ secrets.CACHIX_AUTH_TOKEN }}
```

```

- name: Flake metadata
  run: nix flake metadata

- name: Selected checks
  run: |
    nix build -L \
      .#checks.x86_64-linux.cooltool-runs \
      .#checks.x86_64-linux.ssh-hardening

- name: Build package
  run: nix build .#cooltool -L

- name: Build edge toplevel (optional, heavy)
  run: nix build .#nixosConfigurations.edge.config.system.build.toplevel -L

```

⚠ Warning

Pin action versions to tags or SHAs you have reviewed. Versions above are illustrative—confirm current upstream tags when you wire the repo. Freeze pins harder by the multi-node bar chapter for Capstone.

Theory 3 — Cachix (and friends) in CI

```

- uses: cachix/install-nix-action@v31
  with:
    nix_path: nixpkgs=channel:nixos-26.05
- uses: cachix/cachix-action@v15
  with:
    name: myorg
    authToken: '${{ secrets.CACHIX_AUTH_TOKEN }}'
    # signingKey: '${{ secrets.CACHIX_SIGNING_KEY }}' # if used
- run: nix build .#cooltool -L

```

House rules

Rule	Why
Push from CI (common)	Shared binaries; less laptop upload chaos
Separate PR vs main push rights	Abuse / token theft mitigation
Public keys on consumers	Lab + admin substituters + trusted-public-keys

Rule	Why
Never commit private signing keys	Rotate immediately if leaked

Alternatives

Tool	Notes
Cachix	Hosted; simple
Attic	Self-hosted multi-tenant cache
Harmonia	Serve local store as binary cache
Magic Nix Cache	UX-oriented GitHub cache integration

Any one done well beats none.

Theory 4 — What not to run on free runners

Heavy	Strategy
Full nixosTest matrix	Linux job; fewer nodes; path filters
Multiple toplevels	Build one primary host per PR
Nested KVM	May work on GH-hosted; else self-hosted runner
Secret full decrypt	Dummy configs for CI (the more tests chapter)
Unbounded <code>flake check</code>	Select attrs until budget known

```
on:
  pull_request:
    paths:
      - 'flake.nix'
      - 'flake.lock'
      - 'modules/**'
      - 'hosts/**'
      - 'pkgs/**'
      - 'tests/**'
      - '.github/workflows/nix.yml'
```

Timeouts and disks

```
jobs:
  check:
    runs-on: ubuntu-latest
    timeout-minutes: 90
    steps:
      # ...
```

Out-of-disk: slim checks, GC mid-job only if needed, or larger runners.

Theory 5 — GitLab CI sketch

```
# .gitlab-ci.yml (sketch)
image: debian:bookworm
variables:
  NIX_CONFIG: "experimental-features = nix-command flakes"
before_script:
  - apt-get update && apt-get install -y curl xz-utils
  # install Nix via official/Determinate method - prefer current docs
  - nix flake metadata
check:
  script:
    - nix flake check -L
rules:
  - if: $CI_PIPELINE_SOURCE == "merge_request_event"
  - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

Prefer maintained install methods over ancient `nixos/nix` container tags when flakes misbehave.

Theory 6 — CI vs deploy

Pattern	the CI pipelines chapter stance
CI verifies build/check	Default
CI pushes cache	Preferred
CI deploys to lab	Optional later; protect with environments
CI holds prod SSH keys	High risk; avoid early

Human or gated CD **deploy** after green checks is enough for Stage VI. Capstone the capstone harden chapter revisits.

Worked example — selected checks job

```
- name: Fast checks
  run: |
    set -euo pipefail
    nix build -L \
      .#checks.x86_64-linux.cooltool-runs \
      .#checks.x86_64-linux.ssh-hardening
- name: Optional full check
  if: github.ref == 'refs/heads/main'
  run: nix flake check -L
```

Split fast PR signal from heavier main-branch verification when needed.

Worked example — intentional red PR

```
# on a branch: break a check assertion or package
git checkout -b ci/red-test
# ... introduce fail ...
git commit -am "ci: temporary red"
git push -u origin ci/red-test
# open PR → confirm CI red
# fix → confirm green
```

If you cannot get a red PR, the workflow is not protecting you.

Lab — multi-step

Suggested notes: ~/lab/nixos/ch66

Step 1 — Add workflow file

Commit `.github/workflows/nix.yml` (or GitLab equivalent) to your lab flake repo. Use path filters if monorepo.

Step 2 — First green CI

Push branch; fix until green on a clean runner. Paste run URL into notes.

```
mkdir -p ~/lab/nixos/ch66
echo "CI_URL=..." > ~/lab/nixos/ch66/ci-url.txt
```

Step 3 — Wire cache

Cachix **or** Magic Nix Cache **or** Attic. Confirm second run is faster or shows substitutes.

Step 4 — PR simulation

Open PR that breaks a check; confirm CI red; fix; green.

Step 5 — Secrets hygiene

Ensure tokens only in forge secrets; scan workflow for accidental `echo` of secrets.

Step 6 — Consumer config

Add substituter public keys to lab/admin Nix config; rebuild or set user `nix.conf`.

Step 7 — Document runner needs

KVM? memory? timeout? Write `docs/CI.md` with:

Topic	Content
Triggers	PR/main
Commands	exact <code>nix build</code> lines
Cache	name + public key
Limits	known failures without KVM
Non-goals	deploy-from-CI policy

Step 8 — Status badge (optional)

README badge for the workflow—only if green is honest.

Step 9 — Deploy from CI? (decide)

Write the choice: verify-only vs gated deploy. Default for this day: **verify-only**.

Common gotchas

Symptom / mistake	What to do
Flakes disabled	Installer action enables; set <code>NIX_CONFIG</code>
Disk full on runner	Slim checks; larger runner; GC carefully
Unpinned actions	Pin majors/SHAs per org policy
Cache not used	Public key / auth; path existence; wrong cache name
VM tests fail without KVM	Self-hosted or reduce tests
Monorepo wrong cwd	<code>defaults.run.working-directory</code> / filters
Fork PRs leaking write tokens	Restrict secret access
IFD / eval forever	Fix expressions; select attrs

Checkpoint

- ☐ Workflow exists and runs on PR/push
- ☐ At least one Nix build/check green in CI
- ☐ Intentional red PR observed
- ☐ Cache strategy configured or explicitly deferred with reason
- ☐ `docs/CI.md` notes runner limits and commands
- ☐ Tokens not in git
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "day66: GitHub Actions Nix CI pipeline"
```

Write three personal gotchas before continuing.

GC and store hygiene

GC and store hygiene

Goal: Measure store size, explain **GC roots**, set a retention policy for generations and CI, and run `optimise/GC` safely without deleting live systems.

Why this day exists

Deploy + CI + packaging fill disks silently. Out-of-disk mid-activate is a self-inflicted outage. Hygiene is an operational skill equal to deploy tooling—Capstone hosts die from full `/nix` as often as from bad modules.

Theory 1 — What GC deletes

Garbage collection deletes store paths that are **not reachable from any GC root**.

Root examples	Keeps alive
<code>/run/current-system</code>	Running NixOS closure
<code>/nix/var/nix/profiles/*</code>	System generations, user profiles
<code>~/.local/state/nix/profiles</code>	Modern user profiles
<code>/nix/var/nix/gcroots/*</code>	Explicit pins
Result symlinks in cwd	<code>./result</code> from <code>nix build</code>
Indirect roots / registries	Various tooling pins

```
nix-store --gc --print-roots | head -n 50
nix-collect-garbage --dry-run
nix store gc --dry-run # modern CLI shape; flags evolve-check --help
```

Mental model

GC root → profile/generation/result link → closure of store paths
Anything not reachable is eligible for deletion

Deleting a project directory does **not** free the store if roots still point at builds.

Theory 2 — Generations vs store GC

```
# List system generations
sudo nix-env -p /nix/var/nix/profiles/system --list-generations
# or inspect profile links
ls -l /nix/var/nix/profiles/system*

# Delete old generations then GC
sudo nix-collect-garbage --delete-older-than 14d
# aggressive: remove all old gens of profiles
# sudo nix-collect-garbage -d
```

Command	Effect
<code>nix-collect-garbage</code>	GC unreachable paths
<code>--delete-older-than</code>	Drop old gens then GC
<code>-d</code>	Aggressive old gen removal
<code>nix store gc</code>	Modern CLI GC
<code>nix store optimise</code>	Hardlink identical files

! Important

Deleting generations removes rollback options. Keep enough to survive a bad deploy week—especially before Capstone freeze and DR drills.

User vs system profiles

Home-Manager and user `nix` profile installs have their own generation chains. System GC policy does not automatically equal user hygiene.

```
nix profile list 2>/dev/null || true
ls ~/.local/state/nix/profiles 2>/dev/null || true
```

Theory 3 — Policy template

Host class	Generation retention	Notes
Laptop	7–14 days	Disk pressure
Prod VM	14–30 days + keep last N	Rollback window

Host class	Generation retention	Notes
CI runner	Daily GC; few profiles	Ephemeral preferred
Builder	Weekly GC; keep hot cache set	Balance disk vs rebuild
Capstone lab	14 days through the rollback and tests chapter	Rollback drills need gens

```
# NixOS automatic GC example
{ ... }:
{
  nix.gc = {
    automatic = true;
    dates = "weekly";
    options = "--delete-older-than 14d";
  };
  nix.optimise.automatic = true;
  # proactive free-space GC thresholds (bytes)
  nix.settings.min-free = 10 * 1024 * 1024 * 1024; # 10 GiB
  nix.settings.max-free = 50 * 1024 * 1024 * 1024;
}
```

`min-free` triggers proactive GC when disk is low—excellent on small roots and cloud boxes.

Theory 4 — Finding what holds space

```
df -h /nix
nix path-info -Sh /run/current-system
nix path-info -rS /run/current-system | sort -k2 -n | tail -n 20

# Why is path alive?
nix-store --query --roots /nix/store/...-something

# Old result links (common disk vampires)
find "$HOME" -maxdepth 3 -type l \( -name result -o -name 'result-*' \) 2>/dev/null

# Largest store paths (approx; tools vary)
nix path-info -rS /run/current-system | sort -k2 -h | tail
```

CI-specific roots

Runners accumulate **result** links and old profiles if jobs are not ephemeral. Prefer clean workspaces per job; document GC for self-hosted runners in `docs/CI.md`.

Theory 5 — optimise (hardlinks)

```
nix store optimise
# or
sudo nix-store --optimise
```

Identical files across store paths become hardlinks → often saves GBs when many similar closures exist (multiple generations, overlapping devshells). Safe; CPU-heavy first run.

i Note

Backup tools that do not understand hardlinks may inflate sizes or behave oddly. Know your backup tool's hardlink policy (the backups chapter).

Theory 6 — Incident patterns

Symptom	First checks
No space left on device during build	<code>df -h /nix /</code> ; roots; old gens; result links
Activate failed mid-switch	Free space; incomplete profile; repair with boot generation
GC “deleted my package”	Missing root; rebuild; avoid bare <code>-d</code> in prod
Rollback missing	Generations deleted by policy
Disk full only on CI	Self-hosted runner hygiene

Never run destructive GC on prod without:

1. Confirming current-system root
2. Confirming retention window
3. Preferring `--delete-older-than` over blind `-d`

Worked example — hygiene report script

```
#!/usr/bin/env bash
# scripts/store-report.sh
set -euo pipefail
echo "== disk =="
df -h /nix 2>/dev/null || df -h /
echo "== current system closure =="
nix path-info -Sh /run/current-system 2>/dev/null || true
echo "== gc roots (sample) =="
nix-store --gc --print-roots 2>/dev/null | head -n 40 || true
echo "== result symlinks =="
find "$HOME" -maxdepth 2 -type l -name 'result*' 2>/dev/null || true
echo "== system generations =="
sudo nix-env -p /nix/var/nix/profiles/system --list-generations 2>/dev/null | tail -n 15 ||

chmod +x scripts/store-report.sh
./scripts/store-report.sh | tee ~/lab/nixos/ch67/before.txt
```

Lab — multi-step

Suggested notes: ~/lab/nixos/ch67

Step 1 — Baseline metrics

Record `df -h /nix` (or `/`) and current-system size via `nix path-info -Sh`.

Step 2 — Map roots

Print roots; identify unexpected `result` links and old profiles. Screenshot or tee a sample.

Step 3 — Safe cleanup

Remove stale `./result` from lab dirs; GC **without** aggressive `-d` first:

```
nix store gc
df -h /nix
```

Step 4 — Generation policy

List generations; delete only those outside your written policy window.

```
sudo nix-collect-garbage --delete-older-than 14d
```

Step 5 — Enable automatic GC on lab host

Commit `nix.gc` (+ optional `min-free` / `optimise`) settings; deploy or switch.

Step 6 — optimise

Run `optimise`; record space reclaimed if measurable.

```
df -h /nix | tee ~/lab/nixos/ch67/before-optimise.txt
nix store optimise
df -h /nix | tee ~/lab/nixos/ch67/after-optimise.txt
```

Step 7 — CI runner note

Add GC step for self-hosted runners or rely on ephemeral runners; document in `docs/CI.md`.

Step 8 — Policy doc

Write `docs/STORE-HYGIENE.md` (or section in `RUNBOOK`):

Host	Retention	Auto GC	Notes
lab	14d	weekly	
laptop	7d	weekly	

Step 9 — Rollback still possible?

Confirm at least one previous generation remains on lab after cleanup. If not, you over-pruned—adjust policy.

Step 10 — Profile audit (user + HM)

```
# User profiles / modern nix profile
nix profile list 2>/dev/null | tee ~/lab/nixos/ch67/user-profile.txt || true
# Home-Manager generations if present
ls -la /nix/var/nix/profiles/per-user/"$USER"/ 2>/dev/null || true
```



```
ls -la ~/.local/state/home-manager 2>/dev/null || true
```

Write one sentence: how user-profile retention differs from system `nix.gc` on this host.

Step 11 — Dry-run before aggressive delete

```
# Prefer dry-run / list before destroy
nix-collect-garbage --delete-older-than 30d --dry-run 2>&1 \
  | tee ~/lab/nixos/ch67/gc-dry-run.txt || true
```

If dry-run is unavailable on your Nix version, list generations explicitly and delete only named old gens.

Worked example — min-free incident story

```
Disk at 98% on small cloud root:
1) df -h /nix
2) remove stale result links in ~ and CI workspaces
3) nix-collect-garbage --delete-older-than 7d # laptop policy, not prod
4) enable nix.settings.min-free so next time GC starts earlier
5) consider separate /nix partition next reprovision (Disko)
```

Record whether you hit this during Stage V/VI. Capstone hosts on 20 GiB disks fail predictably without policy.

Common gotchas

Symptom / mistake	What to do
GC “does nothing”	Roots still hold paths
Deleted all rollbacks	Stop bare <code>-d</code> in prod without policy
<code>/nix</code> on small partition	Separate partition; <code>min-free</code>
optimise confuses backups	Hardlinks; tool configuration
Multi-user permissions	GC as root for system profiles
HM gens forgotten	User profile hygiene
Full disk mid-CI	Ephemeral runners; job GC; slimmer checks

Checkpoint

- ☐ Measured store before/after
- ☐ Explained roots with real examples from your machine
- ☐ Retention policy written and applied on lab
- ☐ optimise run once
- ☐ Auto GC or min-free configured
- ☐ At least one rollback generation still present
- ☐ User/HM profile note written
- ☐ docs/STORE-HYGIENE.md (or RUNBOOK section) committed
- ☐ Three personal gotchas

Commit

```
git add .  
git commit -m "day67: GC policy and store hygiene"
```

Write three personal gotchas before continuing.

Secure Boot and TPM (awareness)

Secure Boot and TPM (awareness)

Goal: Document and, if hardware allows, implement a **Lanzaboote** Secure Boot path and/or TPM-backed LUKS unlock—or produce a lab-limits dossier that is honest about what you cannot do on VMs.

! Important

Copy-paste without the upstream guide bricks boots. Treat this day as **process + documentation** first; enable on a **disposable** machine second. Recovery USB and recovery keys before enrollment.

Why this day exists

Fleet images and metal boxes increasingly expect **Secure Boot** and disk encryption that does not beg for a passphrase at every reboot in a datacenter. NixOS’s community answer for SB is primarily **Lanzaboote** (uki/systemd-boot + signed generations). TPM2 unlock pairs with LUKS (the Disko chapter Disko often declares the encrypted volumes).

Capstone B (stretch) lists SB/TPM; Capstone A does not require full enablement—but Stage VI experts must know the **path** and the **failure modes**.

Theory 1 — Problem statement

Threat	Control
Evil maid bootloader swap	Secure Boot + enrolled keys
Disk theft	LUKS (or vendor disk encryption—know the difference)
Passphrase at scale	TPM2 PCR-bound unlock (with enrollment process)
Unsigned kernel modules policies	SB + your key hierarchy
Silent rollback to unsigned attacker kernel	Sign each generation you boot

NixOS’s generation model must **sign** new boots—not only ship a distro shim forever. That is why Lanzaboote exists relative to “turn on SB and hope.”

Theory 2 — Lanzaboote overview (2026)

[Lanzaboote](#) integrates Secure Boot with NixOS (systemd-boot oriented flows).

Typical story (always verify against **current** README for your pin):

1. Enable Secure Boot in firmware (setup mode / enroll keys carefully)
2. Generate or enroll machine keys (PK/KEK/db as relevant to the guide)
3. Import Lanzaboote NixOS modules so boots are signed
4. Keep recovery path (USB installer + known recovery keys)
5. Re-test boot after **each** major change before trusting prod

```
# Conceptual only - follow current Lanzaboote README for 26.05-compatible paths
{
  imports = [ lanzaboote.nixosModules.lanzaboote ];
  boot.loader.systemd-boot.enable = lib.mkForce false; # lanzaboote replaces aspects
  boot.lanzaboote = {
    enable = true;
    pkiBundle = "/etc/secureboot"; # path patterns per docs
  };
}
```

Flake input sketch:

```
# inputs.lanzaboote.url = "github:nix-community/lanzaboote/...";
# inputs.lanzaboote.inputs.nixpkgs.follows = "nixpkgs";
```

⚠ Warning

Firmware Setup Mode, key enrollment, and Microsoft/3rd-party key politics vary by vendor. Laptop OEM SB is not the same as a server you control end-to-end.

Theory 3 — TPM2 + LUKS

Goals:

- Unlock root (or data) volume without typing a passphrase every boot
- Still retain a **recovery passphrase** or recovery key file offline

- Survive policy: which PCRs bind the sealed key

```
# Sketch - systemd-cryptenroll / auto-unlock patterns evolve
{
  boot.initrd.systemd.enable = true; # often required for modern enroll flows
  # boot.initrd.luks.devices."encrypted".crypttabExtraOpts = [ "tpm2-device=auto" ];
  # Exact options: verify on your nixpkgs 26.05 pin and systemd version
}
```

Manual enrollment often looks like:

```
# list TPM
systemd-cryptenroll --tpm2-device=list
ls /sys/class/tpm

# enroll (EXAMPLE - adjust device + PCR set per threat model)
# systemd-cryptenroll --tpm2-device=auto --tpm2-pcrs=7 /dev/nvme0n1p2
```

PCR trade-offs

Choice	Security	Fragility
Fewer PCRs	Weaker binding	Survives more firmware changes
PCR 7 (Secure Boot state) common	Ties to SB	Firmware/SB policy changes may require re-enroll
Many PCRs	Stronger	Breaks after BIOS updates more often

Document which PCRs and the **re-enroll procedure** after BIOS updates. Never TPM-only without recovery credentials.

Theory 4 — VM lab limits

Environment	Secure Boot	TPM
Bare metal modern laptop	Often yes	Often yes
Cloud VM	Sometimes virtual SB/TPM	Vendor-specific
libvirt QEMU	Emulated possible	swtpm possible
Nested cheap VPS	Often no	Often no

QEMU/swtpm note (advanced lab)

You *can* practice pieces with OVMF Secure Boot and swtpm—but it is a project of its own. Stage VI accepts a **document-only track** if hardware refuses you. Do not fake “SB enabled” in notes without evidence.

Theory 5 — Recovery & break-glass

Asset	Storage
Disk recovery key / passphrase	Password manager / sealed paper / offline encrypted store
SB setup mode knowledge	Runbook steps
Unsigned rescue ISO / installer USB	Physical media
Age/sops keys	Separate from disk encryption keys
Old generation unsigned	Know when SB will reject boot

Failure Friday scenario:

- BIOS update → PCR mismatch → TPM unlock fails
- boot installer USB
- unlock with recovery key
- re-enroll TPM
- document time-to-recover

Never enroll TPM-only without a recovery passphrase backup you have **tested**.

Theory 6 — Relation to Disko, deploy, and images

Concern	Link
LUKS layout	the Disko chapter Disko declares partitions; TPM enroll often post-install
Impermanence	the impermanence chapter: encrypted root + persist datasets
Images	the images/generators chapter generators may not equal metal SB story
Deploy	Remote deploy does not replace firmware SB enrollment

Concern	Link
Capstone B	Fleet policy: which classes require SB

Write one paragraph: **which host classes** require SB vs best-effort.

Worked example — dossier template

```
# SECUREBOOT-TPM.md

## Hardware
- Machine:
- Firmware vendor/version:
- Firmware SB state: SetupMode / User / Disabled / Unknown
- TPM: 2.0 visible? (commands + output summary)
- Lab type: metal / libvirt / cloud

## Goals
- [ ] Secure Boot with Lanzaboote
- [ ] LUKS passphrase only
- [ ] LUKS + TPM2
- [ ] Document-only (limits)

## Upstream sources
- Lanzaboote README URL + commit/date consulted:
- NixOS wiki / manual notes:

## Steps performed
1. ...
2. ...

## Recovery
- Recovery key location (not the key itself):
- USB installer location:
- Re-enroll after BIOS update:
- Estimated RTO if TPM fails:

## Residual risks
- ...

## Fleet policy
- Host classes requiring SB:
- Best-effort only:
```

Lab — multi-step

Suggested notes: ~/lab/nixos/ch68

Step 1 — Probe hardware

```
mkdir -p ~/lab/nixos/ch68
bootctl status 2>/dev/null | tee ~/lab/nixos/ch68/bootctl.txt || true
efi-readvar 2>/dev/null | tee ~/lab/nixos/ch68/efi-vars.txt || true
ls -la /sys/class/tpm 2>&1 | tee ~/lab/nixos/ch68/tpm-sysfs.txt
systemd-cryptenroll --tpm2-device=list 2>&1 | tee ~/lab/nixos/ch68/tpm-list.txt || true
```

Step 2 — Choose track

Track	Criteria
A	Metal + SB available → Lanzaboote guided enable on disposable
B	TPM + LUKS available → enroll with recovery key tested
C	VM only → emulate with swtpm or document-only dossier

Write the track choice first; do not improvise on your daily driver.

Step 3 — Read upstream (required even for Track C)

Open current Lanzaboote README + Secure Boot notes relevant to your generation. Save URLs in the dossier. Take notes—do not invent key enrollment sequences from memory.

Step 4 — Recovery assets first

Before any enrollment:

1. Confirm installer USB or rescue media boots
2. Confirm LUKS recovery key unlocks (lab disk)
3. Write locations in dossier

Step 5 — Implement or simulate

If A/B: snapshot/backup first; proceed carefully; capture commands and outcomes.

If C: write why cloud images still matter (signed provenance vs local SB) and what you would do on metal.

Step 6 — Failure plan drill (written)

Write the exact steps to boot installer and decrypt disk if TPM fails after firmware update. Include who holds recovery keys.

Step 7 — Link Disko / LUKS

Map where LUKS is declared and whether TPM enroll is post-install only. Note declarative limits honestly.

Step 8 — Fleet policy paragraph

Which host classes require SB? Which are best-effort? Capstone A vs B expectations.

Step 9 — Tabletop: evil maid + firmware update

Write answers in the dossier (no need to re-brick hardware):

1. Attacker replaces bootloader on stolen laptop with SB off historically—what changes with Lanzaboote enrolled?
2. After OEM BIOS update, TPM unlock fails at initrd—ordered recovery steps?
3. How do age/sops keys relate to disk recovery keys (separate custody)?
4. Can deploy-rs fix a machine that will not boot? (Usually no—firmware/recovery path first.)

Step 10 — Capstone A vs B honesty line

One sentence each:

- Capstone A: SB/TPM **not required**—document limits OK.
 - Capstone B: which pieces become mandatory for *your* stretch goals?
-

Worked example — recovery USB checklist

Recovery USB

- [] Boots on target hardware (tested date: ...)
- [] Contains or can fetch flake tools / nix installer as needed
- [] Operator knows LUKS recovery unlock command
- [] Network plan if flake must be cloned (ethernet? Wi-Fi firmware?)
- [] Time estimate from power-on to decrypted root shell: ...

If you never tested the USB on this machine, Track A/B is incomplete even if enrollment “worked once.”

Common gotchas

Symptom / mistake	What to do
Bricked boot	Installer USB; setup mode; recover keys
PCR mismatch after update	Re-enroll; reconsider PCR set
Lost recovery key	Undecryptable disk; prevention only
Enabled SB without signing new gens	Use Lanzaboote properly
Assumed cloud disk encryption = LUKS	Understand vendor encryption separately
Experiment on only laptop	Use disposable or accept data-loss risk consciously
Docs lag code	Date-stamp upstream guide you followed

Checkpoint

- ☐ Hardware probed with saved outputs
- ☐ Track A/B/C completed with artifacts
- ☐ SECUREBOOT-TPM.md filled
- ☐ Recovery path written **and** assets exist

- ☐ No production host left without recovery keys
- ☐ Upstream sources recorded
- ☐ Tabletop answers written
- ☐ Recovery USB checklist honest
- ☐ Three personal gotchas

Commit

```
git add .  
git commit -m "day68: secure boot / TPM path dossier"
```

Write three personal gotchas before continuing.

Backup story

Backup story

Goal: Pick **one** backup system (**Restic**, **Borg**, or **ZFS send**), declare it in NixOS, and complete a **restore drill of state** (not only a successful backup job log).

i Note

Flakes rebuild systems; they do not recreate Postgres, SQLite, photos, or `/persist`. Pair with the disaster recovery chapter later: rebuild OS from git+keys, restore **state** from backups.

Why this day exists

Capstones fail when “declarative” is confused with “immortal data.” Today is the state half of disaster recovery. A green `restic snapshots` line without a restore is theatre.

Theory 1 — What to back up

Include	Exclude
<code>/persist</code> / service state dirs	<code>/nix/store</code> (rebuild from flake)
Database dumps or consistent snapshots	Ephemeral caches, <code>/tmp</code>
Secrets materials you cannot reissue easily	Cloud provider API caches
User home select paths	result symlinks, build trees
Backup tool config references	The entire OS image as only strategy

Rule: If wipe+reinstall cannot recreate it from git+keys+cache, it needs a backup story.

Inventory questions

1. What does the impermanence chapter impermanence already send to `/persist`?
2. What does your the data service pattern chapter data service write?
3. What keys exist only once (age, SSH CA, ACME account)?

4. What would embarrass you if lost (homelab photos vs throwaway metrics)?

Theory 2 — Tool choice (pick one)

Tool	Strengths	Notes
Restic	Simple; many backends (S3, B2, SFTP, local)	Encrypted snapshots; prune policies
Borg	Dedup; mature; SSH repos	Borgmatic helper common on NixOS
ZFS send/recv	Excellent for ZFS hosts	Not an application-level portable tool

This book accepts any **one** done well. Switching tools mid-day is thrash—pick and finish.

3-2-1 reminder

Idea	Lab minimum
3 copies	Live + backup + offsite note
2 media	Disk + remote/object
1 offsite	Even “friend’s SFTP” or second VPS

Local-only lab is fine for learning **if** you write the offsite sentence for real systems.

Theory 3 — Restic on NixOS (shape)

```
{ config, pkgs, ... }:  
{  
  # Prefer sops-nix for repository password / access keys (the sops-nix chapter)  
  # sops.secrets.restic-password = { ... };  
  
  services.restic.backups.lab = {  
    initialize = true;  
    passwordFile = config.sops.secrets.restic-password.path;  
    repository = "sftp:backup@backup-host:/backups/lab";  
    # or "s3:s3.amazonaws.com/bucket/lab"  
    paths = [  

```

```

    "/persist"
    "/var/lib/myapp"
];
timerConfig = {
    OnCalendar = "daily";
    Persistent = true;
};
pruneOpts = [
    "--keep-daily 7"
    "--keep-weekly 4"
    "--keep-monthly 6"
];
};
}

```

Exact module options: check `man configuration.nix` / search.nixos.org for your **26.05** pin if attribute names shift.

Borgmatic sketch

```

{
  services.borgmatic.enable = true;
  # services.borgmatic.settings = { ... };
  # or environment.etc."borgmatic.d/lab.yaml".text = ...;
}

```

ZFS

```

zfs snapshot tank/persist@day69
zfs send -v tank/persist@day69 | ssh backup zfs recv -F backup/lab/persist

```

Declare snapshot timers via Nix modules or sanoid/syncoid if you go that ecosystem. Still do a **restore drill** (recv to a scratch dataset, mount, verify files).

Theory 4 — Consistency

Data	Technique
Postgres	<code>backupPrepareCommand</code> dump or brief stop / native snapshot
SQLite	Stop service or <code>.backup</code> API; avoid mid-write copies
VM disks	Hypervisor snapshot or fs freeze

Data	Technique
Mutable logs	Optional; size-cap
Mail / queues	App-native export

```
services.restic.backups.lab.backupPrepareCommand = ''
  mkdir -p /var/backup
  ${pkgs.sudo}/bin/sudo -u postgres \
    ${pkgs.postgresql}/bin/pg_dumpall > /var/backup/pg.sql
'';
# include /var/backup/pg.sql in paths
```

Inconsistent DB files that “restored” but won’t start are not a successful drill.

Theory 5 — Restore is the product

```
# Restic example
export RESTIC_REPOSITORY=...
export RESTIC_PASSWORD_FILE=...
restic snapshots
restic restore latest --target /tmp/restore-drill
# verify files; for DB: restore dump into disposable instance
```

Success criteria

Level	Meaning
Files on disk	Necessary but insufficient
Service starts	Better
Application-usable state	Done for the backups chapter
Timed RTO/RPO notes	Ops maturity

Write `RESTORE.md` with commands a tired operator can paste.

Theory 6 — Secrets and backup encryption

Concern	Practice
Repo password	sops-nix / agenix path; not world-readable
Encryption keys only on source host	Fails disaster recovery—offsite custody
Backing up decrypted <code>/run/secrets</code>	Usually wrong; back up encrypted materials + key custody story
Same disk as source	Not a disaster backup

Restic/Borg encrypt client-side; you still need the password after wipe (the disaster recovery chapter).

Worked example — local restic lab (no cloud)

```
{
  services.rectic.backups.local = {
    initialize = true;
    passwordFile = "/persist/secrets/restic-password"; # lab only; prefer sops
    repository = "/mnt/backup-disk/restic-lab";
    paths = [ "/persist/PROOF" "/var/lib/myapp" ];
    timerConfig.OnCalendar = "hourly";
  };
}
```

```
echo important-state | sudo tee /persist/PROOF
sudo systemctl start restic-backups-local.service
sudo systemctl status restic-backups-local.service --no-pager
sudo restic -r /mnt/backup-disk/restic-lab snapshots
sudo rm /persist/PROOF
sudo restic -r /mnt/backup-disk/restic-lab restore latest --target /tmp/r
sudo install -D /tmp/r/persist/PROOF /persist/PROOF
cat /persist/PROOF
```

Worked example — RTO/RPO honesty

```
## Lab targets (not production promises)
- RPO: up to 24h (daily timer) - last successful snapshot time: ...
- RTO: ~30-60 min to restore PROOF + app dir on empty disk (measured: ...)
- Not included: full bare-metal with SB/TPM re-enroll (the Secure Boot/TPM chapter)
```

Lab — multi-step

Suggested notes: ~/lab/nixos/ch69

Step 1 — Choose tool

Write choice + backend (local disk, SFTP, S3-compatible) in NOTES.

Step 2 — State inventory

List paths tied to services you run (reuse the impermanence chapter inventory). Mark each: rebuildable from git vs must backup.

Step 3 — Declare backup job in Nix

Wire secrets properly if possible (sops). Commit module + docs.

Step 4 — First backup

Run service once; list snapshots; capture logs.

```
mkdir -p ~/lab/nixos/ch69
sudo systemctl start restic-backups-lab.service # name may vary
journalctl -u restic-backups-lab.service -n 100 --no-pager \
  | tee ~/lab/nixos/ch69/backup.log
```

Step 5 — Restore drill (required)

Delete or move a **lab** file; restore; verify content byte-for-byte or known string.

Step 6 — DB drill (if you have the data service pattern chapter DB)

Dump → backup → drop table/schema in lab → restore dump → query. If no DB, write N/A with reason.

Step 7 — Offsite note

Even local-only lab needs a sentence on offsite/3-2-1 for real systems.

Step 8 — Runbook

docs/RESTORE.md:

- Prerequisites (password location, repo URL)
- List snapshots
- Restore commands
- Service restart order
- Lab RTO/RPO numbers

Step 9 — Negative cases table

Failure	Detection	Response
Timer silent fail	monitoring / <code>systemctl --failed</code>	
Repo full	prune / expand	
Wrong password after wipe	key custody	
Restored inconsistent DB	prepare hooks	

Common gotchas

Symptom / mistake	What to do
Backed up <code>/nix</code> only	Useless for state; fix paths
Password in world-readable file	sops/agenix; modes
Never pruned	Fill disk; set keep policy
Restored files, app still broken	Consistent DB story
Same disk as source	Not a disaster backup
Encryption keys only on source host	Offsite key custody
“Backup works” without restore	Fail the day until restore proves
Hardlink confusion with optimise	Know tool behavior (the GC hygiene chapter)

Checkpoint

- ☐ One tool chosen and declared in NixOS
- ☐ Backup job succeeds with log proof
- ☐ Restore drill completed with proof
- ☐ `RESTORE.md` written with RTO/RPO honesty
- ☐ Store vs state distinction clear in notes
- ☐ Secrets for repo password handled sanely
- ☐ Three personal gotchas

Commit

```
git add .  
git commit -m "day69: backup + restore drill"
```

Write three personal gotchas before continuing.

Internals

Internals and currency

Internals and currency

This part of the **standalone NixOS book** is organized by **topic** (normal book chapters), not by a calendar. Read linearly when building depth; skip and search when you already operate a flake host and need a reference answer.

Prerequisites. You should already rebuild a flake-defined NixOS host, write or heavily consume modules, pin inputs with `flake.lock`, and have at least one real service path (SSH, proxy, or data) under declarative control. Internals *last* is deliberate: evaluator and daemon vocabulary is most useful after you have scars from rebuilds, not before.

What this part is for. After host, layout, secrets, packaging, and fleet chapters, the remaining skill is **currency under change**: explain why eval hangs, why a type error appears only on merge, who may write the store, how releases and upgrades actually work, and how to keep an architecture honest. Capstone proof (wipe/rebuild, rollback, tests) depends on this literacy even when Capstone code lives in the next part.

Learning outcomes

When you leave this part you should be able to:

1. Force evaluation intentionally, diagnose infinite recursion, and use `trace` / `tryEval` / `seq` / `deepSeq` without cargo-culting them.
2. Choose module types, merge with `mkIf` / `mkMerge` / priorities, and fail bad configs at `eval` time with assertions.
3. Explain multi-user trust, GC roots, sandbox isolation, and substituter signature policy on a live machine.
4. Measure rebuild time (eval vs substitute vs build), tune `max-jobs` / `cores` / builders, and reject “sandbox off for speed.”
5. Describe input-addressed vs content-addressed derivations and keep production hosts on default store semantics until criteria say otherwise.
6. Harden `nixosTest` / flake `checks` so waits are unit/port based, not blind `sleep`.
7. Sample the adjacent ecosystem **once**, with a keep/drop decision that protects Capstone main.

8. Steelman Guix, image-based OS families, and Ansible against *your* workloads.
9. Read official release notes into an IMPACT checklist; plan a staged upgrade with rollback gates.
10. Inject and recover lab failures into a durable playbook; freeze architecture and gaps before Capstone thrash.

Chapter map

Chapter	Theme	You can leave when...
The evaluator	Thunks, WHNF, cycles, trace toolkit	You can reproduce and fix three recursion shapes and force one real flake leaf safely
Module system in depth	Types, merge, priorities, assertions	One production module is typed; bad configs fail at eval
Store and daemon	Multi-user trust, roots, sandbox, signatures	You map live roots and annotate a sandbox build log
Nix performance	Jobs/cores, eval cost, cache, builders	Before/after timings exist with justified settings
CA derivations (awareness)	IA vs CA, experimental flags	Status memo or lab experiment; Capstone stays default store
nixosTest mastery	Driver API, flakes, multi-node, CI	Suite inventory; sleep-only waits removed from critical path
Ecosystem map	devenv / Snowfall / nix-darwin (pick one)	COMPARE.md keep/drop without rewriting Capstone main
Comparative landscape	NixOS vs Guix vs images vs Ansible	Honest scorecard + essay with “when <i>not</i> NixOS”
Reading release notes	26.05 method, IMPACT tags, stateVersion	Annotated checklist against <i>your</i> flake
Upgrade planning	26.05→next runbook, gates, rollback	Dry-run upgrade document with abort criteria
Troubleshooting drills	Inject/recover on disposable VM	2 playbook entries with diagnose/fix/prevent
Architecture review	Map repo → Capstone A gaps	ARCHITECTURE.md + prioritized GAPS.md

How this part connects

Language / modules (earlier parts)

Evaluator + types ← debug eval failures at the source

Store / daemon / perf ← trust, isolation, time budget

CA literacy + tests ← experimental awareness; proof harness

Ecosystem + compare ← adopt tools and OS choices deliberately

Release + upgrade ← currency under nixpkgs cadence

Troubleshoot + arch ← playbooks + freeze before Capstone

Capstone part ← implement, prove, recover

Earlier part	What you reuse here
Concepts	Store paths, derivations, modern CLI
NixOS host	Rebuild modes, custom modules as consumer
Home & layout	Flake layout, locks, checks
Services & security	Secrets, firewall, proxy consumers
Packaging	FODs, caches, remote builders vocabulary
Ops & fleet	nixosTest intro, CI, GC, backups, deploy tools

Suggested study modes

Mode	Approach
Linear depth	Chapters in table order; one lab workspace per chapter under <code>~/lab/nixos/internals/<topic>/</code>
Reference	Jump via symptoms: infinite recursion → evaluator; silent option loss → module priorities; GC surprise → store/daemon

Mode	Approach
Pre-Capstone sprint	Architecture review first if your repo is chaotic; then evaluator + modules + tests; upgrade notes last if pin is already stable
Ops only	Store/daemon, performance, troubleshooting, release notes, upgrade planning

💡 Tip

Prefer disposable VMs and `nixos-rebuild build / nix build ...toplevel` over thrashing a daily driver. Console access before firewall experiments is non-negotiable.

Lab workspace convention

Use topic paths (not calendar folders):

```
mkdir -p ~/lab/nixos/internals/{evaluator,modules,store-daemon,perf,ca,tests,ecosystem,compa
```

Artifacts worth keeping in git (your flake or a notes repo):

Artifact	From
NOTES.md / personal gotchas	Every chapter
TYPES.md, PRIORITIES.md	Modules
ROOTS.md, TRUST.md	Store/daemon
PERF.md, BUILDERS.md	Performance
CA-MEMO.md or experiment logs	CA awareness
SUITE.md	nixosTest mastery
COMPARE.md	Ecosystem
SCORECARD.md, ESSAY.md	Comparative landscape
IMPACT.md	Release notes
UPGRADE-*.md	Upgrade planning
playbook/PL-*.md	Troubleshooting
ARCHITECTURE.md, GAPS.md, CAPSTONE-A.md	Architecture review

Acceptance bar for the part

You do **not** need every optional experiment green. You **do** need:

- ☐ One real infinite-recursion fix documented
- ☐ One module redesigned with real types + 2 assertions
- ☐ Live GC-root classification and sandbox left **on**
- ☐ Before/after rebuild timing with justified **max-jobs/cores**
- ☐ Explicit decision: Capstone on default (input-addressed) store
- ☐ At least one flake check/**nixosTest** without blind **sleep** as the only wait
- ☐ Keep/drop decision for one ecosystem trial **without** rewriting Capstone main
- ☐ Comparative essay that includes when you would *not* choose NixOS
- ☐ **IMPACT.md** for 26.05 (or your pin) with ACTION/VERIFY items
- ☐ Upgrade runbook skeleton with rollback commands
- ☐ 2 playbook entries from real inject/recover drills
- ☐ Architecture + gap list ready to drive Capstone

Baseline and policy

Item	Book default
Nix	2.34.x class — verify with nix --version
nixpkgs / NixOS	26.05 pin in Capstone and lab hosts
Store semantics	Input-addressed default; CA is literacy only
Secrets	Paths only in modules; sops-nix primary
Experimental features	Enable only with revert plan on disposable builders

! Important

No day-calendar framing in this book part. Chapter order is pedagogical, not a schedule. Study pacing is yours.

Common traps in this part

Trap	Better move
Enabling CA on the only builder “for speed”	Memo + criteria; keep Capstone default
deepSeq everywhere to “fix” laziness	Redesign option dependencies
mkForce on every host option	Fix module defaults and boundaries
Tool tourism (devenv + Snowfall + darwin in one weekend)	Try one ; score; decide
Upgrade Friday night with six input bumps	Branch, nixpkgs first, gates, lab first
Architecture doc never opened again	Link from CAPSTONE-A.md; gaps with acceptance tests

Next

→ [The evaluator](#) — thunks, force, and recursion diagnostics.

When architecture and gaps are frozen, continue to [Capstone and recovery](#).

The evaluator

The evaluator

Goal: Explain thunks, force evaluation intentionally, reproduce and fix infinite recursion, and use `trace` / `tryEval` / `seq` / `deepSeq` without cargo-culting them.

i Note

This chapter is **language/evaluator** depth. Prefer a disposable `nix repl` and tiny pure expressions before touching your host flake. A bad recursive module option can hang eval of the whole system.

Why this chapter exists

By the time you reach internals you already *use* laziness every rebuild: modules only force what activation needs. When eval explodes—**infinite recursion encountered**, mysterious `null`, or a multi-minute wait before any build starts—the bug is often a **thunk cycle** or accidental full-tree force, not a broken package.

Internals last (syllabus rule) means you can now map symptoms → mechanisms and fix the design rather than spraying `deepSeq` everywhere.

Theory 1 — Evaluation is a graph of thunks

Values vs thunks

Nix does not compute every subexpression when you write it. An unevaluated expression is a **thunk**: a promise to compute later, holding the expression body plus an environment of free variables.

Concept	Meaning
Thunk	Deferred computation; may be forced 0 or 1 times (then memoized)
WHNF	Weak head normal form: outer constructor known (<code>{...}</code> , <code>[...]</code> , <code>lambda</code> , <code>int</code> , ...) without forcing nested fields
NF	Full normal form: nested values forced recursively

Concept	Meaning
Memoization	After force, the thunk is replaced by the result (shared)

```
let
  x = builtins.trace "forcing x" (1 + 1);
  y = { a = x; b = x; };
in
  y.a
```

You should see "forcing x" **once**. Both a and b share the same thunk identity until forced.

Why modules love laziness

```
# Sketch of the idea - not the real module system
config = {
  services.foo.enable = true;
  # huge attrset of options you never touch...
};
# Only options that are *demanded* by evaluation of the final system get forced.
```

If every option were strict, evaluating a NixOS config would always pay the full cost of every default expression, including unused services.

Eager edges you already know

Construct	Eager?	Notes
<code>import ./file.nix</code>	Path resolve + parse/eval of the file	File body still may return lazy values
Function application <code>f x</code>	Forces <code>f</code> to a function; forces args per function body	Body of <code>f</code> decides strictness
<code>&&</code> / <code>\ \ </code>	Short-circuit	Second arm may never force
<code>if c then a else b</code>	Forces <code>c</code>	Only one branch forces
<code>builtins.seq a b</code>	Forces <code>a</code> to WHNF, returns <code>b</code>	Classic “strictness injection”
<code>builtins.deepSeq a b</code>	Forces <code>a</code> fully, returns <code>b</code>	Expensive; tests and asserts
<code>builtins.trace msg v</code>	Forces nothing about <code>v</code> except returns it	Side effect for debugging
<code>builtins.map f list</code>	Forces list spine as consumed	Element force depends on <code>f</code>

Construct	Eager?	Notes
String interpolation "\${x}"	Forces x to a stringable value	Common accidental force site

Theory 2 — Infinite recursion patterns

Pattern A — Direct self-reference

```
let x = x + 1; in x
# error: infinite recursion encountered
```

Pattern B — Mutual recursion without a base case

```
let
  a = b + 1;
  b = a + 1;
in a
```

Pattern C — Attrset self-reference (legal *shape*, dangerous force)

```
rec {
  a = 1;
  b = a + 1;      # OK
  # c = c;        # infinite if forced
}
```

rec is not evil; forcing a cycle is.

Pattern D — Module / option cycles (production shape)

Classic failure:

```
# module A
config.services.foo.settings.bar = config.services.baz.settings.qux;

# module B
config.services.baz.settings.qux = config.services.foo.settings.bar;
```

Or subtler:

```
options.my.paths.dataDir = lib.mkOption {
  type = lib.types.path;
  default = config.my.paths.root + "/data"; # forces config while defining defaults
};
```

Rule of thumb: defaults and `apply` functions should depend on **options** (or pure inputs), not on deep config that re-enters the same option.

Pattern E — with + shadowing confusion

```
let pkgs = { hello = "ok"; }; in
with pkgs;
let hello = hello; in # which hello?
hello
```

Prefer explicit attr access in teaching examples; `with` is a readability tax on recursion debugging.

Pattern F — `lib.fix` / fixed points gone wrong

```
# conceptual - when extends and self-reference collide badly
lib.fix (self: {
  a = self.b;
  b = self.a;
})
```

Fixed points are powerful for overlays and module-ish composition; cycles still blow up when forced. Prefer small pure functions until you need `fix`.

Theory 3 — Debugging toolkit

`builtins.trace`

```
builtins.trace "here" value
```

Prints to stderr when the `trace` expression itself is forced (i.e. when something needs `value` through the trace).

`builtins.tryEval`

```
builtins.tryEval (throw "boom")
# → { success = false; value = false; } # value is dummy on failure

builtins.tryEval 42
# → { success = true; value = 42; }
```

Use for soft probing—not for swallowing production errors silently. Infinite recursion may **not** be caught the way you hope; catalog behavior on your Nix version.

`seq` vs `deepSeq`

```
builtins.seq { a = throw "late"; } "outer-ok"
# → "outer-ok" # only WHNF of the attrset

builtins.deepSeq { a = throw "late"; } "never"
# → error: late
```

`lib.debug.traceVal` / `traceSeq` / `traceValSeq` (`nixpkgs`)

When you have `lib` from `nixpkgs`:

```
lib.debug.traceVal someAttr
lib.debug.traceValSeqN 2 nested
```

Prefer library helpers over ad-hoc nested `trace` once you are in a flake with `pkgs/lib`.

`NIX_SHOW_STATS` and evaluator metrics

```
NIX_SHOW_STATS=1 nix eval --impure --expr '1+1'
```

Look for counts of thunks, allocations, and GC—useful when comparing “eval is slow” hypotheses, not as a daily dashboard.

`nix-instantiate --eval` / `nix eval`

```
nix eval --expr 'let x = x; in x' # expect infinite recursion
nix eval --json .#nixosConfigurations.lab.config.networking.hostName
```

For module cycles, force **one** leaf option rather than the entire `config`.

Bisect strategy for real flakes

1. Force smallest leaf (hostname, one service enable)
2. Binary-search modules list (comment half of imports)
3. Within suspect module, force each option default path
4. Draw DAG of option dependencies on paper
5. Only then add temporary trace

Remove all temporary traces before merge.

Theory 4 — When to force (and when not to)

Goal	Tool	Risk
Ensure side-effect config is checked early	<code>assertions</code> / <code>warnings</code> in modules	Good default
Fail fast if a secret path missing	<code>assert</code> in module, or activation script	Prefer module-time when pure
Make a function strict in an arg	<code>seq</code> at the start of the body	Can defeat sharing
Serialize entire config for tests	<code>deepSeq</code> or <code>lib.generators.toJSON</code>	Huge cost if done casually
“Fix” mysterious laziness bugs with <code>deepSeq</code> everywhere	Don’t	Hides design issues; tanks eval

Warning

`deepSeq` is a diagnostic scalpel and a test hammer—not a seasoning for every module.

Theory 5 — Eval vs build vs activation (where time goes)

Phase	What runs	Failure shape
Eval	Nix language → derivation graph / config attrset	Type errors, infinite recursion, missing options
Build / substitute	Realize store paths	Compile errors, hash mismatch, sandbox
Activation	Switch system generation	<code>systemd</code> failures, scripts, secret decrypt

Symptoms that feel like “Nix is broken” are often **eval**. Timing with `time nix eval ...` vs `time nix build ...` separates the budget before you tune `max-jobs` (see [Nix performance](#)).

Worked example — Construct, diagnose, fix

Broken

```
# broken-cycle.nix
rec {
  root = "/var/lib/app";
  data = "${data}/files"; # oops: meant root
}
```

```
nix eval -f broken-cycle.nix data
# infinite recursion encountered
```

Fixed

```
rec {
  root = "/var/lib/app";
  data = "${root}/files";
}
```

Module-shaped fix

Prefer:

```
{ lib, config, ... }:
{
  options.my.app.root = lib.mkOption {
    type = lib.types.path;
    default = "/var/lib/app";
  };
  options.my.app.dataDir = lib.mkOption {
    type = lib.types.path;
    default = "${config.my.app.root}/files"; # depends on sibling option via config - OK if
  };
}
```

If `root`'s default ever depended on `dataDir`, you create a cycle. Draw a DAG of option dependencies when stuck.

Force a leaf, not the world

```
# Good: small demand
nix eval .#nixosConfigurations.lab.config.networking.hostName

# Expensive: full system derivation
time nix eval .#nixosConfigurations.lab.config.system.build.toplevel --raw >/dev/null
```

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/evaluator
cd ~/lab/nixos/internals/evaluator
```

Use pure files here before editing Capstone modules.

Lab 1 — Watch memoization

Create `thunks.nix`:

```
let
  expensive = builtins.trace "EXPENSIVE" (builtins.foldl' (a: b: a + b) 0 (builtins.genList
    bag = { x = expensive; y = expensive; });
in
{
  first = bag.x;
  second = bag.y;
}

nix eval -f thunks.nix first
nix eval -f thunks.nix second
nix eval -f thunks.nix
```

Record how many times `EXPENSIVE` prints in each case. Explain sharing in one paragraph in `NOTES.md`.

Acceptance: You can predict print counts before running, then verify.

Lab 2 — Infinite recursion zoo

Create `cycles.nix` with **three** named attributes that each fail differently when forced:

1. Direct `let` cycle
2. Mutual `a/b` cycle
3. `rec set` cycle

Force each with `nix eval -f cycles.nix <name>` and paste the error fingerprints into `NOTES.md`.

Then fix all three; leave the broken versions commented for teaching.

Acceptance: Three broken + three fixed forms committed; each error reproduced once.

Lab 3 — `seq` vs `deepSeq`

```
# force.nix
let
  t = {
    a = builtins.trace "a" 1;
    b = builtins.trace "b" (throw "b-fail");
  };
in {
  onlyWhnf = builtins.seq t "ok-whnf";
  full = builtins.deepSeq (builtins.removeAttrs t ["b"]) "ok-deep";
  # fullBoom = builtins.deepSeq t "never";
}
```

Evaluate `onlyWhnf` and `full`. Uncomment `fullBoom` once and capture the error. Write when WHNF is enough for module work.

Lab 4 — Trace a tricky real eval (your flake)

Pick **one** leaf from your lab host:

```
cd /path/to/your/flake
nix eval .#nixosConfigurations.<host>.config.networking.hostName
# or a service option you care about
```

```
time nix eval .#nixosConfigurations.<host>.config.system.build.toplevel --raw >/dev/null
```

If eval is slow, binary-search:

1. Comment out recently added modules (or use a slimmed `modules = [ ... ] copy`).
2. Force smaller slices with `nix eval ...config.services.<name>.enable`.
3. Add a temporary `lib.trace` only around the suspect option default.

Document the path of the investigation—even if the conclusion is “eval cost is nixpkgs baseline.”

Acceptance: Written investigation path with wall-clock numbers for leaf vs toplevel.

Lab 5 — Intentional soft failure with tryEval

```
# try.nix
{
  ok = builtins.tryEval (1 + 1);
  bad = builtins.tryEval (1 + "x");
}
```

```
nix eval -f try.nix --json
```

Note: type errors vs `throw` vs infinite recursion behave differently under `tryEval`—catalog what you observe on **your** Nix version.

Lab 6 — Module-shaped cycle (optional, high value)

Create a tiny `lib.evalModules` demo with two options whose defaults reference each other. Force eval; capture the error. Redesign so defaults form a DAG (one root option, one derived).

```
# module-cycle-demo.nix - sketch; expand with lib.evalModules
# Show broken defaults, then fixed defaults from a pure root path.
```

Acceptance: One paragraph on why “defaults depending on config” is a design smell when it re-enters the same option.

Lab 7 — Stats snapshot

```
NIX_SHOW_STATS=1 nix eval --impure --expr 'builtins.length (builtins.attrNames (import <nixpkgs> <pkgs> | tee stats-attrnames.txt
```

Compare (roughly) against evaluating a tiny expression. Do **not** over-interpret; note that large attrset demand is expensive.

Failure modes (quick reference)

Symptom	Likely class	First move
<code>infinite recursion encountered</code>	Cycle	Force smaller attr; draw option DAG
Eval hangs, no error	Cycle or huge force	Interrupt; bisect modules
Trace never prints	Not forced	Wrap the <i>used</i> path
Trace floods	Mapped over huge list	Narrow value; use <code>traceValSeqN</code>
<code>tryEval</code> “lies”	Uncatchable error class	Catalog; don’t rely for cycles
Multi-minute silence then build	Eval cost	<code>time nix eval</code> leaf vs toplevel

Common gotchas

Symptom	Theory link
<code>infinite recursion encountered</code> with no file	Cycle in options/ <code>rec</code> ; force a smaller attr
Trace never prints	Expression never forced; wrap the <i>used</i> value
Trace prints thousands of times	You forced a list map; narrow the traced value
Eval hangs, no error	Likely cycle; interrupt and bisect modules
<code>deepSeq</code> “fixed” a bug	You hid a lazy design smell; redesign dependencies
with <code>pkgs</code> ; + name collision	Shadowing; drop <code>with</code> while debugging
Module default uses <code>config</code> of same option	Define default from other options or pure constants
Temporary traces left in production modules	Grep before merge; CI can ban <code>builtins.trace</code> in modules/

Checkpoint

- ☐ Explain thunk / WHNF / memoization in your own words
- ☐ Three infinite-recursion examples constructed and fixed
- ☐ `seq` vs `deepSeq` demo committed
- ☐ Trace notes on one real flake option
- ☐ `NOTES.md` with three personal gotchas
- ☐ No permanent `deepSeq` left in production modules
- ☐ Lab workspace under `~/lab/nixos/internals/evaluator`

Commit

```
git add .  
git commit -m "lab(nixos): evaluator thunks, cycles, and trace discipline"
```

Write three personal gotchas before continuing.

Next

→ [Module system in depth](#) — types, merge, priorities, and assertions that fail at eval.

Module system in depth

Module system in depth

Goal: Use a real **type catalog**, compose with **mkIf** / **mkMerge** / priorities correctly, add assertions, and redesign one production-shaped module so bad configs fail at *eval* time.

i Note

You already *consumed* modules (NixOS host part) and wrote a custom one (custom modules chapter). This chapter is the **types + merge** pass—not another “enable nginx” tutorial.

Why this chapter exists

Untyped **attrs** options accept anything until activation or first request fails. Priority mistakes silently lose settings (**mkDefault** overridden by accident, or **mkForce** used as a hammer). Capstone hosts need modules that **reject nonsense early** and **merge predictably** across roles.

Theory 1 — Anatomy of a module (recall with precision)

A module is a function (or attrset) producing some of:

```
{ config, lib, pkgs, ... }:  
{  
  imports = [ ./peer.nix ];  
  options = { /* option declarations */ };  
  config = { /* option definitions */ };  
}
```

Field	Role
imports	DAG of other modules
options	Schema: types, defaults, descriptions
config	Values / definitions (often under mkIf)
freeform / _module	Advanced; respect unless you know why

Evaluation merges **all** definitions according to type + priority, then checks types and assertions.

Freeform modules (awareness)

Some modules accept undeclared attrs (freeform types). That is convenient for “pass settings through” and dangerous for typos. Prefer explicit options for anything Capstone treats as policy.

Theory 2 — Type catalog (practical reference)

Import `lib.types` as `types` in real modules. Below is the catalog you should be able to **choose from**, not memorize constructor source.

Scalars & paths

Type	Accepts	Notes
<code>types.bool</code>	<code>true/false</code>	Prefer over “stringly” enables
<code>types.int</code> / <code>types.ints.positive</code> / <code>types.ints.u16 ...</code>	Integers with bounds	Ports, counts
<code>types.port</code>	Port range	Prefer over bare int for listeners
<code>types.float</code>	Floats	Rare in OS config
<code>types.str</code>	String	Most text
<code>types.nonEmptyStr</code>	Non-empty string	Hostnames, user names
<code>types.singleLineStr</code>	No newlines	Avoid injection in unit files
<code>types.lines</code> / <code>types.commas</code>	Multi-line / comma merge	Distinct merge semantics
<code>types.path</code>	Path-like	Coerces some strings; store paths common
<code>types.package</code>	Derivation/package	For <code>environment.systemPackages</code> style
<code>types.shellPackage</code>	Package with shell	Specialized
<code>types.passwdEntry</code>	Shadow-safe strings	Passwords (still don’t put secrets in store)
<code>types.str</code>		Domains, tokens shape
<code>types.strMatching</code>	Pattern	
<code>"regex"</code>		
<code>types.enum ["a" "b"]</code>	Closed set	Modes
<code>types.either t1 t2</code>	Union	e.g. <code>bool</code> or <code>str</code>
<code>types.oneOf [t1 t2 ...]</code>	Multi union	Prefer small sets
<code>types.nullOr t</code>	<code>null</code> or <code>t</code>	Optional values

Type	Accepts	Notes
<code>types.uniq t</code>	Unique merge	Only one definition wins (strict)
<code>types.anything</code>	Escape hatch	Avoid in public modules
<code>types.raw</code>	Untyped raw	Advanced / internal

Collections

Type	Merge behavior (intuition)
<code>types.listOf t</code>	Concatenate definitions
<code>types.attrsOf t</code>	Deep-merge by attr name
<code>types.lazyAttrsOf t</code>	Lazier attrs (perf / recursion)
<code>types.attrs</code>	Freeform attrs (weak)
<code>types.attrsWith { ... }</code>	Configurable attrs (modern)
<code>types.loaOf t</code>	Deprecated list-or-attrs legacy

Structure

Type	Use
<code>types.submodule { options = ...; }</code>	Nested option trees
<code>types.submoduleWith { modules = ...; }</code>	Modular submodules
<code>types.deferredModule</code>	Module value delayed
<code>types.optionType</code>	Type of types (rare)

Function-ish / paths to callables

Type	Use
<code>types.functionTo t</code>	Options that are functions
<code>types.raw + convention</code>	When types can't express it

Useful helpers

```
types.addCheck t predicate
types.coercedTo from coerceFn to
lib.mkOption {
  type = types.submodule {
    options.enable = lib.mkEnableOption "this feature";
    options.port = lib.mkOption {
      type = types.port;
      default = 8080;
```

```

    };
};
default = { };
}

```

`lib.mkEnableOption "desc" → bool option, default false, standard description.`

Choosing types under pressure

Situation	Prefer
On/off feature	<code>mkEnableOption</code> / <code>types.bool</code>
Port	<code>types.port</code>
Hostname / domain	<code>types.strMatching</code> or <code>nonEmptyStr</code> + assertion
Multi-instance services	<code>attrsOf (submodule ...)</code>
Free-form upstream JSON	<code>attrs</code> or structured submodule if you own schema
Secret material	Never as string in store; path option only

Theory 3 — Definitions: `mkIf`, `mkMerge`, priorities

`mkIf`

```

config = lib.mkIf config.services.myapp.enable {
  users.users.myapp = { isSystemUser = true; group = "myapp"; };
  # ...
};

```

False conditions **drop** definitions (they do not define `null` everywhere).

`mkMerge`

```

config = lib.mkMerge [
  (lib.mkIf cfg.enable { ... })
  (lib.mkIf cfg.tls { ... })
  { assertions = [ ... ]; }
];

```

Prefer `mkMerge` over huge nested `//` trees when multiple independent conditionals apply.

Priority ladder (high level)

Helper	Intent
<code>mkOptionDefault</code>	Lowest-ish option default layer
<code>mkDefault</code>	Soft default; user config should win
<code>mkOverride n</code>	Numeric priority (<code>mkDefault 1000</code> , normal 100, ...)
<code>mkForce</code>	High priority “I really mean this”
<code>mkFixStrictly</code> / order-related	Advanced ordering

Discipline:

1. Modules ship `mkDefault` for opinionated defaults.
2. Hosts use plain assignments or selective `mkForce`.
3. If you need `mkForce` everywhere, your module boundaries are wrong.

Example conflict

```
# module
services.openssh.enable = lib.mkDefault true;

# host
services.openssh.enable = false; # wins over mkDefault

# role module
networking.firewall.enable = lib.mkForce true;

# host tries
networking.firewall.enable = false; # loses unless higher override
```

Debugging silent loss

When a setting “does nothing”:

1. Search all definitions of that option path.
2. Check for `mkDefault` vs plain vs `mkForce`.
3. Confirm the defining module is actually `imports-ed`.
4. Confirm `mkIf` condition is true.

5. Confirm type merge didn't drop invalid values (eval error should usually surface).

Theory 4 — Assertions & warnings

```
config = {
  assertions = [
    {
      assertion = !(cfg.enable && cfg.package == null);
      message = "myapp: enable requires package";
    }
    {
      assertion = cfg.port != 22 || !config.services.openssh.enable;
      message = "myapp: port 22 conflicts with OpenSSH on this host pattern";
    }
  ];
  warnings = lib.optional cfg.legacyMode ''
    myapp.legacyMode is deprecated; migrate to settings.v2
  '';
};
```

Assertions fail **evaluation**—perfect for Capstone CI (`nix flake check`).

Mechanism	Fails eval?	Use
Type check	Yes	Shape of values
assertions	Yes	Cross-option policy
warnings	No	Deprecations, soft policy
Activation scripts	No (too late for CI purity)	Runtime environment

Theory 5 — Submodules & `attrsOf` services

Pattern for multi-instance:

```
options.services.myapp.instances = lib.mkOption {
  type = types.attrsOf (types.submodule ({ name, ... }: {
    options = {
      enable = lib.mkEnableOption "instance ${name}";
    }
  }));
};
```

```

        port = lib.mkOption { type = types.port; };
    };
    }));
    default = { };
};

```

Then map instances to systemd units with `lib.mapAttrs'`.

```

# sketch
systemd.services = lib.mapAttrs' (name: inst:
  lib.nameValuePair "myapp-${name}" (lib.mkIf inst.enable {
    description = "myapp ${name}";
    # ...
  })
) cfg.instances;

```

Watch for name collisions and firewall port unique assertions across instances.

Theory 6 — Import graphs and role modules

Pattern	When
hosts/lab imports profiles/server.nix	Role defaults
profiles/server imports modules/ssh.nix + modules/hardening.nix	Composition
Host sets <code>my.role = "edge"</code> and one module switches	Fewer imports; more logic

Prefer **shallow import graphs** you can draw. Deep diamond imports with conflicting `mkForce` are how priorities become superstition.

Worked example — Redesign sketch

Before (weak)

```

{ config, lib, ... }:
{
  options.my.proxy = lib.mkOption {
    type = lib.types.attrs;
    default = { };
  };
}

```

```

};
config = {
    # hope the attrs are right...
};
}

```

After (typed)

```

{ config, lib, pkgs, ... }:
let
  inherit (lib) types mkIf mkOption mkEnableOption;
  cfg = config.my.proxy;
in {
  options.my.proxy = {
    enable = mkEnableOption "edge reverse proxy front door";
    domain = mkOption {
      type = types.strMatching "[a-z0-9.-]+";
      example = "lab.example.com";
      description = "Primary public hostname";
    };
    upstream = mkOption {
      type = types.str;
      example = "http://127.0.0.1:8080";
    };
    listenPort = mkOption {
      type = types.port;
      default = 443;
    };
    acme = mkOption {
      type = types.bool;
      default = true;
    };
  };
};

config = mkIf cfg.enable {
  assertions = [
    {
      assertion = cfg.listenPort != 0;
      message = "my.proxy.listenPort must be non-zero";
    }
    {
      assertion = !(cfg.acme && cfg.listenPort == 80);
      message = "my.proxy: ACME mode expects non-80 exclusive conflict check - adjust to y
    }
  ];
};

```

```
    # wire services.nginx / caddy / traefik here with mkDefault
  };
}
```

Bad configs that must fail

```
# wrong type
my.proxy.listenPort = "443";

# failed assertion
my.proxy.enable = true;
my.proxy.listenPort = 0;
```

Save stderr as CI fixtures later (Capstone rebuild proof).

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/modules
cd ~/lab/nixos/internals/modules
```

Copy or reference **one** real module from your lab flake (proxy, app, or secrets-adjacent).

Lab 1 — Type catalog cheat sheet (you write it)

In `TYPES.md`, list **at least 15** types from the catalog with:

- one-line purpose
- one good example value
- one bad value that should fail

Do **not** paste the table from this chapter blindly—rewrite in your words.

Acceptance: 15 rows in your own language; at least 3 collection/structure types.

Lab 2 — Redesign one module

1. Identify untyped `attrs`, raw strings for ports, or missing `mkEnableOption`.
2. Rewrite options with proper types + descriptions.
3. Move conditionals under `mkIf` / `mkMerge`.
4. Add 2 assertions (conflict + required field).
5. Ensure host still evaluates:

```
nix eval .#nixosConfigurations.<host>.config.system.build.toplevel --raw >/dev/null
# or
nixos-rebuild build --flake .#<host>
```

Acceptance: Diff shows types + assertions; toplevel builds.

Lab 3 — Priority drill

Create a tiny three-file demo (can be pure `lib.evalModules`):

```
# eval-demo.nix - sketch
{ lib ? (import <nixpkgs> { }).lib }:
lib.evalModules {
  modules = [
    ({ lib, ... }: {
      options.value = lib.mkOption { type = lib.types.int; };
      config.value = lib.mkDefault 1;
    })
    ({ lib, ... }: {
      config.value = 2; # plain
    })
    ({ lib, ... }: {
      # toggle these in experiments:
      # config.value = lib.mkForce 3;
    })
  ];
}
```

```
nix eval -f eval-demo.nix config.value
```

Record the winner for: default vs plain vs force. Paste results into `PRIORITIES.md`.

Lab 4 — Intentional bad config

Against your redesigned module, write `bad-host.nix` snippets that **must** fail eval (wrong type, failed assertion). Run them and save stderr excerpts—these become CI fixtures later.

Acceptance: Two distinct failure classes with captured stderr.

Lab 5 — Multi-instance sketch (optional)

If your Capstone has only one service instance, still sketch `attrsOf submodule` for a fictional second instance and document how ports would be asserted unique.

Lab 6 — Description quality audit

Pick five options you own. Ensure each has a **description** a stranger could use. Missing descriptions are how “options search” fails future you.

```
# explorative: use nixos-option on a live system or docs
```

Failure modes

Symptom	Fix
Option used but not defined	Declare option or fix path typo
Settings silently ignored	Priority war; search all definitions
Infinite recursion	Defaults cycle (evaluator chapter)
<code>listOf</code> duplicates explode unit names	<code>lib.unique</code> or set-like design
Assertion message useless	Put what and where in message

Common gotchas

Symptom	What to do
The option ... is used but not defined	Missing option decl or wrong path
Infinite recursion in modules	Option default depends on own config cycle (evaluator)
Settings silently ignored	Lost priority war; check <code>mkDefault</code> / <code>mkForce</code>
<code>listOf</code> duplicates explode	Dedup with <code>lib.unique</code> or design set-like options
Submodule names vs <code>name</code> arg	Use <code>{ name, ... }</code> : pattern carefully
Secrets in typed options	Types don't encrypt; still use <code>sops</code> / <code>agenix</code>
Over-using <code>types.attrs</code>	Prefer <code>attrsOf submodule</code> or explicit options
Assertions only on happy path	Add at least one negative fixture

Checkpoint

- ☐ `TYPES.md` with 15 types in your words
- ☐ One production module redesigned with real types
- ☐ 2 assertions
- ☐ Priority demo understood (`PRIORITIES.md`)
- ☐ Bad configs fail at eval with clear messages
- ☐ Host/toplevel still builds
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "lab(nixos): module types catalog and redesigned module"
```

Write three personal gotchas before continuing.

Next

→ [Store and daemon](#) — multi-user trust, GC roots, sandbox, signatures.

Store and daemon

Store and daemon

Goal: Explain multi-user Nix trust boundaries, GC roots, sandbox isolation on Linux, and substituter signature policy—then prove it by reading a real sandbox build log and mapping live roots on your machine.

Why this chapter exists

“It came from the cache” and “sandbox off fixed my build” are not strategies. Capstone CI, remote builders, and disaster recovery all assume you understand **who can write the store**, **what a root pins**, and **what isolation actually guarantees**.

Theory 1 — Single-user vs multi-user

Mode	Builder identity	Typical install	Risk profile
Single-user	Your UID owns <code>/nix/store</code>	Rare on servers; some dev installs	User compromise store compromise
Multi-user	<code>nix-daemon</code> (root) builds; clients speak to daemon	Default multi-user installer / NixOS	Trusted users can influence builds; untrusted cannot freely poison store

On NixOS you almost always run **multi-user** with `nix-daemon.service`.

```
systemctl status nix-daemon
ps aux | grep '[n]ix-daemon'
```

Clients (`nix build`, `nixos-rebuild`) send build goals to the daemon over a Unix socket (`/nix/var/nix/daemon-socket/socket`).

Trusted users

```
nix.settings.trusted-users = [ "root" "@wheel" ];  
# or classic nix.conf: trusted-users = root @wheel
```

Trusted clients may set extra substituters, import unsigned paths (depending on config), and change certain build parameters. **Do not** put random CI runners in **trusted-users** without understanding that.

Allowed users

allowed-users gates who may talk to the daemon at all. Default is often *****.

Threat sketches

Actor	Without trusted	With trusted
Unprivileged shell user	Can request builds of known paths; limited conf changes	May configure extra caches, more build power
Compromised CI job as trusted	Can push/pull trust boundaries	Treat as root-adjacent for store policy
Stolen laptop user session	Profile + user roots	Same + any trusted abilities

Theory 2 — Store paths, validity, and roots

Path anatomy (recall)

```
/nix/store/<hash>-<name>
```

The hash fingerprints the derivation inputs (input-addressed world—the default you live in). The [CA derivations](#) chapter contrasts **content-addressed** experimental paths.

Validity

A path is **valid** if the daemon registered it (built or substituted). Query:

```
nix path-info /nix/store/...-hello-...  
nix path-info -r /nix/store/...-hello-... # closure  
nix path-info -S /nix/store/...           # size  
nix path-info --json /nix/store/... | head
```

GC roots

Garbage collection deletes **unreferenced** valid paths. Roots include:

Root class	Examples
Profiles	<code>/nix/var/nix/profiles/system</code> , per-user profiles
Generations	System generation symlinks
Result symlinks	<code>./result</code> , <code>./result-1</code> from <code>nix build</code>
Runtime roots	Running processes' mapped files (when gcroots scanning enabled)
Explicit roots	<code>nix-store --add-root</code> , GC root dirs under <code>/nix/var/nix/gcroots</code>

```
nix-store --gc --print-roots | head -50
# modern:
nix-store gc --dry-run # careful; prefer print-roots first
ls -la /nix/var/nix/gcroots/
ls -la /nix/var/nix/profiles/system* | head
```

Teaching point: deleting a git repo does **not** free its build products if a profile or **result** link still roots them.

Disk full playbook (operator)

```
df -h /nix
nix-store --gc --print-dead | head
# review old system generations before deleting
sudo nix-collect-garbage -d # aggressive; know what you destroy
```

Prefer generation retention policy on lab hosts over panic GC mid-build.

Theory 3 — Sandbox (Linux)

Why sandbox

Builds should see:

- declared inputs only
- no network (except fixed-output derivations with hashed output)

- clean env, controlled /dev, empty /build

So “works on my laptop” impurities become hard errors.

Mechanisms (Linux)

Rough stack (details evolve; concepts stable):

Mechanism	Role
User namespaces / uid remap	Build user isolation
Mount namespaces	Private /, bind store inputs
PID / IPC / UTS namespaces	Process isolation
Network namespace	No ambient net
Seccomp / syscall filters	Reduce kernel attack surface (where enabled)
sandbox-paths	Extra allowed paths (dangerous if wide)

Config knobs

```
# NixOS
nix.settings.sandbox = true;           # true | false | relaxed
nix.settings.extra-sandbox-paths = [ ];
# nix.settings.sandbox-fallback = false;
```

Value	Meaning
true	Full sandbox (preferred)
relaxed	Some impurities allowed (debug)
false	Off — only for local debugging, never as silent prod default

```
nix show-config | grep -i sandbox
# or
nix config show | grep -i sandbox
```

Fixed-output derivations (FOD)

Fetchers (fetchurl, etc.) may have network **if** the output hash is fixed. Wrong hash → fail. This is intentional purity for source fetching.

Nested virt / KVM note

`nixosTest` and some builds need `kvm` features. That is **not** the same as disabling sandbox. Keep sandbox on; provision KVM correctly on the Elitebook/lab host (see front-matter labs).

Theory 4 — Substituters and signatures

Flow

1. Eval produces derivation goals.
2. Daemon checks local store.
3. Queries substituters (HTTP binary caches).
4. Verifies signatures against **trusted public keys**.
5. Registers paths if valid.

Trust policy

```
nix.settings = {
  substituters = [
    "https://cache.nixos.org"
    "https://myorg.cachix.org"
  ];
  trusted-public-keys = [
    "cache.nixos.org-1:6NCHdD59X431oOgWypbMrAURkbJ16ZPMQFGspcDShjY="
    "myorg.cachix.org-1:..."
  ];
};
```

Misconfiguration	Outcome
Substituter without key	Paths refused (or require trust elevation)
Key without care	You accept binaries from that signer as “as good as build”
<code>require-sigs = false</code>	Dangerous; only for isolated experiments

On NixOS, `nix.settings.require-sigs` should stay **true** in normal life.

Signing your own cache (awareness)

CI often builds → signs → pushes to Cachix/Attic/Harmonia. Capstone hardening wires this into CI; this chapter only requires **why** the public key is in your flake/host config.

Theory 5 — Reading build logs

```
nix build nixpkgs#hello --rebuild # force local rebuild if substitutes exist
nix log /nix/store/...-hello-...drv
# or after failure:
nix log .#packages.x86_64-linux.default
```

Sandbox-related failure fingerprints:

Log signal	Likely cause
Network access disabled / connection errors in non-FOD	Impure net need
No such file or directory for /usr/...	Hardcoded FHS path
Permission denied under /nix/store	Trying to mutate inputs
Hash mismatch	FOD impurity or wrong expected hash
cannot build ... because ... is not valid	Missing input; corrupted store

`strace` on daemon builds is advanced and rarely needed; prefer log literacy first.

Theory 6 — Daemon lifecycle on NixOS

```
systemctl cat nix-daemon.service
systemctl restart nix-daemon # after nix.conf / nix.settings changes when needed
journalctl -u nix-daemon -e --no-pager | tail -100
```

When settings appear “ignored”: multi-user means **daemon** config, not only user `~/.config/nix/nix.conf` (unless allowed). Prefer declarative `nix.settings` on NixOS hosts.

Worked example — Trust interview (self)

Answer in writing:

1. Who can talk to the daemon?
2. Who is trusted?
3. Which substituters and which keys?
4. Is sandbox on?
5. What is your oldest system generation still rooted?
6. What would you change before exposing a shared builder on the LAN?

This interview becomes Capstone TRUST notes.

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/store-daemon
cd ~/lab/nixos/internals/store-daemon
```

Work on a machine with multi-user Nix (NixOS lab VM ideal).

Lab 1 — Map live roots

```
nix-store --gc --print-roots 2>/dev/null | tee roots.txt | head -80
readlink -f /run/current-system || true
ls -la /nix/var/nix/profiles/ | tee profiles.txt
```

In ROOTS.md, classify **10 roots** into: system generation, user profile, result link, gcroots explicit, other. Explain what would free a path you care about.

Acceptance: 10 classified roots; one “how I would free X” paragraph.

Lab 2 — Daemon & trust surface

```
systemctl cat nix-daemon.service | tee daemon-unit.txt
nix config show | tee nix-config.txt
grep -E 'trusted|substitut|sandbox|require-sigs|allowed' nix-config.txt
```

Answer in TRUST.md:

1. Who is trusted?
 2. Which substituters and keys?
 3. Is sandbox on?
 4. What would you change before exposing a shared builder?
-

Lab 3 — Sandbox log literacy

Force a small local build and capture the log:

```
nix build nixpkgs#hello --rebuild -L 2>&1 | tee hello-rebuild.log
# find drv / log:
nix log nixpkgs#hello | tee hello-nix-log.txt
```

If substitutes always win and `--rebuild` isn't available in your version, build a tiny local derivation:

```
# tiny.nix
{ pkgs ? import <nixpkgs> { } }:
pkgs.runCommand "lab-tiny" { } ''
  echo hello > $out
''

nix build -f tiny.nix -L 2>&1 | tee tiny.log
```

Annotate 5 lines in the log (phases, sandbox mentions, store paths).

Acceptance: Annotated log snippet in notes.

Lab 4 — Optional impurity demo (VM only)

Warning

Do this only on a disposable VM. Re-enable sandbox afterward.

1. Note current `sandbox` setting.
2. Temporarily set `sandbox = false` via `nix.settings` or a one-off documented experiment.
3. Observe that impure references may start “working.”
4. **Restore** `sandbox = true` and rebuild the host.
5. Record the moral in `TRUST.md`.

Prefer reading docs + logs over leaving sandbox off.

Lab 5 — Signature refusal mental model

Write a short scenario (no need to actually serve malware):

Cache C serves path P signed by key `K_bad`. Your host only trusts `K_official`.

What should the daemon do? What if you are a `trusted-user` and force import? Document answers—tie to Capstone CI signing later.

Lab 6 — Elitebook / KVM lab acceptance

On the lab host (metal or primary VM):

```
# free space for store + images
df -h / /nix 2>/dev/null || df -h /
# nested virt if host is itself a VM
# egrep -i 'vmx|svm' /proc/cpuinfo | head
```

Acceptance criteria (lab readiness):

- ☐ `nix-daemon` active

- ☐ `sandbox = true`
- ☐ `require-sigs` true (or documented exception)
- ☐ 15% free on store filesystem before heavy rebuilds
- ☐ You know how to open hypervisor console if SSH dies

Common gotchas

Symptom	What to do
<code>cannot connect to daemon</code>	<code>nix-daemon</code> down; socket perms; single-user mismatch
GC deleted something “needed”	Missing root; reinstall profile / rebuild system
Build needs network mid-compile	Package not pure; fix derivation, don’t disable sandbox forever
<code>untrusted substituter</code>	Key missing or not trusted for your user
Rootful Docker “fixed” Nix	Different problem; don’t conflate
Filling disk	Roots + old generations; GC hygiene chapter
Settings ignored	Restart daemon; use declarative <code>nix.settings</code>

Checkpoint

- ☐ Multi-user vs single-user explained
- ☐ `ROOTS.md` with classified roots
- ☐ `TRUST.md` with trusted-users, keys, sandbox
- ☐ One build log annotated
- ☐ Sandbox left **enabled** on lab host
- ☐ Lab free-space / daemon acceptance ticked
- ☐ Three personal gotchas

Commit

```
git add .  
git commit -m "lab(nixos): store daemon sandbox and substituter trust"
```

Write three personal gotchas before continuing.

Next

→ **Nix performance** — measure eval vs build, tune jobs/cores, justify settings.

Nix performance

Nix performance

Goal: Measure where time goes (eval vs download vs build), tune `max-jobs` / `cores` / remote builders / caches, and record a before/after rebuild timing with settings you can justify.

Why this chapter exists

NixOS rebuilds feel “slow” for different reasons. Cranking `--max-jobs 128` on a laptop with 8GB RAM and a spinning disk can thrash harder than serial builds. Capstone CI budgets and lab iteration speed both require **measurement first**.

Theory 1 — The rebuild time budget

A flake host rebuild roughly pays:

```
eval (modules + nixpkgs) → realize (substitute or build) → activate
```

Phase	Dominated by	Knobs
Eval	CPU single-thread-ish, memory, module count, IFD	eval-cache, avoid IFD, slim modules
Substitute	Network, cache hit rate, compression	binary caches, closer mirrors
Build	CPU cores, jobs, IO, remote builders	<code>max-jobs</code> , <code>cores</code> , builders
Activate	Service restarts, switch strategy	<code>switch</code> vs <code>boot</code> vs <code>test</code>

```
# rough wall clock
/usr/bin/time -v nixos-rebuild build --flake .#<host>
# or
time nix build .#nixosConfigurations.<host>.config.system.build.toplevel
```

Separate **cold** (empty download cache / first eval) vs **warm** timings or you will lie to yourself.

Elitebook 16 GB reality

Workload	Expectation
Leaf <code>nix eval</code> hostname	Seconds
Warm toplevel after small edit	Often substitute-heavy; minutes if many new builds
Cold after lock bump	Long; plan coffee + cache
Parallel <code>max-jobs=8</code> link storms	OOM risk; prefer 2-4 jobs

Theory 2 — Jobs vs cores

Definitions

Setting	Meaning
<code>max-jobs</code>	How many derivations realized in parallel
<code>cores</code>	How many cores advertised to each derivation (<code>NIX_BUILD_CORES</code> / <code>enableParallelBuilding</code>)

```
nix.settings = {  
  max-jobs = 4;    # or "auto"  
  cores = 2;      # per-job parallelism  
};
```

CLI overrides:

```
nix build -j 4 --cores 2 .#..  
nixos-rebuild build --flake .#host --max-jobs 4 --cores 2
```

Choosing numbers (rules of thumb)

Hardware	Starting point
4c/8t laptop, 16GB	<code>max-jobs</code> = 2-4, <code>cores</code> = 2
16c desktop, 64GB	<code>max-jobs</code> = 8, <code>cores</code> = 2-4
Many small builds	higher <code>max-jobs</code> , lower <code>cores</code>
Few huge compiles (gcc/llvm)	lower <code>max-jobs</code> , higher <code>cores</code>
Tiny rootfs / low RAM	lower both; watch OOM

Constraint: `max-jobs` * `cores` should not massively exceed physical cores + RAM headroom, especially with linking-heavy packages.

Remote builders

```
nix.distributedBuilds = true;
nix.buildMachines = [ {
  hostname = "builder";
  system = "x86_64-linux";
  maxJobs = 8;
  speedFactor = 2;
  supportedFeatures = [ "nixos-test" "benchmark" "big-parallel" "kvm" ];
  mandatoryFeatures = [ ];
} ];
```

Offload large builds; keep eval local unless you know your setup.

Feature	Why list it
kvm	nixosTest VMs
big-parallel	large compiles allowed
nixos-test	test driver builds

Theory 3 — Eval performance

Eval cache

Modern Nix can cache evaluation results for flakes when inputs unchanged.

```
nix config show | grep -i eval
# features / options evolve; check your version docs for eval-cache
```

Practical wins:

- Stable `flake.lock` → better cache reuse
- Avoid unnecessary `--impure`
- Avoid Import From Derivation (IFD) when possible

IFD cost

```
# expensive pattern: build something at eval time
let pkg = import (pkgs.fetchFromGitHub { ... }) { };
in pkg.something
```

IFD forces realization during eval—serial and painful in CI. Prefer flake inputs and generated files committed or produced in build phase.

Module count

Every module imported has a cost. Shared modules should be **focused**, not a megamodule imported everywhere with giant `mkIf` trees if you can split roles.

Memory

Large evals can exceed default stack/heap. Symptoms: evaluator OOM, thrashing. Fix structure before only raising limits.

```
# rough: watch RSS while evaluating toplevel
/usr/bin/time -v nix eval .#nixosConfigurations.lab.config.system.build.toplevel --raw >/dev
```

Theory 4 — Download & store IO

Lever	Effect
Hit <code>cache.nixos.org</code> + project cache	Avoid local compiles
<code>http2</code> / modern curl stack	Throughput (usually already fine)
Keep disk free	Avoid GC mid-build
<code>min-free</code> / <code>max-free</code> auto-GC	Prevent disk full mid-CI
SSD vs HDD	Link farms and unpack dominate on HDD

```
nix.settings = {
  min-free = 1073741824; # 1 GiB
  max-free = 5368709120; # 5 GiB
};
```

On Elitebook labs with small disks, aggressive auto-GC can surprise you mid-ISO build—know the knobs.

Theory 5 — What *not* to do

Anti-pattern	Why
<code>sandbox = false</code> for speed	Trust/purity regression
Disable binary caches “to force builds” in daily driver	Wastes hours
Infinite <code>max-jobs</code> on laptop	OOM + thermal throttle
Micro-optimizing before cache hits	Measure substitute rate first
Blindly copying someone else’s <code>nix.conf</code>	Hardware differs
Timing only after <code>switch</code>	Includes service downtime noise

Theory 6 — CI budget design

Goal	Technique
PR feedback < N minutes	Cache push from main; build only changed hosts if matrixed
Avoid rebuild world	Pin lock; share <code>/nix/store</code> cache between jobs carefully
<code>nixosTest</code> cost	Nightly <code>checks.slow-*</code> ; PR runs fast subset
Fail fast	Eval/checks before ISO builds

Write target budgets in `PERF.md` even for homelab CI.

Worked example — Measurement protocol

```
# 1) Note config
nix config show | egrep 'max-jobs|cores|substituters' | tee before-config.txt

# 2) Warm caches optionally with a cheap build
nix build nixpkgs#hello

# 3) Wall-clock system build (no switch)
/usr/bin/time -p nix build \
  .#nixosConfigurations.lab.config.system.build.toplevel \
  2>&1 | tee time-before.txt

# 4) Change nix.settings via configuration.nix / flake; rebuild once to apply daemon settings

# 5) Repeat timing → time-after.txt
```

Daemon-level settings change may need:

```
sudo systemctl restart nix-daemon
```

(On NixOS after a config switch that changes `nix.settings`.)

Interpreting results

Pattern	Interpretation
Eval leaf fast, toplevel slow, lots of building	Real compile cost; builders/cores
Many copying path	Substitute/network; cache placement
Eval alone dominates	IFD/modules; not max-jobs
After setting change wall clock worse	Thrash; lower parallelism

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/perf
cd ~/lab/nixos/internals/perf
```

Use your Capstone/lab flake host.

Lab 1 — Baseline timings

Record in `PERF.md`:

Run	Command	Wall seconds	Notes (cold/warm)
A	eval hostname only		
B	nix build toplevel		
C	nixos-rebuild build		

```
time nix eval .#nixosConfigurations.<host>.config.networking.hostName
time nix build .#nixosConfigurations.<host>.config.system.build.toplevel --print-build-logs
```

Capture `nix config show` snapshot as `config-before.txt`.

Acceptance: Three timed runs with cold/warm labels.

Lab 2 — Tune jobs/cores

Pick **one** hypothesis, e.g. “too many parallel link jobs thrash RAM.”

Apply via flake:

```
nix.settings.max-jobs = 2;  
nix.settings.cores = 4;
```

Switch config (or set user `nix.conf` on non-NixOS carefully), restart daemon if needed, re-time the **same** target as Lab 1.

Document winner settings for **this machine only**.

Lab 3 — Cache hit awareness

```
# After a toplevel build, approximate download vs build by reading logs (-L)  
# Count "copying path" vs "building '" lines from a log snippet
```

Optional: temporarily misconfigure a wrong extra substituter (unreachable) and observe timeout cost—then remove it. Note CI implications.

Lab 4 — Eval hygiene checklist

In `PERF.md`, audit your flake for:

- ☐ IFD sites (`import` of derivation results)
- ☐ Accidental `--impure` scripts
- ☐ Oversized `imports` on every host
- ☐ Dev shells pulled into system eval unnecessarily

Fix **one** real issue if found (or document “none found” with evidence).

Lab 5 — Remote builder decision (write-only if no hardware)

Write `BUILDERS.md`: when you would add a remote builder vs buy a better cache. Include `supportedFeatures` needed for `nixosTest` (kvm).

Lab 6 — Memory pressure drill (lab only)

While a parallel build runs:

```
# another terminal
free -h
# watch -n2 free -h
```

If swap thrash appears, lower `max-jobs` and re-time. Record decision.

Acceptance (Elitebook): Documented settings that do not OOM a 16 GB host during a typical host toplevel build.

Common gotchas

Symptom	What to do
Settings in <code>nix.conf</code> ignored	Multi-user: daemon config; restart daemon
Faster jobs, slower wall clock	RAM thrash / disk queue; lower <code>max-jobs</code>
Eval dominates always	Module/IFD issue, not cores
CI OOM on matrix	Cap jobs per runner
<code>auto</code> <code>max-jobs</code> surprises	Pin explicit numbers for servers
Timing after <code>switch</code> includes downtime	Use <code>build</code> for fair comparison
Auto-GC mid-ISO	Raise free space; schedule GC windows

Checkpoint

- ☐ Before/after timings in `PERF.md`
- ☐ Final `max-jobs` / `cores` recorded with rationale

- ☐ Cache vs build awareness notes
- ☐ One eval-hygiene audit completed
- ☐ No sandbox disabled “for speed”
- ☐ Three personal gotchas

Commit

```
git add .  
git commit -m "lab(nixos): rebuild performance measurement and tuning"
```

Write three personal gotchas before continuing.

Next

→ [Content-addressed derivations \(awareness\)](#) — IA vs CA without migrating Capstone.

Content-addressed derivations (awareness)

Content-addressed derivations (awareness)

Goal: Explain input-addressed vs content-addressed derivations, know the **experimental** status and feature flags, run a tiny optional experiment **or** produce a documented status memo with primary-source links—without putting production Capstone (or **26.05** lab hosts) on CA.

! Important

Content-addressed (CA) derivations remain an **experimental** path. This chapter is literacy and a sandbox experiment—not a mandate to migrate your host. Capstone A stays on default store semantics.

Why this chapter exists

Default Nix paths are **input-addressed**: the store hash fingerprints the build plan (derivations + inputs), not only the output bytes. Two builds that produce identical bytes can still get different paths if input metadata differs. CA aims at **output-hash identity**—powerful for early cutoff / better sharing—but the feature surface, UX, and nixpkgs readiness evolve. Experts need an honest mental model before blog hype.

You will hear CA in RFCs, release notes, and conference talks. Literacy prevents cargo-cult **experimental-features** on prod builders.

Theory 1 — Input-addressed world (today’s default)

```
inputs + build recipe → hash → /nix/store/<hash>-name
```

Consequences you already rely on:

Property	Implication
Rebuild with same inputs	Same path (reproducible <i>address</i>)
Different compiler flags	Different path even if binary identical
FODs	Special: output hash fixed; may network
Substitutes	Trust signatures of builders/caches

Trust is “did we run this recipe (or accept a signed substitute for it)?”

FODs vs CA (do not conflate)

	Fixed-output derivation	CA derivations (experimental)
Classic use	fetches, vendoring	general builds
Hash of	declared output	content-addressed store model
Maturity	Everyday	Experimental

FODs already teach “hash the result” for a *class* of builds. CA generalizes the idea; it is not “FODs for everything tomorrow.”

Theory 2 — Content-addressed idea

```
build → output bytes → hash → /nix/store/<hash>-name
```

Aspirations:

Goal	Intuition
Early cutoff	If output already exists with same content hash, skip dependent rebuilds more aggressively
Better dedup	Same bytes converge to same path across “recipes” that don’t matter
FODs generalized	More builds behave like “hash the result”

Challenges:

- Self-references in outputs (files embedding their own store path) need rewriting schemes
- Granularity (per-output CA, etc.)
- Cache/protocol/nixpkgs compatibility
- Debugging mental model shift
- Tooling and UX still moving

Theory 3 — Experimental feature surface

Exact names and defaults **change across Nix versions**. Always check:


```
nix --version
nix config show | grep -i experimental
man nix.conf | less    # search ca-derivations / content-address
```

Historically you will see flags in the family of:

- `ca-derivations`
- related: `dynamic-derivations`, `cgroups`, etc. (orthogonal but often discussed together)

Enable only on a **lab** profile:

```
# ephemeral lab - not Capstone default
nix.settings.experimental-features = [
  "nix-command"
  "flakes"
  "ca-derivations" # if supported by your Nix
];
```

Or local:

```
# illustrative - confirm flag name for your version
nix build --experimental-features 'nix-command flakes ca-derivations' ...
```

Reading status (primary sources)

Prefer in this order:

1. Nix release notes for your exact version
2. Official Nix manual sections on store / experimental features
3. nixpkgs issues/PRs tagged CA (for package readiness)
4. RFCs / discourse RFCs (design intent)

Blogs are optional commentary after primary sources. Version-skew blogs are how people enable the wrong flag.

Theory 4 — What CA does *not* magically fix

Claim	Reality
“CA = perfect reproducibility”	Reproducibility is about <i>determinism</i> of builds; CA is about <i>addressing</i>
“CA replaces sandbox”	Isolation still required
“Turn on CA, everything faster tomorrow”	Needs package + toolchain readiness; may slow or break
“Signatures no longer matter”	You still decide trust for substitutes
“Capstone should adopt CA”	No for this book’s Capstone A

Theory 5 — Relationship to Capstone and 26.05

Capstone should stay on **stable, default** derivation mode unless you explicitly document a lab-only branch. DR wipe/rebuild proofs should not depend on experimental store semantics.

Your **nixos-26.05** pin is about *module and package* stability. CA is a **Nix daemon** / **store** feature layer—orthogonal but coupled in operational risk if enabled fleet-wide.

Optional: track CA as a **roadmap** item in the retrospective—revisit when your Nix pin upgrades and primary sources look greener.

Theory 6 — Trust and substitutes under CA thinking

Even with CA aspirations, operations still answer:

Question	Still true?
Who is allowed to write to my store?	Yes
Which substituters do I trust?	Yes
Do I verify signatures?	Yes (policy)
Can a malicious builder hurt me?	Yes

CA is not a substitute for binary-cache trust discipline or remote-builder trust.

Theory 7 — Operational decision template

```
## Store semantics decision
- Mode: input-addressed (default)
- CA experimental: enabled? no
- Revisit when: <Nix version + release notes criteria>
- Owner: <you>
- Rollback: remove feature flag; rebuild hosts on default
```

Paste into Capstone docs so future-you does not “just try CA” mid-freeze.

Worked example — Conceptual comparison card

Dimension	Input-addressed	Content-addressed (experimental)
Path hash from	Recipe/inputs	Output content (design)
Same bytes, different flags	Different paths	<i>May</i> converge
Ecosystem maturity	Default everywhere	Experimental / partial
Debugging familiarity	High (this book)	Lower; fewer blog runbooks
Prod recommendation (this book)	Yes	Not yet for Capstone

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/ca
cd ~/lab/nixos/internals/ca
```

Lab 1 — Version & feature inventory (required)

```
nix --version | tee nix-version.txt
nix config show > nix-config.txt
grep -iE 'experimental|ca-|content' nix-config.txt || true
```

Open official docs for **your** version; save URLs in `SOURCES.md` (at least 2 primary links).

Write a 10–15 line **status summary**: what CA means, whether your Nix binary advertises the feature, and any warnings in the manual.

Lab 2 — Choose a track

Track A — Optional tiny experiment (if feature available)

1. Enable `ca-derivations` (or current equivalent) **only** in a disposable VM or user `nix.conf` overlay you can revert.
2. Build a trivial derivation twice with intentional non-semantic input noise if docs show how CA modes are requested for derivations (API differs by version—follow current manual).
3. Compare store paths and `nix path-info` output.
4. Capture logs in `experiment/`.
5. **Disable** experimental CA for daily lab work if it complicates Capstone.

If the manual’s API is unclear or broken on your pin, stop and switch to Track B—do not thrash for hours.

Track B — Status memo without enabling (always acceptable)

Write `CA-MEMO.md` (1 page) covering:

1. Input- vs content-addressed definitions
2. Problems CA wants to solve
3. Current experimental flags for your Nix version
4. Known limitations (self-refs, ecosystem)
5. Decision: **not** enabling on Capstone host, with revisit date (e.g. after next release notes)

6. Primary source links

7. Difference from FODs in your own words

Acceptance: Either Track A logs + revert proof, or Track B memo complete.

Lab 3 — Teach-back diagram

In `NOTES.md`, draw (ASCII is fine):

```
[user config] → eval → derivations → (IA path | CA path?) → substitute/build → store
```

Annotate where trust/signatures apply.

Lab 4 — Nixpkgs awareness scan

Search your local nixpkgs (or GitHub) for mentions:

```
# if you have nixpkgs checkout
rg -n "contentAddressed|__contentAddressed|ca-derivations" -g '*.nix' | head
```

Note whether anything in *your* critical path packages advertises CA—likely little for a lab host. Record findings.

Lab 5 — Explicit non-adoption

Add to Capstone docs (or lab notes):

```
## Store semantics
- Mode: input-addressed (default)
- CA: deferred until <date/criteria>
- Owner: <you>
```

This prevents a well-meaning future PR from flipping experimental flags mid-freeze.

Lab 6 — Analogy card (teach others)

Write three sentences max for a teammate:

1. **Input-addressed:** “The address is the recipe’s fingerprint.”
2. **Content-addressed:** “The address wants to be the loaf’s fingerprint.”
3. **Why Capstone waits:** “Experimental + ecosystem partial; DR should not depend on it.”

If you cannot explain without jargon, revise `CA-MEMO.md` until you can.

Interaction with other internals chapters

Chapter	Link to CA literacy
Evaluator	Eval still produces plans; addressing is store-side
Store/daemon	Feature flags live with daemon policy
Performance	Early cutoff is the performance <i>story</i> —measure before enabling
Release notes	Watch Nix + nixpkgs notes for CA maturity signals

Do not enable CA “for speed” without a benchmark and a rollback plan for the builder.

Common gotchas

Symptom / mistake	What to do
Blog for Nix 2.X vs your 2.Y	Trust <code>nix --version</code> + matching manual
Enabled CA on shared prod builder	Revert; isolate experiments
Assuming path equality bit equality always	Still verify; trust model differs
Spending the whole session debugging experimental UX	Fall back to Track B memo
Mixing CA talk with FOD confusion	FODs already hash outputs for <i>fetch</i> class
Capstone depends on CA	Redesign; default store only

Checkpoint

- ☐ `nix-version.txt` + feature inventory
- ☐ `SOURCES.md` with primary links
- ☐ Track A experiment notes **or** Track B `CA-MEMO.md`
- ☐ Explicit Capstone decision: stay input-addressed
- ☐ Three personal gotchas

Commit

```
git add .  
git commit -m "lab(nixos): CA derivations experimental awareness"
```

Write three personal gotchas before continuing.

Next

→ [nixosTest mastery](#) — harden checks for Capstone.

nixosTest mastery

nixosTest mastery

Goal: Harden a real `nixosTest` (or `flake checks`) suite: replace brittle `sleep` waits, use `waitForUnit` / `port` / `file` helpers correctly, handle multi-node networking, and leave CI-ready tests for Capstone.

Why this chapter exists

Ops chapters introduced `nixosTest`. Capstone A **requires** at least one test that catches a real regression. Flaky tests train you to ignore CI—worse than no tests. This chapter is production hygiene for the test layer.

Theory 1 — What `nixosTest` really runs

`pkgs.nixosTest` / `testers.runNixOSTest` (names vary by `nixpkgs` era) builds QEMU VMs from NixOS configs, wires a Python test driver, and runs scripted interactions.

```
# sketch - adjust to your nixpkgs helpers
pkgs.nixosTest {
  name = "lab-proxy";
  nodes.machine = { config, pkgs, ... }: {
    imports = [ ../modules/proxy.nix ];
    my.proxy.enable = true;
    # ...
  };
  testScript = ''
    machine.start()
    machine.wait_for_unit("multi-user.target")
    machine.wait_for_unit("nginx.service")
    machine.succeed("curl -f http://127.0.0.1/")
  ''
}
```

Wire into flakes:

```
checks = {
  lab-proxy = pkgs.nixosTest (import ../tests/lab-proxy.nix { inherit pkgs; });
}
```

```
};
```

```
nix build .#checks.x86_64-linux.lab-proxy -L  
nix flake check
```

i Note

nixpkgs has migrated helpers (`pkgs.testers.runNixOSTest`, `lib.nixos.runTest`, classic `nixosTest`). Use whatever your **26.05** pin documents; the **shape** (nodes + testScript) remains the skill.

Theory 2 — Flake (flaky) sources

Cause	Symptom	Fix
Fixed <code>sleep 10</code> Race before network	Passes on fast hosts, fails on CI DNS/curl fail intermittently	Wait on unit/port/log <code>wait_for_open_port</code> , <code>wait_until_succeeds</code>
Wall-clock timeouts too tight	CI under load fails	Budget timeouts; retry patterns in driver
Shared mutable state	Order-dependent	Independent nodes; pure configs
Missing <code>kvm</code>	Slow TCG or fail	Ensure nested virt / <code>supportedFeatures</code>
Non-determinism in app	Intermittent HTTP 500	Fix app or assert eventually

Theory 3 — Driver API patterns (Python DSL)

Exact methods evolve; concepts stable. Prefer:

```
machine.start()  
machine.wait_for_unit("network-online.target") # if appropriate  
machine.wait_for_unit("myapp.service")  
machine.wait_for_open_port(8080)  
machine.wait_until_succeeds("curl -f http://127.0.0.1:8080/health")  
machine.succeed("systemctl is-active myapp.service")  
machine.fail("curl -f http://127.0.0.1:8080/admin") # expect fail
```

```
machine.copy_from_vm("/var/log/myapp.log", "out")
print(machine.succeed("journalctl -u myapp -n 50 --no-pager"))
```

Anti-pattern

```
machine.succeed("sleep 30")
machine.succeed("curl ...") # still racy if app slower than 30s under load
```

sleep as a **backoff inside wait_until_succeeds** is different from a single blind sleep—prefer the wait helpers.

Multi-node

```
nodes = {
    server = { ... };
    client = { ... };
};

server.wait_for_unit("myapp.service")
client.wait_until_succeeds("curl -f http://server/health")
```

Hostname resolution between nodes is provided by the test network—use node names as hostnames per driver docs.

Theory 4 — Systemd-aware testing

Good services for tests:

- **Type=notify** when possible (ready = really ready)
- Explicit **After=**/**Wants=** for network
- Health endpoint or **ExecStartPost** readiness

If a unit is **active** but still binding ports, wait on **port** not only unit.

```
machine.wait_for_unit("nginx.service")
machine.wait_for_open_port(80)
```

Theory 5 — Secrets and tests

Never require production sops keys for unit tests. Patterns:

Pattern	Use
Test-only plaintext in VM	OK for non-sensitive lab secrets
<code>sops-nix</code> with age key generated in test	Heavier; sometimes worth it
Mock module <code>mkIf config.my.testMode</code>	Injects dummy secrets
Skip TLS in test, test TLS in one dedicated check	Split concerns

Capstone should have **at least one** test without human interactive decryption.

Theory 6 — CI integration

```
# sketch GitHub Actions
- uses: cachix/install-nix-action@...
- run: nix build .#checks.x86_64-linux.lab-proxy -L
```

Needs:

- KVM available **or** accept TCG slowness with longer timeouts
- Binary cache for nixpkgs to avoid rebuilding QEMU world every time
- Artifacts: fail logs on failure

```
nix log .#checks.x86_64-linux.lab-proxy
```

Elitebook / KVM lab acceptance

Check	Command / note
KVM usable	<code>/dev/kvm</code> exists; user in <code>kvm</code> group if required
Nested virt if host is VM	BIOS + hypervisor CPU flags
Test finishes	<code>nix build .#checks... -L</code> exits 0 twice
Disk headroom	Leave space for VM images under store

```
ls -l /dev/kvm || echo "no kvm"
# groups | tr ' ' '\n' | grep -E 'kvm|libvirt' || true
```

Theory 7 — Machine objects and multi-node patterns

```
# conceptual multi-node
start_all()
server.wait_for_unit("nginx.service")
client.wait_until_succeeds("ping -c1 server")
client.succeed("curl -sf http://server | grep -q ok")
```

Node names become hostnames in the test network. Use them deliberately; avoid hard-coded IPs unless the test network docs say so.

Theory 8 — Snapshotting vs rebuild cost

Long tests that rebuild the world each run train you to skip them. Techniques:

Technique	Effect
Smaller module under test	Faster eval/build
Shared fixture attrs	Less duplication
Separate <code>checks.slow-*</code>	CI nightly vs PR

Mastery is green **and** runnable.

Worked example — systemd journal assertion

```
machine.wait_for_unit("lab-tick.service")
machine.succeed("journalctl -u lab-tick.service -n 20 | grep -q tick")
```

Worked example — regression that fails when firewall wrong

```
machine.wait_for_open_port(443)
# backend should not be world-open if design is proxy-only
machine.fail("curl -f --max-time 2 http://127.0.0.1:8080/")
```

Adjust ports to your design; the skill is asserting **policy**, not only liveness.

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/tests
cd ~/lab/nixos/internals/tests
```

Prefer extending your **real** flake checks; if missing, create a minimal test module now.

Lab 1 — Inventory existing tests

```
nix flake show
# list checks
rg -n "nixosTest|runNixOSTest|testScript" -g '*.nix' /path/to/flake
```

In `SUITE.md`, list each check: purpose, flake attribute, known flakes.

Lab 2 — Eliminate sleep-only waits

Find `sleep` in test scripts:

```
rg -n "sleep" -g '*test*' /path/to/flake
```

Replace with `wait_for_unit` / `wait_for_open_port` / `wait_until_succeeds`. Re-run:

```
nix build .#checks.<system>.<name> -L
```

Run **twice** consecutively—tests that pass once and fail once are not done.

Acceptance: Critical path has no blind `sleep` as sole wait; two consecutive greens.

Lab 3 — Add one regression that would have bitten you

Pick a real past bug (wrong firewall port, typo in upstream, module `mkIf` inverted). Write a test that fails if that bug returns.

Lab 4 — Multi-node or client/server (stretch)

If single-node only so far, add a second node that curls the first **or** document why single-node is enough for Capstone A and add a second assertion class instead (journal + HTTP).

Lab 5 — Failure UX

Break the test on purpose (wrong unit name). Confirm the log shows journal excerpts. Add `machine.succeed("journalctl -u ...")` on failure paths if helpful. Record debugging time in `SUITE.md`.

Lab 6 — Flake check wiring audit

```
nix flake show
nix build .#checks.${(nix eval --raw --impure --expr 'builtins.currentSystem')} -L 2>/dev/null
nix flake check -L
```

List every check attr and its purpose in one table.

Lab 7 — Failure screenshot discipline

When a test fails, save:

1. Full command
2. Last 50 log lines
3. Whether rerun without change flakes

Flaky → fix waits; deterministic → fix system under test.

Common gotchas

Symptom	What to do
<code>error: unable to start guest</code>	KVM permissions, nested virt off
Hangs at <code>wait_for_unit</code>	Unit failed; print <code>systemctl status</code> / journal
Works interactively, fails CI	Timeouts; resource; sandbox/kvm
TLS/ACME tests fail offline	Don't use real ACME in VM; use self-signed or http-only mode
Eval of test is huge	Share modules carefully; don't import desktop profiles
Flake check runs <i>all</i> checks	Tag slow tests; <code>nix build</code> specific attr in iteration
Secrets block CI	Dummy secrets / <code>testMode</code>

Checkpoint

- ☐ Suite inventory in `SUITE.md`
- ☐ No critical path depends on blind `sleep` alone
- ☐ Test run twice green
- ☐ At least one regression-oriented assertion
- ☐ Notes on secrets strategy in tests
- ☐ KVM/lab acceptance noted
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "lab(nixos): harden nixosTest suite for capstone"
```

Write three personal gotchas before continuing.

Next

→ [Ecosystem map](#) — try one adjacent tool; keep/drop without Capstone rewrite.

Ecosystem map

Ecosystem map

Goal: Map the adjacent tooling landscape, **try exactly one** of devenv / Snowfall / nix-darwin in a disposable trial, and write a keep/drop decision against **your** flake layout—without rewriting Capstone mid-flight.

i Note

Syllabus rule: **try one, don't collect all.** Tool tourism is how deep curricula die mid-book.

Why this chapter exists

By now your layout is intentional: flake hosts, modules, HM, deploy, tests. The ecosystem constantly offers frameworks that promise less boilerplate. Some are worth it; many are expensive rewrites. Experts sample with a **comparison table**, not a weekend migration of Capstone main.

Theory 1 — Layers of “ecosystem”

Layer	Examples	Problem they target
Devshell UX	devenv, devbox, nix develop , direnv	Fast project environments
Flake structure frameworks	flake-parts, Snowfall, digga (legacy)	Scale modules/systems/homes
Darwin / non-NixOS	nix-darwin, HM standalone	Same declarative story on macOS
Deploy	deploy-rs, Colmena, nixinate	Ops & fleet part
Secrets	sops-nix, agenix	Services & security part
Caches	Cachix, Attic, Harmonia	Packaging / ops
Install / metal	nixos-anywhere, disko, images	Ops & fleet
UX wrappers	nh, nixos-rebuild-ng experiments	Faster day-2 commands
Editors / templates	templates repos, blueprint flakes	Speed of scaffolding

Today’s forced choice is among **devenv**, **Snowfall**, **nix-darwin**—three different layers. Pick the one that matches *your* pain:

If your pain is...	Prefer trying
Polyglot project shells, services in dev	devenv
Many hosts/homes, naming convention fatigue	Snowfall (or double-down on flake-parts you may already use)
You live on macOS half the time	nix-darwin
None of the above	Still pick one; learn by contrast

Theory 2 — devenv (project environments)

What it is: A Nix-powered dev environment tool (CLI + modules) focused on languages, processes, pre-commit, and local services—closer to “compose for dev” than to NixOS hosts.

Typical win: New contributors run one tool and get DB + language toolchain.

Typical cost: Another module system dialect; CI must speak devenv; overlap with **devShells** you already have.

```
# illustrative - follow current devenv docs for install
# nix profile install nixpkgs#devenv
# devenv init
# devenv shell
```

Compare to house default

Concern	nix develop + direnv	devenv
Mental model	Flake devShells	devenv modules + CLI
Services (postgres...)	Manual / process-compose DIY	First-class modules often
Capstone impact	Low if already working	Medium if dual standards

Theory 3 — Snowfall (flake library / structure)

What it is: A convention-heavy library for laying out flakes: systems, modules, packages, homes under standard paths, auto-discovery.

Typical win: Multi-host monorepos with consistent naming and less flake.nix glue.

Typical cost: Framework lock-in; debugging indirection; migration of a working hand layout.

Compare to flake-parts

Concern	Hand layout / flake-parts	Snowfall
Structure	Explicit imports	Convention + library
Learning	Your code is visible	Magic paths
Capstone rewrite cost	Low if already structured	Often high mid-flight

Snowfall and flake-parts both attack “structure,” differently. Trying Snowfall does **not** require abandoning flake-parts forever—decision is local.

Theory 4 — nix-darwin

What it is: Nix modules for macOS system configuration—spiritual cousin of NixOS modules, not a Linux VM.

Typical win: One repo language for laptop (Darwin) + servers (NixOS) + Home Manager.

Typical cost: Darwin modules NixOS modules; GUI/casks world; CI harder; not a substitute for your lab VM.

```
# only on a Mac you own - follow current nix-darwin + flakes docs
# darwin-rebuild switch --flake .#hostname
```

If you have no Mac, you may still **read** a nix-darwin sample config and write a comparison—or pick devenv/Snowfall for a hands-on trial instead.

Shared idea	Divergence
Module options, activation	Different option trees
HM integration common	GUI apps / Homebrew friction
Flakes	Same pin discipline

Theory 5 — Decision framework (keep/drop)

Score 1–5 for your context:

Criterion	Weight idea
Reduces <i>your</i> real weekly pain	High
Clear primary docs	High
Plays with flakes + lock discipline	High
Compatible with sops/deploy/tests	High
Rewrite cost of Capstone	Critical penalty
Bus factor / maintenance	Medium
Fashion / social media mindshare	Zero

Default Capstone policy: adopt only if score says “yes” **and** migration is a branch, not a rewrite of main before disaster recovery proof.

```
decision { keep-current, adopt-later, trial-only-drop }
```

Theory 6 — Ecosystem postcard (vocabulary)

You need names for comparative conversations even without installs:

Name	One-line purpose
flake-parts	Modular flake output composition
nixos-anywhere	Remote install from existing SSH
nh	Friendlier rebuild/search UX
Attic	Self-hosted binary cache
digga	Legacy flake framework (historical literacy)
blueprint / templates	Scaffold speed
nixos-generators	Image formats from NixOS configs
Lanzaboote	Secure Boot UKI/signing path

15 lines total in notes is enough—depth belongs to the **one** trial.

Theory 7 — Integration risks matrix

Trial tool	Risk to secrets	Risk to deploy	Risk to tests
devenv	Low	Low	Low (CI may need devenv)
Snowfall	Medium (path moves)	Medium	Medium (check wiring)
nix-darwin	Low on Linux lab	N/A for NixOS hosts	Separate CI

Never “finish” a trial by force-pushing Capstone restructure without a branch review.

Worked example — COMPARE.md skeleton

```
# COMPARE.md - devenv vs house layout (example)

## Context
- Capstone flake with nixosConfigurations.lab, sops-nix, deploy-rs
- Pain: polyglot shells inconsistent across repos

## Scores (1-5)

| Criterion | House | devenv | Weight |
|-----|-----|-----|-----|
| Weekly pain relief | 2 | 4 | 5 |
| Docs clarity | 4 | 4 | 4 |
| Flake/lock fit | 5 | 4 | 5 |
| sops/deploy/tests | 5 | 3 | 5 |
| Capstone rewrite cost | 5 | 2 | 5 |

## Decision
We will **not** adopt before Capstone freeze because rewrite cost outweighs shell pain;
revisit after Capstone freeze if side projects still hurt.

## Evidence from trial/
- commands run:
- what broke:
- time spent:
```

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/ecosystem/trial
cd ~/lab/nixos/internals/ecosystem
```

Do **not** trial on the only copy of Capstone; branch or separate directory.

Lab 1 — Map your current layout (30 min)

In LAYOUT.md:

```
flake.nix outputs:
  nixosConfigurations: ...
  homeConfigurations / HM-as-module: ...
  devShells: ...
  checks: ...
  packages: ...
  deploy / colmena: ...
modules/: ...
hosts/: ...
secrets/: ...
```

List top 3 pains (e.g. “shells inconsistent”, “host boilerplate”, “no Mac story”).

Lab 2 — Pick one tool and trial

If devenv

1. `devenv init` in `trial/app` or import docs’ minimal flake.
2. Add one language toolchain you actually use.
3. Optional: one service (postgres/redis) if docs make it easy.
4. Enter shell; run a real command (tests/build).
5. Note direnv integration if any.
6. Time spent + friction log.

If Snowfall

1. Scaffold a **tiny** Snowfall flake (one system stub, one module).
2. Do not migrate all hosts.
3. Trace: where does a host definition live? How is it discovered?
4. Compare to your `hosts/lab/default.nix` pattern.
5. Eval something small; capture errors as learning.

If nix-darwin

1. On Mac: flake with one `darwinConfigurations`; apply to a **throwaway** aspect if possible.
2. Or read-only deep dive: clone an example, map options vs NixOS cousins.
3. Document what cannot be shared as-is with NixOS modules.

Timebox: 2–4 hours max for trial hands-on.

Lab 3 — Comparison table (required)

Complete `COMPARE.md` per Theory 5–worked example. Include explicit paragraph:

“We will / will not adopt before Capstone freeze because...”

Lab 4 — Ecosystem postcard

In 15 lines, define in your words: flake-parts, nixos-anywhere, nh, Attic. No install required.

Lab 5 — Capstone protection check

```
# Capstone main should be untouched by trial frameworks
cd /path/to/capstone-flake
git status
git log --oneline -5
```

If you “accidentally” started migrating main, revert or move work to a branch named `experiment/...` and restore main to last known good.

Acceptance: `git status` clean on Capstone main regarding trial frameworks; trial lives under `trial/` or `experiment/*` branch.

Lab 6 — Teach-back (5 bullets)

Explain to a peer:

1. What pain you tried to solve
 2. What you tried
 3. Score highlight
 4. Decision
 5. Revisit criteria
-

Common gotchas

Mistake	What to do
Trying all three “a little”	Violates the chapter; pick one
Rewriting Capstone main branch	Use <code>trial/</code> only
Adopting for aesthetics	Score against pain list
<code>devenv</code> and heavy custom shells forever duplicated	Choose a house standard
Assuming Snowfall NixOS	It’s flake structure, not an OS
<code>nix-darwin</code> as lab substitute	Still need Linux NixOS for Capstone
Infinite template shopping	Timebox trial to hours, not days

Checkpoint

- ☐ LAYOUT.md + top pains
- ☐ One tool trial notes or read-only darwin depth
- ☐ COMPARE.md with keep/drop decision
- ☐ Ecosystem postcard vocabulary
- ☐ Capstone main layout untouched (or only trivial fixes)
- ☐ Three personal gotchas

Commit

```
git add .  
git commit -m "lab(nixos): ecosystem trial and keep/drop decision"
```

Write three personal gotchas before continuing.

Next

→ [Comparative landscape](#) — steelman alternatives to NixOS.

Comparative landscape

Comparative landscape

Goal: Write an honest, concrete trade-off note comparing NixOS to Guix System, image-based/immutable distros, and Ansible-style config management—grounded in **your** workloads, not tribal slogans.

Why this chapter exists

If you cannot say when NixOS is the wrong tool, you do not understand it. Capstone commitment is real effort; this chapter is the “steelman the alternatives” work so your retrospective is adult, not fanfic.

Theory 1 — Comparison dimensions

Score systems on dimensions that matter to operators:

Dimension	Question
Reproducibility	Can two machines converge from the same artifact/definition?
Atomic rollback	Bad update → previous generation quickly?
Abstraction	Packages + services + users in one language?
Learning curve	Time to first safe production change
Ecosystem breadth	Packaging coverage, cloud images, vendor support
Secrecy & compliance	Secrets, audit, SBOM stories
Day-2 ops	Observability, patches, multi-node
Escape hatches	Can you drop to imperative when burning?
Team skill fit	What does your org already know?
Mac/desktop story	Not everything is a Linux server
Migration cost	From <i>your</i> current estate

No system wins all columns. Weighted scores beat vibes.

Theory 2 — NixOS (your baseline)

Strengths you have felt (cite your own chapters):

- Declarative modules + generations
- Flake pins for inputs
- Same language from package to OS
- Binary caches + remote builders
- Strong “rebuild from repo” story
- sops-nix / deploy-rs / Colmena / nixosTest ecosystem

Weaknesses you should admit:

- Steep language + module learning curve
- Eval/build complexity; footguns (IFD, priorities, infinite recursion)
- Docs fragmented across eras (channels vs flakes)
- Some proprietary/vendor agents painful to package
- GUI desktop polish varies by investment
- Experimental features (CA derivations, etc.) not free wins
- Team hiring: fewer “NixOS natives” than Ansible generalists

Write two scars from **your** lab, not from memes.

Theory 3 — Guix System

What it is: GNU Guix—functional package manager + Guix System; Scheme (Guile) instead of Nix language; strong free-software ethos; similar store/generation ideas.

Topic	Guix-ish shape	NixOS-ish shape
Language	Scheme	Nix
Init	shepherd	systemd
Channels / pins	channels	flakes/channels

Topic	Guix-ish shape	NixOS-ish shape
FHS / binary blob culture	Often stricter free software	Pragmatic unfree options common
Community size	Smaller	Larger
Commercial surface	Smaller	Larger blog/job footprint

When Guix may win: Scheme fluency, GNU purity goals, shepherd preference, research alignment.

When NixOS may win for you: systemd ecosystem, larger package set, more operational write-ups, your already-built Capstone investment.

You do **not** need to install Guix today—read primary docs enough to avoid strawmen. If you already know Scheme, say so in the essay.

Theory 4 — Image-based / immutable distros

Examples of the *family* (not a full product review): Fedora CoreOS / Silverblue-style, Bottlerocket, Talos, some appliance OS designs—**image A/B**, ostree or similar, workloads often in containers.

Topic	Image-based	NixOS
Unit of deploy	OS image	Config evaluation → system closure
App delivery	Containers/k8s common	Packages + modules + optional containers
Rollback	Previous image slot	Generations
Host customization	Minimal by design	Extreme (sometimes too much)
“Pets”	Discouraged	Possible (homelab pets common)
Provisioning	Ignition/cloud-init class tools	Disko, nixos-anywhere, images

When images win: Large fleets of identical minimal nodes, Kubernetes workers, locked-down appliances, org already standardized on container platforms.

When NixOS wins: Highly specialized hosts, cross-cutting host services, desire for one repo to express weird topology without baking images every change (though NixOS can also build images).

Hybrid reality: NixOS **build** of images + k8s workers is a valid architecture—not purity cosplay.

Theory 5 — Ansible (and friends)

What it is: Imperative/convergent config management over SSH; YAML playbooks; enormous module library; agentless common. Cousins: Salt, Puppet, Chef—this comparison centers Ansible as the industry default many teams already run.

Topic	Ansible	NixOS
Model	Converge state with tasks	Specify desired system; build closure
Partial apply	Easy, sometimes too easy	Whole-system generations
Drift	Easy without discipline	Harder if you only use nix
Secrets	Vault, etc.	sops/agenix + store discipline
Non-NixOS estate	Excellent	Weak (HM/darwin partial)
Rollback	You build it	Built-in generations
Network gear / mixed OS	Strong ecosystem	Not the point

Hybrid reality: Many orgs use Terraform/cloud for create, Ansible for classic distros, NixOS for greenfield highly automated slices.

When Ansible wins: Heterogeneous fleets (Ubuntu + network gear + weird appliances), existing playbook investment, teams without Nix skills, “change this one file on 50 boxes” urgency.

When NixOS wins: Greenfield Linux hosts you fully own, strong need for rebuildable closures, rollback as a product feature, package+service unity.

Theory 6 — Honesty checklist (avoid tribal takes)

Bad argument	Better
“Ansible is YAML spaghetti, Nix is pure.”	“Our fleet is 200 Ubuntu boxes with existing roles; migration cost exceeds benefits this year.”
“Guix is just Nix with Scheme.”	“Shared functional ideas; different language, init, and community trade-offs.”
“Immutable OS means no SSH ever.”	“SSH policy is orthogonal; image distros still need day-2 access models.”
“NixOS can’t do containers.”	“NixOS can; sometimes image-first platforms optimize for that path harder.”

Bad argument	Better
“Nix solves secrets.”	“sops-nix helps; key custody and policy still human.”

Theory 7 — Decision time-bounds

Prefer time-bounded decisions:

```
Through next nixpkgs release: NixOS Capstone lab remains source of truth for learning hosts.
Ubuntu estate at $work: Ansible remains unless a greenfield slice appears.
Re-evaluate Guix: only if Scheme project requires it.
```

Eternal loyalty is not an engineering output.

Worked example — weighted scorecard fragment

Context: solo homelab + one day-job Ubuntu fleet I don’t control.

```
| Dimension | Weight | NixOS | Guix | Image-family | Ansible |
|-----|-----|-----|-----|-----|-----|
| Reproducibility | 5 | 5 | 5 | 4 | 2 |
| Rollback | 5 | 5 | 5 | 4 | 2 |
| Learning curve | 3 | 2 | 2 | 3 | 4 |
| Team skill (job) | 4 | 1 | 1 | 3 | 5 |
| Specialized hosts | 5 | 5 | 4 | 2 | 3 |
```

Compute crude weighted sums; note uncertainty bands (“±1 on Guix—I read, didn’t run”).

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/compare
cd ~/lab/nixos/internals/compare
```


Lab 1 — Primary sources (skimming with intent)

Collect links in `SOURCES.md` (official or project docs, not only Reddit):

1. Guix System manual intro / features
2. One image-based OS docs page (e.g. CoreOS update model or Talos architecture)
3. Ansible “how it works” / idempotence intro

Spend ~45–60 min reading, not installing everything. Optional: one throwaway VM if time remains—**not** required.

Lab 2 — Scorecard for *your* context

`SCORECARD.md` — rate NixOS / Guix / Image-family / Ansible 1–5 on Theory 1 dimensions. Add “Weight for me.” Compute weighted sum. Note uncertainty.

Define **your** context in 5 bullets (homelab, job, course-only, multi-OS, etc.).

Lab 3 — Essay (required)

`ESSAY.md` — 400–800 words:

1. Where NixOS clearly wins for you (with concrete examples from this book).
2. Where it loses or is overkill.
3. One scenario you would pick Ansible or image-based instead.
4. Whether Guix is interesting to you personally (even if “not now”).
5. What you will *not* rewrite after this essay (protect Capstone).

Tone: steelman alternatives; no meme warfare.

Lab 4 — Elevator version

Compress the essay into a 90-second talk outline (bullet list) for a skeptical SRE. Save as `ELEVATOR.md`.

Include one sentence: “Here’s when I would **not** use NixOS...”

Lab 5 — Capstone implication

One short paragraph in `IMPLICATIONS.md`: does this change Capstone A scope? (Default: **no**—essay is clarity, not a rewrite trigger.)

Lab 6 — Scenario cards (forced choice)

Write three short cards (5–8 sentences each) in `SCENARIOS.md`. For each, pick a primary tool and justify with dimensions from Theory 1:

Card A — Homelab “weird topology”

One edge proxy, one data host, sops secrets, occasional GPU box. You already have Capstone muscle.

Card B — Employer fleet

80 Ubuntu VMs, existing Ansible roles, two network engineers who refuse new DSLs, compliance questionnaire next quarter.

Card C — Kubernetes worker pool

Stateless nodes, image A/B updates preferred by platform team, apps only in containers, host should be minimal.

Do not use these cards to reopen Capstone A scope. They exist to prove you can **switch criteria with context**.

Lab 7 — Quote hygiene

From your primary sources, copy **one** short quotation or paraphrased claim (with URL) that surprised you for each of: Guix, image-based, Ansible. In `SOURCES.md`, note whether you agree after Capstone experience.

This blocks empty essays that restate social media without contact with docs.

Worked example — elevator closer

```
NixOS wins for me when I fully own the Linux host and need rebuildable
closures plus rollback. I would not force it onto our Ubuntu estate this
year-the migration cost dwarfs the benefit. Image-based OS wins for
identical k8s workers. Guix is intellectually close but not my ops path.
Questions?
```

Adapt to your scorecard; keep under 90 seconds spoken.

Common gotchas

Mistake	Fix
Comparing marketing to your scars	Use Capstone experience as data
Installing four OSes today	Read + score; optional one VM if time
Declaring eternal loyalty	Prefer time-bounded decisions
Ignoring team skills	Skills are a real dimension
Strawmanning Guix without reading	Primary sources in <code>SOURCES.md</code>
Using this chapter to restart layout thrash	Protect Capstone; ecosystem chapter already covered tools

Checkpoint

- ☐ SOURCES.md with primary links
- ☐ Weighted SCORECARD.md
- ☐ ESSAY.md 400–800 words, honest trade-offs
- ☐ ELEVATOR.md including when *not* NixOS
- ☐ Capstone protection note
- ☐ Three scenario cards
- ☐ One primary-source surprise note per alternative
- ☐ Three personal gotchas

Commit

```
git add .  
git commit -m "lab(nixos): comparative landscape essay and scorecard"
```

Write three personal gotchas before continuing.

Next

→ [Reading release notes](#) — IMPACT method on official notes.

Reading release notes (26.05)

Reading release notes (26.05)

Goal: Read the **official** NixOS 26.05 “Yarara” release notes (and related nixpkgs/Nix notes as needed), then produce an annotated impact checklist for **your** flake—not a blog summary.

! Important

Baseline for this book: NixOS / nixpkgs **26.05**. Primary source beats social media threads. If a note conflicts with a blog, trust the release notes + your eval.

Why this chapter exists

Upgrades fail when people skip release notes and only chase build errors one by one. The upgrade planning chapter plans 26.05→next; this chapter teaches the **method** on the release you should already be targeting, and capture breakages that still lurk in your modules.

Reading notes is an operator skill equal to writing modules.

Theory 1 — What “a NixOS release” actually is

Artifact	Role
nixpkgs branch nixos-26.05	Package + module tree for the release
Release notes	Human-facing breaking changes, renames, deprecations
State version system.stateVersion	Migration behavior for stateful options
ISO / channel artifacts	Install media (flakes pin git/github instead)
Nix version shipped/tested	Sometimes separate from nixpkgs cadence
flake.lock	Your actual pinned revisions—not the marketing branch name alone

Flakes pin **revisions**. Your `flake.lock` may already be on 26.05 while your brain still follows older blog habits.

```
nix flake metadata
# note nixpkgs locked rev / ref / lastModified

nix eval --raw .#nixosConfigurations.lab.pkgs.lib.version 2>/dev/null || true
# or inspect lock:
# jq '.nodes.nixpkgs' flake.lock
```

Theory 2 — How to read release notes like an operator

Do not read linearly once and forget. Use passes:

Pass A — Inventory (30–45 min)

Skim sections:

- Breaking changes
- Notable renames / option moves
- Service-specific notes for software **you** run
- Deprecated options
- Kernel / boot / systemd highlights
- Security-relevant defaults
- NixOS module system / installer notes

Pass B — Personal filter (45–60 min)

For each note, tag:

Tag	Meaning
N/A	You don't use it
WATCH	Indirect risk
ACTION	Must change config
DONE	Already handled
VERIFY	Needs rebuild/test to confirm

Pass C — Prove with eval/build

```
nixos-rebuild build --flake .#<host>
# or
nix build .#nixosConfigurations.<host>.config.system.build.toplevel -L
```

Search your repo for renamed options:

```
rg -n "oldOption|services\.foo" -g '*.nix'
```

Pass D — Warnings are future breakages

```
evaluation warning: ...
trace: obsolete option ...
```

Paste warnings into `IMPACT.md` as `VERIFY/ACTION`. Next release often hard-errors them.

Theory 3 — `stateVersion` is not a fashion number

```
system.stateVersion = "26.05"; # example - only change with understanding
```

`stateVersion` tells modules how to migrate **on-disk state defaults**. Blindly bumping it to “match release” can break stateful services (Postgres data dirs, user settings, etc.). Release notes and module comments matter more than aesthetics.

Rule for this book: record your current `stateVersion`; do **not** bump it casually while reading notes. If notes require a migration path, put it on the upgrade planning runbook.

Action	When
Leave unchanged	Default
Bump with migration plan	Notes + module docs say so; backups ready
Mismatch with nixpkgs branch	Allowed and common; understand why

Theory 4 — Cross-check Nix itself

Your syllabus pins modern Nix (book baseline mentions **2.34.x** class—**verify yours**). Release notes for **Nix** (the evaluator/daemon) are separate from NixOS notes.


```
nix --version
nix config show | rg -i 'experimental|system|substituter' || true
```

If CI uses a different Nix than your laptop, note the skew in the checklist—flake eval can differ on experimental defaults and sandbox settings.

Surface	Record
Laptop Nix	version
CI Nix	installer action / version
Lab host Nix	from <code>nix --version</code> on host

Theory 5 — Module renames and aliases (kinds of change)

Common historical patterns (examples of *kinds*, not a complete 26.05 dump—you must read the real notes):

Kind	Example pattern	Response
Option rename	<code>services.foo</code> → <code>services.foo-ng</code>	Update; don't live on aliases forever
Nested <code>settings</code>	freeform attr moved under <code>settings</code>	Follow module type
Package rename	overlay breaks	Fix overlay attr
Default flip	<code>enable</code> default changed	Explicit <code>mkForce</code> /set
Kernel package set	boot fails on old nvidia/out-of-tree	Pin or update module
Removed service	module gone	Replace or package yourself

When notes say an option is renamed, prefer the new path. Aliases expire.

Theory 6 — Inputs beyond nixpkgs

Your flake may pin:

Input	Why check notes/changelogs
home-manager	<code>release-26.05</code> pairing
sops-nix	module option moves
deploy-rs / colmena	activation quirks

Input	Why check notes/changelogs
disko	device option changes
lanzaboote	SB flow changes

Release notes for NixOS are necessary but not sufficient. Skim changelogs for **your** inputs when IMPACT is large.

```
nix flake metadata
# for each input: note ref + whether it follows nixpkgs
```

Theory 7 — Severity and scheduling

Tag	SLA for lab Capstone
ACTION (eval broken)	Fix before next switch
ACTION (runtime risk)	Fix before DR proof
VERIFY	Schedule rebuild this week
WATCH	Note in upgrade seeds
N/A	Drop from working set

Do not turn notes reading into a two-week refactor of modules you do not run.

Worked example — IMPACT.md row

```
| Note / change | Tag | Repo paths | Action | Owner/date |
|-----|-----|-----|-----|-----|
| services.foo renamed to services.foo-ng | ACTION | modules/foo.nix:12 | rename option; reb |
| postgres default major | WATCH | hosts/lab/db.nix | verify stateVersion; backup | upgrade |
| plasma something | N/A | - | no desktop | - |
| evaluation warning: obsolete bar | VERIFY | modules/common.nix | rebuild log line 480 | to
```

Aim for **every ACTION and VERIFY** item to reference a file path or explicit N/A reason.

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/release-notes/notes-raw
cd ~/lab/nixos/internals/release-notes
```

Lab 1 — Capture current pin

```
cd /path/to/your/flake
nix flake metadata | tee ~/lab/nixos/internals/release-notes/metadata.txt
nix --version | tee ~/lab/nixos/internals/release-notes/nix-version.txt
rg -n "stateVersion" -g '*.nix' | tee ~/lab/nixos/internals/release-notes/stateVersion-hits.
```

Optional lock peek:

```
jq -r '.nodes.nixpkgs.original,.nodes.nixpkgs.locked' flake.lock \
| tee ~/lab/nixos/internals/release-notes/nixpkgs-lock.txt
```

Lab 2 — Official notes intake

1. Open the official NixOS 26.05 release notes (nixos.org manual / nixpkgs nixos/doc/manual release notes for 26.05).
2. Save the URL in `SOURCES.md`.
3. Export or copy key sections into `notes-raw/` if you want offline annotation (optional).
4. Create `IMPACT.md` with the table from the worked example.

Also add:

- URL for any Nix release notes you consulted
 - HM release notes if you use HM
-

Lab 3 — Repo audit scripts

```
cd /path/to/your/flake
rg -n "mkEnableOption|services\.|programs\." -g '*.nix' | head -n 100 \
  | tee ~/lab/nixos/internals/release-notes/options-sample.txt
# search for deprecated names you found in notes:
# rg -n "deprecatedOptionName" -g '*.nix'
```

For each ACTION, either:

- fix now on a branch, or
- file as upgrade-planning backlog with severity (blocker / nice)

Avoid giant drive-by refactors while annotating—**annotate first**; fix blockers only.

Lab 4 — Build verification

```
nix build .#nixosConfigurations.<host>.config.system.build.toplevel -L \
  2>&1 | tee ~/lab/nixos/internals/release-notes/build.log
```

If warnings appear, paste them into `IMPACT.md` as `VERIFY/ACTION`.

Run relevant checks:

```
nix build .#checks.<system>.<name> -L \
  2>&1 | tee ~/lab/nixos/internals/release-notes/check.log
```

Acceptance: Build log captured; every evaluation warning triaged.

Lab 5 — Personal “gotchas from 26.05”

Write **5 bullets** in `IMPACT.md` under “What I almost missed.” These feed troubleshooting and the retrospective.

Also write `stateVersion` policy in one sentence:

```
## stateVersion policy
We run stateVersion = "..." on host lab. We will not bump it without: backup, notes citation,
```

Lab 6 — Feed upgrade planning

Create `UPGRADE-SEEDS.md` with:

Item	Why it matters for next upgrade
ACTION leftovers	Must clear or plan
WATCH services	Higher test priority
CI Nix skew	Align before upgrade PR

Do not perform the upgrade in this chapter unless you deliberately expand scope.

Common gotchas

Mistake	Fix
Only reading unofficial summaries	Start from official notes
Bumping <code>stateVersion</code> for fun	Read migration semantics first
Assuming lock is 26.05 without checking	<code>nix flake metadata</code>
Ignoring warnings that aren't errors	Warnings become breakages next release
Giant drive-by refactors while annotating	Annotate first; fix blockers only
Forgetting HM/sops inputs	Check paired release branches
Treating blog tips as canonical	Primary notes + your build log

Checkpoint

- ☐ `metadata.txt` + `nix-version.txt`
- ☐ Official 26.05 URL in `SOURCES.md`
- ☐ `IMPACT.md` tagged N/A/WATCH/ACTION/DONE/VERIFY
- ☐ Build (and checks) log captured
- ☐ `stateVersion` policy recorded
- ☐ Five “almost missed” bullets

- ☐ Upgrade seeds noted
- ☐ Three personal gotchas

Commit

```
git add .  
git commit -m "lab(nixos): annotate 26.05 release notes against flake"
```

Write three personal gotchas before continuing.

Next

→ [Upgrade planning](#) — dry-run runbook to the next release.

Upgrade planning

Upgrade planning

Goal: Produce a complete **dry-run** upgrade runbook from 26.05 → 26.11 (next cadence release): input bumps, test gates, staged rollout, rollback, and risk register—**without** requiring a production upgrade today.

i Note

NixOS releases target roughly **May/November** cadence (.05 / .11). 26.11 may be upcoming, in beta, or stable depending on when you read this. The deliverable is the **plan**; execute only on lab when ready.

Why this chapter exists

Release-note literacy without a runbook still produces Friday-night lock bumps. Upgrades are projects: branch, gates, lab first, rollback written cold.

Theory 1 — What actually changes in a flake upgrade

```
flake.nix inputs (nixpkgs url/ref)
  ↓
flake.lock (revs, narHashes)
  ↓
eval modules against new nixpkgs
  ↓
realize new toplevel
  ↓
switch / reboot if kernel
  ↓
stateful service migrations (stateVersion-aware)
```

Step	Failure mode
Lock bump	Unrelated input moves (HM, sops, deploy-rs) break together
Eval	Option removed/renamed; type errors
Build	Package broken; FODs; your overlay

Step	Failure mode
Switch	Service fails activation
Runtime	Data migration, socket changes, API versions
Reboot	Bootloader/kernel/initrd issues

Theory 2 — Staging strategy

Environments

Stage	Purpose
Dev laptop / lab VM	First eval/build/test
CI on branch	Checks + cache populate
Non-prod server	Real networking/secrets
Prod	Last; maybe canary host

Git strategy

```

main (26.05 pin)
  branch upgrade/26.11
    bump nixpkgs only
    fix eval
    bump companion inputs (HM, etc.)
    PR with checklist

```

Avoid one commit that bumps six inputs and refactors modules.

Companion inputs

Home Manager, sops-nix, deploy-rs, disko, agenix—each has branches/tags tracking nixpkgs. Read their upgrade notes. Mismatched HM vs nixpkgs is a classic footgun.

Theory 3 — Tests-first gate

Minimum gates before **switch** on anything important:

```

nix flake check
nix build .#nixosConfigurations.<host>.config.system.build.toplevel
nix build .#checks.<system>.<integration>
# deploy to lab only:
# deploy .#lab -- or colmena apply --on lab

```

Optional: `nixos-rebuild build-vm` / smoke VM.

Policy: red checks no promote.

Theory 4 — Rollback is part of the plan

Layer	Mechanism
Boot	Previous generation in bootloader
Runtime	<code>nixos-rebuild switch --rollback</code> / rollback generation
Git	Revert lock on branch
Secrets	Keys unchanged; ciphertext may need re-encrypt only if tooling changes
Data	DB dumps / zfs snapshots before upgrade

Document **RTO** goals for lab vs prod. Capstone disaster recovery practices wipe/rebuild and rollback drills.

Concrete rollback commands (fill for your path)

```

# local rebuild host
sudo nixos-rebuild switch --rollback
# list generations
sudo nix-env -p /nix/var/nix/profiles/system --list-generations
# deploy-rs / colmena: redeploy previous known-good commit
git revert <lock-bump-commit>
# then redeploy

```

Theory 5 — stateVersion and stateful services

Upgrading nixpkgs automatically bumping stateVersion.

Plan section required:

- Current stateVersion
 - Whether next release notes require migration
 - Snapshot/backup steps for PostgreSQL, Vaultwarden, etc. you actually run
 - Who verifies app-level health post-switch
-

Theory 6 — Communication & windows

Even for homelab:

- Maintenance window
- What breaks for family/users
- Where logs live
- Abort criteria (e.g. “if proxy down >15m, rollback”)

Abort criteria template

Signal	Action
Eval/build red on CI	Do not deploy
systemctl --failed non-empty after switch	Investigate 15m then rollback
Proxy healthcheck fail	Rollback
No console / no SSH	Prefer prevention; fix access before upgrade

Constraint	Plan adjustment
------------	-----------------

Theory 7 — Elitebook / small-lab constraints

Constraint	Plan adjustment
16 GB RAM	Lower <code>max-jobs</code> during upgrade builds
Limited disk	GC old generations after successful upgrade, not mid-build; ensure free space for two closures
Single physical host	Prefer guest upgrade first; snapshot VM
Nested virt	Confirm KVM still works after kernel bump

Runbook template (you will fill this)

Create `UPGRADE-26.11.md` with these sections:

0. Metadata

- Author, date, target hosts
- Current pins (`flake.lock` nixpkgs rev)
- Target ref (`github:NixOS/nixpkgs/nixos-26.11` or unstable interim)

1. Preconditions

- ☐ Backups verified restorable
- ☐ CI green on main
- ☐ Disk space for two closures
- ☐ Access to console/IPMI if remote
- ☐ sops/age keys available offline

2. Release notes

- Links to 26.11 notes (when published) + diff against your 26.05 `IMPACT.md` method
- ACTION items list

3. Technical steps (lab first)

```
git checkout -b upgrade/26.11
# edit flake.nix input for nixpkgs
nix flake lock --update-input nixpkgs
# then carefully update HM / sops / etc.
nix flake check
nix build .#nixosConfigurations.lab.config.system.build.toplevel -L
# deploy lab
# health checks
```

4. Health verification

- HTTP endpoints
- `systemctl --failed`
- Specific app probes
- Logs

5. Rollback

Exact commands for your deploy path (deploy-rs/colmena/rebuild).

6. Risks

Risk	Likelihood	Impact	Mitigation
...			

7. Decision

Go / no-go criteria for prod.

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/upgrade
cd ~/lab/nixos/internals/upgrade
```

Lab 1 — Inventory pins

```
cd /path/to/flake
nix flake metadata --json > ~/lab/nixos/internals/upgrade/metadata.json
cp flake.lock ~/lab/nixos/internals/upgrade/flake.lock.baseline
```

List every input and its tracking branch/tag in `INPUTS.md`.

Lab 2 — Write the full runbook

Complete `UPGRADE-26.11.md` from the template. Mark unknown 26.11 specifics as TBD **when notes ship** but keep the **structure** solid. Reuse the release notes method for notes intake.

Acceptance: All seven template sections present; rollback commands are copy-pasteable for *your* path.

Lab 3 — Dry-run mechanics on lab (optional execution)

If 26.11 exists and you have time:

1. Branch
2. Update nixpkgs **only**
3. Eval and fix until `toplevel` builds
4. **Do not** require prod switch

If 26.11 is not out: dry-run against `nixos-unstable` on a disposable branch/VM **only**, or stop at lock-file rehearsal:

```
# rehearsal of commands without pushing
nix flake lock --update-input nixpkgs --output-lock-file /tmp/test.lock
```

Document what you did / skipped.

Lab 4 — Test plan matrix

TEST-PLAN.md:

Check	Command	Blocks promote?
flake check	<code>nix flake check</code>	Yes
host toplevel	<code>nix build ...</code>	Yes
nixosTest X	...	Yes
manual proxy curl	...	Yes for edge host
backup restore drill	...	Prod only

Lab 5 — Abort & rollback tabletop

Without breaking prod, write the exact rollback commands for:

1. Failed switch, system still booting old generation
2. Booted new generation, services bad
3. Git revert of lock after partial deploy

Acceptance: Three scenarios with commands; no “I’ll figure it out live.”

Lab 6 — Disk space math

```
df -h /nix
nix path-info -S /run/current-system 2>/dev/null || true
```

Record free space and estimated second closure headroom in the runbook preconditions.

Common gotchas

Mistake	Fix
Big-bang all inputs	Stage nixpkgs first
No backup verification	Test restore once
Forgetting companion inputs	HM/sops/disko matrix
Upgrading prod first	Lab → CI → prod
No abort criteria	Write them cold
Bumping stateVersion with release	Separate decision
GC mid-upgrade	Free space first; GC after success

Checkpoint

- ☐ INPUTS.md + lock baseline
- ☐ Complete UPGRADE-26.11.md runbook
- ☐ TEST-PLAN.md with blockers
- ☐ Rollback tabletop written
- ☐ Explicit statement: prod upgraded or **not** (expected: not required)
- ☐ Three personal gotchas

Commit

```
git add .  
git commit -m "lab(nixos): 26.05 to 26.11 upgrade runbook dry-run"
```

Write three personal gotchas before continuing.

Next

→ [Troubleshooting drills](#) — inject, recover, playbook.

Troubleshooting drills

Troubleshooting drills

Goal: Inject 2–3 realistic failures on a **disposable lab VM**, recover them, and write durable playbook entries covering boot, build/eval, service, and network classes.

Warning

Do **not** run injection drills on a daily driver or the only copy of important data. Snapshot the VM first.

Why this chapter exists

Theory without broken systems is cosplay. Capstone DR and rollback assume you already have a personal playbook. This chapter builds it under time pressure.

Theory 1 — Failure class matrix

Class	Symptoms	First tools
Eval	<code>nixos-rebuild</code> dies before build; option errors; infinite recursion	<code>nix eval</code> , bisect modules, evaluator
Build	Compile/hash/sandbox failures	<code>nix log</code> , store and daemon
Boot	Unbootable, emergency shell, wrong generation	Console, bootloader generations, <code>stage-1</code> logs
Activation	Switch fails mid-script	<code>nixos-rebuild</code> output, <code>systemd</code> activation logs
Service	Unit failed, restart loop	<code>systemctl</code> , <code>journalctl</code>
Network	No SSH, DNS, firewall drops	<code>ip</code> , <code>nft</code> / <code>iptables</code> , <code>ss</code> , console
Secrets	Service can't read age/sops path	Permissions, sops keys, activation ordering
Disk/state	Full disk, wrong mount, impermanence surprise	<code>df</code> , <code>mount</code> , disk layout

For each incident, write:

Symptom → Class → Hypothesis → Command evidence → Fix → Prevention

Theory 2 — Golden command palette

```
# system health
systemctl --failed
journalctl -p err -b --no-pager | tail -100
journalctl -u <unit> -e --no-pager

# nix
nix log /nix/store/...drv
nix-store --verify --check-contents # heavy; rare
nixos-rebuild dry-activate --flake .#host

# network
ip a; ip r
ss -lntp
ping -c2 1.1.1.1
resolvectl status || cat /etc/resolv.conf
nft list ruleset | head # if nft

# boot / generations
ls -la /nix/var/nix/profiles/system*
# previous generation boot from bootloader menu
```

Remote when SSH dies

- Hypervisor console
- deploy-rs / colmena won't save you if network is wrong—console will

Elitebook lab specifics

Resource	Note
Console	virt-manager console / virsh console / Incus console
Nested virt	Needed for nixosTest, not for basic SSH drills
Free space	Disk-full injections need headroom to recover

Theory 3 — Injection catalog (choose carefully)

Pick **2–3** from different classes:

#	Injection	Recovery skill
1	Break a module option type / typo in flake host	Eval literacy
2	Firewall deny SSH (on VM with console!)	Network + console
3	Stop/mask a critical unit or bad ExecStart	Service debug
4	Wrong proxy upstream	Service + HTTP
5	Mis-permission a secret path	Secrets activation
6	Fill disk with a large dummy file	Disk hygiene
7	Bad fileSystems entry on snapshot VM only	Boot recovery

Never inject disk/boot failures without a hypervisor snapshot.

Theory 4 — Time discipline

For each incident, timebox:

Phase	Budget
Detect	5 min
Classify	5 min
Fix	20–40 min
Write playbook entry	10 min

If stuck past budget, snapshot rollback and write “what I needed to learn” instead of thrashing.

Theory 5 — Playbook entry format

playbook/PL-XXX.md:

```
# PL-003 - nginx 502 after deploy

## Symptoms
- curl localhost returns 502
- nginx active, upstream down

## Class
```

```
Service / proxy

## Diagnose
systemctl status myapp
journalctl -u myapp -n 100
curl -v upstream

## Fix
...

## Prevention
- nixosTest curl through proxy
- healthcheck in CI

## Related
TLS front door; nixosTest mastery
```

Theory 6 — Rollback as a diagnostic tool

```
sudo nixos-rebuild switch --rollback
# or boot previous generation from loader
```

Rollback answers: “Is this a config regression?” If symptoms vanish, bisect generations / git. If symptoms remain, suspect state/disk/network outside the generation.

Theory 7 — Layered isolation order

When unsure, isolate in this order:

1. Physical/virt console access?
2. Network path (ping/DNS/SSH)?
3. Boot/generation?
4. Eval of flake?
5. Activation/systemd?
6. App/data layer?

Jumping to “rewrite the flake” during a network outage wastes the drill clock.

Worked example — playbook stub

```
## Symptom
SSH hangs after rebuild

## Class
network / firewall

## Diagnose
ss -tlnp; nft list ruleset; journalctl -u sshd

## Fix
re-add allowedTCPPorts = [ 22 ]; console rebuild

## Prevention
nixos-rebuild test first; console attached
```

Lab 0 — Snapshot

```
# hypervisor UI or:
# virsh snapshot-create-as lab troubleshoot-pre --disk-only --atomic
# incus snapshot create nixlab troubleshoot-pre
```

Confirm console access works **before** breaking SSH.

```
mkdir -p ~/lab/nixos/internals/troubleshoot/playbook
cd ~/lab/nixos/internals/troubleshoot
```

Acceptance gate before inject: snapshot exists; console proven; you know wipe boundaries.

Lab 1 — Baseline health

On healthy lab:

```
systemctl --failed | tee baseline-failed.txt
curl -fsS http://127.0.0.1/ || true
nixos-rebuild dry-activate --flake .#<host> | tee baseline-dry.txt
```

Lab 2 — Inject & recover #1 (eval or build)

Example eval injection:

```
# deliberate typo in host config
networking.hostName = 123; # type error

time nixos-rebuild build --flake .#<host> 2>&1 | tee incident1.log
# fix
# re-build green
```

Write playbook/PL-001.md with timing notes.

Lab 3 — Inject & recover #2 (service or secrets)

Example:

- Point systemd ExecStart at wrong binary via your module, or
- Break a runtime secret path after backup of working config

Recover using journal + unit status. Playbook PL-002.md.

Lab 4 — Inject & recover #3 (network) — console required

Example:

```
networking.firewall.allowedTCPPorts = [ ]; # removed 22
# or extra iptables drop - prefer declarative so rebuild can fix
```

From **console**, restore SSH, verify from client. PL-003.md.

If you cannot safely do network injection, substitute a proxy misconfig incident and document why.

Acceptance: At least two distinct classes recovered; network class only with console.

Lab 5 — Compile master index

playbook/README.md table of all PL entries + class tags. This file is Capstone ops gold.

Lab 6 — Timeboxed chaos #4 (state)

Optional fourth injection: delete a non-critical state file (not `/etc/ssh` host keys unless snapshot). Recover from backup notes. Emphasize **state generation**.

Lab 7 — Master index cross-links

Ensure docs/playbooks/index.md links to rebuild/generations rollback, firewall, sops, build debug. Troubleshooting is a graph, not a single page.

Lab 8 — End-state health

```
systemctl --failed
curl -fsS http://127.0.0.1/ || true
# leave host better than you found it
```

Acceptance: Host healthy; snapshot can be deleted only after health confirmed.

Common gotchas

Mistake	Fix
Breaking SSH without console	Snapshot + console first
Only reading theory	Actually inject
Fixing without writing playbook	Entry is the deliverable
Multiple simultaneous injections	One at a time
Leaving host broken overnight	Revert snapshot if needed
Forgetting prevention section	Tests/asserts become prevention

Checkpoint

- ☐ VM snapshot taken
- ☐ 2 incidents injected and fixed
- ☐ 2 playbook entries with diagnose/fix/prevent
- ☐ Master playbook index
- ☐ Host healthy at end of drills
- ☐ Three personal gotchas

Commit

```
git add .  
git commit -m "lab(nixos): troubleshooting drill and ops playbook"
```

Write three personal gotchas before continuing.

Next

→ [Architecture review](#) — freeze map and gaps before Capstone thrash.

Architecture review

Architecture review

Goal: Map earlier parts of this book onto your actual repository, produce `ARCHITECTURE.md`, and a **prioritized gap list** that drives Capstone—before writing more features ad hoc. Assume Capstone hosts pin **NixOS 26.05**.

Tip

Capstone is implementation and proof, not discovery. Teams that skip architecture review thrash: secrets half-done, tests theater, deploy only from a laptop. This chapter freezes an honest map of what exists vs Capstone A requirements.

Why this chapter exists

You have skills across concepts through ops. Capstone fails when those skills never land in **one** coherent flake. Architecture review is the integration gate before code thrash: map, gaps, sequence—then implement.

Theory 1 — Capstone A bar (recall)

Required:

1. 1 NixOS host role understood end-to-end
2. Home Manager **or** minimal service-user story
3. Secrets: sops-nix **or** agenix
4. Reverse-proxy / edge pattern
5. Deploy path: deploy-rs **and/or** Colmena
6. 1 nixosTest or flake check catching real regressions
7. CI builds and preferably pushes a cache

Disaster recovery and rollback chapters add wipe/rebuild + rollback proof. Gaps from this chapter should make Capstone sequencing trivial.

Item	Your status (fill now)
Host role	solid / partial / missing
HM or service user	
Secrets	
Edge/proxy	
Deploy tool	
Tests	
CI + cache	

Theory 2 — Coverage matrix (topic parts)

Part	Theme	Evidence in repo (examples)
Concepts	Foundations, flake shell	<code>devShells</code> , language notes
NixOS host	First host, users, net, systemd	<code>nixosConfigurations.*</code> , host modules
Home & layout	Layout, HM, checks	<code>home-manager</code> , flake structure
Services & security	Secrets, harden, proxy, data	sops, firewall, nginx/caddy
Packaging	Overlays, caches	custom packages, cachix config
Ops & fleet	Disko, deploy, images, tests, CI	deploy configs, workflows, tests
Internals	Evaluator, upgrades	notes artifacts; not all need code

Mark each: **solid** / **partial** / **missing** / **N/A**. Partial without a demo command is missing in disguise.

Theory 3 — Architecture views (write these)

A. Context

Who uses the system? Laptop only? Lab server? Family services? Threat model one paragraph.

B. Containers of code

```
flake.nix
hosts/
modules/
homes/
secrets/
tests/
deploy/
.github/ or .forgejo/
docs/
```

C. Runtime view

```
[client] → [proxy:443] → [app:8080]
      ↓
    [sops secrets]
      ↓
[deploy-rs/colmena from CI or admin]
```

D. Trust view

- Who can decrypt secrets?
- Who is `trusted-users`?
- Which caches are trusted?
- Where do age master keys live offline?

E. Failure view

- Boot rollback
- Deploy failure

- Secret loss (DR needs offline keys)
- Cache outage (can you still build?)

F. Data view

- What is stateful?
- Where are backups?
- Impermanence list if any?

Theory 4 — Prioritization

Score gaps:

Priority	Definition
P0	Capstone A impossible without it
P1	Required for DR proof quality
P2	Hardening / UX polish
P3	Nice / Capstone B

Sequence Capstone as:

Capstone chapter	Suggested focus
Scope and plan	P0 host + module skeleton gaps
Implement	Secrets + proxy E2E
Harden and document	Deploy + CI + cache + docs
Prove rebuild	Tests + freeze
Optional multi-node	Capstone B bar only if A solid
Disaster recovery	Wipe + rebuild timed
Rollback and tests	Rollback drill + suite green
Retrospective	Honest close-out

Adjust if your matrix differs—**write the plan**. A plan you do not write will be renegotiated under panic every session.

Theory 5 — Non-goals for Capstone A

Explicitly list what you will **not** do (protects freeze):

- Multi-region mesh
- Full k3s
- Every self-hosted app
- Capstone B fleet (unless ahead)
- Experimental CA store
- Perfect HA for lab toys

Non-goals are features, not apologies. Scope control is expertise.

Theory 6 — Evidence standards

Claim	Acceptable evidence
“We have sops”	Decrypt path works on host; secrets.nix in git
“We have CI”	Green URL + workflow file
“We have tests”	Check that fails when you break a module
“We deploy”	Log of remote activation this month
“Architecture exists”	ARCHITECTURE.md with diagrams

Screenshots of dashboards without repo paths do not count for Capstone A.

Theory 7 — Dossier structure (feeds Capstone)

Recommended docs layout:

```
docs/  
  ARCHITECTURE.md  
  GAPS.md  
  CAPSTONE-A.md  
  RUNBOOK.md      # deploy/rollback  
  TRUST.md         # keys, caches, trusted-users
```

```
playbooks/          # from troubleshooting
UPGRADE-26.11.md    # from upgrade planning
```

Capstone chapters will tick CAPSTONE-A.md and deepen RUNBOOK/DR.

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/architecture
cd ~/lab/nixos/internals/architecture
```

Work against the real Capstone flake path.

Lab 1 — Automated inventory

```
cd /path/to/flake
find . -name '*.nix' | head -200 | tee ~/lab/nixos/internals/architecture/nix-files.txt
nix flake show 2>&1 | tee ~/lab/nixos/internals/architecture/flake-show.txt
rg -n "sops|agenix|deploy-rs|colmena|home-manager|nixosTest|cachix" -g '*.nix' \
  | tee ~/lab/nixos/internals/architecture/feature-hits.txt
ls -la .github/workflows 2>/dev/null || ls -la .forgejo/workflows 2>/dev/null || echo "no CI"
```

Skim flake-show for dead attrs and surprise outputs.

Lab 2 — Coverage matrix

Fill COVERAGE.md:

Part	Status	Paths	Demo command	Notes
Concepts				
...				
Capstone A items 1–7	each row			

Be brutal. “I watched a video about sops” solid.

Lab 3 — ARCHITECTURE.md (required)

Minimum sections:

1. Purpose & scope
2. Host roles
3. Module map
4. Secrets model
5. Edge/proxy
6. Deploy & CI
7. Test strategy
8. Trust & keys
9. Backup/DR pointers
10. Gap list summary (link)
11. Non-goals
12. Pin / release (26.05) policy

Diagrams: ASCII is enough.

Acceptance: A stranger can find rebuild, secrets, and deploy paths in under five minutes of reading.

Lab 4 — Prioritized backlog

GAPS.md:

ID	Gap	Pri	Capstone chapter	Acceptance test
G01	No CI cache push	P0	Harden/document	workflow uploads
...				

Every P0/P1 needs an acceptance test you could demo in under five minutes.

Lab 5 — Checklist freeze seed

Create CAPSTONE-A.md checklist (boxes empty/partial/done) that Capstone will tick. Include wipe/rebuild and rollback rows even though execution is later.

```
# Capstone A checklist

## Core
- [ ] Host role rebuilds from flake
- [ ] HM or service users
- [ ] Secrets E2E
- [ ] Reverse proxy edge
- [ ] Deploy path
- [ ] nixosTest/check
- [ ] CI (+ cache preferred)

## Proof
- [ ] Wipe + rebuild timed
- [ ] Rollback drill + suite green

## Docs
- [ ] ARCHITECTURE.md current
- [ ] RUNBOOK.md deploy/rollback
```

Lab 6 — Risk register (short)

Three risks that could sink Capstone A:

Risk	Likelihood	Impact	Mitigation
Only age key on lab disk			Offline copy now
No console on remote			Enable before firewall games
Tests don't fail on breaks			Capstone tests priority

Lab 7 — Sequence plan

Write SEQUENCE.md: ordered list of Capstone sessions with intended gap IDs closed. Keep it one page.

Common gotchas

Mistake	Fix
Optimistic “partial = done”	Require demo command proof
Skipping docs because “I know it”	Capstone is proof for future you
40 P0s	Reclassify; cut scope
Planning Capstone B now	Park in non-goals
No acceptance tests on gaps	Unverifiable work
Architecture file never opened again	Link it from CAPSTONE-A.md

Checkpoint

- ☐ Inventory artifacts (`flake-show`, feature hits)
- ☐ `COVERAGE.md` part matrix
- ☐ `ARCHITECTURE.md` complete
- ☐ `GAPS.md` prioritized with Capstone mapping
- ☐ `CAPSTONE-A.md` checklist seeded
- ☐ Risk register + sequence
- ☐ Three personal gotchas

Commit

```
git add .  
git commit -m "lab(nixos): architecture review and capstone gap plan"
```

Write three personal gotchas before continuing.

Next

→ [Capstone and recovery](#) — implement Capstone A and prove rebuildability.

Capstone

Capstone and recovery

Capstone and recovery

This part of the **standalone NixOS book** is organized by **topic** (normal book chapters), not by a calendar. Complete Capstone A before optional multi-node work. Study pacing is yours.

Prerequisites. Architecture review complete (ARCHITECTURE.md, GAPS.md, seeded CAPSTONE-A.md). You can rebuild a flake host, manage secrets without store leakage, and run at least a stub flake check. Internals literacy (eval, modules, store, tests) is assumed when debugging Capstone failures.

What this part is for. Capstone A is not “another service.” It is **proof** that your repository rebuilds a real host role end-to-end: secrets, edge, deploy, tests, CI, wipe/rebuild, and rollback—with documentation an exhausted future-you can follow.

Capstone A vs optional bar

Track	Required?	Bar
Capstone A	Yes	Single host role (or primary lab guest) with full checklist
Multi-node / freeze discipline	Strongly recommended before DR	Feature freeze + tag; optional second node only if A green
Capstone B ideas	No	Fleet patterns—park in retrospective roadmap

! Important

Capstone A (required): flake host · HM or service users · sops/agenix · reverse proxy · deploy-rs/colmena · nixosTest · CI + cache preferred · wipe/rebuild · rollback + green suite · architecture + runbook docs.

Learning outcomes

When you leave this part you should be able to:

1. Close P0 gaps into a building host skeleton with honest checklist ticks.
 2. Wire secrets E2E and a reverse-proxy front door without plaintext secrets in git.
 3. Deploy remotely and run CI that builds (and preferably caches) Capstone artifacts.
 4. Freeze a tagged baseline with known waivers.
 5. Wipe and rebuild from flake + offline keys alone; time the recovery.
 6. Roll back a bad generation and keep the full test suite green.
 7. Write a retrospective and teach plan without starting a rewrite.
-

Chapter map

Chapter	Theme	Deliverable
Scope and plan	Close structural P0s	Green host + users + module hygiene
Implement	Secrets + proxy E2E	Encrypted secrets + front door proof
Harden and document	Deploy + CI + cache	One-command deploy + CI green
Prove rebuildability	Tests + runbook + diagrams	Sufficient suite + RUNBOOK
Optional multi-node bar	Feature freeze (+ optional node)	Annotated freeze tag
Disaster recovery	Wipe + rebuild timed	DR log + fixed docs
Rollback and tests	Generation rollback + suite	Rollback log + green CI
Retrospective	Teach + roadmap	Retro notes; no rewrite

Suggested sequence

Architecture review (internals)

Scope & plan → Implement secrets/proxy

Harden deploy/CI → Prove tests/runbook

Freeze tag → Wipe/rebuild DR

Rollback + full suite → Retrospective

Do **not** wipe before freeze. Do **not** start multi-region mesh mid-Capstone A.

Dossier structure (canonical)

Keep these paths (or equivalent) current throughout Capstone:

```
docs/
  ARCHITECTURE.md
  CAPSTONE-A.md           # live checklist
  GAPS.md                 # remaining / waived
  RUNBOOK.md             # deploy, rollback, health
  SECRETS.md              # key custody
  TLS.md                  # ACME vs lab mode
  DEPLOY.md               # remote path
  DR-PLAN.md              # wipe/rebuild steps
  TRUST.md                # caches, trusted-users
  POST-FREEZE.md          # ideas deferred after freeze
  playbooks/              # PL-*.md from drills
  RETRO.md                # retrospective
```

Doc	Owner chapter
CAPSTONE-A.md	All; start in scope
SECRETS / TLS	Implement
DEPLOY / CI notes	Harden
RUNBOOK / diagrams	Prove rebuild
Freeze tag notes	Multi-node / freeze
DR-PLAN + timing	Disaster recovery
Rollback log	Rollback and tests
RETRO	Retrospective

Live Capstone A checklist (copy into repo)

```
# CAPSTONE-A.md
- [ ] Host role rebuilds from flake
- [ ] HM user OR service user story
- [ ] sops-nix OR agenix wired; service reads secret
- [ ] Reverse proxy edge with TLS or lab HTTP mode documented
- [ ] deploy-rs OR colmena path works from admin machine
- [ ] nixosTest/check catches a real regression
- [ ] CI builds toplevel (and checks)
- [ ] CI pushes to binary cache (preferred)
- [ ] ARCHITECTURE.md current
- [ ] RUNBOOK.md for deploy/rollback
- [ ] Feature freeze tag (recommended before wipe)
- [ ] Wipe + rebuild timed from flake + offline keys
- [ ] Rollback drill + suite green
```

Tick only with **proof commands** recorded (build log path, curl, deploy log).

Lab workspace convention

```
mkdir -p ~/lab/nixos/capstone/{scope,implement,hardening,prove,freeze,dr,rollback,retro}
```

Prefer a single Capstone git branch (capstone-a) over calendar-named branches.

Acceptance bar for Capstone A

- ☐ Every required checklist row has a demo command
- ☐ Secrets never appear as Nix strings in the store graph
- ☐ Front door health probe documented
- ☐ Remote deploy works from admin machine (or documented exception with local-only waiver)
- ☐ At least one behavioral check fails when you break a module
- ☐ DR wipe/rebuild timed with doc fixes committed

- ☐ Rollback proven after intentional bad change
- ☐ Freeze tag (or explicit waiver why not) before DR

Elitebook / KVM lab notes

Concern	Capstone guidance
16 GB RAM	Capstone A on one primary guest; optional second node only if headroom
Snapshots	Snapshot before implement breaks, freeze, and DR
Console	Required before firewall / wipe experiments
Disk	Free space for two system closures + installer ISO
KVM	Needed for nixosTest in prove/rollback chapters

Policy reminders

Policy	Rule
Pin	nixpkgs 26.05 for Capstone A
Store	Default input-addressed; no experimental CA on Capstone
Secrets	Offline private key backup before wipe
Scope	Non-goals stay non-goals until retrospective roadmap
Day framing	None —these are topic chapters

Next

→ [Capstone: scope and plan](#) — close structural P0s and make the checklist real.

Capstone: scope and plan

Capstone: scope and plan

Goal: Close the highest **P0** gaps from the architecture review so the Capstone A skeleton is real: flake host role, module layout, HM or service users, and a live checklist that tracks proof—not vibes.

! Important

Capstone A (required): flake host · HM **or** service users · sops/agenix · reverse proxy · deploy-rs/colmena · nixosTest · CI + cache.

Later Capstone chapters implement secrets/proxy, harden deploy/CI, prove rebuild, optional freeze, DR wipe, rollback, and retrospective.

Why this chapter exists

Architecture review without commits is stationery. This chapter turns **GAPS.md** P0s into code on a branch, with rebuilds you can demonstrate.

Theory — Working agreements for Capstone

Rule	Why
One change → build → commit	Bisectable; safer switch
Lab VM first	Protect daily driver
Checklist is law	CAPSTONE-A.md updated same session
No drive-by rewrites	Freeze before DR
Secrets never in git plaintext	Even “temporary”

Suggested repo shape (adapt, don’t dogma)

```
flake.nix
flake.lock
hosts/<name>/configuration.nix
modules/{proxy,common,users,...}.nix
homes/<user>.nix          # if HM
```

```
secrets/secrets.yaml      # sops-encrypted
tests/*.nix
deploy.nix or colmena hive
.github/workflows/ci.yml
docs/{ARCHITECTURE,CAPSTONE-A,RUNBOOK,GAPS}.md
```

Capstone A checklist (live)

Copy into repo if missing; tick as you go:

```
# CAPSTONE-A.md
- [ ] Host role rebuilds from flake
- [ ] HM user OR service user story
- [ ] sops-nix OR agenix wired; service reads secret
- [ ] Reverse proxy edge with TLS or lab HTTP mode documented
- [ ] deploy-rs OR colmena path works from admin machine
- [ ] nixosTest/check catches a real regression
- [ ] CI builds toplevel (and checks)
- [ ] CI pushes to binary cache (preferred)
- [ ] ARCHITECTURE.md current
- [ ] RUNBOOK.md for deploy/rollback
- [ ] Disaster recovery: wipe + rebuild timed
- [ ] Rollback drill + suite green
```

Target scope for this chapter

From GAPS.md, implement **P0s that unblock structure**, typically:

1. `nixosConfigurations.<lab>` evaluates and builds
2. Host modules split (common + role)
3. Users: interactive HM **or** system service user + data dir
4. Firewall baseline + SSH
5. Placeholder modules for proxy/secrets if not ready (interfaces only)—prefer real stubs that `mkEnableOption` cleanly

Defer full proxy TLS and CI cache to implement/harden chapters if needed—but leave option shapes ready.

Explicit non-goals (this chapter)

Non-goal	Why
Multi-node mesh	Optional bar later
Perfect ACME	Document lab mode later
Full observability stack	P2+
CA store experiments	Internals literacy only

Worked example — Minimal host flake skeleton

```
# flake.nix (sketch)
{
  description = "capstone lab";
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    # home-manager.url = "github:nix-community/home-manager/release-26.05";
    # home-manager.inputs.nixpkgs.follows = "nixpkgs";
    # sops-nix.url = "github:Mic92/sops-nix";
    # sops-nix.inputs.nixpkgs.follows = "nixpkgs";
  };
  outputs = { self, nixpkgs, ... }@inputs: {
    nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      specialArgs = { inherit inputs; };
      modules = [
        ./hosts/lab/configuration.nix
        ./modules/common.nix
        # inputs.sops-nix.nixosModules.sops
        # inputs.home-manager.nixosModules.home-manager
      ];
    };
  };
}

# modules/common.nix
{ lib, pkgs, ... }:
{
  time.timeZone = "UTC";
  networking.firewall.enable = true;
  services.openssh.enable = true;
  environment.systemPackages = with pkgs; [ vim git curl ];
}
```

```
# hosts/lab/configuration.nix
{ config, pkgs, ... }:
{
  imports = [ ./hardware.nix ];
  networking.hostName = "lab";
  # users / HM / roles
  system.stateVersion = "26.05"; # your real value
}
```

Theory — Capstone A success definition (this chapter)

Done	Not done
Green host from flake	Perfect theming
Users + SSH story	Every optional service
Module hygiene applied	Multi-region fleet
Architecture doc updated	Final polish

Scope control is part of the grade.

Theory — Branch discipline

```
git checkout -b capstone-a
# work
# merge only when smoke passes
```

Avoid committing broken main mid-capstone if you use the same repo for the lab host.

Worked example — smoke script

```
#!/usr/bin/env bash
set -euo pipefail
hostname
systemctl is-active sshd
```

```
curl -sf -o /dev/null -w '%{http_code}\n' http://127.0.0.1:... || true
nix flake metadata
```

Keep it boring; run after every significant rebuild.

Rebuild proof steps (skeleton → switch)

Step	Command	Pass means
1	<code>nix flake show</code>	Host attr visible
2	<code>nix build</code> <code>.#nixosConfigurations.lab.config.system.build.toplevel</code> <code>-L</code>	Exit 0
3	<code>nixos-rebuild</code> <code>dry-activate --flake</code> <code>.#lab</code>	No activation surprises
4	<code>sudo nixos-rebuild switch</code> <code>--flake .#lab (lab only)</code>	SSH still works
5	<code>systemctl --failed</code>	Empty or known waivers

Lab 0 — Branch & checklist

```
cd /path/to/capstone-flake
git checkout -b capstone-a
mkdir -p docs ~/lab/nixos/capstone/scope
# seed docs/CAPSTONE-A.md from architecture review if needed
```

Lab 1 — Build green host

```
nix flake show
nix build .#nixosConfigurations.lab.config.system.build.toplevel -L \
  2>&1 | tee ~/lab/nixos/capstone/scope/build.log
```

Fix until green. Prefer build over switch until config is sane; then:


```
sudo nixos-rebuild switch --flake .#lab
# or deploy path if already wired
```

Acceptance: toplevel green; hostname/SSH story demoable.

Lab 2 — Users story

Track HM (workstation-ish)

- Enable HM as NixOS module
- Manage shell + one dotfile
- Prove with home-manager generations or files in place

Track service users (server-ish)

```
users.users.myapp = {
  isSystemUser = true;
  group = "myapp";
  home = "/var/lib/myapp";
  createHome = true;
};
users.groups.myapp = { };
```

Document which track in docs/CAPSTONE-A.md.

Lab 3 — Module hygiene

Ensure at least one custom module uses:

- typed options
- `mkEnableOption`
- `mkIf`
- one assertion

Even if proxy is stubbed:

```
options.my.capstone.proxy.enable = lib.mkEnableOption "edge proxy";
```

Lab 4 — Update architecture & gaps

```
# edit docs/ARCHITECTURE.md paths if moved
# mark GAPS.md items closed with PR/commit refs
```

Write ~/lab/nixos/capstone/scope/NOTES.md: what shipped, what slips to implement.

Lab 5 — Smoke

```
systemctl --failed
hostname
# ssh localhost or from client
```

Lab 6 — Gap register freeze for next chapter

Write docs/capstone-gaps.md with only items that remain for implement/harden. Anything critical for a bootable admin host must be closed now.

Lab 7 — Teach-back outline (5 bullets)

1. How to rebuild
 2. How to roll back
 3. Where secrets will live
 4. What state to back up
 5. What implement will add
-

Failure modes

Symptom	Fix
Hardware config missing	Import hardware or disk
Infinite recursion after split	Option cycles; evaluator chapter
HM + system package duplication	Pick layers deliberately
Switching daily driver	Stop; use VM
Checklist not updated	Same commit as code

Common gotchas

Symptom	What to do
Hardware config missing in flake	Import <code>hardware-configuration.nix</code> or disk
Infinite recursion after module split	Dependency cycles
HM + system package duplication	Pick layers deliberately
Switching daily driver	Stop; use VM
Checklist not updated	Same commit as code

Checkpoint

- ☐ `nix build` toplevel green for lab host
- ☐ HM **or** service user story demoable
- ☐ Modules structured; 1 typed custom module
- ☐ `CAPSTONE-A.md` updated honestly
- ☐ `GAPS.md` P0s reduced or re-dated
- ☐ Commits on capstone branch

Commit

```
git add .  
git commit -m "capstone: host skeleton and user story"
```

Write three personal gotchas before continuing.

Next

→ [Capstone: implement](#) — secrets E2E and reverse-proxy front door.

Capstone: implement

Capstone: implement

Goal: Complete **secrets E2E** (sops-nix or agenix) and the **edge reverse-proxy** path so a service is reachable through the front door without plaintext secrets in git.

! Important

Use **lab-only** tokens. Never commit production credentials. Prefer **sops-nix** as house default; agenix is fine if Capstone already standardized on it—do not run both permanently.

Why this chapter exists

Host skeletons without secrets train bad habits. Proxy without a TLS story (or documented lab HTTP mode) is not an edge pattern. This chapter closes Capstone A items for **secrets + reverse proxy** with proof commands you can paste into the checklist.

Theory 1 — Secrets E2E definition of done

Step	Done means
Encrypt	secrets/*.yaml (or age files) committed encrypted
Keys	Machine age/ssh key or admin key decrypts on host
NixOS wiring	sops.secrets.* or age.secrets.* → path on disk
Consumer	systemd LoadCredential / EnvironmentFile / path read
Negative proof	Repo clone alone cannot read secret without keys
Test story	nixosTest uses dummy secret or test key
Offline custody	Private keys exist off the lab disk

Threat model (short)

Bad outcome	Control
Secret in git plaintext	SOPS/age encrypt before commit
Secret in Nix string → store	Reference paths , never secret values in modules

Bad outcome	Control
World-readable decrypted file	<code>owner</code> / <code>mode</code> / service user match
Lost keys after wipe	Offline backup of age/ssh private material

Theory 2 — sops-nix sketch (preferred)

```
# host module fragment - follow current sops-nix README for key paths
{ config, ... }:
{
  sops.defaultSopsFile = ./secrets/secrets.yaml;
  # common patterns: age ssh host key and/or dedicated age key file
  sops.age.sshKeyPaths = [ "/etc/ssh/ssh_host_ed25519_key" ];
  # sops.age.keyFile = "/var/lib/sops-nix/key.txt";

  sops.secrets."myapp/api_token" = {
    owner = "myapp";
    group = "myapp";
    mode = "0400";
    restartUnits = [ "myapp.service" ];
  };

  systemd.services.myapp = {
    description = "Capstone sample app";
    wantedBy = [ "multi-user.target" ];
    serviceConfig = {
      User = "myapp";
      Group = "myapp";
      EnvironmentFile = config.sops.secrets."myapp/api_token".path;
    };
  };
}
```

.sops.yaml creation rules (idea)

Encrypt lab secrets to **host** recipient + **admin** recipient so you can edit on a workstation and the machine can decrypt at activation.

```
sops secrets/secrets.yaml
```

Theory 3 — agenix sketch (alternative)

```
age.secrets.apiToken = {
  file = ../secrets/apiToken.age;
  owner = "myapp";
  mode = "0400";
};
# service references config.age.secrets.apiToken.path
```

Pick **one** house default for Capstone A.

Theory 4 — Proxy E2E definition of done

Step	Done means
Upstream	App listens localhost (or internal-only) port
Proxy	nginx/caddy/traefik terminates edge
Firewall	Only designed ports public (often 80/443 + SSH)
TLS	ACME or lab self-signed/internal CA or HTTP lab mode documented
Probe	<code>curl -f</code> through proxy hostname/port
Negative	Upstream port not world-open unless intentional
Regression	Test or script fails if upstream link breaks

nginx sketch

```
{ config, lib, ... }:
{
  services.nginx = {
    enable = true;
    recommendedProxySettings = true;
    recommendedTlsSettings = true;
    virtualHosts."lab.example.com" = {
      # forceSSL = true;
      # enableACME = true;
      locations."/\" = {
        proxyPass = "http://127.0.0.1:8080";
        proxyWebsockets = true;
      };
    };
  };
};
networking.firewall.allowedTCPPorts = [ 80 ]; # lab HTTP mode example
```



```
}
```

TLS modes (pick and document in docs/TLS.md)

Mode	When
ACME / public DNS	Real hostname, ports reachable
Self-signed / internal CA	Lab realism without public DNS
HTTP-only lab	Acceptable for Capstone if prod path is written down

Theory 5 — Ordering and dependencies

```
secret decrypt (activation)
  → app service start (reads secret path)
  → proxy start / reload
  → health probe via front door
```

Use `restartUnits` on secrets, `systemd After=/Requires=` where needed, and in tests `wait_for_unit` for **both** app and proxy.

Store discipline reminder

```
# WRONG - leaks into world-readable store
environment.variables.API_TOKEN = "supersecret";

# RIGHT - path only
# EnvironmentFile = config.sops.secrets."myapp/api_token".path;
```

Theory 6 — Module shape for Capstone

```
# modules/proxy.nix (sketch)
{ lib, config, ... }:
let cfg = config.roles.proxy; in
{
  options.roles.proxy = {
```

```
enable = lib.mkEnableOption "edge reverse proxy";
upstream = lib.mkOption {
  type = lib.types.str;
  default = "http://127.0.0.1:8080";
};
hostName = lib.mkOption {
  type = lib.types.str;
  default = "lab.example.com";
};
};
config = lib.mkIf cfg.enable {
  # services.nginx.virtualHosts.${cfg.hostName} = ...
};
}
```

Concrete checklists

Secrets checklist

- ☐ Lab token generated (not production)
- ☐ Ciphertext committed; plaintext never staged
- ☐ Host decrypts after switch
- ☐ Unit user can read path; others cannot
- ☐ Offline private key backup exists
- ☐ Negative: `git grep` no raw token

Proxy checklist

- ☐ Upstream localhost-only (or documented)
- ☐ Front door probe succeeds
- ☐ Firewall matches design
- ☐ `docs/TLS.md` written
- ☐ Negative: external upstream port closed if required

Worked example — Proof commands

```
sudo ls -la /run/secrets 2>/dev/null || sudo ls -la /run/secrets.d 2>/dev/null
systemctl status myapp.service --no-pager
curl -fsS http://127.0.0.1:8080/ | head
curl -fsS -H "Host: lab.example.com" http://127.0.0.1/ | head
git grep -n "the-real-lab-token-value" && echo "LEAK" || echo "ok-no-plaintext"
```

Lab 0 — Branch

```
mkdir -p ~/lab/nixos/capstone/implement
cd /path/to/capstone-flake
git checkout capstone-a # continue Capstone branch
```

Lab 1 — Secrets path E2E

1. Ensure age/ssh keys exist for the lab host; **backup private material offline**.
2. Generate a non-production random token.
3. Encrypt with sops-nix or agenix workflow; commit **ciphertext only**.
4. Wire module → service consumer via path.
5. Switch/deploy; prove service healthy.

```
sudo systemctl restart myapp.service
systemctl is-active myapp.service
journalctl -u myapp.service -n 50 --no-pager
head -n 5 secrets/secrets.yaml
```

Acceptance: Service active; secret path exists; git has ciphertext only.

Lab 2 — Proxy path E2E

1. Enable proxy module with typed options if still weak.
 2. Upstream app on localhost only.
 3. Firewall ports match design.
 4. Choose TLS mode; write docs/TLS.md.
 5. Prove front door with curl.
-

Lab 3 — Failure injection (mini)

1. Wrong secret path or mode → service fails → fix; note in playbook.
 2. Wrong upstream port → 502 → fix.
 3. Leave host **green** at end.
-

Lab 4 — Tests stub toward prove-rebuild chapter

```
machine.wait_for_unit("myapp.service")
machine.wait_for_unit("nginx.service")
machine.wait_for_open_port(80)
machine.succeed("curl -f http://lab/")
```

```
nix build .#checks.x86_64-linux.<test> -L \
  2>&1 | tee ~/lab/nixos/capstone/implement/test.log
```

If full test must wait, leave a **failing checklist note** with reason—not a silent gap.

Lab 5 — Docs & checklist

Doc	Content
docs/ARCHITECTURE.md	Secrets + edge data flow
docs/SECRETS.md	Who can decrypt; key locations; rotation sketch
docs/TLS.md	ACME vs lab mode
docs/CAPSTONE-A.md	Tick secrets + proxy with proof commands

Lab 6 — Offline key custody proof

```
# Document location of offline age/ssh private material
# Do NOT commit private keys
# Verify you can open the offline backup medium
```

Write path (physical) in docs/SECRETS.md offline section.

Common gotchas

Symptom	What to do
Secret in Nix string	Use sops/age paths only
ACME fail on lab	Document offline/lab TLS; don't block Capstone
Proxy works, app ignores secret	Prove unit environment loads file
Permission denied on secret	owner/mode/group match unit User=
Encrypted file looks empty	Confirm sops metadata headers

Checkpoint

- ☐ Secrets E2E with negative git proof
- ☐ Proxy front door probe
- ☐ docs/SECRETS.md + docs/TLS.md
- ☐ Offline keys backed up

- ☐ CAPSTONE-A ticks for secrets + proxy with commands
- ☐ Host green

Commit

```
git add .  
git commit -m "capstone: secrets and reverse-proxy E2E"
```

Write three personal gotchas before continuing.

Next

→ [Capstone: harden and document](#) — remote deploy, CI, cache.

Capstone: harden and document

Capstone: harden and document

Goal: Make **remote deploy** (deploy-rs and/or Colmena) the normal path, and wire **CI** that builds Capstone artifacts and **preferably pushes a binary cache**.

i Note

Baseline: flakes on **nixos-26.05**, secrets already on a path. This chapter is remote delivery + automation—not new app features.

Why this chapter exists

`nixos-rebuild --flake` only on the box is not a deploy story. Capstone A requires a deploy path and CI that would catch breakage before you SSH in panic. Cache push turns CI minutes into laptop seconds.

Theory 1 — Deploy definition of done

Item	Done means
Tool chosen	deploy-rs or Colmena (both OK; one is enough for A)
Config in repo	<code>deploy.nodes</code> or Colmena hive committed
Auth	SSH key or deploy user documented
Magic	One command updates lab host from admin machine
Failure UX	Partial failure doesn't leave mystery state; rollback known
Docs	<code>docs/DEPLOY.md</code> with command + prerequisites + rollback

deploy-rs sketch

```
# flake outputs fragment - follow current deploy-rs docs for 26.05-era usage
{
  inputs.deploy-rs.url = "github:serokell/deploy-rs";
  outputs = { self, nixpkgs, deploy-rs, ... }: {
    nixosConfigurations.lab = /* ... */;
    deploy.nodes.lab = {
```



```

hostname = "lab.local"; # or IP
profiles.system = {
  user = "root";
  path = deploy-rs.lib.x86_64-linux.activate.nixos self.nixosConfigurations.lab;
};
};
};
}

deploy .#lab
# or: nix run github:serokell/deploy-rs -- .#lab

```

Colmena sketch

```

# hive.nix idea
{
  meta.nixpkgs = import (builtins.getFlake "...").inputs.nixpkgs { };
  lab = { ... imports ... };
}

colmena apply --on lab

```

Pick one primary path; document the other as optional.

Theory 2 — CI definition of done

Item	Done means
Workflow file	.github/workflows/ci.yml (or Forgejo/Gitea equivalent)
Triggers	PR + push to main (or Capstone branch policy)
Build	nix build toplevel and/or nix flake check
Logs	Failures visible in CI UI
Cache (preferred)	Push to Cachix/Attic/etc. with secrets in CI vault
Pin	Nix installer version documented

Minimal workflow sketch

```
name: ci
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: cachix/install-nix-action@v27
        with:
          nix_path: nixpkgs=channel:nixos-26.05
      # - uses: cachix/cachix-action@v15
      #   with: { name: mycache, authToken: '${{ secrets.CACHIX_AUTH_TOKEN }}' }
      - run: nix build .#nixosConfigurations.lab.config.system.build.toplevel -L
      - run: nix flake check -L
```

Adapt for KVM runners if nixosTest requires nested virt; otherwise split fast checks vs nightly tests.

Theory 3 — Cache push value

Without project cache	With project cache
Every agent rebuilds your overlays	Shared substitutes
Laptop recompiles CI work	Laptop substitutes CI outputs
Slow feedback	Minutes matter less

If you cannot push a cache yet, **document waiver** and still build in CI. Preferred: push.

Theory 4 — SSH deploy auth checklist

Check	Note
Deploy key or agent	Non-interactive preferred for CI deploy (optional)
root vs sudo user	Match deploy tool docs
Host key verification	known_hosts managed deliberately
Network path	Admin machine can reach lab

Check	Note
Firewall	SSH open; no accidental lockout without console

Theory 5 — RUNBOOK / DEPLOY docs minimum

docs/DEPLOY.md:

1. Prerequisites (keys, network, flake path)
 2. Happy path one-liner
 3. Dry-run / build-only variant
 4. Failure symptoms
 5. Rollback pointer (docs/RUNBOOK.md)
 6. Who is allowed to deploy
-

Concrete checklists

Deploy checklist

- ☐ Tool chosen and committed
- ☐ One-command deploy from admin machine
- ☐ Deploy log saved
- ☐ Rollback path written
- ☐ docs/DEPLOY.md complete

CI checklist

- ☐ Workflow runs on PR
 - ☐ Toplevel or checks green
 - ☐ Cache push **or** explicit waiver
 - ☐ Nix version noted
 - ☐ Badge or URL in CAPSTONE-A
-

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/capstone/harden  
cd /path/to/capstone-flake
```

Snapshot lab VM before first remote deploy if deploy is new.

Lab 1 — Wire deploy tool

1. Add deploy-rs or Colmena input.
2. Point at lab host.
3. Deploy once from admin machine.
4. Capture log:

```
deploy .#lab 2>&1 | tee ~/lab/nixos/capstone/harden/deploy.log
```

Acceptance: Remote activation succeeds; host healthy after.

Lab 2 — CI workflow

1. Add workflow file.
2. Push branch; observe CI.
3. Fix until green.
4. Optionally add Cachix/Attic push.

```
# local parity
nix build .#nixosConfigurations.lab.config.system.build.toplevel -L
nix flake check -L
```

Lab 3 — Hardening pass (light)

Do **not** start new apps. Optionally:

- SSH: password auth off if not already
- Firewall review
- `systemctl --failed clean`
- Fail2ban only if already planned

Document changes in ARCHITECTURE.

Lab 4 — Docs

Doc	Update
docs/DEPLOY.md	Full path
docs/RUNBOOK.md	Deploy + rollback stubs
docs/CAPSTONE-A.md	Tick deploy + CI
docs/TRUST.md	CI cache keys / trusted-users if changed

Lab 5 — Failure tabletop

Write without executing production:

1. Deploy fails mid-activation → what do you do?
 2. CI red on main → who unblocks?
 3. Cache token leaked → rotation steps
-

Lab 6 — Elitebook admin path

If admin machine is the Elitebook host and target is guest:

```
# from host
ping -c1 <guest>
ssh root@<guest> true
deploy .#lab # or colmena
```

Acceptance: Works over libvirt network without manual `nixos-rebuild` on guest.

Common gotchas

Symptom	Fix
SSH permission denied	Keys, user, PermitRootLogin policy
deploy evaluates wrong system	system/attr mismatch
CI OOM	lower jobs; cache; split matrix
nixosTest needs KVM on GA	nightly self-hosted or skip in PR
Cache auth fails	secrets scope; trusted-public-keys on consumers

Checkpoint

- ☐ Remote deploy one-liner works
- ☐ CI builds Capstone artifacts
- ☐ Cache push or waiver documented
- ☐ DEPLOY + RUNBOOK updated
- ☐ CAPSTONE-A ticks with proof
- ☐ Three personal gotchas

Commit

```
git add .  
git commit -m "capstone: deploy path and CI hardening"
```

Write three personal gotchas before continuing.

Next

→ [Capstone: prove rebuildability](#) — tests, runbook, diagrams.

Capstone: prove rebuildability

Capstone: prove rebuildability

Goal: Leave Capstone with a **sufficient test suite**, a **RUNBOOK** an exhausted future-you can follow, and architecture diagrams that match reality—operability, not feature sprawl.

i Note

Baseline: NixOS **26.05**, flakes, sops-nix (or documented agenix), deploy-rs or Colmena. This chapter hardens proof and human interface—not new roles.

Why this chapter exists

Code without operable docs fails disaster recovery. Tests without assertions that match the proxy/secrets design fail rollback drills. This chapter closes Capstone A quality on **tests + run-book + diagrams**: the difference between a demo host and a rebuildable system.

Theory 1 — “Sufficient” tests for Capstone A

Minimum bar:

Test	Intent
Eval/build in CI	Configuration still composes
1 nixosTest or strong VM-less check	Behavioral regression
Secret-safe	No prod keys required in CI
Proxy/app path	Front door works as designed
Optional negative	Firewall deny or failed assertion fixture

Stretch (only if already green):

- Multi-node client→server
- Deploy dry-run / eval in CI
- Dead-man checks for deprecated options

Flake outputs shape

```
# flake fragment - adapt to your layout
{
  checks.x86_64-linux.proxy = pkgs.nixosTest { /* ... */ };
  # packages / apps optional
}
```

```
nix build .#checks.x86_64-linux.proxy -L
nix flake check -L
```

Theory 2 — RUNBOOK audience

Write for **tired you at 2am** after a bad switch:

Section	Content
Identity	Host name, role, IP/DNS
Health probes	curl, systemctl, logs
Deploy	One-liner + prerequisites
Rollback	Generation + git
Secrets	Where keys live; what not to do
Escalation	Console path; snapshot restore

Avoid tutorial prose. Use numbered steps and copy-paste commands.

Theory 3 — Diagrams that match reality

Minimum diagrams (ASCII OK):

1. **Runtime:** client → proxy → app → secret path
2. **Deploy:** admin → tool → host activation
3. **Trust:** who decrypts; which caches

If the diagram shows ACME but lab is HTTP-only, fix diagram or TLS mode.

Theory 4 — Rebuild proof steps (pre-wipe)

You are not wiping yet. Prove *paper* rebuildability:

Step	Evidence
Fresh clone builds	CI or clean machine <code>nix build</code>
Docs list install path	DR-PLAN draft exists
Keys offline	SECRETS.md offline section
stateVersion recorded	ARCHITECTURE / host module
Disk layout documented	disko or hardware notes

Theory 5 — Assertion fixtures

Keep at least one **bad config** snippet that must fail eval (from module deep dive). CI can later evaluate it expecting failure, or you re-run manually before freeze.

```
# expect non-zero
nix eval -f tests/bad-host.nix config 2>&1 | tee bad-host.err
```

Concrete checklists

Tests checklist

- ☐ Suite inventory (SUITE.md or RUNBOOK section)
- ☐ Critical path without blind `sleep`
- ☐ Twice-green local run
- ☐ CI runs checks or documents KVM waiver
- ☐ Regression assertion present

Docs checklist

- ☐ RUNBOOK complete
 - ☐ ARCHITECTURE matches runtime
 - ☐ Diagrams updated
 - ☐ CAPSTONE-A ticks with log paths
-

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/capstone/prove
cd /path/to/capstone-flake
```

Lab 1 — Harden tests

1. Inventory checks.
2. Remove sleep-only waits.
3. Add/verify proxy + app assertions.
4. Run twice:

```
nix build .#checks.<system>.<name> -L 2>&1 | tee ~/lab/nixos/capstone/prove/check1.log
nix build .#checks.<system>.<name> -L 2>&1 | tee ~/lab/nixos/capstone/prove/check2.log
```

Acceptance: two consecutive greens; no sole blind sleep on critical path.

Lab 2 — RUNBOOK

Write or finish docs/RUNBOOK.md with deploy, health, rollback, secrets pointers.

Read it once **without** the repo open on a second machine if possible—note gaps.

Lab 3 — Diagrams

Update ARCHITECTURE with three diagrams. Paste into CAPSTONE notes.

Lab 4 — Paper DR draft

Create docs/DR-PLAN.md draft (steps only; wipe later):

1. Confirm target disk
 2. Boot installer
 3. Partition (disko command or manual)
 4. Install flake
 5. Place keys
 6. Deploy/switch
 7. Health probes
 8. Timing log template
-

Lab 5 — Freeze readiness review

List P0s remaining. If any block health, fix now. Else schedule freeze chapter.

Lab 6 — Negative eval fixture

Commit tests/fixtures/bad-*.nix that fails type or assertion; document expected error fingerprint.

Common gotchas

Mistake	Fix
Docs describe yesterday's layout	Diff ARCHITECTURE vs flake show
Tests only curl localhost app	Include front door
RUNBOOK assumes muscle memory	Numbered commands
CI never runs tests	Wire or waive with reason
Secret keys in CI	Dummy secrets / testMode

Checkpoint

- ☐ Sufficient tests green twice
- ☐ RUNBOOK exhausted-you ready
- ☐ Diagrams match reality
- ☐ DR-PLAN draft exists
- ☐ CAPSTONE-A updated
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "capstone: tests runbook and rebuild proof docs"
```

Write three personal gotchas before continuing.

Next

→ **Optional multi-node bar** — feature freeze (and optional second node).

Capstone: optional multi-node bar

Capstone: optional multi-node bar

Goal: Feature freeze the Capstone A MVP—no new services or toys—only bugfixes, doc fixes, and checklist honesty—so disaster recovery tests a **stable, tagged** artifact. Optionally add a second node **only if** Capstone A is already green and hardware allows.

! Important

After the freeze tag, the only legitimate code changes before wipe are **P0 blockers** that make rebuild or health checks impossible. Everything else goes in `docs/POST-FREEZE.md`.

Why this chapter exists

Wipe/rebuild against a moving target is chaos. Freeze creates a known baseline: annotated git tag, green CI, known gaps **explicitly waived** with risk. Without freeze, disaster-recovery drills measure luck, not process.

Experts freeze early relative to demos. Amateurs keep “just one more module” until the disk wipe and then invent history.

Theory 1 — What “freeze” means

Allowed	Forbidden
Bugfixes for P0/P1 with issue id	New apps / node roles / hostnames
Doc corrections that match code	“Quick” plugin or package installs
Test stability fixes (flakes, waits)	Refactors without bug id
Checklist updates (honest ticks)	Starting Capstone B
Dependency bumps only if build broken	Casual <code>nix flake</code> update
Typo-level RUNBOOK fixes	New sops keys without re-documenting DR

Freeze is a social contract with yourself

If Capstone is a solo lab, freeze still matters: during DR you will be tired and tempted to “fix while rebuilding.” The tag is the line between **recovering a known system** and **redesigning under fire**.

Tag shape

```
git tag -a capstone-a-freeze -m "Capstone A feature freeze"
git push origin capstone-a-freeze # if remote exists
git rev-parse capstone-a-freeze
```

Record tag SHA in docs/CAPSTONE-A.md.

Theory 2 — Waiver register

Every unticked Capstone A row at freeze needs one of:

Disposition	Meaning
Done	Proof command recorded
Waived	Explicit risk + why DR still valid
Blocker	Must fix before freeze tag

```
# docs/POST-FREEZE.md / freeze waivers
| Item | Disposition | Risk | Owner |
|-----|-----|-----|-----|
| CI cache push | Waived | Slower rebuilds | you |
```

Theory 3 — Optional multi-node bar

Capstone A is **single-node sufficient**. Multi-node is optional prestige:

Optional	Description
Second guest	deploy-a or similar with shared modules
Client/server test	nixosTest multi-node
Distinct roles	edge vs data

Config	Advice
--------	--------

Elitebook 16 GB guidance

Config	Advice
Host + 1 guest 4-6 GB	Capstone A default
Host + 2 guests	Only if you drop desktop/k3s toys
Nested heavy tests	Prefer serial; lower max-jobs

Do **not** add a second node if it risks freeze honesty or disk pressure before DR.

Shared modules pattern

```
modules/common.nix
modules/roles/edge.nix
modules/roles/data.nix
hosts/lab/...
hosts/deploy-a/... # optional
```

Avoid copy-paste hosts that drift.

Theory 4 — Freeze gate commands

```
nix flake metadata
nix build .#nixosConfigurations.lab.config.system.build.toplevel -L
nix flake check -L # or specific checks
# deploy once more if remote path is Capstone-critical
systemctl --failed # on lab
```

All green → tag. Any red → fix or waive with eyes open.

Concrete freeze checklist

- ☐ CAPSTONE-A honest ticks
 - ☐ Waivers written
 - ☐ CI green on freeze commit
 - ☐ RUNBOOK matches freeze tag
 - ☐ Offline keys verified present
 - ☐ VM snapshot `pre-dr` taken
 - ☐ `git tag -a capstone-a-freeze`
 - ☐ POST-FREEZE.md for deferred ideas
-

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/capstone/freeze  
cd /path/to/capstone-flake
```

Lab 1 — Honesty pass on CAPSTONE-A

For each box: done with proof, waived, or blocker. No “partial” without a sentence.

Lab 2 — Green suite at freeze

```
nix build .#nixosConfigurations.lab.config.system.build.toplevel -L \  
2>&1 | tee ~/lab/nixos/capstone/freeze/toplevel.log  
nix flake check -L 2>&1 | tee ~/lab/nixos/capstone/freeze/check.log
```

Lab 3 — Tag

```
git status
git tag -a capstone-a-freeze -m "Capstone A feature freeze $(date -I)"
git show capstone-a-freeze --stat | tee ~/lab/nixos/capstone/freeze/tag.show
```

Lab 4 — Optional second node (only if A green)

1. Snapshot first.
 2. Add host with shared modules.
 3. Deploy or rebuild.
 4. Document network and secrets recipients.
 5. If it destabilizes freeze, **revert** and leave as POST-FREEZE.
-

Lab 5 — Snapshot for DR

```
# virsh snapshot-create-as lab pre-dr --disk-only --atomic
# or hypervisor UI
```

Confirm you can revert without using DR path (sanity).

Lab 6 — Freeze announcement (to yourself)

Write 10 lines in NOTES: what is in, what is out, tag SHA, next step = DR wipe.

Common gotchas

Mistake	Fix
Tagging red CI	Fix or waive first
Freeze then “one more feature”	POST-FREEZE.md
Multi-node thrash on 16 GB	Defer
No snapshot before DR	Snapshot now
Lost tag SHA	Record in CAPSTONE-A

Checkpoint

- ☐ Honest CAPSTONE-A + waivers
- ☐ Green build/check logs
- ☐ `capstone-a-freeze` annotated tag
- ☐ pre-dr snapshot
- ☐ Optional multi-node either done or deferred
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "capstone: feature freeze readiness"
# tag after this commit if clean
```

Write three personal gotchas before continuing.

Next

→ [Disaster recovery](#) — wipe and rebuild from flake + keys.

Disaster recovery notes

Disaster recovery notes

Goal: **Wipe** the lab VM/disk and **rebuild** to a working Capstone A host using **only** git/flake + offline keys + your docs—time the recovery and fix documentation gaps that the wipe reveals.

Warning

This chapter **destroys** the lab system disk by design. Confirm target identity three times. Do **not** wipe a daily driver, NAS, or the disk holding your only key copies. Capstone freeze tag + **NixOS 26.05**-aligned installer when possible.

Why this chapter exists

Syllabus completeness: *prove rebuildability*. “It worked once” is not NixOS expertise. DR is the exam for host, secrets, deploy, images, and docs—under stress.

Teams that skip wipe drills ship folklore. You will find the missing step that only lived in your head.

Theory 1 — What “from flake + keys alone” means

Allowed inputs	Disallowed crutches
Git remote / USB bundle of repo	Old disk “just mount and copy /etc”
flake.lock pins	Random unpinned <code>nix-env</code> debris
Offline age/sops private keys	Re-typing secrets from memory into git
Installer ISO / nixos-anywhere / disko	Undocumented manual partitioning folklore
Your docs/DR-PLAN.md + RUNBOOK	Chat scrollbar
Freeze tag (<code>capstone-a-freeze</code>)	“Whatever was on main yesterday”

Ideal proof: new empty disk → installer → disko or manual partitions per docs → install flake → decrypt secrets → deploy/services healthy.

Theory 2 — Recovery paths (pick what you documented)

Path A — Interactive installer + flake

1. Boot NixOS installer ISO (**26.05**-aligned if possible).
2. Network + partition (disko or manual per DR-PLAN).
3. `nixos-install --flake /path/to/flake#lab` (or documented equivalent).
4. Reboot; place keys if not on disk yet.
5. Switch/deploy; health probes.

Path B — nixos-anywhere / remote install

Follow your ops docs; still need keys + freeze tag.

Path C — Image rehydrate

If Capstone ships generators/ISO product path, use only if documented as DR path.

Theory 3 — Timing and RTO

Record wall clock:

Phase	Start	End	Minutes
Confirm target + backups of keys			
Wipe / reinstall base			
First flake build/substitute			
Secrets decrypt + services up			
Health green			
Total			

Lab RTO target: pick a number (e.g. < 2–4 hours for solo lab) and note whether you met it.

Theory 4 — What wipe proves / does not prove

Proves	Does not prove
Docs completeness	Multi-region HA
Key custody	Perfect security
Pin/lock sufficiency	Infinite disk
Deploy path after install	Capstone B

Theory 5 — Data vs system

Layer	DR handling
System closure	Rebuild from flake
Secrets private keys	Offline restore
App state (DB)	Restore from backups chapter artifacts if any
Machine-id / SSH host keys	May change; update sops recipients if needed

If host SSH key is a sops recipient, **re-encrypt** or use dedicated age key that you restore from offline.

Concrete DR checklist

Pre-wipe (mandatory)

- ☐ Freeze tag exists and is the intended recover point
- ☐ Offline keys verified readable
- ☐ Repo pushed / USB clone available
- ☐ DR-PLAN printed or on second device
- ☐ Target disk identity confirmed **three times**
- ☐ Snapshot optional safety net (know if you will refuse to use it)
- ☐ Second machine for docs/timer

Post-rebuild health

- ☐ `hostname` correct
 - ☐ SSH works
 - ☐ `systemctl --failed` clean
 - ☐ Secret path exists; app active
 - ☐ Proxy front door probe
 - ☐ Deploy path still works from admin
-

Lab 0 — Confirm freeze and keys

```
git rev-parse capstone-a-freeze
# open offline key medium
# read docs/DR-PLAN.md on a second device
mkdir -p ~/lab/nixos/capstone/dr
```

Lab 1 — Execute wipe per DR-PLAN

Follow **your** plan. Example shape only:

```
# ON INSTALLER - example placeholders
# lsblk    # confirm disk
# diskpart or manual partition
# nixos-install --flake github:you/capstone?ref=capstone-a-freeze#lab
```

Log every deviation from DR-PLAN in `~/lab/nixos/capstone/dr/DEVIATIONS.md`.

Lab 2 — Time the recovery

Fill the timing table. Be honest about waiting on downloads.

Lab 3 — Secrets rehydrate

```
# restore age key to documented path if needed
# rebuild/switch
systemctl is-active myapp.service
```

If SSH host key recipients broke sops, document fix in SECRETS.md and commit on a **post-DR** branch carefully.

Lab 4 — Health + deploy proof

```
curl -fsS http://127.0.0.1/ || true
# from admin:
deploy .#lab 2>&1 | tee ~/lab/nixos/capstone/dr/redeploy.log
```

Lab 5 — Fix docs (the real deliverable)

Every DEVIATIONS item becomes a DR-PLAN/RUNBOOK patch. Commit doc fixes.

```
git checkout -b capstone/dr-doc-fixes
# edit docs
git commit -m "docs: fix DR gaps found during wipe rebuild"
```

Lab 6 — Optional: refuse snapshot

If you took pre-dr snapshot, either:

- **Hard mode:** do not restore it; live with rebuilt system
- **Teaching mode:** note that snapshot was available but unused

Document which mode you chose.

Common gotchas

Mistake	Fix
Wiped wrong disk	Triple-check; hypervisor snapshot on host only
Keys only on wiped disk	Offline first—always
Installer channel freeze pin	Prefer flake install with lock
sops fails after new host keys	Dedicated age key or re-encrypt
“Fixed” by mounting old root	Invalidates proof; redo
No timing log	Fill table now

Checkpoint

- ☐ Wipe executed on correct target
- ☐ Rebuild to healthy Capstone A
- ☐ Timing table filled
- ☐ DEVIATIONS → doc commits
- ☐ CAPSTONE-A DR row ticked with time
- ☐ Three personal gotchas

Commit

```
git add docs/  
git commit -m "docs: disaster recovery proof and gap fixes"
```

Write three personal gotchas before continuing.

Next

→ [Rollback and tests](#) — generation rollback + full suite green.

Rollback and tests

Rollback and tests

Goal: Prove **generation rollback** works after a bad change, run the **full** test/CI suite green, and finalize the **troubleshooting checklist** that ties playbooks to Capstone reality (post-DR host on NixOS 26.05).

i Note

Disaster recovery proved rebuild from zero. This chapter proves you need not always go to zero: generations + tests + a crisp checklist cover the common middle ground. Do the **real** break—theater fails the exercise.

Why this chapter exists

Most production incidents are not “disk gone.” They are “bad deploy an hour ago.” Without rollback muscle and a short checklist, you re-image for a typo. Capstone A is incomplete until both DR *and* rollback are proven with logs.

Theory 1 — Rollback layers

Layer	When to use	How
Bootloader generation	Unbootable / broken after reboot	Select previous generation at boot
Runtime rollback	System boots but config bad	<code>nixos-rebuild switch --rollback</code> or activate previous profile
Git revert	Want source truth back	<code>git revert /</code> deploy freeze tag
Deploy tool	Remote	Redeploy known-good attr/tag
Full DR	Disk/keys/corruption	Disaster recovery path

Capstone must document **which** you try first for remote lab (console vs SSH).

Commands (lab)

```
# list generations
sudo nix-env -p /nix/var/nix/profiles/system --list-generations
ls -la /nix/var/nix/profiles/system*

# rollback switch
sudo nixos-rebuild switch --rollback

# boot previous (if needed)
# use bootloader menu; or:
# sudo /nix/var/nix/profiles/system-<n>-link/bin/switch-to-configuration switch
```

Theory 2 — Bad change protocol

1. Record current generation number.
2. Introduce **one** deliberate bad change (firewall locks SSH only with console, or break proxy upstream).
3. Deploy/switch.
4. Observe failure with checklist.
5. Rollback.
6. Confirm health.
7. Revert git if needed so main is clean.

Good bad-change	Bad bad-change
Wrong proxy upstream	<code>rm -rf /</code>
Typo in unit ExecStart	Disk wipe
Assertion-triggering option	Destroying offline keys

Theory 3 — Full suite definition

Layer	Command
Local checks	<code>nix flake check -L</code> or explicit check attrs
Toplevel	<code>nix build</code> <code>.#nixosConfigurations.lab.config.system.build.toplevel</code> <code>-L</code>
CI	Green workflow on post-DR commit
Manual probes	RUNBOOK health section

All must be green **after** rollback drill.

Theory 4 — Troubleshooting checklist (finalize)

One page docs/TROUBLESHOOT.md (or RUNBOOK section):

1. Console access?
2. `systemctl --failed`
3. `journalctl -p err -b`
4. Generation / rollback?
5. Eval flake?
6. Secrets path permissions?
7. Firewall / `ss -lnpt`?
8. Playbook index?

Link PL entries from internals troubleshooting.

Theory 5 — State vs generation (again)

Rollback restores **system configuration**, not necessarily:

- Database contents
- Accidental deletes under `/var`
- Lost age keys

If the bad change was data, use backups—not only generations.

Concrete checklists

Rollback drill

- ☐ Generation number before break recorded
- ☐ Bad change applied and failed health
- ☐ Rollback restored health
- ☐ Log saved
- ☐ Git left clean (or intentional fix commit)

Suite

- ☐ flake check / tests green
 - ☐ CI green
 - ☐ Manual probes green
 - ☐ CAPSTONE-A rollback + tests rows ticked
-

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/capstone/rollback
cd /path/to/capstone-flake
# confirm post-DR host healthy
```

Lab 1 — Record baseline generation

```
sudo nix-env -p /nix/var/nix/profiles/system --list-generations \
  | tee ~/lab/nixos/capstone/rollback/generations-before.txt
readlink -f /run/current-system | tee ~/lab/nixos/capstone/rollback/current-system.txt
```

Lab 2 — Intentional break

Example (adjust to your stack):

```
# temporarily wrong upstream
# roles.proxy.upstream = "http://127.0.0.1:1";

sudo nixos-rebuild switch --flake .#lab 2>&1 | tee break.log
curl -fsS http://127.0.0.1/ || echo "expected fail"
```

Lab 3 — Rollback

```
sudo nixos-rebuild switch --rollback 2>&1 | tee ~/lab/nixos/capstone/rollback/rollback.log
# health
curl -fsS http://127.0.0.1/ | head
systemctl --failed
```

Acceptance: health restored without full reinstall.

Lab 4 — Git hygiene

```
git checkout -- path/to/broken.nix # or revert commit
# ensure freeze tag still meaningful
```

Lab 5 — Full suite green

```
nix flake check -L 2>&1 | tee ~/lab/nixos/capstone/rollback/check.log
nix build .#nixosConfigurations.lab.config.system.build.toplevel -L \
  2>&1 | tee ~/lab/nixos/capstone/rollback/toplevel.log
```

Confirm CI green if remote.

Lab 6 — Finalize troubleshooting checklist

Write `docs/TROUBLESHOOT.md`; link playbooks; tick CAPSTONE-A.

Lab 7 — Bootloader path (optional)

If safe, reboot and select previous generation once. Document for remote hosts with console.

Common gotchas

Mistake	Fix
Theater rollback without real break	Do real break
Rollback then leave bad git committed	Clean source
Expecting DB undo from generation	Backups
Skipping CI after DR	Run suite
No generation listed	Profile permissions; use sudo

Checkpoint

- ☐ Rollback log with before/after health
- ☐ Full suite green
- ☐ TROUBLESHOOT checklist finalized
- ☐ CAPSTONE-A complete for rollback+tests
- ☐ Three personal gotchas

Commit

```
git add .  
git commit -m "capstone: rollback drill and suite green"
```

Write three personal gotchas before continuing.

Next

→ [Retrospective](#) — teach plan and roadmap without rewrite thrash.

Retrospective

Retrospective

Goal: Close Capstone with an honest retrospective, a **teach plan** for a friend, and a personal roadmap (next nixpkgs release, nixpkgs contribution, optional Capstone B)—without starting a new rewrite today.

i Note

This chapter is reflection and direction, not another feature. Protect the freeze tag, keys, and runbook—they outrank any blog post.

Why this chapter exists

Expertise compounds when you **teach** and **schedule** the next loop. Without a retrospective, months of labs become a blur. Without a roadmap, Capstone B starts as thrash. Without a teach plan, you never test whether your mental model transfers.

Theory 1 — What you should now be able to do

By Capstone A completion you can:

Skill	Book roots
Explain store, generations, rollback	Concepts, host, internals, DR/rollback
Own a flake NixOS host	NixOS host + layout
Compose modules with types/assertions	Host + internals modules
Manage user or service identity	Host + HM
Keep secrets out of the store	Services + Capstone implement
Front door proxy pattern	Services + Capstone
Deploy remotely (deploy-rs/Colmena)	Ops + Capstone harden
Test with nixosTest	Ops + test mastery + Capstone
CI + cache discipline	Packaging/ops + Capstone
GC/store hygiene	Ops
Backup/restore state	Ops backups
Read release notes / plan upgrades	Internals

Skill	Book roots
Rebuild from zero	Capstone DR
Debug with a playbook	Internals troubleshoot + Capstone
Steelman alternatives	Comparative landscape

Mark each: **solid** / **shaky** / **missing**. Shaky items feed the roadmap.

Theory 2 — Retrospective structure (blameless)

Section	Prompt
Goals	What Capstone A required
Done	What you actually shipped (with tag SHA)
Waived	What you consciously skipped
Surprises	What docs/tests/DR revealed
Pain	Top 3 time sinks
Pride	Top 3 artifacts
Would change	Process, not fantasy rewrites
Thanks	Upstream/docs/people

Tone: operator journal, not marketing.

Theory 3 — Teach plan (1 session)

Design a **half-day** or **1-day** session for a friend who knows Linux but not Nix:

Block	Minutes	Activity
Why Nix exists	20	Store path demo
Rebuild + rollback	40	Live lab guest
Flake tour	30	Your Capstone tree
Secrets story	20	Paths not strings
Q&A	30	Their context

Deliverable: docs/TEACH-PLAN.md with commands that work on **your** freeze tag.

Theory 4 — Roadmap options (pick few)

Option	When it makes sense
Track 26.11 upgrade with your runbook	After notes ship; lab first
nixpkgs PR (package or module fix)	You hit a real bug/gap
Capstone B multi-node	Hardware + freeze honesty
devenv/Snowfall adopt	Only if COMPARE said yes
Secure Boot / Lanzaboote	Hardware allows
Write public post	After teach plan works once

Time-bound items (“by date X”) beat eternal backlogs.

Theory 5 — What not to do today

Do not	Why
Rewrite flake layout	Freeze protection
Enable CA on Capstone	Internals decision
Add five services	Scope creep
Delete DR logs	Evidence
Un-tick Capstone for aesthetics	Honesty

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/capstone/retro
cd /path/to/capstone-flake
```

Lab 1 — Skills matrix

Fill docs/RETRO.md skills table (solid/shaky/missing) from Theory 1.

Lab 2 — Full retrospective writeup

Write sections from Theory 2. Include:

- Freeze tag SHA
- DR total minutes
- Rollback proof one-liner
- Top waiver

Acceptance: 1 page honest prose + tables.

Lab 3 — Teach plan

Write docs/TEACH-PLAN.md. Dry-run the commands yourself once more on lab.

Lab 4 — Roadmap

docs/ROADMAP.md: 3–7 items max, each with trigger date or criterion. Link POST-FREEZE.md ideas selectively.

Lab 5 — Capstone A final sign-off

```
## Sign-off
- Capstone A: complete / complete-with-waivers
- Tag: capstone-a-freeze @ <sha>
- Date:
- Operator:
```

Paste into CAPSTONE-A.md.

Lab 6 — Celebrate without thrash

Optional: export PDF of docs, or demo for a friend. **Do not** open a refactor branch “while it’s fresh.”

Common gotchas

Mistake	Fix
Retro as victory lap only	Include waivers and pain
Teach plan with broken commands	Dry-run
Infinite roadmap	Cap at 7 items
Starting Capstone B tonight	Schedule
Deleting evidence logs	Keep ~/lab/nixos/capstone/**

Checkpoint

- ☐ docs/RETRO.md complete
- ☐ Teach plan dry-runnable
- ☐ Roadmap time-bounded
- ☐ Capstone A sign-off
- ☐ Freeze/docs/keys protected
- ☐ Three personal gotchas

Commit

```
git add docs/  
git commit -m "capstone: retrospective teach plan and roadmap"
```

Write three personal gotchas—and stop building for a day.

Next

Return to earlier parts as reference, or explore [Projects](#) for packaging and distro-branding depth. Capstone A, once signed off, is a **stable base**—not a perpetual construction site.

Projects

Projects index

Projects: packaging and your own distro

Hands-on projects that turn this book’s theory into **store paths you own**. Prefer a **disposable VM** (or a dedicated partition), flakes-first workflows, and commits after each green checkpoint.

Baseline	Value
Nix	2.34 family (<code>nix --version</code>)
NixOS / nixpkgs	26.05 (<code>nixos-26.05</code> flake input)
Style	Flakes-first; modern <code>nix</code> CLI
Safety	Lab machines only; no production secrets in trees

These projects sit **after** (or alongside late stages of) the main syllabus. They do not replace the `stdenv` phases chapter-56 packaging theory—they force you to **ship** small packages and then a branded ISO product.

How to use this part

Learning loop

```
read goal → declare flake/module → rebuild/build  
→ observe store / VM → fix → commit slice → next project
```

1. **Read the goal and constraints** before opening an editor. Know what “done” means.
2. **Type configs yourself.** Copy-paste once to unstick, then retype the critical attrs so they stick.
3. **Break one thing on purpose** (wrong hash, missing `runtimeInputs`, overlay not applied). Roll back with generations or `git checkout`.
4. **Commit a working flake slice** before starting the next project. Tag optional: `project-01-green`.
5. **Write a short lab note** (even five bullets). Future-you will forget which hash field mattered.

Lab environment checklist

Check	Why
Flakes + nix-command enabled	All project flakes assume modern CLI
20–40 GiB free on <code>/nix</code> volume	ISO builds and language deps are fat
KVM / nested virt available	Project 6–7 boot tests
Git installed	Lockfiles and project slices
No production credentials in the tree	Habits transfer to real fleets

```
nix --version
# expect something in the 2.34.x line (or newer compatible)

# optional: isolate lab work
mkdir -p ~/lab/nixos-projects
cd ~/lab/nixos-projects
```

How chapters are structured

Every project chapter follows roughly the same skeleton:

Section	Purpose
Goal	One-sentence outcome
Why	What skill this forces
Prerequisites	Book stages / prior projects
Theory	Concepts you must be able to explain
Lab tree	Suggested layout
Step-by-step	Commands and Nix you type
Acceptance criteria	Binary pass/fail
Pitfalls	Common failure modes
Stretch goals	Optional depth after green

Project path

Track A — Derivations & packaging

Build confidence from the smallest honest package up to language builders and overlays.

#	Chapter	Outcome
1	Hello CLI (<code>stdenv</code>)	<code>nix</code> run a shell/C tool you packaged

#	Chapter	Outcome
2	Script tool + wrap	Declared runtime deps (jq, ...) via wrappers
3	Language package	<code>buildGoModule</code> or <code>buildPythonApplication</code> end-to-end
4	Overlay: pin / patch	Change nixpkgs locally without a hard fork

Track B — Custom NixOS-based distro

Name a product, brand it, build an ISO, ship a multi-edition flake.

#	Chapter	Outcome
5	Distro identity & branding	<code>PRODUCT.md</code> + identity module + assets
6	ISO + themes	Bootable installer/live image with visible brand
7	Complete product flake	Editions, defaults, install story, <code>DEMO.md</code>

Sample assets (lab-ready): [branding/](#) — `logo.svg` (CC0), `os-release` fragment example, Plymouth notes. Copy into your distro flake; **do not** reuse official NixOS marks as your product logo.

Track A (packaging)

01 hello CLI
 02 script + wrap
 03 language builder
 04 overlay pin/patch

Track B (distro product)

05 identity + artwork
 06 ISO + themes
 07 full product flake

Suggested order

- **Linear:** 1 → 2 → 3 → 4 → 5 → 6 → 7 (recommended first pass).
- **Packaging only:** stop after 4 if you only need nixpkgs craft.
- **Distro-first:** you *can* start at 5 if you already completed Stage V packaging days—but Projects 1–2 still teach \$out muscle memory that ISO work assumes.
- **Parallel with syllabus:** Project 1 after the stdenv phases chapter; Project 4 after the overlays chapter + 49; Project 6 after the images/generators chapter.

Time and difficulty (rough)

Project	Lab time	Difficulty	Heavy disk?
1 Hello CLI	1–2 h	Easy	No
2 Script tool	1–3 h	Easy–medium	No
3 Language pkg	2–5 h	Medium	Moderate (Go/Python deps)
4 Overlay	2–4 h	Medium	Low
5 Identity	2–4 h	Medium	Low
6 ISO + themes	3–8 h	Hard	Yes (multi-GB)
7 Product	4–12 h	Hard	Yes

ISO builds can pull large closures. Plan cache space and coffee accordingly.

Repository hygiene (all projects)

```
good project slice
flake.nix
flake.lock          # committed
pkg.nix / modules/ # small, reviewable
NOTES.md or REPORT.md
no secrets, no /etc copies, no private keys
```

Rule	Reason
Commit <code>flake.lock</code>	Reproducible classmate / future rebuilds
Prefer <code>nixos-26.05</code> over <code>nixos-unstable</code> for labs	Stable teaching target for this book
One concern per commit	Easy bisect when Project 6 ISO fails
Document the command that worked	ISO flags and QEMU lines rot in memory

```
# habit after each green project
git add -A
git status    # re-check: no secrets
git commit -m "project-NN: green checkpoint"
```

Cross-links into the main book

When you are stuck on...	Revisit
Phases, \$out, hooks	the stdenv phases chapter (stdenv phases)
Fetchers / FOD hashes	the fetchers/FOD chapter
Go / Python ecosystems	the language ecosystems A chapter–48
Patches & overrides	the patching and overrides chapter
Overlays (light)	the overlays chapter
Images / generators	the images/generators chapter
Module options	the modules as consumer chapter–19, 72
Capstone polish	the capstone scope chapter–90

Projects are **deliberate practice**. Syllabus days teach; these chapters demand a demoable artifact.

Done means (global bar)

You are “done” with this part when **all** of the following hold:

- ☐ You can explain every **store path** your Project 1–3 packages produce (at least one sentence each).
- ☐ Every lab flake has a **committed flake.lock**.
- ☐ **No secrets** in any project repo (tokens, age keys, Wi-Fi PSKs).
- ☐ Project 4 overlay change is **observable** (`nix run` / version / patched string).
- ☐ Distro ISO (Project 6) **boots in a VM** and shows **your** branding on 2 surfaces.
- ☐ Project 7 README explains build → boot → install/upgrade in your own words.

Trademark and ethics (read once)

“NixOS” is a trademark of the **NixOS Foundation**. Building a learning derivative is encouraged; **impersonating** official media is not.

- Choose a **distinct product name** (e.g. YararaOS, MyLabOS).

- Do not ship official snowflake artwork as *your* logo.
 - When docs mention upstream, say “NixOS-based” honestly.
 - See [NixOS community / branding guidance](#) when publishing publicly.
-

Quick start: create the lab root

```
mkdir -p ~/lab/nixos-projects
cd ~/lab/nixos-projects
git init
```

```
# after Project 1 skeleton exists:
# nix flake new hello-cli    # or hand-write flake.nix as in the chapter
```

Work **one directory per project** (or one monorepo with subdirs). Avoid mixing ISO builds into the tiny hello-cli tree—different GC pressure and mental models.

When ready, open [Project 1 — Hello CLI](#).

Project 1: Hello CLI derivation

Project 1 — Package a tiny CLI with `stdenv.mkDerivation`

Goal: Write a reproducible derivation for a small shell or C “hello” tool, expose it from a flake, and run it via `nix build` / `nix run`.

Field	Value
Baseline	Nix 2.34 , nixpkgs nixos-26.05
Time	~1–2 hours
Depends on	the derivations chapter–8 (derivations/flake), the stdenv phases chapter (phases) helpful
Artifact	<code>packages.<system>.default</code> + working apps optional

Why this project exists

If you only install packages from nixpkgs, you never own the **build graph**. This is the smallest honest package: no network in the build, no language ecosystem, no overlay tricks—just **src** → **\$out/bin**.

After this project you should be able to say, without hand-waving:

- What lands in `$out` and why the store path hash changed when you edited the script.
- Why `callPackage` injects `stdenv` and `lib`.
- Why `nix path-info -r` lists more than one path (the **closure**).

Prerequisites

- Flakes enabled (`experimental-features = nix-command flakes`).
- Comfortable with `nix build`, `nix run`, `nix flake metadata`.

- Optional: a C compiler via stdenv (already in the sandbox for C variant).

Theory

Derivation vs package vs app

Term	Meaning in this lab
Derivation	Build plan (inputs + builder) realized into a store path
Package	Usually the derivation's primary output (\$out)
Flake packages	Attrset of packages per system
Flake apps	Named runnable (type = "app" + program absolute path)

Phases you will skip on purpose

Shell-only packaging often sets:

```
dontConfigure = true;  
dontBuild = true;
```

That is honest: there is nothing to `./configure` or `make`. You still run **installPhase** (and hooks). Prefer `runHook preInstall / postInstall` so later overlays and stdenv hooks compose.

\$out is the contract

Everything a dependent might need must appear under `$out` (here: `$out/bin/hello-cli-lab`). Scripts that assume files next to the source tree **outside** `$out` will break for users of your package.

callPackage dependency injection

```
pkgs.callPackage ./pkg.nix { }
```

opens `pkg.nix` as a function `{ stdenv, lib }:` ... and fills arguments from `pkgs`. You rarely pass `stdenv` by hand.

Purity / sandbox

The build sandbox blocks network by default. Project 1 must **not** need `fetchurl` if `src = ../..`. That keeps the first win fast and debuggable.

Lab tree

```
~/lab/nixos-projects/hello-cli/  
flake.nix  
flake.lock  
pkg.nix  
hello.sh      # primary path in this chapter  
hello.c       # optional stretch  
NOTES.md      # your one-paragraph phase note
```

```
mkdir -p ~/lab/nixos-projects/hello-cli  
cd ~/lab/nixos-projects/hello-cli  
git init
```

Step 1 — Source: shell tool

hello.sh:

```
#!/usr/bin/env bash  
set -euo pipefail  
echo "hello from $(basename "$0") - Nix derivation lab"
```

Make it executable in the **repo** for local smoke tests (the install step will set modes in the store either way):

```
chmod +x hello.sh  
./hello.sh
```

i Note

A working script on your PATH is **not** a Nix package yet. The derivation must install a copy into \$out/bin.

Step 2 — Derivation

pkg.nix:

```

{ stdenv, lib }:

stdenv.mkDerivation {
  pname = "hello-cli-lab";
  version = "0.1.0";

  src = ./.;

  dontConfigure = true;
  dontBuild = true;

  installPhase = ''
    runHook preInstall
    mkdir -p $out/bin
    install -m755 hello.sh $out/bin/hello-cli-lab
    runHook postInstall
  '';

  meta = with lib; {
    description = "Tiny lab CLI packaged with stdenv";
    license = licenses.mit;
    platforms = platforms.all;
    mainProgram = "hello-cli-lab";
  };
}

```

Optional: clean the source filter

Without filtering, dirty files (`.git`, `result`, editor swap) can change hashes or bloat `src`. Prefer:

```
src = lib.cleanSource ./.;
```

or a tighter `lib.fileset` / `builtins.path` filter once you outgrow `cleanSource`.

Step 3 — Flake

flake.nix:

```

{
  description = "hello-cli-lab - Project 1";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
}

```



```

outputs = { self, nixpkgs }:
  let
    systems = [
      "x86_64-linux"
      "aarch64-linux"
      "aarch64-darwin"
      "x86_64-darwin"
    ];
    forAll = nixpkgs.lib.genAttrs systems;
  in {
    packages = forAll (system:
      let
        pkgs = nixpkgs.legacyPackages.${system};
      in {
        default = pkgs.callPackage ./pkg.nix { };
        hello-cli-lab = pkgs.callPackage ./pkg.nix { };
      });

    apps = forAll (system: {
      default = {
        type = "app";
        program = "${self.packages.${system}.default}/bin/hello-cli-lab";
      };
    });

    checks = forAll (system: {
      hello-runs = nixpkgs.legacyPackages.${system}.runCommand "hello-cli-check" {
        nativeBuildInputs = [ self.packages.${system}.default ];
      } ''
        hello-cli-lab | grep -q "Nix derivation lab"
        touch $out
      '';
    });
  };
}

```

Tip

`meta.mainProgram` helps `nix run` discover the binary even without a custom `apps` output on newer `nixpkgs` helpers. Keeping an explicit `apps.default` is still clear for labs.

Step 4 — Build, run, inspect

```
nix flake lock
nix build -L
./result/bin/hello-cli-lab

nix run
nix run .#hello-cli-lab

# closure: what else is needed to run this?
nix path-info -rS ./result | tail -n 20
nix path-info -Sh ./result

# optional check output
nix build .#checks.${(nix eval --raw --impure --expr builtins.currentSystem).hello-runs}
```

Record in NOTES.md:

1. Full store path of ./result (resolve symlink).
2. Why more than one path appears in -r.
3. Phases you skipped and why.

```
readlink -f result
nix-store -q --references "${(readlink -f result)}
```

Step 5 — C variant (stretch, same flake)

Optional: hello.c + buildPhase with `$CC hello.c -o hello-cli-c`, install to `$out/bin`, expose as `packages.*.hello-c`. Compare `nix path-info -Sh` vs the shell package (bash vs dynamic linker story).

Theory checkpoints (self-test)

Topic	You should explain out loud
\$out	Install prefix for this package's primary output
pname / version	How default name is formed

Topic	You should explain out loud
<code>callPackage</code>	Dependency injection of <code>stdenv</code> / <code>lib</code>
Closure	Runtime dependency graph of store paths
Purity	Why the sandbox blocks network by default
<code>result</code>	GC root symlink from <code>nix build</code>
<code>dontBuild</code>	Skipping a phase vs empty <code>buildPhase</code>

Pitfalls

Symptom	Likely cause	Fix
<code>permission denied</code> running store binary	Mode not set in install	<code>install -m755</code> or <code>chmod +x</code> in <code>installPhase</code>
Hash changes every commit	Unfiltered <code>src = ./.</code> includes junk	<code>lib.cleanSource</code> / <code>fileset</code>
<code>nix run</code> cannot find binary	Wrong path in <code>apps</code> or missing <code>mainProgram</code>	Align program name with <code>\$out/bin/...</code>
Build sees no <code>hello.sh</code>	Wrong <code>src</code> root or file not committed	Check tree; <code>src</code> must contain the script
Darwin vs Linux surprises	<code>platforms</code> too narrow/wide	Set <code>meta.platforms</code> honestly; test the OS you claim
Infinite recursion in flake	<code>self</code> used before packages defined incorrectly	Keep <code>callPackage</code> on <code>pkgs</code> , not circular <code>self.packages</code> in same attr

Acceptance criteria

- ☐ `nix build` produces `./result/bin/hello-cli-lab` (or your chosen name).

- ☐ `nix run` prints the hello line.
 - ☐ `flake.lock` exists and is committed.
 - ☐ You can point at the **store path** and explain `$out/bin`.
 - ☐ `NOTES.md` has one paragraph: phases skipped and why.
 - ☐ `lib.cleanSource` (or equivalent) is used, **or** you documented why not.
-

Stretch goals

1. Add a trivial `installCheckPhase` / `doInstallCheck` that runs the binary.
 2. Multi-output experiment: put a README in `$out/share/doc/...` (still one output is fine).
 3. Build on a second system attribute (remote builder or second machine).
 4. Replace `installPhase` body with `install -D` one-liner and confirm hooks still matter.
 5. Add `formatter` / `nix fmt` later; not required for Project 1 green.
-

Checkpoint commit message (example)

```
project-01: package hello-cli-lab with stdenv and flake apps
```

Next: [Project 2 — Script tool with runtime deps](#).

Project 2: Script tool derivation

Project 2 — Real script tool with deps and wrap

Goal: Package a Bash (or tiny Python) script that **depends on other programs** (jq, curl, ...) so the **runtime closure is correct**—using `writeShellApplication` or `stdenv + wrapProgram`.

Field	Value
Baseline	Nix 2.34 , nixpkgs nixos-26.05
Time	~1–3 hours
Depends on	Project 1 (or equivalent <code>stdenv</code> comfort)
Artifact	Runnable tool whose references include runtime inputs

Why this project exists

Raw `install` of scripts often shebang-points at `/usr/bin/env bash` and assumes `jq` is on `PATH`. That works on your fat laptop and **fails in the sandbox, on servers, and for users who nix shell only your package**.

Nix’s rule: **runtime tools are dependencies**, not ambient hope. This project makes that rule visceral: remove `jq` from `runtimeInputs`, watch the tool fail closed, restore it, and see the store reference reappear.

Prerequisites

- Project 1 green (you know `$out/bin` and `nix run`).
- Rough idea of *build-time* vs *run-time* inputs.
- Optional reading: the `stdenv` phases chapter phases; wrappers are a packaging pattern, not a new language.

Theory

Three ways to ship shell tools

Helper	Best for	Runtime deps
<code>writeShellApplication</code>	Short tools, clear API	<code>runtimeInputs</code> → PATH wrap
<code>writeShellScriptBin</code>	One-liners / tiny glue	Manual or none
<code>stdenv.mkDerivation</code> + <code>makeWrapper</code>	Multi-file layouts, custom share/	<code>wrapProgram</code>

Prefer `writeShellApplication` for this lab unless you need a multi-file tree.

What the wrapper actually does

Inspect the built binary:

```
nix build
head -40 ./result/bin/weather-lab
```

You will typically see a shell wrapper that **prefixes** `PATH` with store paths of `jq`, `curl`, etc., then execs the real script. The script's shebang is rewritten to a store bash when using the `write*` helpers—another purity win.

Closure vs references

```
nix-store -q --references ./result      # direct refs
nix path-info -r ./result               # full closure walk
nix path-info -rS ./result | sort -nk2  # size-aware
```

If `jq` is a true runtime input, its store path appears in the graph. If you only *mention* `jq` in a comment, it will not.

Build-time vs run-time (again)

Kind	Example	In this project
<code>nativeBuildInputs</code>	<code>makeWrapper</code> , compilers	<code>stdenv</code> +wrap path
<code>buildInputs</code>	libs to link	rare for pure scripts
<code>runtimeInputs</code>	<code>jq</code> , <code>curl</code>	<code>writeShellApplication</code>
propagated*	infect dependents	avoid in this lab

Lab tree

```
~/lab/nixos-projects/script-tool/  
flake.nix  
flake.lock  
weather.nix          # writeShellApplication path  
weather-wrap.nix     # optional stdenv + wrapProgram  
weather.sh           # only if using wrap path  
NOTES.md
```

```
mkdir -p ~/lab/nixos-projects/script-tool  
cd ~/lab/nixos-projects/script-tool
```

Step 1 — Preferred path: writeShellApplication

weather.nix:

```
{ writeShellApplication, jq, curl }:  
  
writeShellApplication {  
  name = "weather-lab";  
  runtimeInputs = [ jq curl ];  
  text = ''  
    set -euo pipefail  
  
    usage() {  
      echo "usage: weather-lab [--live URL]" >&2  
      echo "  default: offline demo JSON (CI-friendly)" >&2  
    }  
  
    if [[ "'${1:-}" == "-h" || "'${1:-}" == "--help" ]]; then  
      usage  
      exit 0  
    fi  
  
    # Default: no network required for reproducible demos / CI.  
    if [[ "'${1:-}" == "--live" ]]; then  
      url="'${2:-https://httpbin.org/json}'"  
      curl -fsSL "$url" | jq -c .  
    else  
      echo '{"temp":21,"city":"lab"}' | jq -r '"\(.city): \(.temp)C"'  
    fi  
  ''
```



```
    '';  
}
```

! Important

In real Nix strings, `''${...}` escapes `${...}` so Bash—not Nix—expands the variable. When you type the file, keep that escaping for any Bash `${...}` inside `'' ... ''` multiline strings.

Flake

flake.nix:

```
{  
  description = "weather-lab - Project 2";  
  
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";  
  
  outputs = { self, nixpkgs }:  
    let  
      systems = [ "x86_64-linux" "aarch64-linux" "aarch64-darwin" "x86_64-darwin" ];  
      forAll = nixpkgs.lib.genAttrs systems;  
    in {  
      packages = forAll (system:  
        let pkgs = nixpkgs.legacyPackages.${system};  
        in {  
          default = pkgs.callPackage ./weather.nix { };  
          weather-lab = pkgs.callPackage ./weather.nix { };  
        });  
  
      apps = forAll (system: {  
        default = {  
          type = "app";  
          program = "${self.packages.${system}.default}/bin/weather-lab";  
        };  
      });  
    };  
}
```

Build and inspect

```
nix flake lock  
nix build -L  
./result/bin/weather-lab  
# expect: lab: 21C
```

```
head -30 ./result/bin/weather-lab
nix-store -q --references ./result
nix path-info -rS ./result | rg 'jq|curl|bash' || true
nix run
```

Step 2 — Fail closed (required demo)

1. Edit `weather.nix`: remove `jq` from `runtimeInputs` (leave `curl` or empty).
2. Rebuild and run:

```
nix build --rebuild -L
./result/bin/weather-lab
# should fail: jq: command not found (or similar)
```

3. Restore `jq`, rebuild, confirm success.
4. In `NOTES.md`, paste **before/after** `nix-store -q --references` lines showing `jq` appear/disappear.

This single demo is the educational core of Project 2.

Step 3 — Alternative: `stdenv` + `wrapProgram`

Use when you already have a multi-file tree or want to see `makeWrapper` explicitly.

`weather.sh`:

```
#!/usr/bin/env bash
set -euo pipefail
echo '{"temp":21,"city":"lab"}' | jq -r '"\(.city): \(.temp)C"'
```

`weather-wrap.nix`:

```
{ stdenv, lib, makeWrapper, jq, curl }:
```

```
stdenv.mkDerivation {
  pname = "weather-lab";
  version = "0.1.0";
  src = lib.cleanSource ./.;
```

```

nativeBuildInputs = [ makeWrapper ];

dontConfigure = true;
dontBuild = true;

installPhase = ''
  runHook preInstall
  mkdir -p $out/bin
  cp weather.sh $out/bin/weather-lab
  chmod +x $out/bin/weather-lab
  wrapProgram $out/bin/weather-lab \
    --prefix PATH : ${lib.makeBinPath [ jq curl ]}
  runHook postInstall
'';

meta = with lib; {
  description = "weather-lab via makeWrapper";
  license = licenses.mit;
  mainProgram = "weather-lab";
  platforms = platforms.all;
};
}

# point flake default at weather-wrap.nix once, build, compare wrapper text
head -40 ./result/bin/weather-lab

```

Note: `wrapProgram` renames the original to `.weather-lab-wrapped` (or similar) and installs a wrapper—inspect both.

Step 4 — Optional live mode (network at runtime)

`--live` uses network when **you run** the tool, not when Nix **builds** it. That is allowed and normal: purity applies to the build sandbox, not to every program you ship.

Still:

- Default offline path keeps CI and classrooms network-free.
- Do not `curl` during `installPhase` without a FOD fetcher—that fights the sandbox.
- Document that live mode needs network + DNS.

```
./result/bin/weather-lab --live https://httpbin.org/json || echo "network optional"
```

Lab tasks

1. Build with `writeShellApplication` and offline JSON path.
 2. Remove `jq` from `runtimeInputs`; show runtime failure.
 3. Restore; show references include `jq`.
 4. Document store path of the wrapper (`readlink -f result + head`).
 5. (Optional) Port once to `wrapProgram`; note differences in `NOTES.md`.
-

Pitfalls

Symptom	Likely cause	Fix
Works in dev shell, fails via <code>nix run</code>	Ambient <code>PATH</code> only	Declare <code>runtimeInputs</code> / wrap
Bash <code>\${var}</code> eaten by Nix <code>curl</code> missing only on live path	Unescaped <code>\${}</code> in <code>' '</code> string Not in <code>runtimeInputs</code>	Use <code>'\${var}'</code> Declare it
Dirty rebuild loops	Outputs under <code>src</code>	<code>cleanSource</code> ; keep result out

Acceptance criteria

- ☐ `nix run` prints `lab: 21C` (or equivalent).
 - ☐ Missing-dep experiment fails closed; notes capture the lesson.
 - ☐ Closure/references include runtime tools (`jq` at minimum).
 - ☐ Wrapper inspected with `head`.
 - ☐ `flake.lock` committed; `NOTES.md` compares wrapper vs plain install.
-

Stretch goals

1. `--version` from installed metadata.
 2. Second binary in the same derivation.
 3. `checks.weather` asserting offline output.
 4. `symlinkJoin` “kit” metapackage; compare closure.
-

Theory recap

Question	Target answer
Why not <code>/usr/bin/jq</code> ?	Not in the closure; not reproducible
Who rewrites shebangs?	<code>write* helpers</code> / <code>patchShebangs</code>
Runtime network impure for Nix?	Build purity runtime policy
When <code>wrapProgram</code> ?	Custom layout, multi-bin

Next: [Project 3 — Language package](#).

Project 3: Language package derivation

Project 3 — Package a Go or Python app with language helpers

Goal: Use a **language-specific builder** (`buildGoModule` or `buildPythonApplication`) so `fetch/build/test` idioms match `nixpkgs` conventions. Finish **one** language end-to-end with real hashes (not `fakeHash` left behind).

Field	Value
Baseline	Nix 2.34 , <code>nixpkgs</code> nixos-26.05
Time	~2–5 hours
Depends on	Projects 1–2; the language ecosystems A chapter–48 helpful
Artifact	<code>packages.<system>.default</code> + <code>REPORT.md</code>

Why this project exists

`stdenv.mkDerivation` can package anything, but language ecosystems already have **conventions**: Go module vendor hashes, Python `pyproject` hooks, test integration. `nixpkgs` helpers encode those conventions so your package looks like every other good citizen in the tree.

This project teaches:

- How **fixed-output-style hashes** pin dependency graphs (`vendorHash`, cargo-like patterns, FOD `src`).
- How to read the **first failed build** that prints the expected hash.
- How language tests map onto `doCheck` / hooks.

Pick **Option A (Go)** *or* **Option B (Python)** and complete it. Doing both is excellent stretch, not required for green.

Prerequisites

- Project 1 flake layout comfort.
- Toolchain on the **host** optional for scaffolding (`go` / `python`); pure Nix builds should still work after hashes are set.
- Network **once** for module download / first hash discovery (unless you vendor offline).

Theory map

Concern	Go (<code>buildGoModule</code>)	Python (<code>buildPythonApplication</code>)
Manifest	<code>go.mod</code> / <code>go.sum</code>	<code>pyproject.toml</code> / <code>setup.cfg</code>
Dep pin	<code>vendorHash</code> or <code>vendor/</code>	Package set + optional FODs
Entry	compiled binary → <code>\$out/bin</code>	<code>console_scripts</code> / <code>scripts</code>
Tests	<code>doCheck</code> , <code>checkFlags</code>	<code>pytestCheckHook</code> , <code>unittest</code>
Common footgun	wrong <code>vendorHash</code> / proxy	missing <code>format/pyproject</code> , native deps

Why helpers beat raw `stdenv` here

Language builders set:

- Correct `nativeBuildInputs` (compilers, `python`, hooks).
- Dependency fetch phases that honor hashes.
- Install layouts that match ecosystem expectations.

You still need to understand `stdenv` phases—the helper **fills** them.

Hash discipline

Field	Meaning
<code>lib.fakeHash</code> / <code>pkgs.lib.fakeSha256</code>	Placeholder that must be replaced
Hash in build error	Authoritative for that input content
Changing <code>src</code> or deps	Must recompute related hashes

Never commit a known-fake hash as “done.”

Lab tree (shared idea)

```
~/lab/nixos-projects/lang-pkg/  
flake.nix  
flake.lock  
pkg.nix  
REPORT.md  
# Go:  
go.mod go.sum main.go  
# or Python:  
pyproject.toml hi_py/__init__.py hi_py/__main__.py  
  
mkdir -p ~/lab/nixos-projects/lang-pkg  
cd ~/lab/nixos-projects/lang-pkg
```

Option A — Go (buildGoModule)

A1. Scaffold module

```
go mod init example.com/hi
```

main.go:

```
package main  
  
import "fmt"  
  
func main() {  
    fmt.Println("hi from buildGoModule")  
}
```

Optional dependency so `vendorHash` is non-trivial (recommended):

```
package main  
  
import (  
    "fmt"  
    "github.com/google/uuid"  
)  
  
func main() {  
    fmt.Println("hi from buildGoModule")  
}
```

```
    fmt.Println(uuid.NewString())
}
```

```
go get github.com/google/uuid@v1.6.0
go mod tidy
```

A2. Package expression

pkg.nix:

```
{ buildGoModule, lib }:

buildGoModule {
  pname = "hi";
  version = "0.1.0";
  src = lib.cleanSource ./.;

  # First build: set to lib.fakeHash and read the error.
  # After vendor/ present, some pins use vendorHash = null - follow your nixpkgs docs.
  vendorHash = lib.fakeHash;

  meta = {
    description = "Project 3 Go lab binary";
    license = lib.licenses.mit;
    mainProgram = "hi";
  };
}
```

A3. Flake

```
{
  description = "lang-pkg Go - Project 3";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      systems = [ "x86_64-linux" "aarch64-linux" "aarch64-darwin" "x86_64-darwin" ];
      forAll = nixpkgs.lib.genAttrs systems;
    in {
      packages = forAll (system:
        let pkgs = nixpkgs.legacyPackages.${system};
        in {
          default = pkgs.callPackage ./pkg.nix { };
        }
      );
    }
```

```

    });

    apps = forAll (system: {
      default = {
        type = "app";
        program = "${self.packages.${system}.default}/bin/hi";
      };
    });
  };
}

```

A4. Hash fix loop

```

nix build 2>&1 | tee /tmp/go-build.log
# Look for: specified: sha256-AAAA... got: sha256-REAL...
# Paste REAL into vendorHash

# rebuild until green
nix build -L
nix run
./result/bin/hi

```

A5. Vendor / tests (optional)

```

go mod vendor    # then set vendorHash per nixpkgs 26.05 docs (often null when vendored)

```

Light test: `main_test.go + doCheck = true` so a failing test fails the derivation.

Option B — Python (buildPythonApplication)

Layout:

```

pyproject.toml
hi_py/__init__.py
hi_py/__main__.py

```

`pyproject.toml` (setuptools; hatchling also fine if hooks match your pin):

```

[build-system]
requires = ["setuptools>=61"]

```

```
build-backend = "setuptools.build_meta"
```

```
[project]
name = "hi-py"
version = "0.1.0"
description = "Project 3 Python lab"
requires-python = ">=3.11"
dependencies = []
```

```
[project.scripts]
hi-py = "hi_py.__main__:main"
```

```
[tool.setuptools.packages.find]
include = ["hi_py*"]
```

hi_py/__main__.py:

```
def main() -> None:
    print("hi from buildPythonApplication")

if __name__ == "__main__":
    main()
```

pkg.nix:

```
{ python3Packages, lib }:

python3Packages.buildPythonApplication {
  pname = "hi-py";
  version = "0.1.0";
  src = lib.cleanSource ./.;

  pyproject = true;
  build-system = [ python3Packages.setuptools ];

  meta = {
    description = "Project 3 Python lab";
    license = lib.licenses.mit;
    mainProgram = "hi-py";
  };
}
```

⚠ Warning

Python attrs evolve (pyproject = true vs older format). **Match a simple package on your 26.05 pin** if eval drifts. Goal: helper lifecycle, not one frozen attr name.

```
nix build -L && nix run && ./result/bin/hi-py
```

Recommended: add one dep (click in pyproject.toml and dependencies = [python3Packages.click];), rebuild, confirm with `nix path-info -r ./result | rg -i click`.

Shared finish

```
nix flake show
nix path-info -rS ./result | tail
nix path-info -Sh ./result
```

REPORT.md (required)

```
# Project 3 report
## Language + helper (why)
## Hashes / pins (what each field fingerprints)
## Closure (`nix path-info -Sh`)
## One failure + fix
## Offline rebuild story after FODs populate
```

Pitfalls

Symptom	Likely cause	Fix
Hash mismatch forever	Edited <code>src</code> / <code>go.sum</code> after hash set	Re-run hash loop
Go wants network in sandbox	Bad vendor / <code>vendorHash</code>	Fix per pin docs
Python missing module at runtime	Dep not declared in Nix	Add to dependencies
<code>nix run</code> misses binary	Name <code>mainProgram</code> / scripts	Align entry names

Symptom	Likely cause	Fix
Dirty src	<code>.venv</code> , <code>result</code> , <code>__pycache__</code>	<code>cleanSource</code> + <code>gitignore</code>

Acceptance criteria

- ☐ `nix run` works for the chosen language binary.
- ☐ Hash fields are **real** (no committed `fakeHash`).
- ☐ `REPORT.md` complete (helper, hashes, closure, one failure).
- ☐ Flake exposes `packages.<system>.default`.
- ☐ Offline rebuild after FODs are populated (documented).
- ☐ `flake.lock` committed.

Stretch goals

1. Ship **both** Go and Python in one flake.
2. Real unit test under `doCheck`.
3. Rust track: `rustPlatform.buildRustPackage` + `cargoHash`.
4. Diff your `pkg.nix` against a similar nixpkgs package (three style notes).

Self-check

1. What does `vendorHash` fingerprint?
2. Why both Python `build-system` and runtime `dependencies`?
3. Where does the runnable live under `$out` for each option?

4. How would you pin a bad upstream version until a fix? ($\rightarrow$ Project 4)

Next: [Project 4 — Overlay pin and patch](#).

Project 4: Overlay pin and patch

Project 4 — Overlay: pin or patch a package

Goal: Change nixpkgs **without forking the tree**: apply a patch and/or pin behavior via an **overlay**, consume it from a flake package output and a `devShell`, and write a short comparison of overlay vs **extra flake input**.

Field	Value
Baseline	Nix 2.34 , nixpkgs nixos-26.05
Time	~2–4 hours
Depends on	the overlays chapter (overlays light), the patching and overrides chapter (patching); Projects 1–3 helpful
Artifact	Working overlay + <code>NOTES.md</code> (overlay vs input)

Why this project exists

Sooner or later you need:

- A one-line fix before upstream merges.
- A temporary pin while 26.05 carries a broken minor.
- The **same** patched tool in a `devShell` **and** a NixOS host.

Overlays modify the `pkgs` fixed-point. Done lightly, they are elegant. Done heavily, they make upgrades impossible. This project practices the light path with an observable demo (patched `hello` greeting or equivalent).

! Important

Overlays are sharp tools. Prefer **module options**, **single-site `overrideAttrs`**, or **separate flake inputs** when they suffice. Learn overlays so you are not helpless—not so every problem becomes one.

Prerequisites

- Can import `nixpkgs { overlays = [ ... ]; }` from a flake.
- Comfortable reading `overrideAttrs (old: { ... })`.
- Optional: `nix build nixpkgs#hello` works on your machine.

Theory

Overlay shape

```
final: prev: {  
  # attrs to add or replace  
}
```

Argument	Meaning
<code>prev</code>	Package set before this overlay
<code>final</code>	Fixed-point after all overlays—use when your new pkg depends on another overridden pkg

```
final: prev: {  
  hello = prev.hello.overrideAttrs (old: {  
    patches = (old.patches or [ ]) ++ [ ./patches/hello-msg.patch ];  
  });  
}
```

Use `prev.hello` for the package you are overriding. Use `final` when composing packages that should see other overlays (e.g. `final.myLib`).

Three layers of change (recap)

Mechanism	Scope	Maintenance
<code>override</code> / <code>overrideAttrs</code> at call site	One use	Lowest
Overlay	Whole <code>pkgs</code> view for flake/host	Medium
Fork / replace input	Anything	Highest

```
nixpkgs → overlay(s) → pkgs.hello (patched)
      ↑
      overrideAttrs / patches
```

Overlay vs separate flake input

Approach	When it shines	Cost
Overlay patch on 26.05	Tiny delta, same channel	Patch may fail on bump
Second input at a rev	Need many packages from another commit	Two evaluations; version skew
<code>pkgs.hello.overrideAttrs</code> only in one module	Single consumer	Easy to forget other consumers

You will document this in `NOTES.md` after Lab B.

Infinite recursion classic

```
# BAD: final.hello depending on final.hello via careless composition
final: prev: {
  hello = final.hello.overrideAttrs (_: { }); # boom
}
```

Prefer `prev` for the base package you modify.

Lab tree

```
~/lab/nixos-projects/overlay-lab/
flake.nix
flake.lock
overlay.nix
patches/
  hello-msg.patch
NOTES.md
```

```
mkdir -p ~/lab/nixos-projects/overlay-lab/patches
cd ~/lab/nixos-projects/overlay-lab
```

Step 1 — Flake wiring (overlay applied once)

flake.nix:

```
{
  description = "overlay-lab - Project 4";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      systems = [ "x86_64-linux" "aarch64-linux" "aarch64-darwin" "x86_64-darwin" ];
      forAll = nixpkgs.lib.genAttrs systems;

      pkgsFor = system:
        import nixpkgs {
          inherit system;
          overlays = [ (import ./overlay.nix) ];
        };
    in {
      packages = forAll (system: {
        default = (pkgsFor system).hello;
        hello = (pkgsFor system).hello;
      });

      devShells = forAll (system:
        let pkgs = pkgsFor system;
        in {
          default = pkgs.mkShell {
            packages = [ pkgs.hello pkgs.jq ];
          };
        });

      # Optional: prove NixOS would see the same pkgs if you import similarly
      # nixosConfigurations.lab = ... nixpkgs.overlays = [ (import ./overlay.nix) ];
    };
}
```

Tip

legacyPackages does **not** automatically apply your overlay. Construct pkgs with `import nixpkgs { overlays = ...; }` (or `nixpkgs.legacyPackages + extend`) deliberately and **reuse** that pkgs everywhere.

Step 2 — Lab A: message patch on hello

A1. Discover the string to patch

```
nix build nixpkgs/nixos-26.05#hello -o /tmp/hello-stock  
/tmp/hello-stock/bin/hello  
# note the exact greeting line on your pin
```

You need a patch against the **source** of that package version. Approaches:

1. **Proper patch file** against upstream **hello** sources (best practice).
2. **Lab-only** postPatch sed (acceptable for learning; document as lab-only).

A2. Lab-fast path: postPatch (allowed for green)

overlay.nix:

```
final: prev: {  
  hello = prev.hello.overrideAttrs (old: {  
    pname = old.pname;  
    postPatch = (old.postPatch or "") + ''  
      # Lab-only: force a recognizable greeting if source layout matches GNU hello.  
      # If this fails, switch to a real patches = [ ./patches/hello-msg.patch ];  
      if [ -f src/hello.c ]; then  
        substituteInPlace src/hello.c \  
          --replace-fail "Hello, world!" "Hello, overlay-lab!"  
      fi  
    '';  
  });  
}
```

Note

`substituteInPlace` / `--replace-fail` behavior depends on `stdenv` helpers on your pin. If flags differ, use `sed -i` carefully or a unified diff patch. The acceptance bar is **observable changed output**, not a perfect GNU patch hobby.

A3. Proper patch path (preferred write-up)

1. Unpack hello sources (from the derivation or upstream tarball matching the pin).
2. Edit the greeting.

3. `diff -u` → `patches/hello-msg.patch`.

4. Overlay:

```
final: prev: {
  hello = prev.hello.overrideAttrs (old: {
    patches = (old.patches or [ ]) ++ [ ./patches/hello-msg.patch ];
  });
}
```

A4. Prove it

```
nix build -L
./result/bin/hello
# must show your modified greeting

nix develop -c hello
# same binary lineage from overlayed pkgs

nix path-info -Sh ./result
```

If output is unchanged, the overlay was not applied or `postPatch` did not match sources—fix, do not proceed to Lab B until this is green.

Step 3 — Lab B: pin via extra input (document, implement lightly)

Sometimes you need an older/newer **package graph**, not a one-line patch.

`flake.nix` sketch (second input):

```
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  # Example only - replace <rev> with a real commit after you pick one:
  # inputs.nixpkgs-tool.url = "github:NixOS/nixpkgs/<rev>";
  # inputs.nixpkgs-tool.inputs = {}; # not always needed

  outputs = { self, nixpkgs /*, nixpkgs-tool */:
    let
      system = "x86_64-linux";
      pkgs = import nixpkgs {
        inherit system;
        overlays = [ (import ./overlay.nix) ];
      };
    in
```

```

    # pkgsTool = import nixpkgs-tool { inherit system; };
  in {
    packages.${system} = {
      default = pkgs.hello;          # patched on 26.05
      # toolFromPin = pkgsTool.curl; # whole package from another rev
    };
  };
}

```

In `NOTES.md`, answer:

1. When is a **second input** cleaner than an overlay?
2. What skew risk appears when `pkgs` and `pkgsTool` both provide libraries?
3. Which approach would you choose for (a) one-line crash fix (b) need cmake 3.x from a different commit?

You do **not** must ship a second input for green if Lab A works—but the written comparison is required.

Step 4 — Optional NixOS consumer + upgrade drill

On a VM: set `nixpkgs.overlays = [ (import ./overlay.nix) ];` and `environment.systemPackages = [ pkgs.hello ];`, rebuild, confirm the patched string.

In `NOTES.md`, write recovery if 26.05 → next stable breaks the patch: refresh patch, drop overlay if fixed upstream, switch to input pin, add `passthru.tests`.

Gotchas

Issue	Notes
Overlay not applied	<code>legacyPackages</code> without <code>extend</code> / separate <code>import</code>
Infinite recursion	<code>final.pkg</code> self-ref; use <code>prev</code>
Patch fails on bump	Comment + CI build of the package
Two <code>nixpkgs</code> copies	Missing <code>follows</code> ; closure duplication
<code>nix shell nixpkgs#hello</code> unpatched	Different package set—expected

Acceptance criteria

- ☐ Overlay changes **observable** behavior.
 - ☐ Visible via flake **packages** and **devShell**.
 - ☐ **NOTES.md**: overlay vs extra input (2 scenarios) + drop-after-upstream plan.
 - ☐ Patch file **or** labeled lab-only **postPatch**.
 - ☐ **flake.lock** committed; no infinite recursion.
-

Stretch goals

1. **passthru.tests** asserting the new string.
 2. Second package depending on patched **hello** via **final**.
 3. Wire overlay into a tiny **nixosConfigurations.demo**.
 4. **lib.composeManyExtensions** with two overlays.
-

Self-check

1. **override** vs **overrideAttrs**?
2. When is **final** required?
3. When is a second input *better* than an overlay?
4. Prove an overlay is active in 30 seconds.

Next: [Project 5 — Distro identity and branding](#).

Project 5: Distro identity and branding

Project 5 — Name your distro: identity, versioning, branding assets

Goal: Define a **product identity** for a NixOS-based derivative: codename, version scheme, artwork, and modules that surface branding (**os-release**, issue/banner, optional loader labels, shipped logo).

This is the “what is this OS called?” project—**before** ISO polish (Project 6) and product packaging (Project 7).

Field	Value
Baseline	Nix 2.34 , NixOS / nixpkgs 26.05
Time	~2–4 hours
Depends on	Stage II host flake comfort; the modules as consumer chapter–19 modules
Artifact	Distro flake slice + PRODUCT.md + branding/

Why this project exists

A “custom distro” that still says **NAME=NixOS** everywhere is only a config repo. Identity work forces you to:

- Pick a **distinct name** (trademark-safe).
- Encode version/codename policy.
- Express branding as **NixOS modules**, not ad-hoc files after install.
- Treat artwork and licenses as first-class inputs.

Projects 6–7 reuse this module graph. Do not skip **PRODUCT.md**.

! Important

Trademark: “NixOS” is a trademark of the NixOS Foundation. Use a distinct product name; say “NixOS-based” when describing heritage. Do not ship official snowflake art as *your* logo. See [community branding guidance](#).

Prerequisites

- A NixOS VM or machine you can rebuild (prefer disposable).
 - Flake-based `nixosConfigurations` experience (the flake hosts chapter+).
 - Optional: copy book sample assets from `content/99-projects/branding/`.
-

Theory

Identity surfaces

Surface	User-visible?	Typical mechanism
<code>/etc/os-release</code>	Tools, neofetch, support scripts	NixOS options / etc files
<code>/etc/issue</code>	TTY login banner	<code>environment.etc.issue</code>
System label	Boot loader generations	<code>system.nixos.label</code>
<code>/etc/<distro>/logo.png</code>	Logos, greeter hooks later	<code>environment.etc</code>
Pretty hostname motd	SSH banners	<code>users.motd</code> / <code>issue.net</code>

Not every surface must be perfect on day one. Pick **3** and make them consistent.

Version scheme for derivatives

Track **base nixpkgs** explicitly:

```
<distro-version> = <nixpkgs-release>.<distro-patch>[-label]
example: 26.05.0-lab1
```

Component	Meaning
26.05	Upstream release train you track
.0	Your product revision on that train
-lab1	Optional channel/codename tag

Record the **flake input rev** in `PRODUCT.md` when you cut a “release.”

Modules as product API

```
options.myDistro.* → config that applies branding
```

Downstream hosts set:

```
myDistro.enable = true;  
myDistro.name = "YararaOS";
```

instead of copy-pasting os-release text into every host.

os-release merge reality

NixOS already generates os-release-related content. On **26.05**, prefer **documented options** when they exist (names evolve—verify on your pin), for example patterns around distro id / pretty name if available. Overwriting `environment.etc."os-release"` can fight the module system.

Lab rule: after rebuild, `cat /etc/os-release` is the source of truth. Adjust the module until **your name** is visible without breaking activation.

Product sheet (PRODUCT.md)

Create and fill:

```
# PRODUCT.md  
  
| Field | Value |  
|-----|-----|  
| Distro name | |  
| Version | 26.05.0-lab1 |  
| Codename | |  
| Homepage | |  
| Bug tracker | |  
| Base | nixos-26.05 |  
| Default edition | minimal |  
| Trademark notes | distinct name; NixOS-based |  
| Flake input rev (optional) | |
```

Example names: YararaOS, MyLabOS, RiverNix—anything **not** “NixOS” alone.

Repo layout

```
my-distro/
  flake.nix
  flake.lock
  PRODUCT.md
  branding/
    logo.svg
    README.md          # license for assets
  modules/
    identity.nix
  hosts/
    lab.nix            # disposable VM/host
    iso.nix            # stub for Project 6
```

Copy sample branding from this book

```
# adjust path to your checkout of this monorepo
cp -r /path/to/books/NixOS/content/99-projects/branding ./branding
# remix logo.svg; keep license notes honest
```

Book kit:

File	Purpose
logo.svg	Sample mark (CC0)
os-release.fragment.example	Field checklist
plymouth-theme-notes.md	Splash strategy (Projects 6–7)
README.md	License + trademark notes

Step 1 — Identity module

modules/identity.nix:

```
{ config, lib, pkgs, ... }:
let
  cfg = config.myDistro;
in {
  options.myDistro = {
    enable = lib.mkEnableOption "distro identity defaults";

    name = lib.mkOption {
```

```

    type = lib.types.str;
    default = "MyLabOS";
    description = "Pretty distro name (distinct from NixOS trademark).";
};

id = lib.mkOption {
    type = lib.types.str;
    default = "mylabos";
    description = "Lowercase ID-like token for os-release ID=";
};

version = lib.mkOption {
    type = lib.types.str;
    default = "26.05.0-lab1";
};

codename = lib.mkOption {
    type = lib.types.str;
    default = "lab";
};

homeUrl = lib.mkOption {
    type = lib.types.str;
    default = "https://example.invalid/";
};

};

config = lib.mkIf cfg.enable {
    system.nixos.label = "${cfg.name}-${cfg.version}";

    # Validate on 26.05: prefer first-class options if present on your pin.
    # The etc approach is a lab lever-adjust when it conflicts with generators.
    environment.etc."os-release".text = lib.mkDefault ''
        NAME="${cfg.name}"
        ID=${cfg.id}
        ID_LIKE=nixos
        VERSION="${cfg.version}"
        VERSION_CODENAME=${cfg.codename}
        PRETTY_NAME="${cfg.name} ${cfg.version} (NixOS-based)"
        HOME_URL="${cfg.homeUrl}"
        DOCUMENTATION_URL="${cfg.homeUrl}docs"
        SUPPORT_URL="${cfg.homeUrl}support"
        BUG_REPORT_URL="${cfg.homeUrl}issues"
    '';

    environment.etc.issue.text = lib.mkDefault ''
        ${cfg.name} ${cfg.version} \n \l

```

```

'';

users.motd = lib.mkDefault ''
  Welcome to ${cfg.name} ${cfg.version} (${cfg.codename})
  Base: NixOS / nixpkgs 26.05 track - lab derivative
'';

environment.etc."${cfg.id}/logo.svg".source = ../branding/logo.svg;

# Optional: make support tooling discover the brand easily
environment.variables.DISTRIB_ID = cfg.id;
};
}

```

Warning

If activation or other modules fight `environment.etc."os-release"`, switch to the **documented 26.05 options** for distro naming and keep `issue` + `logo` + `system.nixos.label` as your guaranteed branding. Record what worked in `PRODUCT.md`.

Use the field checklist in `branding/os-release.fragment.example` when editing.

Step 2 — Lab host + flake

hosts/lab.nix:

```

{ config, lib, pkgs, ... }: {
  imports = [ ../modules/identity.nix ];

  myDistro.enable = true;
  myDistro.name = "MyLabOS";
  myDistro.id = "mylabos";
  myDistro.version = "26.05.0-lab1";

  networking.hostName = "mylabos-lab";

  # Minimal disposable VM assumptions - adapt to your hardware config:
  # boot.loader, fileSystems, users.users.... from your existing lab host.
}

```

flake.nix (sketch):

```

{
  description = "MyLabOS - Project 5 identity";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }: {
    nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux"; # or your arch
      modules = [
        ./hosts/lab.nix
        # ./hardware-configuration.nix # real disk/boot
      ];
    };
  };
}

nix flake lock
sudo nixos-rebuild switch --flake .#lab
# or: nixos-rebuild build-vm --flake .#lab && ./result/bin/run-*-vm

```

Step 3 — Verify identity

On the running system:

```

cat /etc/os-release
cat /etc/issue
cat /etc/motd 2>/dev/null || true
ls -la /etc/mylabos/ # or your cfg.id
hostnamectl 2>/dev/null || true

```

Capture outputs into `lab-evidence.md` or paste under `PRODUCT.md` “Evidence” section.

Optional screenshot: TTY login showing `/etc/issue`.

Step 4 — Artwork + consistency

1. Sample or custom `logo.svg`; license note in `branding/README.md`.
2. No official NixOS snowflake as *your* mark.

3. Prefer SVG; `rsvg-convert -w 256 branding/logo.svg -o branding/logo.png` if a greeter needs PNG later.
 4. Confirm **name/version** match across options defaults, `os-release`, `issue`, `system.nixos.label`, `motd`—drift multiplies in Project 6.
-

Pitfalls

Symptom	Likely cause	Fix
Still says NixOS everywhere	Module not imported / disabled	Import + enable
<code>os-release</code> reverts	Lost merge to generator	Supported options; less force
Logo missing	Bad relative path	Path relative to module file
Trademark risk	Named only “NixOS ...”	Distinct NAME / ID
Rebuild fails after clean eval	Missing boot/fs modules	Add real hardware config

Acceptance criteria

- ☐ `PRODUCT.md` complete (name, version, base, trademark note).
 - ☐ `branding/logo.svg` + license note.
 - ☐ Identity module: clean rebuild of lab host/VM.
 - ☐ `os-release` and/or `issue` show **your** name (document option path if needed).
 - ☐ Logo under `/etc/<id>/` (or equivalent).
 - ☐ `flake.lock` committed; no official NixOS marks as product logo.
-

Stretch goals

1. `myDistro.edition` option varying `motd`.
 2. Doc package installing `PRODUCT.md` under `$out/share/doc/...`
 3. `hosts/iso.nix` stub importing only identity (ready for Project 6).
-

Self-check

1. Why put 26.05 in the product version string?
2. `NAME` vs `ID` in `os-release`?
3. Why modules instead of editing files after first boot?
4. Which surfaces survive headless installs?

Next: [Project 6 — ISO and themes](#).

Project 6: Distro ISO and themes

Project 6 — Installer ISO with themes (Plymouth / greeter / console)

Goal: Produce a **bootable installer ISO** (or live image) for your branded distro using NixOS installer profiles (`isoImage`) and/or `nixos-generators`, with **visible theming**: console colors/fonts, optional Plymouth splash, greeter/wallpaper hooks for a graphical profile, and ISO volume metadata.

Field	Value
Baseline	Nix 2.34 , NixOS 26.05
Time	~3–8 hours (downloads + iteration)
Depends on	Project 5 identity module; the images/generators chapter images helpful
Artifact	<code>.iso</code> that boots in QEMU + branding checklist

Why this project exists

A distro people can **boot** beats a distro that only exists as `nixosConfigurations.laptop`. ISO generation connects:

- Module graph (identity + themes + installer profile)
- Image format (`system.build.isoImage`)
- Runtime verification (QEMU)

You will also learn why **theme options differ by display manager** and why console branding is the reliable baseline.

Prerequisites

- Project 5 green (myDistro identity, logo, PRODUCT.md).
- Disk space: plan for multi-GB store growth.
- KVM recommended:

```
ls /dev/kvm && echo "KVM ok"
```

- Patience: first ISO eval/build is the slowest.

Theory / architecture

```
flake.nix
  nixosConfigurations.iso

    installer profile (minimal or graphical)
    modules/identity.nix      (Project 5)
    modules/themes.nix       (this project)

config.system.build.isoImage

result/iso/*.iso → QEMU / VirtualBox / USB
```

Approach	Notes
modulesPath installer CD profiles nixos-generators format = "iso"	Classic; well documented Same modules, format abstraction (the images/generators chapter)
Graphical Calamares/GNOME profiles	Heavier; better greeter experiments

Theme layers (cheap → expensive)

Layer	Cost	Reliability in QEMU
Console font/colors + issue	Low	High
ISO volume label / isoName	Low	High
Plymouth stock theme	Medium	Medium (needs quiet splash path)
Custom Plymouth package	High	Medium

Layer	Cost	Reliability in QEMU
GDM/SDDM logos	Medium	Option names move across pins

Lab strategy: nail console + os-release + volume label first; add Plymouth/greeter after the ISO boots cleanly.

Lab tree (extends Project 5)

```
my-distro/
  PRODUCT.md
  flake.nix
  flake.lock
  branding/
    logo.svg
    README.md
  modules/
    identity.nix
    themes.nix
  hosts/
    lab.nix
    iso.nix
  NOTES-ISO.md          # build time, size, QEMU line
```

Step 1 — ISO host configuration

hosts/iso.nix:

```
{ config, pkgs, lib, modulesPath, ... }: {
  imports = [
    "${modulesPath}/installer/cd-dvd/installation-cd-minimal.nix"
    # Graphical experiment (much larger) - pick one profile only:
    # "${modulesPath}/installer/cd-dvd/installation-cd-graphical-calamares-gnome.nix"
    ../modules/identity.nix
    ../modules/themes.nix
  ];

  myDistro.enable = true;
  myDistro.name = "MyLabOS";
```

```

myDistro.id = "mylabos";
myDistro.version = "26.05.0-iso1";

# Live image ergonomics
networking.wireless.enable = lib.mkForce false; # avoid iwd/wpa fights on minimal ISOs if
services.openssh.enable = true;

# Installer images often define a `nixos` user - keep passwords per template docs.
# Do not bake production secrets into public ISO flakes.

# ISO metadata - confirm option names on your 26.05 pin if eval fails
isoImage.volumeID = lib.mkDefault "MYLABOS_2605";
isoImage.isoName = lib.mkDefault "mylabos-26.05-x86_64.iso";
}

```

i Note

Exact attribute names under `isoImage.*` can differ slightly by release. If eval errors, `nixos-option` / `source grep` on your pin and adjust. Volume label is a branding surface either way.

Step 2 — Themes module

`modules/themes.nix:`

```

{ config, lib, pkgs, ... }: {
  # --- Console branding (do this first) ---
  console = {
    font = "Lat2-Terminus16";
    colors = [
      # 16-color palette, hex without leading #
      "1d1e28" "ff5555" "50fa7b" "f1fa8c"
      "bd93f9" "ff79c6" "8be9fd" "f8f8f2"
      "6272a4" "ff6e6e" "69ff94" "ffffa5"
      "d6acff" "ff92df" "a4ffff" "ffffff"
    ];
  };

  # --- Plymouth (optional; enable after minimal ISO works) ---
  # boot.plymouth.enable = true;
  # boot.plymouth.theme = "spinner";
  # boot.plymouth.themePackages = [ ]; # custom later

```

```
# --- Graphical greeter hooks (only on graphical installer profiles) ---
# Option paths vary by DM + nixpkgs revision. Verify before relying on them.
# services.displayManager.sddm... / services.xserver.displayManager.gdm...
# services.xserver.displayManager.gdm.logo = ../../branding/logo.png;

environment.systemPackages = with pkgs; [
  git
  vim
  # fastfetch or neofetch - nice for DEMO screenshots
];
}
```

Read book notes: [branding/plymouth-theme-notes.md](#).

Tip

If greeter logo options moved on your pin, **do not block** the project. Ship console + os-release + volume label + logo under `/etc/<id>/`, and document greeter as stretch.

Step 3 — Flake output

```
{
  description = "MyLabOS ISO - Project 6";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }: {
    nixosConfigurations.iso = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      modules = [ ./hosts/iso.nix ];
    };

    # Optional convenience package-like output via apps/scripts in README
  };
}
```

Build

```
nix build -L .#nixosConfigurations.iso.config.system.build.isoImage
ls -lh result/iso/
```

Record in NOTES-ISO.md:

- Wall-clock time
- ISO size
- nixpkgs rev from flake.lock
- Exact build command

Alternative: nixos-generators

```
# sketch - pin nixos-generators and follows nixpkgs
# packages.x86_64-linux.iso = nixos-generators.nixosGenerate {
#   system = "x86_64-linux";
#   format = "iso";
#   modules = [ ./hosts/iso.nix ];
# };
```

Prefer **one** path for the lab so debugging stays sane.

Step 4 — Boot test (QEMU)

```
ISO=$(ls result/iso/*.iso | head -1)
qemu-system-x86_64 \
  -m 2048 \
  -cdrom "$ISO" \
  -boot d \
  -enable-kvm \
  -serial mon:stdio
```

Without KVM, drop `-enable-kvm` and expect slower boots; raise `-m` if the graphical profile OOMs.

VirtualBox / UTM

Import the ISO as an optical drive, boot order CD first. Same verification checklist.

Verification checklist (live session)

Check	Pass?	Notes
Boots to installer/live environment		
cat /etc/os-release shows product name		

Check	Pass?	Notes
<code>cat /etc/issue</code> branded		
Console palette/font applied		
Logo present under <code>/etc/<id>/</code>		
SSH or local login works		
ISO volume label visible (where shown)		
Plymouth (if enabled)		optional

```
# inside live system
cat /etc/os-release
cat /etc/issue
ls /etc/mylabos
```

Step 5 — Branding matrix

Copy into `NOTES-ISO.md` and fill status:

Surface	Status	How verified
os-release / issue		
Console colors/font		
ISO volumeID / isoName		
Logo on live FS		
Plymouth	skipped / ok	
Installer wallpaper/logo	skipped / ok	

Acceptance for Project 6: branding visible on **2 surfaces** in the live environment, ISO boots, build command documented.

Step 6 — Iterate without tears

ISO rebuilds are expensive. Iteration tips:

1. Test **identity module** on `build-vm` / normal host first (Project 5).
2. Change **one** theme knob per ISO rebuild when debugging.

3. Keep a `hosts/iso-minimal.nix` and `hosts/iso-graphical.nix` if you experiment—do not thrash a single file blindly.
4. Use binary caches; do not disable substitutes “for purity theater” on ISO day.
5. Tag git when an ISO is good: `iso-26.05.0-iso1-green`.

```
# eval-only sanity (faster than full ISO when checking option typos)
nix eval .#nixosConfigurations.iso.config.networking.hostName
nix eval .#nixosConfigurations.iso.config.myDistro.name
```

Plymouth (optional after minimal ISO is green)

```
boot.plymouth.enable = true;
boot.plymouth.theme = "spinner";
# later: themePackages + custom theme derivation; test only in a fat VM
```

Timebox custom Plymouth—failures are expensive.

Pitfalls

Symptom	Likely cause	Fix
Out of disk	<code>/nix</code> full	Free tens of GB; careful GC
QEMU hang	Wrong arch / low RAM	Match <code>system</code> ; raise <code>-m</code>
Unbranded live	Modules not on iso host	Fix <code>imports</code>
<code>isoImage.*</code> unknown	Pin difference	Grep installer modules
Graphical ISO never ends	Huge closure	Minimal profile for green

Acceptance criteria

- ☐ ISO builds (`system.build.isoImage` or generators).
- ☐ VM boots live/installer environment.

- ☐ Branding on **2** surfaces.
 - ☐ Build command, size, time in `NOTES-ISO.md`.
 - ☐ ISO version tag (`-iso1`) noted; no production secrets.
-

Stretch goals

1. Graphical installer + greeter logo.
 2. Custom Plymouth from `logo.svg`.
 3. Generators: `qcow2` **and** `iso` from same modules.
 4. Serial QEMU smoke grepping `os-release`.
-

Self-check

1. Why console theming first?
2. Which derivation path built the ISO?
3. How does Project 5 prevent ISO-only drift?
4. What do you drop first under a 30-minute CI budget?

Next: [Project 7 — Complete distro product](#).

Project 7: Complete custom distro product

Project 7 — Ship a complete NixOS-based distro flake

Goal: Assemble a **product-shaped** flake: branded identity, at least two editions (minimal + workstation **or** server), installer ISO, pinned nixpkgs **26.05**, documented install path, first-boot defaults, and a demo script—your own distro in **engineering** terms (not a legal claim on the NixOS trademark).

Field	Value
Baseline	Nix 2.34 , NixOS / nixpkgs 26.05
Time	~4–12 hours (integration)
Depends on	Projects 5–6; packaging Projects 1–4 optional but enriching
Artifact	Public-ready README + DEMO.md + multi-output flake

! Important

Trademark: “NixOS” is a trademark of the NixOS Foundation. Your derivative must use a **distinct name** and follow [NixOS branding guidelines](#) when mentioning the base. This project teaches reproducible OS assembly—not unofficial branding abuse.

Why this project exists

Projects 5–6 create identity and an ISO. Productization means:

- **Editions** as module bundles (not copy-pasted hosts).
- **Channels of truth:** flake lock + docs, not tribal memory.
- **Install + upgrade story** a stranger could follow.
- Honest boundary: what is **your** policy vs upstream NixOS.

Treat this like a tiny open-source OS product even if only you use it.

Product definition of done

#	Requirement
1	Distinct distro name + version in <code>PRODUCT.md</code>
2	<code>flake.lock</code> pins <code>nixos-26.05</code> (or documented stable choice)
3	<code>nixosConfigurations</code> for <code>iso</code> and at least one installed host profile
4	Editions: e.g. <code>minimal</code> and <code>desktop</code> or <code>server</code> module bundles
5	Branding on ISO + installed system path (Projects 5–6)
6	Defaults module: flakes-on, explicit unfree policy, nix settings
7	README: build ISO, install notes, upgrade story
8	<code>DEMO.md</code> with commands + evidence list
9	Optional: CI stub (<code>nix flake check</code> / ISO build in GA)

Target tree

```
my-distro/
  PRODUCT.md
  README.md
  DEMO.md
  LICENSE                # for *your* modules/assets
  flake.nix
  flake.lock
  branding/
    logo.svg
    README.md
  modules/
    identity.nix
    themes.nix
    defaults-nix.nix
    edition-minimal.nix
    edition-desktop.nix  # or edition-server.nix
  hosts/
    iso.nix
    demo-vm.nix
  packages/
    default.nix          # optional distro toolkit
  checks/
    default.nix          # optional
```

Merge everything you already built in Projects 5–6; refactor names for consistency.

Theory: editions vs hosts

```
modules/edition-*.nix    = product SKUs (what kind of system)
hosts/*.nix               = concrete machines / images (where it runs)
```

Layer	Example
Edition	minimal SSH appliance defaults
Host	demo-vm imports minimal + disk/boot for QEMU
Image	iso imports minimal + installer profile + themes

Do **not** put disk device paths inside edition modules.

Step 1 — Defaults module (flakes-first)

modules/defaults-nix.nix:

```
{ lib, ... }: {
  nix = {
    settings = {
      experimental-features = [ "nix-command" "flakes" ];
      # trusted-users: keep tight on real products
      # substituters / trusted-public-keys: document if you run a cache
      auto-optimize-store = lib.mkDefault true;
    };
    # gc.automatic is a product policy - enable only with thought
  };

  # Product policy: be explicit
  nixpkgs.config.allowUnfree = lib.mkDefault false;

  # Prefer flake-pinned nixpkgs over channels in docs and defaults
  # (channel-less mental model for users of your distro)
}
```

Document in README: “This product is flakes-first; classic channels are not the support path.”

Step 2 — Edition modules

Minimal

modules/edition-minimal.nix:

```
{ lib, pkgs, ... }: {
  imports = [
    ./identity.nix
    ./defaults-nix.nix
  ];

  myDistro.enable = lib.mkDefault true;

  services.openssh.enable = lib.mkDefault true;
  networking.firewall.enable = lib.mkDefault true;

  environment.systemPackages = with pkgs; [
    vim
    git
    curl
  ];

  # Users: force consumers to define real users on hosts
}
```

Desktop (or server)

modules/edition-desktop.nix:

```
{ lib, pkgs, ... }: {
  imports = [
    ./edition-minimal.nix
    ./themes.nix
  ];

  # Pick ONE desktop stack and stick to it for the lab
  services.xserver.enable = true;
  services.displayManager.gdm.enable = true; # verify attr path on 26.05
  services.desktopManager.gnome.enable = true; # or plasma6 / something you know

  networking.networkmanager.enable = true;

  environment.systemPackages = with pkgs; [
    firefox
  ];
}
```

```
}
```

Warning

Display manager option paths have moved across releases (services.xserver.displayManager vs services.displayManager, GNOME/Plasma module names). **Eval on your pin** and adjust. The product lesson is edition composition, not memorizing one GNOME attr forever.

Server alternative: skip GUI; add fail2ban posture notes, unattended-upgrade policy **or** flake-based upgrade docs only.

Step 3 — Hosts

Demo VM / installed profile

hosts/demo-vm.nix:

```
{ lib, modulesPath, ... }: {
  imports = [
    (modulesPath + "/virtualisation/qemu-vm.nix") # if using build-vm patterns
    ../modules/edition-minimal.nix
    # ../modules/edition-desktop.nix
  ];

  myDistro.enable = true;
  myDistro.name = "MyLabOS";
  myDistro.version = "26.05.0-prod1";

  networking.hostName = "mylabos-demo";

  users.users.demo = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];
    # initialPassword only for disposable lab VMs - never for public ISOs if weak
    initialPassword = "demo"; # LAB ONLY
  };

  # boot.loader + fileSystems: required for real metal; qemu-vm module helps for demos
}
```

ISO host

Keep Project 6 `hosts/iso.nix` importing identity, themes, installer profile, and **edition-minimal** (not full desktop unless you accept huge ISOs).

Step 4 — Flake product surface

```
{
  description = "MyLabOS - NixOS-based lab distro";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
    in {
      nixosConfigurations = {
        iso = nixpkgs.lib.nixosSystem {
          inherit system;
          modules = [ ./hosts/iso.nix ];
        };
        demo-vm = nixpkgs.lib.nixosSystem {
          inherit system;
          modules = [ ./hosts/demo-vm.nix ];
        };
      };

      packages.${system} = {
        # optional toolkit
        mylabos-tools = pkgs.callPackage ./packages/default.nix { };
      };

      # formatter, checks, devShells - add as you harden the product
      checks.${system}.demo-vm-eval =
        self.nixosConfigurations.demo-vm.config.system.build.toplevel;
    };
}
```

```
nix flake show
nix build -L .#nixosConfigurations.demo-vm.config.system.build.toplevel
nix build -L .#nixosConfigurations.iso.config.system.build.isoImage
nix flake check    # if checks are wired
```

Step 5 — Optional metapackage (toolkit)

packages/default.nix:

```
{ symlinkJoin, hello, git, vim }:  
  
symlinkJoin {  
  name = "mylabos-tools";  
  paths = [ hello git vim ];  
  meta.description = "Convenience toolkit metapackage for MyLabOS demos";  
}
```

Ship only if it clarifies the product; empty packages/ is fine.

Step 6 — Install story (document, then test once)

Write this in README, then execute **once** on a disposable VM disk:

1. Boot ISO (Project 6).
2. Partition (manual or Disko—advanced).
3. Install from your flake:

```
# Conceptual - exact installer flow varies by profile and version.  
# Prefer copying the flake onto the live system or using a reachable git remote.
```

```
sudo nixos-install --flake /path/to/my-distro#demo-vm
```

```
# or, if published:  
# sudo nixos-install --flake github:you/my-distro#demo-vm
```

4. Reboot into installed system.
5. Verify branding (os-release, issue) and flakes defaults.
6. Exercise upgrade path:

```
# on a machine that tracks your flake  
cd /etc/nixos # or your flake path  
nix flake update # policy: when and how you allow this
```

```
sudo nixos-rebuild switch --flake .#demo-vm
```

Document **failure modes**: wrong host attr, missing boot loader, network for first download.

Step 7 — README + DEMO.md (required)

README minimum: what it is (NixOS-based, trademark-safe), quick start, build ISO, QEMU, install, editions, upgrade policy, branding/license, upstream vs yours, limitations. Paste the **exact** commands that worked.

DEMO.md minimum:

1. Build ISO command
2. QEMU one-liner
3. Evidence list (boot, issue, **os-release**, optional fetch)
4. Upgrade: bump flake input → rebuild
5. Limitations (hardware, Secure Boot, blobs, not a Foundation release)

Run the demo once end-to-end before calling Project 7 green.

Step 8 — Channels of truth

Source	Content
flake.lock	Exact nixpkgs (and other inputs)
PRODUCT.md	Name, version, release notes
README.md	Operator docs
modules/*	Behavior
CI (optional)	Prevents silent breakage

Avoid a second brain of classic channels. Binary caches are **optimization**, not identity.

Pitfalls

Symptom	Likely cause	Fix
ISO branded, install not	Host forgot identity import	Fix import chain
Desktop never evals <code>nixos-install</code> misses flake	DM option names on 26.05 Path/remote	Align to pin Copy flake into live image
Weak password on public ISO	Lab password leaked into iso host	Demo-only secrets
Scope explosion	Out-featuring NixOS	Two editions only

Acceptance criteria

- ☐ Two editions as modules (minimal + desktop/server).
- ☐ ISO **and** demo host configs build.
- ☐ Branding on install path **or** dual proof (live ISO + demo VM) with evidence.
- ☐ README + DEMO.md + PRODUCT.md complete.
- ☐ Trademark-safe naming; flakes on; unfree policy explicit.
- ☐ `flake.lock` committed.
- ☐ Note: what you would “sell” vs what remains upstream NixOS.

Stretch goals

1. Calamares-branded graphical installer.
2. Binary cache + documented substituters.
3. `deploy-rs` / Colmena for two demo VMs.
4. `nixosTests` grepping `PRETTY_NAME`; optional GA `nix flake check`.

5. Ship Project 1–4 packages under `packages/`.
-

Capstone reflection (½ page)

In `PRODUCT.md` / `RETRO.md`: what is yours vs forever-upstream; what breaks first on the next stable bump; would you trust this on a travel laptop tomorrow?

Self-check

1. Edition vs host vs image in *your* tree.
 2. Upgrade channel of truth?
 3. How ISO and installed systems avoid drift?
 4. Fifteen-minute demo for a friend?
-

Done with the projects track

Return to the [Projects index](#) and tick the global bar. Optional: fold into capstone days 83–90 or publish the flake as a portfolio piece labeled as a lab derivative.

Branding

Sample distro branding assets (lab only)

These files are **teaching placeholders** for Projects 5–7 (custom NixOS-based distro).

File	Use
logo.svg	System <code>environment.etc</code> , greeter/docs mockups, ISO branding notes
os-release.fragment.example	Fields to merge into identity module experiments
plymouth-theme-notes.md	How to think about splash themes without vendoring huge assets

License

Original simple geometry in `logo.svg` is provided for this monorepo as **CC0-1.0** (public domain dedication) so you can remix freely for personal labs.

Trademark

Do **not** reuse official NixOS snowflake artwork as your distro mark. Use a **distinct name** (e.g. MyNixOS, YararaLab) and follow [NixOS branding guidance](#) when mentioning upstream.

Wire into a flake

```
environment.etc."mynixos/logo.svg".source = ./branding/logo.svg;
```

Or copy into your own repo:

```
cp -r path/to/books/NixOS/content/99-projects/branding ~/src/my-distro/branding
```

Plymouth / greeter theme notes (lab)

Goal

Visible **product identity** on boot/login without shipping megabytes of third-party themes you do not license.

Practical path on 26.05

1. **Console first** — `console.colors`, `console.font`, `/etc/issue` (Projects 5–6). Cheap and reliable in QEMU.
2. **Plymouth** — `boot.plymouth.enable = true` with a **stock** theme (`spinner`, etc.) while learning. Custom themes = package a theme derivation later.
3. **Greeter logo** — option names depend on GDM/SDDM/LightDM and nixpkgs revision. Verify on your pin; if missing, set wallpaper via Home Manager on installed systems.
4. **ISO volume label** — `isoImage.volumeID` / `isoImage.isoName` when using installer profiles.

Custom Plymouth (stretch)

- Theme = directory of script + images under the Plymouth theme path.
- Package with `stdenv.mkDerivation` installing into `$out/share/plymouth/themes/...`
- Reference via `boot.plymouth.themePackages` + `boot.plymouth.theme`.
- Test only in a VM with enough RAM; Plymouth failures are painful on metal.

Assets in this folder

Use `logo.svg` as a greeter/docs mark. Convert to PNG if a greeter requires bitmap:

```
# if rsvg-convert available
rsvg-convert -w 256 logo.svg -o logo.png
```